

Reinforcement Learning: Foundations

Shie Mannor, Yishay Mansour and Aviv Tamar

October 2025

Contents

1	Introduction and Overview	11
1.1	What is Reinforcement Learning?	11
1.2	Motivation for RL	12
1.3	The Need for This Book	13
1.4	Mathematical Models	13
1.5	Book Organization	14
1.6	Bibliography notes	16
2	Preface to the Planning Chapters	17
2.1	Reasoning Under Uncertainty	20
2.2	Objective Optimization	22
2.3	Importance of Small (Finite) Models	23
3	Deterministic Decision Processes	25
3.1	Discrete Time Dynamic Systems	27
3.2	The Finite Horizon Decision Problem	29
3.2.1	Costs and Rewards	29
3.2.2	Optimal Paths	30
3.2.3	Control Policies	30
3.2.4	Reduction between Control Policy Classes	32
3.2.5	Optimal Control Policies	35
3.3	Finite Horizon Dynamic Programming	36
3.4	Shortest Path on a Graph	39
3.4.1	Problem Statement	39
3.4.2	The Dynamic Programming Equation	40
3.4.3	The Bellman-Ford Algorithm	40
3.4.4	Dijkstra's Algorithm	41
3.4.5	Dijkstra's Algorithm for Single Pair Problems	43

3.4.6	From Dijkstra's Algorithm to A*	45
3.5	Average Cost Criterion	47
3.6	Continuous Optimal Control	50
3.6.1	Linear Quadratic Regulator	51
3.6.2	Iterative LQR	54
4	Markov Chains	57
4.1	State Classification	58
4.2	Recurrence	60
4.3	Invariant Distribution	63
4.3.1	Reversible Markov Chains	68
4.3.2	Mixing Time	69
5	Markov Decision Processes and Finite Horizon Dynamic Programming	71
5.1	Markov Decision Process	73
5.2	Performance Criteria	77
5.2.1	Finite Horizon Return	77
5.2.2	Infinite Horizon Problems	80
5.2.3	Stochastic Shortest-Path Problems	81
5.3	Sufficiency of Markov Policies	82
5.4	Finite-Horizon Dynamic Programming	82
5.4.1	The Principle of Optimality	83
5.4.2	Dynamic Programming for Policy Evaluation	83
5.4.3	Dynamic Programming for Policy Optimization	84
5.4.4	The Q Function	87
6	Discounted Markov Decision Processes	89
6.1	Problem Statement	90
6.2	The Fixed-Policy Value Function	91
6.3	Overview: The Main DP Algorithms	95
6.4	Contraction Operators	97
6.4.1	The contraction property	97
6.4.2	The Banach Fixed Point Theorem	98
6.4.3	The Dynamic Programming Operators	99
6.5	Proof of Bellman's Optimality Equation	101
6.6	Value Iteration (VI)	102
6.6.1	Error bounds and stopping rules	103
6.7	Policy Iteration (PI)	104

6.8	A Comparison between VI and PI Algorithms	105
6.9	Further Remarks on Policy Iteration	108
7	Episodic Markov Decision Processes	111
7.1	Definition	112
7.2	Relationship to other models	113
7.2.1	Finite Horizon Return	113
7.2.2	Discounted infinite return	114
7.3	Bellman Equations	114
7.3.1	Value Iteration	115
7.3.2	Policy Iteration	116
7.3.3	Bellman Operators	117
7.3.4	Bellman's Optimality Equations	119
8	Linear Programming Solutions	121
8.1	Background	121
8.2	Linear Program for Finite Horizon MDPs	122
8.3	Linear Program for Discounted MDPs	126
8.4	Further Notes	130
9	Preface to the Learning Chapters	131
9.1	Interacting Online with an Unknown MDP	132
9.1.1	Alternative Learning Models	134
9.1.2	What to Learn in RL?	136
10	Reinforcement Learning: Model Based	139
10.1	Effective horizon of discounted return	140
10.2	Off-Policy Model-Based Learning	141
10.2.1	Mean estimation	142
10.2.2	Influence of reward estimation errors	143
10.2.3	Estimating the transition probabilities	147
10.2.4	Improved sample bound: Approximate Value Iteration (AVI)	152
10.3	On-Policy Learning	154
10.3.1	Learning a Deterministic Decision Process	155
10.3.2	On-policy learning MDP: Explicit Explore or Exploit	158
10.3.3	On-policy learning MDP: R-MAX	162
10.4	Bibliography Notes	165

11 Reinforcement Learning: Model Free	167
11.1 Model Free Learning – the Situated Agent Setting	169
11.2 Q-learning: Deterministic Decision Process	169
11.3 Monte-Carlo Algorithms	173
11.3.1 Generating the samples	173
11.3.2 First visit	175
11.3.3 Every visit	177
11.3.4 Monte-Carlo control	180
11.3.5 Monte-Carlo: pros and cons	181
11.4 Stochastic Approximation	182
11.4.1 Convergence via Contraction	183
11.4.2 Convergence via the ODE method	184
11.4.3 Comparison between the two convergence proof techniques . .	188
11.5 Bootstrapping Algorithms	189
11.5.1 TD(0)	190
11.5.2 Q-learning: Markov Decision Process	194
11.5.3 SARSA: On-Policy Q-Learning	197
11.5.4 Multiple Look-Ahead Algorithms	203
11.6 On Policy vs. Off Policy and Importance Sampling	209
11.7 Bibliography Notes	212
12 Large State Spaces: Value Function Approximation	213
12.1 Approximation approaches	215
12.1.1 Value Function Approximation Architectures	216
12.2 Quantification of Approximation Error	217
12.3 Approximate Policy Evaluation	218
12.3.1 From RL to Supervised Learning	219
12.3.2 Preliminaries – Least Squares Regression	219
12.3.3 Approximate Policy Evaluation: Regression	222
12.3.4 Approximate Policy Evaluation: Bootstrapping	222
12.3.5 The Projected Bellman Equation	223
12.3.6 Least Squares Temporal Differences (LSTD)	228
12.3.7 Projected Value Iteration	229
12.3.8 TD(0) with Function Approximation	231
12.3.9 Episodic MDPs	233
12.4 Approximate Policy Iteration	234
12.4.1 Classification Based Policy Iteration	237
12.4.2 Approximate Policy Iteration with T-step Lookahead	238

12.4.3	Approximate Policy Iteration with Monte-Carlo Tree Search . . .	239
12.4.4	Online Model Free Approximate Policy Iteration – SARSA . . .	242
12.4.5	Batch Model Free Approximate Policy Iteration – Least Squares Policy Iteration (LSPI)	242
12.5	Approximate Value Iteration	243
12.5.1	Online Approximate Value Iteration – Q Learning	244
12.5.2	Batch Approximate Value Iteration – Fitted Q	244
12.5.3	The Deep Q Learning Algorithm	246
12.6	Bibliography Notes	247
13	Large State Space: Policy Gradient Methods	249
13.1	Problem Setting	250
13.2	Policy Representations	250
13.3	The Policy Performance Difference Lemma	251
13.4	Gradient-Based Policy Optimization	255
13.4.1	Finite Differences Methods	256
13.5	Policy Gradient Theorem	257
13.6	Policy Gradient Algorithms	261
13.6.1	REINFORCE: Monte-Carlo updates	262
13.6.2	TD Updates and Compatible Value Functions	265
13.7	Convergence of Policy Gradient	268
13.8	Proximal Policy Optimization	270
13.9	Alternative Proofs for the Policy Gradient Theorem	272
13.9.1	Proof Based on Unrolling the Value Function	272
13.9.2	Proof Based on the Trajectory View	274
13.9.3	Proof Based on Importance Sampling	276
13.10	Bibliography Notes	277
14	Regret Minimization	279
14.1	Multi-Arm Bandit Model	281
14.2	Warmup: Full Information, Two Actions	283
14.3	Stochastic Multi-Arm Bandits: Lower Bound	285
14.4	Explore-Then-Exploit	287
14.5	Improved Regret Minimization Algorithms	290
14.5.1	Refined Confidence Bounds	291
14.5.2	Successive Action Elimination	292
14.5.3	Upper Confidence Bound (UCB) Algorithm	294
14.6	From Multi-Arm Bandits to MDPs	296

14.7	Best Arm Identification	299
14.7.1	Naive Algorithm (PAC criteria)	299
14.7.2	Sequential Halving Algorithm	300
14.8	Bibliography Notes	304
A	Dynamic Programming	305
B	Ordinary Differential Equations	311
B.1	Definitions and Fundamental Results	311
B.1.1	Systems of Linear Differential Equations	313
B.2	Asymptotic Stability	314

List of Figures

1.1	Chapter dependencies	15
3.1	Graphical description of a finite model.	28
3.2	Example of a feasible path.	29
3.3	Control policy vs. feasible path	32
3.4	A finite horizon decision problem.	36
3.5	Shortest path on a finite horizon decision problem.	37
3.6	Grid world	46
3.7	Cart-Pole system	52
5.1	Markov chain	74
5.2	Controlled Markov chain	75
6.1	Example MDP for comparing VI and PI.	106
6.2	Comparison between VI and PI	108
8.1	Comparison between LP and PI	130
9.1	Interaction of an agent with the environment.	132
11.1	First vs. every visit example	174
11.2	Equivalent MDP for Every Visit	179
11.3	Convergence via the ODE method.	186
11.4	Plot for Example 11.1.	189
11.5	TD(0) vs. Monte-Carlo example	190
11.6	Markov Reward Chain	194
12.1	Example: MC vs. TD with function approximation.	222
13.1	Example MDP	255
13.2	Convergence of gradient descent	268

13.3 Aliased grid-world example	276
---	-----

Chapter 1

Introduction and Overview

1.1 What is Reinforcement Learning?

Concisely defined, Reinforcement Learning, abbreviated as RL, is the discipline of learning and acting in environments where sequential decisions are made. That is, the decision made at a given time will be followed by other decisions and therefore the decision maker has to consider the implications of her decision on subsequent decisions.

In the early days of the field, there was an analogy drawn between human learning and computer learning. While the two are certainly tightly connected, this is merely an analogy that serves to motivate and inspire. Other terms that have been used are approximate dynamic programming (ADP), neuro-dynamic programming (NDP), which to us mean the same thing, but focus on a specific collection of techniques that came to grow to the discipline known as “RL”.

Origins of reinforcement learning. Reinforcement learning has roots in quite a few disciplines. Naturally, by our own indoctrination, we are going to look through the lens of Computer Science and Machine Learning. From an engineering perspective, optimal control is the “mother” of RL, and many of the concepts that are used in RL naturally come from optimal control. Other notable origins are in Operations Research, where the initial mathematical frameworks had originated. Additional disciplines include: Neuroscience, Psychology, Statistics and Economics.

The origins of the term “reinforcement learning” is in psychology, where it refers to learning by trial and error. While this inspired much work in the early days of the field, current approaches are mostly based on machine learning and optimal control. We refer the reader to Section 1.6 in [149] for a detailed history of RL as a field.

1.2 Motivation for RL

In recent years there is a renewed interest in RL. The new interest is grounded in emerging applications of RL, and also progress of deep learning that has been impressively applied for solving challenging RL tasks. But for us, the interest comes from the *promise* of RL and its potential to be an effective tool for control and behavior in dynamic environments.

Over the years, RL has proven to be highly successful for playing board games that require long horizon planning. Early in 1962, Arthur Samuel [127] developed a checkers game, which was at the level of the best human. His original framework included many of the ingredients which latter contributed to RL, as well as search heuristics for large domains. Gerald Tesauro in 1992 developed the TD-gammon [157], which used a two layer neural-network to achieve a high performance agent for playing the game of backgammon. The network was trained from scratch, by playing against itself in simulation, and using a temporal differences learning rule. One of the amazing features of TD-gammon was that even in the *first* move, it played a different game move than the typical opening that backgammon grandmasters use. Indeed, this move was later adopted in the backgammon community [158]. More recently, DeepMind has developed AlphaGo – a deep neural-network based agent for playing Go, which was able to beat the best Go players in the world, solving a long-standing challenge for artificial intelligence [139].

To complete the picture of computer board games, we should mention Deep Blue, from 1996, which was able to beat the world champion then, Kasparov [27]. This program mainly built on heuristic search and new hardware was developed to support it. Recently, DeepMind’s AlphaZero matched the best chess programs (which are already much better than any human players), using an RL-based approach [141].

Another domain, popularized by DeepMind, is playing Atari video games [110], which were popular in the 1980’s. DeepMind were able to show that deep neural networks can achieve human level performance, using only the raw video image and the game score as input (and having no additional information about the goal of the game). Importantly, this result reignited the interest of RL in the robotics community, where acting based on raw sensor measurements (a.k.a. ‘end-to-end’) is a promising alternative to the conventional practice of separating decision making into perception, planning, and control components [97].

More recently, interest in RL sparked yet again, as it proved to be an important component in fine tuning large language models to match user preferences, or to accomplish certain tasks [117, 175]. One can think of the sequence of words in a conversation as individual decisions made with some higher level goal in mind, and

RL fits naturally with this view of language generation.

While the RL implementations in each of the different applications mentioned above were very different, the fundamental models and algorithmic ideas were surprisingly similar. These foundations are the topic of this book.

1.3 The Need for This Book

There are already several books on RL. However, while teaching RL in class, we felt that there is a gap between advanced textbooks that focus on one aspect or another of the art, and more general books that opt for readability rather than rigor. Coming from computer science and electrical engineering backgrounds, we like to teach RL in a rigorous, self-contained manner. This book serves this purpose, and is based on our lecture notes for an advanced undergraduate course that we have taught for over a decade at Tel Aviv University and at the Technion.

Complementing this book is a booklet with exercises and exam questions to help students practice the material. These exercises were developed by us and our teaching assistants over the years.

1.4 Mathematical Models

The main mathematical model we will use is the celebrated Markov Decision Process (MDP). The model tries to capture uncertainty in the dynamics of the environment, the actions and our knowledge. The main focus would be on sequential decision making, namely, selecting actions one after the other. The evaluation would consider the long-term effect of the actions, trading off immediate rewards with long-term gains.

In contrast to standard machine learning problems such as regression or classification, the RL model has a notion of a *state*, and the algorithm influences the state through its actions. The algorithm would be faced with an inherent tradeoff between exploitation (getting the most reward given the current information) and exploration (gathering more information about the environment).

There are other models that are useful, such as partially observed MDPs (POMDPs) where the exact state is not fully known, and bandits where the current decision has no effect on subsequent decision. In accordance with most of the literature with the exception of Chapter 14 the book concerns the MDP model.

1.5 Book Organization

The book is thematically comprised of two main parts – *planning* and *learning*.

Planning: The planning theme develops the fundamentals of optimal decision making in the face of uncertainty, under the Markov decision process model. The basic assumption in planning is that the MDP model is *known* (yet, as the model is stochastic, uncertainty must still be accounted for in making decisions). In a preface to the planning section, Chapter 2, we motivate the MDP model and relate it to other models in the planning and control literature. In Chapter 3 we introduce the problem and basic algorithmic ideas in the deterministic setting. In Chapter 4 we review the topic of Markov chains, which the Markov decision process model is based on, and then, in Chapter 5 we introduce the finite horizon MDP model and a fundamental dynamic programming approach. Chapter 6 covers the infinite horizon discounted setting, and Chapter 7 covers the episodic setting. Chapter 8 covers an alternative approach for solving MDPs using a linear programming formulation.

Learning: The learning theme covers decision making when the MDP model is *not known in advance*. In a preface to the learning section, Chapter 9, we motivate this learning problem and relate it to other learning problems in decision making. Chapter 10 introduces the *model-based* approach, where the agent explicitly learns an MDP model from its experience and uses it for planning decisions. Chapter 11 covers an alternative *model-free* approach, where decisions are learned without explicitly building a model. Chapters 12 and 13 address learning of approximately optimal solutions in *large* problems, that is, problems where the underlying MDP model is intractable to solve. Chapter 12 approaches this topic using approximation of the value function, while Chapter 13 considers policy approximations. In Chapter 14 we study the online nature of decision making in more detail, under the framework of regret minimization. In this chapter we mostly focus on the special case of Multi-Arm Bandits, which can be viewed as a MDP with a single state and unknown rewards.

The chapters are organized to be taught in sequence. The diagram in Figure 1.1 depicts the dependency structure between the different chapters, and may be convenient for teachers who opt for a different course structure.

Acknowledgments. We are grateful to our colleagues and students who helped us improve this manuscript. In alphabetical order: Stav Belogolovsky, Esther Derman, Yonathan Efroni, Uri Gadot, Hector Geffner, Nan Jiang, Navdeep Kumar, Tal Lancewicki, Orin Levy, Nadav Merlis, Avinash Mohan, Mirco Mutti, Ofir Nabati,

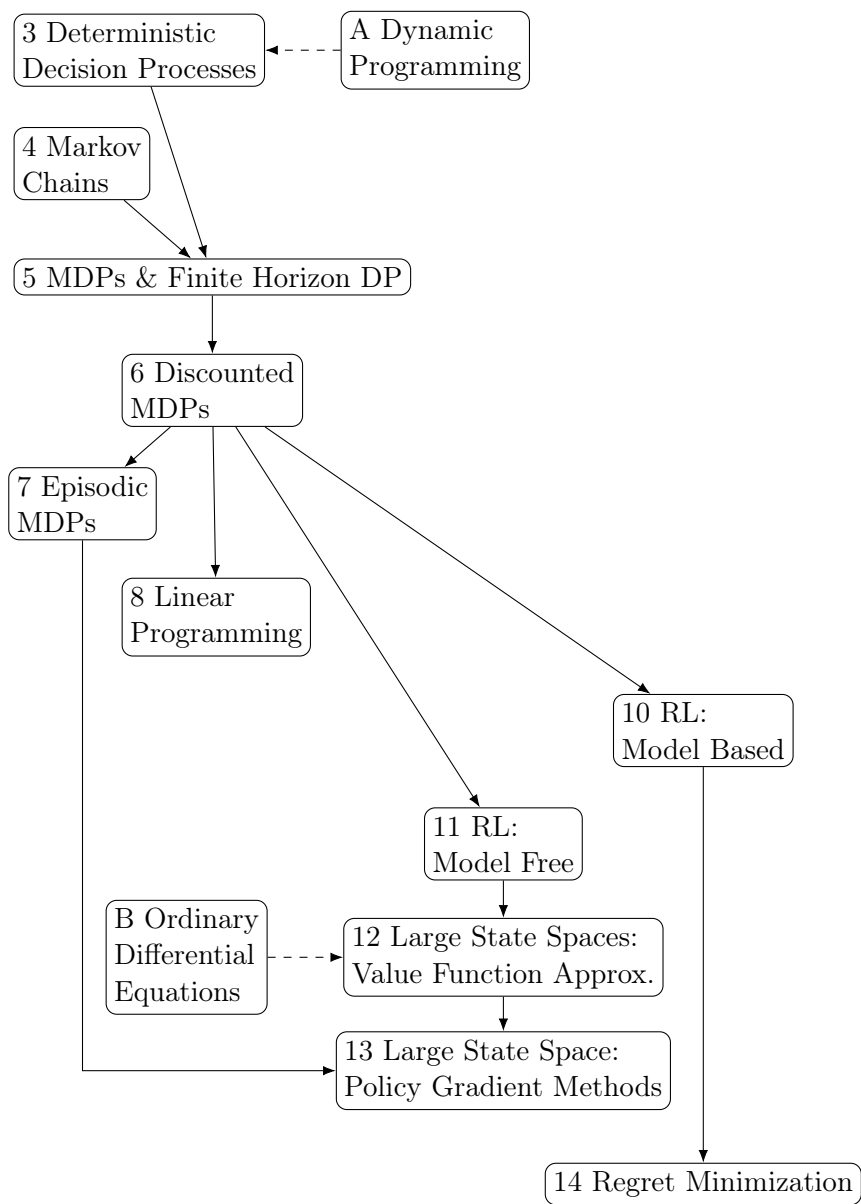


Figure 1.1: Chapter dependencies

Benny Pereth, Uri Sherman, Satinder Singh, and Jonathan Somer. Special thanks is due to Elad Sharony for carefully sieving through this manuscript. Some parts of this book evolved from notes that were initially written by Nahum Shimkin. We thank him deeply for structuring the first versions of the course this book is based on.

1.6 Bibliography notes

Markov decision processes have a long history. The first works that directly addressed Markov Decision Processes and Reinforcement Learning are due to [14] and [60]. The book by Bellman [14], based on a sequence of works by him, introduced the notion of *dynamic programming*, the *principle of optimality* and defined discrete time MDPs. The book of Howard [60], building on his PhD thesis, introduced the policy iteration algorithm as well as a clear algorithmic definition of value iteration. A precursor work by Shapely [136] introduced a discounted MDP model for stochastic games.

There is a variety of books addressing Markov Decision Processes and Reinforcement Learning. Puterman's book [121] gives an extensive exposition of mathematical properties of MDPs, including planning algorithms. Bertsekas and Tsitsiklis [18] and Bertsekas [19] give a stochastic processes approach for reinforcement learning and control.

Sutton and Barto [149] give a general exposition to modern reinforcement learning, which is more focused on implementation issues less focused on mathematical issues. They also give an excellent historical account of the development of RL from its infancy. Szepesvari's monograph [152] gives an outline of basic reinforcement learning algorithms.

Chapter 2

Preface to the Planning Chapters

In the following chapters, we discuss the *planning* problem where a model is known. Before diving in, however, we shall spend some time on defining the various approaches to modeling a sequential decision problem, and motivate our choice to focus on some of them. In the next chapters, we will rigorously cover selected approaches and their implications. This chapter is quite different from the rest of the book, as it discusses epistemological and philosophical issues more than anything else.

We are interested in sequential decision problems in which a sequence of decisions need to be taken to achieve a goal or optimize some performance measure. Some examples include:

Example 2.1 (Board games). *An agent playing a board game such as Tic-Tac-Toe, chess, or backgammon. Board games are typically played against an opponent, and may involve external randomness such as the dice in backgammon. The goal is to play a sequence of moves that lead to winning the game.*

Example 2.2 (Robot Control). *A robot needs to be controlled to perform some task, for example, picking up an object and placing it in a bin, or folding up a piece of cloth. The robot is controlled by applying voltages to its motors, and the goal is to find a sequence of controls that perform the desired task within some time limits.*

Example 2.3 (Inventory Control). *Inventory control is a classical and practical application of sequential decision-making under uncertainty. In its simplest form, a decision maker must determine how much inventory to order at each time period to meet uncertain future demand while balancing ordering costs, holding costs, and stockout penalties. The uncertainty in the demand requires a good policy to adapt to the stochastic nature of customer behavior while accounting for both immediate costs and future implications of current decisions. The (s, S) policy, also known as*

a reorder point-order-up-to policy ([128]), is an elegantly simple yet often optimal approach to inventory control. Under this policy, whenever the inventory level drops to or below a reorder point s , an order is placed to bring the inventory position up to a target level S . While finding the optimal values for s and S is non-trivial, this policy structure has been proven optimal for many important inventory problems under reasonable assumptions. The (s, S) framework provides an excellent example of how constraining the policy space, in this case to just two parameters, can make learning more efficient while still achieving strong performance.

When we are given a sequential decision problem we have to model it from a mathematical perspective. In this book, and in much of the literature, the focus is mostly on the celebrated Markov Decision Process (MDP) model. It should be clear that this is merely a model, i.e., one should not view it as a precise reflection of reality. To quote Box: “*all models are wrong, but some are useful*”. Our goal is to have useful models and as such the MDP model is a perfect example. The MDP model has the following components which we discuss here and provide formally in later chapters. We will use the agent-centric view, assuming an agent interacts with an environment. This agent is sometimes called a “decision maker”, especially in the operations research community.

1. States: A state is the atomic entity that represents all the information needed to predict future rewards of the system. The agent in an MDP can fully observe the state.
2. Actions: An action is what can be affected by the decision maker.
3. Rewards: The rewards represent some numerical measurement that the decision maker wishes to maximize. The reward is assumed to be a function of the current state and the action.
4. Dynamics: The state changes (or transitions) according to the dynamics. This evolution depends only on the current state and the action chosen but not on future or past states or actions.

In planning, it is assumed that all the components are known. The objective of the decision maker is to find a policy, i.e., a mapping from histories of state observations to actions, that maximizes some objective function of the reward. We will adopt the following standard assumptions concerning the planning model:

1. Time is discrete and regular: decisions are made in some predefined decision epochs. For example, every second/month/year. While continuous time is

especially common in robotic applications, we will adhere for simplicity to discrete regular times. In principle, this is not a particularly limiting assumption, as most digital systems inherently discretize the time measurement. However, it may be unnecessary to apply a different control at *every* time step; The semi-MDP model is a common framework to use when the decision epochs are irregular [121], and there is an extensive literature on optimal control in continuous time [80], which we will not consider here.

2. Action space is finite. We will mostly assume that the available actions a decision maker can choose from belong to a finite set. While this assumption may appear natural in board games, or any digital system that is discretized, in some domains such as robotics it is more natural to consider a continuous control setting. For continuous actions, the *structure* of the action space is critical for effective decision making – we will discuss some specific examples, such as a linear dynamical system here. More general continuous and hybrid discrete-continuous models are often studied in the control literature [16] and in operations research [120].
3. State space is finite. The set of possible system states is also assumed to be finite and unstructured. The finiteness assumption is mostly a convenience, as any bounded continuous space can be finely discretized to a finite, but very large set. Indeed, in the second part of this book, we shall study learning-based methods that can handle very large state spaces. For problems where the state space has a known and convenient structure, a model that takes this structure into account can be more appropriate. For example, in a linear controlled dynamical system, which we discuss in Section 3.6, the state space is continuous, and its evolution with respect to the control is linear, leading to a closed form optimal solution when the reward has a particular quadratic structure. In the classical STRIPS and PDDL planning models, which we do not cover here, the state space is a list of binary variables (e.g., a system for robot setting a table may be described by [robot gripper closed = False, cup on table = True, plate on table = False, ...]), and planning algorithms that try to find actions that lead to certain goal states being ‘true’ can take account of this special structure [126].
4. Rewards are all given in a single currency. We assume that the agent has a single reward stream it tries to optimize. Specifically the agent tries to maximize the long term sum of rewards. In some cases, a user may be interested in other statistics of the reward, such as its variance [103], or to balance multiple

types of rewards [102]; we do not cover these cases here.

5. Decision Horizon or Discounting is known. It is assumed in virtually all the literature that the time horizon for the decision process is a known stopping time. That is, the sequential process terminates as a result of an event that depends on history. For discounted problems, the discount factor is assumed to be a part of the problem definition.
6. Markov state evolution. We shall assume that the environment’s reaction to the agent’s actions is fixed (it may be stochastic, but with a fixed distribution), and depends only on the current state. This assumption precludes environments that are *adversarial* to the agent, or systems with multiple independent agents that learn together with our agent [29, 174].

As should be clear from the points above, the MDP model is agnostic to the structure that certain problems may possess, and more specialized models may exploit. The reader may question, therefore, why study such a model for planning. As it turns out, the simplicity and generality of the MDP model is actually a blessing when using it for *learning*, which is the main focus of this book. The reason is that the structure of a specific problem may be implicitly picked up by the learning algorithm, which is designed to identify patterns in data. This strategy has proved to be very valuable in computing decision making policies for problems where structure exists, but is hard to define manually, which is often the case in practice. Indeed, many recent RL success stories, such as mastering the game of Go, managing resources in the complex game of StarCraft, and state-of-the-art continuous control of racing drones, have all used the simple MDP model combined with powerful deep learning methods [138, 170, 72].

There are two other strong modelling assumptions in the MDP model: (1) all uncertainty in decision making is limited to the randomness of the (Markov) state transitions, and (2) the objective can only be specified using rewards. Next, we discuss these two design choices in more detail.

2.1 Reasoning Under Uncertainty

A main objective of planning and learning is to facilitate reasoning under uncertainty. Uncertainty can come in many forms, as humans often encounter every day: when playing a board game, we do not know what our opponent will do; when rolling a dice, we do not know what will be the outcome; when folding laundry, it is likely

that an accurate physical model of the cloth is not available; when driving in a city, we may not observe cars hidden behind a corner; etc.

Here we list common types of uncertainty and, afterwards, discuss how they relate to planning with MDPs.

*Aleatoric*¹ *uncertainty*: Handling inherent randomness that is part of the model. For example, a model can contain an event which occurs with a given probability. For example, in the board game backgammon, the probability of a move is given by throwing two dices.

*Epistemic*² *uncertainty*: Dealing with a lack of knowledge about the model parameters. Sometimes we do not know what the exact model parameters are because more interaction is needed to learn them (this is addressed in the learning part of this book). Sometimes we have a nominal model and the true parameters are only revealed at runtime (this is addressed within the robust MDP framework; see [115]). Sometimes our model is too coarse or simply incorrect – this is known as *model misspecification*.

Partial observability: Reasoning with incomplete information concerning the true state of the system. There are many problems where we just do not have an accurate measurement that can help us predict the future and instead we get to observe partial information concerning the true state of the system. Some would argue that all real problems have some elements of partial observability in them.

We emphasize that for planning and for learning, a model could combine all types of uncertainty. The choice of which type of uncertainty is an important design choice.

The MDP model that we focus on in the planning chapter only accommodates aleatoric uncertainty, through the stochastic state transitions. While this may appear to be a strong limitation, MDPs have proven useful for dealing with more general forms of uncertainty. For example, in the learning chapters, we will ask how to update an MDP model from interaction with the environment, to potentially reduce epistemic uncertainty. For board games, even though MDPs cannot model an adversary, assuming that the opponent is stochastic helps find a robust policy against various opponent strategies. Moreover, by using the concept of *self play* – an agent that learns to play against itself and continually improve – RL has produced the most advanced AI agents for several games, including Chess and Go. For partially observable systems, a fundamental result shows that taking the full observation history as the ‘state’, results in an MDP model for the problem (albeit with a huge state space).

¹The term comes from the Latin word “alea”, which means dice or a game of chance.

²The term is derived from the Greek word “episteme”, meaning knowledge.

2.2 Objective Optimization

A central assumption in planning is that an immediate reward function is given to the decision maker and the objective is to maximize some sort of expected cumulative discounted (or total or average) reward. From the perspective of the planner, this makes the planner’s life easy as the objective is defined in a formal manner. Nevertheless, in many applications, much of the problem is to engineer a “right” reward function. This may be done by understanding the specifications of the problem, or from data of desired behavior, a problem known as *Inverse Reinforcement Learning* [114]. In inverse reinforcement learning the decision maker typically observes trajectories that were sampled by an expert and needs to infer an optimal policy often based on estimating the reward function. While the problem is ill-defined, since many reward functions may lead to similar optimal trajectories, the main issue is to obtain good *generalization* and for that robustness is key.

In some cases, the reward stream is very sparse. For example, in board games the reward is often obtained only at the end of the game in the form of a victory or a loss. While this does not pose a conceptual problem, it may lead to practical problems as we will discuss later in the book. A conceptual solution here is to use “proxy rewards”, which are intermediate reward indicating progress towards a desired goal.

A limitation of the MDP planning model is the underlying assumption that preferences can be succinctly represented through reward functions. Although in principle any preference among trajectories can be represented using a reward function, by extending the state space to include all history, this may be cumbersome and may require a much larger state space. Specifically, the discount factor, which is often assumed a part of the problem specifications, represents a preference between short-term and long-term objectives. Such preferences are often arbitrary.

We finally comment that the assumption that there exists a scalar reward we optimize (through a long term objective) does not hold in many problems. Often, we have several, potentially contradicting, objectives. For example, we may want to minimize power consumption while maximizing throughput in communication networks. In general, part of the reward function engineering pertains to balancing different objectives, even if they are not measured in the same way (“adding apples and oranges”). A different approach is to embrace the multi-objective nature of the decision problems through constrained Markov decision processes [3], or using other approaches [e.g., 102]. Moreover, even when all aspects in expectation of a problem can be amortized with a single reward function the decision maker may need to consider risk issues, such as resilience to rare events. We emphasize that the reward function is a design choice made by the decision maker.

Nevertheless, MDPs with their scalar reward function have proven useful in many practical domains, as the availability of strong algorithms for solving MDPs effectively allows the system engineer to tweak the reward function manually to fit some hard-to-quantify desired behavior.

2.3 Importance of Small (Finite) Models

The next few chapters, and indeed much of the literature, explicitly assume that the models are finite (in terms of actions and states) and even practically small. While this is certainly justified from a pedagogical perspective, there are additional reasons that make small models relevant.

Small models are more interpretable than large ones: it is often the case that a different state captures particular meanings leading to more explainable policies. For example, in inventory control problems, the dynamic programming techniques that we will study can show that for certain simplified problem instances, an *optimal* strategy has the structure of a threshold policy – if the inventory is below some certain threshold then replenish, otherwise do not. Such observations about the structure of optimal policies often inform the design of policies for more complex scenarios.

The language and some fundamental concepts we shall develop for small models, such as the value function, value iteration and policy iteration algorithms, and convergence of stochastic approximation, will also carry over to the learning chapters, which deal with large state spaces and approximations.

Chapter 3

Deterministic Decision Processes

In this chapter we introduce the dynamic system viewpoint of the optimal planning problem, where given a complete model we characterize and compute the optimal policy. We restrict the discussion here to deterministic (rather than stochastic) systems. We consider two basic settings: (1) the finite-horizon decision problem and its recursive solution via finite-horizon Dynamic Programming, and (2) the average cost and its related minimum average weight cycle.

Chapter Orientation

Main Prerequisites:

- Dynamic programming (Appendix A)

Main terminology:

- $\mathbf{s}_t$ state at time t
- $\mathcal{S}_t$ state space at time t
- $\mathbf{a}_t$ action at time t
- $\mathcal{A}_t(\mathbf{s})$ action space at time t for state $\mathbf{s}$
- f_t transition function at time t
- T time horizon

- $r_t(\mathbf{s}, \mathbf{a}), c_t(\mathbf{s}, \mathbf{a})$ reward/cost of state $\mathbf{s}$ and action $\mathbf{a}$ at time t
- h_t feasible path with t steps
- $\mathcal{C}_{0:T}(h_T), \mathcal{V}_{0:T}(h_T)$, abbreviated as $\mathcal{C}_T(h_T), \mathcal{V}_T(h_T)$: cumulative costs/rewards for a path
- π_t policy at time t
- Π_H, Π_M, Π_S history-dependent, Markov, and stationary deterministic policy sets
- $\Pi_{HS}, \Pi_{MS}, \Pi_{SS}$ history-dependent, Markov, and stationary stochastic policy sets
- $\rho_t^\pi(\mathbf{s}, \mathbf{a})$ occupancy of state-action $\mathbf{s}, \mathbf{a}$ at time t under policy π
- $\mathcal{C}_{0:T}^\pi(\mathbf{s}_0), \mathcal{V}_{0:T}^\pi(\mathbf{s}_0)$, abbreviated as $\mathcal{C}_T^\pi(\mathbf{s}_0), \mathcal{V}_T^\pi(\mathbf{s}_0)$: cumulative T -step costs / rewards under policy π starting from state $\mathbf{s}_0$
- π^* optimal control policy
- $\mathcal{V}_t(\mathbf{s})$ value function – maximal cumulative reward from state $\mathbf{s}$ at time t
- $G = (V, E)$ directed graph with nodes V and edges E
- $c(e) = c(v_1, v_2)$ edge cost
- $c(\omega)$ length (sum of edge costs) of path ω
- $d(u, v)$ shortest path length from node u to v
- $\text{Neighbors}(u)$ neighbors of node u
- $\mathcal{C}_{avg}^\pi$ average cost of policy π
- Ω set of all cycles in a graph
- $\mu(\omega), \mu^*$ average cost of cycle ω and minimum average cycle cost

3.1 Discrete Time Dynamic Systems

We consider a discrete-time dynamic system, of the form:

$$\mathbf{s}_{t+1} = f_t(\mathbf{s}_t, \mathbf{a}_t), \quad t = 0, 1, 2, \dots, T-1,$$

where

- t is the time index.
- $\mathbf{s}_t \in \mathcal{S}_t$ is the state variable at time t , and $\mathcal{S}_t$ is the set of possible states at time t .
- $\mathbf{a}_t \in \mathcal{A}_t$ is the control variable at time t , and $\mathcal{A}_t$ is the set of possible control actions at time t .
- $f_t : \mathcal{S}_t \times \mathcal{A}_t \rightarrow \mathcal{S}_{t+1}$ is the state transition function, which defines the *state dynamics* at time t .
- $T > 0$ is the *time horizon* of the system. It can be finite or infinite.

Remark 3.1. *More generally, the set $\mathcal{A}_t$ of available actions may depend on the state at time t , namely: $\mathbf{a}_t \in \mathcal{A}_t(\mathbf{s}_t) \subset \mathcal{A}_t$.*

Remark 3.2. *The system is, in general, time-varying. It is called time invariant, or stationary, if $f_t, \mathcal{S}_t, \mathcal{A}_t$ do not depend on the time t . In that case we write*

$$\mathbf{s}_{t+1} = f(\mathbf{s}_t, \mathbf{a}_t), \quad t = 0, 1, 2, \dots, T-1; \quad \mathbf{s}_t \in \mathcal{S}, \mathbf{a}_t \in \mathcal{A}(\mathbf{s}_t).$$

Remark 3.3. *The state dynamics may be augmented by an output observation:*

$$\mathbf{o}_t = \mathcal{O}_t(\mathbf{s}_t, \mathbf{a}_t),$$

where $\mathbf{o}_t$ is the system observation, or the output. In most of this book we implicitly assume that $\mathbf{o}_t = \mathbf{s}_t$, namely, the current state $\mathbf{s}_t$ is fully observed.

Example 3.1. Linear Dynamic Systems

A well known example of a dynamic system is that of a linear time-invariant system, where:

$$\mathbf{s}_{t+1} = A\mathbf{s}_t + B\mathbf{a}_t$$

with $\mathbf{s}_t \in \mathbb{R}^n$, $\mathbf{a}_t \in \mathbb{R}^m$, $A \in \mathbb{R}^{n \times n}$ and $B \in \mathbb{R}^{n \times m}$. Here the state and action spaces are evidently continuous (and not discrete).

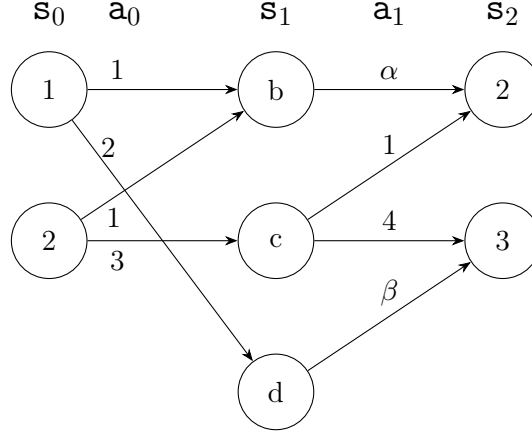


Figure 3.1: Graphical description of a finite model.

Example 3.2. Finite models

Our emphasis here will be on finite state and action models. A finite state space contains a finite number of elements n_t : $\mathcal{S}_t = \{1, 2, \dots, n_t\}$. Similarly, a finite action space implies a finite number $m_t(\mathbf{s})$ of control actions at each stage $\mathbf{s}$:

$$\mathcal{A}_t(\mathbf{s}) = \{1, 2, \dots, m_t(\mathbf{s})\}, \quad \mathbf{s} \in \mathcal{S}_t$$

Graphical description: Finite models (over finite time horizons) can be represented by a corresponding decision graph, as specified in the following example.

Example 3.3. Consider the model specified by Figure 3.1. Here:

- $T = 2$, $\mathcal{S}_0 = \{1, 2\}$, $\mathcal{S}_1 = \{b, c, d\}$, $\mathcal{S}_2 = \{2, 3\}$,
- $\mathcal{A}_0(1) = \{1, 2\}$, $\mathcal{A}_0(2) = \{1, 3\}$, $\mathcal{A}_1(b) = \{\alpha\}$, $\mathcal{A}_1(c) = \{1, 4\}$, $\mathcal{A}_1(d) = \{\beta\}$
- $f_0(1, 1) = b$, $f_0(1, 2) = d$, $f_0(2, 1) = b$, $f_0(2, 3) = c$, $f_1(b, \alpha) = 2$, $f_1(c, 1) = 2$, $f_1(c, 4) = 3$, $f_1(d, \beta) = 3$.

Definition 3.1 (Feasible Path). A feasible path for the specified system is a sequence $(\mathbf{s}_0, \mathbf{a}_0, \dots, \mathbf{s}_{T-1}, \mathbf{a}_{T-1}, \mathbf{s}_T)$ of states and actions, such that $\mathbf{a}_t \in \mathcal{A}_t(\mathbf{s}_t)$ and $\mathbf{s}_{t+1} = f_t(\mathbf{s}_t, \mathbf{a}_t)$.

In the example in Figure 3.2, a feasible path is marked in bold and it is $(1, 2, d, \beta, 3)$.

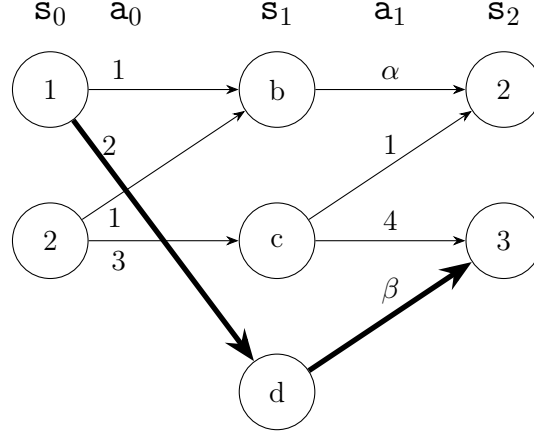


Figure 3.2: Example of a feasible path.

3.2 The Finite Horizon Decision Problem

We proceed to define our first and simplest planning problem. For that we need to specify a *performance objective* for our model, and the notion of *control policies*.

3.2.1 Costs and Rewards

The cumulative cost: Let $\mathbf{h}_T = (\mathbf{s}_0, \mathbf{a}_0, \dots, \mathbf{s}_{T-1}, \mathbf{a}_{T-1}, \mathbf{s}_T)$ denote a T -stage feasible path for the system. Each feasible path $\mathbf{h}_T$ is assigned some cost $\mathcal{C}_T = \mathcal{C}_T(\mathbf{h}_T)$.

The standard definition of the cost $\mathcal{C}_T$ is through the following *cumulative cost functional*:

$$\mathcal{C}_{0:T}(\mathbf{h}_T) = \sum_{t=0}^{T-1} c_t(\mathbf{s}_t, \mathbf{a}_t) + c_T(\mathbf{s}_T).$$

Here:

- $c_t(\mathbf{s}_t, \mathbf{a}_t)$ is the *instantaneous* cost or *single-stage* cost at stage t , state $\mathbf{s}_t$ and action $\mathbf{a}_t$, and c_t is the instantaneous cost function.
- $c_T(\mathbf{s}_T)$ is the *terminal* cost at state $\mathbf{s}_T$, and c_T is the terminal cost function.

We shall refer to $\mathcal{C}_{0:T}$ as the *cumulative T -stage cost*, or just the *cumulative cost*. When it is clear from the context that we are summing the costs from time 0, we will abbreviate the notation for the cumulative cost to $\mathcal{C}_T$. Our objective is to *minimize*

the cumulative cost $\mathcal{C}_T$, by a proper choice of actions. We will define that goal more formally in the next section.

Remark 3.4. *The cost functional defined above is additive in time. Other cost functionals are possible, for example the maximum cost, but additive cost is by far the most common and useful.*

Cost versus reward formulation: It is often more natural to consider *maximizing* reward rather than minimizing cost. In that case, we define the cumulative T-stage return function:

$$\mathcal{V}_{0:T}(\mathbf{h}_T) = \sum_{t=0}^{T-1} \mathbf{r}_t(\mathbf{s}_t, \mathbf{a}_t) + \mathbf{r}_T(\mathbf{s}_T).$$

Here, $\mathbf{r}_t$ is the instantaneous reward and $\mathbf{r}_T$ is the terminal reward, and we may abbreviate the notation as $\mathcal{V}_T$. Clearly, minimizing $\mathcal{C}_T$ is equivalent to maximizing $\mathcal{V}_T$, if we set:

$$\mathbf{r}_t(\mathbf{s}, \mathbf{a}) = -\mathbf{c}_t(\mathbf{s}, \mathbf{a}) \text{ and } \mathbf{r}_T(\mathbf{s}) = -\mathbf{c}_T(\mathbf{s}).$$

We denote by $\mathbb{T}$ the set of time steps for horizon T , i.e., $\mathbb{T} = \{0, 1, \dots, T-1\}$

3.2.2 Optimal Paths

Our first planning problem is the following *T-stage Finite Horizon Problem*:

Definition 3.2 (T-stage Finite Horizon Problem). *For a given initial state $\mathbf{s}_0$, find a feasible path $\mathbf{h}_T = (\mathbf{s}_0, \mathbf{a}_0, \dots, \mathbf{s}_{T-1}, \mathbf{a}_{T-1}, \mathbf{s}_T)$ that minimizes the cost functional $\mathcal{C}_T(\mathbf{h}_T)$, over all feasible paths $\mathbf{h}_T$. Such a feasible path $\mathbf{h}_T$ is called an optimal path from $\mathbf{s}_0$.*

A more general notion than a path is that of a *control policy*, that specifies the action to be taken at each state. Control policies will play an important role in our Dynamic Programming algorithms, and are defined next.

3.2.3 Control Policies

In general we will consider a few classes of control policies. The two basic dimensions in which we will characterize the control policies is their dependence on the history, and their use of randomization.

Definition 3.3 (History-dependent deterministic policy). A *general* or **history-dependent** control policy $\pi = (\pi_t)_{t \in \mathbb{T}}$ is a mapping from each possible history $\mathbf{h}_t = (\mathbf{s}_0, \mathbf{a}_0, \dots, \mathbf{s}_{t-1}, \mathbf{a}_{t-1}, \mathbf{s}_t)$, $t \in \mathbb{T}$, to an action $\mathbf{a}_t = \pi_t(\mathbf{h}_t) \in \mathcal{A}_t$. We denote the set of general policies by Π_H .

Definition 3.4 (Markov deterministic policy). A **Markov** control policy π is allowed to depend only on the current state and time: $\mathbf{a}_t = \pi_t(\mathbf{s}_t)$. We denote the set of Markov policies by Π_M .

Definition 3.5 (Stationary deterministic policy). For stationary models, we may define **stationary** control policies that depend only on the current state. A stationary policy is defined by a single mapping $\pi : \mathcal{S} \rightarrow \mathcal{A}$, so that $\mathbf{a}_t = \pi(\mathbf{s}_t)$ for all $t \in \mathbb{T}$. We denote the set of stationary policies by Π_S .

Evidently, $\Pi_H \supset \Pi_M \supset \Pi_S$.

Randomized (Stochastic) Control policies The control policies defined above specify deterministically the action to be taken at each stage. In some cases we want to allow for a random choice of action.

Definition 3.6 (History-dependent stochastic policy). A *general randomized* (stochastic) control policy assigns to each possible history $\mathbf{h}_t$ a probability distribution $\pi_t(\cdot | \mathbf{h}_t)$ over the action set $\mathcal{A}_t$. That is, $\Pr_{\mathbf{h}_t \sim \pi}[\mathbf{a}_t = \mathbf{a} | \mathbf{h}_t] = \pi_t(\mathbf{a} | \mathbf{h}_t)$. We denote the set of general randomized policies by Π_{HS} .

Definition 3.7 (Markov stochastic policy). Define the set Π_{MS} of Markov randomized (stochastic) control policies, where $\pi_t(\cdot | \mathbf{h}_t)$ is replaced by $\pi_t(\cdot | \mathbf{s}_t)$.

Definition 3.8 (Stationary stochastic policy). Define the set Π_{SS} of stationary randomized (stochastic) control policies, where $\pi_t(\cdot | \mathbf{s}_t)$ is replaced by $\pi(\cdot | \mathbf{s}_t)$.

Note that the set Π_{HS} includes all other policy sets as special cases. For stochastic control policies, we similarly have $\Pi_{HS} \supset \Pi_{MS} \supset \Pi_{SS}$.

Control policies and paths: As mentioned, a deterministic control policy specifies an action for each state, whereas a path specifies an action only for states along the path. The definition of a policy, allows us to consider counter-factual events, namely, what would have been the path if we considered a different action. This distinction is illustrated in Figure 3.3.

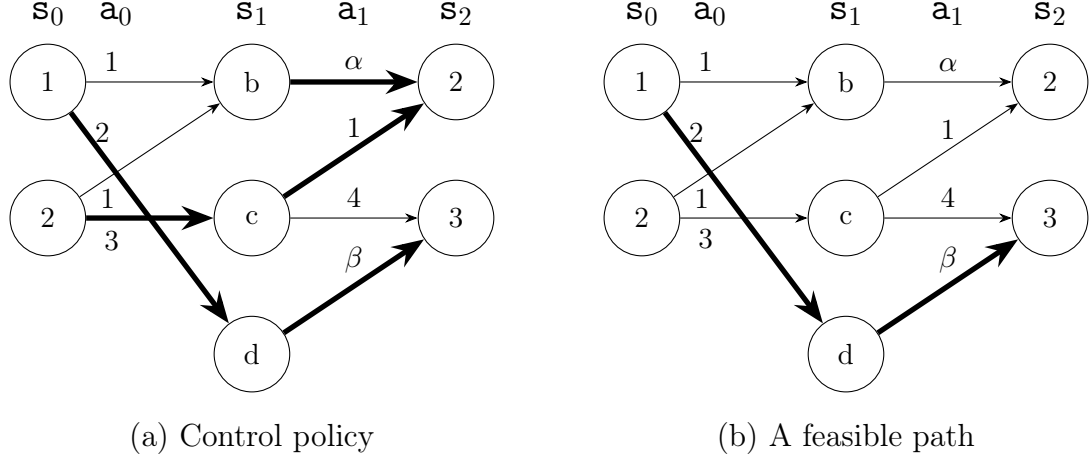


Figure 3.3: Control policy vs. feasible path

Induced Path: A deterministic control policy π , together with an initial state $\mathbf{s}_0$, specify a feasible path $\mathbf{h}_T = (\mathbf{s}_0, \mathbf{a}_0, \dots, \mathbf{s}_{N-1}, \mathbf{a}_{T-1}, \mathbf{s}_T)$. This path may be computed recursively using $\mathbf{a}_t = \pi_t(\mathbf{s}_t)$ and $\mathbf{s}_{t+1} = f_t(\mathbf{s}_t, \mathbf{a}_t)$, for $t = 0, 1, \dots, T-1$.

Remark 3.5. Suppose that for each state $\mathbf{s}_t$, each action $\mathbf{a}_t \in \mathcal{A}_t(\mathbf{s}_t)$ leads to a different state $\mathbf{s}_{t+1}$ (i.e., at most one edge connects any two states). We can then identify each action $\mathbf{a}_t \in \mathcal{A}_t(\mathbf{s}_t)$ with the next state $\mathbf{s}_{t+1} = f_t(\mathbf{s}_t, \mathbf{a}_t)$ it induces. In that case a path may be uniquely specified by the state sequence $(\mathbf{s}_0, \mathbf{s}_1, \dots, \mathbf{s}_T)$.

We define the cumulative cost when starting from state $\mathbf{s}_0$ and following policy π as $\mathcal{C}_T^\pi(\mathbf{s}_0) = \mathcal{C}_T(\mathbf{h}_T)$, where $\mathbf{h}_T$ is an induced path of policy π starting at $\mathbf{s}_0$.

3.2.4 Reduction between Control Policy Classes

We first show a reduction from a general history dependent policies to randomized Markovian policies. The main observation is that the only influence on the cumulative cost is the expected instantaneous cost $\mathbb{E}[c_t(\mathbf{s}_t, \mathbf{a}_t)]$. We define $\rho_t^\pi(\mathbf{s}, \mathbf{a})$ as the occupancy measure when following policy π at time t .

$$\rho_t^\pi(\mathbf{s}, \mathbf{a}) = \Pr_{\mathbf{h}'_{t-1} \sim \pi} [\mathbf{a}_t = \mathbf{a}, \mathbf{s}_t = \mathbf{s}] = \mathbb{E}_{\mathbf{h}'_{t-1} \sim \pi} [\mathbb{I}[\mathbf{s}_t = \mathbf{s}, \mathbf{a}_t = \mathbf{a}] | \mathbf{h}'_{t-1}],$$

where $\mathbf{h}'_{t-1} = (\mathbf{s}_0, \mathbf{a}_0, \dots, \mathbf{s}_{t-1}, \mathbf{a}_{t-1})$ is the history of the first $t-1$ time steps generated using policy π , and the probability and expectation are taken with respect to

the randomness of the policy π . Now we can rewrite the expected cost-to-go as,

$$\mathbb{E}[\mathcal{C}_T^\pi(\mathbf{s}_0)] = \sum_{t=0}^{T-1} \sum_{\mathbf{a} \in \mathcal{A}_t, \mathbf{s} \in \mathcal{S}_t} c_t(\mathbf{s}, \mathbf{a}) \rho_t^\pi(\mathbf{s}, \mathbf{a}),$$

where $\mathbb{E}[\mathcal{C}_T^\pi(\mathbf{s}_0)]$ is the random variable of the cost when starting at state $\mathbf{s}_0$ and following policy π .

This implies that any two policies π and π' for which $\rho_t^\pi(\mathbf{s}, \mathbf{a}) = \rho_t^{\pi'}(\mathbf{s}, \mathbf{a})$, for any time t , state $\mathbf{s}$ and action $\mathbf{a}$, would have the same expected cumulative cost for any cost function, i.e., $\mathbb{E}[\mathcal{C}_T^\pi(\mathbf{s}_0)] = \mathbb{E}[\mathcal{C}_T^{\pi'}(\mathbf{s}_0)]$.

Theorem 3.1. *For any policy $\pi \in \Pi_{HS}$, there is a policy $\pi' \in \Pi_{MS}$, such that for every state $\mathbf{s}$ and action $\mathbf{a}$ and time t we have, $\rho_t^\pi(\mathbf{s}, \mathbf{a}) = \rho_t^{\pi'}(\mathbf{s}, \mathbf{a})$. This implies that,*

$$\mathbb{E}[\mathcal{C}_T^\pi(\mathbf{s}_0)] = \mathbb{E}[\mathcal{C}_T^{\pi'}(\mathbf{s}_0)].$$

Proof. Given the policy $\pi \in \Pi_{HS}$, we define $\pi' \in \Pi_{MS}$ as follows. For every state $\mathbf{s} \in \mathcal{S}_t$ we define,

$$\pi'_t(\mathbf{a}|\mathbf{s}) = \Pr[\mathbf{a}_t = \mathbf{a} | \mathbf{s}_t = \mathbf{s}, \mathbf{h}_{t-1}] = \frac{\rho_t^\pi(\mathbf{s}, \mathbf{a})}{\sum_{\mathbf{a}' \in \mathcal{A}_t} \rho_t^\pi(\mathbf{s}, \mathbf{a}')}.$$

By definition, policy π' is Markovian (depends only on the time t and the realized state $\mathbf{s}$).

We now claim that $\rho_t^\pi(\mathbf{s}, \mathbf{a}) = \rho_t^{\pi'}(\mathbf{s}, \mathbf{a})$. To see this, let us denote $\rho_t^\pi(\mathbf{s}) = \Pr_{\mathbf{h}_{t-1}}[\mathbf{s}_t = \mathbf{s}]$. By construction, we have that $\rho_t^{\pi'}(\mathbf{s}, \mathbf{a}) = \rho_t^{\pi'}(\mathbf{s}) \pi'_t(\mathbf{a}|\mathbf{s}) = \rho_t^{\pi'}(\mathbf{s}) \frac{\rho_t^\pi(\mathbf{s}, \mathbf{a})}{\rho_t^\pi(\mathbf{s})}$. We now show by induction that $\rho_t^{\pi'}(\mathbf{s}) = \rho_t^\pi(\mathbf{s})$. For the base of the induction, by definition we have that $\rho_0^\pi(\mathbf{s}) = \rho_0^{\pi'}(\mathbf{s})$. Assume that $\rho_t^{\pi'}(\mathbf{s}) = \rho_t^\pi(\mathbf{s})$. Then, by the above, we have that $\rho_t^{\pi'}(\mathbf{s}, \mathbf{a}) = \rho_t^\pi(\mathbf{s}, \mathbf{a})$. Then, $\rho_{t+1}^{\pi'}(\mathbf{s}) = \sum_{\mathbf{a}', \mathbf{s}'} \Pr[\mathbf{s}_{t+1} = \mathbf{s} | \mathbf{a}_t = \mathbf{a}', \mathbf{s}_t = \mathbf{s}'] \rho_t^{\pi'}(\mathbf{s}', \mathbf{a}') = \sum_{\mathbf{a}', \mathbf{s}'} \Pr[\mathbf{s}_{t+1} = \mathbf{s} | \mathbf{a}_t = \mathbf{a}', \mathbf{s}_t = \mathbf{s}'] \rho_t^\pi(\mathbf{s}', \mathbf{a}') = \rho_{t+1}^\pi(\mathbf{s})$.

Finally, we obtain that $\rho_t^{\pi'}(\mathbf{s}, \mathbf{a}) = \rho_t^\pi(\mathbf{s}, \mathbf{a})$ for all $t, \mathbf{s}, \mathbf{a}$, and therefore $\mathbb{E}[\mathcal{C}_T^\pi(\mathbf{s}_0)] = \mathbb{E}[\mathcal{C}_T^{\pi'}(\mathbf{s}_0)]$. $\square$

Next we show that for any stochastic Markovian policy there is a deterministic Markovian policy with at most the same cumulative cost.

Theorem 3.2. *For any policy $\pi \in \Pi_{MS}$, there is a policy $\pi' \in \Pi_M$, such that*

$$\mathbb{E}[\mathcal{C}_T^\pi(\mathbf{s}_0)] \geq \mathbb{E}[\mathcal{C}_T^{\pi'}(\mathbf{s}_0)]$$

Proof. The proof is by backward induction on the steps. The inductive claim is:
For any policy $\pi \in \Pi_{MS}$ which is deterministic in $[\mathfrak{t}+1, T]$, there is a policy $\pi' \in \Pi_{MS}$ which is deterministic in $[\mathfrak{t}, T]$ and $E[\mathcal{C}_T^\pi(\mathbf{s}_0)] \geq \mathbb{E}[\mathcal{C}_T^{\pi'}(\mathbf{s}_0)]$, that is, every policy which is deterministic from time $\mathfrak{t}+1$ onward, can be improved by making it deterministic one step earlier, without increasing the associated cost.

Clearly, the theorem follows from the case of $T = 0$. For the base case, $\mathfrak{t} = T$, there are no more actions to take, so the policy is vacuously deterministic and the claim holds trivially.

For the inductive step, assume that policy $\pi \in \Pi_{MS}$ is deterministic in $[\mathfrak{t}+1, T]$.

For every $\mathbf{s}_{\mathfrak{t}+1} \in \mathcal{S}_{\mathfrak{t}+1}$ define

$$\mathcal{C}_{\mathfrak{t}+1:T}^\pi(\mathbf{s}_{\mathfrak{t}+1}) = \mathcal{C}(\text{path}(\mathbf{s}_{\mathfrak{t}+1}, \dots, \mathbf{s}_T)),$$

where $\text{path}(\mathbf{s}_{\mathfrak{t}+1}, \dots, \mathbf{s}_T)$ is the deterministic path from $\mathbf{s}_{\mathfrak{t}+1}$ induced by π .

We define π' to be identical to π for all time steps $\mathfrak{t}' \neq \mathfrak{t}$. We define $\pi'_\mathfrak{t}$ for each $\mathbf{s}_\mathfrak{t} \in \mathcal{S}_\mathfrak{t}$ as follows:

$$\pi'_\mathfrak{t}(\mathbf{s}_\mathfrak{t}) \in \arg \min_{\mathbf{a} \in \mathcal{A}_\mathfrak{t}} c_\mathfrak{t}(\mathbf{s}_\mathfrak{t}, \mathbf{a}) + \mathcal{C}_{\mathfrak{t}+1:T}^\pi(f_\mathfrak{t}(\mathbf{s}_\mathfrak{t}, \mathbf{a})). \quad (3.1)$$

Recall that since we have a Deterministic Decision Process $f_\mathfrak{t}(\mathbf{s}_\mathfrak{t}, \mathbf{a}) \in \mathcal{S}_{\mathfrak{t}+1}$ is the next state if we take action $\mathbf{a}$ in state $\mathbf{s}_\mathfrak{t}$.

For the analysis, note that π and π' are identical until time $\mathfrak{t}$, so they generate exactly the same distribution over paths. At time $\mathfrak{t}$, π' is defined to minimize the cost to go from $\mathbf{s}_\mathfrak{t}$, given that we follow π from $\mathfrak{t}+1$ to T . Therefore the cost can only decrease. Formally, let $\mathbb{E}^\pi[\cdot]$ be the expectation with respect to policy π . We have,

$$\begin{aligned} \mathbb{E}_{\mathbf{s}_\mathfrak{t}}^\pi[\mathcal{C}_{\mathfrak{t}+1:T}^\pi(\mathbf{s}_\mathfrak{t})] &= \mathbb{E}_{\mathbf{s}_\mathfrak{t}}^\pi \mathbb{E}_{\mathbf{a}_\mathfrak{t}}^\pi [c_\mathfrak{t}(\mathbf{s}_\mathfrak{t}, \mathbf{a}_\mathfrak{t}) + \mathcal{C}_{\mathfrak{t}+1:T}^\pi(f_\mathfrak{t}(\mathbf{s}_\mathfrak{t}, \mathbf{a}_\mathfrak{t}))] \\ &\geq \mathbb{E}_{\mathbf{s}_\mathfrak{t}}^\pi \min_{\mathbf{a}_\mathfrak{t} \in \mathcal{A}_\mathfrak{t}} [c_\mathfrak{t}(\mathbf{s}_\mathfrak{t}, \mathbf{a}_\mathfrak{t}) + \mathcal{C}_{\mathfrak{t}+1:T}^\pi(f_\mathfrak{t}(\mathbf{s}_\mathfrak{t}, \mathbf{a}_\mathfrak{t}))] \\ &= \mathbb{E}_{\mathbf{s}_\mathfrak{t}}^{\pi'}[\mathcal{C}_{\mathfrak{t}+1:T}^{\pi'}(\mathbf{s}_\mathfrak{t})], \end{aligned}$$

which completes the inductive proof. $\square$

Remark 3.6. *The above proof extends very naturally for the case of a stochastic MDP, which implies that $f_\mathfrak{t}$ is stochastic. The modification of the proof would simply take an expectation over $f_\mathfrak{t}$ in Eq. (3.1).*

Remark 3.7. *We remark that for the case of deterministic decision processes one can derive a simpler proof, which unfortunately does not extend to stochastic case,*

or other linear return functions. The observation is that any $\pi \in \Pi_{HS}$ induces a distribution over paths. Therefore there is a path p such that $\mathbb{E}[\mathcal{C}_T^\pi(\mathbf{s}_0)] \geq \mathcal{C}(p)$ and for any path p there is a deterministic Markov policy (due to the ability to depend on the time-step).

3.2.5 Optimal Control Policies

Definition 3.9. A control policy $\pi \in \Pi_{MD}$ is called *optimal* if, for each initial state $\mathbf{s}_0$, it induces an optimal path $\mathbf{h}_T$ from $\mathbf{s}_0$.

An alternative definition can be given in terms of policies only. For that purpose, let $\mathbf{h}_T(\pi; \mathbf{s}_0)$ denote the path induced by the policy π from state $\mathbf{s}_0$. For a given return functional $\mathcal{V}_T(\mathbf{h}_T)$, denote $\mathcal{V}_T^\pi(\mathbf{s}_0) = \mathcal{V}_T(\mathbf{h}_T(\pi; \mathbf{s}_0))$. That is, $\mathcal{V}_T^\pi(\mathbf{s}_0)$ is the cumulative return for the path induced by π from $\mathbf{s}_0$.

Definition 3.10. A control policy $\pi \in \Pi_{MD}$ is called *optimal* if, for each initial state $\mathbf{s}_0$, it holds that $\mathcal{V}_T^\pi(\mathbf{s}_0) \geq \mathcal{V}_T^{\tilde{\pi}}(\mathbf{s}_0)$ for any other policy $\tilde{\pi} \in \Pi_{MD}$.

Equivalence of the two definitions can be easily established (we leave this as an exercise). An optimal policy is often denoted by π^* .

The standard T-stage finite-horizon planning problem: Find a control policy π for the T-stage Finite Horizon problem that minimizes the cumulative cost (or maximizes the cumulative return) function.

The naive approach to finding an optimal policy: For finite models (i.e., finite state and action spaces), the number of feasible paths (or control policies) is finite. It is therefore possible, in principle, to enumerate all T-stage paths, compute the cumulative return for each one, and choose the one which gives the largest return. Let us evaluate the number of different paths and control policies. Suppose for simplicity that number of states at each stage is the same: $|\mathcal{S}_t| = n$, and similarly the number of actions at each state is the same: $|\mathcal{A}_t(\mathbf{s})| = m$ (with $m \leq n$ to make the evaluation simpler). The number of feasible T-stage paths for each initial state is seen to be m^T . The number of different policies is m^{nT} . For example, for a fairly small problem with $T = n = m = 10$, we obtain 10^{10} paths for each initial state (and 10^{11} overall), and 10^{100} control policies. Clearly, it is not computationally feasible to enumerate them all. Fortunately, dynamic programming offers a drastic reduction of the computational complexity for this problem, as presented in the next Section.

3.3 Finite Horizon Dynamic Programming

The Dynamic Programming (DP) algorithm breaks down the T -stage finite-horizon problem into T sequential single-stage optimization problems. This results in dramatic improvement in computation efficiency.

The DP technique for dynamic systems is based on a general observation called Bellman's Principle of Optimality [14]. Essentially, it states the following (for deterministic problems): **Any sub-path of an optimal path is itself an optimal path between its end points.**

To see why this should hold, consider a sub-path which is not optimal. We can replace it by an optimal sub-path, and improve the return.

Applying this principle recursively from the last stage backward, obtains the (backward) Dynamic Programming algorithm. Let us first illustrate the idea with following example.

Example 3.4 (Shortest path on a decision graph). *Suppose we wish to find the shortest path (minimum cost path) from the initial node in T steps, on the graph in Figure 3.4.*

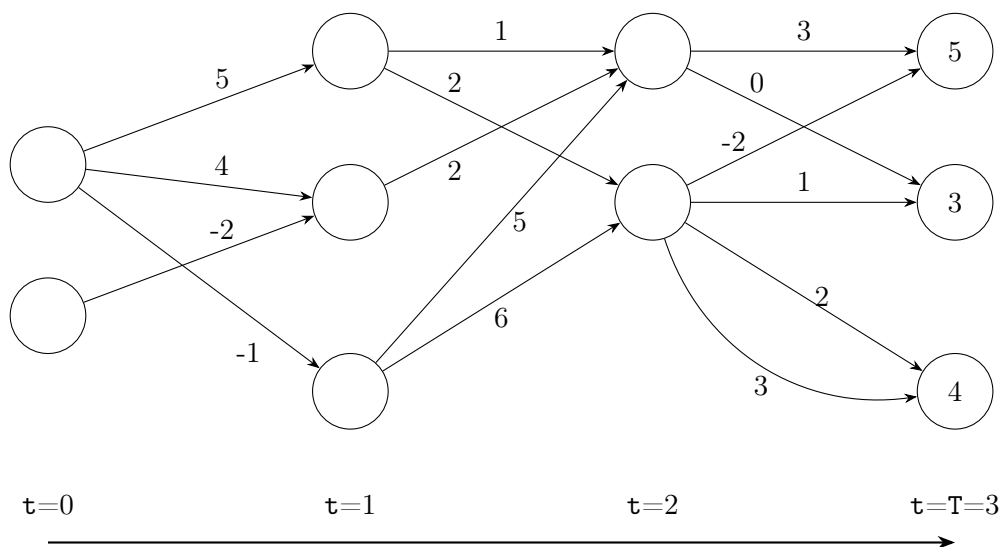


Figure 3.4: A finite horizon decision problem.

The values inside the states are the terminal costs at stage T , the other numbers are the link costs. Using backward recursion, we may obtain that the minimal path

costs from the two initial states are 7 and 3, as well as the optimal paths and an optimal policy. This is depicted in Figure 3.5

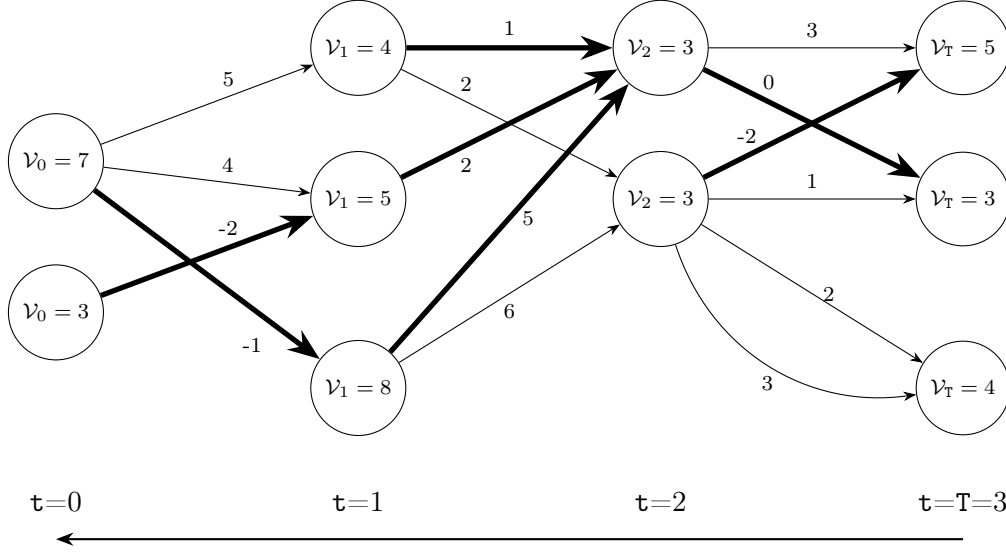


Figure 3.5: Shortest path on a finite horizon decision problem.

We can now describe the DP algorithm. Recall that we consider the dynamic system

$$\begin{aligned} \mathbf{s}_{t+1} &= f_t(\mathbf{s}_t, \mathbf{a}_t), \quad t = 0, 1, 2, \dots, T-1 \\ \mathbf{s}_t &\in \mathcal{S}_t, \quad \mathbf{a}_t \in \mathcal{A}_t(\mathbf{s}_t), \end{aligned}$$

and we wish to maximize the cumulative return:

$$V_{0:T} = \sum_{t=0}^{T-1} r_t(\mathbf{s}_t, \mathbf{a}_t) + r_T(\mathbf{s}_T).$$

The DP algorithm computes recursively a set of **value functions** $V_t : \mathcal{S}_t \rightarrow \mathbb{R}$, where $V_t(\mathbf{s}_t)$ is the maximum cumulative return of an optimal sub-path $\mathbf{h}_{t:T} = (\mathbf{s}_t, \mathbf{a}_t, \dots, \mathbf{s}_T)$ that starts at $\mathbf{s}_t$.

Note that the algorithm involves visiting each state exactly once, proceeding backward in time. For each time instant (or stage) t , the value function $V_t(\mathbf{s})$ is computed for all states $\mathbf{s} \in \mathcal{S}_t$ before proceeding to stage $t-1$. The *backward induction* step of Algorithm 1 (Finite-horizon Dynamic Programming), along with similar equations in the theory of DP, is called **Bellman's equation**.

Algorithm 1 Finite-horizon Dynamic Programming

1: **Initialize the value function:**

2: $\mathcal{V}_T(\mathbf{s}) = r_T(\mathbf{s})$ for all $\mathbf{s} \in \mathcal{S}_T$.

3: **Backward recursion:** For $t = T - 1, \dots, 0$,

4: Compute $\mathcal{V}_t(\mathbf{s}) = \max_{\mathbf{a} \in \mathcal{A}_t} \{r_t(\mathbf{s}, \mathbf{a}) + \mathcal{V}_{t+1}(f_t(\mathbf{s}, \mathbf{a}))\}$ for all $\mathbf{s} \in \mathcal{S}_t$.

5: **Optimal policy:** Choose any control policy $\pi^* = (\pi_t^*)$ that satisfies:

6: $\pi_t^*(\mathbf{s}) \in \arg \max_{\mathbf{a} \in \mathcal{A}_t} \{r_t(\mathbf{s}, \mathbf{a}) + \mathcal{V}_{t+1}(f_t(\mathbf{s}, \mathbf{a}))\}$, for $t = 0, \dots, T - 1$.

Proposition 3.3. *The following holds for finite-horizon dynamic programming:*

1. *The control policy π^* computed in Algorithm 1 (Finite-horizon Dynamic Programming) is an optimal control policy for the T -stage Finite Horizon problem.*
2. *$\mathcal{V}_0(\mathbf{s})$ is the optimal T -stage return from initial state $\mathbf{s}_0 = \mathbf{s}$:*

$$\mathcal{V}_0(\mathbf{s}) = \max_{\pi} \mathcal{V}_{0:T}^{\pi}(\mathbf{s}), \quad \forall \mathbf{s} \in \mathcal{S}_0,$$

where $\mathcal{V}_{0:T}^{\pi}(\mathbf{s})$ is the expected return of policy π when started at state $\mathbf{s}$.

Proof. We show that the computed policy π^* is optimal and its return from time t is $\mathcal{V}_t$. We will establish the following inductive claim:

For any time t and any state $\mathbf{s}$, the path from $\mathbf{s}$ defined by π^ is the maximum return path of length $T - t$. The value of $\mathcal{V}_t(\mathbf{s})$ is the maximum return from $\mathbf{s}$.*

The proof is by a backward induction. For the basis of the induction we have: $t = T$, and the inductive claim follows from the initialization.

Assume the inductive claim holds for t prove for $t + 1$. For contradiction assume there is a higher return path from $\mathbf{s}$. Let the path generated by π^* be $P = (\mathbf{s}, \mathbf{s}_{T-t}^*, \dots, \mathbf{s}_T^*)$. Let $P_1 = (\mathbf{s}, \mathbf{s}_{T-t}, \dots, \mathbf{s}_T)$ be the alternative path with higher return. Let $P_2 = (\mathbf{s}, \mathbf{s}_{T-t}, \mathbf{s}_{T-t-1}', \dots, \mathbf{s}_T')$ be the path generated by following π^* from $\mathbf{s}_{T-t}$. Since P_1 and P_2 are identical except for the last t stages, we can use the inductive hypothesis, which implies that $\mathcal{V}(P_1) \leq \mathcal{V}(P_2)$. From the definition of π^* we have that $\mathcal{V}(P_2) \leq \mathcal{V}(P)$. Hence, $\mathcal{V}(P_1) \leq \mathcal{V}(P_2) \leq \mathcal{V}(P)$, which completes the proof of the inductive hypothesis. $\square$

Let us evaluate the computational complexity of finite horizon DP: there is a total of nT states (excluding the final one), and in each we need m computations. Hence, the number of required calculations is mnT . For the example above with $m = n = T = 10$, we need $O(10^3)$ calculations.

Remark 3.8. *A similar algorithm that proceeds forward in time (from $\mathfrak{t} = 0$ to $\mathfrak{t} = \mathsf{T}$) can be devised. We note that this will not be possible for stochastic systems (i.e., the stochastic MDP model).*

3.4 Shortest Path on a Graph

By definition, the finite horizon problem assumes that we know in advance the number of time steps until termination, as reflected in the cumulative T -stage cost. Some problems, however, are more naturally described by terminating when reaching a particular goal state (not necessarily at a predefined time step). One such class of problems is shortest paths on graphs.

The problem of finding the shortest path over a graph is one of the most fundamental problems in graph theory and computer science. We shall briefly consider here three major algorithms for this problem that are closely related to dynamic programming, namely: The Bellman-Ford algorithm, Dijkstra's algorithm, and A^* . An extensive presentation of the topic can be found in almost any book on algorithms, such as [33, 82, 37].

3.4.1 Problem Statement

We introduce several definitions from graph-theory.

Definition 3.11. *Weighted Graphs:* *Consider a graph $G = (\mathsf{V}, \mathsf{E})$ that consists of a finite set of vertices (or nodes) $\mathsf{V} = \{\mathbf{v}\}$ and a finite set of edges (or links) $\mathsf{E} = \{\mathbf{e}\} \subseteq \mathsf{V} \times \mathsf{V}$. We will consider directed graphs, where each edge $\mathbf{e}$ is equivalent to an ordered pair $(\mathbf{v}_1, \mathbf{v}_2)$ of source and destination nodes, respectively. To each edge $\mathbf{e}$ we assign a real-valued weight (or cost) $c(\mathbf{e}) = c(\mathbf{v}_1, \mathbf{v}_2)$.*

Definition 3.12. *Path:* *A path ω on G from $\mathbf{v}_0$ to $\mathbf{v}_k$ is a sequence $(\mathbf{v}_0, \mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_k)$ of vertices such that $(\mathbf{v}_i, \mathbf{v}_{i+1}) \in \mathsf{E}$. A path is **simple** if all edges in the path are distinct. A **cycle** is a path with $\mathbf{v}_0 = \mathbf{v}_k$.*

Definition 3.13. *Path length:* *The length of a path $c(\omega)$ is the sum of the weights over its edges: $c(\omega) = \sum_{i=1}^k c(\mathbf{v}_{i-1}, \mathbf{v}_i)$.*

A **shortest path** from $\mathbf{u}$ to $\mathbf{v}$ is a path from $\mathbf{u}$ to $\mathbf{v}$ that has the smallest length $c(\omega)$ among such paths. Denote this minimal length as $d(\mathbf{u}, \mathbf{v})$ (with $d(\mathbf{u}, \mathbf{v}) = \infty$ if no path exists from $\mathbf{u}$ to $\mathbf{v}$). The shortest path problem has the following variants:

- Single pair problem: Find the shortest path from a given source vertex u to a given destination vertex v .
- Single source problem: Find the shortest path from a given source vertex u to all other vertices.
- Single destination: Find the shortest path to a given destination node v from all other vertices.
- All pair problem: Find the shortest path from every source vertex u to every destination vertex v .

We note that the single-source and single-destination problems are symmetric and can be treated as one. The all-pair problem can of course be solved by multiple applications of the other algorithms, but there exist algorithms which are especially suited for this problem.

3.4.2 The Dynamic Programming Equation

The DP equation (or Bellman's equation) for the shortest path problem can be written as:

$$d(u, v) = \min \{c(u, u') + d(u', v) : (u, u') \in E\},$$

which holds for any pair of nodes u, v .

The interpretation: $c(u, u') + d(u', v)$ is the length of the path that takes one step from u to u' , and then proceeds optimally. The shortest path is obtained by choosing the best first step. Another version, which singles out the last step, is $d(u, v) = \min \{d(u, v') + c(v', v) : (v', v) \in E\}$. We note that these equations are non-explicit, in the sense that the same quantities appear on both sides. These relations are however at the basis of the following explicit algorithms.

3.4.3 The Bellman-Ford Algorithm

This algorithm solves the single destination (or the equivalent single source) shortest path problem. It allows both positive and negative edge weights. Assume for the moment that there are *no negative-weight cycles*.

The output of the algorithm is $d[v] = d(v, v_d)$, the length of the shortest path from v to v_d , and the policy (routing list) π . A shortest path from vertex v is obtained from π by following the sequence: $v_1 = \pi[v], v_2 = \pi[v_1], \dots, v_d = \pi[v_{k-1}]$. To understand the algorithm, we observe that after round i , $d[v]$ holds the length of the shortest

Algorithm 2 Bellman-Ford Algorithm

```
1: Input: A weighted directed graph  $G$ , and destination node  $v_d$ .
2: Initialization:
3:    $d[v_d] = 0$ ,
4:    $d[v] = \infty$  for all  $v \in V \setminus \{v_d\}$ .
5:    $\triangleright d[v]$  holds the current shortest distance from  $v$  to  $v_d$ .
6: For  $i = 1$  to  $|V| - 1$ 
7:   For each vertex  $v \in V \setminus \{v_d\}$ 
8:      $q[v] = \min_u \{c(v, u) + d[u] \mid (v, u) \in E\}$ 
9:      $\pi[v] \in \arg \min_u \{c(v, u) + d[u] \mid (v, u) \in E\}$ 
10:     $d[v] = q[v] \quad \forall v \in V \setminus \{v_d\}$ 
11: return  $\{d[v], \pi[v] \mid \forall v \in V\}$ 
```

path from v in i steps or less. To see this, observe that the calculations done up to round i are equivalent to the calculations in a finite horizon dynamic programming, where the horizon is i . Since the shortest path takes at most $|V| - 1$ steps, the above claim on optimality follows. The algorithm is provided as Algorithm 2 below.

The running time of the algorithm is $O(|V| \cdot |E|)$. This is because in each round i of the algorithm, each edge e is involved in exactly one update of $d[v]$ for some v . If $\{d[v] : v \in V\}$ does not change at all at some round, then the algorithm may be stopped early.

Remark 3.9. *We have assumed above that no negative-weight cycles exist. In fact the algorithm can be used to check for existence of such cycles: A negative-weight cycle exists if and only if $d[v]$ changes during an additional step ($i = |V|$) of the algorithm.*

Remark 3.10. *The basic scheme above can also be implemented in an asynchronous manner, where each node performs a local update of $d[v]$ at its own time. Further, the algorithm can be started from any initial conditions, although convergence can be slower. This makes the algorithm useful for distributed environments such as internet routing.*

3.4.4 Dijkstra's Algorithm

Dijkstra's algorithm (introduced in 1959, [42]) provides a more efficient algorithm for the single-destination shortest path problem. This algorithm is restricted to non-negative link weights, i.e., $c(v, u) \geq 0$.

The algorithm essentially determines the minimal distance $d(\mathbf{v}, \mathbf{v}_d)$ of the vertices to the destination in order of that distance, namely the closest vertex first, then the second-closest, etc. The algorithm is described below in Algorithm 3. The algorithm maintains a set S of vertices whose minimal distance to the destination has been determined. The other vertices $V \setminus S$ are held in a queue.

Algorithm 3 Dijkstra's Algorithm

```

1: Input: A weighted directed graph  $G$ , and destination node  $\mathbf{v}_d$ .
2: Initialization:
3:    $d[\mathbf{v}_d] = 0$ 
4:    $d[\mathbf{v}] = \infty$  for all  $\mathbf{v} \in V \setminus \{\mathbf{v}_d\}$ 
5:    $\pi[\mathbf{v}] = \emptyset$  for all  $\mathbf{v} \in V$ 
6:    $S = \emptyset$ 
7: while  $S \neq V$  do
8:   Choose  $\mathbf{u} \in V \setminus S$  with minimal value  $d[\mathbf{u}]$ 
9:   Add  $\mathbf{u}$  to  $S$ 
10:  for all  $(\mathbf{v}, \mathbf{u}) \in E$  do
11:    If  $d[\mathbf{v}] > c(\mathbf{v}, \mathbf{u}) + d[\mathbf{u}]$ 
12:       $d[\mathbf{v}] = c(\mathbf{v}, \mathbf{u}) + d[\mathbf{u}]$ 
13:       $\pi[\mathbf{v}] = \mathbf{u}$ 
14:    end for
15: end while
16: return  $\{(d[\mathbf{v}], \pi[\mathbf{v}]) \mid \forall \mathbf{v} \in V\}$ 

```

Let us discuss the running time of Dijkstra's algorithm. Recall that the Bellman-Ford algorithm visits each edge of the graph up to $|V| - 1$ times, leading to a running time of $O(|V| \cdot |E|)$. Dijkstra's algorithm visits each edge only once, which contributes $O(|E|)$ to the running time. The rest of the computation effort is spent on determining the order of node insertion to S .

The vertices in $V \setminus S$ need to be extracted in increasing order of $d[\mathbf{v}]$. This is handled by a min-priority queue, and the complexity of the algorithm depends on the implementation of this queue. With a naive implementation of the queue that simply keeps the vertices in some fixed order, each extract-min operation takes $O(|V|)$ time, leading to overall running time of $O(|V|^2 + |E|)$ for the algorithm. Using a basic (binary heap) priority queue brings the running time to $O((|V| + |E|) \log |V|)$, and a more sophisticated one (Fibonacci heap) can bring it down to $O(|V| \log |V| + |E|)$.

In the following, we prove that Dijkstra's is *complete*, i.e., that it finds the shortest path. Let $d^*[\mathbf{v}]$ denote the shortest path length from $\mathbf{v}$ to $\mathbf{v}_d$.

Theorem 3.4. *Assume that $c(u, v) \geq 0$ for all $u, v \in S$. Then Dijkstra's algorithm terminates with $d[v] = d^*[v]$ for all $v \in S$.*

Proof. We first prove by induction that $d[v] \geq d^*[v]$ throughout the execution of the algorithm. This obviously holds at initialization. Now, assume $d[v] \geq d^*[v] \quad \forall v \in V$ before a relaxation step of edge $(x, y) \in E$. If $d[x]$ changes after the relaxation we have $d[x] = c(x, y) + d[y] \geq c(x, y) + d^*[y] \geq d^*[x]$, where the last inequality is Bellman's equation.

We will next prove by induction that throughout the execution of the algorithm, for each $v \in S$ we have $d[v] = d^*[v]$. The first vertex added to S is v_d , for which the statement holds. Now, assume by contradiction that u is the first node that is going to be added to S for which $d[u] \neq d^*[u]$. We must have that u is connected to v_d , otherwise $d[u] = d^*[u] = \infty$. Let p denote the shortest path from u to v_d . Since p connects a node in $V \setminus S$ to a node in S , it must cross the boundary of S . We can thus write it as $p = u \rightarrow x \rightarrow y \rightarrow v_d$, where $x \in V \setminus S$, $y \in S$, and the path $y \rightarrow v_d$ is inside S . By the induction hypothesis, $d[y] = d^*[y]$. Since x is on the shortest path, it must have been updated when y was inserted into S , so $d[x] = d^*[y] + c(x, y) = d^*[x]$. Since the weights are non-negative, we must have $d[x] = d^*[x] \leq d^*[u] \leq d[u]$ (the last inequality is from the induction proof above). But because both u and x were in S and we chose to update u , we must have $d[x] \geq d[u]$, so $d^*[u] = d[u]$. $\square$

3.4.5 Dijkstra's Algorithm for Single Pair Problems

For the single pair problem, Dijkstra's algorithm can be written in the Single Source Problem formulation, and terminated once the destination node is reached, i.e., when it is popped from the queue. From the discussion above, it is clear that the algorithm will terminate exactly when the shortest path between the source and destination is found. The algorithm is described in Algorithm 4.

In this case, the algorithm may terminate well before visiting all edges of the graph. Therefore, there is no need to explicitly keep the whole graph in memory, and instead we can grow the set of relevant nodes for comparisons as we go along – the *open set* O . Thus, it suffices to have a function that returns the neighbors of a node in the graph, $\text{Neighbors}(u) = \{v : (u, v) \in E\}$ and the cost for an edge $c(u, v)$, to run Dijkstra's algorithm even for very large graphs that do not fit in memory.

Note that in the single source formulation, the output $\pi(v)$ contains the node previous to v in the shortest path from the source node v_s to v . A shortest path from vertex v_s to v_d is obtained from π by following the sequence: $v_d, v_1 = \pi[v_d], v_2 = \pi[v_1], \dots, v_s = \pi[v_{k-1}]$ in reverse order.

Algorithm 4 Dijkstra's Algorithm (Single Pair Problem)

```
1: Input: Functions  $\text{Neighbors}(u)$ ,  $c(u, v)$ , source node  $v_s$ , destination node  $v_d$ .
2: Initialization:
3:    $d[v_s] = 0$ 
4:    $O = \{v_s\}$ 
5:    $S = \emptyset$ 
6: while  $O \neq \emptyset$  do
7:   Choose  $u \in O$  with the minimal value  $d[u]$ 
8:   Add  $u$  to  $S$ , remove  $u$  from  $O$ 
9:   If  $u == v_d$ 
10:    break
11:   for all  $v \in \text{Neighbors}(u), v \notin S$  do
12:     If  $v \notin O$ 
13:       Add  $v$  to  $O$ 
14:        $d[v] = d[u] + c(u, v)$ 
15:        $\pi[v] = u$ 
16:     Else if  $d[v] > d[u] + c(u, v)$ 
17:        $d[v] = d[u] + c(u, v)$ 
18:        $\pi[v] = u$ 
19:   end for
20: end while
21: return  $\{(d[v], \pi[v]) \mid v \in S\}$ 
```

3.4.6 From Dijkstra's Algorithm to A*

Dijkstra's algorithm expands vertices in the order of their distance from the source. When the destination is known (as in the single pair problem), it seems reasonable to bias the search order towards vertices that are closer to the goal.

The A* algorithm [56] implements this idea through the use of a heuristic function $h[v]$, which is an estimate of the distance from vertex v to the goal. It then expands vertices in the order of $d[v] + h[v]$, i.e., the (estimated) length of the shortest path from v_s to v_d that passes through v . The algorithm is described in Algorithm 5.

Algorithm 5 A* Algorithm

```

1: Input: Functions  $\text{Neighbors}(u)$ ,  $c(u, v)$ , source node  $v_s$ , destination node  $v_d$ ,
   heuristic  $h$ .
2: Initialization:
3:    $d[v_s] = 0$ 
4:    $O = \{v_s\}$ 
5:    $S = \emptyset$ 
6: while  $O \neq \emptyset$  do
7:   Choose  $u \in O$  with the minimal value  $d[u] + h[u]$ 
8:   Add  $u$  to  $S$ , remove  $u$  from  $O$ 
9:   If  $u == v_d$ 
10:    break
11:   for all  $v \in \text{Neighbors}(u), v \notin S$  do
12:     If  $v \notin O$ 
13:       Add  $v$  to  $O$ 
14:        $d[v] = d[u] + c(u, v)$ 
15:        $\pi[v] = u$ 
16:     Else if  $d[v] > d[u] + c(u, v)$ 
17:        $d[v] = d[u] + c(u, v)$ 
18:        $\pi[v] = u$ 
19:   end for
20: end while
21: return  $\{(d[v], \pi[v]) \mid v \in S\}$ 

```

Obviously, we cannot expect the estimate $h(v)$ to be exact – if we knew the exact distance then our problem would be solved. However, it turns out that relaxed properties of h are required to guarantee the optimality of A*. We now provide two definitions for desired properties of heuristics.

Definition 3.14. A heuristic is said to be **consistent** if for every adjacent vertices u, v we have that

$$c(v, u) + h[u] - h[v] \geq 0.$$

$$h[v_d] = 0$$

Definition 3.15. A heuristic is said to be **admissible** if it is a lower bound of the shortest path to the goal, i.e., for every vertex u we have that

$$h[u] \leq d[u, v_d],$$

where we recall that $d[u, v]$ denotes the length of the shortest path between u and v .

It is easy to show that every consistent heuristic is also admissible (left as an exercise). It is more difficult to find admissible heuristics that are not consistent. In path finding applications, a popular heuristic that is both admissible and consistent is the Euclidean distance to the goal.

Example 3.5. Consider a path finding problem on a grid-world, depicted in Figure 3.6. An agent (circle) needs to reach the goal (star), and can move only to neighboring cells in the grid that are not obstacles (empty cells). The cost of a move is the distance between the centers of the neighboring cells. It is easy to see that the Euclidean distance to the goal lower bounds the shortest distance on the grid, so the heuristic is admissible. From the triangle inequality of Euclidean geometry, this heuristic is seen to be consistent.

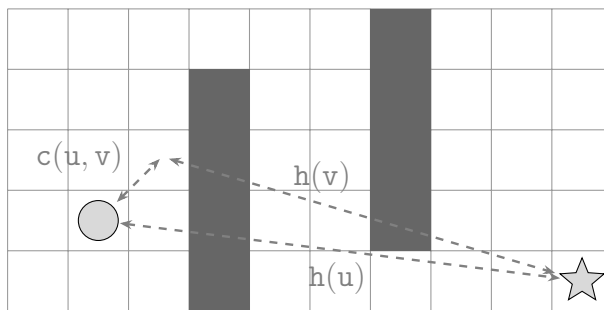


Figure 3.6: Grid world

With a consistent heuristic, A^* is guaranteed to find the shortest path in the graph. With an admissible heuristic, some extra bookkeeping is required to guarantee optimality. We will show optimality for a consistent heuristic by showing that A^* is equivalent to running Dijkstra's algorithm on a graph with modified weights.

Proposition 3.5. *Assume that $c(u, v) \geq 0$ for all $u, v \in S$, and that h is a consistent heuristic. Then the A^* algorithm terminates with $d[v] = d^*[v]$ for all $v \in S$.*

Proof. Define new weights $\hat{c}(u, v) = c(u, v) + h(v) - h(u)$. This transformation does not change the shortest path from v_s to v_d , and the new weights are non-negative due to the consistency property.

The A^* algorithm is equivalent to running Dijkstra's algorithm (for the single pair problem) with the weights $\hat{c}$, and defining $\hat{d}[v] = d[v] + h[v]$. The optimality of A^* therefore follows from the optimality results for Dijkstra's algorithm. $\square$

Remark 3.11. *Actually, a stronger result of optimal efficiency can be shown for A^* : for a given h that is consistent, no other algorithm that is guaranteed to be optimal will explore a smaller set of vertices during the search [56].*

Remark 3.12. *The notion of admissibility is a type of optimism, and is required to guarantee that we do not settle on a suboptimal solution. Later in the book we will see that an analogous idea plays a key role also in learning algorithms.*

Remark 3.13. *In the proof of Proposition 3.5, the idea of changing the cost function to make the problem easier to solve without changing the optimal solution is known as cost shaping, and also plays a role in learning algorithms [113].*

3.5 Average Cost Criterion

The average cost criterion considers the limit of finite time average costs. Formally:

$$\mathcal{C}_{avg}^\pi = \lim_{T \rightarrow \infty} \frac{1}{T} \sum_{t=0}^{T-1} c_t(s_t, a_t),$$

where the trajectory is generated using policy π . The aim is to minimize $\mathbb{E}[\mathcal{C}_{avg}^\pi]$. This implies that any finite prefix has no influence on the final average cost, since its influence vanishes as T goes to infinity. In this section we focus on deterministic decision processes where the problem reduces to inspecting some properties on graphs.

For a deterministic stationary policy, the policy converges to a cycle where every node appears at most once (this is called a simple cycle), and the average cost is the average cost of the edges on the cycle. (Recall, we are considering only deterministic decision processes.)

Given a directed graph $G(V, E)$, let Ω be the collection of all cycles in $G(V, E)$. For each cycle $\omega = (v_1, \dots, v_k)$, we define $c(\omega) = \sum_{i=1}^k c(v_i, v_{i+1})$, where (v_i, v_{i+1}) is the i -th edge in the cycle ω . Let $\mu(\omega) = \frac{c(\omega)}{k}$. The *minimum average cost cycle* is

$$\mu^* = \min_{\omega \in \Omega} \mu(\omega)$$

We show that the minimum average cost cycle is the optimal policy.

Theorem 3.6. *For any Deterministic Decision Process (DDP) the optimal average cost is μ^* , and an optimal policy is π_ω that cycles around a simple cycle of average cost μ^* .*

Proof. Let ω be a cycle of average cost μ^* . Let π_ω be a deterministic stationary policy that first reaches ω and then cycles in ω . Clearly, $\mathcal{C}_{avg}^{\pi_\omega} = \mu^*$.

We will show that for any policy π (possibly in Π_{HS}) we have that $\mathbb{E}[\mathcal{C}_{avg}^\pi] \geq \mu^*$. For contradiction assume that there is a policy π' that has an average cost $\mu^* - \varepsilon$. Consider a sufficiently long run of length T of π' , and fix any trajectory θ of it. We will show that the cumulative cost $\mathcal{C}(\theta) \geq (T - |V|)\mu^*$, which implies that $\mathbb{E}[\mathcal{C}_{avg}^\pi] \geq \mu^* - |V|\mu^*/T$. Then, for $|V|\mu^*/T < \varepsilon$ we have a contradiction.

Given θ , consider the first simple cycle ω_1 in θ . The average cost of ω_1 is $\mu(\omega_1) \geq \mu^*$ and its length is $|\omega_1|$. Delete ω_1 from θ , reducing the number of edges by $|\omega_1|$ and the cumulative cost by $\mu(\omega_1)|\omega_1|$. Given that the average cost in θ is below $\mu^* - \varepsilon$, after deleting ω_1 it will stay below $\mu^* - \varepsilon$. We continue the process until there is no remaining cycles, deleting cycles $\omega_1, \dots, \omega_k$. At the end, since there are no cycles, we have at most $|V|$ nodes remaining, hence $\sum_{i=1}^k |\omega_i| \geq T - |V|$. The cost of θ is at least $\sum_{i=1}^k |\omega_i| \mu(\omega_i) \geq (T - |V|)\mu^*$. This implies that the average cost of θ is at least $\mathbb{E}[\mathcal{C}_{avg}^{\pi'}] \geq (1 - \frac{|V|}{T})\mu^*$. For $\varepsilon > \mu^*|V|/T$ we have $\mathbb{E}[\mathcal{C}_{avg}^{\pi'}] \geq (1 - \frac{|V|}{T})\mu^* > \mu^* - \varepsilon$, which is a contradiction. $\square$

Next we develop an algorithm for computing the minimum average cost cycle, which implies an optimal policy for DDP for average costs. The input is a directed graph $G(V, E)$ with edge cost $c : E \rightarrow \mathbb{R}$.

We first give a characterization of μ^* . Set a root $r \in V$. Let $F_k(v)$ be the set of paths of length k from r to v . Let $d_k(v) = \min_{p \in F_k(v)} c(p)$, where if $F_k(v) = \emptyset$ then $d_k(v) = \infty$, $d_k(v)$ is the lowest cost path from r to v in k steps. The following theorem of Karp [71] gives a characterization of μ^* .

Theorem 3.7. *The value of the minimum cost cycle is*

$$\mu^* = \min_{v \in V} \max_{0 \leq k \leq n-1} \frac{d_n(v) - d_k(v)}{n - k},$$

where we define $\infty - \infty$ as ∞ .

Proof. We assume that the graph has no negative cycle (we can guarantee this by adding a large number M to all the weights). We have two cases, $\mu^* = 0$ and $\mu^* > 0$.

We start with $\mu^* = 0$. This implies that we have in $\mathbf{G}(\mathbf{V}, \mathbf{E})$ a cycle of weight zero. For the theorem it is sufficient to show that,

$$\min_{\mathbf{v} \in \mathbf{V}} \max_{0 \leq k \leq n-1} \{d_n(\mathbf{v}) - d_k(\mathbf{v})\} = 0.$$

For every node $\mathbf{v} \in \mathbf{V}$ there is a path of length $k \in [0, n-1]$ of cost $d(\mathbf{v})$, the cost of the shortest path from $\mathbf{r}$ to $\mathbf{v}$. This implies that

$$\max_{0 \leq k \leq n-1} \{d_n(\mathbf{v}) - d_k(\mathbf{v})\} = d_n(\mathbf{v}) - d(\mathbf{v}) \geq 0$$

We need to show that for some $\mathbf{v} \in \mathbf{V}$ we have $d_n(\mathbf{v}) = d(\mathbf{v})$, which implies that $\min_{\mathbf{v} \in \mathbf{V}} \{d_n(\mathbf{v}) - d(\mathbf{v})\} = 0$.

Consider a cycle ω of cost $\mathcal{C}(\omega) = 0$ (there is one, since $\mu^* = 0$). Let $\mathbf{v}$ be a node on the cycle ω . Consider a shortest path P from $\mathbf{r}$ to $\mathbf{v}$ which then cycles around ω and has length at least n . The path P is a shortest path to $\mathbf{v}$ (although not necessarily simple). This implies that any sub-path of P is also a shortest path. Let P' be a sub-path of P of length n and let it end in $\mathbf{u} \in \mathbf{V}$. Path P' is a shortest path to $\mathbf{u}$, since it is a prefix of a shortest path P . This implies that the cost of P' is $d(\mathbf{u})$. Since P' is of length n , by construction, we have that $d_n(\mathbf{u}) = d(\mathbf{u})$. Therefore, $\min_{\mathbf{v} \in \mathbf{V}} \{d_n(\mathbf{v}) - d(\mathbf{v})\} = 0$, which completes the case that $\mu^* = 0$.

For $\mu^* > 0$ we subtract a constant $\Delta = \mu^*$ from all the costs in the graph. This implies that for the new costs we have a zero *cost* cycle and no negative cycle. We can now apply the previous case. It only remains to show that the formula changes by exactly $\Delta = \mu^*$.

Formally, for every edge $e \in \mathbf{E}$ let $\mathbf{c}'(e) = \mathbf{c}(e) - \Delta$. For any path p we have $\mathcal{C}'(p) = \mathcal{C}(p) - |p|\Delta$, and for any cycle ω we have $\mu'(\omega) = \mu(\omega) - \Delta$. This implies that for $\Delta = \mu^*$ we have a cycle of cost zero and no negative cycles. We now consider

the formula,

$$\begin{aligned}
0 = (\mu')^* &= \min_{\mathbf{v} \in \mathbf{V}} \max_{0 \leq k \leq n-1} \left\{ \frac{d'_n(\mathbf{v}) - d'_k(\mathbf{v})}{n - k} \right\} \\
&= \min_{\mathbf{v} \in \mathbf{V}} \max_{0 \leq k \leq n-1} \left\{ \frac{d_n(\mathbf{v}) - n\Delta - d_k(\mathbf{v}) + k\Delta}{n - k} \right\} \\
&= \min_{\mathbf{v} \in \mathbf{V}} \max_{0 \leq k \leq n-1} \left\{ \frac{d_n(\mathbf{v}) - d_k(\mathbf{v})}{n - k} - \Delta \right\} \\
&= \min_{\mathbf{v} \in \mathbf{V}} \max_{0 \leq k \leq n-1} \left\{ \frac{d_n(\mathbf{v}) - d_k(\mathbf{v})}{n - k} \right\} - \Delta.
\end{aligned}$$

Therefore we have,

$$\mu^* = \Delta = \min_{\mathbf{v} \in \mathbf{V}} \max_{0 \leq k \leq n-1} \left\{ \frac{d_n(\mathbf{v}) - d_k(\mathbf{v})}{n - k} \right\}$$

which completes the proof. $\square$

We now explain how to recover the minimum average cost cycle. The basic idea is to recover the cycle from the minimizing vertices in the formula, but some care is needed to be taken. It is true that for some minimizing pair $(\mathbf{v}, k)$ the path of length n from $\mathbf{r}$ to $\mathbf{v}$ has a cycle of length $n - k$, which is the suffix of the path. The solution is that for the path p , from $\mathbf{r}$ to $\mathbf{v}$ of length n , any simple cycle is a minimum average cost cycle. (See [30].) The running time of computing the minimum average cost cycle is $O(|\mathbf{V}| \cdot |\mathbf{E}|)$.

3.6 Continuous Optimal Control

In this section we consider optimal control of continuous, deterministic, and fully observed systems in discrete time. In particular, consider the following problem:

$$\begin{aligned}
&\min_{\mathbf{a}_0, \dots, \mathbf{a}_T} \sum_{t=0}^T c_t(\mathbf{s}_t, \mathbf{a}_t), \\
&s.t. \quad \mathbf{s}_{t+1} = f_t(\mathbf{s}_t, \mathbf{a}_t),
\end{aligned} \tag{3.2}$$

where $\mathbf{s}_t \in \mathbb{R}^N$, $\mathbf{a}_t \in \mathbb{R}^M$, and the initial state $\mathbf{s}_0$ is given. Here c_t is a (non-linear) cost function at time t , and f_t describes the (non-linear) dynamics at time t . We assume here that f_t and c_t are differentiable.

A simple approach for solving Problem 3.2 is using gradient based optimization. Note that we can expand the terms in the sum using the known dynamics function and initial state:

$$\begin{aligned}\mathcal{V}(\mathbf{a}_0, \dots, \mathbf{a}_T) &= \sum_{t=0}^T c_t(\mathbf{s}_t, \mathbf{a}_t) \\ &= c_0(\mathbf{s}_0, \mathbf{a}_0) + c_1(f_0(\mathbf{s}_0, \mathbf{a}_0), \mathbf{a}_1) + \dots + c_T(f_{T-1}(f_{T-2}(\dots), \mathbf{a}_{T-1}), \mathbf{a}_T).\end{aligned}$$

Using our differentiability assumption, we can compute $\frac{\partial f_t}{\partial \mathbf{s}_t}, \frac{\partial f_t}{\partial \mathbf{a}_t}, \frac{\partial c_t}{\partial \mathbf{s}_t}, \frac{\partial c_t}{\partial \mathbf{a}_t}$. Thus, using repeated application of the chain rule, we can calculate $\frac{\partial \mathcal{V}}{\partial \mathbf{a}_t}$, and optimize $\mathcal{V}$ using gradient descent. There are, however, two potential issues with this approach. The first is that we will only be guaranteed a locally optimal solution. The second is that in practice, a first-order gradient optimization algorithm often converges slowly.

We will now show a different approach. We will first show that for linear systems and quadratic costs, Problem 3.2 can be solved using dynamic programming. This problem is often called a Linear Quadratic Regulator (LQR). We will then show how to extend the LQR solution to non-linear problems using linearization, resulting in an iterative LQR algorithm (iLQR).

3.6.1 Linear Quadratic Regulator

We now restrict our attention to linear-quadratic problems of the form:

$$\begin{aligned}\min_{\mathbf{a}_0, \dots, \mathbf{a}_T} \quad & \sum_{t=0}^T c_t(\mathbf{s}_t, \mathbf{a}_t), \\ \text{s.t.} \quad & \mathbf{s}_{t+1} = A_t \mathbf{s}_t + B_t \mathbf{a}_t, \\ & c_t = \mathbf{s}_t^\top Q_t \mathbf{s}_t + \mathbf{a}_t^\top R_t \mathbf{a}_t, \forall t = 0, \dots, T-1, \\ & c_T = \mathbf{s}_T^\top Q_T \mathbf{s}_T.\end{aligned}\tag{3.3}$$

where $\mathbf{s}_0$ is given, $A_t \in \mathbb{R}^{N \times N}$, $B_t \in \mathbb{R}^{N \times M}$, $Q_t = Q_t^\top \geq 0$ is a symmetric $N \times N$ non-negative definite state-cost matrix, and $R_t = R_t^\top > 0$ is a symmetric $M \times M$ positive definite control-cost matrix.

Example 3.6 (Cart-Pole). *Many physical systems can be described by locally-linear equations of motions. The following is a classic control problem.*

Consider an inverted pendulum on a cart, as depicted in Figure 3.7, where the goal is to stabilize the pendulum in an upright position. Letting x denote the horizontal

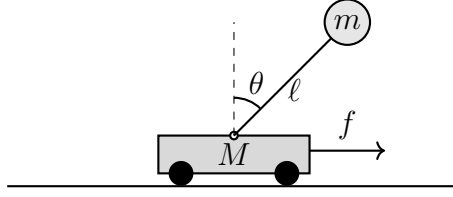


Figure 3.7: Cart-Pole system. A horizontal force f can be applied to the cart with mass M , and the goal is to maintain the mass-less pole of length ℓ with a mass m attached to its tip in an upright position.

position of the cart, and θ being the angle of the pole, the equations of motion for this system are (see Appendix B for an introduction to differential equations):

$$(M + m) \ddot{x} - m\ell \ddot{\theta} \cos \theta + m\ell \dot{\theta}^2 \sin \theta = f,$$

$$m\ell \ddot{\theta} - mg \sin \theta = m\ddot{x} \cos \theta,$$

where g denotes gravity. The equations of motion are in continuous time. We shall now approximate them as a linear system in discrete time.

We first linearize the system around the equilibrium $\theta = 0$. Using the small-angle approximations, and assuming small velocities:

$$\sin \theta \approx \theta, \quad \cos \theta \approx 1, \quad \dot{\theta}^2 \approx 0,$$

we obtain a linear system:

$$(M + m) \ddot{x} - m\ell \ddot{\theta} = f,$$

$$\ell \ddot{\theta} - g\theta = \ddot{x},$$

which can be written as

$$\ddot{x} = \frac{mg}{M} \theta + \frac{1}{M} f,$$

$$\ddot{\theta} = \frac{g}{\ell} \left(1 + \frac{n}{M}\right) \theta + \frac{1}{Mt} f.$$

Define the state and action as:

$$\mathbf{s} = \begin{pmatrix} x \\ \dot{x} \\ \theta \\ \dot{\theta} \end{pmatrix}, \quad \mathbf{a} = f.$$

Plugging in the equations above gives

$$\dot{\mathbf{s}} = \begin{pmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & \frac{mg}{M} & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & \frac{g}{\ell} \left(1 + \frac{n}{M}\right) & 0 \end{pmatrix} \mathbf{s} + \begin{pmatrix} 0 \\ \frac{1}{M} \\ 0 \\ \frac{1}{Ml} \end{pmatrix} \mathbf{a}.$$

Let us now discretize the time into Δt units, that is, let $\mathbf{s}_{\mathbf{t}} = \mathbf{s}(\mathbf{t} \cdot \Delta t)$. A first-order approximation of the dynamics gives $\mathbf{s}_{\mathbf{t}+1} \approx \mathbf{s}_{\mathbf{t}} + \dot{\mathbf{s}}_{\mathbf{t}} \cdot \Delta t$, therefore

$$\mathbf{s}_{\mathbf{t}+1} = \begin{pmatrix} 1 & \Delta t & 0 & 0 \\ 0 & 1 & \frac{mg\Delta t}{M} & 0 \\ 0 & 0 & 1 & \Delta t \\ 0 & 0 & \frac{g}{\ell} \left(1 + \frac{n}{M}\right) \Delta t & 1 \end{pmatrix} \mathbf{s}_{\mathbf{t}} + \begin{pmatrix} 0 \\ \frac{\Delta t}{M} \\ 0 \\ \frac{\Delta t}{Ml} \end{pmatrix} \mathbf{a}_{\mathbf{t}}.$$

Let us now consider the cost function. Choosing

$$\mathbf{c}_{\mathbf{t}} = \mathbf{s}_{\mathbf{t}}^{\top} \begin{pmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix} \mathbf{s}_{\mathbf{t}} + \mathbf{a}_{\mathbf{t}}^{\top} \alpha \mathbf{a}_{\mathbf{t}}$$

penalizes the angle from an upright position, and the control cost for $\alpha > 0$. Note that we obtained a linear quadratic regulator problem (cf. Equation 3.3).

We will solve Problem 3.3 using dynamic programming. Let $\mathcal{V}_{\mathbf{t}}(\mathbf{s})$ denote the value function of a state at time $\mathbf{t}$, that is,

$$\mathcal{V}_{\mathbf{t}}(\mathbf{s}) = \min_{\mathbf{a}_{\mathbf{t}}, \dots, \mathbf{a}_{\mathbf{T}}} \sum_{\mathbf{t}'=\mathbf{t}}^{\mathbf{T}} \mathbf{c}_{\mathbf{t}'}(\mathbf{s}_{\mathbf{t}'}, \mathbf{a}_{\mathbf{t}'}) \quad \text{s.t.} \quad \mathbf{s}_{\mathbf{t}} = \mathbf{s}.$$

Proposition 3.8. *The value function has a quadratic form: $\mathcal{V}_{\mathbf{t}}(\mathbf{s}) = \mathbf{s}^{\top} P_{\mathbf{t}} \mathbf{s}$, and $P_{\mathbf{t}} = P_{\mathbf{t}}^{\top}$ positive semi-definite (all its eigenvalues are positive). Furthermore, the optimal policy is a linear function of the state $\pi_{\mathbf{t}}^*(\mathbf{s}) = K_{\mathbf{t}} \mathbf{s}$.*

Proof. We prove by induction. For $\mathbf{t} = \mathbf{T}$, this holds by definition, as $\mathcal{V}_{\mathbf{T}}(\mathbf{s}) = \mathbf{s}^{\top} Q_{\mathbf{T}} \mathbf{s}$. Now, assume that $\mathcal{V}_{\mathbf{t}+1}(\mathbf{s}) = \mathbf{s}^{\top} P_{\mathbf{t}+1} \mathbf{s}$. We have that

$$\begin{aligned} \mathcal{V}_{\mathbf{t}}(\mathbf{s}) &= \min_{\mathbf{a}_{\mathbf{t}}} [\mathbf{s}^{\top} Q_{\mathbf{t}} \mathbf{s} + \mathbf{a}_{\mathbf{t}}^{\top} R_{\mathbf{t}} \mathbf{a}_{\mathbf{t}} + \mathcal{V}_{\mathbf{t}+1}(A_{\mathbf{t}} \mathbf{s} + B_{\mathbf{t}} \mathbf{a}_{\mathbf{t}})] \\ &= \min_{\mathbf{a}_{\mathbf{t}}} [\mathbf{s}^{\top} Q_{\mathbf{t}} \mathbf{s} + \mathbf{a}_{\mathbf{t}}^{\top} R_{\mathbf{t}} \mathbf{a}_{\mathbf{t}} + (A_{\mathbf{t}} \mathbf{s} + B_{\mathbf{t}} \mathbf{a}_{\mathbf{t}})^{\top} P_{\mathbf{t}+1} (A_{\mathbf{t}} \mathbf{s} + B_{\mathbf{t}} \mathbf{a}_{\mathbf{t}})] \\ &= \mathbf{s}^{\top} Q_{\mathbf{t}} \mathbf{s} + (A_{\mathbf{t}} \mathbf{s})^{\top} P_{\mathbf{t}+1} (A_{\mathbf{t}} \mathbf{s}) + \min_{\mathbf{a}_{\mathbf{t}}} [\mathbf{a}_{\mathbf{t}}^{\top} (R_{\mathbf{t}} + B_{\mathbf{t}}^{\top} P_{\mathbf{t}+1} B_{\mathbf{t}}) \mathbf{a}_{\mathbf{t}} + 2(A_{\mathbf{t}} \mathbf{s})^{\top} P_{\mathbf{t}+1} (B_{\mathbf{t}} \mathbf{a}_{\mathbf{t}})] \\ &= \mathbf{s}^{\top} (Q_{\mathbf{t}} + A_{\mathbf{t}}^{\top} P_{\mathbf{t}+1} A_{\mathbf{t}}) \mathbf{s} + \min_{\mathbf{a}_{\mathbf{t}}} [\mathbf{a}_{\mathbf{t}}^{\top} (R_{\mathbf{t}} + B_{\mathbf{t}}^{\top} P_{\mathbf{t}+1} B_{\mathbf{t}}) \mathbf{a}_{\mathbf{t}} + 2\mathbf{s}^{\top} A_{\mathbf{t}}^{\top} P_{\mathbf{t}+1} B_{\mathbf{t}} \mathbf{a}_{\mathbf{t}}] \end{aligned}$$

The objective is quadratic in $\mathbf{a}_t$, and solving the minimization gives

$$\mathbf{a}_t^* = -(R_t + B_t^\top P_{t+1} B_t)^{-1} B_t^\top P_{t+1} A_t \mathbf{s} \triangleq K_t \mathbf{s}.$$

Substituting back $\mathbf{a}_t^*$ in the expression for $\mathcal{V}_t(\mathbf{s})$ gives a quadratic expression in $\mathbf{s}$. $\square$

From the construction in the proof of Proposition 3.8 one can recover the sequence of optimal controllers $\mathbf{a}_t^*$. By substituting the optimal controls in the forward dynamics equation, one can also recover the optimal state trajectory.

Note that the DP solution is *globally* optimal for the LQR problem. Interestingly, the computational complexity is polynomial in the *dimension* of the state, and linear in the time horizon. In contrast, we could have pursued the dynamic programming approach from Section 3.3 by first discretizing the continuous state and action spaces into finite sets. However, the size of the state and action spaces using discretization grow exponentially with the dimension, making such an approach infeasible for high dimensional systems. This is commonly termed *the curse of dimensionality*. The efficiency of the LQR approach is due to the special structure of the dynamics and cost function, and does not hold in general.

Remark 3.14. *Note that the DP computation resulted in a sequence of linear feedback controllers. It turns out that these controllers are optimal in the presence of Gaussian noise added to the dynamics.*

A similar derivation holds for the system where the cost has an additional linear component (in the action and state):

$$\begin{aligned} & \min_{\mathbf{a}_0, \dots, \mathbf{a}_T} \sum_{t=0}^T c_t(\mathbf{s}_t, \mathbf{a}_t), \\ & s.t. \quad \mathbf{s}_{t+1} = A_t \mathbf{s}_t + B_t \mathbf{a}_t + C_t, \\ & \quad c_t = [\mathbf{s}_t, \mathbf{a}_t]^\top W_t [\mathbf{s}_t, \mathbf{a}_t] + Z_t [\mathbf{s}_t, \mathbf{a}_t] + Y_t, \forall t = 0, \dots, T. \end{aligned}$$

In this case, the optimal control is of the form $\mathbf{a}_t^* = K_t \mathbf{s} + \kappa_t$, for some matrices K_t and vectors κ_t .

3.6.2 Iterative LQR

We now return to the original non-linear problem (3.2). If we linearize the dynamics and quadratize the cost, we can plug in the LQR solution we obtained above. Namely,

given some reference trajectory $\hat{\mathbf{s}}_0, \hat{\mathbf{a}}_0, \dots, \hat{\mathbf{s}}_T, \hat{\mathbf{a}}_T$, we apply a Taylor approximation:

$$\begin{aligned} f_t(\mathbf{s}_t, \mathbf{a}_t) &\approx f_t(\hat{\mathbf{s}}_t, \hat{\mathbf{a}}_t) + \nabla_{\mathbf{s}_t, \mathbf{a}_t} f_t(\hat{\mathbf{s}}_t, \hat{\mathbf{a}}_t) [\mathbf{s}_t - \hat{\mathbf{s}}_t, \mathbf{a}_t - \hat{\mathbf{a}}_t] \\ \mathbf{c}_t(\mathbf{s}_t, \mathbf{a}_t) &\approx \mathbf{c}_t(\hat{\mathbf{s}}_t, \hat{\mathbf{a}}_t) + \nabla_{\mathbf{s}_t, \mathbf{a}_t} \mathbf{c}_t(\hat{\mathbf{s}}_t, \hat{\mathbf{a}}_t) [\mathbf{s}_t - \hat{\mathbf{s}}_t, \mathbf{a}_t - \hat{\mathbf{a}}_t] \\ &\quad + \frac{1}{2} [\mathbf{s}_t - \hat{\mathbf{s}}_t, \mathbf{a}_t - \hat{\mathbf{a}}_t]^\top \nabla_{\mathbf{s}_t, \mathbf{a}_t}^2 \mathbf{c}_t(\hat{\mathbf{s}}_t, \hat{\mathbf{a}}_t) [\mathbf{s}_t - \hat{\mathbf{s}}_t, \mathbf{a}_t - \hat{\mathbf{a}}_t]. \end{aligned} \quad (3.4)$$

If we define $\delta_{\mathbf{s}} = \mathbf{s} - \hat{\mathbf{s}}$, $\delta_{\mathbf{a}} = \mathbf{a} - \hat{\mathbf{a}}$, then the Taylor approximation gives an LQR problem for $\delta_{\mathbf{s}}, \delta_{\mathbf{a}}$. The optimal LQR controller is $\mathbf{a}_t^* = K_t(\mathbf{s}_t - \hat{\mathbf{s}}_t) + \kappa_t + \hat{\mathbf{a}}_t$. By running this controller on the non-linear system, we obtain a new reference trajectory and iteratively seek a good solution for this trajectory. This is done by iterating between the linear approximation and the forward non-linear dynamics as described in Algorithm 6. Also note that the controller $\mathbf{a}_t^* = K_t(\mathbf{s}_t - \hat{\mathbf{s}}_t) + \alpha \kappa_t + \hat{\mathbf{a}}_t$ for $\alpha \in [0, 1]$ smoothly transitions from the previous trajectory ($\alpha = 0$) to the new trajectory ($\alpha = 1$). Therefore we can interpret α as a step size, to guarantee that we stay within the Taylor approximation limits.

Algorithm 6 Iterative LQR

- 1: Initialize a control sequence $\hat{\mathbf{a}}_0, \dots, \hat{\mathbf{a}}_T$ (e.g., by zeros).
 - 2: Run a forward simulation of the controls in the nonlinear system to obtain a state trajectory $\hat{\mathbf{s}}_0, \dots, \hat{\mathbf{s}}_T$.
 - 3: Linearize the dynamics and quadratize the cost (Eq. 3.4), and solve using LQR.
 - 4: By running a forward simulation of the control $\mathbf{a}_t^* = K_t(\mathbf{s}_t - \hat{\mathbf{s}}_t) + \alpha \kappa_t + \hat{\mathbf{a}}_t$ on the non-linear system, perform a line search for the optimal α according to the non-linear cost.
 - 5: For the found α , run a forward simulation to obtain a new trajectory $\hat{\mathbf{s}}_0, \hat{\mathbf{a}}_0, \dots, \hat{\mathbf{s}}_T, \hat{\mathbf{a}}_T$. Go to step 3.
-

In practice, the iLQR algorithm can converge much faster than the simple gradient descent approach.

A treatment of LQR appears in [78]. Our presentation of the iterative LQR follows [160], which is closely related to differential dynamic programming [63].

Chapter 4

Markov Chains

Our goal in extending the deterministic decision making framework of Chapter 3 to stochastic models is to develop a mathematical model for decision making under uncertainty. With the goal of presenting such a model in mind, in this chapter we cover the fundamentals of the Markov chain stochastic process. A detailed treatment can be found in various books, e.g., [96, 121].

Chapter Orientation

Main Prerequisites:

- Probability theory

Main terminology:

- X_t state of the Markov chain at time t
- $p_{i,j}$ probability of transitioning from state i to state j
- $p_{i,j}^{(m)}$ m -step transition probabilities
- P transition matrix
- p_0 initial state distribution
- d_i period of state i
- T_i return time to state i

- μ invariant distribution

A *Markov chain* $\{X_t, t = 0, 1, 2, \dots\}$, with $X_t \in X$, is a discrete-time stochastic process, over a finite or countable state-space X , that satisfies the following Markov property:

$$\Pr(X_{t+1} = j | X_t = i, X_{t-1}, \dots, X_0) = \Pr(X_{t+1} = j | X_t = i).$$

We focus on time-homogeneous Markov chains, where

$$\Pr(X_{t+1} = j | X_t = i) = \Pr(X_1 = j | X_0 = i) \triangleq p_{i,j}.$$

The $p_{i,j}$'s are the transition probabilities, which satisfy $p_{i,j} \geq 0$, and for each $i \in X$ we have $\sum_{j \in X} p_{i,j} = 1$, namely, $\{p_{i,j} : j \in X\}$ is a distribution on the next state following state i . The matrix $P = (p_{i,j})$ is the transition matrix. The matrix is row-stochastic (each row sums to 1 and all entries non-negative).

Given the initial distribution p_0 of X_0 , namely $\Pr(X_0 = i) = p_0(i)$, we obtain the finite-dimensional distributions for states $i_0, i_1, \dots, i_t$:

$$\Pr(X_0 = i_0, X_1 = i_1, \dots, X_t = i_t) = p_0(i_0)p_{i_0,i_1} \cdot \dots \cdot p_{i_{t-1},i_t}.$$

Define $p_{i,j}^{(m)} = \Pr(X_m = j | X_0 = i)$, the m -step transition probabilities. It is easy to verify that $p_{i,j}^{(m)} = [P^m]_{ij}$, where P^m is the m -th power of the matrix P .

Example 4.1. Consider the following two state Markov chain, with transition probability P and initial distribution p_0 , as follows:

$$P = \begin{pmatrix} 0.4 & 0.6 \\ 0.2 & 0.8 \end{pmatrix} \quad p_0 = (0.5 \quad 0.5)$$

Initially, we have both states equally likely. After one step, the distribution of states is $p_1 = p_0 P = (0.3, 0.7)$. After two steps we have $p_2 = p_1 P = p_0 P^2 = (0.26, 0.74)$. The limit of this sequence would be $p_\infty = (0.25, 0.75)$, which is called the steady state distribution, and would be discussed later.

4.1 State Classification

We classify the Markov chain given the ability to transition between states. We start by defining the *accessibility* (or *reachability*) relation between two states, which capture the fact that the Markov state can transition from one state to another with some positive probability.

Definition 4.1. State j is accessible (or reachable) from i (denoted by $i \rightarrow j$) if $p_{i,j}^{(m)} > 0$ for some $m \geq 1$.

For a finite state space X we can compute the accessibility property as follows. Construct a directed graph $G(X, E)$ where the vertices are the states X and there is a directed edge (i, j) if $p_{i,j} > 0$. State j is accessible from state i iff there exists a directed path in $G(X, E)$ from i to j .

Note that the relation is transitive. If $i \rightarrow j$ and $j \rightarrow k$ then $i \rightarrow k$. This follows since $i \rightarrow j$ implies that there is m_1 such that $p_{i,j}^{(m_1)} > 0$. Similarly, since $j \rightarrow k$ there is m_2 such that $p_{j,k}^{(m_2)} > 0$. Therefore, for $m = m_1 + m_2$ we have $p_{i,k}^{(m)} > p_{i,j}^{(m_1)} p_{j,k}^{(m_2)} > 0$. States i and j are *communicating* states (or communicate) if $i \rightarrow j$ and $j \rightarrow i$. For a finite X , this implies that in $G(X, E)$ there is both a directed path from i to j and from j to i .

Definition 4.2. A communicating class (or just class) is a maximal collection of states that communicate.

For a finite state space X , this implies that in $G(X, E)$ we have i and j in the same strongly connected component of the graph. (A strongly connected component has a directed path between any pair of vertices.)

Definition 4.3. The Markov chain is irreducible if all states belong to a single class (i.e., all states communicate with each other).

For a finite state space X , this implies that $G(X, E)$ is strongly connected.

Next, we define the period of a state in a Markov chain. The period intuitively captures the fact that the time between any two occurrences of the state is always divisible by some integer, which is the period.

Definition 4.4. State i has a period $d_i = \text{GCD}\{m \geq 1 : p_{i,i}^{(m)} > 0\}$, where GCD is the greatest common divisor. A state is aperiodic if $d_i = 1$.

State i is periodic with period $d_i \geq 2$ if $p_{i,i}^{(m)} = 0$ for $m \pmod{d_i} \neq 0$ and for any m such that $p_{i,i}^{(m)} > 0$ we have $m \pmod{d_i} = 0$. If a state i is aperiodic, then there exists an integer m_0 such that for any $m \geq m_0$ we have $p_{i,i}^{(m)} > 0$. Periodicity is a class property: all states in the same class have the same period. Specifically, if some state is *aperiodic*, then all states in the class are aperiodic.

Claim 4.1. For any two states i and j with periods d_i and d_j , in the same communicating class, we have $d_i = d_j$.

Proof. For contradiction, assume that $d_j \pmod{d_i} \neq 0$. Since they are in the same communicating class, we have a trajectory from i to j of length $m_{i,j}$ and from j to i of length $m_{j,i}$. This implies that $(m_{i,j} + m_{j,i}) \pmod{d_i} = 0$. Now, there is a trajectory (which is a cycle) of length $m_{j,j}$ from j back to j such that $m_{j,j} \pmod{d_i} \neq 0$ (otherwise d_i divides the period of j). Consider the path from i to itself of length $m_{i,j} + m_{j,j} + m_{j,i}$. We have that $(m_{i,j} + m_{j,j} + m_{j,i}) \pmod{d_i} = m_{j,j} \pmod{d_i} \neq 0$. This is a contradiction to the definition of d_i . Therefore, $d_j \pmod{d_i} = 0$ and similarly $d_i \pmod{d_j} = 0$, which implies that $d_i = d_j$. $\square$

The claim shows that periodicity is a class property, and all the states in a class have the same period.

4.2 Recurrence

We define the property that a state will reappear in the Markov chain. A *recurrent state* is a state which with probability 1 will reappear in the Markov chain, otherwise it is a *transient state*. The formal definition is following.

Definition 4.5. State i is recurrent if $\Pr(X_{\mathbf{t}} = i \text{ for some } \mathbf{t} \geq 1 | X_0 = i) = 1$. Otherwise, state i is transient.

We can relate the state property of recurrent and transient to the expected number of returns to a state.

Claim 4.2. State i is transient iff $\sum_{m=1}^{\infty} p_{i,i}^{(m)} < \infty$.

Proof. Assume that state i is transient. Let $q_i = \Pr(X_{\mathbf{t}} = i \text{ for some } \mathbf{t} \geq 1 | X_0 = i)$. Since state i is transient we have $q_i < 1$. Let Z_i be the number of times the trajectory returns to state i . Note that Z_i is geometrically distributed with parameter q_i , namely $\Pr[Z_i = k] = q_i^k(1 - q_i)$. Therefore the expected number of returns to state i is $\mathbb{E}[Z] = \sum_{k=0}^{\infty} k(1 - q_i)q_i^k = (1 - q_i) \sum_{k=0}^{\infty} kq_i^k = (1 - q_i) \frac{q_i}{(1 - q_i)^2} = \frac{q_i}{1 - q_i}$ and is finite. The expected number of returns to state i is equivalently $\sum_{m=1}^{\infty} p_{i,i}^{(m)}$, and hence if a state is transient we have $\sum_{m=1}^{\infty} p_{i,i}^{(m)} < \infty$.

For the other direction, assume that $\sum_{m=1}^{\infty} p_{i,i}^{(m)} < \infty$. This implies that there is an m_0 such that $\sum_{m=m_0}^{\infty} p_{i,i}^{(m)} < 1/2$. Consider the probability of returning to i within m_0 steps. This implies that $\Pr(X_{\mathbf{t}} = i \text{ for some } t \geq m_0 | X_0 = i) < 1/2$. Now consider the probability $q'_i = \Pr(X_{\mathbf{t}} = i \text{ for some } m_0 \geq \mathbf{t} \geq 1 | X_0 = i)$. If $q'_i < 1$, this implies that $\Pr(X_{\mathbf{t}} = i \text{ for some } \mathbf{t} \geq 1 | X_0 = i) < q'_i + (1 - q'_i)/2 = (1 + q'_i)/2 < 1$, which implies that state i is transient. If $q'_i = 1$, this implies that after at most m_0

steps we are guaranteed to return to i , hence the expected number of return to state i is infinite, i.e., $\sum_{m=1}^{\infty} p_{i,i}^{(m)} = \infty$. This is in contradiction to the assumption that $\sum_{m=1}^{\infty} p_{i,i}^{(m)} < \infty$. $\square$

Claim 4.3. *Recurrence is a class property.*

Proof. To see this consider two states i and j in the same communicating class and i is recurrent. Since i is recurrent $\sum_{m=1}^{\infty} p_{i,i}^{(m)} = \infty$. Since j is accessible from i there is a k such that $p_{i,j}^{(k)} > 0$. Since i is accessible from j , there exists a k' such that $p_{j,i}^{(k')} > 0$. We can lower bound $\sum_{m=1}^{\infty} p_{j,j}^{(m)}$ by $\sum_{m=1}^{\infty} p_{j,j}^{(m)} \geq p_{j,i}^{(k')} \left(\sum_{m=1}^{\infty} p_{i,i}^{(m)} \right) p_{i,j}^{(k)} = \infty$. Therefore we showed that state j is recurrent. $\square$

Claim 4.4. *If states i and j are in the same recurrent (communicating) class, then state j is (eventually) reached from state i with probability 1, namely, $\Pr(X_t = j \text{ for some } t \geq 1 | X_0 = i) = 1$.*

Proof. This follows from the fact that both states occur infinitely often, with probability 1. $\square$

Definition 4.6. *Let T_i be the first return time to state i , defined as the minimum $t \geq 1$ such that $X_t = i$, given $X_0 = i$. That is, $T_i = \inf\{t \geq 1 : X_t = i\}$.*

Claim 4.5. *If i is a recurrent state, then $T_i < \infty$ w.p. 1.*

Proof. Since otherwise, there is a positive probability that we never return to state i , and hence state i is not recurrent. $\square$

We now would like consider the expected time to return to a state. The claim shows that any recurrent state has a finite return time, but this does not imply that the expected return time is finite. (For example, consider a random variable T such that $\Pr[T = 2^k] = 2^{-k}$. In this example, $\mathbb{E}[T] = \sum_k 2^k 2^{-k} = \sum_k 1 = \infty$.)

Definition 4.7. *State i is positive recurrent if $\mathbb{E}(T_i) < \infty$, and null recurrent if $\mathbb{E}(T_i) = \infty$.*

We now show that in the case of finite state space, we do not have null recurrent states.

Claim 4.6. *If the state space X is finite, all recurrent states are positive recurrent.*

Proof. This follows since the set of states that are null recurrent cannot have transitions from positive recurrent states and cannot have a transition to transient states. If the chain never leaves the set of null recurrent states, then some state would have a return time which is at most the size of the set. If there is a positive probability of leaving the set (and never returning) then the states are transient. (See the proof of Theorem 4.10 for a more formal proof of a similar claim for countable Markov Chains.) $\square$

In the following we illustrate some of the notions that we define. we start with the classic random walk on the integers, where all integer (states) are null recurrent.

Example 4.2. Random walk Consider the following Markov chain over the integers. The states are the integers. The initial state is 0. At each state i , with probability $\frac{1}{2}$ we move to $i + 1$ and with probability $\frac{1}{2}$ to $i - 1$. Namely, $p_{i,i+1} = \frac{1}{2}$, $p_{i,i-1} = \frac{1}{2}$, and $p_{i,j} = 0$ for $j \notin \{i - 1, i + 1\}$. We will show that T_i is finite with probability 1 and $\mathbb{E}[T_i] = \infty$. This implies that all the states are null recurrent.

To compute $\mathbb{E}[T_i]$ consider what happens after one and two steps. Let $Z_{i,j}$ be the time to move from i to j . Note that we have,

$$\mathbb{E}[T_i] = 1 + 0.5\mathbb{E}[Z_{i+1,i}] + 0.5\mathbb{E}[Z_{i-1,i}] = 1 + \mathbb{E}[Z_{1,0}],$$

since, due to symmetry, $\mathbb{E}[Z_{i,i+1}] = \mathbb{E}[Z_{i+1,i}] = \mathbb{E}[Z_{1,0}]$.

After two steps we are either back to i , or at state $i + 2$ or state $i - 2$. For $\mathbb{E}[T_i]$ we have that,

$$\begin{aligned} \mathbb{E}[T_i] &= 2 + \frac{1}{4}\mathbb{E}[Z_{i+2,i}] + \frac{1}{4}\mathbb{E}[Z_{i-2,i}] \\ &= 2 + \frac{1}{4}(\mathbb{E}[Z_{i+2,i+1}] + \mathbb{E}[Z_{i+1,i}]) + \frac{1}{4}(\mathbb{E}[Z_{i-2,i-1}] + \mathbb{E}[Z_{i-1,i}]) \\ &= 2 + \mathbb{E}[Z_{1,0}], \end{aligned}$$

where the first identity uses the fact that $Z_{i+2,i} = Z_{i+2,i+1} + Z_{i+1,i}$, since in order to reach from state $i + 2$ to state i we need to first reach from state $i + 2$ state $i + 1$, and then from state $i + 1$ to state i .

This implies that we have

$$1 + \mathbb{E}[Z_{1,0}] = \mathbb{E}[T_i] = 2 + \mathbb{E}[Z_{1,0}]$$

Clearly, there is no finite value for $\mathbb{E}[Z_{1,0}]$ which will satisfy both equations, which implies $\mathbb{E}[Z_{1,0}] = \infty$, and hence $\mathbb{E}[T_i] = \infty$.

To show that state 0 is a recurrent state, note that the probability that at time $2k$ we are at state 0 is exactly $p_{0,0}^{(2k)} = \binom{2k}{k} 2^{-2k} \approx \frac{c}{\sqrt{k}}$ (using Stirling's approximation), for some constant $c > 0$. This implies that

$$\sum_{m=1}^{\infty} p_{0,0}^{(m)} \approx \sum_{m=1}^{\infty} \frac{c}{\sqrt{m}} = \infty$$

and therefore state 0 is recurrent. (By symmetry, this shows that all the states are recurrent.)

Note that this Markov chain has a period of 2. This follows since any trajectory starting at 0 and returning to 0 has an equal number of $+1$ and -1 and therefore of even length. Any even number n has a trajectory of this length that starts at 0 and returns to 0, for example, having $\frac{n}{2}$ times $+1$ followed by $\frac{n}{2}$ times -1 .

The next example is a simple modification of the random walk, where each time we either return to the origin or continue to the next integer with equal probability. This Markov chain will have all (non-negative) integers as positive recurrent states.

Example 4.3. Random walk with jumps. Consider the following Markov chain over the integers. The states are the integers. The initial state is 0. At each state i , with probability $\frac{1}{2}$ we move to $i + 1$ and with probability $\frac{1}{2}$ we return to 0. Namely, $p_{i,i+1} = \frac{1}{2}$, $p_{i,0} = \frac{1}{2}$, and $p_{i,j} = 0$ for $j \notin \{0, i + 1\}$. We will show that $\mathbb{E}[T_i] < \infty$ (which implies that T_i is finite with probability 1).

From any state we return to 0 with probability $\frac{1}{2}$, therefore $\mathbb{E}[T_0] = 2$ (The return time is 1 with probability $\frac{1}{2}$, 2 with probability $(\frac{1}{2})^2$, k with probability $(\frac{1}{2})^k$, and computing the expectation gives $\sum_{k=1}^{\infty} \frac{k}{2^k} = 2$). We will show that for state i we have $\mathbb{E}[T_i] \leq 2 + 2 \cdot 2^i$. We will decompose T_i to two parts. The first is the return to 0, this part has expectation 2. The second is to reach state i from state 0. Consider an epoch as the time between two visits to 0. The probability that an epoch would reach i is exactly 2^{-i} . The expected time of an epoch is 2 (the expected time to return to state 0). The expected time to return to state 0, given that we did not reach state i is less than 2. Therefore, $\mathbb{E}[T_i] \leq 2 + 2 \cdot 2^i$. Note that this Markov chain is aperiodic.

4.3 Invariant Distribution

In general, we have an expected state distribution of the Markov chain depending on the time and the start state. We would like to capture what happens as we run the Markov chain for a long time. Intuitively, we would like to see convergence of the state distribution. For this we need the Markov chain to be aperiodic, since otherwise

the periodicity will not allow convergence, since a periodic state will switch from zero to non-zero probability. We would also like to have the Markov chain irreducible, otherwise some states will have a vanishing but non-zero probability. This limit distribution is called in a few different names, all meaning the same concept: invariant distribution, stationary distribution, and steady state distribution.

The probability vector $\mu = (\mu_i)$ is an *invariant distribution* (or *stationary distribution* or *steady state distribution*) for the Markov chain if $\mu^\top P = \mu^\top$, namely

$$\mu_j = \sum_i \mu_i p_{i,j} \quad \forall j.$$

Clearly, if $X_t \sim \mu$ then $X_{t+1} \sim \mu$. If $X_0 \sim \mu$, then the Markov chain (X_t) is a *stationary stochastic process*.

Theorem 4.7. *Let (X_t) be an irreducible and aperiodic Markov chain over a finite state space X with transition matrix P . Then there is a unique distribution $\mu > 0$ such that $\mu^\top P = \mu^\top > 0$.*

Proof. Assume that x is an eigenvector of P with eigenvalue λ , i.e., we have $Px = \lambda x$. Since P is a stochastic matrix, we have $\|Px\|_\infty \leq \|x\|_\infty$, which implies that $|\lambda| \leq 1$. Since the matrix P is row stochastic, $P\vec{1} = \vec{1}$, which implies that P has a right eigenvalue of 1 and this is the maximal eigenvalue. Since the sets of right and left eigenvalues are identical for square matrices, we conclude that there is x such that $x^\top P = x^\top$. Our first task is to show that there is such an x with $x \geq 0$.

Since the Markov chain is irreducible and aperiodic, there is an integer m , such that P^m has all the entries strictly positive. Namely, for any $i, j \in X$ we have $p_{i,j}^{(m)} > 0$.

We now show a general property of positive matrices (matrices where all the entries are strictly positive). Let $A = P^m$ be a positive matrix and x an eigenvector of A with eigenvalue 1. First, if x has complex entries then $Re(x)$ and $Im(x)$ are eigenvectors of A of eigenvalue 1 and one of them is non-zero. Therefore we can assume that $x \in \mathbb{R}^d$. We would like to show that there is an $x \geq 0$ such that $x^\top A = x^\top$. If $x \geq 0$ we are done. If $x \leq 0$ we can take $x' = -x$ and we are done. We need to show that x cannot have both positive and negative entries.

For contradiction, assume that we have $x_k > 0$ and $x_{k'} < 0$. This implies that for any weight vector $w > 0$ we have $|x^\top w| < |x|^\top w$, where $|x|$ is point-wise absolute value. Therefore,

$$\sum_j |x_j| = \sum_j \left| \sum_i x_i A_{i,j} \right| < \sum_j \sum_i |x_i| A_{i,j} = \sum_i |x_i| \sum_j A_{i,j} = \sum_i |x_i|,$$

where the first identity follows since x is an eigenvector. The second since P is strictly positive. The third is a change of order of summation. The last follows since P is a row stochastic matrix, so each row sums to 1. Clearly, we reached a contradiction, and therefore x cannot have both positive and negative entries.

We have shown so far that there exists a μ such that $\mu^\top P = \mu^\top$ and $\mu \geq 0$. This implies that $\frac{\mu}{\|\mu\|_1}$ is a steady state distribution. Since $A = P^m$ is strictly positive, then $\mu^\top = \mu^\top A > 0$.

To show the uniqueness of μ , assume we have x and y such that $x^\top P = x^\top$ and $y^\top P = y^\top$ and $x \neq y$. Recall that we showed that in such a case both $x > 0$ and $y > 0$. Then there is a linear combination $z = ax + by$ such that for some i we have $z_i = 0$. Since $z^\top P = z^\top$, we have showed that z is strictly positive, i.e., $z > 0$, which is a contradiction. Therefore, $x = y$, and hence μ is unique. $\square$

We define the average fraction that a state $j \in X$ occurs, given that we start with an initial state distribution x_0 up to time m , as follows:

$$\pi_j^{(m)} = \frac{1}{m} \sum_{t=1}^m \mathbb{I}(X_t = j).$$

Theorem 4.8. *Let (X_t) be an irreducible and aperiodic Markov chain over a finite state space X with transition matrix P . Let μ be the stationary distribution of P . Then, for any $j \in X$ we have,*

$$\mu_j = \lim_{m \rightarrow \infty} \mathbb{E}[\pi_j^{(m)}] = \frac{1}{\mathbb{E}[T_j]}.$$

Proof. We have that

$$\mathbb{E}[\pi_j^{(m)}] = \mathbb{E}\left[\frac{1}{m} \sum_{t=1}^m \mathbb{I}(X_t = j)\right] = \frac{1}{m} \sum_{t=1}^m \Pr[X_t = j | X_0 = x_0] = \frac{1}{m} \sum_{t=1}^m x_0^\top P^t e_j,$$

where e_j denotes a vector of zeros, with 1 only in the j 's element. Let $v_1, \dots, v_n$ be the eigenvectors of P with eigenvalues $\lambda_1 \geq \dots \geq \lambda_n$. By Theorem 4.7 we have that $v_1 = \mu$, the stationary distribution and $\lambda_1 = 1 > \lambda_i$ for $i \geq 2$. Rewrite $x_0 = \sum_i \alpha_i v_i$. Since P^m is a stochastic matrix, $x_0^\top P^m$ is a distribution, and therefore $\lim_{m \rightarrow \infty} x_0^\top P^m = \mu$ (this is known as the limit law of Markov Chains. For a proof, see [69], Theorem 1.3 on p.85-86).

We will be interested in the limit $\pi_j = \lim_{m \rightarrow \infty} \pi_j^{(m)}$, and mainly in the expected value $\mathbb{E}[\pi_j]$. From the above we have that $\mathbb{E}[\pi_j] = \mu_j$.

A different way to express $\mathbb{E}[\pi_j]$ is using a variable time horizon, with a fixed number of occurrences of j . Let $T_{k,j}$ be the time between the k and $k+1$ occurrence of state j . This implies that

$$\lim_{m \rightarrow \infty} \frac{1}{m} \sum_{t=1}^m \mathbb{I}(X_t = j) = \lim_{n \rightarrow \infty} \frac{n}{\sum_{k=1}^n T_{k,j}}.$$

Note that the $T_{k,j}$ are i.i.d. because of the Markov assumption and equivalent to T_j . By the law of large numbers we have that $\frac{1}{n} \sum_{k=1}^n T_{k,j}$ converges to $\mathbb{E}[T_j]$. Therefore,

$$\mathbb{E}[\pi_j] = \frac{1}{\mathbb{E}[T_j]}.$$

□

We have established the following general theorem.

Theorem 4.9 (Recurrence of finite Markov chains). *Let (X_t) be an irreducible, aperiodic Markov chain over a finite state space X . Then the following properties hold:*

1. *All states are **positive recurrent***
2. *There exists a **unique stationary distribution** μ , where $\mu_i = \mathbb{E}[T_i]^{-1}$.*
3. ***Convergence to the stationary distribution:** $\lim_{t \rightarrow \infty} \Pr[X_t = j] = \mu_j \quad (\forall j)$*
4. ***Ergodicity:** For any finite f : $\lim_{t \rightarrow \infty} \frac{1}{t} \sum_{s=0}^{t-1} f(X_s) = \sum_i \mu_i f(i) \triangleq \pi \cdot f$.*

We prove this theorem for the case that P is a diagonalizable matrix. In the proof we sketch how to extend this result to the general case.

Proof. From Theorem 4.7, we have that $\mu > 0$, and from Theorem 4.8 we have that $\mathbb{E}[T_i] = \mu_i^{-1} < \infty$. This establishes (1) and (2).

For any initial distribution x_0 we have that

$$\Pr[X_t = j] = x_0^\top P^t e_j,$$

where e_j denotes a vector of zeros, with 1 only in the j 's element. Let $v_1, \dots, v_n$ be the eigenvectors of P with eigenvalues $\lambda_1 \geq \dots \geq \lambda_n$. By Theorem 4.7 we have that

$v_1 = \mu$, the stationary distribution and $\lambda_1 = 1 > \lambda_i$ for $i \geq 2$. Assume that P is diagonalizable, and rewrite $x_0 = \sum_i \alpha_i v_i$.¹ We have that

$$\Pr[X_t = j] = \sum_i \alpha_i \lambda_i^t v_i^\top e_j,$$

and therefore $\lim_{t \rightarrow \infty} \Pr[X_t = j] = \lambda_1 \mu^\top e_j = \lambda_1 \mu_j$. Since P^t is a stochastic matrix, $x_0^\top P^t$ is a distribution, and therefore $\lambda_1 = 1$. This establishes (3).

Finally, we establish (4) following the proof of Theorem 4.8:

$$\begin{aligned} \lim_{t \rightarrow \infty} \frac{1}{t} \sum_{s=0}^{t-1} f(X_s) &= \lim_{t \rightarrow \infty} \frac{1}{t} \sum_{s=0}^{t-1} \sum_i \mathbb{I}(X_s = i) f(i) \\ &= \sum_i f(i) \lim_{t \rightarrow \infty} \frac{1}{t} \sum_{s=0}^{t-1} \mathbb{I}(X_s = i) \\ &= \sum_i \mu_i f(i). \end{aligned} \tag{4.1}$$

□

For countable Markov chains, unlike finite Markov chains, there are additional possibilities beyond recurrence.

Theorem 4.10 (Countable Markov chains). *Let (X_t) be an irreducible Markov chain over a countable state space X . Then: Either (i) all states are positive recurrent, or (ii) all states are null recurrent, or (iii) all states are transient.*

Proof. Let i be a positive recurrent state. We will show that all states are positive recurrent. For any state j , since the Markov chain is irreducible, we have for some $m_1, m_2 \geq 0$ that $p_{j,i}^{(m_1)}, p_{i,j}^{(m_2)} > 0$. This implies that the expected return time to state j is upper bounded by the returning to i , transitionning from j to i and from i to j . That is:

$$\mathbb{E}[T_j] \leq \frac{1}{p_{j,i}^{(m_1)}} + \mathbb{E}[T_i] + \frac{1}{p_{i,j}^{(m_2)}},$$

where the first term is the expected time to reach state i from and hence j is positive recurrent.

If there is no positive recurrent state, let i be a null recurrent state, then we will show that all states are null recurrent. For any state j , since the Markov chain is

¹If P is not diagonalizable, we can write $x_0 = \sum_i \alpha_i v_i$ where v_i is a generalized eigenvector. By Theorem 4.7 we still have that $v_1 = \mu$, and $\lambda_1 = 1 > |\lambda_i|$ for $i \geq 2$. Noting that for any Jordan block J with eigenvalue $|\lambda_i| < 1$ we have $\lim_{t \rightarrow \infty} J^t = 0$, and we may obtain a similar result: $\lim_{t \rightarrow \infty} \Pr[X_t = j] = \lambda_1 \mu_j$.

irreducible, we have for some $m_1, m_2 \geq 0$ that $p_{j,i}^{(m_1)}, p_{i,j}^{(m_2)} > 0$. This implies that $\sum_{m=0}^{\infty} p_{j,j}^{(m)} = \infty$ is at least $\sum_{m=0}^{\infty} p_{j,i}^{(m_1)} p_{i,i}^{(m)} p_{i,j}^{(m_2)} = \infty$, since we have $\sum_{m=0}^{\infty} p_{i,i}^{(m)} = \infty$, since i is a recurrent state. This implies that j is a recurrent state. Since there are no positive recurrent states, it has to be that j is a null recurrent state.

If there are no positive or null recurrent states, then all states are transient. $\square$

4.3.1 Reversible Markov Chains

Suppose there exists a probability vector $\mu = (\mu_i)$ so that

$$\mu_i p_{i,j} = \mu_j p_{j,i}, \quad i, j \in X. \quad (4.2)$$

Then it is easy to verify by direct summation that μ is an invariant distribution for the Markov chain defined by $(p_{i,j})$. This follows because $\sum_i \mu_i p_{i,j} = \sum_i \mu_j p_{j,i} = \mu_j \sum_i p_{j,i} = \mu_j$, where we used $\sum_i p_{j,i} = 1$. The equations (4.2) are called the *detailed balance equations*. A Markov chain that satisfies these equations is called *reversible*.

Example 4.4 (Discrete-time queue). Consider a discrete-time queue, with queue length $X_{\mathbf{t}} \in \mathbb{N}_0 = \{0, 1, 2, \dots\}$. At time $\mathbf{t}$, $A_{\mathbf{t}}$ new jobs arrive, and then up to $S_{\mathbf{t}}$ jobs can be served, so that

$$X_{\mathbf{t}+1} = (X_{\mathbf{t}} + A_{\mathbf{t}} - S_{\mathbf{t}})^+.$$

Suppose that $(S_{\mathbf{t}})$ is a sequence of i.i.d. RVs, and similarly $(A_{\mathbf{t}})$ is a sequence of i.i.d. RVs, with $(S_{\mathbf{t}})$, $(A_{\mathbf{t}})$ and X_0 mutually independent. It may then be seen that $(X_{\mathbf{t}}, \mathbf{t} \geq 0)$ is a Markov chain. Suppose further that each $S_{\mathbf{t}}$ is a Bernoulli RV with parameter q , namely $P(S_{\mathbf{t}} = 1) = q$, $P(S_{\mathbf{t}} = 0) = 1 - q$. Similarly, let $A_{\mathbf{t}}$ be a Bernoulli RV with parameter p . Then

$$p_{i,j} = \begin{cases} p(1-q) & : j = i + 1 \\ (1-p)(1-q) + pq & : j = i, \quad i > 0 \\ (1-p)q & : j = i - 1, \quad i > 0 \\ (1-p) + pq & : j = i = 0 \\ 0 & : \text{otherwise} \end{cases}$$

Denote $\lambda = p(1-q)$, $\eta = (1-p)q$, and $\rho = \lambda/\eta$. The detailed balance equations for this case are:

$$\mu_i p_{i,i+1} = \mu_i \lambda = \mu_{i+1} \eta = \mu_{i+1} p_{i+1,i}, \quad \forall i \geq 0$$

These equations have a solution with $\sum_i \mu_i = 1$ if and only if $\rho < 1$. The solution is $\mu_i = \mu_0 \rho^i$, with $\mu_0 = 1 - \rho$. This is therefore the stationary distribution of this queue.

4.3.2 Mixing Time

The mixing time measures how fast the Markov chain converges to the steady state distribution [96]. We first define the *Total Variation* distance between distributions D_1 and D_2 as:

$$\|D_1 - D_2\|_{TV} = \max_{B \subseteq X} \{|D_1(B) - D_2(B)|\} = \frac{1}{2} \sum_{x \in X} |D_1(x) - D_2(x)|.$$

The *mixing time* τ is defined as the time to reach a total variation of at most $1/4$ from the initial state:

$$\|s_0 P^\tau - \mu\|_{TV} = \|p^{(\tau)} - \mu\|_{TV} \leq \frac{1}{4} \|s_0 - \mu\|_{TV},$$

where μ is the steady state distribution and $p^{(\tau)}$ is the state distribution after τ steps starting with an initial state distribution s_0 .

Note that after 2τ time steps we have

$$\|s_0 P^{2\tau} - \mu\|_{TV} = \|p^{(\tau)} P^\tau - \mu\|_{TV} \leq \frac{1}{4} \|p^{(\tau)} - \mu\|_{TV} \leq \frac{1}{4^2} \|s_0 - \mu\|_{TV}.$$

In general, after $k\tau$ time steps we have

$$\|s_0 P^{k\tau} - \mu\|_{TV} = \|p^{((k-1)\tau)} P^\tau - \mu\|_{TV} \leq \frac{1}{4} \|p^{((k-1)\tau)} - \mu\|_{TV} \leq \frac{1}{4^k} \|s_0 - \mu\|_{TV}.$$

where the formal proof follows by induction on $k \geq 1$.

Chapter 5

Markov Decision Processes and Finite Horizon Dynamic Programming

In Chapter 3 we considered multi-stage decision problems for *deterministic* systems. In many problems of interest, the system dynamics also involves *randomness*, which leads us to stochastic decision problems. In this chapter we introduce the basic model of Markov Decision Processes (MDP), which will be considered throughout the rest of the book, and discuss optimal decision making in the finite horizon setting.

Chapter Orientation

Main Prerequisites:

- Deterministic decision processes (Chapter 3)
- Markov chains (Chapter 4)

Main terminology:

- $\mathbf{s}_t$ state at time t
- $\mathcal{S}_t$ state space at time t
- $\mathbf{a}_t$ action at time t

- $\mathcal{A}_t(\mathbf{s})$ action space at time t for state $\mathbf{s}$
- $p_t(\mathbf{s}'|\mathbf{s}, \mathbf{a})$ transition probability at time t
- $p(\mathbf{s}'|\mathbf{s}, \mathbf{a})$ stationary transition probability
- $\mathbf{h}_t$ history at time t
- π control policy
- π_t policy at time t
- π^* optimal control policy
- $\Pi_{HD}, \Pi_{MD}, \Pi_{SD}$ history-dependent, Markov, and stationary deterministic policy sets
- $\Pi_{HS}, \Pi_{MS}, \Pi_{SS}$ history-dependent, Markov, and stationary stochastic policy sets
- p_0 initial state distribution
- $\mathbf{r}_t(\mathbf{s}, \mathbf{a})$ reward function of state $\mathbf{s}$ and action $\mathbf{a}$ at time t
- $\tilde{\mathbf{r}}_t(\mathbf{s}_t, \mathbf{a}_t, \mathbf{s}_{t+1})$ reward function of state $\mathbf{s}_t$, action $\mathbf{a}_t$, and next state $\mathbf{s}_{t+1}$
- R_t reward obtained at time t
- $R_{\max}$ bound on reward
- T fixed time horizon
- $\mathcal{V}^\pi(\mathbf{s})$ expected cumulative reward starting from state $\mathbf{s}$ and following policy π for T steps
- $\mathcal{V}^*(\mathbf{s})$ maximal expected cumulative reward in T steps starting from state $\mathbf{s}$
- $\mathcal{V}_t^\pi(\mathbf{s})$ value function: expected reward-to-go from state $\mathbf{s}$ at time t under policy π
- $\mathcal{V}_t^*(\mathbf{s})$ optimal value function for state $\mathbf{s}$ and time t

- $V_t(\mathbf{s})$ value of state $\mathbf{s}$, time t in the dynamic programming algorithm
- $Q_t^*(\mathbf{s}, \mathbf{a})$ state-action value function: maximal expected reward to go from state $\mathbf{s}$ and action $\mathbf{a}$ at time t

5.1 Markov Decision Process

A Markov Decision Process consists of two main parts:

1. A controlled dynamic system, with stochastic evolution.
2. A performance objective to be optimized.

In this section we describe the first part, which is modeled as a controlled Markov chain.

Consider a controlled dynamic system, defined over:

- A discrete time axis $\mathbb{T} = \{0, 1, \dots, T-1\}$ (finite horizon), or $\mathbb{T} = \{0, 1, 2, \dots\}$ (infinite horizon). To simplify the discussion we refer below to the infinite horizon case, which can always be “truncated” at T if needed.
- A finite state space $\mathcal{S}$, where $\mathcal{S}_t \subset \mathcal{S}$ is the set of possible states at time $t \in \mathbb{T}$.
- A finite action set $\mathcal{A}$, where $\mathcal{A}_t(\mathbf{s}) \subset \mathcal{A}$ is the set of possible actions at time $t \in \mathbb{T}$ and state $\mathbf{s} \in \mathcal{S}_t$, and $\mathcal{A}_t = \cup_{\mathbf{s} \in \mathcal{S}_t} \mathcal{A}_t(\mathbf{s})$ is the set of possible actions in time $t \in \mathbb{T}$.

State transition probabilities: Suppose that at time t we are in state $\mathbf{s}_t = \mathbf{s}$, and choose an action $\mathbf{a}_t = \mathbf{a}$. The next state $\mathbf{s}_{t+1} = \mathbf{s}'$ is then determined randomly according to a probability distribution $\mathbf{p}_t(\cdot | \mathbf{s}, \mathbf{a})$ on $\mathcal{S}_{t+1}$. That is,

$$\Pr(\mathbf{s}_{t+1} = \mathbf{s}' | \mathbf{s}_t = \mathbf{s}, \mathbf{a}_t = \mathbf{a}) = \mathbf{p}_t(\mathbf{s}' | \mathbf{s}, \mathbf{a}), \quad \mathbf{s}' \in \mathcal{S}_{t+1}$$

The probability $\mathbf{p}_t(\mathbf{s}' | \mathbf{s}, \mathbf{a})$ is the *transition probability* from state $\mathbf{s}$ to state $\mathbf{s}'$ for a given action $\mathbf{a}$. We naturally require that $\mathbf{p}_t(\mathbf{s}' | \mathbf{s}, \mathbf{a}) \geq 0$, and $\sum_{\mathbf{s}' \in \mathcal{S}_{t+1}} \mathbf{p}_t(\mathbf{s}' | \mathbf{s}, \mathbf{a}) = 1$ for all $\mathbf{s} \in \mathcal{S}_t, \mathbf{a} \in \mathcal{A}_t(\mathbf{s})$.

Implicit in this definition is the controlled-Markov property:

$$\Pr(\mathbf{s}_{t+1} = \mathbf{s}' | \mathbf{s}_t, \mathbf{a}_t, \dots, \mathbf{s}_0, \mathbf{a}_0) = \Pr(\mathbf{s}_{t+1} = \mathbf{s}' | \mathbf{s}_t, \mathbf{a}_t).$$

The set of probability distributions

$$\mathbf{P} = \{\mathbf{p}_t(\cdot | \mathbf{s}, \mathbf{a}) : \mathbf{s} \in \mathcal{S}_t, \mathbf{a} \in \mathcal{A}_t(\mathbf{s}), t \in \mathbb{T}\},$$

is called the *transition law* or *transition kernel* of the controlled Markov process.

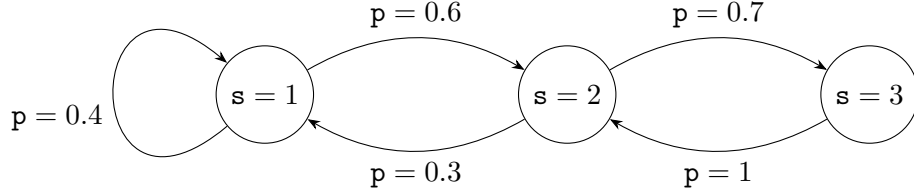


Figure 5.1: Markov chain

Stationary Models: The controlled Markov chain is called stationary or time-invariant if the transition probabilities do not depend on the time $\mathbf{t}$. That is:

$$\forall \mathbf{t}, \quad p_{\mathbf{t}}(\mathbf{s}'|\mathbf{s}, \mathbf{a}) \equiv p(\mathbf{s}'|\mathbf{s}, \mathbf{a}), \quad \mathcal{S}_{\mathbf{t}} \equiv \mathcal{S}, \quad \mathcal{A}_{\mathbf{t}}(\mathbf{s}) \equiv \mathcal{A}(\mathbf{s}).$$

Graphical Notation: The state transition probabilities of a Markov chain are often illustrated via a state transition diagram, such as in Figure 5.1 and 5.2.

In figure 5.1 we have three states $\mathcal{S} = \{1, 2, 3\}$. The transition are marked by the arrows, and labeled with their probabilities. For example: from state 1 we transition to state 2 with probability 0.6 and with probability 0.4 we stay in state 1.

A graphical description of a controlled Markov chain is a bit more complicated because of the additional action variable. We obtain the diagram (drawn for state $\mathbf{s} = 1$ only, and for a given time $\mathbf{t}$) in Figure 5.2, reflecting the following transition probabilities:

$$p(\mathbf{s}' = 2|\mathbf{s} = 1, \mathbf{a} = 1) = 1$$

$$p(\mathbf{s}'|\mathbf{s} = 1, \mathbf{a} = 2) = \begin{cases} 0.3 & : \mathbf{s}' = 1 \\ 0.2 & : \mathbf{s}' = 2 \\ 0.5 & : \mathbf{s}' = 3 \end{cases}$$

State-equation notation: The stochastic state dynamics can be equivalently defined in terms of a state equation of the form

$$\mathbf{s}_{\mathbf{t}+1} = f_{\mathbf{t}}(\mathbf{s}_{\mathbf{t}}, \mathbf{a}_{\mathbf{t}}, w_{\mathbf{t}}),$$

where $w_{\mathbf{t}}$ is a random variable (RV). If $(w_{\mathbf{t}})_{\mathbf{t} \geq 0}$ is a sequence of independent RVs, and further each $w_{\mathbf{t}}$ is independent of the “past” $(\mathbf{s}_{\mathbf{t}-1}, \mathbf{a}_{\mathbf{t}-1}, \dots, \mathbf{s}_0)$, then $(\mathbf{s}_{\mathbf{t}}, \mathbf{a}_{\mathbf{t}})_{\mathbf{t} \geq 0}$ is a controlled Markov process. For example, the state transition law of the last

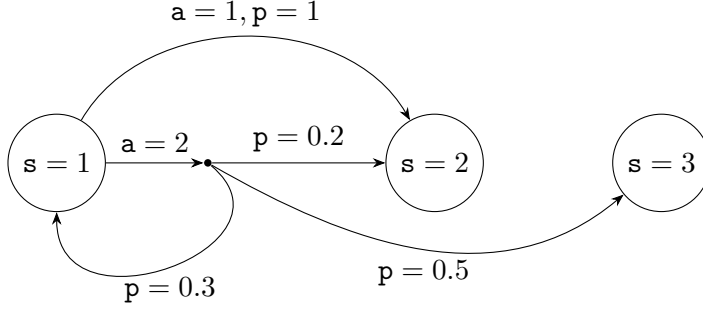


Figure 5.2: Controlled Markov chain

example can be written in this way, using $w_t \in \{4, 5, 6\}$, with $p_w(4) = 0.3$, $p_w(5) = 0.2$, $p_w(6) = 0.5$ and, for $\mathbf{s}_t = 1$:

$$\begin{aligned} f_t(1, 1, w_t) &= 2 \\ f_t(1, 2, w_t) &= w_t - 3 \end{aligned}$$

This state algebraic equation notation is especially useful for problems with continuous state space, but also for some models with discrete states. Equivalently, we can write

$$f_t(1, 2, w_t) = 1 \cdot \mathbb{I}[w_t = 4] + 2 \cdot \mathbb{I}[w_t = 5] + 3 \cdot \mathbb{I}[w_t = 6],$$

where $\mathbb{I}[\cdot]$ is the indicator function.

Next we recall the definitions of control policies from Chapter 3.

Control Policies

- A general or **history-dependent deterministic** control policy $\pi = (\pi_t)_{t \in \mathbb{T}}$ is a mapping from each possible history $\mathbf{h}_t = (\mathbf{s}_0, \mathbf{a}_0, \dots, \mathbf{s}_{t-1}, \mathbf{a}_{t-1}, \mathbf{s}_t)$, and time $t \in \mathbb{T}$, to an action $\mathbf{a}_t = \pi_t(\mathbf{h}_t) \in \mathcal{A}_t$. We denote the set of general policies by Π_{HD} .
- A **Markov deterministic** control policy π is allowed to depend on the current state and time only, i.e., $\mathbf{a}_t = \pi_t(\mathbf{s}_t)$. We denote the set of Markov deterministic policies by Π_{MD} .
- For stationary models, we may define **stationary deterministic** control policies that depend on the current state alone. A stationary policy is defined by a single mapping $\pi : \mathcal{S} \rightarrow \mathcal{A}$, so that $\mathbf{a}_t = \pi(\mathbf{s}_t)$ for all $t \in \mathbb{T}$. We denote the set of stationary policies by Π_{SD} .

- Evidently, $\Pi_{HD} \supset \Pi_{MD} \supset \Pi_{SD}$.

Randomized Control policies

- The control policies defined above specify deterministically the action to be taken at each stage. In some cases we want to allow for a random choice of action.
- A general randomized control policy assigns to each possible history $\mathbf{h}_t$ a probability distribution $\pi_t(\cdot|\mathbf{h}_t)$ over the action set $\mathcal{A}_t$. That is, $\Pr(\mathbf{a}_t = \mathbf{a}|\mathbf{h}_t) = \pi_t(\mathbf{a}|\mathbf{h}_t)$. We denote the set of history-dependent stochastic policies by Π_{HS} .
- Similarly, we can define the set Π_{MS} of Markov stochastic control policies, where $\pi_t(\cdot|\mathbf{h}_t)$ is replaced by $\pi_t(\cdot|\mathbf{s}_t)$, and the set Π_{SS} of stationary stochastic control policies, where $\pi_t(\cdot|\mathbf{s}_t)$ is replaced by $\pi(\cdot|\mathbf{s}_t)$, namely the policy is independent of the time.
- Note that the set Π_{HS} includes all other policy sets as special cases.

The Induced Stochastic Process Let $\mathbf{p}_0 = \{\mathbf{p}_0(\mathbf{s}), \mathbf{s} \in \mathcal{S}_0\}$ be a probability distribution for the initial state $\mathbf{s}_0$. (Many times we will assume that the initial state is deterministic and given by $\mathbf{s}_0$.) A control policy $\pi \in \Pi_{HS}$, together with the transition law $\mathbf{P} = \{\mathbf{p}_t(\mathbf{s}'|\mathbf{s}, \mathbf{a})\}$ and the initial state distribution $\mathbf{p}_0 = (\mathbf{p}_0(\mathbf{s}), \mathbf{s} \in \mathcal{S}_0)$, induces a probability distribution over any finite state-action sequence $\mathbf{h}_T = (\mathbf{s}_0, \mathbf{a}_0, \dots, \mathbf{s}_{T-1}, \mathbf{a}_{T-1}, \mathbf{s}_T)$, given by

$$\Pr(\mathbf{h}_T) = \mathbf{p}_0(\mathbf{s}_0) \prod_{t=0}^{T-1} \mathbf{p}_t(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t) \pi_t(\mathbf{a}_t|\mathbf{h}_t),$$

where $\mathbf{h}_t = (\mathbf{s}_0, \mathbf{a}_0, \dots, \mathbf{s}_{t-1}, \mathbf{a}_{t-1}, \mathbf{s}_t)$. To see this, observe the recursive relation:

$$\begin{aligned} \Pr(\mathbf{h}_{t+1}) &= \Pr(\mathbf{h}_t, \mathbf{a}_t, \mathbf{s}_{t+1}) = \Pr(\mathbf{s}_{t+1}|\mathbf{h}_t, \mathbf{a}_t) \Pr(\mathbf{a}_t|\mathbf{h}_t) \Pr(\mathbf{h}_t) \\ &= \mathbf{p}_t(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t) \pi_t(\mathbf{a}_t|\mathbf{h}_t) \Pr(\mathbf{h}_t). \end{aligned}$$

In the last step we used the conditional Markov property of the controlled chain: $\Pr(\mathbf{s}_{t+1}|\mathbf{h}_t, \mathbf{a}_t) = \mathbf{p}_t(\mathbf{s}_{t+1}|\mathbf{s}_t, \mathbf{a}_t)$, and the definition of the control policy π_t . The required formula follows by recursion.

Therefore, the state-action sequence $\mathbf{h}_\infty = (\mathbf{s}_k, \mathbf{a}_k)_{k \geq 0}$ can now be considered a stochastic process. We denote the probability law of this stochastic process by

$\Pr^{\pi, \mathbf{p}_0}(\cdot)$. The corresponding expectation operator is denoted by $\mathbb{E}^{\pi, \mathbf{p}_0}(\cdot)$, where the expectation is over state-action sequences generated using the stochastic process $\Pr^{\pi, \mathbf{p}_0}(\cdot)$. When the initial state $\mathbf{s}_0$ is deterministic (i.e., $\mathbf{p}_0(\mathbf{s})$ is concentrated on a single state $\mathbf{s}$), we may simply write $\Pr^{\pi, \mathbf{s}}(\cdot)$ or $\Pr^{\pi}(\cdot | \mathbf{s}_0 = \mathbf{s})$.

Under a Markov control policy, the state sequence $(\mathbf{s}_t)_{t \geq 0}$ becomes a *Markov chain*, with transition probabilities:

$$\Pr(\mathbf{s}_{t+1} = \mathbf{s}' | \mathbf{s}_t = \mathbf{s}) = \sum_{\mathbf{a} \in \mathcal{A}_t} \mathbf{p}_t(\mathbf{s}' | \mathbf{s}, \mathbf{a}) \pi_t(\mathbf{a} | \mathbf{s}).$$

This follows since:

$$\begin{aligned} \Pr(\mathbf{s}_{t+1} = \mathbf{s}' | \mathbf{s}_t = \mathbf{s}) &= \sum_{\mathbf{a} \in \mathcal{A}_t} \Pr(\mathbf{s}_{t+1} = \mathbf{s}', \mathbf{a}_t = \mathbf{a} | \mathbf{s}_t = \mathbf{s}) \\ &= \sum_{\mathbf{a} \in \mathcal{A}_t} \Pr(\mathbf{s}_{t+1} = \mathbf{s}' | \mathbf{s}_t = \mathbf{s}, \mathbf{a}_t = \mathbf{a}) \Pr(\mathbf{a}_t = \mathbf{a} | \mathbf{s}_t = \mathbf{s}) \\ &= \sum_{\mathbf{a} \in \mathcal{A}_t} \mathbf{p}_t(\mathbf{s}' | \mathbf{s}, \mathbf{a}) \pi_t(\mathbf{a} | \mathbf{s}). \end{aligned}$$

If the controlled Markov chain is stationary (time-invariant) and the control policy is stationary, then the induced Markov chain is stationary as well.

Remark 5.1. *For most non-learning optimization problems, Markov policies suffice to achieve the optimum.*

Remark 5.2. *Implicit in these definitions of control policies is the assumption that the current state $\mathbf{s}_t$ can be fully observed before the action $\mathbf{a}_t$ is chosen. If this is not the case we need to consider the problem of a Partially Observed MDP (POMDP), which is more involved and is not discussed in this book.*

5.2 Performance Criteria

5.2.1 Finite Horizon Return

Consider the finite-horizon return, with a fixed time horizon T . As in the deterministic case, we are given an immediate reward function $\mathbf{r}_t = \{\mathbf{r}_t(\mathbf{s}, \mathbf{a}) : \mathbf{s} \in \mathcal{S}_t, \mathbf{a} \in \mathcal{A}_t\}$ for $0 \leq t \leq T-1$, and a terminal reward function $\mathbf{r}_T = \{\mathbf{r}_T(\mathbf{s}) : \mathbf{s} \in \mathcal{S}_T\}$. We assume that reward functions are bounded, $|\mathbf{r}_t|, |\mathbf{r}_T| \in [0, \mathbf{R}_{\max}]$. The obtained reward is $R_t = \mathbf{r}_t(\mathbf{s}_t, \mathbf{a}_t)$ at times $t \leq T-1$, and $R_T = \mathbf{r}_T(\mathbf{s}_T)$ at the last stage. (Note

that $\mathbf{s}_t, \mathbf{a}_t$ and $\mathbf{s}_T$ are random variables that depend both on the policy π and the stochastic transitions.) Our general goal is to maximize the cumulative return:

$$\sum_{t=0}^T R_t = \sum_{t=0}^{T-1} r_t(\mathbf{s}_t, \mathbf{a}_t) + r_T(\mathbf{s}_T).$$

However, since the system is stochastic, the cumulative return will generally be a random variable, and we need to specify in which sense to maximize it. A natural first option is to consider the expected value of the return. That is, define:

$$\mathcal{V}^\pi(\mathbf{s}) = \mathbb{E}^\pi \left[\sum_{t=0}^T R_t | \mathbf{s}_0 = \mathbf{s} \right] \equiv \mathbb{E}^{\pi, \mathbf{s}} \left[\sum_{t=0}^T R_t \right].$$

Here π is the control policy as defined above, and $\mathbf{s}$ denotes the initial state. Hence, $\mathcal{V}^\pi(\mathbf{s})$ is the expected cumulative return under the control policy π . Our goal is to find an optimal control policy that maximizes $\mathcal{V}^\pi(\mathbf{s})$.

Remark 5.3. *Reward dependence on the next state: In some problems, the obtained reward may depend on the next state as well: $R_t = \tilde{r}_t(\mathbf{s}_t, \mathbf{a}_t, \mathbf{s}_{t+1})$. For control purposes, when we only consider the expected value of the reward, we can reduce this reward function to the usual one by defining*

$$r_t(\mathbf{s}, \mathbf{a}) \triangleq \mathbb{E}[R_t | \mathbf{s}_t = \mathbf{s}, \mathbf{a}_t = \mathbf{a}] \equiv \sum_{\mathbf{s}' \in S} p(\mathbf{s}' | \mathbf{s}, \mathbf{a}) \tilde{r}_t(\mathbf{s}, \mathbf{a}, \mathbf{s}').$$

Remark 5.4. *Random rewards: The reward R_t may also be random, namely a random variable whose distribution depends on $(\mathbf{s}_t, \mathbf{a}_t)$. This can also be reduced to our standard model for planning purposes by looking at the expected value of R_t , namely*

$$r_t(\mathbf{s}, \mathbf{a}) = \mathbb{E}[R_t | \mathbf{s}_t = \mathbf{s}, \mathbf{a}_t = \mathbf{a}].$$

Remark 5.5. *Risk-sensitive criteria: The expected cumulative return is by far the most common goal for planning. However, it is not the only one possible. For example, one may consider the following risk-sensitive return function:*

$$\mathcal{V}_\lambda^\pi(\mathbf{s}) = \frac{1}{\lambda} \log \mathbb{E}^{\pi, \mathbf{s}} \left(\exp \left(\lambda \sum_{t=0}^T R_t \right) \right).$$

For $\lambda > 0$, the exponent gives higher weight to high rewards, and the opposite for $\lambda < 0$.

The key challenge: stochastic transitions It is important to understand what is the ‘key’ challenge in MDPs – is it the stochastic rewards, the stochastic transitions, or both. The next several results show that an MDP with stochastic rewards may be replaced with an MDP with deterministic rewards. The opposite is not true.

In the case that the rewards are stochastic, but have a discrete support, we can construct an equivalent MDP in which all the rewards are deterministic and trajectories have the same distribution of rewards. This implies that the important challenge is the stochastic state transition function, and the rewards can be assumed to be deterministic. Formally, given a trajectory we define a *rewards trajectory* as the sub-trajectory that includes only the rewards, i.e., for a trajectory $(\mathbf{s}_0, \mathbf{a}_0, \mathbf{r}_0, \mathbf{s}_1, \dots)$ the reward trajectory is $(\mathbf{r}_0, \mathbf{r}_1, \dots)$.

Theorem 5.1. *Given an MDP $M(\mathcal{S}, \mathcal{A}, \mathbf{P}, \mathbf{r}, \mathbf{s}_0)$, where the rewards are stochastic, with support $\mathcal{K} = \{0, \dots, k-1\}$, there is an MDP $M'(\mathcal{S} \times \mathcal{K}, \mathcal{A}, \mathbf{P}', \mathbf{r}', \mathbf{s}'_0)$, where the rewards are deterministic, and a mapping of policies π of M to π' policies of M' , such that: running π in M for horizon T generates reward trajectory $R = (R_0, \dots, R_T)$ and running π' in M' for horizon $T+1$ generates reward trajectory $R' = (R_1, \dots, R_{T+1})$, and the distributions of R and R' are identical.*

Proof. For simplicity we assume that the MDP is loop-free, namely you can reach any state at most once in a trajectory. This is mainly to simplify the notation.

The basic idea is to encode the rewards in the states of M' which are $\mathcal{S} \times \mathcal{K} = \mathcal{S}'$. For each $(\mathbf{s}, i) \in \mathcal{S}'$ and action $\mathbf{a} \in \mathcal{A}$ we have $\mathbf{p}'_t((\mathbf{s}', j)|(\mathbf{s}, i), \mathbf{a}) = \mathbf{p}_t(\mathbf{s}'|\mathbf{s}, \mathbf{a}) \Pr[R_t(\mathbf{s}, \mathbf{a}) = j]$, and $\mathbf{p}'_T((\mathbf{s}', j)|(\mathbf{s}, i)) = \mathbb{I}(\mathbf{s}' = \mathbf{s}) \Pr[R_T(\mathbf{s}) = j]$. The reward is $\mathbf{r}'_t((\mathbf{s}, i), \mathbf{a}) = i$. The initial state is $\mathbf{s}'_0 = (\mathbf{s}_0, 0)$.

For any policy $\pi(\mathbf{a}|\mathbf{s})$ in M we have a policy π' in M' where $\pi'(\mathbf{a}|\mathbf{s}, i) = \pi(\mathbf{a}|\mathbf{s})$.

We map trajectories of M to trajectories of M' which have identical probabilities. A trajectory $(\mathbf{s}_0, \mathbf{a}_0, R_0, \mathbf{s}_1, \mathbf{a}_1, R_1, \mathbf{s}_2, \dots, R_T)$ is mapped to $((\mathbf{s}_0, 0), \mathbf{a}_0, 0, (\mathbf{s}_1, R_0), \mathbf{a}_1, R_0, (\mathbf{s}_2, R_1), \dots, R_{T+1})$. Let R and R' be the respective reward trajectories. Clearly, the two trajectories have identical probabilities. This implies that the rewards trajectories R and R' have identical probabilities (up to a shift of one in the index). $\square$

Theorem 5.1 requires the number of rewards to be bounded, and guarantees that the reward distribution be identical. In the case that the rewards are continuous, we can have a similar guarantee for expected return functions.

Theorem 5.2. *Given an MDP $M(\mathcal{S}, \mathcal{A}, \mathbf{P}, \mathbf{r}, \mathbf{s}_0)$, where the rewards are stochastic, with support $[0, 1]$, there is an MDP $M'(\mathcal{S}, \mathcal{A}, \mathbf{P}, \mathbf{r}', \mathbf{s}_0)$, where the rewards are stochastic, with support $\{0, 1\}$, such that for any policy $\pi \in \Pi_{MS}$ the distribution of the expected rewards trajectory is identical.*

Proof. We simply change the reward of $(\mathbf{s}, \mathbf{a})$ to be $\{0, 1\}$ by changing them to be a Bernoulli random variables with a parameter $\mathbf{r}_t(\mathbf{s}, \mathbf{a})$, i.e., $\Pr[R_t = 1 | \mathbf{s}, \mathbf{a}] = \mathbf{r}_t(\mathbf{s}, \mathbf{a})$ and $\Pr[R_t = 0 | \mathbf{s}, \mathbf{a}] = 1 - \mathbf{r}_t(\mathbf{s}, \mathbf{a})$, and we recall that $\mathbf{r}_t(\mathbf{s}, \mathbf{a})$ denotes the expected reward for $\mathbf{s}, \mathbf{a}$ in the original MDP M . Clearly, the expected value of the rewards in M and M' is identical. Further, since $\pi \in \Pi_{MS}$, it depends only on $\mathbf{s}$ and $\mathbf{t}$, which implies that the behavior (states and actions) will be identical in M and M' . $\square$

We have also established the following corollary.

Corollary 5.3. *Given an MDP $M(\mathcal{S}, \mathcal{A}, \mathbf{P}, \mathbf{r}, \mathbf{s}_0)$ and horizon $\mathbf{T}$, where the rewards are stochastic with support $[0, 1]$, there is an MDP $M'(\mathcal{S} \times \{0, 1\}, \mathcal{A}, \mathbf{P}', \mathbf{r}', \mathbf{s}'_0)$, and horizon $\mathbf{T}+1$, where the rewards are deterministic, and a mapping of policies $\pi \in \Pi_{MS}$ of M to $\pi' \in \Pi_{MD}$ policies of M' , such that $\mathcal{V}^{\pi, M}(\mathbf{s}_0) = \mathcal{V}^{\pi', M'}(\mathbf{s}'_0)$.*

5.2.2 Infinite Horizon Problems

We next consider planning problems that extend to an infinite time horizon, $\mathbf{t} = 0, 1, 2, \dots$. Such planning problems arise when the system in question is expected to operate for a long time, or a large number of steps, possibly with no specific “closing” time. Infinite horizon problems are most often defined for stationary problems. In that case, they enjoy the important advantage that optimal policies can be found among the class of stationary policies. We will restrict attention here to stationary models. As before, we have the immediate reward function $\mathbf{r}(\mathbf{s}, \mathbf{a})$, which extends to all $\mathbf{t} \geq 0$. The expected reward obtained at stage $\mathbf{t}$ is $\mathbb{E}[R_t] = \mathbf{r}(\mathbf{s}_t, \mathbf{a}_t)$.

Discounted return: The most common performance criterion for infinite horizon problems is the expected discounted return:

$$\mathcal{V}_\gamma^\pi(\mathbf{s}) = \mathbb{E}^\pi \left[\sum_{t=0}^{\infty} \gamma^t \mathbf{r}(\mathbf{s}_t, \mathbf{a}_t) | \mathbf{s}_0 = \mathbf{s} \right] \equiv \mathbb{E}^{\pi, \mathbf{s}} \left[\sum_{t=0}^{\infty} \gamma^t \mathbf{r}(\mathbf{s}_t, \mathbf{a}_t) \right],$$

where $0 < \gamma < 1$ is the discount factor. Mathematically, the discount factor ensures convergence of the sum (whenever the reward sequence is unbounded). This makes the problem “well behaved”, and relatively easy to analyze. The discounted return is discussed in Chapter 6.

Average return: Here we are interested to maximize the long-term average return. The most common definition of the long-term average return is,

$$\mathcal{V}_{avg}^\pi(\mathbf{s}) = \liminf_{T \rightarrow \infty} \mathbb{E}^{\pi, \mathbf{s}} \left[\frac{1}{T} \sum_{t=0}^{T-1} \mathbf{r}(\mathbf{s}_t, \mathbf{a}_t) \right].$$

The theory of average-return planning problems is more involved, and relies to a larger extent on the theory of Markov chains. (See Chapter 4 for planning in Deterministic Decision Processes with the average return criteria.)

5.2.3 Stochastic Shortest-Path Problems

In an important class of planning problems, the time horizon is not set beforehand, but rather the problem continues until a certain event occurs. This event can be defined as reaching some goal state. Let $\mathcal{S}_G \subset \mathcal{S}$ define the set of *goal states*. Define

$$\tau = \inf\{\mathbf{t} \geq 0 : \mathbf{s}_{\mathbf{t}} \in \mathcal{S}_G\}$$

as the first time in which a goal state is reached. The total expected return for this problem is defined as:

$$\mathcal{V}_{sp}^{\pi}(\mathbf{s}) = \mathbb{E}^{\pi, \mathbf{s}} \left[\sum_{\mathbf{t}=0}^{\tau-1} r(\mathbf{s}_{\mathbf{t}}, \mathbf{a}_{\mathbf{t}}) + r_G(\mathbf{s}_{\tau}) \right].$$

Here $r_G(\mathbf{s})$, $\mathbf{s} \in \mathcal{S}_G$ specified the reward at goal states. Note that the length of the run, τ , is a random variable.

Stochastic shortest path includes, naturally, the finite horizon case. This can be shown by creating a leveled MDP where at each time step we move to the next level and terminate at level T . Specifically, we define a new state space $\mathcal{S}' = \mathcal{S} \times \mathbb{T}$, transition function $\mathbf{p}((\mathbf{s}', \mathbf{t} + 1) | (\mathbf{s}, \mathbf{t}), \mathbf{a}) = \mathbf{p}(\mathbf{s}' | \mathbf{s}, \mathbf{a})$ and goal states $\mathcal{S}_G = \{(\mathbf{s}, T) : \mathbf{s} \in \mathcal{S}\}$.

Stochastic shortest path includes also the discounted infinite horizon. To see that, add a new goal state, and from each state with probability $1 - \gamma$ jump to the goal state and terminate. The expected return of a policy would be the same in both models. Specifically, we add a state $\mathbf{s}_G$ and modify the transition probability to $\mathbf{p}'$, such that $\mathbf{p}'(\mathbf{s}_G | \mathbf{s}, \mathbf{a}) = 1 - \gamma$, for any state $\mathbf{s} \in \mathcal{S}$ and action $\mathbf{a} \in \mathcal{A}$ and $\mathbf{p}'(\mathbf{s}' | \mathbf{s}, \mathbf{a}) = \gamma \mathbf{p}(\mathbf{s}' | \mathbf{s}, \mathbf{a})$. The probability that we do not terminate by time $\mathbf{t}$ is exactly $\gamma^{\mathbf{t}}$. Therefore, the expected return is $\mathbb{E}^{\pi, \mathbf{s}}[\sum_{\mathbf{t}=0}^{\infty} \gamma^{\mathbf{t}} r(\mathbf{s}_{\mathbf{t}}, \mathbf{a}_{\mathbf{t}})]$ which is identical to the discounted return.

This class of problems provides a natural extension of the standard shortest-path problem to stochastic settings. Some conditions on the system dynamics and reward function must be imposed for the problem to be well posed (e.g., that a goal state may be reached with probability one). Stochastic shortest path problems are also known as *episodic* MDP problems. (See more in Chapter 7.)

5.3 Sufficiency of Markov Policies

In all the performance criteria defined above, the criterion is composed of sums of terms of the form $\mathbb{E}[r_t(\mathbf{s}_t, \mathbf{a}_t)]$. It follows that if two control policies induce the same marginal probability distributions $q_t(\mathbf{s}_t, \mathbf{a}_t)$ over the state-action pairs $(\mathbf{s}_t, \mathbf{a}_t)$ for all $t \geq 0$, they will have the same performance for any linear return function.

Using this observation, the next claim implies that it is enough to consider the set of (stochastic) Markov policies in the above planning problems.

Proposition 5.4. *Let $\pi \in \Pi_{HS}$ be a general (history-dependent, stochastic) control policy. Let*

$$p_t^{\pi, \mathbf{s}_0}(\mathbf{s}, \mathbf{a}) = P^{\pi, \mathbf{s}_0}(\mathbf{s}_t = \mathbf{s}, \mathbf{a}_t = \mathbf{a}), \quad (\mathbf{s}, \mathbf{a}) \in \mathcal{S}_t \times \mathcal{A}_t$$

Denote the marginal distributions induced by $q_t(\mathbf{s}_t, \mathbf{a}_t)$ on the state-action pairs $(\mathbf{s}_t, \mathbf{a}_t)$, for all $t \geq 0$. Then there exists a stochastic Markov policy $\tilde{\pi} \in \Pi_{MS}$ that induces the same marginal probabilities (for all initial states $\mathbf{s}_0$).

In Chapter 3 we showed for Deterministic Decision Process for the finite horizon that there is an optimal deterministic policy. The proof that every stochastic history dependent strategy has an equivalent stochastic Markovian policy (Theorem 3.1) showed how to generate the same state-action distribution, and applies to other setting as well. The proof that every stochastic Markovian policy has an equivalent (or better) deterministic Markovian policy (Theorem 3.2) depended on the finite horizon, but it is easy to extend it to any linear return function as well. (We leave the formal proof as an exercise to the reader.)

5.4 Finite-Horizon Dynamic Programming

Recall that we consider the expected total reward criterion, which we denote as

$$\mathcal{V}^\pi(\mathbf{s}_0) = \mathbb{E}^{\pi, \mathbf{s}_0} \left[\sum_{t=0}^{T-1} r_t(\mathbf{s}_t, \mathbf{a}_t) + r_T(\mathbf{s}_T) \right],$$

where π is the control policy used, and $\mathbf{s}_0$ is a given initial state. We wish to maximize the expected return $\mathcal{V}^\pi(\mathbf{s}_0)$ over all control policies, and find an optimal policy π^* that achieves the maximal expected return $\mathcal{V}^*(\mathbf{s}_0)$ for all initial states $\mathbf{s}_0$. Thus,

$$\mathcal{V}^*(\mathbf{s}_0) \triangleq \mathcal{V}^{\pi^*}(\mathbf{s}_0) = \max_{\pi \in \Pi_{HS}} \mathcal{V}^\pi(\mathbf{s}_0).$$

5.4.1 The Principle of Optimality

The celebrated principle of optimality (stated by Bellman) applies to a large class of multi-stage optimization problems, and is at the heart of Dynamic Programming. As a general principle, it states that:

The tail of an optimal policy is optimal for the “tail problem”.

This principle is not an actual claim, but rather a guiding principle that can be applied in different ways to each problem. For example, considering our finite-horizon problem, let $\pi^* = (\pi_0, \dots, \pi_{T-1})$ denote an optimal Markov policy, where π_t is the policy used at time t in the horizon. Take any state $\mathbf{s}_t = \mathbf{s}'$ which has a positive probability to be reached under π^* , namely $P^{\pi^*, \mathbf{s}_0}(\mathbf{s}_t = \mathbf{s}') > 0$. Then the tail policy $\pi_{t:T}^* = (\pi_t, \dots, \pi_{T-1})$ is optimal for the “tail” criterion $\mathcal{V}_{t:T}^{\pi^*}(\mathbf{s}') = \mathbb{E}^{\pi^*} [\sum_{k=t}^T R_k | \mathbf{s}_t = \mathbf{s}']$.

Note that the reverse is not true. The prefix of the optimal policy is not optimal for the “prefix” problem. When we plan for a long horizon, we might start with non-greedy actions, so we can improve our return in later time steps. Specifically, the first action taken does not have to be the optimal action for horizon $T = 1$, for which the greedy action is optimal.

5.4.2 Dynamic Programming for Policy Evaluation

As a “warmup”, let us evaluate the reward of a given policy. Let $\pi = (\pi_0, \dots, \pi_{T-1})$ be a given deterministic Markov policy. Define the following reward-to-go function, or *value* function:

$$\mathcal{V}_t^\pi(\mathbf{s}) = \mathbb{E}^\pi \left[\sum_{k=t}^T R_k \middle| \mathbf{s}_t = \mathbf{s} \right]$$

Observe that $\mathcal{V}_0^\pi(\mathbf{s}_0) = \mathcal{V}^\pi(\mathbf{s}_0)$.

Lemma 5.5 (Fixed Policy Value Iteration). *The values $\mathcal{V}_t^\pi(\mathbf{s})$ may be computed by the backward recursion:*

$$\mathcal{V}_t^\pi(\mathbf{s}) = \left\{ r_t(\mathbf{s}, a) + \sum_{\mathbf{s}' \in \mathcal{S}_{t+1}} p_t(\mathbf{s}' | \mathbf{s}, a) \mathcal{V}_{t+1}^\pi(\mathbf{s}') \right\}_{a=\pi_t(\mathbf{s})}, \quad \forall \mathbf{s} \in \mathcal{S}_t$$

for $t = T - 1, \dots, 0$, starting with $\mathcal{V}_T^\pi(\mathbf{s}) = r_T(\mathbf{s})$.

Proof. Observe that:

$$\begin{aligned}
\mathcal{V}_t^\pi(\mathbf{s}) &= \mathbb{E}^\pi \left[R_t + \sum_{k=t+1}^T R_k \middle| \mathbf{s}_t = \mathbf{s}, \mathbf{a}_t = \pi_t(\mathbf{s}) \right] \\
&= \mathbb{E}^\pi \left[R_t + \mathbb{E}^\pi \left[\sum_{k=t+1}^T R_k \middle| \mathbf{s}_{t+1} \right] \middle| \mathbf{s}_t = \mathbf{s}, \mathbf{a}_t = \pi_t(\mathbf{s}) \right] \\
&= \mathbb{E}^\pi \left[r_t(\mathbf{s}_t, \mathbf{a}_t) + \mathcal{V}_{t+1}^\pi(\mathbf{s}_{t+1}) \middle| \mathbf{s}_t = \mathbf{s}, \mathbf{a}_t = \pi_t(\mathbf{s}) \right] \\
&= r_t(\mathbf{s}, \pi_t(\mathbf{s})) + \sum_{\mathbf{s}' \in \mathcal{S}_{t+1}} p_t(\mathbf{s}' | \mathbf{s}, \pi_t(\mathbf{s})) \mathcal{V}_{t+1}^\pi(\mathbf{s}').
\end{aligned}$$

The first identity is simply writing the value function explicitly, starting at state $\mathbf{s}$ at time t and using action $\mathbf{a} = \pi_t(\mathbf{s})$. We split the sum to R_t , the immediate reward, and the sum of other latter rewards. The second identity uses the law of total probability, we are conditioning on state $\mathbf{s}_{t+1}$, and taking the expectation over it. The third identity observes that the expected value of the sum is actually the value function at $\mathbf{s}_{t+1}$. The last identity writes the expectation over $\mathbf{s}_{t+1}$ explicitly. This completes the proof of the lemma. $\square$

Remark 5.6. Note that $\sum_{\mathbf{s}' \in \mathcal{S}_{t+1}} p_t(\mathbf{s}' | \mathbf{s}, \mathbf{a}) \mathcal{V}_{t+1}^\pi(\mathbf{s}') = \mathbb{E}^\pi[\mathcal{V}_{t+1}^\pi(\mathbf{s}_{t+1}) | \mathbf{s}_t = \mathbf{s}, \mathbf{a}_t = \mathbf{a}]$.

Remark 5.7. For the more general reward function $\tilde{r}_t(\mathbf{s}, \mathbf{a}, \mathbf{s}')$, the recursion takes the form

$$\mathcal{V}_t^\pi(\mathbf{s}) = \left\{ \sum_{\mathbf{s}' \in \mathcal{S}_{t+1}} p_t(\mathbf{s}' | \mathbf{s}, \mathbf{a}) (\tilde{r}_t(\mathbf{s}, \mathbf{a}, \mathbf{s}') + \mathcal{V}_{t+1}^\pi(\mathbf{s}')) \right\}_{\mathbf{a}=\pi_t(\mathbf{s})}.$$

A similar observation applies to the Dynamic Programming equations in the next section.

5.4.3 Dynamic Programming for Policy Optimization

We next define the optimal value function at each time $0 \leq t \leq T$:

$$\mathcal{V}_t^*(\mathbf{s}) = \max_{\pi^t} \mathbb{E}^{\pi^t} \left[\sum_{k=t}^T R_k \middle| \mathbf{s}_t = \mathbf{s} \right], \quad \mathbf{s} \in \mathcal{S}_t,$$

where the maximum is taken over “tail” policies $\pi^{\mathbf{t}} = (\pi_{\mathbf{t}}, \dots, \pi_{T-1})$ that start from time $\mathbf{t}$ and until time $T - 1$. Note that $\pi^{\mathbf{t}}$ is allowed to be a general policy, i.e., history-dependent and stochastic, i.e., each $\pi_k \in \pi^{\mathbf{t}}$ can depend on the history from time $\mathbf{t}$ to time k . Obviously, $\mathcal{V}_0^*(\mathbf{s}_0) = \mathcal{V}^*(\mathbf{s}_0)$.

Theorem 5.6 (Finite-horizon Dynamic Programming). *The following holds:*

1. *Backward recursion:* We have that $\mathcal{V}_T^*(\mathbf{s}) = r_T(\mathbf{s})$ for all $\mathbf{s} \in \mathcal{S}_T$.
For $\mathbf{t} = T - 1, \dots, 0$, we have:

$$\mathcal{V}_{\mathbf{t}}^*(\mathbf{s}) = \max_{\mathbf{a} \in \mathcal{A}_{\mathbf{t}}} \left\{ r_{\mathbf{t}}(\mathbf{s}, \mathbf{a}) + \sum_{\mathbf{s}' \in \mathcal{S}_{\mathbf{t}+1}} p_{\mathbf{t}}(\mathbf{s}' | \mathbf{s}, \mathbf{a}) \mathcal{V}_{\mathbf{t}+1}^*(\mathbf{s}') \right\}, \quad \mathbf{s} \in \mathcal{S}_{\mathbf{t}}.$$

2. *Optimal policy:* Any Markov policy π^* that satisfies, for $\mathbf{t} = 0, \dots, T - 1$,

$$\pi_{\mathbf{t}}^*(\mathbf{s}) \in \arg \max_{\mathbf{a} \in \mathcal{A}_{\mathbf{t}}} \left\{ r_{\mathbf{t}}(\mathbf{s}, \mathbf{a}) + \sum_{\mathbf{s}' \in \mathcal{S}_{\mathbf{t}+1}} p_{\mathbf{t}}(\mathbf{s}' | \mathbf{s}, \mathbf{a}) \mathcal{V}_{\mathbf{t}+1}^*(\mathbf{s}') \right\}, \quad \forall \mathbf{s} \in \mathcal{S}_{\mathbf{t}},$$

is an optimal control policy. Furthermore, π^ maximizes $\mathcal{V}^{\pi}(\mathbf{s}_0)$ simultaneously for every initial state $\mathbf{s}_0 \in \mathcal{S}_0$.*

Note that Theorem 5.6 specifies an optimal control policy which is a deterministic Markov policy. Further, the theorem prescribes an algorithm for computing the optimal value function and policy.

Algorithm 7 Finite-Horizon Dynamic Programming

1: **Initialize:**

2: For all $\mathbf{s} \in \mathcal{S}_T$, set:

$$V_T(\mathbf{s}) = r_T(\mathbf{s})$$

3: **For** $\mathbf{t} = T - 1, T - 2, \dots, 0$

4: **For all** $\mathbf{s} \in \mathcal{S}_{\mathbf{t}}$

5: Set:

$$V_{\mathbf{t}}(\mathbf{s}) = \max_{\mathbf{a} \in \mathcal{A}_{\mathbf{t}}} \left\{ r_{\mathbf{t}}(\mathbf{s}, \mathbf{a}) + \sum_{\mathbf{s}' \in \mathcal{S}_{\mathbf{t}+1}} p_{\mathbf{t}}(\mathbf{s}' | \mathbf{s}, \mathbf{a}) V_{\mathbf{t}+1}(\mathbf{s}') \right\}$$

6: Store the action $\pi_{\mathbf{t}}^*(\mathbf{s})$ that achieves the maximum.

Theorem 5.6 claims that the values computed in Algorithm 7 satisfy $V_{\mathbf{t}}(\mathbf{s}) = \mathcal{V}_{\mathbf{t}}^*(\mathbf{s})$ for all $\mathbf{t} \in 0, \dots, T$, and that the deterministic policy stored by the algorithm is optimal. We now prove the theorem.

Proof. Part (i):

We use induction to show that the stated backward recursion in Algorithm 7 indeed yields the optimal value function $\mathcal{V}_t^*$. The idea is simple, but some care is needed with the notation since we consider general policies, and not just Markov policies.

For the base of the induction we start with $t = T$. The equality $\mathcal{V}_T^*(s) = r_T(s)$ follows directly from the definition of $\mathcal{V}_T^*$. Therefore, we have $V_T = \mathcal{V}_T^*$.

We proceed by backward induction. Suppose that $V_{t+1}(s)$ is the optimal value function for time $t+1$, i.e., $V_{t+1}(s) = \mathcal{V}_{t+1}^*(s)$. We need to show that $V_t(s) = \mathcal{V}_t^*(s)$, where

$$V_t(s) \triangleq \max_{a \in \mathcal{A}_t} \left\{ r_t(s, a) + \sum_{s' \in \mathcal{S}_{t+1}} p_t(s'|s, a) V_{t+1}(s') \right\}.$$

We will first establish that $\mathcal{V}_t^*(s) \geq V_t(s)$, and then that $\mathcal{V}_t^*(s) \leq V_t(s)$.

(a) We first show that $\mathcal{V}_t^*(s) \geq V_t(s)$. For that purpose, it is enough to find a policy π^t so that $\mathcal{V}_t^{\pi^t}(s) = V_t(s)$, since $\mathcal{V}_t^*(s) \geq \mathcal{V}_t^{\pi}(s)$ for any policy π .

Fix $s \in \mathcal{S}_t$, and define π^t as follows: Choose $a_t = \bar{a}$, where

$$\bar{a} \in \arg \max_{a \in \mathcal{A}_t} \left\{ r_t(s, a) + \sum_{s' \in \mathcal{S}_{t+1}} p_t(s'|s, a) V_{t+1}(s') \right\},$$

and then, after observing $s_{t+1} = s'$, proceed with the optimal tail policy $\pi^{t+1}(s')$ that obtains $\mathcal{V}_{t+1}^{\pi^{t+1}(s')}(s') = V_{t+1}(s')$. Proceeding similarly to the proof of Lemma 5.5 (value iteration for a fixed policy), we obtain:

$$\mathcal{V}_t^{\pi^t}(s) = r_t(s, \bar{a}) + \sum_{s' \in \mathcal{S}_{t+1}} p_t(s'|s, \bar{a}) \mathcal{V}_{t+1}^{\pi^{t+1}(s')}(s') \quad (5.1)$$

$$= r_t(s, \bar{a}) + \sum_{s' \in \mathcal{S}_{t+1}} p_t(s'|s, \bar{a}) V_{t+1}(s') = V_t(s), \quad (5.2)$$

as was required.

(b) To establish $\mathcal{V}_t^*(s) \leq V_t(s)$, it is enough to show that $\mathcal{V}_t^{\pi^t}(s) \leq V_t(s)$ for any (general, randomized) "tail" policy π^t .

Fix $s \in \mathcal{S}_t$. Consider then some tail policy $\pi^t = (\pi_t, \dots, \pi_{T-1})$. Note that this means that $a_k \sim \pi_k(a|h_{t:k})$, where $h_{t:k} = (s_t, a_t, s_{t+1}, a_{t+1}, \dots, s_k)$. For each state-action pair $s \in \mathcal{S}_t$ and $a \in \mathcal{A}_t$, let $(\pi^t|s, a)$ denote the tail policy π^{t+1} from time $t+1$ onward which is obtained from π^t given that $s_t = s$, $a_t = a$. As before, by

value iteration for a fixed policy,

$$\mathcal{V}_t^{\pi_t}(\mathbf{s}) = \sum_{\mathbf{a} \in \mathcal{A}_t} \pi_t(\mathbf{a}|\mathbf{s}) \left\{ r_t(\mathbf{s}, \mathbf{a}) + \sum_{\mathbf{s}' \in \mathcal{S}_{t+1}} p_t(\mathbf{s}'|\mathbf{s}, \mathbf{a}) \mathcal{V}_{t+1}^{(\pi_t|\mathbf{s}, \mathbf{a})}(\mathbf{s}') \right\}.$$

But since V_{t+1} is optimal,

$$\begin{aligned} \mathcal{V}_t^{\pi_t}(\mathbf{s}) &\leq \sum_{\mathbf{a} \in \mathcal{A}_t} \pi_t(\mathbf{a}|\mathbf{s}) \left\{ r_t(\mathbf{s}, \mathbf{a}) + \sum_{\mathbf{s}' \in \mathcal{S}_{t+1}} p_t(\mathbf{s}'|\mathbf{s}, \mathbf{a}) V_{t+1}(\mathbf{s}') \right\} \\ &\leq \max_{\mathbf{a} \in \mathcal{A}_t} \left\{ r_t(\mathbf{s}, \mathbf{a}) + \sum_{\mathbf{s}' \in \mathcal{S}_{t+1}} p_t(\mathbf{s}'|\mathbf{s}, \mathbf{a}) V_{t+1}(\mathbf{s}') \right\} = V_t(\mathbf{s}), \end{aligned}$$

which is the required inequality in (b).

Part (ii) The main point is to show that it is sufficient that the optimal policy would be Markov (rather than history dependent) and deterministic (rather than stochastic).

We will only sketch the proof. Let π^* be the (Markov) policy defined in part 2 of Theorem 5.6. Our goal is to show that the value function of π^* coincides with that of the optimal policy, which we showed is equal to V_t that we computed. Once we show that, we prove that π^* is optimal.

Consider the value iteration (Lemma 5.5). The updates for $\mathcal{V}_t^{\pi^*}$ in the value iteration, given the action selection of π^* , are identical to those of V_t . This implies that $\mathcal{V}_t^{\pi^*} = V_t$ (formally, by induction on t). Since V_t is the optimal value function, it implies that π^* is an optimal policy. $\square$

5.4.4 The Q Function

Let

$$Q_t^*(\mathbf{s}, \mathbf{a}) \triangleq r_t(\mathbf{s}, \mathbf{a}) + \sum_{\mathbf{s}' \in \mathcal{S}_{t+1}} p_t(\mathbf{s}'|\mathbf{s}, \mathbf{a}) \mathcal{V}_{t+1}^*(\mathbf{s}').$$

This is known as the optimal state-action value function, or simply as the *Q-function*. $Q_t^*(\mathbf{s}, \mathbf{a})$ is the expected return from stage t onward, if we choose $\mathbf{a}_t = \mathbf{a}$ and then proceed optimally.

Theorem 5.6 can now be succinctly expressed as

$$\mathcal{V}_t^*(\mathbf{s}) = \max_{\mathbf{a} \in \mathcal{A}_t} Q_t^*(\mathbf{s}, \mathbf{a}),$$

and

$$\pi_t^*(\mathbf{s}) \in \arg \max_{\mathbf{a} \in \mathcal{A}_t} Q_t^*(\mathbf{s}, \mathbf{a}).$$

The Q function provides the basis for the Q -learning algorithm, which is one of the basic Reinforcement Learning algorithms, and would be discussed in Chapter 11.

Chapter 6

Discounted Markov Decision Processes

This chapter covers the basic theory and main solution methods for stationary MDPs over an infinite horizon, with the discounted return criterion, which we will refer to as *discounted MDPs*.

The discounted return problem is the most “mathematically well behaved” among all infinite horizon problems (such as average return and stochastic shortest path), and its theory is relatively simple, both in the planning and the learning contexts. For that reason, as well as its usefulness, we will consider here the discounted problem and its solution in some detail.

Chapter Orientation

Main Prerequisites:

- Finite horizon MDPs (Chapter 5)

Main terminology:

- $\mathcal{S}$ state space
- $\mathcal{A}$ action space
- $p(s'|s, a)$ transition probability
- P^π state transition matrix under policy π

- $\mathbf{r}(\mathbf{s}, \mathbf{a})$ reward function of state $\mathbf{s}$ and action $\mathbf{a}$
- γ discount factor
- $\mathcal{V}_\gamma^\pi(\mathbf{s})$ (abbreviated as $\mathcal{V}^\pi(\mathbf{s})$) discounted value of state $\mathbf{s}$ under policy π
- $\mathcal{V}_\gamma^*(\mathbf{s})$ (abbreviated as $\mathcal{V}^*(\mathbf{s})$) optimal discounted value of state $\mathbf{s}$
- $\mathcal{V}_k$ value at iteration k of the VI algorithm
- π_k policy at iteration k of the PI algorithm
- $\mathcal{T}^\pi$ dynamic programming operator for fixed policy
- $\mathcal{T}^*$ optimal Bellman operator

6.1 Problem Statement

We consider a stationary (time-invariant) MDP, with a finite state space $\mathcal{S}$, finite action set $\mathcal{A}$, a fixed reward function $\mathbf{r}(\mathbf{s}, \mathbf{a})$, and a fixed transition kernel $\mathbf{P} = \{\mathbf{p}(\mathbf{s}'|\mathbf{s}, \mathbf{a})\}$ over the infinite time horizon $\mathbb{T} = \{0, 1, 2, \dots\}$.

Our goal is to maximize the expected discounted return, which is defined for each control policy π and initial state $\mathbf{s}_0 = \mathbf{s}$ as follows:

$$\begin{aligned}\mathcal{V}_\gamma^\pi(\mathbf{s}) &= \mathbb{E}^\pi \left[\sum_{t=0}^{\infty} \gamma^t \mathbf{r}(\mathbf{s}_t, \mathbf{a}_t) \middle| \mathbf{s}_0 = \mathbf{s} \right] \\ &\equiv \mathbb{E}^{\pi, \mathbf{s}} \left[\sum_{t=0}^{\infty} \gamma^t \mathbf{r}(\mathbf{s}_t, \mathbf{a}_t) \right],\end{aligned}$$

where $\mathbb{E}^{\pi, \mathbf{s}}$ uses the distribution induced by policy π starting at state $\mathbf{s}$. Here,

- $\mathbf{r} : \mathcal{S} \times \mathcal{A} \rightarrow \mathbb{R}$ is the (running, immediate, or instantaneous) expected reward function, i.e., $\mathbf{r}(\mathbf{s}, \mathbf{a}) = \mathbb{E}[\mathbf{r}|\mathbf{s}, \mathbf{a}]$.
- $\gamma \in (0, 1)$ is the discount factor.

We observe that $\gamma < 1$ ensures convergence of the infinite sum (since the rewards $\mathbf{r}(\mathbf{s}_t, \mathbf{a}_t)$ are uniformly bounded because $\mathcal{S}$ and $\mathcal{A}$ are finite). With $\gamma = 1$ we obtain the total return criterion, which is harder to handle due to possible divergence of the sum.

Let $\mathcal{V}_\gamma^*(\mathbf{s})$ denote the maximal expected value of the discounted return, over all (possibly history dependent and randomized) control policies, i.e.,

$$\mathcal{V}_\gamma^*(\mathbf{s}) = \sup_{\pi \in \Pi_{HS}} \mathcal{V}_\gamma^\pi(\mathbf{s}).$$

Our goal is to find an optimal control policy π^* that attains that maximum (for all initial states), and compute the numeric value of the optimal return $\mathcal{V}_\gamma^*(\mathbf{s})$. As we shall see, for this problem there always exists an optimal deterministic stationary policy.

Remark 6.1. *As usual, the discounted performance criterion can be defined in terms of cost:*

$$\mathcal{C}_\gamma^\pi(\mathbf{s}) = \mathbb{E}^{\pi, \mathbf{s}} \left[\sum_{t=0}^{\infty} \gamma^t c(\mathbf{s}_t, \mathbf{a}_t) \right],$$

where $c(\mathbf{s}, \mathbf{a})$ is the running cost function. Our goal is then to minimize the discounted cost $\mathcal{C}_\gamma^\pi(\mathbf{s})$. Since minimizing cost and maximizing reward are equivalent, we will focus on maximizing rewards throughout the book.

6.2 The Fixed-Policy Value Function

We start the analysis by defining and computing the value function for a fixed stationary policy. This intermediate step is required for later analysis of our optimization problem, and also serves as a gentle introduction to the value iteration approach.

For a stationary policy $\pi : \mathcal{S} \rightarrow \mathcal{A}$, we define the value function $\mathcal{V}_\gamma^\pi(\mathbf{s})$, $\mathbf{s} \in \mathcal{S}$ simply as the corresponding discounted return:

$$\mathcal{V}_\gamma^\pi(\mathbf{s}) \triangleq \mathbb{E}^{\pi, \mathbf{s}} \left[\sum_{t=0}^{\infty} \gamma^t r(\mathbf{s}_t, \mathbf{a}_t) \right], \quad \forall \mathbf{s} \in \mathcal{S}.$$

To reduce clutter we will omit the γ subscript in the following and abbreviate $\mathcal{V}_\gamma^\pi(\mathbf{s})$ as $\mathcal{V}^\pi(\mathbf{s})$.

Lemma 6.1. *For $\pi \in \Pi_{SD}$, the value function $\mathcal{V}^\pi$ satisfies the following set of $|\mathcal{S}|$ linear equations:*

$$\mathcal{V}^\pi(\mathbf{s}) = r(\mathbf{s}, \pi(\mathbf{s})) + \gamma \sum_{\mathbf{s}' \in \mathcal{S}} p(\mathbf{s}' | \mathbf{s}, \pi(\mathbf{s})) \mathcal{V}^\pi(\mathbf{s}'), \quad \forall \mathbf{s} \in \mathcal{S}. \quad (6.1)$$

Proof. We first note that

$$\begin{aligned}\mathcal{V}^\pi(\mathbf{s}) &\triangleq \mathbb{E}^\pi \left[\sum_{t=0}^{\infty} \gamma^t \mathbf{r}(\mathbf{s}_t, \mathbf{a}_t) \middle| \mathbf{s}_0 = \mathbf{s} \right] \\ &= \mathbb{E}^\pi \left[\sum_{t=1}^{\infty} \gamma^{t-1} \mathbf{r}(\mathbf{s}_t, \mathbf{a}_t) \middle| \mathbf{s}_1 = \mathbf{s} \right],\end{aligned}$$

since both the model and the policy are stationary and in particular time invariant. Now,

$$\begin{aligned}\mathcal{V}^\pi(\mathbf{s}) &= \mathbf{r}(\mathbf{s}, \pi(\mathbf{s})) + \mathbb{E}^\pi \left[\sum_{t=1}^{\infty} \gamma^t \mathbf{r}(\mathbf{s}_t, \pi(\mathbf{s}_t)) \middle| \mathbf{s}_0 = \mathbf{s} \right] \\ &= \mathbf{r}(\mathbf{s}, \pi(\mathbf{s})) + \mathbb{E}^\pi \left[\mathbb{E}^\pi \left[\sum_{t=1}^{\infty} \gamma^t \mathbf{r}(\mathbf{s}_t, \pi(\mathbf{s}_t)) \middle| \mathbf{s}_0 = \mathbf{s}, \mathbf{s}_1 = \mathbf{s}' \right] \middle| \mathbf{s}_0 = \mathbf{s} \right] \\ &= \mathbf{r}(\mathbf{s}, \pi(\mathbf{s})) + \sum_{\mathbf{s}' \in \mathcal{S}} \mathbf{p}(\mathbf{s}' | \mathbf{s}, \pi(\mathbf{s})) \mathbb{E}^\pi \left[\sum_{t=1}^{\infty} \gamma^t \mathbf{r}(\mathbf{s}_t, \pi(\mathbf{s}_t)) \middle| \mathbf{s}_1 = \mathbf{s}' \right] \\ &= \mathbf{r}(\mathbf{s}, \pi(\mathbf{s})) + \gamma \sum_{\mathbf{s}' \in \mathcal{S}} \mathbf{p}(\mathbf{s}' | \mathbf{s}, \pi(\mathbf{s})) \mathbb{E}^\pi \left[\sum_{t=1}^{\infty} \gamma^{t-1} \mathbf{r}(\mathbf{s}_t, \mathbf{a}_t) \middle| \mathbf{s}_1 = \mathbf{s}' \right] \\ &= \mathbf{r}(\mathbf{s}, \pi(\mathbf{s})) + \gamma \sum_{\mathbf{s}' \in \mathcal{S}} \mathbf{p}(\mathbf{s}' | \mathbf{s}, \pi(\mathbf{s})) \mathcal{V}^\pi(\mathbf{s}').\end{aligned}$$

The first equality is by the definition of the value function. The second equality follows from the law of total expectation, conditioning $\mathbf{s}_1 = \mathbf{s}'$ and taking the expectation over it. By definition $\mathbf{a}_t = \pi(\mathbf{s}_t)$. The third equality follows similarly to the finite-horizon case (Lemma 5.5, in Chapter 5). The fourth is simple algebra, taking one multiple of the discount factor γ outside. The last, by the observation in the beginning of the proof. $\square$

We can write the linear equations in (6.1) in vector form as follows. Define the column vector $\mathbf{r}^\pi = (\mathbf{r}^\pi(\mathbf{s}))_{\mathbf{s} \in \mathcal{S}}$ with components $\mathbf{r}^\pi(\mathbf{s}) = \mathbf{r}(\mathbf{s}, \pi(\mathbf{s}))$, and the transition matrix $\mathbf{P}^\pi$ with components $\mathbf{P}^\pi(\mathbf{s}' | \mathbf{s}) = \mathbf{p}(\mathbf{s}' | \mathbf{s}, \pi(\mathbf{s}))$. Finally, let $\mathcal{V}^\pi$ denote a column vector with components $\mathcal{V}^\pi(\mathbf{s})$. Then (6.1) is equivalent to the linear equation set

$$\mathcal{V}^\pi = \mathbf{r}^\pi + \gamma \mathbf{P}^\pi \mathcal{V}^\pi. \quad (6.2)$$

Lemma 6.2. *For any policy $\pi \in \Pi_{SD}$ the set of linear equations (6.1) or (6.2), with $\mathcal{V}^\pi$ as variables, has a unique solution $\mathcal{V}^\pi$, which is given by*

$$\mathcal{V}^\pi = (I - \gamma \mathbf{P}^\pi)^{-1} \mathbf{r}^\pi.$$

Proof. We only need to show that the square matrix $I - \gamma \mathbf{P}^\pi$ is non-singular. Let (λ_i) denote the eigenvalues of the matrix $\mathbf{P}^\pi$. Since $\mathbf{P}^\pi$ is a stochastic matrix (row sums are 1), then $|\lambda_i| \leq 1$ (see the proof of Theorem 4.7). Now, the eigenvalues of $I - \gamma \mathbf{P}^\pi$ are $(1 - \gamma \lambda_i)$, and satisfy $|1 - \gamma \lambda_i| \geq 1 - \gamma > 0$. $\square$

Combining Lemma 6.1 and Lemma 6.2, we obtain the following.

Proposition 6.3. *For any $\pi \in \Pi_{SD}$. The value function $\mathcal{V}^\pi = [\mathcal{V}^\pi(\mathbf{s})]$ is the unique solution of Equation (6.2), and is given by*

$$\mathcal{V}^\pi = (I - \gamma \mathbf{P}^\pi)^{-1} \mathbf{r}^\pi.$$

Proposition 6.3 provides a closed-form formula for computing $\mathcal{V}^\pi$. However, for large systems, computing the inverse $(I - \gamma \mathbf{P}^\pi)^{-1}$ may be computationally expensive. In that case, the following value iteration algorithm provides an alternative, iterative method for computing $\mathcal{V}^\pi$.

Algorithm 8 Fixed-policy Value Iteration

- 1: **Initialization:** Set $\mathcal{V}_0 = (\mathcal{V}_0(\mathbf{s}))_{\mathbf{s} \in \mathcal{S}}$ arbitrarily.
 - 2: **For** $n = 0, 1, 2, \dots$
 - 3: Set $\mathcal{V}_{n+1}(\mathbf{s}) = \mathbf{r}(\mathbf{s}, \pi(\mathbf{s})) + \gamma \sum_{\mathbf{s}' \in \mathcal{S}} \mathbf{p}(\mathbf{s}' | \mathbf{s}, \pi(\mathbf{s})) \mathcal{V}_n(\mathbf{s}') \quad \forall \mathbf{s} \in \mathcal{S}$
-

Note that Line 3 in Algorithm 8 can equivalently be written in matrix form as:

$$\mathcal{V}_{n+1} = \mathbf{r}^\pi + \gamma \mathbf{P}^\pi \mathcal{V}_n.$$

Proposition 6.4 (Convergence of fixed-policy value iteration). *We have that $\mathcal{V}_n \rightarrow \mathcal{V}^\pi$ component-wise, that is,*

$$\lim_{n \rightarrow \infty} \mathcal{V}_n(\mathbf{s}) = \mathcal{V}^\pi(\mathbf{s}), \quad \forall \mathbf{s} \in \mathcal{S}.$$

Proof. Note that,

$$\begin{aligned} \mathcal{V}_1(\mathbf{s}) &= \mathbf{r}(\mathbf{s}, \pi(\mathbf{s})) + \gamma \sum_{\mathbf{s}' \in \mathcal{S}} \mathbf{p}(\mathbf{s}' | \mathbf{s}, \pi(\mathbf{s})) \mathcal{V}_0(\mathbf{s}') \\ &= \mathbb{E}^\pi [\mathbf{r}(\mathbf{s}_0, \mathbf{a}_0) + \gamma \mathcal{V}_0(\mathbf{s}_1) | \mathbf{s}_0 = \mathbf{s}]. \end{aligned}$$

Continuing similarly, we obtain that

$$\mathcal{V}_n(\mathbf{s}) = \mathbb{E}^\pi \left[\sum_{t=0}^{n-1} \gamma^t \mathbf{r}(\mathbf{s}_t, \mathbf{a}_t) + \gamma^n \mathcal{V}_0(\mathbf{s}_n) \middle| \mathbf{s}_0 = \mathbf{s} \right].$$

Note that $\mathcal{V}_n(\mathbf{s})$ is the n -stage discounted return, with terminal reward $\mathbf{r}_n(\mathbf{s}_n) = \mathcal{V}_0(\mathbf{s}_n)$. Comparing with the definition of $\mathcal{V}^\pi$, we can see that

$$\mathcal{V}^\pi(\mathbf{s}) - \mathcal{V}_n(\mathbf{s}) = \mathbb{E}^\pi \left[\sum_{t=n}^{\infty} \gamma^t \mathbf{r}(\mathbf{s}_t, \mathbf{a}_t) - \gamma^n \mathcal{V}_0(\mathbf{s}_n) \middle| \mathbf{s}_0 = \mathbf{s} \right].$$

Denoting $R_{\max} = \max_{\mathbf{s}, \mathbf{a}} |\mathbf{r}(\mathbf{s}, \mathbf{a})|$, $\bar{\mathcal{V}}_0 = \max_{\mathbf{s}} |\mathcal{V}_0(\mathbf{s})|$ we obtain

$$|\mathcal{V}^\pi(\mathbf{s}) - \mathcal{V}_n(\mathbf{s})| \leq \gamma^n \left(\frac{R_{\max}}{1 - \gamma} + \bar{\mathcal{V}}_0 \right)$$

which converges to 0 since $\gamma < 1$. □

Comments:

- The proof provides an explicit bound on $|\mathcal{V}^\pi(\mathbf{s}) - \mathcal{V}_n(\mathbf{s})|$. It may be seen that the convergence is exponential, with rate $O(\gamma^n)$.
- Using vector notation, it may be seen that

$$\mathcal{V}_n = \mathbf{r}^\pi + \gamma \mathbf{P}^\pi \mathbf{r}^\pi + \dots + (\gamma \mathbf{P}^\pi)^{n-1} \mathbf{r}^\pi + (\gamma \mathbf{P}^\pi)^n \mathcal{V}_0 = \sum_{t=0}^{n-1} (\gamma \mathbf{P}^\pi)^t \mathbf{r}^\pi + (\gamma \mathbf{P}^\pi)^n \mathcal{V}_0.$$

$$\text{Similarly, } \mathcal{V}^\pi = \sum_{t=0}^{\infty} (\gamma \mathbf{P}^\pi)^t \mathbf{r}^\pi.$$

In summary:

- Proposition 6.3 allows to compute $\mathcal{V}^\pi$ by solving a set of $|\mathcal{S}|$ linear equations.
- Proposition 6.4 computes $\mathcal{V}^\pi$ by an infinite recursion, that converges exponentially fast.

6.3 Overview: The Main DP Algorithms

We now return to the optimal planning problem formulated in Section 6.1. Recall that $\mathcal{V}_\gamma^*(\mathbf{s}) = \sup_{\pi \in \Pi_{HS}} \mathcal{V}_\gamma^\pi(\mathbf{s})$ is the optimal discounted return. We further denote

$$\mathcal{V}^*(\mathbf{s}) \triangleq \mathcal{V}_\gamma^*(\mathbf{s}), \quad \forall \mathbf{s} \in \mathcal{S},$$

and refer to $\mathcal{V}^*$ as the optimal value function. Depending on the context, we consider $\mathcal{V}^*$ either as a function $\mathcal{V}^* : \mathcal{S} \rightarrow \mathbb{R}$, or as a column vector $\mathcal{V}^* = [\mathcal{V}^*(\mathbf{s})]_{\mathbf{s} \in \mathcal{S}}$.

The following optimality equation provides an explicit characterization of the value function, and shows that an optimal stationary policy can easily be computed if the value function is known. (See the proof in Section 6.5.)

Theorem 6.5 (Bellman's Optimality Equation). *The following statements hold:*

1. $\mathcal{V}^*$ is the unique solution of the following set of (nonlinear) equations:

$$\mathcal{V}(\mathbf{s}) = \max_{\mathbf{a} \in \mathcal{A}} \left\{ \mathbf{r}(\mathbf{s}, \mathbf{a}) + \gamma \sum_{\mathbf{s}' \in \mathcal{S}} \mathbf{p}(\mathbf{s}' | \mathbf{s}, \mathbf{a}) \mathcal{V}(\mathbf{s}') \right\}, \quad \forall \mathbf{s} \in \mathcal{S}. \quad (6.3)$$

2. Any stationary policy π^* that satisfies

$$\pi^*(\mathbf{s}) \in \arg \max_{\mathbf{a} \in \mathcal{A}} \left\{ \mathbf{r}(\mathbf{s}, \mathbf{a}) + \gamma \sum_{\mathbf{s}' \in \mathcal{S}} \mathbf{p}(\mathbf{s}' | \mathbf{s}, \mathbf{a}) \mathcal{V}^*(\mathbf{s}') \right\} \quad \forall \mathbf{s} \in \mathcal{S},$$

is an optimal policy (for any initial state $\mathbf{s}_0 \in \mathcal{S}$).

The optimality equation (6.3) is non-linear, and generally requires iterative algorithms for its solution. The main iterative algorithms are **value iteration** and **policy iteration**. In the following we provide the algorithms and the basic claims. Later in this chapter we formally prove the results regarding value iteration (Section 6.6) and policy iteration (Section 6.7).

Algorithm 9 Value Iteration (VI)

- 1: **Input:** number of iterations n
 - 2: **Initialization:** Set $\mathcal{V}_0 = (\mathcal{V}_0(\mathbf{s}))_{\mathbf{s} \in \mathcal{S}}$ arbitrarily
 - 3: **For** $k = 0, 1, 2, \dots, n - 1$
 - 4: Set $\mathcal{V}_{k+1}(\mathbf{s}) = \max_{\mathbf{a} \in \mathcal{A}} \left\{ \mathbf{r}(\mathbf{s}, \mathbf{a}) + \gamma \sum_{\mathbf{s}' \in \mathcal{S}} \mathbf{p}(\mathbf{s}' | \mathbf{s}, \mathbf{a}) \mathcal{V}_k(\mathbf{s}') \right\}, \quad \forall \mathbf{s} \in \mathcal{S}$
-

Theorem 6.6 (Convergence of value iteration). *We have $\lim_{n \rightarrow \infty} \mathcal{V}_n = \mathcal{V}^*$ (component-wise). The rate of convergence is exponential, at rate $O(\gamma^n)$.*

Proof. Recalling the finite horizon dynamic programming recursion in Theorem 5.6, we observe that the sequence of computations in Algorithm 9 is the same as in finite horizon dynamic programming with a stationary MDP model (i.e., $p_k(s'|s, a) = p(s'|s, a)$), discounted rewards $r_k(s, a) = \gamma^k r(s, a)$, and a terminal reward equal to the arbitrary initial value in Algorithm 9 discounted by γ^n , i.e., $r_T(s) = \gamma^n \mathcal{V}_0(s)$. Using Theorem 5.6, it follows that after n iterations we have that $\mathcal{V}_n$ is the optimal value of a corresponding finite horizon MDP:

$$\mathcal{V}_n(s) = \max_{\pi} \mathbb{E}^{\pi, s} \left[\sum_{t=0}^{n-1} \gamma^t r(s_t, a_t) + \gamma^n \mathcal{V}_0(s_n) \right].$$

Comparing to the optimal value function

$$\mathcal{V}^*(s) = \max_{\pi} \mathbb{E}^{\pi, s} \left[\sum_{t=0}^{\infty} \gamma^t r(s_t, a_t) \right],$$

it may be seen that that

$$|\mathcal{V}_n(s) - \mathcal{V}^*(s)| \leq \gamma^n \left(\frac{R_{\max}}{1 - \gamma} + \|\mathcal{V}_0\|_{\infty} \right).$$

As $\gamma < 1$, this implies that $\mathcal{V}_n$ converges to $\mathcal{V}_{\gamma}^*$ exponentially fast. $\square$

The value iteration algorithm iterates over the value functions, with asymptotic convergence. The policy iteration algorithm iterates over stationary policies, with each new policy better than the previous one. This algorithm converges to the optimal policy in a finite number of steps.

Algorithm 10 Policy Iteration (PI)

- 1: **Initialization:** choose some stationary policy π_0 .
 - 2: **For** $k = 0, 1, 2, \dots$
 - 3: **Policy Evaluation:** Compute $\mathcal{V}^{\pi_k}$.
 - 4: (For example, use the explicit formula $\mathcal{V}^{\pi_k} = (I - \gamma P^{\pi_k})^{-1} r^{\pi_k}$)
 - 5: **Policy Improvement:** Compute π_{k+1} , a greedy policy with respect to $\mathcal{V}^{\pi_k}$:
 - 6: $\pi_{k+1}(s) \in \arg \max_{a \in \mathcal{A}} \{r(s, a) + \gamma \sum_{s' \in \mathcal{S}} p(s'|s, a) \mathcal{V}^{\pi_k}(s')\}, \forall s \in \mathcal{S}$.
 - 7: **If** $\pi_{k+1} = \pi_k$ (or if $\mathcal{V}^{\pi_k}$ satisfies the optimality equation)
 - 8: **Stop**
-

Theorem 6.7 (Convergence of policy iteration). *The following statements hold:*

1. Each policy π_{k+1} is improving over the previous one π_k , in the sense that $\mathcal{V}^{\pi_{k+1}} \geq \mathcal{V}^{\pi_k}$ (component-wise).
2. $\mathcal{V}^{\pi_{k+1}} = \mathcal{V}^{\pi_k}$ if and only if π_k is an optimal policy.
3. Consequently, since the number of stationary deterministic policies is finite, π_k converges to the optimal policy after a finite number of steps.

Remark 6.2. An additional solution method for DP planning relies on a Linear Programming formulation of the problem. See chapter 8.

6.4 Contraction Operators

The basic proof methods of the DP results mentioned above rely on the concept of a *contraction operator*. We provide here the relevant mathematical background, and illustrate the contraction properties of some basic Dynamic Programming operators.

6.4.1 The contraction property

Recall that a norm $\|\cdot\|$ over $\mathbb{R}^n$ is a real-valued function $\|\cdot\| : \mathbb{R}^d \rightarrow \mathbb{R}^+$ such that, for any pair of vectors $x, y \in \mathbb{R}^d$ and scalar $a \in \mathbb{R}$,

1. $\|ax\| = |a| \cdot \|x\|$,
2. $\|x + y\| \leq \|x\| + \|y\|$,
3. $\|x\| = 0$ only if $x = 0$.

Common examples are the p-norm $\|x\|_p = (\sum_{i=1}^d |x_i|^p)^{1/p}$ for $p \geq 1$, and in particular the Euclidean norm ($p = 2$). Here we will mostly use the max-norm:

$$\|x\|_\infty = \max_{1 \leq i \leq d} |x_i|.$$

Unless otherwise mentioned, we will omit the ∞ and use $\|\cdot\|$ to abbreviate the max-norm throughout the book.

Let $T : \mathbb{R}^d \rightarrow \mathbb{R}^d$ be a vector-valued function over $\mathbb{R}^d$ ($d \geq 1$). We equip $\mathbb{R}^d$ with some norm $\|\cdot\|$, and refer to T as an *operator* over $\mathbb{R}^d$. Thus, $T(v) \in \mathbb{R}^d$ for any $v \in \mathbb{R}^d$. We also denote $T^n(v) = T(T^{n-1}(v))$ for $n \geq 2$. For example, $T^2(v) = T(T(v))$.

Definition 6.1. The operator T is called a contraction operator with respect to a norm $\|\cdot\|$, if there exists $\beta \in (0, 1)$ (the contraction coefficient) such that

$$\|T(v_1) - T(v_2)\| \leq \beta \|v_1 - v_2\|,$$

for any $v_1, v_2 \in \mathbb{R}^d$. In which case, T is called a β -contraction operator.

When the norm is clear from the context, we will omit explicit mentioning of it.

6.4.2 The Banach Fixed Point Theorem

The following celebrated result applies to contraction operators. While we quote the result for $\mathbb{R}^d$, we note that it applies in much greater generality to any Banach space (a complete normed space), with essentially the same proof.

Theorem 6.8 (Banach's fixed point theorem). Let $T : \mathbb{R}^d \rightarrow \mathbb{R}^d$ be a contraction operator with respect to some norm $\|\cdot\|$. Then

1. The equation $T(v) = v$ has a unique solution $\mathcal{V}^* \in \mathbb{R}^d$.
2. For any $v_0 \in \mathbb{R}^d$, $\lim_{n \rightarrow \infty} T^n(v_0) = \mathcal{V}^*$. In fact, $\|T^n(v_0) - \mathcal{V}^*\| \leq O(\beta^n)$, where β is the contraction coefficient.

Proof. Fix any v_0 and define $v_{n+1} = T(v_n)$. We will show that: (1) there exists a limit to the sequence, and (2) the limit is a fixed point of T .

Existence of a limit v^* of the sequence v_n

We show that the sequence of v_n is a Cauchy sequence. We consider two elements v_n and v_{n+m} and bound the distance between them.

$$\begin{aligned} \|v_{n+m} - v_n\| &= \left\| \sum_{k=0}^{m-1} v_{n+k+1} - v_{n+k} \right\| \\ &\leq \sum_{k=0}^{m-1} \|v_{n+k+1} - v_{n+k}\| \quad (\text{according to the triangle inequality}) \\ &= \sum_{k=0}^{m-1} \|T^{n+k}(v_1) - T^{n+k}(v_0)\| \\ &\leq \sum_{k=0}^{m-1} \beta^{n+k} \|v_1 - v_0\| \quad (\text{contraction } (n+k) \text{ times}) \\ &= \frac{\beta^n(1 - \beta^m)}{1 - \beta} \|v_1 - v_0\| \end{aligned}$$

Since the coefficient decreases as n increases, for any $\varepsilon > 0$ there exists $N > 0$ such that for all $n, m \geq N$, we have $\|v_{n+m} - v_n\| < \varepsilon$. This implies that the sequence is a Cauchy sequence, and hence the sequence v_n has a limit because the space is complete. Let us call this limit v^* . Next we show that v^* is a fixed point of the operator T .

The limit v^* is a fixed point

We need to show that $T(v^*) = v^*$, or equivalently by the third property of norms we need to show that $\|T(v^*) - v^*\| = 0$.

$$\begin{aligned}
0 &\leq \|T(v^*) - v^*\| \\
&\leq \|T(v^*) - v_n\| + \|v_n - v^*\| \text{ (according to the triangle inequality)} \\
&= \|T(v^*) - T(v_{n-1})\| + \|v_n - v^*\| \\
&\leq \beta \underbrace{\|v^* - v_{n-1}\|}_{\rightarrow 0} + \underbrace{\|v_n - v^*\|}_{\rightarrow 0}
\end{aligned}$$

Since v^* is the limit of v_n , i.e., $\lim_{n \rightarrow \infty} \|v_n - v^*\| = 0$ hence

$$\|Tv^* - v^*\| = 0.$$

Thus, v^* is a fixed point of the operator T .

Uniqueness of v^*

Assume that $T(v_1) = v_1$, $T(v_2) = v_2$, and $v_1 \neq v_2$. Then

$$\|v_1 - v_2\| = \|T(v_1) - T(v_2)\| \leq \beta \|v_1 - v_2\|$$

Hence, this is in contradiction to $\beta < 1$. Therefore, v^* is unique. $\square$

6.4.3 The Dynamic Programming Operators

We next define the basic dynamic programming operators, and show that they contraction operators.

Definition 6.2. For a fixed stationary policy $\pi : \mathcal{S} \rightarrow \mathcal{A}$, define the **Fixed Policy Dynamic Programming Operator** $\mathcal{T}^\pi : \mathbb{R}^{|\mathcal{S}|} \rightarrow \mathbb{R}^{|\mathcal{S}|}$ as follows: For any $\mathcal{V} = (\mathcal{V}(\mathbf{s})) \in \mathbb{R}^{|\mathcal{S}|}$,

$$(\mathcal{T}^\pi(\mathcal{V}))(\mathbf{s}) = \mathbf{r}(\mathbf{s}, \pi(\mathbf{s})) + \gamma \sum_{\mathbf{s}' \in \mathcal{S}} \mathbf{p}(\mathbf{s}' | \mathbf{s}, \pi(\mathbf{s})) \mathcal{V}(\mathbf{s}'), \quad \forall \mathbf{s} \in \mathcal{S}.$$

In our column-vector notation, this is equivalent to $\mathcal{T}^\pi(\mathcal{V}) = \mathbf{r}^\pi + \gamma \mathbf{P}^\pi \mathcal{V}$.

Definition 6.3. Define the discounted-return *Dynamic Programming Operator* $\mathcal{T}^* : \mathbb{R}^{|\mathcal{S}|} \rightarrow \mathbb{R}^{|\mathcal{S}|}$ as follows: For any $\mathcal{V} = (\mathcal{V}(\mathbf{s})) \in \mathbb{R}^{|\mathcal{S}|}$,

$$(\mathcal{T}^*(\mathcal{V}))(\mathbf{s}) = \max_{\mathbf{a} \in \mathcal{A}} \left\{ \mathbf{r}(\mathbf{s}, \mathbf{a}) + \gamma \sum_{\mathbf{s}' \in \mathcal{S}} \mathbf{p}(\mathbf{s}' | \mathbf{s}, \mathbf{a}) \mathcal{V}(\mathbf{s}') \right\}, \quad \forall \mathbf{s} \in \mathcal{S}$$

We note that $\mathcal{T}^\pi$ is a linear operator, while $\mathcal{T}^*$ is generally non-linear due to the maximum operation.

Let $\|\mathcal{V}\|_\infty \triangleq \max_{\mathbf{s} \in \mathcal{S}} |\mathcal{V}(\mathbf{s})|$ denote the max-norm of $\mathcal{V}$. Recall that $0 < \gamma < 1$.

Theorem 6.9 (Contraction property). *The following statements hold:*

1. $\mathcal{T}^\pi$ is a γ -contraction operator with respect to the max-norm, namely $\|\mathcal{T}^\pi(\mathcal{V}_1) - \mathcal{T}^\pi(\mathcal{V}_2)\|_\infty \leq \gamma \|\mathcal{V}_1 - \mathcal{V}_2\|_\infty$ for all $\mathcal{V}_1, \mathcal{V}_2 \in \mathbb{R}^{|\mathcal{S}|}$.
2. Similarly, $\mathcal{T}^*$ is a γ -contraction operator with respect to the max-norm.

Proof. 1. Fix $\mathcal{V}_1, \mathcal{V}_2$. For every state $\mathbf{s}$,

$$\begin{aligned} \|(\mathcal{T}^\pi(\mathcal{V}_1))(\mathbf{s}) - (\mathcal{T}^\pi(\mathcal{V}_2))(\mathbf{s})\| &= \left\| \gamma \sum_{\mathbf{s}' \in \mathcal{S}} \mathbf{p}(\mathbf{s}' | \mathbf{s}, \pi(\mathbf{s})) (\mathcal{V}_1(\mathbf{s}') - \mathcal{V}_2(\mathbf{s}')) \right\| \\ &\leq \gamma \sum_{\mathbf{s}' \in \mathcal{S}} \mathbf{p}(\mathbf{s}' | \mathbf{s}, \pi(\mathbf{s})) \|\mathcal{V}_1(\mathbf{s}') - \mathcal{V}_2(\mathbf{s}')\| \\ &\leq \gamma \sum_{\mathbf{s}' \in \mathcal{S}} \mathbf{p}(\mathbf{s}' | \mathbf{s}, \pi(\mathbf{s})) \|\mathcal{V}_1 - \mathcal{V}_2\|_\infty \\ &= \gamma \|\mathcal{V}_1 - \mathcal{V}_2\|_\infty. \end{aligned}$$

The first inequality is since $\mathbf{p}(\mathbf{s}' | \mathbf{s}, \pi(\mathbf{s})) \geq 0$, and the last equality is since $\sum_{\mathbf{s}' \in \mathcal{S}} \mathbf{p}(\mathbf{s}' | \mathbf{s}, \pi(\mathbf{s})) = 1$. Since this result holds for every $\mathbf{s} \in \mathcal{S}$ the required inequality follows.

2. The proof here is more intricate due to the maximum operation. As before, we need to show that $\|\mathcal{T}^*(\mathcal{V}_1)(\mathbf{s}) - \mathcal{T}^*(\mathcal{V}_2)(\mathbf{s})\| \leq \gamma \|\mathcal{V}_1 - \mathcal{V}_2\|_\infty$. Fixing the state $\mathbf{s}$, we consider separately the positive and negative parts of the absolute value:
 - (a) Showing $\mathcal{T}^*(\mathcal{V}_1)(\mathbf{s}) - \mathcal{T}^*(\mathcal{V}_2)(\mathbf{s}) \leq \gamma \|\mathcal{V}_1 - \mathcal{V}_2\|_\infty$: Let $\bar{\mathbf{a}}$ denote an action that attains the maximum in $\mathcal{T}^*(\mathcal{V}_1)(\mathbf{s})$, namely

$$\bar{\mathbf{a}} \in \arg \max_{\mathbf{a} \in \mathcal{A}} \left\{ \mathbf{r}(\mathbf{s}, \mathbf{a}) + \gamma \sum_{\mathbf{s}' \in \mathcal{S}} \mathbf{p}(\mathbf{s}' | \mathbf{s}, \mathbf{a}) \mathcal{V}_1(\mathbf{s}') \right\}.$$

Then

$$\mathcal{T}^*(\mathcal{V}_1)(\mathbf{s}) = r(\mathbf{s}, \bar{\mathbf{a}}) + \gamma \sum_{\mathbf{s}' \in \mathcal{S}} p(\mathbf{s}'|\mathbf{s}, \bar{\mathbf{a}}) \mathcal{V}_1(\mathbf{s}')$$

$$\mathcal{T}^*(\mathcal{V}_2)(\mathbf{s}) \geq r(\mathbf{s}, \bar{\mathbf{a}}) + \gamma \sum_{\mathbf{s}' \in \mathcal{S}} p(\mathbf{s}'|\mathbf{s}, \bar{\mathbf{a}}) \mathcal{V}_2(\mathbf{s}')$$

Since the same action $\bar{\mathbf{a}}$ appears in both expressions, we can now continue to show the inequality (a) similarly to 1. Namely,

$$\begin{aligned} (\mathcal{T}^*(\mathcal{V}_1))(\mathbf{s}) - (\mathcal{T}^*(\mathcal{V}_2))(\mathbf{s}) &\leq \gamma \sum_{\mathbf{s}' \in \mathcal{S}} p(\mathbf{s}'|\mathbf{s}, \bar{\mathbf{a}}) (\mathcal{V}_1(\mathbf{s}') - \mathcal{V}_2(\mathbf{s}')) \\ &\leq \gamma \sum_{\mathbf{s}' \in \mathcal{S}} p(\mathbf{s}'|\mathbf{s}, \bar{\mathbf{a}}) \|\mathcal{V}_1 - \mathcal{V}_2\|_\infty = \gamma \|\mathcal{V}_1 - \mathcal{V}_2\|_\infty. \end{aligned}$$

(b) Showing $\mathcal{T}^*(\mathcal{V}_2)(\mathbf{s}) - \mathcal{T}^*(\mathcal{V}_1)(\mathbf{s}) \leq \gamma \|\mathcal{V}_1 - \mathcal{V}_2\|_\infty$. Similarly to the proof of (a) we have

$$\mathcal{T}^*(\mathcal{V}_2)(\mathbf{s}) - \mathcal{T}^*(\mathcal{V}_1)(\mathbf{s}) \leq \gamma \|\mathcal{V}_2 - \mathcal{V}_1\|_\infty = \gamma \|\mathcal{V}_1 - \mathcal{V}_2\|_\infty.$$

The inequalities (a) and (b) together imply that $\|\mathcal{T}^*(\mathcal{V}_1)(\mathbf{s}) - \mathcal{T}^*(\mathcal{V}_2)(\mathbf{s})\| \leq \gamma \|\mathcal{V}_1 - \mathcal{V}_2\|_\infty$. Since this holds for any state $\mathbf{s}$, it follows that $\|\mathcal{T}^*(\mathcal{V}_1) - \mathcal{T}^*(\mathcal{V}_2)\|_\infty \leq \gamma \|\mathcal{V}_1 - \mathcal{V}_2\|_\infty$. □

6.5 Proof of Bellman's Optimality Equation

We prove in this section Theorem 6.5, which is restated here:

Theorem (Bellman's Optimality Equation). *The following statements hold:*

1. $\mathcal{V}^*$ is the unique solution of the following set of (nonlinear) equations:

$$\mathcal{V}(\mathbf{s}) = \max_{\mathbf{a} \in \mathcal{A}} \left\{ r(\mathbf{s}, \mathbf{a}) + \gamma \sum_{\mathbf{s}' \in \mathcal{S}} p(\mathbf{s}'|\mathbf{s}, \mathbf{a}) \mathcal{V}(\mathbf{s}') \right\}, \quad \forall \mathbf{s} \in \mathcal{S}. \quad (6.4)$$

2. Any stationary policy π^* that satisfies

$$\pi^*(\mathbf{s}) \in \arg \max_{\mathbf{a} \in \mathcal{A}} \left\{ r(\mathbf{s}, \mathbf{a}) + \gamma \sum_{\mathbf{s}' \in \mathcal{S}} p(\mathbf{s}'|\mathbf{s}, \mathbf{a}) \mathcal{V}^*(\mathbf{s}') \right\}, \quad \forall \mathbf{s} \in \mathcal{S}$$

is an optimal policy (for any initial state $\mathbf{s}_0 \in \mathcal{S}$).

We observe that the Optimality equation in part 1 is equivalent to $\mathcal{V} = \mathcal{T}^*(\mathcal{V})$ where $\mathcal{T}^*$ is the optimal DP operator from the previous section, which was shown to be a contraction operator with coefficient γ . The proof also uses the value iteration property of Theorem 6.6.

Proof of Theorem 6.5: We prove each part.

1. As $\mathcal{T}^*$ is a contraction operator, existence and uniqueness of the solution to $\mathcal{V} = \mathcal{T}^*(\mathcal{V})$ follows from the Banach fixed point theorem (Theorem 6.8). Let $\widehat{V}$ denote the solution. It also follows by Theorem 6.8 that $(\mathcal{T}^*)^n(\mathcal{V}_0) \rightarrow \widehat{V}$ for any $\mathcal{V}_0$. By Theorem 6.6 we have that $(\mathcal{T}^*)^n(\mathcal{V}_0) \rightarrow \mathcal{V}^*$, hence $\widehat{V} = \mathcal{V}^*$, so that $\mathcal{V}^*$ is indeed the unique solution of $V = \mathcal{T}^*(V)$.
2. By definition of π^* we have

$$T^{\pi^*}(\mathcal{V}^*) = \mathcal{T}^*(\mathcal{V}^*) = \mathcal{V}^*,$$

where the last equality follows from part 1. Thus, the optimal value function satisfies the equation $T^{\pi^*}(\mathcal{V}^*) = \mathcal{V}^*$. But we already know (from Proposition 6.3) that $\mathcal{V}^{\pi^*}$ is the unique solution of that equation, hence $\mathcal{V}^{\pi^*} = \mathcal{V}^*$. This implies that π^* achieves the optimal value (for any initial state), and is therefore an optimal policy as stated. □

6.6 Value Iteration (VI)

The value iteration algorithm allows to compute the optimal value function $\mathcal{V}^*$ iteratively to any required accuracy. The Value Iteration algorithm (Algorithm 9) can be stated as follows:

1. Start with any initial value function $\mathcal{V}_0 = (\mathcal{V}_0(\mathbf{s}))$.
2. Compute recursively, for $n = 0, 1, 2, \dots$,

$$\mathcal{V}_{n+1}(\mathbf{s}) = \max_{\mathbf{a} \in \mathcal{A}} \sum_{\mathbf{s}' \in \mathcal{S}} p(\mathbf{s}' | \mathbf{s}, \mathbf{a}) [r(\mathbf{s}, \mathbf{a}, \mathbf{s}') + \gamma \mathcal{V}_n(\mathbf{s}')], \quad \forall \mathbf{s} \in \mathcal{S}.$$

3. Apply a stopping rule to obtain a required accuracy (see below).

In terms of the DP operator $\mathcal{T}^*$, value iteration is simply stated as:

$$\mathcal{V}_{n+1} = \mathcal{T}^*(\mathcal{V}_n), \quad n \geq 0.$$

Note that the number of operations for each iteration is $O(|\mathcal{A}| \cdot |\mathcal{S}|^2)$. Theorem 6.6 states that $\mathcal{V}_n \rightarrow \mathcal{V}^*$, exponentially fast.

6.6.1 Error bounds and stopping rules

While we showed an exponential convergence rate, it is important to have a criterion that depends only on the observed quantities. That is, we wish to bound the difference between $\mathcal{V}_n$ and $\mathcal{V}^*$ (which is unobserved) in terms of $\mathcal{V}_\ell$ for different ℓ which are observed.

Lemma 6.10. *If $\|\mathcal{V}_{n+1} - \mathcal{V}_n\| < \varepsilon \cdot \frac{1-\gamma}{2\gamma}$ then $\|\mathcal{V}_{n+1} - \mathcal{V}^*\| < \frac{\varepsilon}{2}$ and $\|\mathcal{V}^{\pi_{n+2}} - \mathcal{V}^*\| \leq \varepsilon$, where π_{n+2} is the greedy policy w.r.t. $\mathcal{V}_{n+1}$.*

Proof. Assume that $\|\mathcal{V}_{n+1} - \mathcal{V}_n\| < \varepsilon \cdot \frac{1-\gamma}{2\gamma}$, we show that $\|\mathcal{V}^{\pi_{n+2}} - \mathcal{V}^*\| < \varepsilon$, which would make the policy π_{n+2} ε -optimal. We bound the difference between $\mathcal{V}^{\pi_{n+2}}$ and $\mathcal{V}^*$. We consider the following:

$$\|\mathcal{V}^{\pi_{n+2}} - \mathcal{V}^*\| \leq \|\mathcal{V}^{\pi_{n+2}} - \mathcal{V}_{n+1}\| + \|\mathcal{V}_{n+1} - \mathcal{V}^*\|. \quad (6.5)$$

We now bound each part of the sum separately:

$$\begin{aligned} \|\mathcal{V}^{\pi_{n+2}} - \mathcal{V}_{n+1}\| &= \|\mathcal{T}^{\pi_{n+2}}(\mathcal{V}^{\pi_{n+2}}) - \mathcal{V}_{n+1}\| \quad (\mathcal{V}^{\pi_{n+2}} \text{ is the fixed point of } \mathcal{T}^{\pi_{n+2}}) \\ &\leq \|\mathcal{T}^{\pi_{n+2}}(\mathcal{V}^{\pi_{n+2}}) - \mathcal{T}^*(\mathcal{V}_{n+1})\| + \|\mathcal{T}^*(\mathcal{V}_{n+1}) - \mathcal{V}_{n+1}\|. \end{aligned}$$

Since π_{n+2} is maximal over the actions using $\mathcal{V}_{n+1}$, it implies that $\mathcal{T}^{\pi_{n+2}}(\mathcal{V}_{n+1}) = \mathcal{T}^*(\mathcal{V}_{n+1})$ and we conclude that:

$$\begin{aligned} \|\mathcal{V}^{\pi_{n+2}} - \mathcal{V}_{n+1}\| &\leq \|\mathcal{T}^{\pi_{n+2}}(\mathcal{V}^{\pi_{n+2}}) - \mathcal{T}^{\pi_{n+2}}(\mathcal{V}_{n+1})\| + \|\mathcal{T}^*(\mathcal{V}_{n+1}) - \mathcal{T}^*(\mathcal{V}_n)\| \\ &\leq \gamma \|\mathcal{V}^{\pi_{n+2}} - \mathcal{V}_{n+1}\| + \gamma \|\mathcal{V}_{n+1} - \mathcal{V}_n\|. \end{aligned}$$

Rearranging, this implies that,

$$\|\mathcal{V}^{\pi_{n+2}} - \mathcal{V}_{n+1}\| \leq \frac{\gamma}{1-\gamma} \|\mathcal{V}_{n+1} - \mathcal{V}_n\| < \frac{\gamma}{1-\gamma} \cdot \varepsilon \cdot \frac{1-\gamma}{2\gamma} = \frac{\varepsilon}{2}.$$

For the second part of the sum we derive similarly that:

$$\begin{aligned} \|\mathcal{V}_{n+1} - \mathcal{V}^*\| &\leq \|\mathcal{V}_{n+1} - \mathcal{T}^*(\mathcal{V}_{n+1})\| + \|\mathcal{T}^*(\mathcal{V}_{n+1}) - \mathcal{V}^*\| \\ &= \|\mathcal{T}^*(\mathcal{V}_n) - \mathcal{T}^*(\mathcal{V}_{n+1})\| + \|\mathcal{T}^*(\mathcal{V}_{n+1}) - \mathcal{T}^*(\mathcal{V}^*)\| \\ &\leq \gamma \|\mathcal{V}_n - \mathcal{V}_{n+1}\| + \gamma \|\mathcal{V}_{n+1} - \mathcal{V}^*\|, \end{aligned}$$

and therefore

$$\|\mathcal{V}_{n+1} - \mathcal{V}^*\| \leq \frac{\gamma}{1-\gamma} \|\mathcal{V}_{n+1} - \mathcal{V}_n\| < \frac{\gamma}{1-\gamma} \cdot \varepsilon \cdot \frac{1-\gamma}{2\gamma} = \frac{\varepsilon}{2}$$

Returning to inequality (6.5), it follows:

$$\|\mathcal{V}^{\pi_{n+2}} - \mathcal{V}^*\| \leq \frac{\varepsilon}{2} + \frac{\varepsilon}{2} = \varepsilon.$$

Therefore, the selected policy π_{n+2} is ε -optimal. □

6.7 Policy Iteration (PI)

The policy iteration algorithm, introduced by Howard [60], computes an optimal policy π^* in a finite number of steps. This number is typically small (on the same order as $|\mathcal{S}|$). There is a significant body of work to bound the number of iterations as a function of the number of states and actions, for more, see the bibliography remarks in Chapter 6.9.

The basic principle behind PI is *policy improvement*. Let π be a stationary policy, and let $\mathcal{V}^\pi$ denote its value function. A stationary policy $\bar{\pi}$ is called π -improving if it is a greedy policy with respect to $\mathcal{V}^\pi$, namely

$$\bar{\pi}(\mathbf{s}) \in \arg \max_{\mathbf{a} \in \mathcal{A}} \left\{ r(\mathbf{s}, \mathbf{a}) + \gamma \sum_{\mathbf{s}' \in \mathcal{S}} p(\mathbf{s}' | \mathbf{s}, \mathbf{a}) \mathcal{V}^\pi(\mathbf{s}') \right\}, \quad \forall \mathbf{s} \in \mathcal{S}.$$

Lemma 6.11 (Policy Improvement). *Let π be a stationary policy and $\bar{\pi}$ be a π -improving policy. We have $\mathcal{V}^{\bar{\pi}} \geq \mathcal{V}^\pi$ (component-wise), and $\mathcal{V}^{\bar{\pi}} = \mathcal{V}^\pi$ if and only if π is an optimal policy.*

Proof. Observe first that

$$\mathcal{V}^\pi = \mathcal{T}^\pi(\mathcal{V}^\pi) \leq \mathcal{T}^*(\mathcal{V}^\pi) = \mathcal{T}^{\bar{\pi}}(\mathcal{V}^\pi).$$

The first equality follows since $\mathcal{V}^\pi$ is the value function for the policy π , the inequality follows because of the maximization in the definition of $\mathcal{T}^*$, and the last equality by definition of the improving policy $\bar{\pi}$.

It is easily seen that $\mathcal{T}^\pi$ is a monotone operator (for any policy π), namely $\mathcal{V}_1 \leq \mathcal{V}_2$ implies $\mathcal{T}^\pi(\mathcal{V}_1) \leq \mathcal{T}^\pi(\mathcal{V}_2)$. Applying $\mathcal{T}^{\bar{\pi}}$ repeatedly to both sides of the above inequality $\mathcal{V}^\pi \leq \mathcal{T}^{\bar{\pi}}(\mathcal{V}^\pi)$ therefore gives

$$\mathcal{V}^\pi \leq \mathcal{T}^{\bar{\pi}}(\mathcal{V}^\pi) \leq (\mathcal{T}^{\bar{\pi}})^2(\mathcal{V}^\pi) \leq \dots \leq \lim_{n \rightarrow \infty} (\mathcal{T}^{\bar{\pi}})^n(\mathcal{V}^\pi) = \mathcal{V}^{\bar{\pi}}, \quad (6.6)$$

where the last equality follows by Proposition 6.4. This establishes the first claim.

We now show that π is optimal if and only if $\mathcal{V}^{\bar{\pi}} = \mathcal{V}^\pi$. We showed that $\mathcal{V}^{\bar{\pi}} \geq \mathcal{V}^\pi$. If $\mathcal{V}^{\bar{\pi}} > \mathcal{V}^\pi$ then clearly π is not optimal. Assume that $\mathcal{V}^{\bar{\pi}} = \mathcal{V}^\pi$. We have the following identities:

$$\mathcal{V}^\pi = \mathcal{V}^{\bar{\pi}} = \mathcal{T}^{\bar{\pi}}(\mathcal{V}^{\bar{\pi}}) = \mathcal{T}^{\bar{\pi}}(\mathcal{V}^\pi) = \mathcal{T}^*(\mathcal{V}^\pi),$$

where the first equality is by our assumption. The second equality follows since $\mathcal{V}^{\bar{\pi}}$ is the fixed point of its operator $\mathcal{T}^{\bar{\pi}}$. The third follows since we assume that $\mathcal{V}^{\bar{\pi}} = \mathcal{V}^\pi$. The last equality follows since $\mathcal{T}^{\bar{\pi}}$ and $\mathcal{T}^*$ are identical on $\mathcal{V}^\pi$.

We have established that: $\mathcal{V}^\pi = \mathcal{T}^*(\mathcal{V}^\pi)$, and hence $\mathcal{V}^\pi$ and π is a fixed point of $\mathcal{T}^*$ and therefore, by Theorem 6.5, policy π is optimal. $\square$

The policy iteration algorithm performs successive rounds of policy improvement, where each policy π_{k+1} improves the previous one π_k . Since the number of stationary deterministic policies is bounded, so is the number of strict improvements, and the algorithm must terminate with an optimal policy after a finite number of iterations.

In terms of computational complexity, Policy Iteration requires $O(|\mathcal{A}| \cdot |\mathcal{S}|^2 + |\mathcal{S}|^3)$ operations per iteration, while Value Iteration requires $O(|\mathcal{A}| \cdot |\mathcal{S}|^2)$ per iteration. However, in many cases the Policy Iteration has a smaller number of iterations than Value Iteration, as we show in the next section. Another consideration is that the number of iterations of Value Iteration increases as the discount factor γ approaches 1, while the number of policies (which upper bound the number of iterations of Policy Iteration) is independent of γ .

6.8 A Comparison between VI and PI Algorithms

In this section we will compare the convergence rate of the VI and PI algorithms. We show that, assuming that the two algorithms begin with the same approximated value, the PI algorithm converges in less iterations.

Theorem 6.12. *Let $\{VI_n\}$ be the sequence of values created by the VI algorithm (where $VI_{n+1} = \mathcal{T}^*(VI_n)$) and let $\{PI_n\}$ be the sequence of values created by PI algorithm, i.e., $PI_n = \mathcal{V}^{\pi_n}$. If the initial VI value satisfies $VI_0 = PI_0$, then for all n we have $VI_n \leq PI_n \leq \mathcal{V}^*$.*

Proof. The proof is by induction on n .

Induction Basis: By construction $VI_0 = PI_0$. Since $PI_0 = \mathcal{V}^{\pi_0}$, it is clearly bounded by $\mathcal{V}^*$.

Induction Step: Assume that $VI_n \leq PI_n$. For VI_{n+1} we have,

$$VI_{n+1} = \mathcal{T}^*(VI_n) = \mathcal{T}^{\pi'}(VI_n),$$

where π' is the greedy policy w.r.t. VI_n , i.e.,

$$\pi'(\mathbf{s}) \in \arg \max_{a \in A} \{r(\mathbf{s}, a) + \gamma \sum_{\mathbf{s}' \in \mathcal{S}} p(\mathbf{s}' | \mathbf{s}, a) VI_n(\mathbf{s}')\} \quad \forall \mathbf{s} \in \mathcal{S}.$$

Since $VI_n \leq PI_n$, and $\mathcal{T}^{\pi'}$ is monotonic it follows that:

$$\mathcal{T}^{\pi'}(VI_n) \leq \mathcal{T}^{\pi'}(PI_n).$$

Since $\mathcal{T}^*$ is upper bounding any $\mathcal{T}^\pi$:

$$\mathcal{T}^{\pi'}(PI_n) \leq \mathcal{T}^*(PI_n).$$

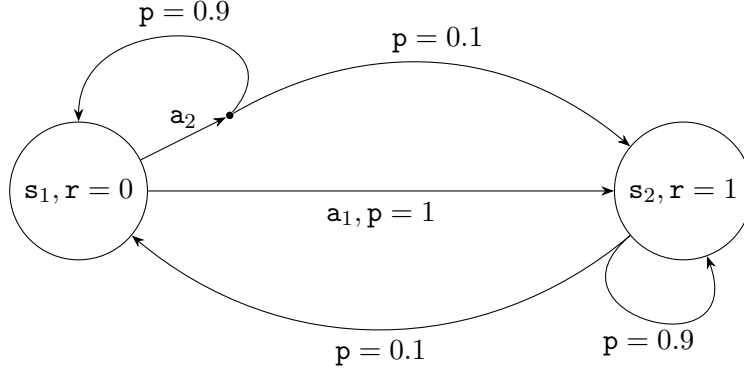


Figure 6.1: Example MDP for comparing VI and PI.

The policy determined by PI algorithm in iteration $n + 1$ is π_{n+1} and we have:

$$\mathcal{T}^*(PI_n) = \mathcal{T}^{\pi_{n+1}}(PI_n).$$

From the definition of π_{n+1} (cf. Eq. 6.6), we have

$$\mathcal{T}^{\pi_{n+1}}(PI_n) \leq \mathcal{V}^{\pi_{n+1}} = PI_{n+1}.$$

Therefore, $VI_{n+1} \leq PI_{n+1}$. Since $PI_{n+1} = \mathcal{V}^{\pi_{n+1}}$, it implies that $PI_{n+1} \leq \mathcal{V}^*$. $\square$

The next example demonstrates several important differences between value iteration and policy iteration.

Example 6.1. We consider a 2-state MDP as depicted in Figure 6.1, with action selection only at state $\mathbf{s}_1$, and a discount factor $\gamma = 0.5$. Let the policy in state $\mathbf{s}_1$ be parametrized by x :

$$x = \pi(\mathbf{a}_1), \quad \text{and hence} \quad \pi(\mathbf{a}_2) = 1 - x.$$

The corresponding Bellman equations for the value functions are:

$$\begin{aligned} \mathcal{V}^\pi(\mathbf{s}_1) &= \gamma [x\mathcal{V}^\pi(\mathbf{s}_2) + (1-x)(0.1\mathcal{V}^\pi(\mathbf{s}_2) + 0.9\mathcal{V}^\pi(\mathbf{s}_1))] , \\ \mathcal{V}^\pi(\mathbf{s}_2) &= 1 + \gamma (0.9\mathcal{V}^\pi(\mathbf{s}_2) + 0.1\mathcal{V}^\pi(\mathbf{s}_1)) . \end{aligned}$$

Thus, substituting $\gamma = 0.5$:

$$\mathcal{V}^\pi(\mathbf{s}_1) = 0.5 [(0.9x + 0.1)\mathcal{V}^\pi(\mathbf{s}_2) + 0.9(1-x)\mathcal{V}^\pi(\mathbf{s}_1)] .$$

Rearranging, we obtain

$$\mathcal{V}^\pi(\mathbf{s}_1) = \frac{0.5(0.9x + 0.1)}{0.55 + 0.45x} \mathcal{V}^\pi(\mathbf{s}_2).$$

Now, substitute into the $\mathbf{s}_2$ equation to obtain:

$$\mathcal{V}^\pi(\mathbf{s}_2) = \frac{11 + 9x}{6 + 4.5x}, \quad \mathcal{V}^\pi(\mathbf{s}_1) = \frac{9x + 1}{6 + 4.5x}.$$

Policy Iteration: we compute PI iterates starting from a policy π_0 that chooses $\mathbf{a}_2$ in $\mathbf{s}_1$, i.e., $x = 0$. Thus,

$$\begin{aligned} \mathcal{V}^{\pi_0}(\mathbf{s}_1) &= \frac{9 \cdot 0 + 1}{6 + 4.5 \cdot 0} = \frac{1}{6} \approx 0.16667, \\ \mathcal{V}^{\pi_0}(\mathbf{s}_2) &= \frac{11 + 9 \cdot 0}{6 + 4.5 \cdot 0} = \frac{11}{6} \approx 1.83333. \end{aligned}$$

Using the value function for π_0 , we compute the action-value functions in $\mathbf{s}_1$:

$$\begin{aligned} \mathcal{Q}^{\pi_0}(\mathbf{s}_1, \mathbf{a}_1) &= 0 + 0.5\mathcal{V}^{\pi_0}(\mathbf{s}_2) \approx 0.5 \cdot 1.83333 = 0.91667, \\ \mathcal{Q}^{\pi_0}(\mathbf{s}_1, \mathbf{a}_2) &= 0 + 0.5(0.1\mathcal{V}^{\pi_0}(\mathbf{s}_2) + 0.9\mathcal{V}^{\pi_0}(\mathbf{s}_1)) \\ &\approx 0.5(0.18333 + 0.15) \approx 0.16667. \end{aligned}$$

Since $\mathcal{Q}^{\pi_0}(\mathbf{s}_1, \mathbf{a}_1) > \mathcal{Q}^{\pi_0}(\mathbf{s}_1, \mathbf{a}_2)$, the improved policy π_1 chooses $\mathbf{a}_1$ (i.e., $x = 1$). Then,

$$\begin{aligned} \mathcal{V}^{\pi_1}(\mathbf{s}_1) &= \frac{9 \cdot 1 + 1}{6 + 4.5 \cdot 1} = \frac{10}{10.5} \approx 0.95238, \\ \mathcal{V}^{\pi_1}(\mathbf{s}_2) &= \frac{11 + 9 \cdot 1}{6 + 4.5 \cdot 1} = \frac{20}{10.5} \approx 1.90476. \end{aligned}$$

Policy π_1 is optimal, therefore PI will halt after one iteration.

Value Iteration: the VI updates are as follows.

$$\begin{aligned} \mathcal{V}_{k+1}(\mathbf{s}_1) &= \max \{0.5\mathcal{V}_k(\mathbf{s}_2), 0.5(0.1\mathcal{V}_k(\mathbf{s}_2) + 0.9\mathcal{V}_k(\mathbf{s}_1))\}, \\ \mathcal{V}_{k+1}(\mathbf{s}_2) &= 1 + 0.5[0.9\mathcal{V}_k(\mathbf{s}_2) + 0.1\mathcal{V}_k(\mathbf{s}_1)]. \end{aligned}$$

In Figure 6.2 we illustrate the differences between the VI and PI algorithms for this MDP, by inspecting the values obtained during the algorithm runs. First, in the solid line, we plot the values of all possible policies in this MDP (by varying x in $[0, 1]$). Note that the two policy iteration steps correspond to the two end points on this line. Next, we plot the VI iterates. We mark in circles the iterates for which the greedy policy (with respect to $\mathcal{V}_k$) is optimal, while a triangle indicates otherwise. We make several important observations:

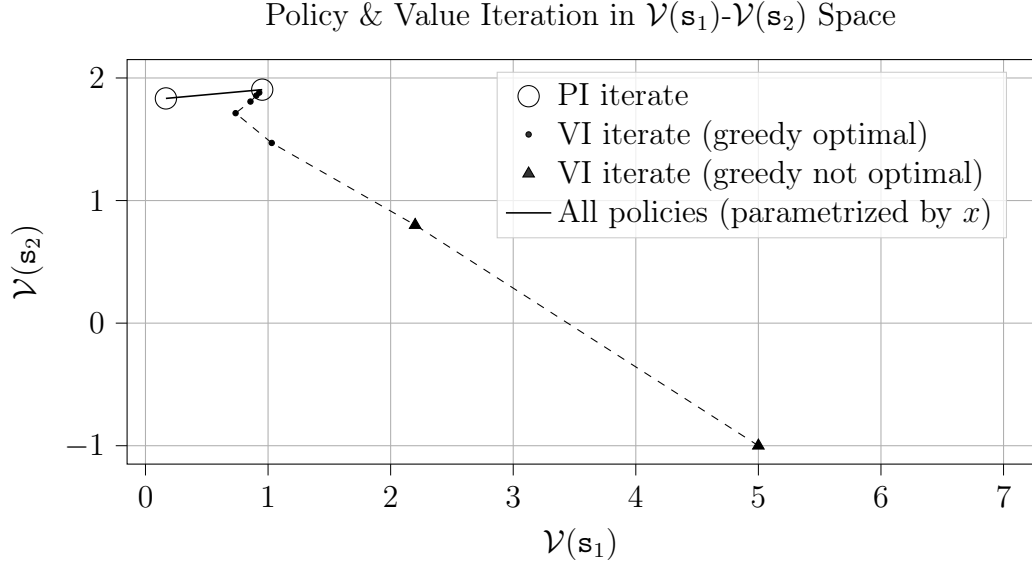


Figure 6.2: Comparison between VI and PI for the MDP in Figure 6.1. PI starts with a policy $x = 0$, and VI is initialized with $\mathcal{V}_0(\mathbf{s}_1) = 5, \mathcal{V}_0(\mathbf{s}_2) = -1$.

- While the values of PI steps correspond to values of actual policies, the VI iterates $\mathcal{V}_k$ do not necessarily correspond to any possible value functions (in the example, the VI iterates do not intersect the solid line). Recall that we are only guaranteed that VI converges to the value of an optimal policy.
- Note that the greedy policy with respect to VI iterate converges to an optimal policy before the VI algorithm converged (in this case, already at $k = 2$). Still, PI converges faster.

6.9 Further Remarks on Policy Iteration

Policy iteration originated in the work of Howard [60]. There has been significant interest in bounding the number of iteration of policy iterations, with a dependency only on the number of states and actions. A simple upper bound is the number of policies, $|\mathcal{A}|^{|\mathcal{S}|}$, since each policy is selected at most once. The work of [107] shows a lower bound of $\Omega(2^{|\mathcal{S}|/2})$ for a special class of policy iteration, where only a single state of all improving states is updated and two actions. The work of [104] shows that if the policy iteration updates with all the improving states (as it is define here) then the number of iterations is at most $O(|\mathcal{A}|^{|\mathcal{S}|}/|\mathcal{S}|)$. The work of [47] shows a n -state and

$\Theta(n)$ action MDP for which the policy iteration requires $\Omega(2^{n/7})$ iterations for the average cost return, and [59] for the discounted return. Surprisingly, for a constant discount factor, the bound on the number of iterations is polynomial [173, 55].

Chapter 7

Episodic Markov Decision Processes

Episodic MDPs provide a natural extension of the standard finite horizon deterministic total cost MDP setting to a stochastic setting. When we view Stochastic Shortest Paths (SSP) as an extension of the graph theoretic notion of shortest paths, we can motivate it by having the edges not completely deterministic, but rather having a probability of ending in a different state. Probably a better view, is to think of the edges as general actions, which induce a distribution over the next state. The goal state can be either a single state or a set of states, both notions would be equivalent.

The SSP problem includes an important sub-category, which is the *episodic MDP* model. In an episodic MDP we are guaranteed to complete the episode in (expected) finite time, regardless of the policy we employ. This will not be true for a general SSP, as some policies might get ‘stuck in a loop’ and never terminate.

Some conditions on the system dynamics and reward function must be imposed for the problem to be well posed (e.g., that a goal state may be reached with probability one). In this chapter we focus on a particular setting; a more extensive treatment can be found in [19].

Chapter Orientation

Main Prerequisites:

- Discounted MDPs (Chapter 6)

Main terminology:

- $\mathcal{S}_G$ set of goal states

- $r_G(\mathbf{s})$ reward at goal state $\mathbf{s}$, assumed to be 0
- τ termination time
- $\mathcal{V}_{ssp}^\pi$ value of state $\mathbf{s}$ under policy π
- $\mathcal{V}_{ssp}^*(\mathbf{s})$ optimal value of state $\mathbf{s}$
- $\mathbf{r}^\pi$ reward vector excluding goal state
- $\mathbf{P}^\pi$ transition matrix excluding goal state

7.1 Definition

We consider a stationary (time-invariant) MDP, with a finite state space $\mathcal{S}$, finite action set $\mathcal{A}$, a transition kernel $\mathbf{P} = \{\mathbf{p}(\mathbf{s}'|\mathbf{s}, \mathbf{a})\}$, and rewards $\mathbf{r}(\mathbf{s}, \mathbf{a})$.

The SSP model is an important class of planning problems, where the time horizon is not set beforehand, but rather the problem continues until a certain event occurs. This event can be defined as reaching some goal state. Let $\mathcal{S}_G \subset \mathcal{S}$ denote the set of *goal states*.

Definition 7.1 (Termination time). *Define the termination time as the random variable*

$$\tau = \inf\{\mathbf{t} \geq 0 : \mathbf{s}_{\mathbf{t}} \in \mathcal{S}_G\},$$

the first time in which a goal state is reached, or infinity otherwise.

We shall make the following assumption on the MDP, which states that for any policy, we will always reach a goal state in finite time.

Assumption 7.1. *The state space is finite, and for any policy π , we have that $\tau < \infty$ with probability 1.*

For the case of positive rewards, Assumption 7.1 guarantees that the agent cannot get ‘stuck in a loop’ and obtain infinite reward.¹ This is similar to the assumption on no negative cycles in deterministic shortest paths. When the rewards are negative, the agent will be driven to reach the goal state as quickly as possible, and in principle, Assumption 7.1 could be relaxed. We will keep it nonetheless, as it will significantly simplify our analysis.

¹The finite state space, by Claim 4.6, guarantees that the *expected* termination time is also finite.

Remark 7.1. *It is easy to find MDPs where Assumption 7.1 does not hold. For example, any MDP with an action that does not change the state, i.e., $p(\mathbf{s}|\mathbf{s}, \mathbf{a}) = 1$ for some non-goal state $\mathbf{s}$ and action $\mathbf{a}$. Realistic problems, however, usually have a predefined hard bound on the length of an episode, and in such cases the MDP can be formulated such that Assumption 7.1 holds.*

The total expected return for Stochastic Shortest Path problem is defined as:

$$\mathcal{V}_{ssp}^{\pi}(\mathbf{s}) = \mathbb{E}^{\pi, \mathbf{s}} \left[\sum_{t=0}^{\tau-1} r(\mathbf{s}_t, \mathbf{a}_t) + r_G(\mathbf{s}_{\tau}) \right].$$

Here $r_G(\mathbf{s})$, $\mathbf{s} \in \mathcal{S}_G$ specifies the reward at goal states. Note that the expectation is taken also over the random length of the run τ .

To simplify the notation, in the following we will assume a single goal state $\mathcal{S}_G = \{\mathbf{s}_G\}$, and that $r_G(\mathbf{s}_G) = 0$.² We therefore write the value as

$$\mathcal{V}_{ssp}^{\pi}(\mathbf{s}) = \begin{cases} \mathbb{E}^{\pi, \mathbf{s}} \left[\sum_{t=0}^{\tau-1} r(\mathbf{s}_t, \mathbf{a}_t) \right], & \mathbf{s} \neq \mathbf{s}_G, \\ 0, & \mathbf{s} = \mathbf{s}_G. \end{cases} \quad (7.1)$$

Our objective is to find a policy that maximizes $\mathcal{V}_{ssp}^{\pi}(\mathbf{s})$. Let π^* be the optimal policy and let $\mathcal{V}_{ssp}^*(\mathbf{s})$ be its value, which is the maximal value from each state $\mathbf{s}$.

7.2 Relationship to other models

We now show that the SSP formulation generalizes several previous models we studied.

7.2.1 Finite Horizon Return

Stochastic shortest path includes, naturally, the finite horizon case. This can be shown by creating a leveled MDP where at each time step we move to the next level and terminate at level T . Specifically, we define a new state space $\mathcal{S}' = \mathcal{S} \times \mathbb{T}$. For any $\mathbf{s} \in \mathcal{S}$, action $\mathbf{a} \in \mathcal{A}$ and time $i \in \mathbb{T}$ we define a transition function $p'((\mathbf{s}', i+1)|(\mathbf{s}, i), \mathbf{a}) = p(\mathbf{s}'|\mathbf{s}, \mathbf{a})$, and goal states $\mathcal{S}_G = \{(\mathbf{s}, T) : \mathbf{s} \in \mathcal{S}\}$. Clearly, Assumption 7.1 is satisfied here.

²This does not reduce the generality of the problem, as we can modify the MDP by adding another state with deterministic reward r_G that transitions to a state in $\mathcal{S}_G$ with zero reward deterministically.

7.2.2 Discounted infinite return

Stochastic shortest path includes also the discounted infinite horizon. To see that, add a new goal state, and from each state with probability $1 - \gamma$ jump to the goal state and terminate. Clearly, Assumption 7.1 is satisfied here too for $\gamma > 0$.

The expected return of a policy would be the same in both models. Specifically, we add a state $\mathbf{s}_G$, such that $\mathbf{p}'(\mathbf{s}_G|\mathbf{s}, \mathbf{a}) = 1 - \gamma$, for any state $\mathbf{s} \in \mathcal{S}$ and action $\mathbf{a} \in \mathcal{A}$ and $\mathbf{p}'(\mathbf{s}'|\mathbf{s}, \mathbf{a}) = \gamma \mathbf{p}(\mathbf{s}'|\mathbf{s}, \mathbf{a})$. The probability that we do not terminate by time $\mathbf{t}$ is exactly $\gamma^{\mathbf{t}}$. Therefore the expected return is $\mathbb{E}[\sum_{t=0}^{\infty} \gamma^t \mathbf{r}(\mathbf{s}_t, \mathbf{a}_t)]$ which is identical to the discounted return.

7.3 Bellman Equations

We now extend the Bellman equations to the SSP setting. We begin by noting that once the goal state has been reached, we do not care anymore about the state transitions, and therefore, without loss of generality, we can consider an MDP where $\mathbf{p}(\mathbf{s}_G|\mathbf{s}_G, \mathbf{a}) = 1$ $\mathbf{r}(\mathbf{s}_G, \mathbf{a}) = 0$ for all $\mathbf{a}$.

Consider a Markov stationary policy π . Define the column vector $\mathbf{r}^\pi = (\mathbf{r}^\pi(\mathbf{s}))_{\mathbf{s} \in \mathcal{S} \setminus \mathbf{s}_G}$ with components $\mathbf{r}^\pi(\mathbf{s}) = \mathbf{r}(\mathbf{s}, \pi(\mathbf{s}))$, and the transition matrix $\mathbf{P}^\pi$ with components $\mathbf{P}^\pi(\mathbf{s}'|\mathbf{s}) = \mathbf{p}(\mathbf{s}'|\mathbf{s}, \pi(\mathbf{s}))$ for all $\mathbf{s}, \mathbf{s}' \in \mathcal{S} \setminus \mathbf{s}_G$. Finally, let $\mathcal{V}_{ssp}^\pi$ denote a column vector with components $\mathcal{V}_{ssp}^\pi(\mathbf{s})$. The next results extends Bellman's equation for a fixed policy to the SSP setting.

Proposition 7.1. *The value function $\mathcal{V}_{ssp}^\pi$ is finite, and is the unique solution to the Bellman equation*

$$\mathcal{V} = \mathbf{r}^\pi + \mathbf{P}^\pi \mathcal{V}, \quad (7.2)$$

i.e.,

$$\mathcal{V} = (I - \mathbf{P}^\pi)^{-1} \mathbf{r}^\pi$$

and $(I - \mathbf{P}^\pi)$ is invertible.

Proof. From Assumption 7.1, every state $\mathbf{s} \neq \mathbf{s}_G$ is transient. For any $i, j \in \mathcal{S} \setminus \mathbf{s}_G$ let $q_{i,j} = \Pr(\mathbf{s}_{\mathbf{t}} = j \text{ for some } \mathbf{t} \geq 1 | \mathbf{s}_0 = i)$. Since state i is transient we have $q_{i,i} < 1$. Let $Z_{i,j}$ be the number of times the trajectory returns to state j when starting from state i . Note that $Z_{i,j}$ is distributed as $\Pr(Z_{i,j} = k) = q_{i,j} q_{j,j}^{(k-1)} (1 - q_{j,j})$ for $k > 0$, and $\Pr(Z_{i,j} = 0) = 1 - q_{i,j}$. Therefore the expected number of visits to state j when starting from state i is $\mathbb{E}[Z_{i,j}] = q_{i,j} (1 - q_{j,j}) \sum_{k \geq 1} k q_{j,j}^{k-1} = \frac{q_{i,j}}{1 - q_{j,j}}$, and is finite.

We can write the value function as

$$\mathcal{V}_{ssp}^\pi(\mathbf{s}) = \sum_{\mathbf{s}' \in \mathcal{S} \setminus \mathbf{s}_G} \mathbb{E}[Z_{\mathbf{s}, \mathbf{s}'}] \mathbf{r}^\pi(\mathbf{s}') < \infty,$$

so the value function is well defined. Now, note that

$$\mathcal{V}_{ssp}^\pi(\mathbf{s}) = \sum_{t=0}^{\infty} \sum_{\mathbf{s}' \in \mathcal{S} \setminus \mathbf{s}_G} \Pr(\mathbf{s}_t = \mathbf{s}' | \mathbf{s}_0 = \mathbf{s}) \mathbf{r}^\pi(\mathbf{s}').$$

Similarly to result for Markov chains, we have that

$$\Pr(\mathbf{s}_t = j | \mathbf{s}_0 = i) = [(\mathbf{P}^\pi)^t]_{ij},$$

therefore,

$$\mathcal{V}_{ssp}^\pi = \sum_{t=0}^{\infty} (\mathbf{P}^\pi)^t \mathbf{r}^\pi.$$

Now, consider the equation (7.2). By unrolling the right hand side and noting that $\lim_{t \rightarrow \infty} (\mathbf{P}^\pi)^t = 0$ because the states are transient we obtain

$$\mathcal{V} = \mathbf{r}^\pi + \mathbf{P}^\pi \mathbf{r}^\pi + (\mathbf{P}^\pi)^2 \mathcal{V} = \dots = \sum_{t=0}^{\infty} (\mathbf{P}^\pi)^t \mathbf{r}^\pi = \mathcal{V}_{ssp}^\pi.$$

We have thus shown that the linear Equation 7.2 has a unique solution $\mathcal{V}_{ssp}^\pi$, and so the claim follows. $\square$

Remark 7.2. *At first sight, it seems that Equation 7.2 is simply Bellman's equation for the discounted setting (6.2), just with $\gamma = 1$. The subtle yet important differences are that Equation 7.2 considers states $\mathcal{S} \setminus \mathbf{s}_G$, and Proposition 7.1 requires Assumption 7.1 to hold, while in the discounted setting the discount factor guarantees that a solution exists for any MDP.*

7.3.1 Value Iteration

Consider the Value Iteration algorithm for SSP, in Algorithm 11.

Theorem 7.2 (Convergence of value iteration). *Let Assumption 7.1 hold. We have $\lim_{n \rightarrow \infty} \mathcal{V}_n = \mathcal{V}_{ssp}^*$ (component-wise).*

Algorithm 11 Value Iteration (for SSP problems)

- 1: **Initialization:** Set $\mathcal{V}_0 = (\mathcal{V}_0(\mathbf{s}))_{\mathbf{s} \in \mathcal{S} \setminus \mathbf{s}_G}$ arbitrarily, $\mathcal{V}_0(\mathbf{s}_G) = 0$.
 - 2: **For** $n = 0, 1, 2, \dots$
 - 3: Set $\mathcal{V}_{n+1}(\mathbf{s}) = \max_{\mathbf{a} \in \mathcal{A}} \left\{ r(\mathbf{s}, \mathbf{a}) + \sum_{\mathbf{s}' \in \mathcal{S} \setminus \mathbf{s}_G} p(\mathbf{s}' | \mathbf{s}, \mathbf{a}) \mathcal{V}_n(\mathbf{s}') \right\}, \quad \forall \mathbf{s} \in \mathcal{S} \setminus \mathbf{s}_G$
-

Proof. Using our previous results on value iteration for the finite-horizon problem, namely the proof of Proposition 6.4, it follows that

$$\mathcal{V}_n(\mathbf{s}) = \max_{\pi} \mathbb{E}^{\pi, \mathbf{s}} \left[\sum_{t=0}^{n-1} R_t + \mathcal{V}_0(\mathbf{s}_n) \right].$$

Since any policy reaches the goal state with probability 1, and after reaching the goal state the agent stays at the goal and receives 0 reward, we can write the optimal value function as

$$\mathcal{V}_{ssp}^*(\mathbf{s}) = \max_{\pi} \mathbb{E}^{\pi, \mathbf{s}} \left[\sum_{t=0}^{\tau} R_t \right] = \max_{\pi} \mathbb{E}^{\pi, \mathbf{s}} \left[\sum_{t=0}^{\infty} R_t \right].$$

It may be seen that

$$\lim_{n \rightarrow \infty} |\mathcal{V}_n(\mathbf{s}) - \mathcal{V}_{ssp}^*(\mathbf{s})| = \lim_{n \rightarrow \infty} \left| \max_{\pi} \mathbb{E}^{\pi, \mathbf{s}} \left[\sum_{t=n}^{\infty} R_t \right] - \mathcal{V}_0(\mathbf{s}_n) \right| = 0,$$

where the last equality is since Assumption 7.1 guarantees that with probability 1 the goal state will be reached, and from that time onwards the agent will receive 0 reward. $\square$

7.3.2 Policy Iteration

The Policy Iteration for the SSP setting is given in Algorithm 12.

The next theorem shows that policy iteration converges to an optimal policy. The proof is the same as in the discounted setting, i.e., Theorem 6.7.

Theorem 7.3 (Convergence of policy iteration for SSP). *The following statements hold:*

1. Each policy π_{k+1} is improving over the previous one π_k , in the sense that $\mathcal{V}^{\pi_{k+1}} \geq \mathcal{V}^{\pi_k}$ (component-wise).

Algorithm 12 Policy Iteration (SSP)

- 1: **Initialization:** choose some stationary policy π_0 .
 - 2: **For** $k = 0, 1, 2, \dots$
 - 3: **Policy Evaluation:** Compute $\mathcal{V}^{\pi_k}$.
 - 4: (For example, use the explicit formula $\mathcal{V}^{\pi_k} = (I - \mathbf{P}^{\pi_k})^{-1} \mathbf{r}^{\pi_k}$)
 - 5: **Policy Improvement:** Compute π_{k+1} , a greedy policy with respect to $\mathcal{V}^{\pi_k}$:
 - 6: $\pi_{k+1}(\mathbf{s}) \in \arg \max_{\mathbf{a} \in \mathcal{A}} \left\{ \mathbf{r}(\mathbf{s}, \mathbf{a}) + \sum_{\mathbf{s}' \in \mathcal{S} \setminus \mathbf{s}_G} \mathbf{p}(\mathbf{s}' | \mathbf{s}, \mathbf{a}) \mathcal{V}^{\pi_k}(\mathbf{s}') \right\}, \forall \mathbf{s} \in \mathcal{S} \setminus \mathbf{s}_G.$
 - 7: **If** $\pi_{k+1} = \pi_k$ (or if $\mathcal{V}^{\pi_k}$ satisfies the optimality equation)
 - 8: **Stop**
-

2. $\mathcal{V}^{\pi_{k+1}} = \mathcal{V}^{\pi_k}$ if and only if π_k is an optimal policy.

3. Consequently, since the number of stationary policies is finite, π_k converges to the optimal policy after a finite number of steps.

7.3.3 Bellman Operators

Let us define the Bellman operators.

Definition 7.2. For a fixed stationary policy $\pi : \mathcal{S} \rightarrow \mathcal{A}$, define the Fixed Policy Dynamic Programming Operator $\mathcal{T}^\pi : \mathbb{R}^{|\mathcal{S}|-1} \rightarrow \mathbb{R}^{|\mathcal{S}|-1}$ as follows: For any $V = (V(\mathbf{s})) \in \mathbb{R}^{|\mathcal{S}|-1}$,

$$(\mathcal{T}^\pi(V))(\mathbf{s}) = \mathbf{r}(\mathbf{s}, \pi(\mathbf{s})) + \sum_{\mathbf{s}' \in \mathcal{S} \setminus \mathbf{s}_G} \mathbf{p}(\mathbf{s}' | \mathbf{s}, \pi(\mathbf{s})) V(\mathbf{s}'), \quad \forall \mathbf{s} \in \mathcal{S} \setminus \mathbf{s}_G.$$

In our column-vector notation, this is equivalent to $\mathcal{T}^\pi(V) = \mathbf{r}^\pi + \mathbf{P}^\pi V$.

Definition 7.3. Define the Dynamic Programming Operator $\mathcal{T}^* : \mathbb{R}^{|\mathcal{S}|-1} \rightarrow \mathbb{R}^{|\mathcal{S}|-1}$ as follows: For any $V = (V(\mathbf{s})) \in \mathbb{R}^{|\mathcal{S}|-1}$,

$$(\mathcal{T}^*(V))(\mathbf{s}) = \max_{\mathbf{a} \in \mathcal{A}} \left\{ \mathbf{r}(\mathbf{s}, \mathbf{a}) + \sum_{\mathbf{s}' \in \mathcal{S} \setminus \mathbf{s}_G} \mathbf{p}(\mathbf{s}' | \mathbf{s}, \mathbf{a}) V(\mathbf{s}') \right\}, \quad \forall \mathbf{s} \in \mathcal{S} \setminus \mathbf{s}_G$$

In the discounted MDP setting, we relied on the discount factor to show that the dynamic programming operators are contractions. Here, we will use Assumption 7.1 to show a weaker contraction-type result.

For any policy π (not necessarily stationary), Assumption 7.1 implies that for $\mathbf{t} = |\mathcal{S}|$, we must have that $\Pr(\mathbf{s}_{\mathbf{t}} = \mathbf{s}_G | \mathbf{s}_0 = \mathbf{s}) > 0$ for all $\mathbf{s} \in \mathcal{S}$, since otherwise,

the Markov chain corresponding to π would have a state that is not communicating with $\mathbf{s}_G$. Let

$$\epsilon = \min_{\pi} \min_{\mathbf{s}} \Pr(\mathbf{s}_t = \mathbf{s}_G | \mathbf{s}_0 = \mathbf{s}),$$

which is well defined since the space of policies is compact and the occupancy probability at time t is a continuous function over the policy space (hence the minimum is attained and we do not need an infimum). Moreover, $\epsilon > 0$. Therefore, we have that for a stationary Markov policy π ,

$$\sum_j [(\mathbf{P}^\pi)^{|\mathcal{S}|}]_{ij} < 1 - \epsilon, \forall i \in |\mathcal{S}| - 1, \quad (7.3)$$

and, for any set of $|\mathcal{S}|$ Markov stationary policies $\pi_1, \dots, \pi_{|\mathcal{S}|}$,

$$\sum_j [\prod_{k=1, \dots, |\mathcal{S}|} \mathbf{P}^{\pi_k}]_{ij} < 1 - \epsilon, \forall i \in |\mathcal{S}| - 1. \quad (7.4)$$

From these results, we have that both $(\mathbf{P}^\pi)^{|\mathcal{S}|}$ and $\prod_{k=1, \dots, |\mathcal{S}|} \mathbf{P}^{\pi_k}$ are $(1 - \epsilon)$ -contractions. We are now ready to show the contraction property of the dynamic programming operators.

Theorem 7.4. *Let Assumption 7.1 hold. Then $(\mathcal{T}^\pi)^{|\mathcal{S}|}$ and $(\mathcal{T}^*)^{|\mathcal{S}|}$ are $(1 - \epsilon)$ -contractions.*

Proof. The proof is similar to the proof of Theorem 6.9, and we only describe the differences. For $\mathcal{T}^\pi$, note that

$$|((\mathcal{T}^\pi)^{|\mathcal{S}|}(\mathcal{V}_1))(\mathbf{s}) - ((\mathcal{T}^\pi)^{|\mathcal{S}|}(\mathcal{V}_2))(\mathbf{s})| = |((\mathbf{P}^\pi)^{|\mathcal{S}|}[\mathcal{V}_1 - \mathcal{V}_2])(\mathbf{s})|,$$

and use the fact that $(\mathbf{P}^\pi)^{|\mathcal{S}|}$ is a $(1 - \epsilon)$ -contraction to proceed as in Theorem 6.9.

For $(\mathcal{T}^*)^{|\mathcal{S}|}$, note that

$$\begin{aligned} ((\mathcal{T}^*)^{|\mathcal{S}|}(\mathcal{V}_1))(\mathbf{s}) &= \max_{\mathbf{a}_0, \dots, \mathbf{a}_{|\mathcal{S}|-1}} \mathbf{r}(\mathbf{s}, \mathbf{a}_0) + \sum_{\mathbf{s}'} \Pr(\mathbf{s}_1 = \mathbf{s}' | \mathbf{s}_0 = \mathbf{s}, \mathbf{a}_0) \mathbf{r}(\mathbf{s}', \mathbf{a}_1) \\ &\quad + \sum_{\mathbf{s}'} \Pr(\mathbf{s}_2 = \mathbf{s}' | \mathbf{s}_0 = \mathbf{s}, \mathbf{a}_0, \mathbf{a}_1) \mathbf{r}(\mathbf{s}', \mathbf{a}_2) + \dots \\ &\quad + \sum_{\mathbf{s}'} \Pr(\mathbf{s}_{|\mathcal{S}|} = \mathbf{s}' | \mathbf{s}_0 = \mathbf{s}, \mathbf{a}_0, \dots, \mathbf{a}_{|\mathcal{S}|-1}) \mathcal{V}_1(\mathbf{s}'). \end{aligned}$$

To show $(\mathcal{T}^*)^{|\mathcal{S}|}(\mathcal{V}_1)(\mathbf{s}) - (\mathcal{T}^*)^{|\mathcal{S}|}(\mathcal{V}_2)(\mathbf{s}) \leq (1 - \epsilon) \|\mathcal{V}_1 - \mathcal{V}_2\|_\infty$: Let $\bar{\mathbf{a}}_0, \dots, \bar{\mathbf{a}}_{|\mathcal{S}|-1}$ denote actions that attains the maximum in $(\mathcal{T}^*)^{|\mathcal{S}|}(\mathcal{V}_1)(\mathbf{s})$. Then proceed similarly as in the proof of Theorem 6.9, and use the fact that $\prod_{k=1, \dots, |\mathcal{S}|} \mathbf{P}^{\pi_k}$ is a $(1 - \epsilon)$ -contraction. $\square$

Remark 7.3. While $\mathcal{T}^\pi$ and $\mathcal{T}^*$ are not necessarily contractions in the sup-norm, they can be shown to be contractions in a weighted sup-norm; see, e.g., [19]. For our discussion here, however, the fact that $(\mathcal{T}^\pi)^{|S|}$ and $(\mathcal{T}^*)^{|S|}$ are contractions will suffice.

7.3.4 Bellman's Optimality Equations

We are now ready to state the optimality equations for the SSP setting.

Theorem 7.5 (Bellman's Optimality Equation for SSP). *The following statements hold:*

1. $\mathcal{V}_{ssp}^*$ is the unique solution of the following set of (nonlinear) equations:

$$\mathcal{V}(\mathbf{s}) = \max_{\mathbf{a} \in \mathcal{A}} \left\{ r(\mathbf{s}, \mathbf{a}) + \sum_{\mathbf{s}' \in \mathcal{S} \setminus \mathbf{s}_G} p(\mathbf{s}' | \mathbf{s}, \mathbf{a}) \mathcal{V}(\mathbf{s}') \right\}, \quad \forall \mathbf{s} \in \mathcal{S} \setminus \mathbf{s}_G. \quad (7.5)$$

2. Any stationary policy π^* that satisfies

$$\pi^*(\mathbf{s}) \in \arg \max_{\mathbf{a} \in \mathcal{A}} \left\{ r(\mathbf{s}, \mathbf{a}) + \sum_{\mathbf{s}' \in \mathcal{S} \setminus \mathbf{s}_G} p(\mathbf{s}' | \mathbf{s}, \mathbf{a}) \mathcal{V}^{\pi^*}(\mathbf{s}') \right\}, \quad \forall \mathbf{s} \in \mathcal{S} \setminus \mathbf{s}_G$$

is an optimal policy (for any initial state $\mathbf{s}_0 \in \mathcal{S}$).

Sketch Proof of Theorem 7.5: The proof is similar to the proof of the discounted setting, but we cannot use Theorem 6.8 directly as we have not shown that $\mathcal{T}^*$ is a contraction. However, a relatively simple extension of the Banach fixed point theorem holds also when $(\mathcal{T}^*)^k$ is a contraction, for some integer k (see, e.g., Theorem 2.4 in [92]). Therefore the proof follows, with Theorem 7.2 replacing Theorem 6.6. $\square$

Chapter 8

Linear Programming Solutions

An alternative approach to value and policy iteration is the linear programming method. Here the optimal control problem is formulated as a linear program (LP), which can be solved efficiently using standard LP solvers. In this chapter we will briefly overview the Linear Programming in general and the Linear Program approach for planning in MDPs. We do not attempt in any way to cover the area of linear programming optimization. There are many excellent books that cover linear programming, for example [70, 32, 169].

Chapter Orientation

Main Prerequisites:

- Finite horizon MDPs (Chapter 5)
- Discounted MDPs (Chapter 6)

8.1 Background

A Linear Program (LP) is an optimization problem that involves minimizing (or maximizing) a linear objective function subject to linear constraints. A standard form of an LP is

$$\text{minimize } b^\top x, \quad \text{subject to } Ax \geq c, x \geq 0, \quad (8.1)$$

where $x = (x_1, x_2, \dots, x_n)^\top$ is a vector of real variables arranged as a column vector. The set of linear constraints defines a *convex polytope* in $\mathbb{R}^n$, namely a closed and

convex set U that is the intersection of a finite number of half-spaces. The set U has a finite number of vertices, which are points that cannot be represented as a convex combination of other points in U . If U is bounded, it equals the convex combination of its vertices. It can be seen that an optimal solution (if finite) will be in one of these vertices.

The LP problem has been extensively studied, and many efficient solvers exist. In 1947, Danzig introduced the Simplex algorithm, which essentially moves greedily along neighboring vertices, and may run in time exponential in the number of constraints. In the 1980's effective algorithms (interior point and others) were introduced which have polynomial (in the number of constraints) time guarantees.

One of the most important notions in a linear program is *duality*, which in many cases allows to gain insight to the solutions of a linear program. The following is the definition of the dual LP.

Duality: The *dual* of the LP in (8.1) is defined as the following LP:

$$\text{maximize } c^\top y, \quad \text{subject to } A^\top y \leq b, y \geq 0. \quad (8.2)$$

The two dual LPs have the same optimal value, and (in many cases) the solution of one can be obtained from that of the other. The common optimal value can be understood by the following computation:

$$\begin{aligned} \min_{x \geq 0, Ax \geq c} b^\top x &= \min_{x \geq 0} \max_{y \geq 0} \{b^\top x + y^\top (c - Ax)\} \\ &= \max_{y \geq 0} \min_{x \geq 0} \{c^\top y + x^\top (b - Ay)\} = \max_{y \geq 0, Ay \leq b} c^\top y, \end{aligned}$$

where the second equality follows by the min-max theorem.

Note: For an LP of the form:

$$\text{minimize } b^\top x, \quad \text{subject to } Ax \geq c,$$

the dual is

$$\text{maximize } c^\top y, \quad \text{subject to } A^\top y = b, y \geq 0.$$

8.2 Linear Program for Finite Horizon MDPs

Our goal is to derive both the primal and the dual linear programs for the finite horizon case. The linear programs for the discounted return is in Section 8.3 and average reward, which is similar in spirit, is omitted.

Representing a policy: The first step is to decide how to represent a policy, then compute its expected return, and finally, maximize over all policies. Given a policy $\pi(\mathbf{a}|\mathbf{s})$ we have seen how to compute its expected return by solving a set of linear equations. (See Lemma 5.5 in Chapter 5.4.2.) However, we are interested in representing a policy in a way which will allow us to maximize over all policies.

The first natural attempt is to write variables which represent a deterministic policy, since we know that there is a deterministic optimal policy. We can have a variable $z(\mathbf{s}, \mathbf{a})$ for each action $\mathbf{a} \in \mathcal{A}$ and state $\mathbf{s} \in \mathcal{S}$. The variable will represent whether in state $\mathbf{s}$ we perform action $\mathbf{a}$. This can be represented by the constraints $z(\mathbf{s}, \mathbf{a}) \in \{0, 1\}$ and $\sum_{\mathbf{a}} z(\mathbf{s}, \mathbf{a}) = 1$ for every $\mathbf{s} \in \mathcal{S}$. Given $z(\mathbf{s}, \mathbf{a})$ we define a policy $\pi(\mathbf{a}|\mathbf{s}) = z(\mathbf{s}, \mathbf{a})$.

One issue that immediately arises is that the Boolean constraints $z(\mathbf{s}, \mathbf{a}) \in \{0, 1\}$ are not linear. We can relax the deterministic policies to stochastic policies and have $z(\mathbf{s}, \mathbf{a}) \geq 0$ and $\sum_{\mathbf{a}} z(\mathbf{s}, \mathbf{a}) = 1$. Given $z(\mathbf{s}, \mathbf{a})$ we still define a policy $\pi(\mathbf{a}|\mathbf{s}) = z(\mathbf{s}, \mathbf{a})$, but now in each state we have a distribution over actions.

The next step is to compute the return of the policy as a *linear function*. The main issue that we have is that in order to compute the return of a policy from state $\mathbf{s}$ we need to also compute the probability that the policy reaches the state $\mathbf{s}$. This probability can be computed by summing over all states $\mathbf{s}'$, the probability of reaching the state $\mathbf{s}'$ times the probability of performing action $\mathbf{a}'$ in state $\mathbf{s}'$, i.e., $q(\mathbf{s}) = \sum_{\mathbf{s}'} q(\mathbf{s}') z(\mathbf{s}', \mathbf{a}') p(\mathbf{s}|\mathbf{a}', \mathbf{s}')$, where $q(\mathbf{s})$ is the probability of reaching state $\mathbf{s}$. The issue that we have is that *both* $q(\cdot)$ and $z(\cdot, \cdot)$ are variables, and therefore the resulting computation is not linear in the variables.

There is a simple fix here, we can define $x(\mathbf{s}, \mathbf{a}) = q(\mathbf{s}) z(\mathbf{s}, \mathbf{a})$, namely, $x(\mathbf{s}, \mathbf{a})$ is the probability of reaching state $\mathbf{s}$ and performing action $\mathbf{a}$. Given $x(\mathbf{s}, \mathbf{a})$ we can define a policy $\pi(\mathbf{a}|\mathbf{s}) = \frac{x(\mathbf{s}, \mathbf{a})}{\sum_{\mathbf{a}'} x(\mathbf{s}, \mathbf{a}')}$. For the finite horizon return, since we are interested in Markov policies, we will add an index for the time and have $x_{\mathbf{t}}(\mathbf{s}, \mathbf{a})$ as the probability that in time $\mathbf{t}$ we are in state $\mathbf{s}$ and perform action $\mathbf{a}$. Recall that in Section 3.2.4 we saw that a sufficient set of parameters is $\Pr_{\mathbf{h}'_{\mathbf{t}-1}}[\mathbf{a}_{\mathbf{t}} = \mathbf{a}, \mathbf{s}_{\mathbf{t}} = \mathbf{s}] = \mathbb{E}_{\mathbf{h}'_{\mathbf{t}-1}} [\mathbb{I}[\mathbf{s}_{\mathbf{t}} = \mathbf{s}, \mathbf{a}_{\mathbf{t}} = \mathbf{a}] | \mathbf{h}'_{\mathbf{t}-1}]$, where $\mathbf{h}'_{\mathbf{t}-1} = (\mathbf{s}_0, \mathbf{a}_0, \dots, \mathbf{s}_{\mathbf{t}-1}, \mathbf{a}_{\mathbf{t}-1})$. We are essentially using those same parameters here.

The variables: For each time $\mathbf{t} \in \mathbb{T} = \{0, \dots, T\}$, state $\mathbf{s}$ and action $\mathbf{a}$ we will have a variable $x_{\mathbf{t}}(\mathbf{s}, \mathbf{a}) \in [0, 1]$ that indicates the probability that at time $\mathbf{t}$ we are at state $\mathbf{s}$ and perform action $\mathbf{a}$. For the terminal states $\mathbf{s}$ we will have a variable $x_T(\mathbf{s}) \in [0, 1]$ that will indicate the probability that we terminate at state $\mathbf{s}$.

The feasibility constraints: Given that we decided on the representation of $x(\mathbf{s}, \mathbf{a})$, we now need to define what is the set of feasible solution for them. The simple constraints are the non-negativity constraints, i.e., $x_{\mathbf{t}}(\mathbf{s}, \mathbf{a}) \geq 0$ and $x_{\mathbf{T}}(\mathbf{s}) \geq 0$.

Our main set of constraints will need to impose the dynamics of the MDP. We can view the feasibility constraints as flow constraints, stating that the probability mass that leaves state $\mathbf{s}$ at time $\mathbf{t}$ is equal by the probability mass of reaching state $\mathbf{s}$ at time $\mathbf{t} - 1$. Formally,

$$\sum_{\mathbf{a}} x_{\mathbf{t}}(\mathbf{s}, \mathbf{a}) = \sum_{\mathbf{s}', \mathbf{a}'} x_{\mathbf{t}-1}(\mathbf{s}', \mathbf{a}') p_{\mathbf{t}-1}(\mathbf{s} | \mathbf{s}', \mathbf{a}').$$

and for terminal states simply

$$x_{\mathbf{T}}(\mathbf{s}) = \sum_{\mathbf{s}', \mathbf{a}'} x_{\mathbf{T}-1}(\mathbf{s}', \mathbf{a}') p_{\mathbf{T}-1}(\mathbf{s} | \mathbf{s}', \mathbf{a}').$$

The objective Given the variables $x_{\mathbf{t}}(\mathbf{s}, \mathbf{a})$ and $x_{\mathbf{T}}(\mathbf{s})$ we can write the expected return, which we would like to maximize, as

$$\sum_{\mathbf{t}, \mathbf{s}, \mathbf{a}} r_{\mathbf{t}}(\mathbf{s}, \mathbf{a}) x_{\mathbf{t}}(\mathbf{s}, \mathbf{a}) + \sum_{\mathbf{s}} r_{\mathbf{T}}(\mathbf{s}) x_{\mathbf{T}}(\mathbf{s})$$

The main observation is that the expected objective depends only on the probabilities of being at time $\mathbf{t}$ in state $\mathbf{s}$ and performing action $\mathbf{a}$.

Primal LP: Combining the above we derive the following linear program. (Recall that $\mathcal{S}_{\mathbf{t}} \subset \mathcal{S}$ is the set of possible states at time $\mathbf{t} \in \mathbb{T}$.)

$$\max_{x_t(\mathbf{s}, \mathbf{a}), x_T(\mathbf{s})} \sum_{\mathbf{t}, \mathbf{s}, \mathbf{a}} r_t(\mathbf{s}, \mathbf{a}) x_t(\mathbf{s}, \mathbf{a}) + \sum_{\mathbf{s}} r_T(\mathbf{s}) x_T(\mathbf{s})$$

such that

$$\sum_{\mathbf{a}} x_t(\mathbf{s}, \mathbf{a}) \leq \sum_{\mathbf{s}', \mathbf{a}'} x_{t-1}(\mathbf{s}', \mathbf{a}') p_{t-1}(\mathbf{s} | \mathbf{s}' \mathbf{a}'). \quad \forall \mathbf{s} \in \mathcal{S}_t, t \in \mathbb{T}$$

$$x_T(\mathbf{s}) \leq \sum_{\mathbf{s}', \mathbf{a}'} x_{T-1}(\mathbf{s}', \mathbf{a}') p_{T-1}(\mathbf{s} | \mathbf{s}' \mathbf{a}') \quad \forall \mathbf{s} \in \mathcal{S}_T$$

$$x_t(\mathbf{s}, \mathbf{a}) \geq 0 \quad \forall \mathbf{s} \in \mathcal{S}_t, \mathbf{a} \in \mathcal{A}, t \in \{0, \dots, T-1\}$$

$$\sum_{\mathbf{a}} x_0(\mathbf{s}_0, \mathbf{a}) = 1$$

$$x_0(\mathbf{s}, \mathbf{a}) = 0, \quad \forall \mathbf{s} \in \mathcal{S}_0, \mathbf{s} \neq \mathbf{s}_0$$

Remarks: First, note that we replaced the flows equalities with inequalities. In the optimal solution, since we are maximizing, and since the rewards are non-negative, those flow inequalities will become equalities.

Second, note that we do not explicitly upper bound $x_t(\mathbf{s}, \mathbf{a}) \leq 1$, although it should clearly hold in any feasible solution. While we do not impose it explicitly, this is implicit in the linear program. To observe this, let $\Phi(t) = \sum_{\mathbf{s}, \mathbf{a}} x_t(\mathbf{s}, \mathbf{a})$. From the initial conditions we have that $\Phi(0) = 1$. When we sum the flow condition (first inequality) over the states we have that $\Phi(t) \leq \Phi(t-1)$. This implies that $\Phi(t) \leq 1$. Again, in the optimal solution we will maximize those values and we will have $\Phi(t) = \Phi(t-1)$.

Dual LP: Given the primal linear program we can derive the dual linear program.

$$\min_{z_t(\mathbf{s})} z_0(\mathbf{s}_0)$$

such that

$$z_T(\mathbf{s}) = r_T(\mathbf{s}) \quad \forall \mathbf{s} \in \mathcal{S}_T$$

$$z_t(\mathbf{s}) \geq r_t(\mathbf{s}, \mathbf{a}) + \sum_{\mathbf{s}'} z_{t+1}(\mathbf{s}') p_t(\mathbf{s}' | \mathbf{s}, \mathbf{a}), \quad \forall \mathbf{s} \in \mathcal{S}_t, \mathbf{a} \in \mathcal{A}, t \in \mathbb{T},$$

$$z_t(\mathbf{s}) \geq 0 \quad \forall \mathbf{s} \in \mathcal{S}_t, t \in \mathbb{T}$$

One can identify the dual random variables $z_t(\mathbf{s})$ with the optimal value function $\mathcal{V}_t^*(\mathbf{s})$.

The logic for the dual LP is the following. The constraints at time T are setting the terminal cost for each node, and this initializes the optimal value function correctly for time T . The constraints at time t are lower bounding the value function at state $\mathbf{s}$ given the value function at times $t + 1$. The Bellman optimality equations require that the value function would be the maximum value over the actions, i.e., $\mathcal{V}_t^*(\mathbf{s}) = \max_{\mathbf{a}} \{r_t(\mathbf{s}, \mathbf{a}) + \sum_{\mathbf{s}'} \mathcal{V}_{t+1}^*(\mathbf{s}') p_t(\mathbf{s}'|\mathbf{s}, \mathbf{a})\}$. We impose in the LP only an inequality, but we will reach the equality in the optimal solution, since we will perform a minimization over $z_t(\cdot)$. The minimization is technically done only over the initial state $\mathbf{s}_0$ and initial time $t = 0$. However, this minimization implies a minimization of $z_t(\mathbf{s})$ for any state $\mathbf{s}$ and time t .

At the optimal solution of the dual linear program one can show that we have

$$z_t(\mathbf{s}) = \max_{\mathbf{a}} \{r_t(\mathbf{s}, \mathbf{a}) + \sum_{\mathbf{s}'} z_{t+1}(\mathbf{s}') p_t(\mathbf{s}'|\mathbf{s}, \mathbf{a})\}, \quad \forall \mathbf{s} \in \mathcal{S}_t, t \in \mathbb{T},$$

which are the familiar Bellman optimality equations.

8.3 Linear Program for Discounted MDPs

In this section we will use linear programming to derive the optimal policy for discounted return. The resulting program would be very similar to that of the finite horizon, however, we do need to make a few changes to accommodate for the introduction of a discount factor γ and the fact that the horizon is infinite (and the policy is time-homogeneous). Again, we will see that both the primal and dual programs will play an important part in defining the optimal policy. A slight difference from the finite horizon return case, is that for the discounted return case the optimal policy does not depend on the start state, so we compute the optimal policy independent from the start state.

We will start with the primal linear program, which will compute the optimal policy. In the finite horizon return we had for each time t state $\mathbf{s}$ and action $\mathbf{a}$ a variable $x_t(\mathbf{s}, \mathbf{a})$. In the discounted return case we will consider stationary policies, so we will drop the dependency on the time t . In addition we will replace the probabilities by *discounted fraction of time*. Namely, for each state $\mathbf{s}$ and action $\mathbf{a}$ we will have a variable $x(\mathbf{s}, \mathbf{a})$ that will indicate the discounted fraction of time we are at state $\mathbf{s}$ and perform action $\mathbf{a}$.

To better understand what we mean by the discounted fraction of time consider a fixed stationary policy π and a trajectory $(\mathbf{s}_0, \dots)$ generated by π . Define the discounted time of state-action $(\mathbf{s}, \mathbf{a})$ in the trajectory as $X^\pi(\mathbf{s}, \mathbf{a}) = \sum_t \gamma^t \mathbb{I}(\mathbf{s}_t =$

$\mathbf{s}, \mathbf{a}_t = \mathbf{a}$), which is a random variable. We are interested in $x^\pi(\mathbf{s}, \mathbf{a}) = \mathbb{E}[X^\pi(\mathbf{s}, \mathbf{a})]$ which is the expected discounted fraction of time policy π is in state $\mathbf{s}$ and performs action $\mathbf{a}$. This discounted fraction of time would be very handy in defining the objective as well as defining the flow constraints.

Given the discounted fraction of time values $x(\mathbf{s}, \mathbf{a})$ for every $\mathbf{s} \in \mathcal{S}$ and $\mathbf{a} \in \mathcal{A}$ we essentially have all the information we need. First, the discounted fraction of time that we are in a state $\mathbf{s} \in \mathcal{S}$ is simply $x(\mathbf{s}) = \sum_{\mathbf{a} \in \mathcal{A}} x(\mathbf{s}, \mathbf{a})$. We can recover a policy that generates those discounted fraction of times by setting,

$$\pi(\mathbf{a}|\mathbf{s}) = \frac{x(\mathbf{s}, \mathbf{a})}{\sum_{\mathbf{a}' \in \mathcal{A}} x(\mathbf{s}, \mathbf{a}')}.$$

All this is under the assumption that the discounted fraction of time values $x(\mathbf{s}, \mathbf{a})$ were generated by some policy. However, in the linear program we will need to guarantee that indeed those values are feasible, namely, can be generated by the given dynamics. For this we will introduce feasibility constraints.

The feasibility constraints: As in the finite horizon case, our main constraint will be flow constraints, stating that the discounted fraction of time we reach state $\mathbf{s}$ equals the discounted fraction of time we exit it, times the discounted factor. (We are multiplying by the discount factor since we are moving one step to the future.) Technically, it will be sufficient to use only an upper bound, and in the optimal solution, maximizing the expected return, there will be an equality. Formally, for $\mathbf{s} \in \mathcal{S}$,

$$\sum_{\mathbf{a}} x(\mathbf{s}, \mathbf{a}) \leq \gamma \sum_{\mathbf{s}', \mathbf{a}'} x(\mathbf{s}', \mathbf{a}') p(\mathbf{s}|\mathbf{s}'\mathbf{a}') + \mathbb{I}(\mathbf{s} = \mathbf{s}_0)$$

For the initial state $\mathbf{s}_0$ we add 1 for the incoming flow, since initially we start in it, and not reach it from another state.

At first sight the dependency on the initial state $\mathbf{s}_0$ might be surprising, since the optimal policy does not depend on the initial state. However, we would be maximizing the discounted return from the initial state, i.e., the optimal value function $\mathcal{V}^*(\mathbf{s}_0)$. Clearly, the optimal value function does depend on the state, which will result in a dependency on the initial state in the LP.

Let us verify that indeed the constraints show that when we sum over all states and actions we get the correct value of $1/(1-\gamma)$, which equals the infinite discounted sum $\sum_t \gamma^t$. If we sum the inequalities over all states, we have

$$\sum_{\mathbf{s}, \mathbf{a}} x(\mathbf{s}, \mathbf{a}) \leq \gamma \sum_{\mathbf{s}', \mathbf{a}'} x(\mathbf{s}', \mathbf{a}') \sum_{\mathbf{s}} p(\mathbf{s}|\mathbf{s}'\mathbf{a}') = \gamma \sum_{\mathbf{s}', \mathbf{a}'} x(\mathbf{s}', \mathbf{a}') + 1,$$

which implies that $\sum_{\mathbf{s}, \mathbf{a}} x(\mathbf{s}, \mathbf{a}) \leq 1/(1 - \gamma)$, as we should expect. Namely, in each time we are in some state, therefore, the sum over states should be $\sum_t \gamma^t = 1/(1 - \gamma)$.

The objective: The discounted return, which we would like to maximize, is $\mathbb{E}[\sum_t \gamma^t \mathbf{r}(\mathbf{s}_t, \mathbf{a}_t)]$. (Again, note that the expectation implicitly depends on the initial state $\mathbf{s}_0$.) We can regroup the sum by state and action and have

$$\sum_{\mathbf{s}, \mathbf{a}} \mathbb{E} \left[\sum_t \gamma^t \mathbf{r}(\mathbf{s}_t, \mathbf{a}_t) \mathbb{I}(\mathbf{s}_t = \mathbf{s}, \mathbf{a}_t = \mathbf{a}) \right],$$

which is equivalent to

$$\sum_{\mathbf{s}, \mathbf{a}} \mathbf{r}(\mathbf{s}, \mathbf{a}) \mathbb{E} \left[\sum_t \gamma^t \mathbb{I}(\mathbf{s}_t = \mathbf{s}, \mathbf{a}_t = \mathbf{a}) \right].$$

Since our variable are $x(\mathbf{s}, \mathbf{a}) = \mathbb{E}[\sum_t \gamma^t \mathbb{I}(\mathbf{s}_t = \mathbf{s}, \mathbf{a}_t = \mathbf{a})]$, the expected return would be

$$\sum_{\mathbf{s}, \mathbf{a}} \mathbf{r}(\mathbf{s}, \mathbf{a}) x(\mathbf{s}, \mathbf{a})$$

Primal LP: Combining all the above, the resulting linear program is the following.

$$\max_{x(\mathbf{s}, \mathbf{a})} \sum_{\mathbf{s}, \mathbf{a}} \mathbf{r}(\mathbf{s}, \mathbf{a}) x(\mathbf{s}, \mathbf{a})$$

such that

$$\begin{aligned} \sum_{\mathbf{a}} x(\mathbf{s}, \mathbf{a}) &\leq \gamma \sum_{\mathbf{s}', \mathbf{a}'} x(\mathbf{s}', \mathbf{a}') p(\mathbf{s} | \mathbf{s}' \mathbf{a}') + \mathbb{I}(\mathbf{s} = \mathbf{s}_0) & \forall \mathbf{s} \in \mathcal{S}, \mathbf{a} \in \mathcal{A}, \\ x(\mathbf{s}, \mathbf{a}) &\geq 0 & \forall \mathbf{s} \in \mathcal{S}, \mathbf{a} \in \mathcal{A}. \end{aligned}$$

Dual LP: Given the primal linear program we can derive the dual linear program.

$$\min_{z(\mathbf{s})} z(\mathbf{s}_0)$$

such that

$$\begin{aligned} z(\mathbf{s}) &\geq \mathbf{r}(\mathbf{s}, \mathbf{a}) + \gamma \sum_{\mathbf{s}'} z(\mathbf{s}') p(\mathbf{s}' | \mathbf{s}, \mathbf{a}), & \forall \mathbf{s} \in \mathcal{S}, \mathbf{a} \in \mathcal{A}, \\ z(\mathbf{s}) &\geq 0 & \forall \mathbf{s} \in \mathcal{S}. \end{aligned}$$

One can identify the dual random variables $z(\mathbf{s})$ with the optimal value function $\mathcal{V}^*(\mathbf{s})$.

To gain insight to the dual LP, recall the Bellman optimality equations: $\mathcal{V}^*(\mathbf{s}) = \max_{\mathbf{a}} \{ \mathbf{r}(\mathbf{s}, \mathbf{a}) + \gamma \sum_{\mathbf{s}'} \mathcal{V}^*(\mathbf{s}') \mathbf{p}(\mathbf{s}' | \mathbf{s}, \mathbf{a}) \}$. As in the finite horizon case, we impose only inequalities. The minimization over $z(\mathbf{s}_0)$ guarantees that the inequalities would be tight for at least one action. Note that we can minimize any $z(\mathbf{s})$, since the value function of the discounted return does not depend on the initial state.

At the optimal solution of the dual linear program one can show that we have

$$z(\mathbf{s}) = \max_{\mathbf{a}} \left\{ \mathbf{r}(\mathbf{s}, \mathbf{a}) + \gamma \sum_{\mathbf{s}'} z(\mathbf{s}') \mathbf{p}(\mathbf{s}' | \mathbf{s}, \mathbf{a}) \right\}, \quad \forall \mathbf{s} \in \mathcal{S},$$

which are the familiar Bellman optimality equations.

We conclude this section by extending Example 6.1 to include the linear programming solution.

Example 8.1 (Example 6.1 cont'd). *Consider the 2-state MDP of Example 6.1. The constraints in the dual linear program here are:*

$$\begin{aligned} z(\mathbf{s}_1) &\geq 0 + 0.5 z(\mathbf{s}_2), \\ z(\mathbf{s}_1) &\geq 0 + 0.5 [0.1 z(\mathbf{s}_2) + 0.9 z(\mathbf{s}_1)], \\ z(\mathbf{s}_2) &\geq 1 + 0.5 [0.9 z(\mathbf{s}_2) + 0.1 z(\mathbf{s}_1)], \end{aligned}$$

which can be simplified to:

$$\begin{aligned} z(\mathbf{s}_1) &\geq 0.5 z(\mathbf{s}_2), \\ z(\mathbf{s}_2) &\leq 11 z(\mathbf{s}_1), \\ z(\mathbf{s}_2) &\geq \frac{1}{11} z(\mathbf{s}_1) + \frac{20}{11}, \end{aligned}$$

defining the feasible region in $(z(\mathbf{s}_1), z(\mathbf{s}_2))$ -space (the polytope). Note that the second constraint is obviated by the first constraint. The unique intersection of the first-and third-constraint lines is exactly

$$z(\mathbf{s}_1) = \frac{20}{21} \approx 0.95, \quad z(\mathbf{s}_2) = 2 z(\mathbf{s}_1) = \frac{40}{21} \approx 1.90,$$

which is exactly the optimal $(\mathcal{V}^*(\mathbf{s}_1), \mathcal{V}^*(\mathbf{s}_2))$ pair. Figure 8.1 illustrates the polytope, the optimal value function, and compares with policy iteration. Note in particular that the only intersection between any feasible $z(\mathbf{s})$ and $\mathcal{V}^\pi(\mathbf{s})$ is at the optimal point: $\mathcal{V}^*(\mathbf{s})$ is both a lower bound for $z(\mathbf{s})$ and an upper bound for $\mathcal{V}^\pi(\mathbf{s})$.

MDP Value Space: All Policies vs. LP Polytope

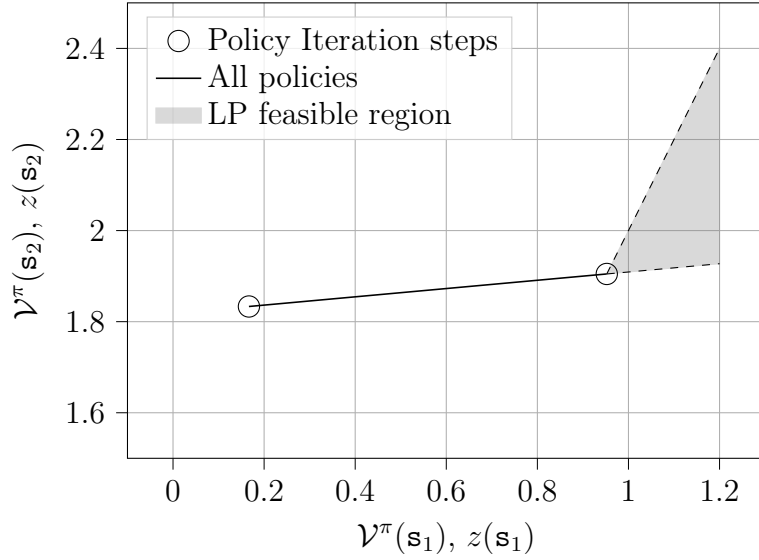


Figure 8.1: Comparison between LP and PI for the MDP in Figure 6.1. We plot the polytope for $z(s_1), z(s_2)$ on the same plot of the values $V^\pi(s_1), V^\pi(s_2)$, for every possible policy π . Note that the polytope intersects with the values only at the optimal policy $V^*(s_1), V^*(s_2)$.

8.4 Further Notes

A rigorous treatment of the relationship between linear programming and MDP planning is given in [121]. A more reinforcement learning specific approach can be found in [40, 41].

There are works that use a linear programming approach to derive strongly polynomial algorithms. Specifically, for deterministic MDPs we have polynomial time algorithms which are based on linear programming [101, 119]. An important application of the LP approach is in constrained MDPs [3].

Chapter 9

Preface to the Learning Chapters

Up until now, we have discussed *planning* under a known model, such as the MDP. Indeed, the algorithms we discussed made extensive use of the model, such as iterating over all the states, actions, and transitions. In the remainder of this book, we shall tackle the *learning* setting – how to make decisions when the model is not known in advance, or too large for iterating over it, precluding the use of the planning methods described earlier. Before diving in, however, we shall spend some time on defining the various approaches to modeling a learning problem. In the next chapters, we will rigorously cover some of these approaches. This chapter, similarly to Chapter 2, is quite different than the rest of the book, as it discusses epistemological issues more than anything else.

In the machine learning literature, perhaps the most iconic learning problem is *supervised learning*, where we are given a training dataset of N samples, $X_1, X_2, \dots, X_N$, sampled i.i.d. from some distribution, and corresponding labels $Y_1, \dots, Y_N$, generated by some procedure. We can think of Y_i as the supervisor’s answer to the question “what to do when the input is X_i ?”. The learning problem, then, is to use this data to find some function $Y = f(X)$, such that when given a new sample X' from the data distribution (not necessarily in the dataset), the output of $f(X')$ will be similar to the corresponding label Y' (which is not known to us). A successful machine learning algorithm therefore exhibits *generalization* to samples outside its training set.

Measuring the success of a supervised learning algorithm in practice is straightforward – by measuring the average error it makes on a test set sampled from the data distribution. The Probably Approximately Correct (PAC) framework is a common framework for providing theoretical guarantees for a learning algorithm. For a given algorithm, a PAC sample complexity result, also known as an “upper bound”,

would compute the expected error (ϵ) that can be guaranteed with a given error probability (δ) for a given sample size (N). A PAC lower bound is an impossibility result arguing that no algorithm can learn with an expected error less than ϵ with success probability more than $1 - \delta$ with fewer than N samples. PAC results are therefore important for understanding how efficient a learning algorithm is (e.g., how the expected error and failure probability reduce with N).

In reinforcement learning, we are interested in learning how to solve sequential decision problems. We shall now discuss the main learning model, why it is useful, how to measure success and provide guarantees, and also briefly mention some alternative learning models that are outside the scope of this book.

9.1 Interacting Online with an Unknown MDP

The common RL model is inspired by models of behavioral psychology, where an agent (e.g., a rat) needs to learn some desired behavior (e.g., navigate a maze), by reinforcing the desired behavior with some reward (e.g., giving the rat food upon exiting the maze). The key distinction with supervised learning, is that the agent is not given direct supervision about its actions (i.e., how to navigate the maze), but must understand what actions are good only from the reward signal.

To a great extent, much of the RL literature implements this model as interacting with an MDP whose parameters are unknown. As depicted in Figure 9.1, at each time step $t = 1, 2, \dots, N$, the agent can observe the current state $\mathbf{s}_t$, take an action $\mathbf{a}_t$, and subsequently obtain an observation from the environment (MDP) about the current reward $r(\mathbf{s}_t, \mathbf{a}_t)$ and next state $\mathbf{s}_{t+1} \sim p(\cdot | \mathbf{s}_t, \mathbf{a}_t)$.

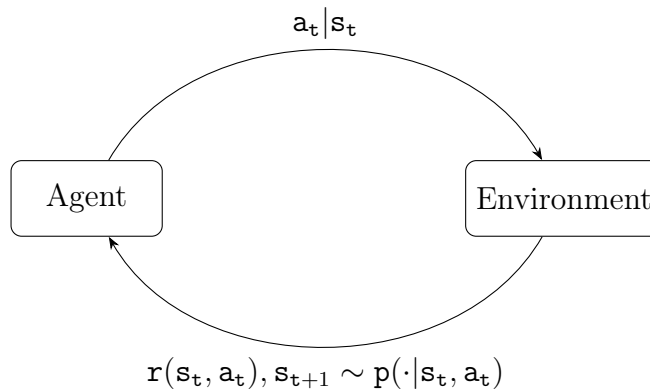


Figure 9.1: Interaction of an agent with the environment.

We can think of the N training samples in RL as tuples $(\mathbf{s}_t, \mathbf{a}_t, \mathbf{r}(\mathbf{s}_t, \mathbf{a}_t), \mathbf{s}_{t+1})_{t=0}^N$, and the goal of the learning agent is to eventually (for N large enough) perform well in the environment, that is, learn a policy for the MDP that is near optimal. Note that in this learning model, the agent cannot make any explicit use of the MDP model (rewards and transitions), but only obtain samples of them in the states it visited.

The reader may wonder – why is such a learning model useful at all? After all, it’s quite hard to imagine real world problems as in Figure 9.1, where the agent starts out without *any* knowledge about the world and must learn everything only from the reinforcement signal. As it turns out, RL algorithms essentially learn to solve MDPs without requiring an explicit MDP model, and can therefore be applied even to very large MDPs, for which the planning methods in the previous chapters do not apply. The important insight is that if we have an RL algorithm, and a *simulator* of the MDP, capable of generating $\mathbf{r}(\mathbf{s}_t, \mathbf{a}_t)$ and $\mathbf{s}_{t+1} \sim \mathbf{p}(\cdot | \mathbf{s}_t, \mathbf{a}_t)$, then we can run the RL algorithm with the simulator replacing the real environment. To date, almost all RL successes in game playing, control, and decision making have been obtained under this setting.

Another motivation for this learning model comes from the field of adaptive control [4]. If the agent has an imperfect model of the MDP (what we called epistemic uncertainty in Chapter 2), any policy it computes using it may be suboptimal. To overcome this error, the agent can try and correct its model of the MDP or adapt its policy during interaction with the real environment. Indeed, RL is very much related to adaptive optimal control [150], which studies a similar problem.

In contrast with the supervised learning model, where measuring success was straightforward, we shall see that defining a goodness measure for an RL agent is more involved, and we shall discuss some prominent ideas in the literature.

Regret, PAC, and asymptotic guarantees. Consider that we evaluate the agent based the cumulative reward it can obtain in the MDP. Naturally, we should expect that with enough interactions with the environment, any reasonable RL algorithm should converge to obtaining as much reward as an optimal policy would. That is, as the number of training samples N goes to infinity, the value of the agent’s policy should converge to the optimal value function $\mathcal{V}^*$. Such an asymptotic result will guarantee that the algorithm is *fundamentally sound*, and does not make any systematic errors.

To compare the learning efficiency of different RL algorithms, it is more informative to look at finite-sample guarantees. A direct extension of the PAC framework to the RL setting could be: bound the sub-optimality ϵ of the value of the learned policy

with respect to an optimal policy, after taking N samples from the environment, with probability $1 - \delta$ (the probability is with respect to the stochasticity of the MDP transitions). A corresponding practical evaluation is to first train the agent for N time steps, and then evaluate the learned policy.

The problem with the PAC approach is that we only care about the reward collected *after* learning, but not the reward obtained *during* learning. For some problems, such as online marketing or finance, we may want to maximize revenue *all throughout* learning. A useful measure for this is the *regret*,

$$Regret(N) = \sum_{t=0}^N \mathbf{r}_t^* - \sum_{t=0}^N \mathbf{r}(\mathbf{s}_t, \mathbf{a}_t),$$

which measures the difference between the cumulative reward the agent obtained on the N samples and the sum of rewards that an optimal policy would have obtained (with the same amount of time steps N), denoted here as $\mathbf{r}_t^*$. Any algorithm that converges to an optimal policy would have $\frac{1}{N}Regret(N) \rightarrow 0$, but we can also compare algorithms by the *rate* that the average regret decreases.

Interestingly, for an algorithm to be optimal in terms of regret, it must balance between *exploration* – taking actions that yield information about the MDP, and *exploitation* – taking actions that simply yield high reward. This is different from PAC, where the agent should in principle devote all the N samples for exploration.

9.1.1 Alternative Learning Models

Humans are perhaps the best example we have for agents learning general, well performing decision making. Even though the common RL model was inspired from behavioral psychology, its specific mathematical formulation is much more limited than the general decision making we may imagine as humans. In the following, we discuss some limitations of the RL model, and alternative decision making formulations that address them. These models are outside the scope of this book.

The challenges of learning from rewards (revisited) We have already discussed the difficulty of specifying decision making problems using reward functions in the preface to the planning, Chapter 2. In the RL model, we assume that we can evaluate the observed interaction of the agent with environment by scalar rewards. This is easy if we have an MDP model or simulator, but often difficult otherwise. For example, if we want to use RL to automatically train a robot to perform some task (e.g., fold a piece of cloth), we need to write a reward function that can evaluate

whether the cloth was folded or not – a difficult task in itself. We can also directly query a human expert for evaluating the agent. However, it turns out that humans find it easier to rank different interactions than to associate their performance with a scalar reward. The field of RL from Human Feedback (RLHF) studies such evaluation models, and has been instrumental for tuning chatbots using RL [117]. It is also important to emphasize that in the RL model defined above, the agent *is only concerned with maximizing reward*, leading to behavior that can be very different from human decision making. As argued by Lake et al. [91] in the context of video games, humans can easily imagine how to play the game differently, e.g., how to lose the game as quickly as possible, or how to achieve certain goals, but such behaviors are outside the desiderata of the standard RL problem; extensions of the RL problem include more general reward evaluations such as ‘obtain a reward higher than x ’ [145, 31], or goal-based formulations [66], and a key question is how to train agents that *generalize* to new goals.

Bayesian vs Frequentist The RL model described above is *frequentist*¹ in nature – the agent interacts with a fixed, but unknown, MDP. An alternative paradigm is *Bayesian RL* [50], where we assume some prior distribution over possible MDPs that the agent can interact with, and update the agent’s belief about the “real” (but unknown) MDP using the data samples. The Bayesian prior is a convenient method to specify prior knowledge that the agent may have before learning, and the Bayesian formulation offers a principled solution to the exploration-exploitation tradeoff – the agent can calculate in advance how much information any action would yield (i.e., how it would affect the belief).

Offline reinforcement learning A common situation that is often encountered in practice is that of offline RL. In its purest form, in offline learning, the agent is given recorded trajectories from some historical policy and needs to learn an optimal policy based on them. When the historical data is created by a strong agent, perhaps even optimal, one often tries to use *behavioral cloning* to learn a policy that will be as similar as possible to the one that created the data. However, the data itself may be sampled from a suboptimal policy and in which case, since the agent cannot explore

¹The Bayesian and Frequentist approaches are two fundamental schools of thought in statistics that differ in how they interpret probability and approach inference. In frequentist inference, parameters are considered fixed but unknown quantities, and inference is made by examining how an estimator would perform in repeated sampling. Bayesian inference treats parameters as sampled from a prior distribution, and calculates the posterior parameter probability after observing data, using Bayes rule.

further, the variance of the learned policy may be large even when the offline data is huge. We note that the pure offline and pure online RL settings are two extremes. Often, one has both: plenty of offline data and the opportunity to interact with an environment in an online manner.

Generalization to changes in the MDP A stark difference between RL and supervised learning is what we mean by generalization. While in supervised learning we evaluate the agent’s decision making on test problems unseen during training, in the RL problem described above the agent is trained and tested *on the same MDP*. At test time, the agent may encounter states that it has not visited during training and in this sense must generalize, but the main focus of the learning problem is how to take actions in the MDP that eventually lead to learning a good policy.

Several alternative learning paradigms explored generalization in sequential decision making. In Meta RL [13], the agent can interact with several *training MDPs* during learning, but is then tested on a similar, yet unseen, *test MDP*. If the training and test MDPs are sampled from some distribution, meta RL relates to Bayesian RL, where the prior is the training MDP distribution, and PAC-style guarantees can be provided on how many training MDPs are required to obtain near Bayes-optimal performance [155]. A related paradigm is contextual MDPs, where repeated interactions with several MDPs are considered at test time, and regret bounds can capture the tradeoff between identifying the MDPs and maximizing rewards [54]. More generally, *transfer learning* in RL concerns how to transfer knowledge between different decision making problems [156, 81]. It is also possible to search for policies that work well across many different MDPs, and are therefore robust enough to generalize to changes in the MDP. One approach, commonly termed *domain randomization*, trains a single policy on an ensemble of different MDPs [159]. Another approach optimizes a policy for the worst case MDP in some set, based on the robust MDP formulation [115]. Yet another learning setting is lifelong RL, where an agent interacts with an MDP that gradually changes over time [79].

9.1.2 What to Learn in RL?

In the next chapters we shall explore several approaches to the RL problem. Relating to the underlying MDP model, we shall apply a learning-based approach to different MDP-related quantities.

A straightforward approach is *model-based*. In this approach, the agent learns the rewards and transitions of the MDP, and uses them to compute a policy using planning algorithms. A key question here is how to take actions that would guarantee

that the agent sufficiently explores all states of the MDP.

An alternative approach is *model-free*. Interestingly, the agent can learn optimal behavior without explicitly estimating the MDP parameters. This can be done by directly estimating either the value function, or the optimal policy. In particular, this approach will allow us to use function approximation to generalize the learned value or policy to states that the agent has not seen during training, potentially allowing us to handle MDPs with large state spaces.

Chapter 10

Reinforcement Learning: Model Based

Until now we looked at planning problems, where we are given a complete model of the MDP, and the goal is to either evaluate a given policy or compute the optimal policy. In this chapter we will start looking at learning problems, where we need to learn from interaction. This chapter will concentrate on *model based* learning, where the main goal is to learn an accurate model of the MDP and use it. In following chapters we will look at *model free* learning, where we learn a value function or a policy without recovering the actual underlying model.

Chapter Orientation

Main Prerequisites:

- Discounted MDPs (Chapter 6)

Main terminology:

- $R_t(\mathbf{s}, \mathbf{a})$ random reward for state $\mathbf{s}$ and action $\mathbf{a}$ at time t
- $R(\mathbf{s}, \mathbf{a})$ random reward for state $\mathbf{s}$ and action $\mathbf{a}$ (stationary model)
- $\hat{r}_t(\mathbf{s}, \mathbf{a})$ estimated expected reward
- $\hat{p}(\mathbf{s}'|\mathbf{s}, \mathbf{a})$ estimate transition probabilities

- $\widehat{\mathcal{V}}^\pi, \widehat{\mathcal{V}}_\gamma^\pi$ finite horizon and discounted value functions for policy π on estimated model
- σ trajectory $(\mathbf{s}_0, \mathbf{a}_0, \dots, \mathbf{s}_{T-1}, \mathbf{a}_{T-1}, \mathbf{s}_T)$
- $\widehat{\pi}^*$ optimal policy on the estimated model
- $\widehat{M}$ estimated model (reward and transitions)
- $\mathcal{V}^\pi(\mathbf{s}; M), \mathcal{V}_\gamma^\pi(\mathbf{s}; M)$ finite horizon and discounted value of state $\mathbf{s}$ under model M
- $\widehat{V}_n$ value at iteration n of approximate value iteration algorithm
- $\mathcal{V}_n$ value at iteration n of value iteration algorithm (no approximation)
- $\widetilde{M}$ the ‘explored’ model (in an on-policy learning algorithm)
- $\tilde{\mathbf{p}}(\cdot|\mathbf{s}, \mathbf{a}), \widehat{\mathbf{p}}(\cdot|\mathbf{s}, \mathbf{a})$ transitions and rewards in the explored model

10.1 Effective horizon of discounted return

Before we start looking at the learning setting, we will show a “reduction” from discounted return to finite horizon return. The main issue will be to show that the discounted return has an *effective horizon* such that rewards beyond it have a negligible effect on the discounted return.

Theorem 10.1. *Given a discount factor γ , the discounted return in the first $T = \frac{1}{1-\gamma} \log \frac{R_{\max}}{\varepsilon(1-\gamma)}$ time steps, is within ε of the total discounted return.*

Proof. Recall that the rewards are $\mathbf{r}_t \in [-R_{\max}, R_{\max}]$. Fix an infinite sequence of rewards $(\mathbf{r}_0, \dots, \mathbf{r}_t, \dots)$. We would like to consider the following difference:

$$\left| \sum_{t=0}^{\infty} \mathbf{r}_t \gamma^t - \sum_{t=0}^{T-1} \mathbf{r}_t \gamma^t \right| = \left| \sum_{t=T}^{\infty} \mathbf{r}_t \gamma^t \right| \leq \frac{\gamma^T}{1-\gamma} R_{\max}.$$

We want this difference to be bounded by ε , hence

$$\frac{\gamma^T}{1-\gamma} R_{\max} \leq \varepsilon.$$

This is equivalent to,

$$T \log(1/\gamma) \geq \log \frac{R_{\max}}{\varepsilon(1-\gamma)}.$$

Using $\log(1-x) \leq -x$ with $x = 1-\gamma$ gives $\log(\gamma) \leq -(1-\gamma)$. So, $\log(1/\gamma) = -\log \gamma \geq 1-\gamma$. If it holds that

$$T \geq \frac{1}{1-\gamma} \log \frac{R_{\max}}{\varepsilon(1-\gamma)},$$

then

$$T \log(1/\gamma) \geq \frac{\log(1/\gamma)}{1-\gamma} \log \frac{R_{\max}}{\varepsilon(1-\gamma)} \geq \log \frac{R_{\max}}{\varepsilon(1-\gamma)},$$

and the difference above is thus bounded by ε .

□

10.2 Off-Policy Model-Based Learning

Throughout this chapter we follow the finite horizon and discounted MDP notation introduced in Chapters 5 and 6. Since we focus on estimating a model from data, we make explicit the notion that rewards may be stochastic, and we denote $R_t(\mathbf{s}, \mathbf{a})$ a random variable for the reward obtained when visiting state $\mathbf{s}$ and action $\mathbf{a}$ at time t . For stationary models, we denote the random reward $R(\mathbf{s}, \mathbf{a})$. We assume that the reward at a state and action is independent of any other random variables in the model. Note that according to our notation, $r_t(\mathbf{s}, \mathbf{a}) = \mathbb{E}[R_t(\mathbf{s}, \mathbf{a})]$ and $r(\mathbf{s}, \mathbf{a}) = \mathbb{E}[R(\mathbf{s}, \mathbf{a})]$.

In the off-policy setting we would have access to previous executed trajectories, and we would like to use them to learn. Naturally, we will have to make some assumption about the trajectories. Intuitively, we will need to assume that they are sufficiently exploratory.

We will decompose the trajectories to quadruples, which are composed of $(\mathbf{s}, \mathbf{a}, \mathbf{r}, \mathbf{s}')$ where $\mathbf{r}$ is sampled from $R(\mathbf{s}, \mathbf{a})$ and $\mathbf{s}'$ is a sample of the random $p(\cdot|\mathbf{s}, \mathbf{a})$. The question we ask in the off-policy setting is how many samples we need from *each* state-action pair in order to learn a sufficiently well-performing policy.

The model-based approach means that our goal is to output an MDP $(\mathcal{S}, \mathcal{A}, \hat{r}, \hat{p})$, where $\mathcal{S}$ is the set of states, $\mathcal{A}$ is the set of actions, $\hat{r}(\mathbf{s}, \mathbf{a})$ is the approximate expected reward of $R(\mathbf{s}, \mathbf{a}) \in [0, R_{\max}]$, and $\hat{p}(\mathbf{s}'|\mathbf{s}, \mathbf{a})$ is the approximate probability of reaching state $\mathbf{s}'$ when we are in state $\mathbf{s}$ and doing action $\mathbf{a}$. Intuitively, we would like the approximate model and the true model have a similar expected value for any policy.

10.2.1 Mean estimation

We start with a basic mean estimation problem, which appears in many settings including supervised learning. Suppose we are given access to a random variable $R \in [0, 1]$ and would like to approximate its mean $\mu = \mathbb{E}[R]$. We observe m samples of R , which are $R_1, \dots, R_m$, and compute their observed mean $\hat{\mu} = \frac{1}{m} \sum_{i=1}^m R_i$.

By the law of large numbers we know that when m goes to infinity we have that $\hat{\mu}$ converges to μ . We would like to have concrete finite convergence bounds, mainly to derive the value of m as a function of the desired accuracy ε . For this we use concentration bounds (known as Chernoff-Hoeffding bounds). The bounds have both an additive form and a multiplicative form, given as follows:

Lemma 10.2 (Chernoff-Hoeffding). *Let $R_1, \dots, R_m$ be m i.i.d. samples of a random variable $R \in [0, 1]$. Let $\mu = \mathbb{E}[R]$ and $\hat{\mu} = \frac{1}{m} \sum_{i=1}^m R_i$. For any $\varepsilon \in (0, 1)$ we have,*

$$\Pr[|\mu - \hat{\mu}| \geq \varepsilon] \leq 2e^{-2\varepsilon^2 m}$$

In addition,

$$\Pr[\hat{\mu} \leq (1 - \varepsilon)\mu] \leq e^{-\varepsilon^2 \mu m / 2} \quad \text{and} \quad \Pr[\hat{\mu} \geq (1 + \varepsilon)\mu] \leq e^{-\varepsilon^2 \mu m / 3}$$

We will refer to the first bound as *additive* and the second set of bounds as *multiplicative*.

Using the additive bound of Lemma 10.2, we can derive the following sample complexity bound.

Corollary 10.3. *Let $R_1, \dots, R_m$ be m i.i.d. samples of a random variable $R \in [0, 1]$. Let $\mu = \mathbb{E}[R]$ and $\hat{\mu} = \frac{1}{m} \sum_{i=1}^m R_i$. Fix $\varepsilon, \delta > 0$. Then, for $m \geq \frac{1}{2\varepsilon^2} \log(2/\delta)$, with probability at least $1 - \delta$, we have that $|\mu - \hat{\mu}| \leq \varepsilon$.*

We can now use the above concentration bound in order to estimate the expected rewards. For each state-action $(\mathbf{s}, \mathbf{a})$ let $\hat{\mathbf{r}}(\mathbf{s}, \mathbf{a}) = \frac{1}{m} \sum_{i=1}^m R_i(\mathbf{s}, \mathbf{a})$ be the average of m samples. We can show the following:

Claim 10.4. *Given $m \geq \frac{R_{\max}^2}{2\varepsilon^2} \log \frac{2|S||A|}{\delta}$ samples for each state action $(\mathbf{s}, \mathbf{a})$, then, with probability at least $1 - \delta$, we have for every $(\mathbf{s}, \mathbf{a})$ that $|\mathbf{r}(\mathbf{s}, \mathbf{a}) - \hat{\mathbf{r}}(\mathbf{s}, \mathbf{a})| \leq \varepsilon$.*

Proof. First, we will need to scale the random variables to $[0, 1]$, which will be achieved by dividing them by $R_{\max}$. Then, by the Chernoff-Hoeffding bound (Corollary 10.3), using $\varepsilon' = \frac{\varepsilon}{R_{\max}}$ and $\delta' = \frac{\delta}{|S||A|}$, we have that for each $(\mathbf{s}, \mathbf{a})$ we have that $|\frac{\mathbf{r}(\mathbf{s}, \mathbf{a})}{R_{\max}} - \frac{\hat{\mathbf{r}}(\mathbf{s}, \mathbf{a})}{R_{\max}}| \leq \frac{\varepsilon}{R_{\max}}$ holds with probability at least $1 - \frac{\delta}{|S||A|}$.

We bound the probability over all state-action pairs using a union bound,

$$\begin{aligned} \Pr \left[\exists (\mathbf{s}, \mathbf{a}) : \left| \frac{\mathbf{r}(\mathbf{s}, \mathbf{a})}{R_{\max}} - \frac{\hat{\mathbf{r}}(\mathbf{s}, \mathbf{a})}{R_{\max}} \right| > \frac{\varepsilon}{R_{\max}} \right] &\leq \sum_{(\mathbf{s}, \mathbf{a})} \Pr \left[\left| \frac{\mathbf{r}(\mathbf{s}, \mathbf{a})}{R_{\max}} - \frac{\hat{\mathbf{r}}(\mathbf{s}, \mathbf{a})}{R_{\max}} \right| > \frac{\varepsilon}{R_{\max}} \right] \\ &\leq \sum_{(\mathbf{s}, \mathbf{a})} \frac{\delta}{|\mathcal{S}| |\mathcal{A}|} = \delta. \end{aligned}$$

Therefore, we have that, with probability at least $1 - \delta$, for every $(\mathbf{s}, \mathbf{a})$ simultaneously we have $|\mathbf{r}(\mathbf{s}, \mathbf{a}) - \hat{\mathbf{r}}(\mathbf{s}, \mathbf{a})| \leq \varepsilon$. $\square$

10.2.2 Influence of reward estimation errors

We would like to quantify the influence of having inaccurate estimates of the rewards. We will look both at the finite horizon return and the discounted return. We start with the case of finite horizon.

Influence of reward estimation errors: Finite horizon

Fix a stochastic Markov policy $\pi \in \Pi_{MS}$. We want to compare the return using the rewards $\mathbf{r}_t(\mathbf{s}, \mathbf{a})$ versus the rewards $\hat{\mathbf{r}}_t(\mathbf{s}, \mathbf{a})$ and the terminal reward $\mathbf{r}_T(\mathbf{s})$ versus the terminal reward $\hat{\mathbf{r}}_T(\mathbf{s})$. We will assume that for every $(\mathbf{s}, \mathbf{a})$ and $\mathbf{t}$ we have $|\mathbf{r}_t(\mathbf{s}, \mathbf{a}) - \hat{\mathbf{r}}_t(\mathbf{s}, \mathbf{a})| \leq \varepsilon$ and $|\mathbf{r}_T(\mathbf{s}) - \hat{\mathbf{r}}_T(\mathbf{s})| \leq \varepsilon$. We will show that the difference in return is bounded by $\varepsilon(T + 1)$, where T is the finite horizon.

Define the expected return of a policy π with the true rewards

$$\mathcal{V}^\pi(\mathbf{s}_0) = \mathbb{E}^{\pi, \mathbf{s}_0} \left[\sum_{t=0}^{T-1} \mathbf{r}_t(\mathbf{s}_t, \mathbf{a}_t) + \mathbf{r}_T(\mathbf{s}_T) \right],$$

and with the estimated rewards

$$\hat{\mathcal{V}}^\pi(\mathbf{s}_0) = \mathbb{E}^{\pi, \mathbf{s}_0} \left[\sum_{t=0}^{T-1} \hat{\mathbf{r}}_t(\mathbf{s}_t, \mathbf{a}_t) + \hat{\mathbf{r}}_T(\mathbf{s}_T) \right].$$

We are interested in bounding the difference between the two expectations, which is,

$$error(\pi) = |\mathcal{V}^\pi(\mathbf{s}_0) - \hat{\mathcal{V}}^\pi(\mathbf{s}_0)|.$$

Note that in both cases we use the true transition probabilities in the expectation. For a given trajectory $\sigma = (\mathbf{s}_0, \mathbf{a}_0, \dots, \mathbf{s}_{T-1}, \mathbf{a}_{T-1}, \mathbf{s}_T)$ we define

$$error(\pi, \sigma) = \left(\sum_{t=0}^{T-1} \mathbf{r}_t(\mathbf{s}_t, \mathbf{a}_t) + \mathbf{r}_T(\mathbf{s}_T) \right) - \left(\sum_{t=0}^{T-1} \hat{\mathbf{r}}_t(\mathbf{s}_t, \mathbf{a}_t) + \hat{\mathbf{r}}_T(\mathbf{s}_T) \right).$$

Taking the expectation over trajectories we define,

$$\text{error}(\pi) = |\mathbb{E}^{\pi, \mathbf{s}_0}[\text{error}(\pi, \sigma)]|.$$

Lemma 10.5. *Assume that for every state-action $(\mathbf{s}, \mathbf{a})$ and time $\mathbf{t}$ we have $|\mathbf{r}_t(\mathbf{s}, \mathbf{a}) - \hat{\mathbf{r}}_t(\mathbf{s}, \mathbf{a})| \leq \varepsilon$ and for every $\mathbf{s}$ we have $|\mathbf{r}_T(\mathbf{s}) - \hat{\mathbf{r}}_T(\mathbf{s})| \leq \varepsilon$. Then, for any policy $\pi \in \Pi_{MS}$ we have $\text{error}(\pi) \leq \varepsilon(T + 1)$.*

Proof. Since $\pi \in \Pi_{MS}$ it implies that π depends only on the time $\mathbf{t}$ and state $\mathbf{s}_t$. Therefore, probability of each trajectory $\sigma = (\mathbf{s}_0, \mathbf{a}_0, \dots, \mathbf{s}_{T-1}, \mathbf{a}_{T-1}, \mathbf{s}_T)$ is the same under the true rewards $\mathbf{r}_t(\mathbf{s}, \mathbf{a})$ and the estimated rewards $\hat{\mathbf{r}}_t(\mathbf{s}, \mathbf{a})$,

For each trajectory $\sigma = (\mathbf{s}_0, \mathbf{a}_0, \dots, \mathbf{s}_{T-1}, \mathbf{a}_{T-1}, \mathbf{s}_T)$, we have,

$$\begin{aligned} |\text{error}(\pi, \sigma)| &= \left| \sum_{t=0}^{T-1} (\mathbf{r}_t(\mathbf{s}_t, \mathbf{a}_t) + \mathbf{r}_T(\mathbf{s}_T)) - \sum_{t=0}^{T-1} (\hat{\mathbf{r}}_t(\mathbf{s}_t, \mathbf{a}_t) + \hat{\mathbf{r}}_T(\mathbf{s}_T)) \right| \\ &= \left| \sum_{t=0}^{T-1} (\mathbf{r}_t(\mathbf{s}_t, \mathbf{a}_t) - \hat{\mathbf{r}}_t(\mathbf{s}_t, \mathbf{a}_t)) + (\mathbf{r}_T(\mathbf{s}_T) - \hat{\mathbf{r}}_T(\mathbf{s}_T)) \right| \\ &\leq \sum_{t=0}^{T-1} |\mathbf{r}_t(\mathbf{s}_t, \mathbf{a}_t) - \hat{\mathbf{r}}_t(\mathbf{s}_t, \mathbf{a}_t)| + |\mathbf{r}_T(\mathbf{s}_T) - \hat{\mathbf{r}}_T(\mathbf{s}_T)| \\ &\leq \varepsilon T + \varepsilon. \end{aligned}$$

The lemma follows since $\text{error}(\pi) = |\mathbb{E}^{\pi, \mathbf{s}_0}[\text{error}(\pi, \sigma)]| \leq \varepsilon(T + 1)$, and the bound hold for every trajectory σ . $\square$

Computing approximate optimal policy: finite horizon

We now describe how to compute a near optimal policy for the finite horizon case. We start with the sample requirement. We need a sample of size $m \geq \frac{R_{\max}^2}{2\varepsilon^2} \log \frac{2|S||\mathcal{A}|T}{\delta}$ for each random variable $\mathbf{R}_t(\mathbf{s}, \mathbf{a})$ and $\mathbf{R}_T(\mathbf{s})$. Given the sample, we compute the rewards estimates $\hat{\mathbf{r}}_t(\mathbf{s}, \mathbf{a})$ and $\hat{\mathbf{r}}_T(\mathbf{s})$. By Claim 10.4, with probability at least $1 - \delta$, for every state $\mathbf{s} \in \mathcal{S}$ and action $\mathbf{a} \in \mathcal{A}$, we have $|\mathbf{r}_t(\mathbf{s}, \mathbf{a}) - \hat{\mathbf{r}}_t(\mathbf{s}, \mathbf{a})| \leq \varepsilon$ and $|\mathbf{r}_T(\mathbf{s}) - \hat{\mathbf{r}}_T(\mathbf{s})| \leq \varepsilon$. Now we can compute the optimal policy $\hat{\pi}^*$ for the estimated rewards $\hat{\mathbf{r}}_t(\mathbf{s}, \mathbf{a})$ and $\hat{\mathbf{r}}_T(\mathbf{s})$. The main goal is to show that $\hat{\pi}^*$ is a near optimal policy.

Theorem 10.6. *Assume that for every state-action $(\mathbf{s}, \mathbf{a})$ and time $\mathbf{t}$ we have $|\mathbf{r}_t(\mathbf{s}, \mathbf{a}) - \hat{\mathbf{r}}_t(\mathbf{s}, \mathbf{a})| \leq \varepsilon$ and for every $\mathbf{s}$ we have $|\mathbf{r}_T(\mathbf{s}) - \hat{\mathbf{r}}_T(\mathbf{s})| \leq \varepsilon$. Then,*

$$\mathcal{V}^{\pi^*}(\mathbf{s}_0) - \mathcal{V}^{\hat{\pi}^*}(\mathbf{s}_0) \leq 2\varepsilon(T + 1).$$

Proof. By Lemma 10.5, for any policy $\pi \in \Pi_{MS}$, we have that $error(\pi) \leq \varepsilon(T+1)$. This implies that

$$\mathcal{V}^{\pi^*}(\mathbf{s}_0) - \widehat{\mathcal{V}}^{\pi^*}(\mathbf{s}_0) \leq error(\pi^*) \leq \varepsilon(T+1),$$

and

$$\widehat{\mathcal{V}}^{\widehat{\pi}^*}(\mathbf{s}_0) - \mathcal{V}^{\widehat{\pi}^*}(\mathbf{s}_0) \leq error(\widehat{\pi}^*) \leq \varepsilon(T+1).$$

Since $\widehat{\pi}^*$ is optimal for $\widehat{\mathbf{r}}_t$ we have,

$$\widehat{\mathcal{V}}^{\pi^*}(\mathbf{s}_0) \leq \widehat{\mathcal{V}}^{\widehat{\pi}^*}(\mathbf{s}_0).$$

The theorem follows by adding the three inequalities. $\square$

Influence of reward estimation errors: discounted return

Fix a Markov stochastic policy $\pi \in \Pi_{MS}$. Again, define the expected return of policy π with the true rewards

$$\mathcal{V}_\gamma^\pi(\mathbf{s}_0) = \mathbb{E}^{\pi, \mathbf{s}_0} \left[\sum_{t=0}^{\infty} \gamma^t \mathbf{r}(\mathbf{s}_t, \mathbf{a}_t) \right],$$

and with the estimated rewards

$$\widehat{\mathcal{V}}_\gamma^\pi(\mathbf{s}_0) = \mathbb{E}^{\pi, \mathbf{s}_0} \left[\sum_{t=0}^{\infty} \gamma^t \widehat{\mathbf{r}}(\mathbf{s}_t, \mathbf{a}_t) \right].$$

We are interested in bounding the difference between the two

$$error(\pi) = |\mathcal{V}_\gamma^\pi(\mathbf{s}_0) - \widehat{\mathcal{V}}_\gamma^\pi(\mathbf{s}_0)|.$$

For a given trajectory $\sigma = (\mathbf{s}_0, \mathbf{a}_0, \dots)$ we define

$$error(\pi, \sigma) = \sum_{t=0}^{\infty} \gamma^t \mathbf{r}(\mathbf{s}_t, \mathbf{a}_t) - \sum_{t=0}^{\infty} \gamma^t \widehat{\mathbf{r}}(\mathbf{s}_t, \mathbf{a}_t).$$

Again, taking the expectation over trajectories we define,

$$error(\pi) = |\mathbb{E}^{\pi, \mathbf{s}_0}[error(\pi, \sigma)]|.$$

Lemma 10.7. *Assume that for every state-action $(\mathbf{s}, \mathbf{a})$ we have $|\mathbf{r}(\mathbf{s}, \mathbf{a}) - \widehat{\mathbf{r}}(\mathbf{s}, \mathbf{a})| \leq \varepsilon$. Then, for any policy $\pi \in \Pi_{MS}$ we have $error(\pi) \leq \frac{\varepsilon}{1-\gamma}$.*

Proof. Consider an infinite trajectory $\sigma = (\mathbf{s}_0, \mathbf{a}_0, \dots)$, and let $\sigma_T = (\mathbf{s}_0, \mathbf{a}_0, \dots, \mathbf{s}_{T-1}, \mathbf{a}_{T-1})$ denote its first T state-action pairs. Since the policy $\pi \in \Pi_{MS}$ is Markov, the probability of σ_T is the same under $\mathbf{r}(\mathbf{s}, \mathbf{a})$ and $\hat{\mathbf{r}}(\mathbf{s}, \mathbf{a})$. For each trajectory σ_T , we have,

$$\begin{aligned} \text{error}(\pi, \sigma_T) &= \left| \sum_{t=0}^T \mathbf{r}(\mathbf{s}_t, \mathbf{a}_t) \gamma^t - \sum_{t=0}^T \hat{\mathbf{r}}(\mathbf{s}_t, \mathbf{a}_t) \gamma^t \right| \\ &\leq \sum_{t=0}^T |\mathbf{r}(\mathbf{s}_t, \mathbf{a}_t) - \hat{\mathbf{r}}(\mathbf{s}_t, \mathbf{a}_t)| \gamma^t \\ &\leq \frac{\varepsilon}{1 - \gamma}. \end{aligned}$$

For each trajectory σ , we have,

$$\begin{aligned} \text{error}(\pi, \sigma) &= \left| \sum_{t=0}^{\infty} \mathbf{r}(\mathbf{s}_t, \mathbf{a}_t) \gamma^t - \sum_{t=0}^{\infty} \hat{\mathbf{r}}(\mathbf{s}_t, \mathbf{a}_t) \gamma^t \right| \\ &\leq \text{error}(\pi, \sigma_T) + \left| \sum_{t=T+1}^{\infty} \mathbf{r}(\mathbf{s}_t, \mathbf{a}_t) \gamma^t - \sum_{t=T+1}^{\infty} \hat{\mathbf{r}}(\mathbf{s}_t, \mathbf{a}_t) \gamma^t \right| \\ &\leq \text{error}(\pi, \sigma_T) + \frac{\gamma^{T+1} 2\mathbf{R}_{\max}}{1 - \gamma}. \end{aligned}$$

So, for any T , we have $\text{error}(\pi) = |E^{\pi, \mathbf{s}_0}[\text{error}(\pi, \sigma)]| \leq |E^{\pi, \mathbf{s}_0}[\text{error}(\pi, \sigma_T)]| + \frac{\gamma^{T+1} 2\mathbf{R}_{\max}}{1 - \gamma} \underbrace{\leq}_{T \rightarrow \infty} \frac{\varepsilon}{1 - \gamma}$. $\square$

Computing approximate optimal policy: discounted return

We now describe how to compute a near optimal policy for the discounted return. We need a sample of size $m \geq \frac{\mathbf{R}_{\max}^2}{2\varepsilon^2} \log \frac{2|S||A|}{\delta}$ for each random variable $\mathbf{R}(\mathbf{s}, \mathbf{a})$. Given the sample, we compute $\hat{\mathbf{r}}(\mathbf{s}, \mathbf{a})$. As we saw in the finite horizon case, with probability at least $1 - \delta$, we have for every $(\mathbf{s}, \mathbf{a})$ that $|\mathbf{r}(\mathbf{s}, \mathbf{a}) - \hat{\mathbf{r}}(\mathbf{s}, \mathbf{a})| \leq \varepsilon$. Now we can compute the policy $\hat{\pi}^*$ for the estimated rewards $\hat{\mathbf{r}}(\mathbf{s}, \mathbf{a})$. Again, the main goal is to show that $\hat{\pi}^*$ is a near optimal policy.

Theorem 10.8. *Assume that for every state-action $(\mathbf{s}, \mathbf{a})$ we have $|\mathbf{r}(\mathbf{s}, \mathbf{a}) - \hat{\mathbf{r}}(\mathbf{s}, \mathbf{a})| \leq \varepsilon$. Then,*

$$\mathcal{V}_{\gamma}^{\pi^*}(\mathbf{s}_0) - \mathcal{V}_{\gamma}^{\hat{\pi}^*}(\mathbf{s}_0) \leq \frac{2\varepsilon}{1 - \gamma}.$$

Proof. By Lemma 10.7 for any $\pi \in \Pi_{SS}$ we have $error(\pi) \leq \frac{\varepsilon}{1-\gamma}$. Therefore,

$$\mathcal{V}_\gamma^{\pi^*}(\mathbf{s}_0) - \widehat{\mathcal{V}}_\gamma^{\pi^*}(\mathbf{s}_0) \leq error(\pi^*) \leq \frac{\varepsilon}{1-\gamma},$$

and

$$\widehat{\mathcal{V}}_\gamma^{\widehat{\pi}^*}(\mathbf{s}_0) - \mathcal{V}_\gamma^{\widehat{\pi}^*}(\mathbf{s}_0) \leq error(\widehat{\pi}^*) \leq \frac{\varepsilon}{1-\gamma}.$$

Since $\widehat{\pi}^*$ is optimal for $\widehat{\mathbf{r}}$ we have,

$$\widehat{\mathcal{V}}_\gamma^{\pi^*}(\mathbf{s}_0) \leq \widehat{\mathcal{V}}_\gamma^{\widehat{\pi}^*}(\mathbf{s}_0).$$

The theorem follows by adding the three inequalities. $\square$

10.2.3 Estimating the transition probabilities

We now estimate the transition probabilities. Again, we will look at the observed model. Namely, for a given state-action $(\mathbf{s}, \mathbf{a})$, we consider m i.i.d. transitions $(\mathbf{s}, \mathbf{a}, \mathbf{s}'_i)$, for $1 \leq i \leq m$. We define the observed transition distribution,

$$\widehat{\mathbf{p}}(\mathbf{s}'|\mathbf{s}, \mathbf{a}) = \frac{|\{i : \mathbf{s}'_i = \mathbf{s}'\}|}{m}.$$

Our main goal would be to evaluate the observed model as a function of the sample size m .

We start with a general well-known observation about distributions.¹

Theorem 10.9. *Let q_1 and q_2 be two distributions over $\mathcal{S}$. Let $f : \mathcal{S} \rightarrow [0, F_{max}]$. Then,*

$$|\mathbb{E}_{s \sim q_1}[f(\mathbf{s})] - \mathbb{E}_{s \sim q_2}[f(\mathbf{s})]| \leq F_{max} \|q_1 - q_2\|_1,$$

where $\|q_1 - q_2\|_1 = \sum_{\mathbf{s} \in \mathcal{S}} |q_1(\mathbf{s}) - q_2(\mathbf{s})|$.

Proof. Consider the following derivation,

$$\begin{aligned} |\mathbb{E}_{s \sim q_1}[f(\mathbf{s})] - \mathbb{E}_{s \sim q_2}[f(\mathbf{s})]| &= \left| \sum_{\mathbf{s} \in \mathcal{S}} f(\mathbf{s})q_1(\mathbf{s}) - \sum_{\mathbf{s} \in \mathcal{S}} f(\mathbf{s})q_2(\mathbf{s}) \right| \\ &\leq \sum_{\mathbf{s} \in \mathcal{S}} f(\mathbf{s}) |q_1(\mathbf{s}) - q_2(\mathbf{s})| \\ &\leq F_{max} \|q_1 - q_2\|_1, \end{aligned}$$

¹A more refined analysis, which we avoid here for sake of simplicity, can yield $|\mathbb{E}_{s \sim q_1}[f(\mathbf{s})] - \mathbb{E}_{s \sim q_2}[f(\mathbf{s})]| \leq \frac{F_{max}}{2} \|q_1 - q_2\|_1$. See, e.g., Prop. 4.5 in [96].

where the first identity is the explicit expectation, the second is by the triangle inequality, and the third is by bounding the values of f by the maximum possible value. $\square$

When we measure the distance between two Markov chains M_1 and M_2 , it is natural to consider the next state distributions of each state i , namely $M[i, \cdot]$. The distance between the next state distribution for state i can be measured by the L_1 norm, i.e., $\|M_1[i, \cdot] - M_2[i, \cdot]\|_1$. We take the worst case over states, and define $\|M\|_{\infty,1} = \max_i \sum_j |M[i, j]|$. The measure that we will consider is $\|M_1 - M_2\|_{\infty,1}$, and assume that $\|M_1 - M_2\|_{\infty,1} \leq \alpha$, namely, that for any state, the next state distributions differ by at most α in norm L_1 .

Clearly if $\alpha \approx 0$ then the distributions will be almost identical, but we would like to have a quantitative bound on the difference, which will allow us to derive an upper bound of the required sample size m .

Theorem 10.10. *Assume that $\|M_1 - M_2\|_{\infty,1} \leq \alpha$. Let $q_1^{\mathfrak{t}}$ and $q_2^{\mathfrak{t}}$ be the distribution over states after trajectories of length $\mathfrak{t}$ of M_1 and M_2 , respectively. Then,*

$$\|q_1^{\mathfrak{t}} - q_2^{\mathfrak{t}}\|_1 \leq \alpha \mathfrak{t}.$$

Proof. Let p_0 be the distribution of the start state. Then $q_1^{\mathfrak{t}} = p_0^{\top} M_1^{\mathfrak{t}}$ and $q_2^{\mathfrak{t}} = p_0^{\top} M_2^{\mathfrak{t}}$. The proof is by induction on $\mathfrak{t}$. Clearly, for $\mathfrak{t} = 0$ we have $q_1^0 = q_2^0 = p_0^{\top}$.

We start with a few basic facts about matrix norms. Recall that $\|M\|_{\infty,1} = \max_i \sum_j |M[i, j]|$. Then,

$$\begin{aligned} \|z^{\top} M\|_1 &= \sum_j \left| \sum_i z[i] M[i, j] \right| \\ &\leq \sum_{i,j} |z[i]| |M[i, j]| \\ &= \sum_i |z[i]| \sum_j |M[i, j]| \\ &\leq \|z\|_1 \|M\|_{\infty,1}. \end{aligned} \tag{10.1}$$

This implies the following two simple facts. First, let q be a distribution, hence, $\|q\|_1 = 1$, and M a matrix such that $\|M\|_{\infty,1} \leq \alpha$. Then,

$$\|qM\|_1 \leq \|q\|_1 \|M\|_{\infty,1} \leq \alpha. \tag{10.2}$$

Second, let M be a row-stochastic matrix, this implies that $\|M\|_{\infty,1} = 1$. Then,

$$\|zM\|_1 \leq \|z\|_1 \|M\|_{\infty,1} \leq \|z\|_1. \tag{10.3}$$

For the induction step, let $z^{\mathbf{t}} = q_1^{\mathbf{t}} - q_2^{\mathbf{t}}$, and assume that $\|z^{\mathbf{t}-1}\|_1 \leq \alpha(\mathbf{t}-1)$. We have,

$$\begin{aligned}\|q_1^{\mathbf{t}} - q_2^{\mathbf{t}}\|_1 &= \|p_0^\top M_1^{\mathbf{t}} - p_0^\top M_2^{\mathbf{t}}\|_1 \\ &= \|q_1^{\mathbf{t}-1} M_1 - (q_1^{\mathbf{t}-1} - z^{\mathbf{t}-1}) M_2\|_1 \\ &\leq \|q_1^{\mathbf{t}-1} (M_1 - M_2)\|_1 + \|z^{\mathbf{t}-1} M_2\|_1 \\ &\leq \alpha + \alpha(\mathbf{t}-1) = \alpha\mathbf{t},\end{aligned}$$

where the last inequality is derived as follows: for the first term we used Eq. (10.2), and for the second term we used Eq. (10.3) with the inductive claim. $\square$

Approximate model and simulation lemma

We define an α -approximate model as follows.

Definition 10.1. A model $\widehat{M}$ is an α -approximate model of M if for every state-action $(\mathbf{s}, \mathbf{a})$ we have: (1) $|\widehat{\mathbf{r}}(\mathbf{s}, \mathbf{a}) - \mathbf{r}(\mathbf{s}, \mathbf{a})| \leq \alpha$ and (2) $\|\widehat{\mathbf{p}}(\cdot|\mathbf{s}, \mathbf{a}) - \mathbf{p}(\cdot|\mathbf{s}, \mathbf{a})\|_1 \leq \alpha$.

Since we have two different models, we define the value of a policy π in a model M as $\mathcal{V}^\pi(\mathbf{s}_0; M)$.

The following simulation lemma, for the finite horizon case, guarantees that approximate models have similar return.

Lemma 10.11. Fix $\alpha \leq \min\{\frac{\varepsilon}{2T}, \frac{\varepsilon}{2\mathbf{R}_{\max}T^2}\}$, and assume that model $\widehat{M}$ is an α -approximate model of M . For the finite horizon return, for any policy $\pi \in \Pi_{MS}$, we have

$$|\mathcal{V}^\pi(\mathbf{s}_0; M) - \mathcal{V}^\pi(\mathbf{s}_0; \widehat{M})| \leq \varepsilon$$

Proof. Consider the difference in the dynamics. By Theorem 10.10 the distance between the state distributions of M and $\widehat{M}$ at time $\mathbf{t}$ is bounded by $\alpha\mathbf{t}$. Since the maximum reward is $\mathbf{R}_{\max}$, by Theorem 10.9 the difference is bounded by $\sum_{\mathbf{t}=0}^T \alpha\mathbf{t}\mathbf{R}_{\max} \leq \alpha T^2 \mathbf{R}_{\max}$. For $\alpha \leq \frac{\varepsilon}{2\mathbf{R}_{\max}T^2}$ it implies that the difference is at most $\varepsilon/2$. The difference in the rewards can result in a maximum return difference of αT . For $\alpha \leq \frac{\varepsilon}{2T}$ it implies that the difference is at most $\varepsilon/2$. The total difference is therefore at most ε . $\square$

We now present the simulation lemma for the discounted return case, which also guarantees that approximate models have similar return.

Lemma 10.12. Fix $\alpha \leq \frac{(1-\gamma)^2\varepsilon}{2\mathbf{R}_{\max}}$, and assume that model $\widehat{M}$ is an α -approximate model of M . For the discounted return, for any policy $\pi \in \Pi_{SS}$, we have

$$|\mathcal{V}_\gamma^\pi(\mathbf{s}_0; M) - \mathcal{V}_\gamma^\pi(\mathbf{s}_0; \widehat{M})| \leq \varepsilon.$$

Proof. The difference in the rewards can result in a maximum return difference of $\alpha/(1 - \gamma)$. Consider The difference in the dynamics. By Theorem 10.10 the distance between the state distributions of M and $\widehat{M}$ at time $\mathfrak{t}$ is bounded by $\alpha\mathfrak{t}$. Since the maximum reward is $\mathbf{R}_{\max}$, by Theorem 10.9 the difference is bounded by $\sum_{\mathfrak{t}=0}^{\infty} \alpha\mathfrak{t}\mathbf{R}_{\max}\gamma^{\mathfrak{t}}$. The sum

$$\sum_{\mathfrak{t}=0}^{\infty} \alpha\mathbf{R}_{\max}\mathfrak{t}\gamma^{\mathfrak{t}} = \alpha\mathbf{R}_{\max} \frac{\gamma}{1 - \gamma} \sum_{\mathfrak{t}=0}^{\infty} \mathfrak{t}\gamma^{\mathfrak{t}-1}(1 - \gamma) = \alpha\mathbf{R}_{\max} \frac{\gamma}{(1 - \gamma)^2} < \alpha\mathbf{R}_{\max} \frac{1}{(1 - \gamma)^2},$$

where the last equality uses the expected value of a geometric distribution with parameter γ . Using the bound for α implies that the difference is at most ε . $\square$

Putting it all together

We want with high probability $(1 - \delta)$ to have an α -approximate model. For this we need to bound the sample size needed to approximate a distribution in the norm L_1 . Here, Bretagnolle Huber-Carol inequality comes in handy.

Lemma 10.13 (Bretagnolle Huber-Carol). *Let X be a random variable taking values in $\{1, \dots, k\}$, where $\Pr[X = i] = p_i$. Assume we sample X for n times and observe the value i in $\hat{n}_i$ outcomes. Then,*

$$\Pr \left[\sum_{i=1}^k \left| \frac{\hat{n}_i}{n} - p_i \right| \geq \lambda \right] \leq 2^{k+1} e^{-2n\lambda^2}$$

For completeness we give the proof. (The proof can also be found as Proposition A6.6 of [165], and there the bound is smaller by a factor of 2.)

Proof. Note that,

$$\sum_{i=1}^k \left| \frac{\hat{n}_i}{n} - p_i \right| = 2 \max_{S \subset [k]} \sum_{i \in S} \frac{\hat{n}_i}{n} - p_i,$$

which follows by taking $S = \{i : \frac{\hat{n}_i}{n} \geq p_i\}$.

We can now perform a concentration bound (Chernoff-Hoeffding, Lemma 10.2) for each subset $S \subset [k]$, and get that the deviation is λ with probability at most $e^{-2n\lambda^2}$. Using a union bound over all 2^k subsets S we derive the lemma. $\square$

The above lemma implies that to get, with probability at least $1 - \delta$, accuracy α for each state-action $(\mathbf{s}, \mathbf{a})$, it is sufficient to sample $m = O\left(\frac{|\mathcal{S}| + \log(|\mathcal{S}| |\mathcal{A}| / \delta)}{\alpha^2}\right)$ samples

for each state-action $(\mathbf{s}, \mathbf{a})$. Plugging in the value of α , for the finite horizon, we have

$$m = O\left(\frac{R_{\max}^2}{\varepsilon^2} T^4 (|\mathcal{S}| + \log(|\mathcal{S}| |\mathcal{A}|/\delta))\right),$$

and for the discounted return

$$m = O\left(\frac{R_{\max}^2}{\varepsilon^2} \frac{1}{(1-\gamma)^4} (|\mathcal{S}| + \log(|\mathcal{S}| |\mathcal{A}|/\delta))\right).$$

Assume we have a sample of m for each state-action $(\mathbf{s}, \mathbf{a})$. Then, with probability at least $1 - \delta$, we have an α -approximate model $\widehat{M}$. We compute an optimal policy $\widehat{\pi}^*$ for $\widehat{M}$. This implies that $\widehat{\pi}^*$ is a 2ε -optimal policy. Namely,

$$|\mathcal{V}^*(\mathbf{s}_0) - \mathcal{V}^{\widehat{\pi}^*}(\mathbf{s}_0)| \leq 2\varepsilon.$$

When considering the total sample size, we need to consider all state-action pairs. For the finite horizon (not necessarily stationary), the total sample size is

$$m|\mathcal{S}| |\mathcal{A}| T = O\left(\frac{R_{\max}^2}{\varepsilon^2} T^5 |\mathcal{A}| \left[|\mathcal{S}|^2 + |\mathcal{S}| \log(|\mathcal{S}| |\mathcal{A}|/\delta)\right]\right)$$

and for the discounted return (stationary model)

$$m|\mathcal{S}| |\mathcal{A}| = O\left(\frac{R_{\max}^2}{\varepsilon^2 (1-\gamma)^4} |\mathcal{A}| \left[|\mathcal{S}|^2 + |\mathcal{S}| \log(|\mathcal{S}| |\mathcal{A}|/\delta)\right]\right).$$

We can now look on the dependency of our sample complexity and its dependence on the various parameters.

1. The required sample size scales like $\frac{R_{\max}^2}{\varepsilon^2}$, which looks like the right bound, even for estimation of a random variable's expectation.
2. The dependency on the horizon is necessary, although it is probably not optimal. In [36] a sample bound of $O\left(\frac{|\mathcal{S}|^2 |\mathcal{A}| T^2 R_{\max}^2}{\varepsilon^2} \log \frac{1}{\delta}\right)$ is given.
3. The dependency on the number of states $|\mathcal{S}|$ and actions $|\mathcal{A}|$, is due to the fact that we like a very high approximation of the next state distribution. We need to approximate $|\mathcal{S}|^2 |\mathcal{A}|$ parameters, so for this task the bound is reasonable. However, we will show that if we restrict the task to compute an approximate optimal policy we can reduce the sample size by a factor of approximately $|\mathcal{S}|$.

10.2.4 Improved sample bound: Approximate Value Iteration (AVI)

We would like to exhibit a better sample complexity, for the very interesting case of deriving an approximately optimal policy. The following approach is off-policy, but not model based, as we will not build an explicit model $\widehat{M}$. Instead, the construction and proof would use the samples to approximate the Value Iteration algorithm (see Chapter 6.6). Recall, that the Value Iteration algorithm works as follows. Initially, we set the values arbitrarily,

$$\mathcal{V}_0 = \{\mathcal{V}_0(\mathbf{s})\}_{\mathbf{s} \in \mathcal{S}}.$$

In iteration n we compute for every $\mathbf{s} \in \mathcal{S}$

$$\begin{aligned} \mathcal{V}_{n+1}(\mathbf{s}) &= \max_{\mathbf{a} \in \mathcal{A}} \left\{ \mathbf{r}(\mathbf{s}, \mathbf{a}) + \gamma \sum_{\mathbf{s}' \in \mathcal{S}} \mathbf{p}(\mathbf{s}' | \mathbf{s}, \mathbf{a}) \mathcal{V}_n(\mathbf{s}') \right\} \\ &= \max_{\mathbf{a} \in \mathcal{A}} \left\{ \mathbf{r}(\mathbf{s}, \mathbf{a}) + \gamma \mathbb{E}_{\mathbf{s}' \sim \mathbf{p}(\cdot | \mathbf{s}, \mathbf{a})} [\mathcal{V}_n(\mathbf{s}')] \right\}. \end{aligned}$$

We showed that $\lim_{n \rightarrow \infty} \mathcal{V}_n = \mathcal{V}^*$, and that the error rate is $O(\frac{\gamma^n}{1-\gamma} \mathbf{R}_{\max})$. This implies that if we run for N iterations, where $N = \frac{1}{1-\gamma} \log \frac{\mathbf{R}_{\max}}{\varepsilon(1-\gamma)}$, we have an error of at most ε (see Chapter 6.6.)

We would like to approximate the Value Iteration algorithm using a sample. Namely, for each state-action $(\mathbf{s}, \mathbf{a})$ we have a sample of size m , i.e., $\{(\mathbf{s}, \mathbf{a}, \mathbf{r}_i, \mathbf{s}'_i)\}_{i=1}^m$. The Approximate Value Iteration (AVI) using the sample would be,

$$\widehat{\mathcal{V}}_{n+1}(\mathbf{s}) = \max_{\mathbf{a} \in \mathcal{A}} \left\{ \widehat{\mathbf{r}}(\mathbf{s}, \mathbf{a}) + \gamma \frac{1}{m} \sum_{i=1}^m \widehat{\mathcal{V}}_n(\mathbf{s}'_i) \right\},$$

where $\widehat{\mathbf{r}}(\mathbf{s}, \mathbf{a}) = \frac{1}{m} \sum_{i=1}^m \mathbf{r}_i(\mathbf{s}, \mathbf{a})$.

The intuition is that if we have a large enough sample, AVI will approximate the Value Iteration. One way to guarantee the approximation is by requiring that the empirical transition probabilities are close to the true ones, as we did in the previous section. However, this comes at a cost of $O(|\mathcal{S}|^2 |\mathcal{A}|)$ samples, from the Bretagnolle Huber-Carol bound. Instead, we note the following less restrictive result:

Corollary 10.14. *Let X be a random variable taking values in $\{1, \dots, k\}$, where $\Pr[X = i] = p_i$. Assume we sample X for m times and observe the value i in $\hat{n}_i$ outcomes. Let $\hat{p}_i = \frac{\hat{n}_i}{m}$ denote the empirical distribution. Consider N functions $f_1, \dots, f_N$ from $\{1, \dots, k\}$ to $[0, 1]$, and let $\widehat{\mathbb{E}}[f_j] = \sum_{i=1, \dots, k} \hat{p}_i f_j(i)$ denote their*

empirical means. For $m \geq \frac{1}{2\varepsilon^2} \log(2N/\delta)$, with probability at least $1 - \delta$, for all $j \in \{1, \dots, k\}$,

$$\left| \hat{\mathbb{E}}[f_j] - \mathbb{E}[f_j(X)] \right| \leq \varepsilon.$$

Proof. Let A_j denote the event $\left| \hat{\mathbb{E}}[f_j] - \mathbb{E}[f_j(X)] \right| > \varepsilon$. From the Chernoff-Hoffding bound (Corollary 10.3), using $\delta' = \frac{\delta}{N}$, we have that $\Pr(A_j) \leq \frac{\delta}{N}$. From the union bound, $\Pr\left(\bigcup_{j \in \{1, \dots, k\}} A_j\right) \leq \sum_{j \in \{1, \dots, k\}} \Pr(A_j) = \delta$. $\square$

We set m such that, with probability at least $1 - \delta$, for every state-action $(\mathbf{s}, \mathbf{a})$ and any iteration $n \in [1, N]$ we have:

$$\left| \mathbb{E} \left[\hat{V}_n(\mathbf{s}') \right] - \frac{1}{m} \sum_{i=1}^m \hat{V}_n(\mathbf{s}'_i) \right| \leq \varepsilon',$$

and also

$$|\hat{\mathbf{r}}(\mathbf{s}, \mathbf{a}) - \mathbf{r}(\mathbf{s}, \mathbf{a})| \leq \varepsilon'.$$

This holds for $m = O\left(\frac{\mathcal{V}_{\max}^2}{\varepsilon'^2} \log(N|\mathcal{S}| |\mathcal{A}|/\delta)\right)$, where $\mathcal{V}_{\max}$ bounds the maximum value. I.e., for finite horizon $\mathcal{V}_{\max} = \mathbf{T} \mathbf{R}_{\max}$ and for discounted return $\mathcal{V}_{\max} = \frac{\mathbf{R}_{\max}}{1-\gamma}$.

Assume that for every state $\mathbf{s} \in \mathcal{S}$ we have

$$\left| \hat{V}_n(\mathbf{s}) - \mathcal{V}_n(\mathbf{s}) \right| \leq \lambda.$$

Then,

$$\begin{aligned} & \left| \hat{V}_{n+1}(\mathbf{s}) - \mathcal{V}_{n+1}(\mathbf{s}) \right| = \\ & = \left| \max_a \left\{ \hat{\mathbf{r}}(\mathbf{s}, \mathbf{a}) + \gamma \frac{1}{m} \sum_{i=1}^m \hat{V}_n(\mathbf{s}'_i) \right\} - \max_a \left\{ \mathbf{r}(\mathbf{s}, \mathbf{a}) + \gamma \mathbb{E}_{\mathbf{s}' \sim \mathbf{p}(\cdot|\mathbf{s}, \mathbf{a})} [\mathcal{V}_n(\mathbf{s}')] \right\} \right| \\ & \leq \max_a \left| \hat{\mathbf{r}}(\mathbf{s}, \mathbf{a}) + \gamma \frac{1}{m} \sum_{i=1}^m \hat{V}_n(\mathbf{s}'_i) - \mathbf{r}(\mathbf{s}, \mathbf{a}) - \gamma \mathbb{E}_{\mathbf{s}' \sim \mathbf{p}(\cdot|\mathbf{s}, \mathbf{a})} [\mathcal{V}_n(\mathbf{s}')] \right| \\ & \leq \max_a \left\{ \left| \hat{\mathbf{r}}(\mathbf{s}, \mathbf{a}) - \mathbf{r}(\mathbf{s}, \mathbf{a}) \right| + \gamma \left| \frac{1}{m} \sum_{i=1}^m \hat{V}_n(\mathbf{s}'_i) - \mathbb{E}_{\mathbf{s}' \sim \mathbf{p}(\cdot|\mathbf{s}, \mathbf{a})} [\mathcal{V}_n(\mathbf{s}')] \right| \right\} \\ & \leq \varepsilon' + \gamma \left| \frac{1}{m} \sum_{i=1}^m \hat{V}_n(\mathbf{s}'_i) - \mathbb{E}_{\mathbf{s}' \sim \mathbf{p}(\cdot|\mathbf{s}, \mathbf{a})} [\hat{V}_n(\mathbf{s}')] \right| \\ & \quad + \gamma \left| \mathbb{E}_{\mathbf{s}' \sim \mathbf{p}(\cdot|\mathbf{s}, \mathbf{a})} [\hat{V}_n(\mathbf{s}')] - \mathbb{E}_{\mathbf{s}' \sim \mathbf{p}(\cdot|\mathbf{s}, \mathbf{a})} [\mathcal{V}_n(\mathbf{s}')] \right| \\ & \leq \varepsilon' + \gamma \varepsilon' + \gamma \lambda = (1 + \gamma) \varepsilon' + \gamma \lambda. \end{aligned}$$

Since $\widehat{V}_0(\mathbf{s}) = \mathcal{V}_0(\mathbf{s})$, the recurrence above gives:

$$\left| \widehat{V}_n(\mathbf{s}) - \mathcal{V}_n(\mathbf{s}) \right| \leq (1 + \gamma) \varepsilon' \sum_{i=0}^{n-1} \gamma^i \leq \frac{(1 + \gamma) \varepsilon'}{1 - \gamma} \leq \frac{2\varepsilon'}{1 - \gamma}.$$

Therefore, if we sample $m = O\left(\frac{\mathcal{V}_{\max}^2}{(1-\gamma)^2 \varepsilon^2} \log \frac{N|\mathcal{S}||\mathcal{A}|}{\delta}\right)$, we have that with probability at least $1 - \delta$ for every state-action $(\mathbf{s}, \mathbf{a})$ the approximation error ε' is at most $(1 - \gamma)\varepsilon$. This implies that the Approximate Value Iteration has error at most ε . The work of [2] shows that the simple maximum likelihood estimator also achieves this optimal min-max bound.

The main result is that we can run Approximate Value Iteration algorithm for N iterations and approximate well the optimal value function and policy.

Theorem 10.15. *Given for every state-action pair a sample of size*

$$m = O\left(\frac{\mathcal{R}_{\max}^2}{(1 - \gamma)^4 \varepsilon^2} \log \frac{|\mathcal{S}| |\mathcal{A}| \log\left(\frac{\mathcal{R}_{\max}}{\varepsilon(1-\gamma)}\right)}{(1 - \gamma)\delta}\right)$$

Running the Approximate Value Iteration for $N = \frac{1}{1-\gamma} \log \frac{\mathcal{R}_{\max}}{\varepsilon(1-\gamma)}$ iterations results in an ε -approximation of the optimal value function.

The implicit drawback of the above theorem is that we are approximating only the optimal policy, and cannot evaluate an arbitrary policy.

10.3 On-Policy Learning

In the off-policy setting, when given some trajectories, we learn the model and use it to get an approximate optimal policy. Essentially, we assumed that the trajectories are exploratory enough, in the sense that each state-action $(\mathbf{s}, \mathbf{a})$ has a sufficient number of samples. In the online setting it is the responsibility of the learner to perform the exploration. This will be the main challenge of this section.

We will consider two (similar) tasks. The first is to reconstruct the MDP to sufficient accuracy. Given such a reconstruction we can compute the optimal policy for it and be guaranteed that it is a near optimal policy in the true MDP. The second is to reconstruct only the parts of the MDP which have a significant influence on the optimal policy. In this case we will be able to show that in most time steps we are playing a near optimal action.

10.3.1 Learning a Deterministic Decision Process

Recall that a Deterministic Decision Process (DDP) is modeled by a directed graph, where the states are the vertices, and each action is associated with an edge. For simplicity we will assume that the graph is strongly connected, i.e., there is a directed path between any two states (see Chapter 3).

We will start by showing how to recover the DDP. The basic idea is rather simple. We partition the state-action pairs to **known** and **unknown**. Initially all states-action pairs are **unknown**. Each **unknown** state-action that we execute is moved to **known**. In each step we look for a path from the current state to some **unknown** state-action pair. When all the state-action pairs are **known** we are done. This implies that we have at most $|\mathcal{S}| |\mathcal{A}|$ iterations, and since the maximum length of such a path is at most $|\mathcal{S}|$, the total number of time steps would be bounded by $|\mathcal{S}|^2 |\mathcal{A}|$.

To compute a path from the **known** state-action pairs to some **unknown** state-action pair, we reduce this task to a planning task in DDP. For each **known** state-action pair we define the reward to be zero and the next state to be the next state in the DDP (which we already observed, since the state-action is **known**). For each **unknown** state-action pair we define the reward to be $R_{\max}$ and the next state as the same state, i.e., we stay in the same state. We can now solve for the optimal policy (infinite horizon average reward) of our model.² As long as there are unobserved state-action pairs, the optimal policy will reach one of them.

Theorem 10.16. *For any strongly connected DDP there is a strategy ρ which recovers the DDP in at most $O(|\mathcal{S}|^2 |\mathcal{A}|)$*

Proof. We first define the explored model. Given an observation set $\{(\mathbf{s}_t, \mathbf{a}_t, \mathbf{r}_t, \mathbf{s}_{t+1})\}$, we define an explored model $\widetilde{M}$, where $\tilde{f}(\mathbf{s}_t, \mathbf{a}_t) = \mathbf{s}_{t+1}$ and $\tilde{r}(\mathbf{s}, \mathbf{a}) = 0$. For $(\mathbf{s}, \mathbf{a})$ which do not appear in the observation set, we define $\tilde{f}(\mathbf{s}, \mathbf{a}) = \mathbf{s}$ and $\tilde{r}(\mathbf{s}, \mathbf{a}) = R_{\max}$.

We can now present the on-policy exploration algorithm.

²Alternatively, we could define the unknown states as goal states and solve using shortest path algorithms.

Algorithm 13 On-Policy Exploration for a Deterministic Decision Process

- 1: **Initialize:**
 - 2: Set $\widetilde{M}_0$ such that $\widetilde{f}(\mathbf{s}, \mathbf{a}) = \mathbf{s}$ and $\widetilde{r}(\mathbf{s}, \mathbf{a}) = R_{\max}$ for all $(\mathbf{s}, \mathbf{a})$
 - 3: Choose initial state $\mathbf{s}_0 \in \mathcal{S}$ arbitrarily
 - 4: **For** $t = 0, 1, 2, \dots$
 - 5: Compute $\widetilde{\pi}_t^* \in \Pi_{SD}$, the optimal policy for $\widetilde{M}_t$ (infinite horizon average reward)
 - 6: **If** the return of $\widetilde{\pi}_t^*$ on $\widetilde{M}_t$ is zero: **terminate**
 - 7: Use action: $\mathbf{a}_t = \widetilde{\pi}_t^*(\mathbf{s}_t)$
 - 8: Observe reward $\mathbf{r}_t$ and next state $\mathbf{s}_{t+1}$, add $(\mathbf{s}_t, \mathbf{a}_t, \mathbf{r}_t, \mathbf{s}_{t+1})$ to observation set
 - 9: Modify $\widetilde{M}_t$ to $\widetilde{M}_{t+1}$ by setting:
 - 10: $\widetilde{f}(\mathbf{s}_t, \mathbf{a}_t) = \mathbf{s}_{t+1}$
 - 11: $\widetilde{r}(\mathbf{s}_t, \mathbf{a}_t) = 0$
-

We claim that at termination we have observed each state-action pair at least once. Otherwise, there will be state-action pairs that would have a reward of $R_{\max}$ and at least one of those pairs would be reachable from the current **known** states. So the optimal policy would have a return of $R_{\max}$ contradicting the fact that it had return of zero.

The time to termination can be bounded by $O(|\mathcal{S}|^2|\mathcal{A}|)$, since we have $|\mathcal{S}| |\mathcal{A}|$ state-action pairs and while we did not terminate, we reach an **unknown** state-action pair after at most $|\mathcal{S}|$ steps.

After the algorithm terminates, define the following model. Given the observations during the run of the algorithm $\{(\mathbf{s}_t, \mathbf{a}_t, \mathbf{r}_t, \mathbf{s}_{t+1})\}$, we define the observed model $\widehat{M}$, where $\widehat{f}(\mathbf{s}_t, \mathbf{a}_t) = \mathbf{s}_{t+1}$ and $\widehat{r}(\mathbf{s}, \mathbf{a}) = \mathbf{r}_t$. This model is exactly the true DDP M since it includes all state-action pairs, and for each it has the correct reward and next state. (We are using the fact that for a DDP multiple observations of the same state and action result in identical observations.) $\square$

The above algorithm reconstructs the model completely. We can be slightly more refined. We can define an *optimistic* model, whose return upper bounds that of the true model. We can then solve for the optimal policy in the optimistic model, and if it does not reach a new state-action pair (after sufficiently long time) then it has to be the true optimal policy.

We first define the optimistic observed model. Given an observation set $\{(\mathbf{s}_t, \mathbf{a}_t, \mathbf{r}_t, \mathbf{s}_{t+1})\}$, we define an optimistic observed model $\widehat{M}$, where $\widehat{f}(\mathbf{s}_t, \mathbf{a}_t) = \mathbf{s}_{t+1}$ and $\widehat{r}(\mathbf{s}_t, \mathbf{a}_t) = \mathbf{r}_t$. For $(\mathbf{s}, \mathbf{a})$ which do not appear in the observation set, we define $\widehat{f}(\mathbf{s}, \mathbf{a}) = \mathbf{s}$ and $\widehat{r}(\mathbf{s}, \mathbf{a}) = R_{\max}$.

First, we claim that for any $\pi \in \Pi_{SS}$ the optimistic observed model $\widehat{M}$ can only increase the value compared to the true model M . Namely,

$$\widehat{\mathcal{V}}^\pi(\mathbf{s}; \widehat{M}) \geq \mathcal{V}^\pi(\mathbf{s}; M).$$

The increase holds for any trajectory, and note that once π reaches $(\mathbf{s}, \mathbf{a})$ that was not observed, its reward will be $R_{\max}$ forever. (This is since $\pi \in \Pi_{SS}$.)

We can now present the on-policy learning algorithm.

Algorithm 14 Optimistic On-Policy Algorithm for a Deterministic Decision Process

- 1: **Initialize:**
 - 2: Set $\widehat{M}_0$ such that $\widehat{f}(\mathbf{s}, \mathbf{a}) = \mathbf{s}$ and $\widehat{r}(\mathbf{s}, \mathbf{a}) = R_{\max}$ for all $(\mathbf{s}, \mathbf{a})$
 - 3: Choose initial state $\mathbf{s}_0 \in \mathcal{S}$ arbitrarily
 - 4: **For** $t = 0, 1, 2, \dots$
 - 5: Compute $\widehat{\pi}_t^* \in \Pi_{SD}$, the optimal policy for $\widehat{M}_t$ (infinite horizon average reward)
 - 6: Use action: $\mathbf{a}_t = \widehat{\pi}_t^*(\mathbf{s}_t)$
 - 7: Observe reward $\mathbf{r}_t$ and next state $\mathbf{s}_{t+1}$, add $(\mathbf{s}_t, \mathbf{a}_t, \mathbf{r}_t, \mathbf{s}_{t+1})$ to observation set
 - 8: Modify $\widehat{M}_t$ to $\widehat{M}_{t+1}$ by setting:
 - 9: $\widehat{f}(\mathbf{s}_t, \mathbf{a}_t) = \mathbf{s}_{t+1}$
 - 10: $\widehat{r}(\mathbf{s}_t, \mathbf{a}_t) = \mathbf{r}_t$
-

We can now state the convergence of the algorithm to the optimal policy.

Theorem 10.17. *After $\tau \leq |\mathcal{S}|^2|\mathcal{A}|$ time steps the policy $\widehat{\pi}_\tau^*$ never changes and it is optimal for the true model M .*

Proof. We first claim that the model $\widehat{M}_t$ can change at most $|\mathcal{S}| |\mathcal{A}|$ times (i.e., $\widehat{M}_t \neq \widehat{M}_{t+1}$). Each time we change the observed model $\widehat{M}_t$, we observe a new $(\mathbf{s}, \mathbf{a})$ for the first time. Since there are $|\mathcal{S}| |\mathcal{A}|$ such pairs, this bounds the number of changes of $\widehat{M}_t$.

Next, we show that we either make a change in $\widehat{M}_t$ during the next $|\mathcal{S}|$ steps or we never make any more changes. The model M is deterministic; if we do not change the policy in the next $|\mathcal{S}|$ time steps, the policy $\widehat{\pi}_\tau^* \in \Pi_{SD}$ reaches a cycle and continues on this cycle forever. Hence, the model will never change.

We showed that the number of changes is at most $|\mathcal{S}| |\mathcal{A}|$, and the time between changes is at most $|\mathcal{S}|$. This implies that after time $\tau \leq |\mathcal{S}|^2|\mathcal{A}|$ we never change.

The return of $\widehat{\pi}_\tau^*$ after time τ is identical in $\widehat{M}_\tau$ and M , since all the edges it traverses are **known**. Therefore, $\mathcal{V}^{\widehat{\pi}_\tau^*}(\mathbf{s}; M) = \mathcal{V}^{\widehat{\pi}_\tau^*}(\mathbf{s}; \widehat{M}_\tau)$. Since $\widehat{\pi}_\tau^*$ is the optimal

policy in $\widehat{M}_\tau$ we have that $\mathcal{V}^{\widehat{\pi}_\tau^*}(\mathbf{s}; \widehat{M}_\tau) \geq \mathcal{V}^{\pi^*}(\mathbf{s}; \widehat{M}_\tau)$, where π^* is the optimal policy in M . By the optimism we have $\mathcal{V}^{\pi^*}(\mathbf{s}; \widehat{M}_\tau) \geq \mathcal{V}^{\pi^*}(\mathbf{s}; M)$. We established that $\mathcal{V}^{\widehat{\pi}_\tau^*}(\mathbf{s}; M) \geq \mathcal{V}^{\pi^*}(\mathbf{s}; M)$, but due to the optimality of π^* we have $\pi^* = \widehat{\pi}_\tau^*$. $\square$

Remark 10.1. *In this section we used the infinite horizon average reward, however this is not critical. If we are interested in the finite horizon, or the discounted return, we can use them to define the optimal policy, and the claims would be almost identical.*

10.3.2 On-policy learning MDP: Explicit Explore or Exploit (E^3)

We will now extend the techniques we developed for DDP to a general MDP. We will move from infinite horizon average reward to finite horizon with a stationary MDP and fixed initial state, mainly for simplicity, however, the techniques presented can be applied to a variety of return criteria.

The main difference between a DDP and MDP is that in a DDP it is sufficient to have a single state-action sample $(\mathbf{s}, \mathbf{a})$ to know both the reward and the next state. In a general MDP we need to have a larger number of state-action samples of $(\mathbf{s}, \mathbf{a})$ to approximate it well. (Recall that to have an α -approximate model it is sufficient to have from each state-action pair $m = O(\alpha^{-2}(|\mathcal{S}| + \log(T|\mathcal{S}| |\mathcal{A}|/\delta)))$ samples.) Otherwise, the algorithms would be very similar.

For the analysis there is an additional issue. In the DDP, we reached an **unknown** state-action pair in at most $|\mathcal{S}|$ steps. For the finite horizon return that would imply that we either are guarantee to explore a new **unknown** state-action pair or that we have explored all the relevant state-action pairs. This deterministic guarantee will no longer be true in an MDP, as we have transition probabilities. For this reason, in the analysis, we will need to keep track of the probability of reaching an **unknown** state-action pair. We will show that as long as this probability is high, we keep exploring (and with a reasonable probability discovering new **unknown** state-action pair). In addition, when this probability will be small, we will show that we can compute a near optimal policy using the known part of the model.

We start with the E^3 (Explicit Explore or Exploit) algorithm [76]. The algorithm learns the MDP model by sampling each state-action pair m times. The main task would be to generate those m samples. (A technical point would be that some state-action pairs might have very low probability under any policy, such state-action pairs would be implicitly ignored.)

As in the DDP we will maintain an explored model. Given an observation set $\{(\mathbf{s}_t, \mathbf{a}_t, \mathbf{r}_t, \mathbf{s}_{t+1})\}$, we define a state-action $(\mathbf{s}, \mathbf{a})$ pair **known** if we have at least m

times $\mathbf{t}_i$, $1 \leq i \leq m$, where $\mathbf{s}_{\mathbf{t}_i} = \mathbf{s}$ and $\mathbf{a}_{\mathbf{t}_i} = \mathbf{a}$, otherwise it is **unknown**. We define the observed distribution of a **known** state-action $(\mathbf{s}, \mathbf{a})$ to be

$$\hat{\mathbf{p}}(\mathbf{s}'|\mathbf{s}, \mathbf{a}) = \frac{|\{\mathbf{t}_i : \mathbf{s}_{\mathbf{t}_i+1} = \mathbf{s}', \mathbf{s}_{\mathbf{t}_i} = \mathbf{s}, \mathbf{a}_{\mathbf{t}_i} = \mathbf{a}\}|}{m},$$

and the observed reward to be,

$$\hat{\mathbf{r}}(\mathbf{s}, \mathbf{a}) = \frac{1}{m} \sum_{i=1}^m \mathbf{r}_{\mathbf{t}_i}.$$

We define the explored model $\widetilde{M}$ as follows. We add a new state $\tilde{\mathbf{s}}$ (this state is not a “real state” but is useful for defining the algorithm and in the analysis). For each **known** state-action $(\mathbf{s}, \mathbf{a})$, we set the next state distribution $\tilde{\mathbf{p}}(\cdot|\mathbf{s}, \mathbf{a})$ to be the observed distribution $\hat{\mathbf{p}}(\cdot|\mathbf{s}, \mathbf{a})$, and the reward to be zero, i.e., $\tilde{\mathbf{r}}(\mathbf{s}, \mathbf{a}) = 0$. For **unknown** state-action $(\mathbf{s}, \mathbf{a})$, we define $\tilde{\mathbf{p}}(\mathbf{s}' = \tilde{\mathbf{s}}|\mathbf{s}, \mathbf{a}) = 1$ and $\tilde{\mathbf{r}}(\mathbf{s}, \mathbf{a}) = 1$. For state $\tilde{\mathbf{s}}$ we have $\tilde{\mathbf{p}}(\mathbf{s}' = \tilde{\mathbf{s}}|\tilde{\mathbf{s}}, \mathbf{a}) = 1$ and $\tilde{\mathbf{r}}(\tilde{\mathbf{s}}, \mathbf{a}) = 0$ for any action $\mathbf{a} \in \mathcal{A}$. The terminal reward of any state $\mathbf{s}$ is zero, i.e., $\tilde{\mathbf{r}}_{\mathbf{T}}(\mathbf{s}) = 0$. Note that the expected value of any policy π in $\widetilde{M}$ is *exactly* the probability it will reach an **unknown** state-action pair.

We can now specify the E^3 (Explicit Explore or Exploit) algorithm. The algorithm has two stages. In the **explore** stage, we collect transitions until all states become **known**. Afterwards, we build an approximate MDP from the collected transitions, and begin the **exploit** stage: act optimally according to the approximate model.

The algorithm takes as input m , how many samples we need to change a state-action from **unknown** to **known**, and $\varepsilon \in (0, 1)$, an accuracy threshold.

Algorithm 15 E^3 : Explicit Explore or Exploit Algorithm

- 1: **Parameters:**
 - 2: m — sample threshold for marking $(\mathbf{s}, \mathbf{a})$ as **known**
 - 3: T — finite horizon parameter
 - 4: $\varepsilon \in (0, 1)$ — accuracy threshold
 - 5: Initial state $\mathbf{s}_0$
 - 6: **Initialize:**
 - 7: Let $\tilde{\mathbf{s}}$ be a special absorbing placeholder state:
 - 8: $\tilde{\mathbf{p}}(\tilde{\mathbf{s}}|\tilde{\mathbf{s}}, \mathbf{a}) = 1, \quad \tilde{\mathbf{r}}(\tilde{\mathbf{s}}, \mathbf{a}) = 0$
 - 9: Construct model $\widetilde{M}_0$ as follows:
 - 10: All $(\mathbf{s}, \mathbf{a})$ are **unknown**:
 - 11: $\tilde{\mathbf{p}}(\tilde{\mathbf{s}}|\mathbf{s}, \mathbf{a}) = 1, \quad \tilde{\mathbf{r}}(\mathbf{s}, \mathbf{a}) = 1$
 - 12: **Explore Phase:**
 - 13: **For** $t = 0, 1, 2, \dots$
 - 14: Compute $\tilde{\pi}_t^*$, the optimal policy for $\widetilde{M}_t$ from $\mathbf{s}_0$ with horizon T
 - 15: **If** expected return of $\tilde{\pi}_t^*$ on $\widetilde{M}_t \leq \varepsilon/2$: **break**
 - 16: Run $\tilde{\pi}_t^*$ from $\mathbf{s}_0$ to obtain trajectory: $(\mathbf{s}_0, \mathbf{a}_0, \mathbf{r}_0, \mathbf{s}_1, \dots, \mathbf{s}_T)$
 - 17: Add to observations: $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{r}_i, \mathbf{s}_{i+1})$ for $0 \leq i < T$
 - 18: For each $(\mathbf{s}, \mathbf{a})$ newly becoming **known**:
 - 19: Update $\widetilde{M}_{t+1}$ with:
 - 20: $\tilde{\mathbf{p}}(\cdot|\mathbf{s}, \mathbf{a}) = \widehat{\mathbf{p}}(\cdot|\mathbf{s}, \mathbf{a})$
 - 21: $\tilde{\mathbf{r}}(\mathbf{s}, \mathbf{a}) = 0$
 - 22: **Exploit Phase**
 - 23: Construct MDP M' as follows:
 - 24: For **known** $(\mathbf{s}, \mathbf{a})$: empirical $\widehat{\mathbf{p}}(\cdot|\mathbf{s}, \mathbf{a})$ and $\widehat{\mathbf{r}}(\mathbf{s}, \mathbf{a})$
 - 25: For **unknown** $(\mathbf{s}, \mathbf{a})$: $\widehat{\mathbf{p}}(\mathbf{s}|\mathbf{s}, \mathbf{a}) = 1, \quad \widehat{\mathbf{r}}(\mathbf{s}, \mathbf{a}) = R_{\max}$
-

Theorem 10.18. Let $m \geq \frac{|S| + \log(|S| |\mathcal{A}|/\delta)}{\alpha^2}$ and $\alpha = \frac{\varepsilon/4}{R_{\max} T^2}$. The E^3 (Explicit Explore or Exploit) algorithm recovers an MDP M' , such that for any policy π the expected return on M' and M differ by at most $\varepsilon(TR_{\max} + 1)$, i.e.,

$$|\mathcal{V}_{M'}^\pi(\mathbf{s}_0) - \mathcal{V}_M^\pi(\mathbf{s}_0)| \leq \varepsilon TR_{\max} + \varepsilon.$$

In addition, the expected number of time steps until termination is at most $O(mT|S| |\mathcal{A}|/\varepsilon)$

Proof. We set the sample size m such that, with probability at least $1 - \delta$, we have that for every state $\mathbf{s}$ and action $\mathbf{a}$ we have that both the observed and true next

state distribution are α close and the difference between the observed and true reward is at most α . Namely, $\|p(\cdot|\mathbf{s}, \mathbf{a}) - \widehat{p}(\cdot|\mathbf{s}, \mathbf{a})\|_1 \leq \alpha$ and $|\mathbf{r}(\mathbf{s}, \mathbf{a}) - \widehat{\mathbf{r}}(\mathbf{s}, \mathbf{a})| \leq \alpha$. By Lemma 10.13, it is sufficient to have that $m \geq c \frac{|S| + \log(|S| |\mathcal{A}|/\delta)}{\alpha^2}$, for some $c > 0$.

Let $\widetilde{M}_{\mathbf{t}}$ be the model at time $\mathbf{t}$. We define an intermediate model $\widetilde{M}'_{\mathbf{t}}$ to be the model where we replace the observed next-state distributions with the true next state distributions for the **known** state-action pairs. Since the two models are α -approximate, by Theorems 10.9 and 10.10, their expected return differ by at most $\alpha T^2 R_{\max} \leq \varepsilon/4$.

Note that the probability of reaching some **unknown** state in the true model M and the intermediate model $\widetilde{M}'_{\mathbf{t}}$ at time $\mathbf{t}$ is identical. This is since the two models agree on the **known** states, and once an **unknown** state is reached, we are done.

We will show that as long as the probability of reaching some **unknown** state in the true model is large (larger than 0.75ε) we will not terminate. This will guarantee that when we terminate the probability of reaching any **unknown** state is negligible, and hence we can conceptually ignore such states and still be near optimal. The second part is to show that we do terminate and bound the expected time until termination. For this part we will show that once every policy has a low probability of reaching some **unknown** state in the true model (less than 0.25ε) then we will terminate.

Assume there is a policy π that at time $\mathbf{t}$ in the true model M has a probability of at least $(3/4)\varepsilon$ to reach an **unknown** state. (Note that the sets of **known** and **unknown** states change with $\mathbf{t}$.) Recall that this implies that π has the same probability in $\widetilde{M}'_{\mathbf{t}}$. Therefore, this policy π has a probability of at least $(1/2)\varepsilon$ to reach an **unknown** state in $\widetilde{M}_{\mathbf{t}}$ since $\widetilde{M}'_{\mathbf{t}}$ and $\widetilde{M}_{\mathbf{t}}$ are α -approximate. This implies that we will not terminate while there is such a policy π .

Similarly, once at time $\mathbf{t}$, every policy π in the true model M has a probability of at most $(1/4)\varepsilon$ to reach an **unknown** state, then we are guaranteed to terminate. This is since the probability of π to reach an **unknown** state is identical in M and $\widetilde{M}'_{\mathbf{t}}$. Since the expected return of π in $\widetilde{M}'_{\mathbf{t}}$ and $\widetilde{M}$ differ by at most $\varepsilon/4$, the probability of π to reach an **unknown** state in $\widetilde{M}_{\mathbf{t}}$ is at most $\varepsilon/2$. This is exactly our termination condition, and we will terminate.

Assume termination at time $\mathbf{t}$. At time $\mathbf{t}$ every policy π has a probability of at most $(1/2)\varepsilon$ to reach some **unknown** state in $\widetilde{M}_{\mathbf{t}}$. This implies that π has a probability of at most $(3/4)\varepsilon$ to reach some **unknown** state in M .

After the algorithm terminates, we define the model M' using the observed distributions and rewards for any **known** state-action pair. Since every **known** state-action pair is sampled m times, we have that with probability $1 - \delta$ the model M' is an α -approximation of the true model M , in the **known** state-action pairs.

When we compare $|\mathcal{V}_{M'}^\pi(\mathbf{s}_0) - \mathcal{V}_M^\pi(\mathbf{s}_0)|$ we separate the difference due to trajectories that include **unknown** states and due to trajectories in which all the states are **known** states. The contribution of trajectories with **unknown** states is at most $\varepsilon \text{TR}_{\max}$, since the probability of reaching any **unknown** state is at most $(3/4)\varepsilon < \varepsilon$ and the maximum return is $\text{TR}_{\max}$. The difference in trajectories in which all the states are **known** states is at most $\varepsilon/4 < \varepsilon$ since M and M' are α approximate, and the selection of α guarantees that the difference in expectation is at most $\varepsilon/4$ (Lemma 10.11).

In each iteration, until we terminate, we have a probability of at least $\varepsilon/4$ to reach some **unknown** state-action. We can reach unknown state-action pairs at most $m|\mathcal{S}| |\mathcal{A}|$ times, before they become **known**. Therefore the expected number of time steps is $O(mT|\mathcal{S}| |\mathcal{A}|/\varepsilon)$. $\square$

10.3.3 On-policy learning MDP: R-MAX

In this section we introduce **R-max** [25]. The main difference between **R-MAX** and E^3 is that **R-MAX** will have a single continuous phase, and there will be no need to explicitly switch from exploration to exploitation.

Similar to the DDP, we will use the principle of *Optimism in face of uncertainty*. Namely, we substitute the **unknown** quantities by the maximum possible values. In addition, similar to DDP and E^3 , we will partition the state-action pairs $(\mathbf{s}, \mathbf{a})$ to **known** and **unknown**. The main difference from DDP, and similar to E^3 , is that in a DDP it is sufficient to have a single sample to move $(\mathbf{s}, \mathbf{a})$ from **unknown** to **known**. In a general MDP we need to have a larger sample to move $(\mathbf{s}, \mathbf{a})$ from **unknown** to **known**.

We can now specify the **R-MAX** algorithm. The algorithm takes as input m , how many samples we need to change a state-action from **unknown** to **known**

Algorithm 16 R-MAX: Optimistic On-Policy Model-Based RL

- 1: **Parameters:**
 - 2: m — number of samples needed for $(\mathbf{s}, \mathbf{a})$ to become **known**
 - 3: T — finite horizon parameter
 - 4: Initial state $\mathbf{s}_0$
 - 5: **Initialization:**
 - 6: Build model $\widehat{M}_0$: for all $(\mathbf{s}, \mathbf{a})$:
 - 7: Set $p(\mathbf{s}'|\mathbf{s}, \mathbf{a}) = 1$ if $\mathbf{s}' = \mathbf{s}$, else 0 (states return to themselves)
 - 8: Set $r(\mathbf{s}, \mathbf{a}) = R_{\max}$
 - 9: Mark $(\mathbf{s}, \mathbf{a})$ as **unknown**
 - 10: **For** $t = 0, 1, 2, \dots$
 - 11: Compute $\widehat{\pi}_t^*$, the optimal policy for $\widehat{M}_t$ with horizon T
 - 12: Execute $\widehat{\pi}_t^*$ to generate trajectory: $(\mathbf{s}_0, \mathbf{a}_0, \mathbf{r}_0, \mathbf{s}_1, \dots, \mathbf{s}_T)$
 - 13: Add to observations: $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{r}_i, \mathbf{s}_{i+1})$ for $0 \leq i < T$
 - 14: Update model $\widehat{M}_t$ to $\widehat{M}_{t+1}$: for each $(\mathbf{s}, \mathbf{a})$:
 - 15: **If** $(\mathbf{s}, \mathbf{a})$ is **unknown** and observed at least m times:
 - 16: Set $p(\cdot|\mathbf{s}, \mathbf{a}) = \widehat{p}(\cdot|\mathbf{s}, \mathbf{a})$
 - 17: Set $r(\mathbf{s}, \mathbf{a}) = \widehat{r}(\mathbf{s}, \mathbf{a})$
 - 18: Mark $(\mathbf{s}, \mathbf{a})$ as **known**
-

Note that we update each $(\mathbf{s}, \mathbf{a})$ only once, when it moves from **unknown** to **known**.

There are two main differences between R-MAX and E^3 . First, when a state-action becomes **known**, we set the reward to be the observed reward (and not zero, as in E^3). Second, there is no test for termination, but we continuously run the algorithm (although at some point the policy will stop changing).

Here is the basic intuition for algorithm R-MAX. We consider the finite horizon return with horizon T . In each episode we run $\widehat{\pi}_t^*$ for T time steps. Either, with some non-negligible probability we explore a state-action $(\mathbf{s}, \mathbf{a})$ which is **unknown**, in this case we make progress on the exploration. This can happen at most $m|\mathcal{S}||\mathcal{A}|$ times. Alternatively, with high probability we do not reach any state-action $(\mathbf{s}, \mathbf{a})$ which is **unknown**, in which case we are optimal on the observed model, and near optimal on the true model.

For the analysis define an event NEW_t , which is the event that we visit some **unknown** state-action $(\mathbf{s}, \mathbf{a})$ during the iteration t .

Claim 10.19. *With probability $1 - \delta$, for the return of $\widehat{\pi}_t^*$, we have,*

$$\mathcal{V}^{\widehat{\pi}_t^*}(\mathbf{s}_0) \geq \mathcal{V}^*(\mathbf{s}_0) - \Pr[NEW_t]TR_{\max} - \lambda,$$

where λ is the approximation error for any two models which are α -approximate.

Proof. Let π^* be the optimal policy in the true model M . Since we selected policy $\hat{\pi}_t^*$ for our model $\widehat{M}_t$, we have $\mathcal{V}^{\hat{\pi}_t^*}(\mathbf{s}_0; \widehat{M}_t) \geq \mathcal{V}^{\pi^*}(\mathbf{s}_0; \widehat{M}_t)$.

We now define an intermediate model $\widehat{M}'_t$ which replaces the transitions and rewards in the **known** state-action pairs by the true transition probabilities and rewards. We have that $\widehat{M}'_t$ and $\widehat{M}_t$ are α -approximate, with probability at least $1 - \delta$. By the definition of λ we have $\mathcal{V}^{\pi^*}(\mathbf{s}_0; \widehat{M}_t) \geq \mathcal{V}^{\pi^*}(\mathbf{s}_0; \widehat{M}'_t) - \lambda$. In addition, $\mathcal{V}^{\pi^*}(\mathbf{s}_0; \widehat{M}'_t) \geq \mathcal{V}^{\pi^*}(\mathbf{s}_0; M) = \mathcal{V}^*(\mathbf{s}_0)$, since in $\widehat{M}'_t$ we only increased the rewards of the **unknown** state-action pairs such that when we reach them we are guarantee maximal rewards until the end of the trajectory.

For our policy $\hat{\pi}_t^*$ we have that $\mathcal{V}^{\hat{\pi}_t^*}(\mathbf{s}_0; \widehat{M}'_t) + \lambda \geq \mathcal{V}^{\hat{\pi}_t^*}(\mathbf{s}_0; \widehat{M}_t)$, since the models are α -approximate. In M and $\widehat{M}'_t$, any trajectory that does not reach any **unknown** state-action pair, has the same probability in both models. This implies that $\mathcal{V}^{\hat{\pi}_t^*}(\mathbf{s}_0; M) \geq \mathcal{V}^{\hat{\pi}_t^*}(\mathbf{s}_0; \widehat{M}'_t) - \Pr[NEW_t] \text{TR}_{\max}$, since the maximum return is $\text{TR}_{\max}$. Combining all the inequalities derives the claim. $\square$

We set the sample size m such that $\lambda \leq \varepsilon/2$.

We consider two cases, depending on the probability of NEW_t . First, we consider the case that the probability of NEW_t is small. If $\Pr[NEW_t] \leq \frac{\varepsilon}{2\text{TR}_{\max}}$, then $\mathcal{V}^{\hat{\pi}_t^*}(\mathbf{s}_0) \geq \mathcal{V}^*(\mathbf{s}_0) - \varepsilon/2 - \varepsilon/2$, since we assume that $\lambda \leq \varepsilon/2$.

Second, we consider the case that the probability of NEW_t is large. If $\Pr[NEW_t] > \frac{\varepsilon}{2\text{TR}_{\max}}$. Then, there is a good probability to visit an **unknown** state-action pair $(\mathbf{s}, \mathbf{a})$, but this can happen at most $m|\mathcal{S}| |\mathcal{A}|$ times. Therefore, the expected number of such iterations is at most $m|\mathcal{S}| |\mathcal{A}| \frac{2\text{TR}_{\max}}{\varepsilon}$. Using the Chernoff-Hoeffding bound in the multiplicative form (Lemma 10.2 with $\varepsilon = 1/2$), this implies the following theorem.

Theorem 10.20. *With probability at least $1 - \delta$, algorithm **R-MAX** will be ε -optimal, i.e., have an expected return at least $\mathcal{V}^* - \varepsilon$, in all iterations except for at most*

$$24m|\mathcal{S}| |\mathcal{A}| \frac{\text{TR}_{\max}}{\varepsilon} \log \left(\frac{1}{\delta} \right)$$

iterations.

Remark: Note that we do not guarantee a termination after which we can fix the policy. The main technical issue that we have is that the probability of the event NEW_t is not monotone non-increasing. This is since when we switch policies, we might considerably increase the probability of reaching **unknown** state-action pairs.

For this reason we settle for a weaker guarantee that the number of sub-optimal iterations is bounded. Note that in E^3 we separated the exploration and exploitation and have a clear transition between the two, and therefore we can terminate and output a near-optimal policy.

10.4 Bibliography Notes

The simplest model for model-based reinforcement learning is the generative model, which allows to sample directly state-action pairs, without the need for exploration. The generative model was first introduced by Kearns and Singh [75]. This work also introduced the approximate value iteration to establish the reduced sample complexity for the optimal policy, as shown in Theorem 10.15. The work of Azar et al. [7] give both a upper and lower bounds of $\Theta\left(\frac{R_{max}^2|\mathcal{S}||\mathcal{A}|}{(1-\gamma)^3\varepsilon^2}\log\frac{|\mathcal{S}||\mathcal{A}|}{\delta}\right)$.

The work of [36] gives a Probably Approximately Correct (PAC) bound for reinforcement learning, for the finite horizon, an upper bound of $\tilde{O}\left(\frac{R_{max}^2|\mathcal{S}|^2T^2|\mathcal{A}|}{\varepsilon^2}\log(1/\delta)\right)$ and lower bound of $\tilde{\Omega}\left(\frac{R_{max}^2|\mathcal{S}||\mathcal{A}|T^2}{\varepsilon^2}\right)$. Other PAC bounds for MDPs include [153, 154, 93].

The PhD thesis of Kakade [67] introduced the PAC-MDP model. The model considers the number of episodes in which the learner's policy expected value is worse than ϵ away from the optimal value. The R-MAX algorithm [25] was presented before the introduction of the PAC-MDP model, although conceptually it falls in this category. The PAC-MDP model has been further studied in [147, 146, 98]. The analysis of the R-MAX algorithm as a PAC-MDP algorithm appears in [147, 153].

Another line of model-based learning algorithms is based on learning the dynamics, without considering the rewards. Later, the learner can adapt to any reward function and derive an optimal policy for it, which is also named "Best Policy Identification (BPI)". The first work in this direction is [48] which gives an efficient algorithm for the discounted return, with a reset assumption. The Explicit Explore or Exploit (E^3) algorithm of [76] improves in both allowing a wide range of return functions and does not need the reset assumption.

The term "reward free exploration" is due to [65] which give a polynomial complexity using a reduction to online learning. The work of [73] improves the bound, and their algorithm is based on that of [48], and show that $O(|\mathcal{S}|^2|\mathcal{A}|T^4\log(1/\delta)/\varepsilon^2)$ episodes learns a near optimal model. This bound was improved in [108], reducing the dependency on the horizon from T^4 to T^3 .

Chapter 11

Reinforcement Learning: Model Free

In this chapter, we consider model-free learning algorithms. The main idea of model-free algorithms is to avoid learning the MDP model directly. The model-based methodology was the following. During the learning, we estimate model of the MDP, and later, we derive the optimal policy of the estimated model. The main point was that an optimal policy of a near-accurate MDP is a near-optimal policy in the true MDP.

The model-free methodology will be different. We will never learn an estimated model, but rather we will directly learn the value function of the MDP. The value function can be either the Q -function (as is the case in Q -learning and SARSA) or the V -function (as is the case in Temporal Difference (TD) algorithms and the Monte Carlo approach).

We will first look at the case of deterministic MDPs, and develop a Q -learning algorithm that learns the Q -function directly from interaction with the MDP. We will then extend our approach to general MDPs, where our handling of stochasticity will be based on the *stochastic approximation* technique. We will first look at learning V^π for a fixed policy, using either temporal difference or Monte-Carlo methods, and then look at learning the optimal Q -function, using the Q -learning and SARSA methods. At the end of the chapter we have a few miscellaneous topics, including, evaluating one policy while following a different policy (using importance sampling) and the distinction between on-policy and off-policy algorithms.

Chapter Orientation

Main Prerequisites:

- Discounted MDPs (Chapter 6)

Main terminology:

- $\widehat{Q}_t$ estimate of the Q function at iteration t of the learning algorithm
- $\widehat{V}_t$ estimate of the V function at iteration t of the learning algorithm, at iteration t
- α step size in a learning algorithm
- $\alpha_t(s)$ step size that depends on the iteration and state
- G observed sum of rewards in a trajectory
- G^s observed sum of rewards starting from the first visit to state s
- G^{sj} observed sum of rewards starting from time step j
- $G_{i,j}$ observed sum of rewards in episode i starting from time step j
- ε random action probability in a ε -greedy policy
- X_t vector of parameters in the stochastic approximation algorithm at iteration t
- ω_t noise term in the stochastic approximation algorithm at iteration t
- h_{t-1} history up to time t in the stochastic approximation algorithm
- H contraction mapping in the stochastic approximation algorithm
- X^* fixed point of the stochastic approximation algorithm
- $\mathcal{D}$ replay buffer
- Δ_t temporal difference at time t
- $\Delta_t^{(n)}$ n -step temporal difference at time t

11.1 Model Free Learning – the Situated Agent Setting

This chapter mostly concerns the discounted MDP model, and we follow the notations of Chapter 6.

The learning setting we consider involves an agent that sequentially *interacts* with an MDP, where by interaction we mean that at time t the agent can observe the current state $\mathbf{s}_t$, the current action $\mathbf{a}_t$, the current reward $\mathbf{r}_t = \mathbf{r}(\mathbf{s}_t, \mathbf{a}_t)$, and the resulting next state $\mathbf{s}_{t+1} \sim P(\cdot | \mathbf{s}_t, \mathbf{a}_t)$. Throughout the interaction, the agent collects *transition tuples*, $(\mathbf{s}_t, \mathbf{a}_t, \mathbf{r}_t, \mathbf{s}_{t+1})$, which will effectively be the *data* used for learning the MDP’s value function. That is, all our learning algorithms will take as input transition tuples, and output estimates of value functions. For some algorithms, the time index of tuples in the data is not important, and we shall sometimes denote the tuples as $(\mathbf{s}, \mathbf{a}, \mathbf{r}, \mathbf{s}')$, understanding that both notations above are equivalent.

As with any learning method, the data we learn from has substantial influence on what we can ultimately learn. In our setting, the agent can *control* the data distribution through its choice of actions. For example, if the agent chooses actions according to a Markov policy π , we should expect to obtain tuples that roughly follow the stationary distribution of the Markov chain corresponding to π . If π is very different from the optimal policy, for example, this data may not be very useful for estimating $\mathcal{V}^*$. Therefore, different from the supervised machine learning methodology, in reinforcement learning the agent must consider both how to learn from data, but also how to *collect* it. As we shall see, the agent will need to *explore* the MDP’s state space in its data collection, to guarantee that the optimal value function can be learned. In this chapter we shall devise several heuristics for effective exploration. In proceeding chapters we will dive deeper into how to provably explore effectively.

11.2 Q-learning: Deterministic Decision Process

The celebrated Q-learning algorithm is among the most popular and fundamental model-free RL methods. To demonstrate some key ideas of Q-learning, we start with a simplified learning algorithm that is suitable for a *Deterministic Decision Process (DDP)* model, namely:

$$\mathbf{s}_{t+1} = f(\mathbf{s}_t, \mathbf{a}_t) \quad \text{and} \quad \mathbf{r}_t = \mathbf{r}(\mathbf{s}_t, \mathbf{a}_t).$$

We have already discussed DDPs in Chapter 3.

We consider the discounted return criterion:

$$\mathcal{V}^\pi(\mathbf{s}) = \sum_{t=0}^{\infty} \gamma^t \mathbf{r}(\mathbf{s}_t, \mathbf{a}_t), \quad \text{given } \mathbf{s}_0 = \mathbf{s}, \mathbf{a}_t = \pi(\mathbf{s}_t)$$

$$\mathcal{V}^*(\mathbf{s}) = \max_{\pi} \mathcal{V}^\pi(\mathbf{s}),$$

where $\mathcal{V}^*$ is the value function of the optimal policy.

Recall our definition of the $\mathcal{Q}$ -function (or *state-action value function*), specialized to the present deterministic setting:

$$\mathcal{Q}^*(\mathbf{s}, \mathbf{a}) = \mathbf{r}(\mathbf{s}, \mathbf{a}) + \gamma \mathcal{V}^*(f(\mathbf{s}, \mathbf{a})).$$

The optimality equation is then

$$\mathcal{V}^*(\mathbf{s}) = \max_{\mathbf{a}} \mathcal{Q}^*(\mathbf{s}, \mathbf{a}),$$

or, in terms of $\mathcal{Q}^*$:

$$\mathcal{Q}^*(\mathbf{s}, \mathbf{a}) = \mathbf{r}(\mathbf{s}, \mathbf{a}) + \gamma \max_{\mathbf{a}'} \mathcal{Q}^*(f(\mathbf{s}, \mathbf{a}), \mathbf{a}').$$

The $\mathcal{Q}$ -learning algorithm runs as follows:

Algorithm 17 $\mathcal{Q}$ -learning (for deterministic decision processes)

- 1: **Initialize:** Set $\hat{\mathcal{Q}}_0(\mathbf{s}, \mathbf{a}) = 0$, for all $\mathbf{s}, \mathbf{a}$.
- 2: **For** $t = 0, 1, 2, \dots$
- 3: Select action $\mathbf{a}_t$
- 4: Observe $(\mathbf{s}_t, \mathbf{a}_t, \mathbf{r}_t, \mathbf{s}'_t)$, where $\mathbf{s}'_t = f(\mathbf{s}_t, \mathbf{a}_t)$.
- 5: Update:

$$\hat{\mathcal{Q}}_{t+1}(\mathbf{s}_t, \mathbf{a}_t) = \mathbf{r}_t + \gamma \max_{\mathbf{a}'} \hat{\mathcal{Q}}_t(\mathbf{s}'_t, \mathbf{a}'),$$

$$\hat{\mathcal{Q}}_{t+1}(\mathbf{s}, \mathbf{a}) = \hat{\mathcal{Q}}_t(\mathbf{s}, \mathbf{a}) \quad \forall \mathbf{s}, \mathbf{a} \neq \mathbf{s}_t, \mathbf{a}_t.$$

We emphasize that throughout this chapter we consider an infinite horizon discounted setting, and in the notation $\hat{\mathcal{Q}}_t(\mathbf{s}, \mathbf{a})$ the subscript t denotes the iteration of the learning algorithm (and not the decision time step in a finite horizon setting).

The update in $\mathcal{Q}$ -learning is an example of a technique that is often termed “bootstrapping”¹, where we use our current, and possibly inaccurate, value estimate $\hat{\mathcal{Q}}_t$, to

¹Related to the saying “to pull oneself up by one’s bootstraps”.

improve the accuracy of the same value function. The intuition for why this makes sense comes from the discount factor: Since in the update $\mathbf{r}_t + \gamma \max_{\mathbf{a}'} \hat{Q}_t(\mathbf{s}'_t, \mathbf{a}')$ the first term is accurate (the reward is not estimated), and the second term is multiplied by the discount factor, we expect that if $\gamma < 1$ our updated value would suffer less from the inaccuracy.

The Q -learning algorithm is an off-policy algorithm, namely, it does not specify how to choose the actions $\mathbf{a}_t$, and this can be done using various exploration methods. To guarantee convergence of Q -learning, we need to have some assumption about the sequence of actions selected, as is the case in the theorem below. We shall later discuss exploration methods that satisfy this assumption.

Theorem 11.1 (Convergence of Q -learning for DDP).

Assume a DDP model. If each state-action pair is visited infinitely-often, then $\lim_{t \rightarrow \infty} \hat{Q}_t(\mathbf{s}, \mathbf{a}) = Q^(\mathbf{s}, \mathbf{a})$, for all $(\mathbf{s}, \mathbf{a})$.*

Proof. The proof would be done by considering the maximum difference between $\hat{Q}_t$ and Q^* . Let

$$\delta_t \triangleq \|\hat{Q}_t - Q^*\|_\infty = \max_{\mathbf{s}, \mathbf{a}} |\hat{Q}_t(\mathbf{s}, \mathbf{a}) - Q^*(\mathbf{s}, \mathbf{a})|.$$

The first step is to show that after an update at time t the difference at the updated state-action $(\mathbf{s}_t, \mathbf{a}_t)$ between $\hat{Q}_t$ and Q^* can be bounded by $\gamma\delta_t$. This does not imply that δ_t would shrink, since it is the maximum over all state-action pair. Later we show that eventually, after we update each state-action pairs at least once, then we are guaranteed to have the difference shrink by a factor of at least γ .

First, at every stage t :

$$\begin{aligned} |\hat{Q}_{t+1}(\mathbf{s}_t, \mathbf{a}_t) - Q^*(\mathbf{s}_t, \mathbf{a}_t)| &= |(r_t + \gamma \max_{\mathbf{a}'} \hat{Q}_t(\mathbf{s}'_t, \mathbf{a}')) - (r_t + \gamma \max_{\mathbf{a}''} Q^*(\mathbf{s}'_t, \mathbf{a}''))| \\ &= \gamma |\max_{\mathbf{a}'} \hat{Q}_t(\mathbf{s}'_t, \mathbf{a}') - \max_{\mathbf{a}''} Q^*(\mathbf{s}'_t, \mathbf{a}'')| \\ &\leq \gamma \max_{\mathbf{a}'} |\hat{Q}_t(\mathbf{s}'_t, \mathbf{a}') - Q^*(\mathbf{s}'_t, \mathbf{a}')| \\ &\leq \gamma \delta_t, \end{aligned}$$

where the first inequality uses the fact that $|\max_{x_1} f_1(x_1) - \max_{x_2} f_2(x_2)| \leq \max_x |f_1(x) - f_2(x)|$, and the second inequality follows from the bound on $\|\hat{Q}_t - Q^*\|_\infty = \delta_t$. This implies that the difference at $(\mathbf{s}_t, \mathbf{a}_t)$ is bounded by $\gamma\delta_t$, but this does not imply that $\delta_{t+1} \leq \gamma\delta_t$, since it is the maximum over all state-action pairs.

Next, we show that eventually $\delta_{t+\tau}$ would be at most $\gamma\delta_t$. Consider now some interval $[t, t_1]$ over which each state-action pairs $(\mathbf{s}, \mathbf{a})$ appear at least once. Using

the above relation and simple induction, it follows that $\delta_{t_1} \leq \gamma \delta_t$. Since each state-action pair is visited infinitely often, there is an infinite number of such intervals, and since $\gamma < 1$, it follows that $\delta_t \rightarrow 0$, as t goes to infinity. $\square$

Remark 11.1. *Note that the Q -learning algorithm does not need to receive a continuous trajectory, but can receive arbitrary quadruples $(\mathbf{s}_t, \mathbf{a}_t, \mathbf{r}_t, \mathbf{s}'_t)$. We do need that for any state-action pair $(\mathbf{s}, \mathbf{a})$ we have infinitely many times t for which $\mathbf{s}_t = \mathbf{s}$ and $\mathbf{a}_t = \mathbf{a}$. That said, a practical method for obtaining the samples is by continuously taking actions in the MDP and using the t 's quadruple in the observed trajectory $\mathbf{s}_0, \mathbf{a}_0, \mathbf{r}_0, \mathbf{s}_1, \mathbf{a}_1, \mathbf{r}_1 \dots$ for the t 's update in algorithm.*

Remark 11.2. *We could have also relaxed the update to use a step-size $\alpha \in (0, 1)$ as follows: $\hat{Q}_{t+1}(\mathbf{s}_t, \mathbf{a}_t) := (1 - \alpha)\hat{Q}_t(\mathbf{s}_t, \mathbf{a}_t) + \alpha \left(\mathbf{r}_t + \gamma \max_{\mathbf{a}'} \hat{Q}_t(\mathbf{s}'_t, \mathbf{a}') \right)$. The proof follows similarly, only with a bound $|\hat{Q}_{t+1}(\mathbf{s}_t, \mathbf{a}_t) - Q^*(\mathbf{s}_t, \mathbf{a}_t)| \leq (1 - \alpha(1 - \gamma)) \delta_t$, and it is clear that $(1 - \alpha(1 - \gamma)) < 1$ when $\gamma < 1$. For the deterministic case, there is no reason to choose $\alpha < 1$. However, we shall see that taking smaller update steps will be important in the non-deterministic setting.*

Remark 11.3. *We note that in the model based setting, if we have a single sample for each state-action pair $(\mathbf{s}, \mathbf{a})$, then we can completely reconstruct the DDP. The challenge in the model free setting is that we are not reconstructing the model, but rather running a direct approximation of the value function. The DDP model is used here mainly to give intuition to the challenges that we will later encounter in the MDP model.*

Let us complete the introduction of Q learning with a discussion on how to select actions that are ‘exploratory’ enough. An obvious choice would be to select actions randomly, e.g., from a uniform distribution. However, that would mean that most of the data samples would correspond to sub-optimal decisions. The following simple strategy balances between choosing actions that seem optimal with respect to the current Q estimate and random actions.

Definition 11.1 (ε -greedy policy). *With probability $1 - \varepsilon$ select $\mathbf{a}_t \in \arg \max_{\mathbf{a}} \hat{Q}_t(\mathbf{s}_t, \mathbf{a})$. Otherwise, select $\mathbf{a}_t$ randomly from a uniform distribution over $\mathcal{A}$.*

Clearly, for any $\varepsilon > 0$ the ε -greedy policy will guarantee that all actions are sampled infinitely often. Later in this chapter we will analyze this strategy in more detail.

11.3 Monte-Carlo Algorithms

We shall now investigate model-free learning in MDPs. We shall start with the simplest setting – policy evaluation, using a simple estimation technique that is often termed Monte-Carlo.

Monte-Carlo methods learn directly from experience in a model free way. The idea is very simple. In order to estimate the value of a state under a given policy, i.e., $\mathcal{V}^\pi(\mathbf{s})$, we consider trajectories of the policy π from state $\mathbf{s}$ and average them. The method does not assume any dependency between the different states, and does not even assume a Markovian environment, which is both a plus (less assumptions) and a minus (longer time to learn – a non-Markovian environment could, for example, have the reward in a state depend on the number of visits to the state in the episode). We will concentrate on the case of an episodic MDP, namely, generating finite length episodes in each trajectory. A special case of an episodic MDP is a finite horizon return, where all the episodes have the same length.

Assume we have a fixed policy π , which for each state $\mathbf{s}$ selects action $\mathbf{a}$ with probability $\pi(\mathbf{a}|\mathbf{s})$. Using π we generate an episode $(\mathbf{s}_1, \mathbf{a}_1, \mathbf{r}_1, \dots, \mathbf{s}_\tau, \mathbf{a}_\tau, \mathbf{r}_\tau)$. The observed return of the episode is $G = \sum_{i=1}^{\tau} \mathbf{r}_i$. We are interested in the expected return of an episode conditioned on the initial state, i.e., $\mathcal{V}^\pi(\mathbf{s}) = \mathbb{E}[\sum_{i=1}^{\tau} \mathbf{r}_i | \mathbf{s}_1 = \mathbf{s}]$. Recall that the length of the episode, τ , is a random variable.

Fix a state $\mathbf{s}$, and assume we observed returns $G_1^{\mathbf{s}}, \dots, G_m^{\mathbf{s}}$, all starting at state $\mathbf{s}$. The Monte-Carlo estimate for the state $\mathbf{s}$ would be $\hat{\mathcal{V}}^\pi(\mathbf{s}) = \frac{1}{m} \sum_{i=1}^m G_i^{\mathbf{s}}$. The main issue that remains is how do we generate the samples $G_i^{\mathbf{s}}$ for a state $\mathbf{s}$. Clearly, if we assume we can reset the MDP to any state, we are done. However, such a *reset assumption* is not realistic in many applications. For this reason, we do not want to assume that we can reset the MDP to any state $\mathbf{s}$ and start an episode from it.

11.3.1 Generating the samples

Initial state only We use only the initial state of the episode. Namely, given an episode $(\mathbf{s}_1, \mathbf{a}_1, \mathbf{r}_1, \dots, \mathbf{s}_\tau, \mathbf{a}_\tau, \mathbf{r}_\tau)$ we update only $\hat{\mathcal{V}}^\pi(\mathbf{s}_1)$. This is clearly an unbiased estimate, but has many drawbacks. First, it is most likely not the case that every state can be an initial state. Second, it seems very wasteful to update only a single state per episode.

First visit We update every state that appears in the episode, but update it only once. Given an episode $(\mathbf{s}_1, \mathbf{a}_1, \mathbf{r}_1, \dots, \mathbf{s}_\tau, \mathbf{a}_\tau, \mathbf{r}_\tau)$ for each state $\mathbf{s}$ that appears in the episode, we consider the first appearance of $\mathbf{s}$, say $\mathbf{s}_j$, and update $\hat{\mathcal{V}}^\pi(\mathbf{s})$ using

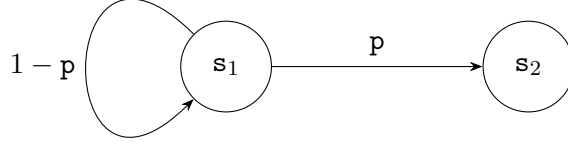


Figure 11.1: First vs. every visit example

$G^{\mathbf{s}} = \sum_{i=j}^{\tau} \mathbf{r}_i$. Namely, we compute the actual return from the first visit to state $\mathbf{s}$, and use it to update our approximation. This is clearly an unbiased estimator of the return from state $\mathbf{s}$, e.g., $\mathbb{E}[G^{\mathbf{s}}] = \mathcal{V}^{\pi}(\mathbf{s})$.

Every visit We do an update at each step of the episode. Namely, given an episode $(\mathbf{s}_1, \mathbf{a}_1, \mathbf{r}_1, \dots, \mathbf{s}_{\tau}, \mathbf{a}_{\tau}, \mathbf{r}_{\tau})$ for each state $\mathbf{s}_j$ that appears in the episode, we update each $\hat{\mathcal{V}}^{\pi}(\mathbf{s}_j)$ using $G^{\mathbf{s}_j} = \sum_{i=j}^{\tau} \mathbf{r}_i$. We compute the actual return from every state $\mathbf{s}_j$ until the end and use it to update our approximation. Note that a state can be updated multiple times in a single episode using this approach. We will later show that this estimator is biased, due to the dependency between different updates of the same state in the same episode.

First versus Every visit: To better understand the difference between first visit and every visit we consider the following simple test case. We have a two state MDP, actually a Markov Chain. In the initial state $\mathbf{s}_1$ we have a reward of 1 and with probability $1 - p$ we stay in that state and with probability p move to the terminating state $\mathbf{s}_2$. See Figure 11.1.

The expected value is $\mathcal{V}(\mathbf{s}_1) = 1/p$, which is the expected duration of an episode. (Note that the return of an episode is its length, since all the rewards are 1.) Assume that we observe a single trajectory, $(\mathbf{s}_1, \mathbf{s}_1, \mathbf{s}_1, \mathbf{s}_1, \mathbf{s}_2)$, and all the rewards are 1. What would be a reasonable estimate for the expected return from $\mathbf{s}_1$?

First visit takes the naive approach, considers the return from the first occurrence of $\mathbf{s}_1$, which is 4, and uses this as an estimate. **Every visit** considers four runs from state $\mathbf{s}_1$, we have: $(\mathbf{s}_1, \mathbf{s}_1, \mathbf{s}_1, \mathbf{s}_1, \mathbf{s}_2)$ with return 4, $(\mathbf{s}_1, \mathbf{s}_1, \mathbf{s}_1, \mathbf{s}_2)$ with return 3, $(\mathbf{s}_1, \mathbf{s}_1, \mathbf{s}_2)$ with return 2, and $(\mathbf{s}_1, \mathbf{s}_2)$ with return 1. **Every visit** averages the four

and has $G = (4 + 3 + 2 + 1)/4 = 2.5$. On the face of it, the estimate of 4 seems to make more sense. We will return to this example later.

11.3.2 First visit

Consider the First Visit Monte-Carlo updates. Assume that for state $\mathbf{s}$ we have updates $G_1^{\mathbf{s}}, \dots, G_m^{\mathbf{s}}$. Our estimate would be $\hat{\mathcal{V}}^\pi(\mathbf{s}) = (1/m) \sum_{i=1}^m G_i^{\mathbf{s}}$. Since the different $G_i^{\mathbf{s}}$ are independent, we can use a concentration bound to claim that the error is small. Actually we will need two different bounds. The first will say that if we run n episodes, then with high probability we have at least m episodes in which state $\mathbf{s}$ appears. The second will say that if we have m episodes in which state $\mathbf{s}$ appears, then we will have a good approximation of the value function at $\mathbf{s}$. For the first part, the bound will clearly depend on the probability of reaching state $\mathbf{s}$ in an episode. Call a state $\mathbf{s}$ α -good if the probability that π visits $\mathbf{s}$ in an episode is at least α . The following theorem relates the number of episodes to the accuracy in estimating the value function.

Theorem 11.2. *Assume that we execute n episodes using policy π and each episode has length at most T . Then, with probability $1 - \delta$, for any α -good state $\mathbf{s}$, we have $|\hat{\mathcal{V}}^\pi(\mathbf{s}) - \mathcal{V}^\pi(\mathbf{s})| \leq \lambda$, assuming $n \geq 2m/\alpha$ and $m = 8(T^2/\lambda^2) \log(2|\mathcal{S}|/\delta)$.*

Proof. The proof shows that if we sample n episodes, then the expected number of episodes in which we visit state $\mathbf{s}$ is at least $2m$, and with high probability at least m episodes visit state $\mathbf{s}$.

Let $p(\mathbf{s})$ be the probability that policy π visits state $\mathbf{s}$ in an episode. Since $\mathbf{s}$ is α -good, the expected number of episodes in which $\mathbf{s}$ appears is $p(\mathbf{s})n \geq 2m$. Using the relative Chernoff–Hoeffding bound (Lemma 10.2, with $\epsilon = 1/2$) we have that the probability that we have at least m samples of state $\mathbf{s}$ is at least $1 - \delta/(2|\mathcal{S}|)$

Given that we have at least m samples from state $\mathbf{s}$ using the additive Chernoff–Hoeffding bound (Lemma 10.2) we have with probability at least $1 - \delta/(2|\mathcal{S}|)$ that $|\hat{\mathcal{V}}^\pi(\mathbf{s}) - \mathcal{V}^\pi(\mathbf{s})| \leq \lambda$. (Since episodes have return in the range $[0, T]$, to use Lemma 10.2 we need to normalize to $[0, 1]$ by dividing the rewards by T , which creates the T^2 term in m .)²

Finally, the theorem follows from a union bound over the bad events. $\square$

²A more refined bound can be derived by noticing that the variance of the return of an episode is bounded by T and not T^2 , and using an appropriate concentration bound, say, the Bernstein inequality.

Next, we relate the First Visit Monte-Carlo updates to the maximum likelihood model for the MDP. Going back to the example of Figure 11.1 and observing the sequence $(\mathbf{s}_1, \mathbf{s}_1, \mathbf{s}_1, \mathbf{s}_1, \mathbf{s}_2)$, the only unknown parameter is p .

The maximum likelihood approach would select the value of p that would maximize the probability of observing the sequence $(\mathbf{s}_1, \mathbf{s}_1, \mathbf{s}_1, \mathbf{s}_1, \mathbf{s}_2)$. The likelihood of the sequence is, $(1 - p)^3 p$, and we solve for

$$p^* = \arg \max (1 - p)^3 p.$$

Taking the derivative we have $(1 - p)^3 - 3(1 - p)^2 p = 0$, which give $p^* = 1/4$. For the maximum likelihood (ML) model M we have $p^* = 1/4$ and therefore $\mathcal{V}(\mathbf{s}_1; M) = 4$.

In general the Maximum Likelihood model value does not always coincide with the First Visit Monte-Carlo estimate. However we can make the following interesting connection.

Clearly, when updating state $\mathbf{s}$ using First Visit, we ignore all the episodes that do not include $\mathbf{s}$, and also for each of the remaining episodes, that do include $\mathbf{s}$, we ignore the prefix until the first appearance of $\mathbf{s}$. Let us modify the sample by deleting those parts (episodes in which $\mathbf{s}$ does not appear, and for each episode that $\mathbf{s}$ appears, start it at the first appearance of $\mathbf{s}$). Call this the *reduced sample for state $\mathbf{s}$* .

Maximum Likelihood model The maximum likelihood model, given a set of episodes, is simply the observed model studied in Chapter 10. (We will not show here that the observed model is indeed the maximum likelihood model, but it is a good exercise for the reader to show it.) Namely, for each state-action pair $(\mathbf{s}, \mathbf{a})$ let $n(\mathbf{s}, \mathbf{a})$ be the number of times it appears, let $n(\mathbf{s}, \mathbf{a}, \mathbf{s}')$ be the number of times $\mathbf{s}'$ is observed following executing action $\mathbf{a}$ in state $\mathbf{s}$. The observed transition model is $\hat{p}(\mathbf{s}'|\mathbf{s}, \mathbf{a}) = n(\mathbf{s}, \mathbf{a}, \mathbf{s}')/n(\mathbf{s}, \mathbf{a})$. Assume that in the i -th execution of action $\mathbf{a}$ in state $\mathbf{s}$ we observe a reward $\mathbf{r}_i$ then the observed reward is $\hat{\mathbf{r}}(\mathbf{s}, \mathbf{a}) = (1/n(\mathbf{s}, \mathbf{a})) \sum_{i=1}^{n(\mathbf{s}, \mathbf{a})} \mathbf{r}_i$.

Definition 11.2. *The Maximum Likelihood model M has rewards $\hat{\mathbf{r}}(\mathbf{s}, \mathbf{a})$ and transition probabilities $\hat{p}(\mathbf{s}'|\mathbf{s}, \mathbf{a})$.*

Theorem 11.3. *Let M be the maximum likelihood MDP for the reduced sample for state $\mathbf{s}_0$. The expected value of $\mathbf{s}_0$ in M , i.e., $\mathcal{V}(\mathbf{s}_0; M)$, is identical to the First Visit estimate of $\mathbf{s}_0$, i.e., $\hat{\mathcal{V}}^\pi(\mathbf{s}_0)$.*

Proof. Assume that we have N episodes in the reduced sample for state $\mathbf{s}_0$ and the sum of the rewards in the i -th episode is G_i . The First Visit Monte Carlo estimate would be $\hat{\mathcal{V}}^\pi(\mathbf{s}_0) = (1/N) \sum_{i=1}^N G_i$.

Consider the maximum likelihood model. Since we have a fixed deterministic policy, we can ignore actions, and define $n(\mathbf{s}) = n(\mathbf{s}, \pi(\mathbf{s}))$, $n(\mathbf{s}, \mathbf{s}') = n(\mathbf{s}, \pi(\mathbf{s}), \mathbf{s}')$, and $\hat{\mathbf{r}}(\mathbf{s}, \pi(\mathbf{s})) = \hat{\mathbf{r}}(\mathbf{s})$. We set the initial state $\mathbf{s}_0$ to be the state we are updating.

We want to compute the expected number of visits $\mu(\mathbf{s})$ to each state $\mathbf{s}$ in the ML model M . We will show that $\mu(\mathbf{s}) = n(\mathbf{s})/N$. This implies that the expected reward for state $\mathbf{s}_0$ in M would be

$$\mathcal{V}^\pi(\mathbf{s}_0; M) = \sum_v \mu(v) \hat{\mathbf{r}}(v) = \sum_v \frac{n(v)}{N} \frac{1}{n(v)} \sum_{i=1}^{n(v)} \mathbf{r}_i^v = \frac{1}{N} \sum_{j=1}^N G_j,$$

where $\mathbf{r}_i^v$ is the i -th reward observation from state v , and the last equality follows by changing the order of summation (from states to episodes).

It remains to show that $\mu(\mathbf{s}) = n(\mathbf{s})/N$. We have the following identities. For $\mathbf{s} \neq \mathbf{s}_0$:

$$\mu(\mathbf{s}) = \sum_{\mathbf{s}'} \hat{\mathbf{p}}(\mathbf{s}|\mathbf{s}') \mu(\mathbf{s}').$$

For the initial state $\mathbf{s}_0$ we have

$$\mu(\mathbf{s}_0) = 1 + \sum_{\mathbf{s}'} \hat{\mathbf{p}}(\mathbf{s}_0|\mathbf{s}') \mu(\mathbf{s}').$$

Note that $n(\mathbf{s}') = \sum_{\mathbf{s}} n(\mathbf{s}, \mathbf{s}')$ for $\mathbf{s}' \neq \mathbf{s}_0$ and $n(\mathbf{s}_0) = N + \sum_{\mathbf{s}} n(\mathbf{s}, \mathbf{s}_0)$, and recall that $\hat{\mathbf{p}}(\mathbf{s}'|\mathbf{s}) = n(\mathbf{s}, \mathbf{s}')/n(\mathbf{s})$. One can verify the identities by plugging in $\mu(\mathbf{s}) = n(\mathbf{s})/N$. $\square$

11.3.3 Every visit

The **First Visit** updates are unbiased, since the different updates are from different episodes. For each episode the update is an independent unbiased sample of the return. For **Every Visit** the situation is more complicated, since there are different updates from the same episode, and therefore they are dependent. The first issue that we have to resolve is how we would like to average the **Every Visit** updates. Let $G_{i,j}^{\mathbf{s}}$ be the j -th update in the i -th episode for state $\mathbf{s}$. Let n_i be the number of updates in episode i and N the overall number of episodes.

One way to average the updates is to average for each episode the updates and average across episodes. Namely,

$$\frac{1}{N} \sum_{i=1}^N \frac{1}{n_i} \sum_{j=1}^{n_i} G_{i,j}.$$

An alternative approach is to sum the updates and divide by the number of updates,

$$\frac{\sum_{i=1}^N \sum_{j=1}^{n_i} G_{i,j}}{\sum_{i=1}^N n_i}.$$

We will use the latter scheme, but it is worthwhile to understand the difference between the two. Consider for example a case where we have 10 episodes, in 9 of them we have a single visit to $\mathbf{s}$ and a return of 1, and in the 10-th we have 11 visits to $\mathbf{s}$ and all the returns are zero. The first averaging would give an estimate of 9/10 while the second would give an estimate of 9/20.

Consider the case of Figure 11.1. For a single episode of length τ we have that the sum of the rewards is $\tau(\tau+1)/2$, since there are updates of lengths $\tau, \dots, 1$ and recall that the return equals the episode duration since all rewards are 1. The number of updates is τ , so we have that the estimate of a single episode is $(\tau+1)/2$. When we take the expectation we have that $\mathbb{E}[(\tau+1)/2] = (1/p + 1)/2$ which is different from the expected value of $1/p$. (Recall that the **Every Visit** updates τ times using values $\tau, \dots, 1$. In addition, $\mathbb{E}[\tau] = 1/p$ which is also the expected value.) If we have a single episode then both averaging schemes are identical.

When we have multiple episodes, we can see the difference between the two averaging schemes. The first will be biased random variables of $\mathbb{E}[(\tau+1)/2] = (1/p + 1)/2$, so it will converge to this value rather than $1/p$. The second scheme, which we will use in **Every Visit** updates, will have the bias decrease with the number of episodes. The reason is that we sum separately the returns, and the number of occurrences. Note that from the law of large numbers we have $\frac{1}{N} \sum_{i=1}^N \sum_{j=1}^{n_i} G_{i,j} \rightarrow \mathbb{E}[\sum_{j=1}^n G_j]$, and $\frac{1}{N} \sum_{i=1}^N n_i \rightarrow \mathbb{E}[n]$, therefore using Slutsky's theorem,

$$\mathcal{V}^{ev}(\mathbf{s}) = \frac{\sum_{i=1}^N \sum_{j=1}^{n_i} G_{i,j}}{\sum_{i=1}^N n_i} \rightarrow \frac{\mathbb{E}[\sum_{j=1}^n G_j]}{\mathbb{E}[n]} = \frac{\mathbb{E}[\tau(\tau+1)/2]}{\mathbb{E}[\tau]} = \frac{\mathbb{E}[\tau^2] + \mathbb{E}[\tau]}{2\mathbb{E}[\tau]} = \frac{1}{p},$$

since $\mathbb{E}[\tau^2] = 2/p^2 - 1/p$. This implies that if we average many episodes we will get an almost unbiased estimate using **Every Visit**.

We did all this on the example of Figure 11.1, but this indeed generalizes. Given an arbitrary episodic MDP, consider the following mapping. For each episode, mark the places where state $\mathbf{s}$ appears (the state we want to approximate its value). We now have a distribution of rewards from going from $\mathbf{s}$ back to $\mathbf{s}$. Since we are in an episodic MDP, we also have to terminate, and for this we can add another state, from which we transition from state $\mathbf{s}$ and have the reward distribution as the rewards from the last appearance of $\mathbf{s}$ until the end of the episode. This implies that we have two states MDP as described in Figure 11.2.

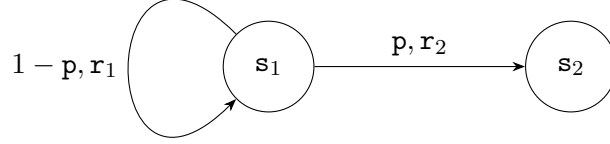


Figure 11.2: Equivalent MDP for **Every Visit**

For this MDP, the value is $\mathcal{V}(\mathbf{s}_1) = \frac{1-p}{p}\mathbf{r}_1 + \mathbf{r}_2$. The single episode expected estimate of **Every Visit** is $\mathcal{V}(\mathbf{s}_1) = \frac{1-p}{2p}\mathbf{r}_1 + \mathbf{r}_2$. The m episodes expected estimate of **Every Visit** is $\mathcal{V}(\mathbf{s}_1) = \frac{m}{m+1}\frac{1-p}{p}\mathbf{r}_1 + \mathbf{r}_2$. This implies that if we have a large number of episodes the bias of the estimate becomes negligible. (For more details, see Theorem 7 in [143].)

Every visit and squared loss

We have related the **First Visit** to the Maximum Likelihood (although on a slightly modified sample). In the following we would like to relate the **Every Visit** to the square loss, showing the benefit in **Every Visit**. Actually, the relation is rather general, since the minimizer the square loss is simply the empirical average, which **Every Visit** implicitly computes.

We now formally relate **Every Visit** to squared error estimation. Assume we have $\mathbf{s}_{i,j}$ as the j -th state in the i -th episode, and it has return $G_{i,j}$, i.e., the sum of the rewards from in episode i from step j until the end of the episode. Let $\hat{\mathcal{V}}(\mathbf{s}_{i,j})$ be the estimate of **Every Visit** for state $\mathbf{s}_{i,j}$. (Note that states $\mathbf{s}_{i,j}$ are not unique, and we can have $\mathbf{s} = \mathbf{s}_{i_1,j_1} = \mathbf{s}_{i_2,j_2}$.) The squared error is,

$$SE = \frac{1}{2} \sum_{i,j} (\hat{\mathcal{V}}(\mathbf{s}_{i,j}) - G_{i,j})^2.$$

For a fixed state $\mathbf{s}$ we have

$$SE(\mathbf{s}) = \frac{1}{2} \sum_{i,j:\mathbf{s}=\mathbf{s}_{i,j}} (\hat{\mathcal{V}}(\mathbf{s}) - G_{i,j})^2,$$

and the total squared error is $SE = \sum_{\mathbf{s}} SE(\mathbf{s})$.

Our goal is to select a value $\hat{\mathcal{V}}^{se}(\mathbf{s})$ for every state, which would minimize the SE. The minimization is achieved by minimizing the square error of each $\mathbf{s}$, and setting the values

$$\hat{\mathcal{V}}^{se}(\mathbf{s}) = \frac{\sum_{i,j:\mathbf{s}=\mathbf{s}_{i,j}} G_{i,j}}{|(i,j) : \mathbf{s} = \mathbf{s}_{i,j}|},$$

which is exactly the **Every Visit** Monte-carlo estimate for state $\mathbf{s}$.

11.3.4 Monte-Carlo control

We can also use the Monte-Carlo methodology to learn the optimal policy. The main idea is to learn the $\mathcal{Q}^\pi$ function. This is done by simply updating for every $(\mathbf{s}, \mathbf{a})$. (The updates can be either **Every Visit** or **First Visit**.) The problem is that we need the policy to be “exploring”, otherwise we will not have enough information about the actions the policy does not perform.

For the control, we can maintain an estimates of the $\mathcal{Q}^\pi$ function, where the current policy is π . After we have a good estimate of $\mathcal{Q}^\pi$ we can switch to a policy which is greedy with respect to $\mathcal{Q}^\pi$. Namely, each time we reach a state $\mathbf{s}$, we select a “near-greedy” action, for example, using the ε -greedy policy of Definition 11.1.

We will show that updating from one ε -greedy policy to another ε -greedy policy, using policy improvement, does increase the value of the policy. This will guarantee that we will not cycle, and eventually converge.

Recall that an ε -greedy policy can be defined in the following way. For every state $\mathbf{s}$ there is an action $\bar{\mathbf{a}}_{\mathbf{s}}$, which is the preferred action. The policy does the following: (1) with probability $1 - \varepsilon$ selects action $\bar{\mathbf{a}}_{\mathbf{s}}$, and (2) with probability ε , selects each action $\mathbf{a} \in \mathcal{A}$, with probability $\varepsilon/|\mathcal{A}|$.

Assume we have an ε -greedy policy π_1 . Compute $\mathcal{Q}^{\pi_1}$ and define π_2 to be ε -greedy with respect to $\mathcal{Q}^{\pi_1}$.

Theorem 11.4. *For any ε -greedy policy π_1 , the ε -greedy improvement policy π_2 has $\mathcal{V}^{\pi_2} \geq \mathcal{V}^{\pi_1}$.*

Proof. Let $\bar{\mathbf{a}}_{\mathbf{s}} = \arg \max_{\mathbf{a}} \mathcal{Q}^{\pi_1}(\mathbf{s}, \mathbf{a})$ be the greedy action w.r.t. $\mathcal{Q}^{\pi_1}$. We first lower bound the expected value of $\mathcal{Q}^{\pi_1}$ when selecting actions using policy π_2 and $\bar{\mathbf{a}}_{\mathbf{s}} \in$

$\arg \max_{\mathbf{a}_s} \mathcal{Q}^{\pi_1}(\mathbf{s}, \mathbf{a}_s)$:

$$\begin{aligned}
\mathbb{E}_{\mathbf{a} \sim \pi_2(\cdot|\mathbf{s})}[\mathcal{Q}^{\pi_1}(\mathbf{s}, \mathbf{a})] &= \sum_{\mathbf{a} \in \mathcal{A}} \pi_2(\mathbf{a}|\mathbf{s}) \mathcal{Q}^{\pi_1}(\mathbf{s}, \mathbf{a}) \\
&= \frac{\varepsilon}{|\mathcal{A}|} \sum_{\mathbf{a} \in \mathcal{A}} \mathcal{Q}^{\pi_1}(\mathbf{s}, \mathbf{a}) + (1 - \varepsilon) \mathcal{Q}^{\pi_1}(\mathbf{s}, \bar{\mathbf{a}}_s) \\
&\geq \frac{\varepsilon}{|\mathcal{A}|} \sum_{\mathbf{a} \in \mathcal{A}} \mathcal{Q}^{\pi_1}(\mathbf{s}, \mathbf{a}) + (1 - \varepsilon) \sum_{\mathbf{a} \in \mathcal{A}} \frac{\pi_1(\mathbf{a}|\mathbf{s}) - \varepsilon/|\mathcal{A}|}{1 - \varepsilon} \mathcal{Q}^{\pi_1}(\mathbf{s}, \mathbf{a}) \\
&= \sum_{\mathbf{a} \in \mathcal{A}} \pi_1(\mathbf{a}|\mathbf{s}) \mathcal{Q}^{\pi_1}(\mathbf{s}, \mathbf{a}) = \mathcal{V}^{\pi_1}(\mathbf{s}).
\end{aligned}$$

The inequality follows since we are essentially concentrating of the action that $\pi_1(\cdot|\mathbf{s})$ selects with probability $1 - \varepsilon$, and clearly $\bar{\mathbf{a}}_s$, by definition, guarantees a higher value.

It remains to show, similar to the basic policy improvement, that we have

$$\mathcal{V}^{\pi_2}(\mathbf{s}) \geq \max_{\mathbf{a}} \mathcal{Q}^{\pi_1}(\mathbf{s}, \mathbf{a}) \geq \mathbb{E}_{\mathbf{a} \sim \pi_2(\cdot|\mathbf{s})}[\mathcal{Q}^{\pi_1}(\mathbf{s}, \mathbf{a})] \geq \mathcal{V}^{\pi_1}(\mathbf{s}).$$

Basically, we need to re-write the Bellman optimality operator to apply to ε -greedy policies as follows:

$$\begin{aligned}
(\mathcal{T}_\varepsilon^* \mathcal{V})(\mathbf{s}) &= \max_{\mathbf{a}} \left[(1 - \varepsilon)(r(\mathbf{s}, \mathbf{a}) + \gamma \mathbb{E}_{\mathbf{s}' \sim \mathbf{p}(\cdot|\mathbf{s}, \mathbf{a})}[\mathcal{V}(\mathbf{s}')]) \right. \\
&\quad \left. + \varepsilon \left(\sum_{\mathbf{a}''} \frac{\varepsilon}{|\mathcal{A}|} (r(\mathbf{s}, \mathbf{a}'') + \gamma \mathbb{E}_{\mathbf{s}'' \sim \mathbf{p}(\cdot|\mathbf{s}, \mathbf{a}'')}[\mathcal{V}(\mathbf{s}'')]) \right) \right].
\end{aligned}$$

Clearly $\mathcal{T}_\varepsilon^*(V)$ is monotone in V , and for $\mathcal{V}^{\pi_1}$ we have $\mathcal{T}_\varepsilon^*(\mathcal{V}^{\pi_1}) = \mathcal{T}^{\pi_2}(\mathcal{V}^{\pi_1})$. Since $\mathcal{T}^{\pi_2}(\mathcal{V}^{\pi_1}) = \mathbb{E}_{\mathbf{a} \sim \pi_2(\cdot|\mathbf{s})}[\mathcal{Q}^{\pi_1}(\mathbf{s}, \mathbf{a})]$, this implies that,

$$\mathcal{T}^{\pi_2}(\mathcal{V}^{\pi_1}) = \mathcal{T}_\varepsilon^*(\mathcal{V}^{\pi_1}) \geq \mathcal{T}^{\pi_1}(\mathcal{V}^{\pi_1}) = \mathcal{V}^{\pi_1}$$

We can continue to apply $\mathcal{T}^{\pi_2}$ and due to the monotonicity have

$$(\mathcal{T}^{\pi_2})^k(\mathcal{V}^{\pi_1}) \geq (\mathcal{T}^{\pi_2})^{k-1}(\mathcal{V}^{\pi_1}) \geq \dots \geq \mathcal{V}^{\pi_1}$$

Since $\lim_{k \rightarrow \infty} (\mathcal{T}^{\pi_2})^k(\mathcal{V}^{\pi_1}) = \mathcal{V}^{\pi_2}$ we are done. □

11.3.5 Monte-Carlo: pros and cons

The main benefits of the Monte-Carlo updates are:

1. Very simple and intuitive
2. Does not assume the environment is Markovian
3. Extends naturally to function approximation (more in future chapters)
4. Unbiased updates (using **First Visit**).

The main drawback of the Monte-Carlo updates are:

1. Need to wait for the end of episode to update.
2. Suited mainly for episodic environment.
3. Biased updates (using **Every Visit**).

Going back to the $\mathcal{Q}$ -learning algorithm in Section 11.2, we see that Monte-Carlo methods do not use the bootstrapping idea, which can mitigate the first two drawbacks, by updating the estimates online, *before* an episode is over. In the following we will develop bootstrapping based methods for model-free learning in MDPs. To facilitate our analysis of these methods, we shall first describe a general framework for online algorithms.

11.4 Stochastic Approximation

In this section we present the general framework of stochastic approximation algorithms. We here outline the main definitions and results of that literature. We will later use it to show convergence of online learning algorithms.

The stochastic approximation algorithm takes the following general form:

$$X_{\mathfrak{t}+1} = X_{\mathfrak{t}} + \alpha_{\mathfrak{t}}(f(X_{\mathfrak{t}}) + \omega_{\mathfrak{t}}), \quad (11.1)$$

where $X \in \mathbb{R}^d$ is a vector of parameters that we update, f is a deterministic function, $\alpha_{\mathfrak{t}}$ is a step size, and $\omega_{\mathfrak{t}}$ is a noise term that is zero-mean and bounded in some sense (that will be defined later).

We will be interested in the long-term behavior of the algorithm in Eq. 11.1, and in particular, whether the iterates $X_{\mathfrak{t}}$ can be guaranteed to converge as $\mathfrak{t} \rightarrow \infty$. Without the noise term, Eq. 11.1 describes a simple recurrence relation, with behavior that is determined by the function f and the step size (see Remark 11.2 for an example). In this case, convergence can be guaranteed if f has certain structure, such as a contraction property. In the following, we separate our presentation to

two different structures of f . The first is the said contraction, with convergence to a fixed point, while the second relates Eq. 11.1 to an ordinary differential equation (ODE), and looks at convergence to stable equilibrium points of the ODE. While the technical details of each approach are different, the main idea is similar: in both cases we will choose step sizes that are large enough such that the *expected* update converges, yet are small enough such that the noise terms do not take the iterates too far away from the expected behavior. The contraction method will be used to analyse the model-free learning algorithms in this section, while the ODE method will be required for the analysis in later chapters, when function approximation is introduced.

11.4.1 Convergence via Contraction

In the flavor of the stochastic approximation we consider here, there is a state space $\mathcal{S}$, and the iterates update an approximation $X(\mathbf{s})$ at each iteration by modifying a single element $X(\mathbf{s}_t)$, where the sequence of state updates $\mathbf{s}_t \in \mathcal{S}$ is chosen arbitrarily. The iterative algorithm takes the following general form:

$$X_{t+1}(\mathbf{s}) = \begin{cases} (1 - \alpha_t(\mathbf{s}))X_t(\mathbf{s}) + \alpha_t(\mathbf{s})((HX_t)(\mathbf{s}) + \omega_t(\mathbf{s})), & \mathbf{s} = \mathbf{s}_t \\ X_t(\mathbf{s}), & \text{otherwise} \end{cases}.$$

This can be seen as a special case of the general stochastic approximation form in Eq. 11.1, where $f(X) = H(X) - X$, and a state-dependent step size that is non-zero only for the current state $\mathbf{s}_t$.

We call a sequence of learning rates $\{\alpha_t(\mathbf{s})\}$ is *well formed* if for every $\mathbf{s} \in \mathcal{S}$ we have: (1) $\sum_t \alpha_t(\mathbf{s})I(\mathbf{s}_t = \mathbf{s}) = \infty$, and (2) $\sum_t \alpha_t^2(\mathbf{s})I(\mathbf{s}_t = \mathbf{s}) = O(1)$, with probability 1.

Remark 11.4. *The step size rule $\alpha_t(\mathbf{s})$ may depend on the history. An instance that is particularly useful is $\alpha_t(\mathbf{s}_t) = 1/(1 + \# \text{ visits to state } \mathbf{s}_t)$. Note also that a sequence that is well formed implicitly implies that every state is visited infinitely often.*

We will mainly look at (B, γ) *well behaved* iterative algorithms, where $B > 0$ and $\gamma \in (0, 1)$, which have the following properties:

1. Step size: sequence of learning rates $\{\alpha_t(\mathbf{s})\}$ is *well formed*.
2. Noise: $\mathbb{E}[\omega_t(\mathbf{s})|h_{t-1}] = 0$ and $|\omega_t(\mathbf{s})| \leq B$, where h_{t-1} is the history up to time t , that is $h_{t-1} = \{\mathbf{s}_0, \omega_0, \dots, \omega_{t-1}, \mathbf{s}_t\}$.

3. Pseudo-contraction: There exists X^* such that for any X we have $\|HX - X^*\|_\infty \leq \gamma\|X - X^*\|_\infty$. (This implies that $HX^* = X^*$. Note that any contraction operator H is also a pseudo contraction, with X^* being the unique fixed point of H , cf. Theorem 6.8)

The following is the convergence theorem for well behaved iterative algorithms.

Theorem 11.5 (Contracting Stochastic Approximation: convergence).

Let X_t be a sequence that is generated by a (B, γ) well behaved iterative algorithm. Then X_t converges with probability 1 to X^ .*

We will not give a proof of this important theorem, but we will try to sketch the main proof methodology; a complete proof can be found in Proposition 4.4 of [19].

There are two distinct parts to the iterative algorithms. The part (HX_t) is contracting, in a deterministic manner. If we had only this part (say, $\omega_t = 0$ always) then the contraction property of H will give the convergence (as we saw before in Remark 11.2). The main challenge is the addition of the stochastic noise ω_t . The noise is unbiased, so on average the expectation is zero. Also, the noise is bounded by a constant B . This implies that if we average the noise over a long time interval, then the average should be very close to zero.

The proof considers the $\|X_t - X^*\|$, and works in phases. In phase i , at any time t in the phase we have $\|X_t - X^*\| \leq \lambda_i$. In each phase we have a deterministic contraction using the operator H . The deterministic contraction implies that the space contracts by a factor $\gamma < 1$. Taking into account the step size α_i , following Remark 11.2, let $\tilde{\gamma}_i = (1 - \alpha_i(1 - \gamma)) < 1$. We have to take care of the stochastic noise. We make the phase long enough so that the average of the noise is less than $\lambda_i(1 - \tilde{\gamma}_i)/2$ factor. This implies that the space contracts by $\lambda_{i+1} \leq \tilde{\gamma}_i\lambda_i + (1 - \tilde{\gamma}_i)\lambda_i/2 = \lambda_i(1 + \tilde{\gamma}_i)/2 < \lambda_i$. To complete our proof, we need to show that the decreasing sequence λ_i converges to zero. Without loss of generality, let $\lambda_0 = 1$. Then we need to evaluate $\lambda_\infty = \prod_{i=0}^{\infty} (1 - \alpha_i \frac{1-\gamma}{2})$. We have

$$\exp\left(\log\left(\prod_{i=0}^{\infty}\left(1 - \alpha_i \frac{1-\gamma}{2}\right)\right)\right) = \exp\left(\sum_{i=0}^{\infty} \log\left(1 - \alpha_i \frac{1-\gamma}{2}\right)\right) \approx \exp\left(-\frac{1-\gamma}{2} \sum_{i=0}^{\infty} \alpha_i\right),$$

which converges to zero for a well-behaved algorithm, due to the first step size rule.

11.4.2 Convergence via the ODE method

Here we will consider the general stochastic approximation form in Eq. 11.1, and we will not limit ourselves to algorithms that update only certain states, but may update the whole vector X_t at every time step.

The asymptotic behavior of the stochastic approximation algorithm is closely related to the solutions of a certain ODE (Ordinary Differential Equation)³, namely

$$\frac{d}{dt}\theta(t) = f(\theta(t)),$$

or $\dot{\theta} = f(\theta)$. The relation between the discrete-time stochastic approximation iterates and the ODE in continuous time is depicted in Figure 11.3 and detailed in the following.

Given $\{X_{\mathbf{t}}, \alpha_{\mathbf{t}}\}$, we define a *continuous-time* process $\theta(t)$ as follows. Let

$$t_{\mathbf{t}} = \sum_{k=0}^{\mathbf{t}-1} \alpha_k.$$

Define

$$\theta(t_{\mathbf{t}}) = X_{\mathbf{t}},$$

and use linear interpolation in-between the $t_{\mathbf{t}}$'s. Thus, the time-axis t is rescaled according to the gains $\{\alpha_{\mathbf{t}}\}$.

Note that over a fixed Δt , the “total gain” is approximately constant:

$$\sum_{k \in K(t, \Delta t)} \alpha_k \simeq \Delta t,$$

where $K(t, \Delta t) = \{k : t \leq t_k < t + \Delta t\}$. Plugging in the update of Eq. 11.1, we have

$$\theta(t + \Delta t) = \theta(t) + \sum_{\mathbf{t} \in K(t, \Delta t)} \alpha_{\mathbf{t}} [f(X_{\mathbf{t}}) + \omega_{\mathbf{t}}].$$

We now make two observations about the terms in the sum above:

1. For large t , $\alpha_{\mathbf{t}}$ becomes small and the summation is over many terms; thus the noise term is approximately “averaged out”: $\sum \alpha_{\mathbf{t}} \omega_{\mathbf{t}} \rightarrow 0$.
2. For small Δt , $X_{\mathbf{t}}$ is approximately constant over $K(t, \Delta t)$: $f(X_{\mathbf{t}}) \simeq f(\theta(t))$.

We thus obtain:

$$\theta(t + \Delta t) \simeq \theta(t) + \Delta t \cdot f(\theta(t)),$$

and rearranging gives,

$$\frac{\theta(t + \Delta t) - \theta(t)}{\Delta t} \simeq f(\theta(t)).$$

³We provide an introduction to ODEs in Appendix B.

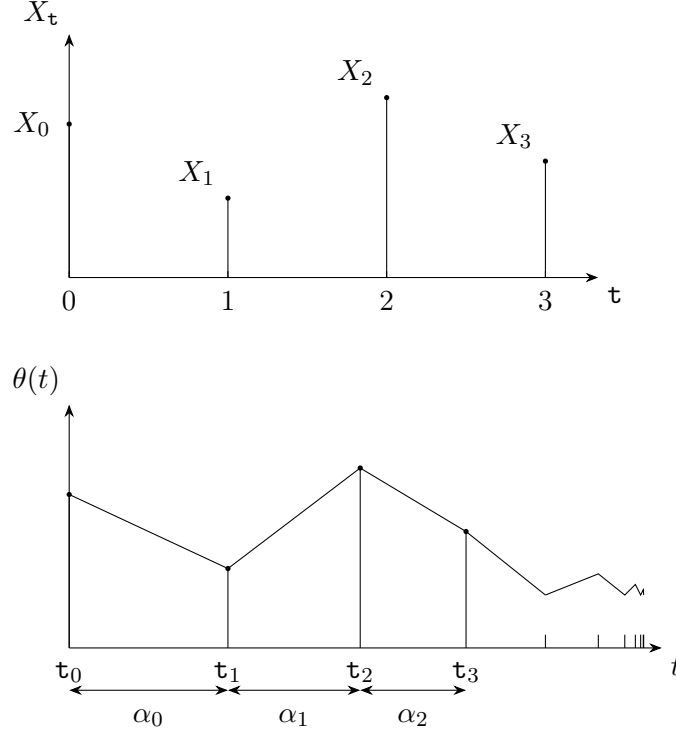


Figure 11.3: Convergence via the ODE method. Top figure: the discrete time process. Bottom figure: continuous time interpolation.

For $\Delta t \rightarrow 0$, this reduces to the ODE:

$$\dot{\theta}(t) = f(\theta(t)).$$

We shall now discuss the convergence of the stochastic approximation iterates. As $t \rightarrow \infty$, we “expect” that the estimates $\{X_t\}$ will follow a trajectory of the ODE $\dot{\theta} = f(\theta)$ (under the above time normalization). Thus, convergence can be expected to *equilibrium points* (also termed *stationary points* or *fixed points*) of the ODE, that is, points X^* that satisfy $f(X^*) = 0$. We will also require the equilibrium to be *globally asymptotically stable*, as follows.

Definition 11.3. *An ODE $\dot{\theta}(t) = f(\theta(t))$ has a globally asymptotically stable equilibrium point X^* , if $f(X^*) = 0$, and for any θ_0 we have that $\lim_{t \rightarrow \infty} \theta(t) = X^*$.*

We will mainly look at (A, B) well behaved iterative algorithms, where $B > 0$, which have the following properties:

1. Step size: the sequence of learning rates $\{\alpha_t(\mathbf{s}, \mathbf{a})\}$ is *well formed*,
2. Noise: $\mathbb{E}[\omega_t(\mathbf{s})|h_{t-1}] = 0$ and $|\omega_t(\mathbf{s})| \leq A + B\|X_t\|^2$, for some norm $\|\cdot\|$ on $\mathbb{R}^d$, where h_{t-1} is the history up to time t ,
3. f is Lipschitz continuous,
4. The ODE $\dot{\theta}(t) = f(\theta(t))$ has a globally asymptotically stable equilibrium X^* ,
5. The sequence X_t is bounded with probability 1.

We now give a convergence result (Theorem 2 in [22]).

Theorem 11.6 (Stochastic Approximation: ODE convergence).

Let X_t be a sequence that is generated by a (A, B) well behaved iterative algorithm. Then X_t converges with probability 1 to X^ .*

Remark 11.5. *More generally, even if the ODE is not globally stable, X_t can be shown to converge to an invariant set of the ODE (e.g., a limit cycle).*

Remark 11.6. *A major assumption in the last result is the boundedness of (X_t) . In general this assumption has to be verified independently. However, there exist several results that rely on further properties of f to deduce boundedness, and hence convergence. One technique is to consider the function $f_c(\theta) = f(c\theta)/c$, $c \geq 1$. If $f_c(\theta) \rightarrow f_\infty(\theta)$ uniformly, one can consider the ODE with f_∞ replacing f (Theorem 7 in [22]). In particular, for a linear f , we have that $f_c = f$, and this result shows that boundedness is guaranteed. We make this explicit in the following theorem.*

Theorem 11.7 (Stochastic Approximation: ODE convergence for linear systems).

Let X_t be a sequence that is generated by a iterative algorithm that satisfies the first 4 conditions of an (A, B) well behaved algorithm, and f is linear in θ . Then X_t converges with probability 1 to X^ .*

Remark 11.7. *Another technique to guarantee boundedness is to use projected iterates:*

$$X_{t+1} = \text{Proj}_\Gamma[X_t + \alpha_t((f(X_t) + \omega_t))]$$

where Proj_Γ is a projection onto some convex set Γ . A simple example is when Γ is a box, so that the components of X are simply truncated at their minimal and maximal values. If Γ is a bounded set then the estimated sequence $\{X_t\}$ is guaranteed to be bounded. However, in this case the corresponding ODE must account for the projection, and is $\dot{\theta}(t) = f(\theta(t)) + \zeta$, where ζ is zero in the interior of Γ , and on the

boundary of Γ , ζ is the infinitesimal change to θ required to keep it in Γ . In this case, to show convergence using Theorem 11.6, we must verify that X^* is still a globally asymptotically stable equilibrium, that is, we must verify that the projection did not add a spurious stable point on the boundary of Γ . A thorough treatment of this idea is presented in [86].

11.4.3 Comparison between the two convergence proof techniques

The two proof techniques above are qualitatively different, and also require different conditions to be applied. For the contraction approach, establishing that the iterates are bounded with probability 1 is not required, while with the ODE method, this is known to be a big technical difficulty in some applications. For linear ODEs, however, as Theorem 11.7 shows, this is not an issue. Another important difference is that some recurrence relations may converge even though they are not necessarily contractions. In such cases, the ODE method is more suitable. We next give a simple example of such a system. In Chapter 12, we will encounter a similar case when establishing convergence of an RL algorithm with linear function approximation.

Example 11.1. Consider the following linear recurrence equation in $\mathbb{R}^2$, where for simplicity we omit the noise term

$$X_{t+1} = X_t + \alpha_t A X_t,$$

where $A \in \mathbb{R}^{2 \times 2}$. Clearly, $X^* = [0, 0]$ is a fixed point. Let $X_0 = [0, 1]$, and consider two different values of the matrix A , namely, $A_{\text{contraction}} = \begin{bmatrix} -0.9 & -0.9 \\ 0 & -0.9 \end{bmatrix}$, and $A_{\text{no-contraction}} = \begin{bmatrix} -3 & -3 \\ 2.1 & 1.9 \end{bmatrix}$. Note that the resulting operator $H(X) = X + AX$ can be either $H_{\text{contraction}} = \begin{bmatrix} 0.1 & -0.9 \\ 0 & 0.1 \end{bmatrix}$, and $H_{\text{no-contraction}} = \begin{bmatrix} -2 & -3 \\ 2.1 & 2.9 \end{bmatrix}$. It can be verified that $\|H_{\text{contraction}} X\| < \|X\|$ for any $X \neq 0$. However, note that $\|H_{\text{no-contraction}} X_0\| = \|[-3, 2.9]\| > \|[0, 1]\|$, therefore $H_{\text{no-contraction}}$ is not a contraction in the Euclidean norm (nor in any other weighted p -norm).

The plot in Figure 11.4 shows the evolution of the recurrence when starting from X_0 , for a constant step size $\alpha_t = 0.2$. Note that both iterates converge to X^* , as it is an asymptotically stable fixed point for the ODE $\dot{X} = AX$ for both values of A . However, the iterates for $H_{\text{contraction}}$ always reduce the distance to X^* , while the iterates for $H_{\text{no-contraction}}$ do not. Thus, for $H_{\text{no-contraction}}$, only the ODE method would have worked for showing convergence.

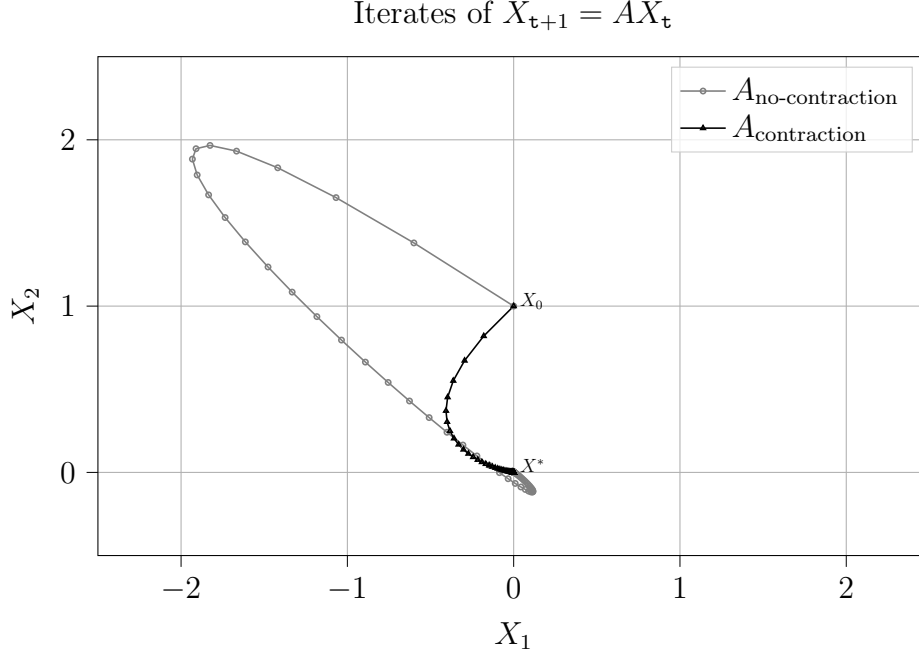


Figure 11.4: Plot for Example 11.1.

11.5 Bootstrapping Algorithms

We now return to the problem of policy evaluation, and explore the following idea: how to use *bootstrapping* to evaluate the policy by combining the observed samples with our current estimate of the value function. We illustrate the potential of this idea in the following example.

Figure 11.5 shows a simple example (taken from [149]) with four states $S = \{A, B, C, D\}$ where $\{C, D\}$ are terminal states and in $\{A, B\}$ there is one action (essentially, the policy selects a unique action), and the transitions and rewards are such that $\mathcal{V}(A) = \mathcal{V}(B) = 0.75$. Assume we observe eight episodes. One episode is $(A, 0, B, 0, C)$, one episode $(B, 0, C)$, and six episodes $(B, 1, D)$. We would like to estimate the value function of the non-terminal states. For $\mathcal{V}(B)$ we use Monte Carlo, which gives $\hat{\mathcal{V}}_{\text{MC}}(B) = 6/8 = 0.75$.

The interesting question, however, is what should be the estimate for A ? MC will average only the trajectories that include A and will therefore get $\hat{\mathcal{V}}_{\text{MC}}(A) = 0$ (only one trajectory which gives 0 reward). But we know from the Bellman equation that $\mathcal{V}(A) = \mathbf{r}(A) + \mathcal{V}(B)$, so an alternative bootstrapping estimate is $\hat{\mathcal{V}}(A) = 0 + \hat{\mathcal{V}}_{\text{MC}}(B) = 0.75$, which is much more accurate as it effectively takes into account

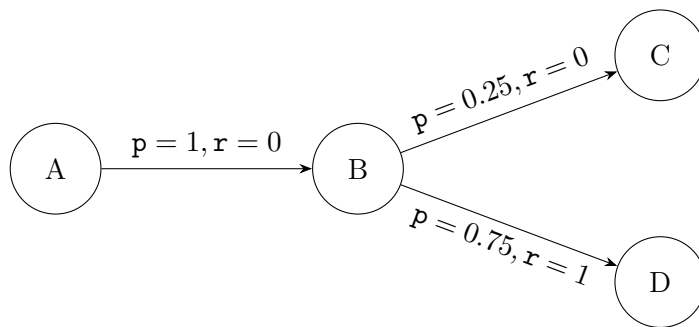


Figure 11.5: TD(0) vs. Monte-Carlo example

the 6 additional trajectories used for estimating $\hat{\mathcal{V}}_{\text{MC}}(B)$.

To better understand the above example, consider the empirical MDP estimated from the samples: it will have a transition from A to B with probability 1 and reward 0, a transition from B to C with probability 0.25 and reward 0, and a transition from B to D with probability 0.75 and reward 1 (see Figure 11.5). The value of A in the empirical model is 0.75, agreeing with the bootstrapping estimate.

To write the Bellman equation in the example above we needed to know the MDP model beforehand, which is not practical. Can we develop a method that uses bootstrapping without knowing the model? This is the Temporal Difference (TD) approach, which works in a model-free, online fashion. We will start with $TD(0)$ which uses only the most recent observations for the updates, and continue with the Q-learning and SARSA algorithms which extend this idea from policy evaluation to control. Afterwards we will also consider methods that allow for a longer look-ahead, leading to the $TD(\lambda)$ algorithm that averages multiple look-ahead estimations. Unlike Monte-Carlo algorithms, TD methods use incomplete episodes for the updates, and are not restricted to episodic MDPs.

11.5.1 TD(0)

Fix a policy $\pi \in \Pi_{SD}$, stationary and deterministic. The goal is to learn the value function $\mathcal{V}^\pi(\mathbf{s})$ for every $\mathbf{s} \in \mathcal{S}$. (The same goal as Monte-Carlo learning, but we focus on the discounted setting.) The TD algorithms will maintain an estimate of the value function of the policy π , i.e., maintain an estimate $\hat{\mathcal{V}}_{\mathbf{t}}(\mathbf{s})$ for $\mathcal{V}^\pi(\mathbf{s})$ at iteration $\mathbf{t}$ of the algorithm. The TD algorithms will use their estimates $\hat{\mathcal{V}}$ for the updates. This implies that unlike Monte-Carlo, there will be an interaction between the estimates of different states and at different times.

As a starting point, we can recall the *value iteration* algorithm:

$$\mathcal{V}_{t+1}(\mathbf{s}) = \mathbb{E}^\pi[\mathbf{r}(\mathbf{s}, \pi(\mathbf{s})) + \gamma \mathcal{V}_t(\mathbf{s}')].$$

We have shown in Proposition 6.4 that value iteration converges, namely $\mathcal{V}_t \rightarrow \mathcal{V}^\pi$ as $t \rightarrow \infty$.

Assume we sample $(\mathbf{s}_t, \mathbf{a}_t, \mathbf{r}_t, \mathbf{s}_{t+1})$. Let $\hat{\mathcal{V}}_t$ be our estimation at time t , and we sample $(\mathbf{s}_t, \mathbf{a}_t, \mathbf{r}_t, \mathbf{s}_{t+1})$. Then, we can use the sample to approximate the value iteration update at state $\mathbf{s}_t$. The expected update is

$$\hat{\mathcal{V}}_{t+1}(\mathbf{s}_t) := \mathbb{E}^\pi[\mathbf{r}_t + \gamma \hat{\mathcal{V}}_t(\mathbf{s}_{t+1})] = \mathbb{E}^\pi[\mathbf{r}(\mathbf{s}, \mathbf{a}) + \gamma \hat{\mathcal{V}}_t(\mathbf{s}') | \mathbf{s} = \mathbf{s}_t, \mathbf{a} = \pi(\mathbf{s})].$$

The $TD(0)$ algorithm will update in the direction of a single-sample estimate of this expectation, namely, $[\mathbf{r}_t + \gamma \hat{\mathcal{V}}_t(\mathbf{s}_{t+1})]$.

Algorithm 18 Temporal Difference $TD(0)$ Learning Algorithm

- 1: **Initialize:** Set $\hat{\mathcal{V}}_0(\mathbf{s})$ arbitrarily for all $\mathbf{s}$.
- 2: **For** $t = 0, 1, 2, \dots$
- 3: Observe: $(\mathbf{s}_t, \mathbf{a}_t \sim \pi(\mathbf{s}_t), \mathbf{r}_t, \mathbf{s}_{t+1})$.
- 4: Update:

$$\hat{\mathcal{V}}_{t+1}(\mathbf{s}) = \begin{cases} \hat{\mathcal{V}}_t(\mathbf{s}_t) + \alpha_t(\mathbf{s}_t) [\mathbf{r}_t + \gamma \hat{\mathcal{V}}_t(\mathbf{s}_{t+1}) - \hat{\mathcal{V}}_t(\mathbf{s}_t)], & \mathbf{s} = \mathbf{s}_t \\ \hat{\mathcal{V}}_t(\mathbf{s}), & \text{otherwise} \end{cases},$$

where $\alpha_t(\mathbf{s})$ is the step size for $\mathbf{s}$ at time t .

We define the *temporal difference* to be

$$\Delta_t = \mathbf{r}_t + \gamma \hat{\mathcal{V}}_t(\mathbf{s}_{t+1}) - \hat{\mathcal{V}}_t(\mathbf{s}_t).$$

The $TD(0)$ update becomes:

$$\hat{\mathcal{V}}_{t+1}(\mathbf{s}_t) = \hat{\mathcal{V}}_t(\mathbf{s}_t) + \alpha_t(\mathbf{s}_t) \Delta_t.$$

As the $TD(0)$ update uses a potentially incorrect $\hat{\mathcal{V}}_t(\mathbf{s}_{t+1})$ to update $\hat{\mathcal{V}}_t(\mathbf{s}_t)$, it is natural to question whether the errors accumulate or vanish as the algorithm progresses. The convergence of value iteration hints that the expected direction of the update is sound, however, we need to account for the stochasticity of the update – we replaced value iteration's $\mathbb{E}^\pi[\mathbf{r}_t + \gamma \hat{\mathcal{V}}_t(\mathbf{s}_{t+1})]$ with a sample, $\mathbf{r}_t + \gamma \hat{\mathcal{V}}_t(\mathbf{s}_{t+1})$. We will show that $TD(0)$ is an instance of the stochastic approximation algorithm, as presented in previously in Section 11.4, and the convergence proof will follow.

Theorem 11.8 (Convergence of $TD(0)$). *If the sequence of learning rates $\{\alpha_t(\mathbf{s})\}$ is well formed then $\hat{\mathcal{V}}_t$ converges to $\mathcal{V}^\pi$ as $t \rightarrow \infty$, with probability 1.*

Proof. We first define a linear operator H for the policy π ,

$$(Hv)(\mathbf{s}) = \mathbf{r}(\mathbf{s}, \pi(\mathbf{s})) + \gamma \sum_{\mathbf{s}'} \mathbf{p}(\mathbf{s}'|\mathbf{s}, \pi(\mathbf{s}))v(\mathbf{s}').$$

Note that H is exactly the operator $\mathcal{T}^\pi$ we defined in Section 6.4.3. Theorem 6.9 shows that the operator H is γ -contracting.

We now would like to re-write the $TD(0)$ update to be a stochastic approximation iterative algorithm. The $TD(0)$ update is,

$$\hat{\mathcal{V}}_{t+1}(\mathbf{s}_t) = (1 - \alpha_t(\mathbf{s}_t))\hat{\mathcal{V}}_t(\mathbf{s}_t) + \alpha_t(\mathbf{s}_t)\Phi_t,$$

where $\Phi_t = \mathbf{r}_t + \gamma\hat{\mathcal{V}}_t(\mathbf{s}_{t+1})$. We would like to consider the expected value of Φ_t . Clearly, $\mathbb{E}[\mathbf{r}_t] = \mathbf{r}(\mathbf{s}_t, \pi(\mathbf{s}_t))$ and $\mathbf{s}_{t+1} \sim \mathbf{p}(\cdot|\mathbf{s}_t, \mathbf{a}_t)$. This implies that $\mathbb{E}[\Phi_t] = (H\hat{\mathcal{V}}_t)(\mathbf{s}_t)$. Therefore, we can define the noise term ω_t as follows,

$$\omega_t(\mathbf{s}_t) = [\mathbf{r}_t + \gamma\hat{\mathcal{V}}_t(\mathbf{s}_{t+1})] - (H\hat{\mathcal{V}}_t)(\mathbf{s}_t),$$

and have $\mathbb{E}[\omega_t|h_{t-1}] = 0$. We can bound $|\omega_t| \leq \mathcal{V}_{\max} = \frac{\mathbf{R}_{\max}}{1-\gamma}$, since the value function is bounded by $\mathcal{V}_{\max}$. Returning to $TD(0)$, we can write

$$\hat{\mathcal{V}}_{t+1}(\mathbf{s}_t) = (1 - \alpha_t(\mathbf{s}_t))\hat{\mathcal{V}}_t(\mathbf{s}_t) + \alpha_t(\mathbf{s}_t)[(H\hat{\mathcal{V}}_t)(\mathbf{s}_t) + \omega_t(\mathbf{s}_t)].$$

The requirement of the step sizes follows since they are well formed. The noise ω_t has both $\mathbb{E}[\omega_t|h_{t-1}] = 0$ and $|\omega_t| \leq \mathcal{V}_{\max}$. The operator H is γ -contracting with a fix-point $\mathcal{V}^\pi$. Therefore, using Theorem 11.5, we established Theorem 11.8. $\square$

If we consider the number of samples used, Algorithm 18 may appear a bit wasteful – we use every sample only once, to effect a small change on the value estimate. We now introduce the *replay buffer* – a conceptual mechanism that will allow to make multiple updates from each sample. In this section we will use the replay buffer to derive a theoretical claim about $TD(0)$, but it has proven to be a useful practical tool in various sampling-based RL algorithms.

Definition 11.4 (Replay Buffer). *A replay buffer $\mathcal{D}$ at time τ with capacity N stores the last N samples $\{(\mathbf{s}_t, \mathbf{a}_t, \mathbf{r}_t, \mathbf{s}_{t+1})\}_{t=\tau-N+1}^\tau$. The replay buffer can be queried for a sample, upon which it returns $(\mathbf{s}_i, \mathbf{a}_i, \mathbf{r}_i, \mathbf{s}_{i+1})$, where $i \sim \text{Unif}(\tau-N+1, \tau)$. Repeated queries are sampled independently and with replacement.*

We can use the replay buffer in line 3 of the TD(0) algorithm to replace the random sample with a query from the replay buffer. We can now ask – given a fixed replay buffer with N samples, what would TD(0) converge to?

The following theorem states that the value of the policy π on the maximum likelihood model given the samples in the replay buffer (Definition 11.2), which is the empirical model, is identical to that of TD(0) run on the replay buffer to convergence.

Theorem 11.9. *Consider a replay buffer $\mathcal{D}$ with N samples. Let $\mathcal{V}_{TD}^\pi$ be the estimated value function of π when we run TD(0) until convergence (sampling from the replay buffer). Let $\mathcal{V}_{EM}^\pi$ be the value function of π on the empirical model calculated from the samples in the replay buffer. Then, $\mathcal{V}_{TD}^\pi = \mathcal{V}_{EM}^\pi$.*

Proof sketch. The update of TD(0) is $\hat{\mathcal{V}}(\mathbf{s}_t) = \hat{\mathcal{V}}(\mathbf{s}_t) + \alpha_t(\mathbf{s}_t)\Delta_t$, where $\Delta_t = \mathbf{r}_t + \gamma\hat{\mathcal{V}}(\mathbf{s}_{t+1}) - \hat{\mathcal{V}}(\mathbf{s}_t)$. At convergence we have $\mathbb{E}[\Delta_t] = 0$ and hence,

$$\hat{\mathcal{V}}(\mathbf{s}) = \frac{1}{n(\mathbf{s}, \mathbf{a})} \sum_{\mathbf{s}_{t+1}: \mathbf{s}_t=\mathbf{s}, \mathbf{a}_t=\mathbf{a}} \mathbf{r}_t + \gamma\hat{\mathcal{V}}(\mathbf{s}_{t+1}) = \hat{\mathbf{r}}(\mathbf{s}) + \gamma\mathbb{E}_{\mathbf{s}' \sim \hat{\mathbf{p}}(\cdot|\mathbf{s}, \mathbf{a})}[\hat{\mathcal{V}}(\mathbf{s}')],$$

where $\mathbf{a} = \pi(\mathbf{s})$. □

It is worth to compare the above theorem to the case of Monte Carlo (Theorem 11.3). In the TD case we are using the entire sample, and we have the same ML model for any state $\mathbf{s}$. In the Monte-Carlo case we used a reduced sample for state $\mathbf{s}$, which depends on the state $\mathbf{s}$, and therefore we have a different ML model for each state, based on its reduced sample.

The next example further demonstrates the statistical benefit of TD(0) versus Monte Carlo.

Example 11.2. *Consider the Markov Chain in Figure 11.6. State $\mathbf{s}_4$ is terminal with zero reward. The initial state is either $\mathbf{s}_1$, with probability $1 - \beta$, or $\mathbf{s}_2$, with probability β , both with zero reward. The reward for state $\mathbf{s}_3$ is a Bernoulli(0.5) random variable. Therefore $\mathcal{V}(\mathbf{s}_1) = \mathcal{V}(\mathbf{s}_2) = \mathcal{V}(\mathbf{s}_3) = 0.5$.*

We ask how many sampled trajectories are required to get an ε -approximation of $\mathcal{V}(\mathbf{s}_2)$, i.e., $|\hat{\mathcal{V}}(\mathbf{s}_2) - 0.5| \approx \varepsilon$. The Monte-Carlo method will require $O(1/(\beta\varepsilon^2))$ episodes, out of which only $O(1/\varepsilon^2)$ start at $\mathbf{s}_2$. On the other hand, TD(0) will require a replay buffer of only $O(1/\varepsilon^2 + 1/\beta)$ since the estimate of $\mathbf{s}_3$ will converge after $1/\varepsilon^2$ episodes which start from either $\mathbf{s}_1$ or $\mathbf{s}_2$, and we only need to observe 1 sample from $\mathbf{s}_2$ – an event that occurs with high probability with $O(1/\beta)$ trajectories.

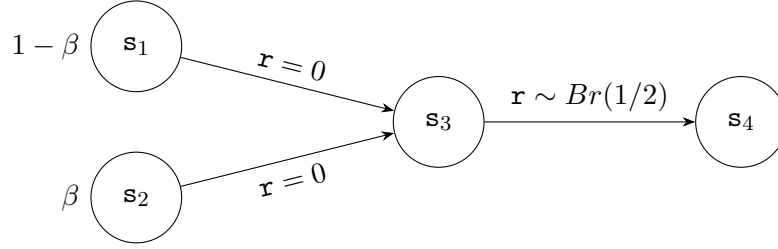


Figure 11.6: Markov Reward Chain

11.5.2 Q -learning: Markov Decision Process

We now extend the Q -learning algorithm from DDP to MDP. The main difference is that now we will need to average multiple observations to converge to the value of Q^* . For this we introduce learning rates for each state-action pair, $\alpha_t(\mathbf{s}, \mathbf{a})$. We allow the learning rate to depend both on the state $\mathbf{s}$, action $\mathbf{a}$ and time $\mathbf{t}$. For example $\alpha_t(\mathbf{s}, \mathbf{a}) = 1/n$ where n is the number of times we updated $(\mathbf{s}, \mathbf{a})$ up to time $\mathbf{t}$. The following is the definition of the algorithm.

Algorithm 19 Q -learning

- 1: **Initialize:** Set $\hat{Q}_0(\mathbf{s}, \mathbf{a}) = 0$, for all $\mathbf{s}, \mathbf{a}$.
- 2: **For** $\mathbf{t} = 0, 1, 2, \dots$
- 3: Observe: $(\mathbf{s}_t, \mathbf{a}_t, \mathbf{r}_t, \mathbf{s}'_t)$.
- 4: Update:

$$\hat{Q}_{t+1}(\mathbf{s}, \mathbf{a}) = \begin{cases} \hat{Q}_t(\mathbf{s}_t, \mathbf{a}_t) + \alpha_t(\mathbf{s}_t, \mathbf{a}_t) \left[\mathbf{r}_t + \gamma \max_{\mathbf{a}'} \hat{Q}_t(\mathbf{s}'_t, \mathbf{a}') - \hat{Q}_t(\mathbf{s}_t, \mathbf{a}_t) \right], & (\mathbf{s}, \mathbf{a}) = (\mathbf{s}_t, \mathbf{a}_t) \\ \hat{Q}_t(\mathbf{s}, \mathbf{a}), & \text{otherwise} \end{cases}$$

It is worth to try and gain some intuition regarding the Q learning algorithm. Let $\Gamma_t = \mathbf{r}_t + \gamma \max_{\mathbf{a}'} \hat{Q}_t(\mathbf{s}'_t, \mathbf{a}') - \hat{Q}_t(\mathbf{s}_t, \mathbf{a}_t)$. For simplicity assume we already converged, $\hat{Q}_t = Q^*$. Then we have that $\mathbb{E}[\Gamma_t] = 0$ and (on average) we maintain that $\hat{Q}_t = Q^*$. Clearly, we do not want to assume that we converge, since this is the entire goal of the algorithm. The main challenge in showing the convergence is that in the updates we use $\hat{Q}_t$ rather than Q^* . We also need to handle the stochastic nature of the updates, where there are both stochastic rewards and stochastic state transitions.

The next theorem states the main convergence property of Q -learning.

Theorem 11.10 (Q -learning convergence).

*Assume every state-action pair $(\mathbf{s}, \mathbf{a})$ occurs infinitely often, and the sequence of learning rates $\{\alpha_t(\mathbf{s}, \mathbf{a})\}$ is well formed. Then, $\hat{Q}_t$ converges with probability 1 to Q^**

Note that the statement of the theorem has two requirements. The first is that every state-action pair occurs infinitely often. This is clearly required for convergence (per state-action). Since Q -learning is an off-policy algorithm, it has no influence on the sequence of state-action it observes, and therefore we have to make this assumption. The second requirement is two properties regarding the learning rates α . The first states that the learning rates are large enough that we can (potentially) reach a value. The second states that the learning rates are sufficiently small (sum of squares finite) so that we will be able to converge locally.

We will show the convergence proof by the general technique of stochastic approximation.

Q -learning as a Stochastic Approximation

After we introduced the stochastic approximation algorithms, and their convergence theorem, we show that the Q -learning algorithm is a stochastic approximation algorithm, and thus converges. To show that Q -learning algorithm is a stochastic approximation algorithm we need to introduce an operator H and the noise ω .

We first define the operator H ,

$$(Hq)(\mathbf{s}, \mathbf{a}) = \sum_{\mathbf{s}'} p(\mathbf{s}'|\mathbf{s}, \mathbf{a}) [r(\mathbf{s}, \mathbf{a}) + \gamma \max_{\mathbf{a}'} q(\mathbf{s}', \mathbf{a}')].$$

The contraction of H is established as follows,

$$\begin{aligned} \|Hq_1 - Hq_2\|_\infty &= \gamma \max_{\mathbf{s}, \mathbf{a}} \left| \sum_{\mathbf{s}'} p(\mathbf{s}'|\mathbf{s}, \mathbf{a}) [\max_{b_1} q_1(\mathbf{s}', b_1) - \max_{b_2} q_2(\mathbf{s}', b_2)] \right| \\ &\leq \gamma \max_{\mathbf{s}, \mathbf{a}} \max_{b, \mathbf{s}'} |q_1(\mathbf{s}', b) - q_2(\mathbf{s}', b)| \\ &\leq \gamma \|q_1 - q_2\|_\infty. \end{aligned}$$

In this section we re-write the Q -learning algorithm to follow the iterative stochastic approximation algorithms, so that we will be able to apply Theorem 11.5.

Recall that,

$$\hat{Q}_{t+1}(\mathbf{s}_t, \mathbf{a}_t) := (1 - \alpha_t(\mathbf{s}_t, \mathbf{a}_t)) \hat{Q}_t(\mathbf{s}_t, \mathbf{a}_t) + \alpha_t(\mathbf{s}_t, \mathbf{a}_t) [r_t + \gamma \max_{\mathbf{a}'} \hat{Q}_t(\mathbf{s}'_t, \mathbf{a}')]]$$

Let $\Phi_t = r_t + \gamma \max_{a'} \hat{Q}_t(s_{t+1}, a')$. This implies that $\mathbb{E}[\Phi_t] = (H\hat{Q}_t)(s_t, a_t)$. We can define the noise term as $\omega_t(s_t, a_t) = \Phi_t - (H\hat{Q}_t)(s_t, a_t)$ and have $\mathbb{E}[\omega_t(s_t, a_t)|h_{t-1}] = 0$. In addition $|\omega_t(s_t, a_t)| \leq V_{\max} = \frac{R_{\max}}{1-\gamma}$.

We can now rewrite the Q -learning, as follows,

$$\hat{Q}_{t+1}(s_t, a_t) := (1 - \alpha_t(s_t, a_t))\hat{Q}_t(s_t, a_t) + \alpha_t(s_t, a_t)[(H\hat{Q}_t)(s_t, a_t) + \omega_t(s_t, a_t)]$$

In order to apply Theorem 11.5, we have the properties of the noise ω_t , and of the contraction of H . Therefore, we can derive Theorem 11.10, since the step size requirement is part of the theorem.

Step Size Selection Rules

The step size has important implication for the convergence of the Q -learning algorithm, and more importantly, on the rate convergence. For the convergence, we need for the step size to have two properties. The first is that sum diverges, i.e., $\sum_t \alpha_t(s, a)I(s_t = s, a_t = a) = \infty$, which intuitively implies that we can potentially reach any value. This is important, since we might have errors on the way, and this guarantees that the step sizes are large enough to possibly correct any error. (It does not guarantee that it will correct the errors, only that the step size is large enough to allow for it.)

The second requirement from the step size is that the sum of squares converges, i.e., $\sum_t \alpha_t^2(s, a)I(s_t = s, a_t = a) = O(1)$. This requirement is that the step size are not too large. It will guarantee that once we are close to the correct value, the step size will be small enough that we actually converge, and not bounce around.

Consider the following experiment. Suppose from some time τ we update using only Q^* , then we clearly would like to converge. For simplicity assume there is no noise, i.e., $\omega_t = 0$ for $t \geq \tau$ and assume a single state, i.e., $Q^* \in \mathbb{R}$. For the update we have that $Q_{t+1} = (1 - \alpha_t)Q_t + \alpha_t Q^*$ or equivalently, $Q_{t+1} = \beta Q_\tau + (1 - \beta)Q^*$, where $\beta = \prod_{i=\tau}^t (1 - \alpha_i)$. We like to have that β converges to 0 and a sufficient condition is that $\sum_i \alpha_i = \infty$. The small enough step size will guarantee that we will converge even when the noise is not zero (but bounded).

Popular choices for the step size are based on the number of visits to (s, a) , which we denote by $n(s, a)$. Two leading examples are:

1. *Linear step size:* $\alpha_t(s, a) = 1/n(s, a)$. We have that $\sum_{n=1}^N 1/n \approx \ln(N)$ and therefore $\sum_{n=1}^{\infty} 1/n = \infty$. Also, $\sum_{n=1}^{\infty} 1/n^2 = \pi^2/6 = O(1)$.
2. *Polynomial step size:* For $\theta \in (1/2, 1)$ we have $\alpha_t(s, a) = 1/(n(s, a))^\theta$. We have that $\sum_{n=1}^N 1/n^\theta \approx (1 - \theta)^{-1} N^{1-\theta}$ and therefore $\sum_{n=1}^{\infty} 1/n^\theta = \infty$. Also, $\sum_{n=1}^{\infty} 1/n^{2\theta} \leq \frac{1}{2\theta-1}$, since $2\theta > 1$.

The linear step size, although many times popular in practice, might lead to slow converges. Here is a simple example. We have a single state $\mathbf{s}$ and single action $\mathbf{a}$ and $\mathbf{r}(\mathbf{s}, \mathbf{a}) = 0$. However, suppose we start with $\hat{Q}_0(\mathbf{s}, \mathbf{a}) = 1$. We will analyze the convergence with the linear step size. Our update is,

$$\hat{Q}_t = \left(1 - \frac{1}{t}\right) \hat{Q}_{t-1} + \frac{1}{t} [0 + \gamma \hat{Q}_{t-1}] \left(1 - \frac{1-\gamma}{t}\right) \hat{Q}_{t-1}.$$

When we solve the recursion we get that $\hat{Q}_t = \Theta(1/t^{1-\gamma})$.⁴ This implies that for $t \leq (1/\varepsilon)^{1/(1-\gamma)}$ we have $\hat{Q}_t \geq \varepsilon$.

In contrast, if we use a polynomial step size, we have,

$$\hat{Q}_t = \left(1 - \frac{1}{t^\theta}\right) \hat{Q}_{t-1} + \frac{1}{t^\theta} [0 + \gamma \hat{Q}_{t-1}] = \left(1 - \frac{1-\gamma}{t^\theta}\right) \hat{Q}_{t-1}.$$

When we solve the recursion we get that $\hat{Q}_t = \Theta(e^{-(1-\gamma)t^{1-\theta}})$. This implies that for $t \geq \frac{1}{1-\gamma} \log^{1/(1-\theta)}(1/\varepsilon)$ we have $\hat{Q}_t \leq \varepsilon$. This is a poly-logarithmic dependency on ε , which is much better. Also, note that θ is under our control, and we can set for example $\theta = 2/3$. Note that unlike θ , the setting of the discount factor γ has a huge influence on the objective function and the effective horizon.

How to obtain samples

We remind the reader that Q -learning algorithm is an off-policy algorithm, and as such it does not specify how obtain the samples in line 3 of Algorithm 19. All it requires is that the samples be exploratory enough to guarantee convergence. Thus, the samples may be collected independently using some exploration policy.

In practice, Q -learning is often applied in an on-policy setting, in which the sample is obtained by interacting with the environment at each time step. In this case, the ε -greedy action selection (Definition 11.1) is a common choice to balance exploration with exploitation.

11.5.3 SARSA: On-Policy Q-Learning

The Q -learning algorithm that we presented is an off-policy algorithm. This implies that it has no control over the action selection. The benefit is that it does not face the exploration exploitation trade-off and its only goal is to approximate the optimal Q function.

⁴ $\hat{Q}_t = \prod_{i=1}^t (1 - (1-\gamma)/i) \approx \prod_{i=1}^t e^{-(1-\gamma)/i} = e^{-\sum_{i=1}^t (1-\gamma)/i} \approx e^{-(1-\gamma) \ln t} = t^{-(1-\gamma)}.$

While Q -learning can be run on-policy, as discussed above, in this section we present a variant of Q -learning that is explicitly on-policy, and interestingly, the choice of actions in the algorithm will drive the convergence of the algorithm to near-optimal behavior. Another motivation for this approach, as we shall see later in this chapter, is that it can easily be modified to use multi-step bootstrapping.

The specific algorithm that we present is called SARSA, and pseudo-code is given in Algorithm 20. The name comes from the fact that the feedback we observe $(\mathbf{s}_t, \mathbf{a}_t, \mathbf{r}_t, \mathbf{s}_{t+1}, \mathbf{a}_{t+1})$, ignoring the subscripts spells SARSA. Note that since it is an on-policy algorithm, the actions are actually under the control of the algorithm, and we need to specify how to select them in Line 4 of the algorithm.

When designing the algorithm we need to think of two contradicting objectives in selecting the actions. The first is the need to explore, perform each action infinitely often. This implies that we need, for each state $\mathbf{s}$ and action $\mathbf{a}$, to have that $\sum_t \pi_t(\mathbf{a}|\mathbf{s}) = \infty$. Then by the Borel-Cantelli lemma we have with probability 1 an infinite number of times that we select action $\mathbf{a}$ in state $\mathbf{s}$ (actually, we need independence of the events, or at least a Martingale property, which holds in our case). On the other hand we would like not only our estimates to converge, as done in Q -learning, but also the return to be near optimal. For this we need the action selection to converge to being greedy with respect to the Q function.

Algorithm 20 SARSA

- 1: **Initialize:** Set $\hat{Q}_0(\mathbf{s}, \mathbf{a}) = 0$, for all $\mathbf{s}, \mathbf{a}$.
- 2: **For** $t = 0, 1, 2, \dots$
- 3: Observe: $(\mathbf{s}_t, \mathbf{a}_t, \mathbf{r}_t, \mathbf{s}_{t+1})$.
- 4: Select $\mathbf{a}_{t+1} = \pi(\mathbf{s}_{t+1}; \hat{Q}_t)$.
- 5: Update:

$$\hat{Q}_{t+1}(\mathbf{s}_t, \mathbf{a}_t) := \hat{Q}_t(\mathbf{s}_t, \mathbf{a}_t) + \alpha_t(\mathbf{s}_t, \mathbf{a}_t) \left[\mathbf{r}_t + \gamma \hat{Q}_t(\mathbf{s}_{t+1}, \mathbf{a}_{t+1}) - \hat{Q}_t(\mathbf{s}_t, \mathbf{a}_t) \right]$$

Selecting the action: As we discussed before, one of the main tasks of an on-policy algorithm is to select the actions. It would be natural to select the action in state $\mathbf{s}_t$ as a function of our current approximation $\hat{Q}_t$ of the optimal Q function.

Given a state $\mathbf{s}$ and a Q , we first define the greedy action in state $\mathbf{s}$ according to Q as

$$\bar{\mathbf{a}} \in \arg \max_{\mathbf{a}} Q(\mathbf{s}, \mathbf{a}).$$

The first idea might be to simply select the greedy action $\bar{a}$, however this might be devastating. The main issue is that we might be avoiding exploration. Some actions might look better due to errors in $\hat{Q}_t$, and we will continue to execute them and not gain any information about alternative actions.

For a concrete example, consider an MDP with a single state and two actions a_a and a_b . The reward of action a_a and a_b are a Bernoulli random variables with parameters $1/3$ and $3/4$, respectively. Assume we initialize $\hat{Q}_0$ to be $\hat{Q}_0(s, a_a) = 0.01$ and $\hat{Q}_0(s, a_b) = 0$. If we execute action $a_0 = a_a$ first, then we have $\hat{Q}_1(s, a_a) > 0$ and $\hat{Q}_1(s, a_b) = 0$. If we select the greedy action, we will *always* select the same action $a_t = a_a$ for all t . We will both be sub-optimal in the return and never explore a_b which will result that we will not converge to Q^* . For this reason we shall not deterministically select the greedy action.

Remark 11.8. *Note that with greedy action selection, the update in SARSA becomes equivalent to the update in Q-learning. However, as outlined above, we understand that SARSA may not converge to an optimal Q function due to it's action selection rule.*

In the following we will present two simple ways to select the action by $\pi(s; \hat{Q})$ stochastically. Both methods will give all actions a non-zero probability, and thus guarantee exploration.

ε -greedy: As already introduced in Definition 11.1, the ε_n -greedy, has as a parameter a sequence of ε_n and selects the actions as follows. Let $n_t(s)$ be the number of times state s was visited up to time t . At time t in state s policy ε_n -greedy (1) with probability $1 - \varepsilon_n$ sets $\pi(s; Q) = \bar{a}$, where $n = n_t(s)$, and (2) with probability $\varepsilon_n/|\mathcal{A}|$, selects $\pi(s; Q) = a$, for each $a \in \mathcal{A}$. Common values for ε_n are linear, $\varepsilon_n = 1/n$, or polynomial, $\varepsilon_n = 1/n^\theta$ for $\theta \in (0.5, 1)$, or constant, $\varepsilon_n = \varepsilon$.

Soft-max The *soft-max*, has as a parameter a sequence of $\beta_t \geq 0$ and selects $\pi(s; Q) = a$, for each $a \in \mathcal{A}$, with probability $\frac{e^{\beta_t Q(s, a)}}{\sum_{a' \in \mathcal{A}} e^{\beta_t Q(s, a')}}$. Note that for $\beta_t = 0$ we get the uniform distribution and for $\beta_t \rightarrow \infty$ we get the maximum. We would like the schedule of the β_t to go to infinity (become greedy) but need it to be slow enough (so that each action appears infinitely often).

SARSA convergence: We will show the convergence of $\hat{Q}_t$ to an ε -close approximation of Q^* under the ε -greedy policies, for a fixed ε . First, we will define an appropriate operator

$$(\mathcal{T}^{*, \varepsilon} q)(s, a) = r(s, a) + \gamma \mathbb{E}_{s' \sim p(\cdot | s, a)} \left[\frac{\varepsilon}{|\mathcal{A}|} \sum_{b' \in \mathcal{A}} q(s', b') + (1 - \varepsilon) \max_{a' \in \mathcal{A}} q(s', a') \right].$$

We claim that $\mathcal{T}^{*,\varepsilon}$ is a γ -contracting operator. This follows since,

$$\begin{aligned} (\mathcal{T}^{*,\varepsilon} q_1 - \mathcal{T}^{*,\varepsilon} q_2)(\mathbf{s}, \mathbf{a}) &= \gamma(1 - \varepsilon) \mathbb{E}_{\mathbf{s}' \sim \mathbf{p}(\cdot|\mathbf{s}, \mathbf{a})} \left[\max_{\mathbf{a}' \in \mathcal{A}} q_1(\mathbf{s}', \mathbf{a}') - \max_{\mathbf{a}'' \in \mathcal{A}} q_2(\mathbf{s}', \mathbf{a}'') \right] \\ &\quad + \gamma \frac{\varepsilon}{|\mathcal{A}|} \sum_{b' \in \mathcal{A}} \mathbb{E}_{\mathbf{s}' \sim \mathbf{p}(\cdot|\mathbf{s}, \mathbf{a})} [q_1(\mathbf{s}', b') - q_2(\mathbf{s}', b')] \\ &\leq \gamma \|q_1 - q_2\|_{\infty}. \end{aligned}$$

Therefore, $\mathcal{T}^{*,\varepsilon}$ will converge to a fix-point $\mathcal{Q}^{*,\varepsilon}$. Now we want to relate the fix-point of $\mathcal{Q}^{*,\varepsilon}$ to the optimal $\mathcal{Q}^*$, which is a fixed point of $\mathcal{T}^*$. For $\mathcal{Q}^{*,\varepsilon}$, since it is the fixed point of $\mathcal{T}^{*,\varepsilon}$, we have

$$\mathcal{Q}^{*,\varepsilon}(\mathbf{s}, \mathbf{a}) = \mathbf{r}(\mathbf{s}, \mathbf{a}) + \gamma \mathbb{E}_{\mathbf{s}' \sim \mathbf{p}(\cdot|\mathbf{s}, \mathbf{a})} \left[\frac{\varepsilon}{|\mathcal{A}|} \sum_{b' \in \mathcal{A}} \mathcal{Q}^{*,\varepsilon}(\mathbf{s}', b') + (1 - \varepsilon) \max_{\mathbf{a}' \in \mathcal{A}} \mathcal{Q}^{*,\varepsilon}(\mathbf{s}', \mathbf{a}') \right].$$

For $\mathcal{Q}^*$, since it is the fix point of $\mathcal{T}^*$, we have

$$\mathcal{Q}^*(\mathbf{s}, \mathbf{a}) = \mathbf{r}(\mathbf{s}, \mathbf{a}) + \gamma \mathbb{E}_{\mathbf{s}' \sim \mathbf{p}(\cdot|\mathbf{s}, \mathbf{a})} [\max_{\mathbf{a}' \in \mathcal{A}} \mathcal{Q}^*(\mathbf{s}', \mathbf{a}')].$$

Let $\Delta^\varepsilon = \|\mathcal{Q}^{*,\varepsilon} - \mathcal{Q}^*\|_{\infty}$. We have

$$\begin{aligned} \mathcal{Q}^{*,\varepsilon}(\mathbf{s}, \mathbf{a}) - \mathcal{Q}^*(\mathbf{s}, \mathbf{a}) &= (1 - \varepsilon) \gamma \mathbb{E}_{\mathbf{s}' \sim \mathbf{p}(\cdot|\mathbf{s}, \mathbf{a})} \left[\max_{\mathbf{a}' \in \mathcal{A}} \mathcal{Q}^{*,\varepsilon}(\mathbf{s}', \mathbf{a}') - \max_{\mathbf{a}'' \in \mathcal{A}} \mathcal{Q}^*(\mathbf{s}', \mathbf{a}'') \right] \\ &\quad + \varepsilon \gamma \mathbb{E}_{\mathbf{s}' \sim \mathbf{p}(\cdot|\mathbf{s}, \mathbf{a})} \left[\frac{1}{|\mathcal{A}|} \sum_{b' \in \mathcal{A}} \mathcal{Q}^{*,\varepsilon}(\mathbf{s}', b') - \max_{\mathbf{a}'' \in \mathcal{A}} \mathcal{Q}^*(\mathbf{s}', \mathbf{a}'') \right] \\ &\leq \gamma(1 - \varepsilon) \Delta^\varepsilon + \varepsilon \gamma V_{\max}. \end{aligned}$$

Let $(\mathbf{s}, \mathbf{a})$ be the state-action that determines Δ^ε . Then we have

$$\Delta^\varepsilon \leq \gamma \Delta^\varepsilon + \varepsilon \gamma V_{\max},$$

which implies that

$$\Delta^\varepsilon \leq \varepsilon \frac{\gamma V_{\max}}{1 - \gamma}.$$

Theorem 11.11. *Using ε -greedy the SARSA algorithm converges to $\mathcal{Q}^{*,\varepsilon}$ and $\|\mathcal{Q}^{*,\varepsilon} - \mathcal{Q}^*\|_{\infty} \leq \varepsilon \frac{\gamma V_{\max}}{1 - \gamma}$.*

The convergence of the return of SARSA to that of Q^* is more delicate. Recall that we do not have such a claim about Q -learning, since it is an off-policy method, and does not prescribe a policy. We shall show that the value of the ε -greedy policy with respect to $Q^{*,\varepsilon}$ is not much worse than the optimal value.

The following lemma shows that if we have a strategy which is greedy with respect to a near optimal Q function, then the policy is near optimal.

Lemma 11.12. *Let Q such that $\|Q - Q^*\|_\infty \leq \Delta$, and π the greedy policy w.r.t. Q , i.e., $\pi(\mathbf{s}) \in \arg \max_{\mathbf{a}} Q(\mathbf{s}, \mathbf{a})$. Then, $V^*(\mathbf{s}) - V^\pi(\mathbf{s}) \leq \frac{2\Delta}{1-\gamma}$.*

Proof. First, we show that for any state $\mathbf{s}$ we have $V^*(\mathbf{s}) - Q^*(\mathbf{s}, \pi(\mathbf{s})) \leq 2\Delta$. Since $\|Q - Q^*\|_\infty \leq \Delta$ we have $|Q^*(\mathbf{s}, \pi(\mathbf{s})) - Q(\mathbf{s}, \pi(\mathbf{s}))| \leq \Delta$ and $|Q^*(\mathbf{s}, \mathbf{a}^*) - Q(\mathbf{s}, \mathbf{a}^*)| \leq \Delta$, where $\mathbf{a}^*$ is the optimal action in state $\mathbf{s}$. This implies that $Q^*(\mathbf{s}, \mathbf{a}^*) - Q^*(\mathbf{s}, \pi(\mathbf{s})) \leq Q(\mathbf{s}, \mathbf{a}^*) - Q(\mathbf{s}, \pi(\mathbf{s})) + 2\Delta$. Since policy π is greedy w.r.t. Q we have $Q(\mathbf{s}, \pi(\mathbf{s})) \geq Q(\mathbf{s}, \mathbf{a}^*)$, and hence $V^*(\mathbf{s}) - Q^*(\mathbf{s}, \pi(\mathbf{s})) = Q^*(\mathbf{s}, \mathbf{a}^*) - Q^*(\mathbf{s}, \pi(\mathbf{s})) \leq 2\Delta$.

Next,

$$V^*(\mathbf{s}) = Q^*(\mathbf{s}, \mathbf{a}^*) \leq Q^*(\mathbf{s}, \pi(\mathbf{s})) + 2\Delta = \mathbb{E}[r_0] + \gamma \mathbb{E}[V^*(\mathbf{s}_1)] + 2\Delta,$$

where $\mathbf{r}_0 = \mathbb{E}[R(\mathbf{s}, \pi(\mathbf{s}))]$ and $\mathbf{s}_1$ is the state reached when doing action $\pi(\mathbf{s})$ in state $\mathbf{s}$. As we roll out to time t we have,

$$V^*(\mathbf{s}) \leq \mathbb{E} \left[\sum_{i=0}^{t-1} \gamma^i r_i \right] + \gamma^t \mathbb{E}[V^*(\mathbf{s}_t)] + \sum_{i=1}^t 2\Delta \gamma^i,$$

where r_i is the reward in time i in state $\mathbf{s}_i$, $\mathbf{s}_{i+1}$ is the state reached when doing action $\pi(\mathbf{s}_i)$ in state $\mathbf{s}_i$, and we start with $\mathbf{s}_0 = \mathbf{s}$. This implies that in the limit we have

$$V^*(\mathbf{s}) \leq V^\pi(\mathbf{s}) + \frac{2\Delta}{1-\gamma},$$

since $V^\pi(\mathbf{s}) = \mathbb{E}[\sum_{i=0}^{\infty} \gamma^i r_i]$. □

The above lemma uses the greedy policy, but as we discussed before, we would like to add exploration. We would like to claim that if ε is small, then the difference in return between the greedy policy and the ε -greedy policy would be small. We will show a more general result, showing that for any policy, if we add a perturbation of ε to the action selection, then the effect on the expected return is at most $O(\varepsilon)$.

Fix a policy π and let π_ε be a policy such that for any state $\mathbf{s}$ we have that $\|\pi(\cdot|\mathbf{s}) - \pi_\varepsilon(\cdot|\mathbf{s})\|_1 \leq \varepsilon$. Namely, there is a policy $\rho(\mathbf{a}|\mathbf{s})$ such that $\pi_\varepsilon(\mathbf{a}|\mathbf{s}) = (1 - \varepsilon)\pi(\mathbf{a}|\mathbf{s}) + \varepsilon\rho(\mathbf{a}|\mathbf{s})$. Hence, at any state, with probability at least $1 - \varepsilon$ policy π_ε and policy π use the same action selection.

Lemma 11.13. Fix π_ε and policy π such that for any state $\mathbf{s}$ we have that $\|\pi(\cdot|\mathbf{s}) - \pi_\varepsilon(\cdot|\mathbf{s})\|_1 \leq \varepsilon$. Then, for any state $\mathbf{s}$ we have

$$|\mathcal{V}^{\pi_\varepsilon}(\mathbf{s}) - \mathcal{V}^\pi(\mathbf{s})| \leq \frac{\varepsilon\gamma\mathbf{R}_{\max}}{(1-\gamma)(1-\gamma(1-\varepsilon))} \leq \frac{\varepsilon\mathbf{R}_{\max}}{(1-\gamma)^2}.$$

Proof. Let $\mathbf{r}_t$ be the reward of policy π at time t . By definition $\mathcal{V}^\pi(\mathbf{s}) = \mathbb{E}[\sum_{t=0}^{\infty} \gamma^t \mathbf{r}_t]$. The probability that policy π_ε never deviated from π until time t is $(1-\varepsilon)^t$. Therefore we can lower bound the expected reward of policy π_ε by $\mathcal{V}^{\pi_\varepsilon}(\mathbf{s}) \geq \mathbb{E}[\sum_{t=0}^{\infty} (1-\varepsilon)^t \gamma^t \mathbf{r}_t]$.

Consider the difference between the expected returns,

$$\mathcal{V}^\pi(\mathbf{s}) - \mathcal{V}^{\pi_\varepsilon}(\mathbf{s}) \leq \mathbb{E}[\sum_{t=0}^{\infty} \gamma^t \mathbf{r}_t] - \mathbb{E}[\sum_{t=0}^{\infty} (1-\varepsilon)^t \gamma^t \mathbf{r}_t]$$

Since the rewards are bounded, namely, $\mathbf{r}_t \in [0, \mathbf{R}_{\max}]$, the difference is maximized if we set all the rewards to $\mathbf{R}_{\max}$, and have

$$\begin{aligned} \mathcal{V}^\pi(\mathbf{s}) - \mathcal{V}^{\pi_\varepsilon}(\mathbf{s}) &\leq \sum_{t=0}^{\infty} \gamma^t \mathbf{R}_{\max} - \sum_{t=0}^{\infty} (1-\varepsilon)^t \gamma^t \mathbf{R}_{\max} \\ &= \frac{\mathbf{R}_{\max}}{1-\gamma} - \frac{\mathbf{R}_{\max}}{1-\gamma(1-\varepsilon)} \\ &= \frac{\varepsilon\gamma\mathbf{R}_{\max}}{(1-\gamma)(1-\gamma(1-\varepsilon))} \end{aligned}$$

□

We can now combine the results of Theorem 11.11, Lemma 11.12, and Lemma 11.13 and claim that SARSA with ε -greedy converges to a near optimal policy.

Theorem 11.14. The SARSA algorithm with an ε -greedy policy with a fixed ε , converges to a policy with value $\mathcal{V}^{\pi_\varepsilon}$, and we have that for all state $\mathbf{s} \in \mathcal{S}$,

$$\mathcal{V}^{\pi_\varepsilon}(\mathbf{s}) \geq \mathcal{V}^*(\mathbf{s}) - \frac{2\varepsilon(\gamma\mathbf{V}_{\max} + \mathbf{R}_{\max})}{(1-\gamma)^2}.$$

The bound in Theorem 11.14 depends linearly on the fixed exploration parameter ε . In practice, we may want to decrease ε with time to converge to an optimal value. With an appropriate schedule ε_n , convergence may also be established using a refined stochastic approximation technique, see [142] for more details.

Algorithms for Episodic MDPs

Modifying the learning algorithms above from the discounted to the episodic setting requires a simple but important change. We show it here for Q -learning, but the extension to the other algorithms is immediate.

Algorithm 21 Q -Learning for Episodic MDPs

- 1: **Initialize:** Set $\hat{Q}_0(\mathbf{s}, \mathbf{a}) = 0$, for all $\mathbf{s}, \mathbf{a}$.
- 2: **For** $t = 0, 1, 2, \dots$
- 3: Observe: $(\mathbf{s}_t, \mathbf{a}_t, \mathbf{r}_t, \mathbf{s}'_t)$.
- 4: Update:

$$\hat{Q}_{t+1}(\mathbf{s}_t, \mathbf{a}_t) := \begin{cases} \hat{Q}_t(\mathbf{s}_t, \mathbf{a}_t) + \alpha_t(\mathbf{s}_t, \mathbf{a}_t) \left[\mathbf{r}_t + \max_{\mathbf{a}'} \hat{Q}_t(\mathbf{s}'_t, \mathbf{a}') - \hat{Q}_t(\mathbf{s}_t, \mathbf{a}_t) \right] & \mathbf{s}'_t \notin \mathcal{S}_G \\ \hat{Q}_t(\mathbf{s}_t, \mathbf{a}_t) + \alpha_t(\mathbf{s}_t, \mathbf{a}_t) \left[\mathbf{r}_t - \hat{Q}_t(\mathbf{s}_t, \mathbf{a}_t) \right] & \mathbf{s}'_t \in \mathcal{S}_G \end{cases}$$

Note that we removed the discount factor, and also explicitly used the fact that the value of a goal state is 0. The latter is critical for the algorithm to converge, under the Assumption 7.1 that a goal state will always be reached.

11.5.4 Multiple Look-Ahead Algorithms

The $TD(0)$ algorithm uses only the current reward and state. Given $(\mathbf{s}_t, \mathbf{a}_t, \mathbf{r}_t, \mathbf{s}_{t+1})$ it updates $\Delta_t = R_t^{(1)}(\mathbf{s}_t) - \hat{V}(\mathbf{s}_t)$ where $R_t^{(1)}(\mathbf{s}_t) = \mathbf{r}_t + \gamma \hat{V}(\mathbf{s}_{t+1})$, and note that we drop the iteration subscript from $\hat{V}$ for readability in this subsection. We can also consider a two step look-ahead as follows. Given $(\mathbf{s}_t, \mathbf{a}_t, \mathbf{r}_t, \mathbf{s}_{t+1}, \mathbf{a}_{t+1}, \mathbf{r}_{t+1}, \mathbf{s}_{t+2})$ we can update using $\Delta_t^{(2)} = R_t^{(2)}(\mathbf{s}_t) - \hat{V}(\mathbf{s}_t)$ where $R_t^{(2)}(\mathbf{s}_t) = \mathbf{r}_t + \gamma \mathbf{r}_{t+1} + \gamma^2 \hat{V}(\mathbf{s}_{t+2})$. Using the same logic, we have that this is a temporal difference that uses two time steps.

We can generalize this to any n -step look-ahead and define $R_t^{(n)}(\mathbf{s}_t) = \sum_{i=0}^{n-1} \gamma^i \mathbf{r}_{t+i} + \gamma^n \hat{V}(\mathbf{s}_{t+n})$ and updates $\Delta_t^{(n)} = R_t^{(n)}(\mathbf{s}_t) - \hat{V}(\mathbf{s}_t)$.

We can relate the $\Delta_t^{(n)}$ to the Δ_t as follows:

$$\Delta_t^{(n)} = \sum_{i=0}^{n-1} \gamma^i \Delta_{t+i}.$$

This follows since

$$\begin{aligned}
\sum_{i=0}^{n-1} \gamma^i \Delta_{\mathbf{t}+i} &= \sum_{i=0}^{n-1} \gamma^i \mathbf{r}_{\mathbf{t}+i} + \sum_{i=0}^{n-1} \gamma^{i+1} \hat{\mathcal{V}}(\mathbf{s}_{\mathbf{t}+i+1}) - \sum_{i=0}^{n-1} \gamma^i \hat{\mathcal{V}}(\mathbf{s}_{\mathbf{t}+i}) \\
&= \sum_{i=0}^{n-1} \gamma^i \mathbf{r}_{\mathbf{t}+i} + \gamma^n \hat{\mathcal{V}}(\mathbf{s}_{\mathbf{t}+n}) - \hat{\mathcal{V}}(\mathbf{s}_{\mathbf{t}}) \\
&= R_{\mathbf{t}}^{(n)}(\mathbf{s}_{\mathbf{t}}) - \hat{\mathcal{V}}(\mathbf{s}_{\mathbf{t}}) = \Delta_{\mathbf{t}}^{(n)}.
\end{aligned}$$

Using the n -step look-ahead we have the update $\hat{\mathcal{V}}(\mathbf{s}_{\mathbf{t}}) := \hat{\mathcal{V}}(\mathbf{s}_{\mathbf{t}}) + \alpha_{\mathbf{t}} \Delta_{\mathbf{t}}^{(n)}$ where $\Delta_{\mathbf{t}}^{(n)} = R_{\mathbf{t}}^{(n)}(\mathbf{s}_{\mathbf{t}}) - \hat{\mathcal{V}}(\mathbf{s}_{\mathbf{t}})$. We note that $R_{\mathbf{t}}^{(n)}$ corresponds to the operator $(\mathcal{T}^\pi)^n$, which is γ^n -contracting, namely

$$\|(\mathcal{T}^\pi)^n(V_1) - (\mathcal{T}^\pi)^n(V_2)\|_\infty \leq \gamma^n \|V_1 - V_2\|_\infty.$$

We can use any parameter n for the n -step look-ahead. If the episode ends before step n we can pad it with zero rewards. This implies that for $n = \infty$ we have that n -step look-ahead is simply the Monte-Carlo estimate. However, we need to select some parameter n . An alternative idea is to simply average over the possible parameters n . One simple way to average is to use exponential averaging with a parameter $\lambda \in (0, 1)$. This implies that the weight of each parameter n is $(1 - \lambda)\lambda^{n-1}$, and that the sum of the weights is 1.

This leads us to the $TD(\lambda)$ update:

$$\hat{\mathcal{V}}(\mathbf{s}_{\mathbf{t}}) := \hat{\mathcal{V}}(\mathbf{s}_{\mathbf{t}}) + \alpha_{\mathbf{t}}(1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} \Delta_{\mathbf{t}}^{(n)}.$$

Remark: While both γ and λ are used to generate exponential decaying values, their goals are very different. The discount parameter γ defines the objective of the MDP, the goal that we like to maximize. The exponential averaging parameter λ is used by the learning algorithm to average over the different look-ahead parameters, and is selected to optimize the algorithm's convergence to the value defined by γ .

The above describes the forward view of $TD(\lambda)$, where we average over future rewards. If we will try to implement it in a strict way this will lead us to wait until the end of the episode, since we will need to first observe all the rewards. Fortunately, there is an equivalent *backward view* form of $TD(\lambda)$. The backward view updates the value estimates at each time step, using incomplete information. At the end of the episode, the updates of the forward and backward updates will be the same.

The basic idea of the backward view is the following. Fix a time $\mathbf{t}$ and a state $\mathbf{s} = \mathbf{s}_{\mathbf{t}}$. We have at time $\mathbf{t}$ a temporal difference $\Delta_{\mathbf{t}} = \mathbf{r}_{\mathbf{t}} + \gamma \hat{\mathcal{V}}(\mathbf{s}_{\mathbf{t}+1}) - \hat{\mathcal{V}}(\mathbf{s}_{\mathbf{t}})$. Consider how this $\Delta_{\mathbf{t}}$ affects all the previous times $\tau < \mathbf{t}$ where $\mathbf{s}_{\tau} = \mathbf{s} = \mathbf{s}_{\mathbf{t}}$. The influence is exactly $(\gamma\lambda)^{\mathbf{t}-\tau} \Delta_{\mathbf{t}}$. This implies that for every previous time step τ we can perform a part of their $TD(\lambda)$ update. More elegantly, however, we can aggregate all those updates to a single operation by introducing another state-dependent variable. Let,

$$e_{\mathbf{t}}(\mathbf{s}) = \sum_{\tau \leq \mathbf{t}: \mathbf{s}_{\tau} = \mathbf{s}} (\gamma\lambda)^{\mathbf{t}-\tau} = \sum_{\tau=1}^{\mathbf{t}} (\gamma\lambda)^{\mathbf{t}-\tau} \mathbb{I}(\mathbf{s}_{\tau} = \mathbf{s}).$$

The above $e_{\mathbf{t}}(\mathbf{s})$ defines the *eligibility trace* and we can compute it online using

$$e_{\mathbf{t}}(\mathbf{s}) = \gamma\lambda e_{\mathbf{t}-1}(\mathbf{s}) + \mathbb{I}(\mathbf{s} = \mathbf{s}_{\mathbf{t}}), \quad \forall \mathbf{s} \in \mathcal{S},$$

where we initialize the eligibility trace to 0 for all states. We obtain the update

$$\hat{\mathcal{V}}(\mathbf{s}) := \hat{\mathcal{V}}(\mathbf{s}) + \alpha_{\mathbf{t}} e_{\mathbf{t}}(\mathbf{s}) \Delta_{\mathbf{t}}, \quad \forall \mathbf{s} \in \mathcal{S}.$$

Note that in the backward view of $TD(\lambda)$ we update the values of all states at each iteration. The special case $\lambda = 0$ is the $TD(0)$ algorithm, where we have that the eligibility trace becomes $e_{\mathbf{t}}(\mathbf{s}) = I(\mathbf{s} = \mathbf{s}_{\mathbf{t}})$, and we recover the $TD(0)$ algorithm that updates only $\mathbf{s}_{\mathbf{t}}$ according to $\hat{\mathcal{V}}(\mathbf{s}_{\mathbf{t}}) := \hat{\mathcal{V}}(\mathbf{s}_{\mathbf{t}}) + \alpha_{\mathbf{t}} \Delta_{\mathbf{t}}$. Pseudo-code for $TD(\lambda)$ is given in Algorithm 22.

Algorithm 22 $TD(\lambda)$ Learning Algorithm

- 1: **Initialize:** Set $\hat{\mathcal{V}}_0(\mathbf{s})$ arbitrarily and $e_{-1}(\mathbf{s}) = 0$ for all $\mathbf{s} \in \mathcal{S}$.
- 2: **For** $\mathbf{t} = 0, 1, 2, \dots$
- 3: Observe: $(\mathbf{s}_{\mathbf{t}}, \mathbf{a}_{\mathbf{t}} \sim \pi(\mathbf{s}_{\mathbf{t}}), \mathbf{r}_{\mathbf{t}}, \mathbf{s}_{\mathbf{t}+1})$
- 4: Compute TD error:

$$\Delta_{\mathbf{t}} = \mathbf{r}_{\mathbf{t}} + \gamma \hat{\mathcal{V}}_{\mathbf{t}}(\mathbf{s}_{\mathbf{t}+1}) - \hat{\mathcal{V}}_{\mathbf{t}}(\mathbf{s}_{\mathbf{t}})$$

- 5: Update eligibility traces:

$$e_{\mathbf{t}}(\mathbf{s}) = \gamma\lambda e_{\mathbf{t}-1}(\mathbf{s}) + \mathbb{I}[\mathbf{s} = \mathbf{s}_{\mathbf{t}}], \quad \forall \mathbf{s} \in \mathcal{S}$$

- 6: Update value function:

$$\hat{\mathcal{V}}_{\mathbf{t}+1}(\mathbf{s}) = \hat{\mathcal{V}}_{\mathbf{t}}(\mathbf{s}) + \alpha_{\mathbf{t}} \cdot e_{\mathbf{t}}(\mathbf{s}) \cdot \Delta_{\mathbf{t}}, \quad \forall \mathbf{s} \in \mathcal{S}$$

	Δ_0	Δ_1	Δ_2	Δ_3	Δ_4	Δ_5	Δ_6	Δ_7
$\mathbf{s}_0 = \mathbf{s}$	1	$\lambda\gamma$	$(\lambda\gamma)^2$	$(\lambda\gamma)^3$	$(\lambda\gamma)^4$	$(\lambda\gamma)^5$	$(\lambda\gamma)^6$	$(\lambda\gamma)^7$
$\mathbf{s}_2 = \mathbf{s}$			1	$\lambda\gamma$	$(\lambda\gamma)^2$	$(\lambda\gamma)^3$	$(\lambda\gamma)^4$	$(\lambda\gamma)^5$
$\mathbf{s}_5 = \mathbf{s}$						1	$\lambda\gamma$	$(\lambda\gamma)^2$
	$e_0(\mathbf{s})$	$e_1(\mathbf{s})$	$e_2(\mathbf{s})$	$e_3(\mathbf{s})$	$e_4(\mathbf{s})$	$e_5(\mathbf{s})$	$e_6(\mathbf{s})$	$e_7(\mathbf{s})$

Table 11.1: An example for $TD(\lambda)$ updates of state $\mathbf{s}$ that occurs at times 0, 2 and 5, i.e., $\mathbf{s}_0 = \mathbf{s}$, $\mathbf{s}_2 = \mathbf{s}$, and $\mathbf{s}_5 = \mathbf{s}$. The forward update appears in the rows. Each column is the coefficients of the update of Δ_i , and their sum equals $e_i(\mathbf{s})$.

To summarize, the benefit of $TD(\lambda)$ is that it interpolates between $TD(0)$ and Monte-carlo updates, and in practice often achieves superior performance to both. Similarly to $TD(0)$, $TD(\lambda)$ can also be written as a stochastic approximation iterative algorithm, and its convergence can be established (see [62] for details). In the next section we show the equivalence of the forward and backward $TD(\lambda)$ updates.

The Equivalence of the Forward and Backward View

We would like to show that the forward and backward views indeed result in the same overall update.

Consider the following intuition. Assume that state $\mathbf{s}$ occurs in time τ_1 . This occurrence will contribute to the forward view $\sum_{\mathbf{t}=0}^{\infty} \lambda^{\mathbf{t}} \gamma^{\mathbf{t}} \Delta_{\tau_1+\mathbf{t}}$. The same contribution applies to any time τ_j where state $\mathbf{s}$ occurs. The sum of those contributions would be $\sum_{j=1}^M \lambda^{\mathbf{t}} \gamma^{\mathbf{t}} \Delta_{\tau_j+\mathbf{t}}$, where M is the number of occurrences of state $\mathbf{s}$. Now we compute the contribution of any update Δ_i . The updates of Δ_i will contribute to any update of state $\mathbf{s}$ which occurs at time $\tau_j \leq i$. The total update of Δ_i would be $\sum_{\tau_j \leq i} (\lambda\gamma)^{i-\tau_j} \Delta_i$. Note that $e_i(\mathbf{s})$ at time i equals to $\sum_{\tau_j \leq i} (\lambda\gamma)^{i-\tau_j}$, which implies that the update equals $e_i(\mathbf{s}) \Delta_i$. So the sum of the updates should be equivalent. Table 11.1 has an illustrative example. We will derive it more formally in the proof that follows.

For the forward view we define the updates to be $\Delta V_{\mathbf{t}}^F(\mathbf{s}) = \alpha(R_{\mathbf{t}}^{\lambda} - V_{\mathbf{t}}(\mathbf{s}))$, where $R_{\mathbf{t}}^{\lambda} = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} R_{\mathbf{t}}^{(n)}$. Equivalently, $\Delta V_{\mathbf{t}}^F(\mathbf{s}) = \alpha(1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} \Delta_{\mathbf{t}}^{(n)}$.

For the backward view we define the updates to be $\Delta V_{\mathbf{t}}^B(\mathbf{s}) = \alpha \Delta_{\mathbf{t}} e_{\mathbf{t}}(\mathbf{s})$, where the eligibility trace is $e_{\mathbf{t}}(\mathbf{s}) = \sum_{k=0}^{\mathbf{t}} (\lambda\gamma)^{\mathbf{t}-k} \mathbb{I}(\mathbf{s} = \mathbf{s}_k)$.

Theorem 11.15. *For any state $\mathbf{s}$*

$$\sum_{\mathbf{t}=0}^{\infty} \Delta V_{\mathbf{t}}^B(\mathbf{s}) = \sum_{\mathbf{t}=0}^{\infty} \Delta V_{\mathbf{t}}^F(\mathbf{s}) \mathbb{I}(\mathbf{s}_{\mathbf{t}} = \mathbf{s}).$$

Proof. First, we note a useful fact for interchanging the order of summations:

$$\sum_{i=0}^{\infty} \sum_{j=0}^i f(i, j) = \sum_{j=0}^{\infty} \sum_{i=j}^{\infty} f(i, j). \quad (11.2)$$

Consider the sum of the forward updates for state $\mathbf{s}$:

$$\begin{aligned} \sum_{\mathbf{t}=0}^{\infty} \Delta V_{\mathbf{t}}^F(\mathbf{s}) \mathbb{I}(\mathbf{s} = \mathbf{s}_{\mathbf{t}}) &= \sum_{\mathbf{t}=0}^{\infty} \alpha(1 - \lambda) \sum_{n=\mathbf{t}}^{\infty} \lambda^{n-\mathbf{t}} \Delta_{\mathbf{t}}^{(n-\mathbf{t})} \mathbb{I}(\mathbf{s} = \mathbf{s}_{\mathbf{t}}) \\ &= \sum_{\mathbf{t}=0}^{\infty} \alpha(1 - \lambda) \sum_{n=\mathbf{t}}^{\infty} \lambda^{n-\mathbf{t}} \sum_{i=0}^{n-\mathbf{t}} \gamma^i \Delta_{\mathbf{t}+i} \mathbb{I}(\mathbf{s} = \mathbf{s}_{\mathbf{t}}) \\ &= \sum_{\mathbf{t}=0}^{\infty} \alpha(1 - \lambda) \sum_{k=0}^{\infty} \lambda^k \sum_{i=0}^k \gamma^i \Delta_{\mathbf{t}+i} \mathbb{I}(\mathbf{s} = \mathbf{s}_{\mathbf{t}}) \\ &= \sum_{\mathbf{t}=0}^{\infty} \alpha(1 - \lambda) \sum_{i=0}^{\infty} \sum_{k=i}^{\infty} \lambda^k \gamma^i \Delta_{\mathbf{t}+i} \mathbb{I}(\mathbf{s} = \mathbf{s}_{\mathbf{t}}) \\ &= \sum_{\mathbf{t}=0}^{\infty} \alpha(1 - \lambda) \sum_{i=0}^{\infty} \lambda^i \gamma^i \Delta_{\mathbf{t}+i} \mathbb{I}(\mathbf{s} = \mathbf{s}_{\mathbf{t}}) \sum_{k=i}^{\infty} \lambda^{k-i} \\ &= \sum_{\mathbf{t}=0}^{\infty} \sum_{i=0}^{\infty} \alpha(\lambda \gamma)^i \Delta_{\mathbf{t}+i} \mathbb{I}(\mathbf{s} = \mathbf{s}_{\mathbf{t}}) \\ &= \sum_{\mathbf{t}=0}^{\infty} \sum_{k=\mathbf{t}}^{\infty} \alpha(\lambda \gamma)^{k-\mathbf{t}} \Delta_k \mathbb{I}(\mathbf{s} = \mathbf{s}_{\mathbf{t}}), \end{aligned} \quad (11.3)$$

where the first identity is the definition, the second identity follows since $\Delta_{\mathbf{t}}^{(n)} = \sum_{i=0}^n \gamma^i \Delta_{\mathbf{t}+i}$, in the third identity we substitute k for $n - \mathbf{t}$. In the fourth equality we use (11.2) and in the fifth equality we reorder the terms and use $\lambda^k = \lambda^{k-i} \lambda^i$. In the sixth equality we use $\sum_{i=0}^{\infty} (1 - \lambda) \lambda^i = 1$. In the last identity we substitute k for $i + \mathbf{t}$.

For the backward view for state $\mathbf{s}$ we have

$$\begin{aligned}
\sum_{\mathbf{t}=0}^{\infty} \Delta V_{\mathbf{t}}^B(\mathbf{s}) &= \sum_{\mathbf{t}=0}^{\infty} \alpha \Delta_{\mathbf{t}}(\mathbf{s}) e_{\mathbf{t}}(\mathbf{s}) \\
&= \sum_{\mathbf{t}=0}^{\infty} \alpha \Delta_{\mathbf{t}}(\mathbf{s}) \sum_{k=0}^{\mathbf{t}} (\gamma \lambda)^{\mathbf{t}-k} \mathbb{I}(\mathbf{s} = \mathbf{s}_k) \\
&= \sum_{k=0}^{\infty} \sum_{\mathbf{t}=k}^{\infty} \alpha (\gamma \lambda)^{\mathbf{t}-k} \Delta_{\mathbf{t}}(\mathbf{s}) \mathbb{I}(\mathbf{s} = \mathbf{s}_k). \tag{11.4}
\end{aligned}$$

Note that if we interchange k and $\mathbf{t}$ in Eq. (11.3) and in Eq. (11.4), then we have the identical expressions. $\square$

The astute reader might have noticed a subtle difference between the equivalence proved in Theorem 11.15 and the implementation of $TD(\lambda)$ in Algorithm 22: in our proof we ignored the fact that future temporal difference terms may have a different value function due to the value function updates in previous steps. In [167], a more elaborate backward view that takes the updates into account was proposed.

The $SARSA(\lambda)$ Algorithm

We can use the idea of eligibility traces also in other algorithms, such as SARSA. Recall that given $(\mathbf{s}_{\mathbf{t}}, \mathbf{a}_{\mathbf{t}}, \mathbf{r}_{\mathbf{t}}, \mathbf{s}_{\mathbf{t}+1}, \mathbf{a}_{\mathbf{t}+1})$, the update of SARSA is

$$\mathbf{r}_{\mathbf{t}} + \gamma \hat{Q}_{\mathbf{t}}(\mathbf{s}_{\mathbf{t}+1}, \mathbf{a}_{\mathbf{t}+1}) - \hat{Q}_{\mathbf{t}}(\mathbf{s}_{\mathbf{t}}, \mathbf{a}_{\mathbf{t}}).$$

Similarly, we can define an n -step look-ahead $q_{\mathbf{t}}^{(n)} = \sum_{i=0}^{n-1} \gamma^i \mathbf{r}_{\mathbf{t}+i} + \gamma^n \hat{Q}_{\mathbf{t}}(\mathbf{s}_{\mathbf{t}+n}, \mathbf{a}_{\mathbf{t}+n})$ and set $\hat{Q}_{\mathbf{t}+1}(\mathbf{s}_{\mathbf{t}}, \mathbf{a}_{\mathbf{t}}) = \hat{Q}_{\mathbf{t}}(\mathbf{s}_{\mathbf{t}}, \mathbf{a}_{\mathbf{t}}) + \alpha_{\mathbf{t}} (q_{\mathbf{t}}^{(n)} - \hat{Q}_{\mathbf{t}}(\mathbf{s}_{\mathbf{t}}, \mathbf{a}_{\mathbf{t}}))$.

We can now define $SARSA(\lambda)$ using exponential averaging with parameter λ . Namely, we define $q_{\mathbf{t}}^{\lambda} = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} q_{\mathbf{t}}^{(n)}$. This makes the forward view of $SARSA(\lambda)$ to be $\hat{Q}_{\mathbf{t}+1}(\mathbf{s}_{\mathbf{t}}, \mathbf{a}_{\mathbf{t}}) = \hat{Q}_{\mathbf{t}}(\mathbf{s}_{\mathbf{t}}, \mathbf{a}_{\mathbf{t}}) + \alpha_{\mathbf{t}} (q_{\mathbf{t}}^{\lambda} - \hat{Q}_{\mathbf{t}}(\mathbf{s}_{\mathbf{t}}, \mathbf{a}_{\mathbf{t}}))$. Similar to $TD(\lambda)$, we can implement a backward view using eligibility traces, see Algorithm 23.

Algorithm 23 SARSA(λ) Learning Algorithm

- 1: **Initialize:** Set $\hat{Q}_0(\mathbf{s}, \mathbf{a}) = 0$ and $e_{-1}(\mathbf{s}, \mathbf{a}) = 0$ for all $\mathbf{s} \in \mathcal{S}, \mathbf{a} \in \mathcal{A}$.
- 2: **For** $t = 0, 1, 2, \dots$
- 3: Observe: $(\mathbf{s}_t, \mathbf{a}_t, \mathbf{r}_t, \mathbf{s}_{t+1})$.
- 4: Select: $\mathbf{a}_{t+1} = \pi(\mathbf{s}_{t+1}; \hat{Q}_t)$.
- 5: Compute TD error:

$$\Delta_t = \mathbf{r}_t + \gamma \hat{Q}_t(\mathbf{s}_{t+1}, \mathbf{a}_{t+1}) - \hat{Q}_t(\mathbf{s}_t, \mathbf{a}_t)$$

- 6: Update eligibility traces:

$$e_t(\mathbf{s}, \mathbf{a}) = \gamma \lambda e_{t-1}(\mathbf{s}, \mathbf{a}) + \mathbb{I}[\mathbf{s} = \mathbf{s}_t, \mathbf{a} = \mathbf{a}_t], \quad \forall \mathbf{s} \in \mathcal{S}, \mathbf{a} \in \mathcal{A}$$

- 7: Update $\hat{Q}$ values:

$$\hat{Q}_{t+1}(\mathbf{s}, \mathbf{a}) = \hat{Q}_t(\mathbf{s}, \mathbf{a}) + \alpha_t \cdot e_t(\mathbf{s}, \mathbf{a}) \cdot \Delta_t, \quad \forall \mathbf{s} \in \mathcal{S}, \mathbf{a} \in \mathcal{A}$$

11.6 On Policy vs. Off Policy and Importance Sampling

In this section we elaborate on the distinction between on-policy and off-policy algorithms. Recall that in an on-policy method the action selection strategy is dictated by the algorithm (e.g., TD(0) and SARSA), while in an off-policy method it can be arbitrary (e.g., Q -learning). Note that even though an off-policy method does not dictate the action selection strategy, we may still require some conditions on the data samples generated by the actions to guarantee convergence (e.g., infinitely often state visitations in Q -learning).

When we use a replay buffer in our algorithm, the on-policy vs. off-policy distinction becomes even more important. For an off-policy method like Q -learning, any previous sample $(\mathbf{s}_t, \mathbf{a}_t, \mathbf{r}_t, \mathbf{s}_{t+1})$ that we encountered is valid for use in the update, therefore we can use an arbitrarily large replay buffer, and maximally exploit all the samples we have collected – important if samples are costly. Algorithm 24 demonstrates this explicitly by performing K updates of the Q function for every sample collected.

Algorithm 24 Q -Learning with Replay Buffer

- 1: **Initialize:** Set $\hat{Q}(\mathbf{s}, \mathbf{a}) = 0$, for all $\mathbf{s}$, $\mathbf{a}$, empty replay buffer $\mathcal{D} = \emptyset$.
- 2: **For** $t = 0, 1, 2, \dots$
- 3: Observe sample $(\mathbf{s}_t, \mathbf{a}_t, \mathbf{r}_t, \mathbf{s}'_t)$ and add to $\mathcal{D}$
- 4: **For** $i = 0, 1, \dots, K$
- 5: Sample $(\mathbf{s}, \mathbf{a}, \mathbf{r}, \mathbf{s}')$ from $\mathcal{D}$
- 6: Update:

$$\hat{Q}(\mathbf{s}, \mathbf{a}) := \hat{Q}(\mathbf{s}, \mathbf{a}) + \alpha_t(\mathbf{s}, \mathbf{a}) \left[\mathbf{r} + \gamma \max_{\mathbf{a}'} \hat{Q}(\mathbf{s}', \mathbf{a}') - \hat{Q}(\mathbf{s}, \mathbf{a}) \right],$$

For an on-policy method like SARSA, on the other hand, we can only use samples from the most recent policy, which means that we are limited to a small replay buffer, and effectively must discard samples from early policies.

A natural question arises: can we use samples from some behavior policy π_b in an on-policy algorithm that expects samples from a policy π ? The key technical idea we build on is *importance sampling* – how to evaluate an expectation under some distribution using samples from a different distribution.

Importance Sampling

Importance sampling is a simple general technique to estimate the mean with respect to a given distribution, while sampling from a different distribution. To be specific, let Q be the sampling distribution and P the evaluation distribution. The basic idea is the following:

$$\mathbb{E}_{x \sim P}[f(x)] = \sum_x P(x) f(x) = \sum_x Q(x) \frac{P(x)}{Q(x)} f(x) = \mathbb{E}_{x \sim Q} \left[\frac{P(x)}{Q(x)} f(x) \right].$$

This implies that given a sample $\{x_1, \dots, x_m\}$ from Q , we can estimate $\mathbb{E}_{x \sim P}[f(x)]$ using $\sum_{i=1}^m \frac{P(x_i)}{Q(x_i)} f(x_i)$. Importance sampling gives an unbiased estimator, but the variance of the estimator might be huge, since it depends on $P(x)/Q(x)$.

We would like to apply the idea of importance sampling to learning in MDPs. Assume that there is a behavior policy π_b that selects the actions, and there is a policy π that we would like to evaluate. For the importance sampling, given a trajectory, we need to take the ratio of probabilities under π and π_b .

$$\frac{\Pr^\pi(\mathbf{s}_1, \mathbf{a}_1, \mathbf{r}_1, \dots, \mathbf{s}_T, \mathbf{a}_T, \mathbf{r}_T, \mathbf{s}_{T+1})}{\Pr^{\pi_b}(\mathbf{s}_1, \mathbf{a}_1, \mathbf{r}_1, \dots, \mathbf{s}_T, \mathbf{a}_T, \mathbf{r}_T, \mathbf{s}_{T+1})} = \prod_{t=1}^T \frac{\pi(\mathbf{a}_t | \mathbf{s}_t)}{\pi_b(\mathbf{a}_t | \mathbf{s}_t)},$$

where the equality follows since the reward and transition probabilities are identical, and cancel out.

For Monte-Carlo policy evaluation, the estimates would be

$$G^{\pi/\pi_b} = \prod_{t=1}^T \frac{\pi(\mathbf{a}_t|\mathbf{s}_t)}{\pi_b(\mathbf{a}_t|\mathbf{s}_t)} \left(\sum_{t=1}^T \mathbf{r}_t \right),$$

and the update for a step size α would be

$$\hat{\mathcal{V}}^\pi(\mathbf{s}_1) = \hat{\mathcal{V}}^\pi(\mathbf{s}_1) + \alpha \left(G^{\pi/\pi_b} - \hat{\mathcal{V}}^\pi(\mathbf{s}_1) \right).$$

These updates might be huge, since we are multiplying the ratios of many small numbers.

For the $TD(0)$, the importance-weighted estimate is

$$\Delta_t^{\pi/\pi_b} = \frac{\pi(\mathbf{a}_t|\mathbf{s}_t)}{\pi_b(\mathbf{a}_t|\mathbf{s}_t)} \left(\mathbf{r}_t + \gamma \hat{\mathcal{V}}(\mathbf{s}_{t+1}) \right)$$

and the temporal difference updates will be

$$\hat{\mathcal{V}}^\pi(\mathbf{s}_1) = \hat{\mathcal{V}}^\pi(\mathbf{s}_1) + \alpha (\Delta_t^{\pi/\pi_b} - \hat{\mathcal{V}}^\pi(\mathbf{s}_1)).$$

This update is much more stable, since we have only one factor multiplying the observed reward.

Example 11.3. *Consider an MDP with a single state and two actions (also called multi-arm bandit, which we will cover in Chapter 14). We consider a finite horizon return with parameter T . Policy π_b at each time selects one of the two actions uniformly at random. The policy π selects action one always.*

Using the Monte Carlo approach, when considering complete trajectories, only after expected 2^T trajectories we have a trajectory in which for T times action one was selected. (Note that the update will have weight 2^T .)

Using the $TD(0)$ updates, each time action one is selected by π we can update the estimates of π (with a factor of 2).

To compare the two approaches, consider the number of trajectories required to get an ϵ approximation for the return of π . Using Monte-Carlo, we need $O(T2^T/\epsilon^2)$ trajectories, in expectation. In contrast, for $TD(0)$ we need only $O(T/\epsilon^2)$ trajectories. The huge gap is due to the fact that $TD(0)$ utilizes partial trajectories while Monte-Carlo requires the entire trajectory to agree with π .

11.7 Bibliography Notes

The Monte-Carlo approach dates back to the 1940's [109]. Monte Carlo methods were introduced to reinforcement learning in [8]. The comparison of the **First Visit** and **Every Visit** of Monte-Carlo algorithms is based on [143].

The Temporal differences method was introduced in [148], where $TD(0)$ is introduced. The $TD(\lambda)$ was analyzed in [38, 39]. A more refined version, termed “True Online $TD(\lambda)$ ” and its analysis appears in [167, 166].

Q-learning was introduced and first analyzed in [171]. The step size analysis of Q-learning and non-asymptotic convergence rates were derived in [46]. The asymptotic convergence of Q-learning and Temporal differences was given in [62, 163]. The SARSA algorithm was introduced in [125, 143], and its convergence proved in [142]. The expected SARSA was presented in [168].

The stochastic approximation method was introduced by Robbins and Monro [124]. For an extensive survey of this literature we refer to [86]. The ODE method was pioneered by Ljung [99, 100] and further developed by Kushner [87, 85]. An extensive treatment on stochastic approximation appears in [86, 17, 22]

Chapter 12

Large State Spaces: Value Function Approximation

This chapter starts looking at the case where the MDP is large. In the current chapter we will look at approximating the value function. In the next chapter we will consider learning directly a policy and optimizing it.

When we talk about a large MDP, it can be due to a few different reasons. The most common is having a large state space $\mathcal{S}$. For example, Backgammon has over 10^{20} states, Go has over 10^{170} , and robot control typically has a continuous state space. The *curse of dimensionality* is a common term for this problem, and relates to states that are composed of several state variables. For example, the configuration of a robot manipulator with N joints can be described using N variables for the angles at each joint. Assuming that each variable can take on M different values, the size of the state space, M^N , grows exponentially with the number of state variables N .

Another reason why an MDP can be large is the size of the action space $\mathcal{A}$, which can even be continuous in some applications (say, robots). Finally, we might have complex dynamics $\mathbf{P}$ that are hard to describe succinctly (e.g., the next state is the result of a complex simulation) or are not even known to sufficient accuracy.

Recall Bellman's dynamic programming equation,

$$\mathcal{V}(\mathbf{s}) = \max_{\mathbf{a} \in \mathcal{A}} \left\{ \mathbf{r}(\mathbf{s}, \mathbf{a}) + \gamma \sum_{\mathbf{s}' \in \mathcal{S}} \mathbf{p}(\mathbf{s}' | \mathbf{s}, \mathbf{a}) \mathcal{V}(\mathbf{s}') \right\} \quad \forall \mathbf{s} \in \mathcal{S}.$$

Dynamic programming requires knowing the model transitions $\mathbf{P}$ and is only feasible for small problems, where iterating over all states and actions is feasible. The model-free and model-based learning algorithms described in Chapters 11 and 10 do not require knowing the model, but require storing either value estimates *for each* state

and action (in model free), or state transition probabilities for every possible state, action, and next state (in model based). Scaling up our planning and RL algorithms to very large state and action spaces is the challenge we shall address in this chapter.

Chapter Orientation

Main Prerequisites:

- Model Free RL (Chapter 11)

Main terminology:

- $\mathbf{w}$ parameters (weights) in function approximation
- $\hat{\mathcal{V}}^\pi(\mathbf{s}; \mathbf{w})$ approximation of value function for policy π with weights $\mathbf{w}$
- $\hat{\mathcal{V}}^*(\mathbf{s}; \mathbf{w})$ approximation of optimal value function with weights $\mathbf{w}$
- $\hat{\mathcal{Q}}^\pi(\mathbf{s}, \mathbf{a}; \mathbf{w})$ approximation of state-action value function for policy π with weights $\mathbf{w}$
- $\hat{\mathcal{Q}}^*(\mathbf{s}, \mathbf{a}; \mathbf{w})$ approximation of optimal state-action value function with weights $\mathbf{w}$
- $\phi(\mathbf{s})$ feature vector for state $\mathbf{s}$
- $\phi(\mathbf{s}, \mathbf{a})$ feature vector for state $\mathbf{s}$ and action $\mathbf{a}$
- Φ matrix with $\phi(\mathbf{s})$ on its rows
- $\hat{\Phi}$ empirical feature matrix (a.k.a. design matrix)
- ξ vector of state probabilities
- Ξ diagonal matrix with ξ on its diagonal
- $\|\cdot\|_\xi$ ξ -weighted Euclidean norm
- Π_ξ linear projection w.r.t. ξ -weighted Euclidean norm
- $\hat{\mathcal{S}}$ linear subspace spanned by Φ

- A, b matrix and vector that define the solution of the projected Bellman equation, in Equation (12.10)
- $\pi_{\text{greedy}}(\mathcal{V})$ greedy policy w.r.t. value function $\mathcal{V}$
- $\pi_{\text{T-greedy}}(\mathcal{V})$ T-step greedy policy w.r.t. value function $\mathcal{V}$

12.1 Approximation approaches

There are 4 main approaches for handling large MDPs:

1. **Myopic:** When $p(\mathbf{s}'|\mathbf{s}, \mathbf{a})$ is approximately uniform across $\mathbf{a}$ (that is, the actions do not have much effect on the transition to the next state), we may ignore the state transitions and simply use $\pi(\mathbf{s}) \approx \underset{\mathbf{a} \in \mathcal{A}}{\operatorname{argmax}} \{r(\mathbf{s}, \mathbf{a})\}$. If $r(\mathbf{s}, \mathbf{a})$ is not known exactly – replace it with an estimate.
2. **Lookahead policies:** also known as rolling horizon/model-predictive control. At each step $\mathbf{t}$, observe the state $\mathbf{s}_{\mathbf{t}}$ and simulate a horizon of T steps to calculate

$$\pi = \underset{\pi' \in \Pi}{\operatorname{argmax}} \mathbb{E}^{\pi'} \left[\sum_{\mathbf{t}'=\mathbf{t}}^{\mathbf{t}+T} r(\mathbf{s}_{\mathbf{t}'}, \mathbf{a}_{\mathbf{t}'}) \middle| \mathbf{s}_{\mathbf{t}} \right].$$

Then, apply the *first* action $\pi(\mathbf{s}_{\mathbf{t}})$ from the calculated T-step optimal policy, observe the resulting state transition to $\mathbf{s}_{\mathbf{t}+1}$, and repeat.

3. **Policy function approximation**

Assume the policy is of some parametric form $\pi(\mathbf{s}) = \pi(\mathbf{s}; \mathbf{w})$, $\mathbf{w} \in \mathbb{W}$, and optimize over parameters $\mathbf{w}$.

4. **Value function approximation**

Assume the value function is of some parametric form and optimize over parameters $\mathbf{w}$; the current chapter focuses on this approach. When considering a value function approximation, there are a few interpretations of what exactly we mean. Given a policy π , it can either be: (1) mapping from a state $\mathbf{s}$ to its expected return, i.e., $\hat{\mathcal{V}}^{\pi}(\mathbf{s}; \mathbf{w})$. (2) mapping from state-action pairs $(\mathbf{s}, \mathbf{a})$ to their expected return, i.e., $\hat{\mathcal{Q}}^{\pi}(\mathbf{s}, \mathbf{a}; \mathbf{w})$. (3) mapping from states $\mathbf{s}$ to expected return of each action, i.e., $\{\hat{\mathcal{Q}}^{\pi}(\mathbf{s}, \mathbf{a}_i; \mathbf{w}) : \mathbf{a}_i \in \mathcal{A}\}$. All the interpretations are valid, and our discussion will not distinguish between them (actually, for

$\{\widehat{Q}^\pi(\mathbf{s}, \mathbf{a}_i; \mathbf{w}) : \mathbf{a}_i \in \mathcal{A}\}$ we implicitly assume that the number of actions is small). We shall also be interested in approximating the optimal value function, and the corresponding approximations are denoted $\widehat{V}^*(\mathbf{s}; \mathbf{w})$, $\widehat{Q}^*(\mathbf{s}, \mathbf{a}; \mathbf{w})$, and $\{\widehat{Q}^*(\mathbf{s}, \mathbf{a}_i; \mathbf{w}) : \mathbf{a}_i \in \mathcal{A}\}$, respectively.

Given an approximate $\widehat{Q}^*(\mathbf{s}, \mathbf{a}; \mathbf{w})$, we can derive an approximately optimal policy by choosing the *greedy* action with respect to $\widehat{Q}^*(\mathbf{s}, \mathbf{a}; \mathbf{w})$.

The 4 approaches above for dealing with large MDPs are not mutually exclusive and in practice, combining different approaches often produces the best performance. For example, a common approach is to combine a T step lookahead with an approximate terminal value function,

$$\pi(\mathbf{s}_t) = \operatorname{argmax}_{\pi' \in \Pi} \mathbb{E}^{\pi'} \left[\sum_{t'=t}^{t+T-1} r(\mathbf{s}_{t'}) + \widehat{V}^*(\mathbf{s}_{t+T}) \right].$$

We shall also see, in the next chapter, that value function approximations will be a useful component in approximate policy optimization. In the rest of this chapter, we focus on value function approximation. We will consider (mainly) the discounted return with a discount parameter $\gamma \in (0, 1)$. The results extend naturally to the finite horizon and episodic settings.

12.1.1 Value Function Approximation Architectures

We now discuss how to build the approximating function. For this we can consider popular hypothesis classes from the machine learning literature. For example: (1) Linear functions, (2) Neural networks, (3) Decision trees, (4) Nearest neighbors, (5) Fourier or wavelet basis, etc. We will concentrate here on linear functions.

In a linear function approximation, we represent the value as a weighted combination of some d features:

$$\widehat{V}^\pi(\mathbf{s}; \mathbf{w}) = \sum_{j=1}^d \mathbf{w}_j \phi_j(\mathbf{s}) = \mathbf{w}^T \phi(\mathbf{s}),$$

where $\mathbf{w} \in \mathbb{R}^d$ are the function approximation parameters and $\phi(\mathbf{s}) \in \mathbb{R}^d$ are the states' *features* (a.k.a. basis functions). Similarly, for state-action value functions, we use state-action features, $\phi(\mathbf{s}, \mathbf{a}) \in \mathbb{R}^d$, and approximate the value by $\widehat{Q}^\pi(\mathbf{s}, \mathbf{a}; \mathbf{w}) = \mathbf{w}^T \phi(\mathbf{s}, \mathbf{a})$.

Popular examples of state feature vectors include radial basis functions $\phi_j(\mathbf{s}) \propto \exp\left(-\frac{\|\mathbf{s}-\mu_j\|^2}{2\sigma_j^2}\right)$, and tile features, where $\phi_j(\mathbf{s}) = 1$ for a set of states $A_j \subset \mathcal{S}$, and

$\phi_j(\mathbf{s}) = 0$ otherwise. For state-action features, when the number of actions is finite $\mathcal{A} = \{1, 2, \dots, |\mathcal{A}|\}$, a common approach is to extend the state features independently for every action. That is, consider the following construction for $\phi(\mathbf{s}, i) \in \mathbb{R}^{d \cdot |\mathcal{A}|}$, $i \in \mathcal{A}$:

$$\phi(\mathbf{s}, i)^T = \left[\underbrace{\mathbf{0}^T, \mathbf{0}^T, \dots, \mathbf{0}^T}_{i-1 \text{ times}}, \phi(\mathbf{s})^T, \underbrace{\mathbf{0}^T, \dots, \mathbf{0}^T}_{|\mathcal{A}|-i \text{ times}} \right],$$

where $\mathbf{0}$ is a vector of d zeros.

However, for most interesting settings designing appropriate features is a difficult problem that requires significant domain knowledge. In the following, we assume that the features $\phi(\mathbf{s})$ (or $\phi(\mathbf{s}, \mathbf{a})$) are given to us in advance, and we concentrate on general methods for learning weights $\mathbf{w}$ that minimize the approximation error as best as possible, with respect to the available features.

Remark 12.1 (Tabular features). *Note that the value functions without function approximation studied earlier (sometimes referred to as the tabular setting, since the value function can be represented as a table of states and values) may be seen as a special case of linear function approximation, by choosing tabular features $\phi(\mathbf{s}, \mathbf{a}) \in \mathbb{R}^{|\mathcal{S}| \times |\mathcal{A}|}$, such that $[\phi(\mathbf{s}, \mathbf{a})]_{\mathbf{s}', \mathbf{a}'} = \mathbb{I}\{\mathbf{s}' = \mathbf{s}, \mathbf{a}' = \mathbf{a}\}$. In this case, each element $\mathbf{s}, \mathbf{a}$ of the weight vector will encode the value of that particular state $\mathbf{s}$ and action $\mathbf{a}$. Tabular features are useful as a sanity check – the algorithms we develop with linear function approximation should provide the same guarantees we already established in the setting without function approximation when using tabular features.*

12.2 Quantification of Approximation Error

Before we start the discussion on learning methods, we take a small detour to discuss the effect of error in the value function. Assume we have a value function $\hat{\mathcal{V}}$ such that $\|\hat{\mathcal{V}} - \mathcal{V}^*\|_\infty \leq \varepsilon$. Let $\hat{\pi}$ be the greedy policy with respect to $\hat{\mathcal{V}}$, namely,

$$\hat{\pi}(\mathbf{s}) = \arg \max_{\mathbf{a}} [\mathbf{r}(\mathbf{s}, \mathbf{a}) + \gamma \mathbb{E}_{\mathbf{s}' \sim \mathbf{p}(\cdot | \mathbf{s}, \mathbf{a})} [\hat{\mathcal{V}}(\mathbf{s}')]].$$

Theorem 12.1. *Let $\hat{\mathcal{V}}$ such that $\|\hat{\mathcal{V}} - \mathcal{V}^*\|_\infty \leq \varepsilon$ and $\hat{\pi}$ be the greedy policy with respect to $\hat{\mathcal{V}}$. Then,*

$$\|\mathcal{V}^{\hat{\pi}} - \mathcal{V}^*\|_\infty \leq \frac{2\gamma\varepsilon}{1-\gamma}.$$

Proof. Consider two operators $\mathcal{T}^\pi$ and $\mathcal{T}^*$ (see Chapter 6.4.3). The first, $\mathcal{T}^\pi$, is

$$(\mathcal{T}^\pi v)(\mathbf{s}) = \mathbf{r}(\mathbf{s}, \pi(\mathbf{s})) + \gamma \mathbb{E}_{\mathbf{s}' \sim \mathbf{p}(\cdot | \mathbf{s}, \pi(\mathbf{s}))} [v(\mathbf{s}')],$$

and it converges to $\mathcal{V}^\pi$ (see Theorem 6.9). The second, $\mathcal{T}^*$, is

$$(\mathcal{T}^*v)(\mathbf{s}) = \max_{\mathbf{a}} [\mathbf{r}(\mathbf{s}, \mathbf{a}) + \gamma \mathbb{E}_{\mathbf{s}' \sim \mathbf{p}(\cdot|\mathbf{s}, \mathbf{a})} [v(\mathbf{s}')]],$$

and it converges to $\mathcal{V}^*$ (see Theorem 6.9). In addition, we have shown that both $\mathcal{T}^\pi$ and $\mathcal{T}^*$ are γ -contracting (see Theorem 6.9).

Since $\hat{\pi}$ is greedy with respect to $\hat{\mathcal{V}}$ we have $\mathcal{T}^{\hat{\pi}}\hat{\mathcal{V}} = \mathcal{T}^*\hat{\mathcal{V}}$ (but this does not hold for other value functions $\mathcal{V}' \neq \hat{\mathcal{V}}$).

Then,

$$\begin{aligned} \|\mathcal{V}^{\hat{\pi}} - \mathcal{V}^*\|_\infty &= \|\mathcal{T}^{\hat{\pi}}\mathcal{V}^{\hat{\pi}} - \mathcal{V}^*\|_\infty \\ &\leq \|\mathcal{T}^{\hat{\pi}}\mathcal{V}^{\hat{\pi}} - \mathcal{T}^{\hat{\pi}}\hat{\mathcal{V}}\|_\infty + \|\mathcal{T}^{\hat{\pi}}\hat{\mathcal{V}} - \mathcal{V}^*\|_\infty \\ &\leq \gamma\|\mathcal{V}^{\hat{\pi}} - \hat{\mathcal{V}}\|_\infty + \|\mathcal{T}^*\hat{\mathcal{V}} - \mathcal{T}^*\mathcal{V}^*\|_\infty \\ &\leq \gamma\|\mathcal{V}^{\hat{\pi}} - \hat{\mathcal{V}}\|_\infty + \gamma\|\hat{\mathcal{V}} - \mathcal{V}^*\|_\infty \\ &\leq \gamma(\|\mathcal{V}^{\hat{\pi}} - \mathcal{V}^*\|_\infty + \|\mathcal{V}^* - \hat{\mathcal{V}}\|_\infty) + \gamma\|\hat{\mathcal{V}} - \mathcal{V}^*\|_\infty, \end{aligned}$$

where in the second inequality we used the fact that since π is greedy with respect to $\hat{\mathcal{V}}$ then $\mathcal{T}^{\hat{\pi}}\hat{\mathcal{V}} = \mathcal{T}^*\hat{\mathcal{V}}$.

Reorganizing the inequality and recalling that $\|\mathcal{V}^* - \hat{\mathcal{V}}\|_\infty \leq \varepsilon$, we have

$$(1 - \gamma)\|\mathcal{V}^{\hat{\pi}} - \mathcal{V}^*\|_\infty \leq 2\varepsilon\gamma,$$

and the theorem follows. $\square$

The above theorem states that if we have small errors in L_∞ norm, the effect of the errors on the expected return is bounded. However, in most cases we will not be able to guarantee an approximation in norm L_∞ . In fact, even if $\mathcal{V}^*$ is given to us, it is infeasible to compute $\|\mathcal{V}^{\hat{\pi}} - \mathcal{V}^*\|_\infty$ in the large state space setting, as it requires iterating over *all* states.

Intuitively, a more feasible guarantee is that some *average* error is small. In the following, we shall see that this condition can be represented mathematically as a weighted L_2 norm. Extending Theorem 12.1 to a weighted L_2 norm is possible, but is technically involved [112], and we will not derive it in this book. Nevertheless, we shall next study learning algorithms that have a guaranteed average error bound.

12.3 Approximate Policy Evaluation

We begin our investigation with the problem of approximating the value function of a given policy. We shall later discuss methods that improve a policy, or approximate the optimal value function.

12.3.1 From RL to Supervised Learning

To learn the value function, we would like to reduce our reinforcement learning problem to a supervised learning problem. This will enable us to use any of the many techniques of machine learning to address the problem. Let us consider the basic ingredients of supervised learning. The first ingredient is a labeled training set that is sampled i.i.d.

Let us start by considering an idealized setting. Fix a policy π , and consider learning its value function $\hat{V}^\pi$. To apply supervised learning, we should generate a training set, i.e.,

$$\{(\mathbf{s}_1, \mathcal{V}^\pi(\mathbf{s}_1)), \dots, (\mathbf{s}_m, \mathcal{V}^\pi(\mathbf{s}_m))\},$$

where $\mathcal{V}^\pi(\mathbf{s})$ is the label of a sample state $\mathbf{s}$.

We first need to discuss how to sample the states $\mathbf{s}_i$ in an i.i.d. way. We can generate a trajectory, but we need to be careful, since adjacent states are definitely dependent! One solution is to space the sampling from the trajectory using the mixing time of π .¹ This will give us samples $\mathbf{s}_i$ which are sampled (almost) from the stationary distribution of π and are (almost) independent. In the episodic setting, we can sample different episodes, and states from different episodes are guaranteed to be independent.

Second, we need to define a loss function, which will tradeoff the different approximation errors. Since the value is a real scalar, a natural candidate is the average squared error loss, $\frac{1}{m} \sum_{i=1}^m (\hat{V}^\pi(\mathbf{s}_i) - \mathcal{V}^\pi(\mathbf{s}_i))^2$. With this loss, the corresponding supervised learning problem is least squares regression.

The hardest, and most confusing, ingredient is generating the labels $\mathcal{V}^\pi(\mathbf{s}_i)$. In supervised machine learning we assume that someone gives us the labels to build a classifier. However, in our problem, the value function is exactly what we want to learn, and it is not realistic to assume any ground truth samples from it!

Our main task, therefore, would be to replace the ground truth labels with quantities that we can measure, using simulation or interaction with the system. We shall start by formally defining least squares regression in a way that will be convenient to extend later to RL.

12.3.2 Preliminaries – Least Squares Regression

To simplify our formulation, we will assume that the state space may be very large, but finite. Equivalently, we will consider a regression problem where the independent variable can only take a finite set of values.

¹See Chapter 4 for definition.

Assume we have some function $y = f(x)$, where $y \in \mathbb{R}$, $x \in X$, and X is finite. As usual, x is termed the independent variable, while y is the dependent variable. We assume that data is generated by sampling i.i.d. from a distribution $\xi(x)$, and the labels are noisy. That is, we are given N labeled samples $\{(x_1, y_1), \dots, (x_N, y_N)\}$, where $x_i \sim \xi(x)$, $y_i = f(x_i) + \omega(x_i)$, and $\omega(x)$ is an independent zero-mean noise (which may depend on the state).

Our goal is to fit to our data a parametric function $g(x; \mathbf{w}) : X \rightarrow \mathbb{R}$, where $\mathbf{w} \in \mathbb{R}^d$, such that g approximates f well. The Least Squares approach solves the following problem:

$$\hat{\mathbf{w}}_{LS} = \min_{\mathbf{w}} \frac{1}{N} \sum_{i=1}^N (g(x_i; \mathbf{w}) - y_i)^2. \quad (12.1)$$

A practical iterative algorithm for solving (12.1) is the stochastic gradient descent (SGD) method, which updates the parameters by

$$\mathbf{w}_{i+1} = \mathbf{w}_i - \alpha_i (g(x_i; \mathbf{w}_i) - y_i) \nabla_{\mathbf{w}} g(x_i; \mathbf{w}_i), \quad (12.2)$$

where α_i is some step size schedule, such as $\alpha_i = 1/i$, and $\nabla_{\mathbf{w}} g(x_i; \mathbf{w}_i)$ means the gradient of $g(x_i; \mathbf{w})$ with respect to $\mathbf{w}$ evaluated at $\mathbf{w}_i$.

When g is linear in some features $\phi(x)$, i.e., $g(x; \mathbf{w}) = \mathbf{w}^T \phi(x)$, the least squares solution can be calculated explicitly. Let $\hat{\Phi} \in \mathbb{R}^{N \times d}$ be a matrix with $\phi(x_i)$ in its rows, often called the *design matrix*. Similarly, let $\hat{Y} \in \mathbb{R}^{N \times 1}$ be a vector of y_i 's. Then, Equation (12.1) can be written as

$$\hat{\mathbf{w}}_{LS} = \min_{\mathbf{w}} \frac{1}{N} (\hat{\Phi} \mathbf{w} - \hat{Y})^T (\hat{\Phi} \mathbf{w} - \hat{Y}) = \min_{\mathbf{w}} \frac{1}{N} \left(\mathbf{w}^T (\hat{\Phi}^T \hat{\Phi}) \mathbf{w} - 2 \mathbf{w}^T \hat{\Phi}^T \hat{Y} + \hat{Y}^T \hat{Y} \right). \quad (12.3)$$

Noting that (12.3) is a quadratic form, the least squares solution is:

$$\hat{\mathbf{w}}_{LS} = (\hat{\Phi}^T \hat{\Phi})^{-1} \hat{\Phi}^T \hat{Y}. \quad (12.4)$$

We now show that the LS solution (12.4) converges to the *expected* least squares solution when $N \rightarrow \infty$. Without loss of generality, we assume that the states are ordered, $1, 2, \dots, |X|$. Let $\xi \in \mathbb{R}^{|X|}$ denote a vector of probabilities for each state $\xi(x)$, and define the diagonal matrix $\Xi = \text{diag}(\xi) \in \mathbb{R}^{|X| \times |X|}$. Further, let $\Phi \in \mathbb{R}^{|X| \times d}$ be a matrix with $\phi(x)$ as its rows, and let $Y \in \mathbb{R}^{|X|}$ be a vector of $f(x)$.

Proposition 12.2. *Assume that $\Phi^T \Xi \Phi$ is not singular (i.e., no feature is a linear combination of other features and each state has a positive probability). Then $\lim_{N \rightarrow \infty} \hat{\mathbf{w}}_{LS} = \mathbf{w}_{LS}$, where*

$$\mathbf{w}_{LS} = (\Phi^T \Xi \Phi)^{-1} \Phi^T \Xi Y.$$

Proof. From the law of large numbers, we have that

$$\begin{aligned}
\lim_{N \rightarrow \infty} \frac{1}{N} \hat{\Phi}^T \hat{\Phi} &= \lim_{N \rightarrow \infty} \frac{1}{N} \sum_{i=1}^N \phi(x_i) \phi(x_i)^T \\
&= \mathbb{E}_{x \sim \xi(x)} [\phi(x) \phi(x)^T] \\
&= \sum_x \xi(x) \phi(x) \phi(x)^T \\
&= \Phi^T \Xi \Phi.
\end{aligned}$$

Similarly, $\lim_{N \rightarrow \infty} \frac{1}{N} \hat{\Phi}^T \hat{Y} = \Phi^T \Xi Y$. Plugging into Eq. (12.4) completes the proof. $\square$

Using the stochastic approximation technique, a similar result holds for the SGD update.

Proposition 12.3. *Consider the SGD update in Eq. (12.2) with linear features,*

$$\mathbf{w}_{i+1} = \mathbf{w}_i - \alpha_i (\mathbf{w}^T \phi(x_i) - y_i) \phi(x_i).$$

Assume that $\Phi^T \Xi \Phi$ is not singular, and that the step sizes satisfy $\sum_i \alpha_i = \infty$ and $\sum_i \alpha_i^2 < \infty$. Then $\mathbf{w}_i \rightarrow \mathbf{w}_{LS}$ almost surely.

Note that the expected LS solution can also be written as the solution to the following *expected least squares* problem:

$$\mathbf{w}_{LS} = \min_{\mathbf{w}} (\Phi \mathbf{w} - Y)^T \Xi (\Phi \mathbf{w} - Y). \quad (12.5)$$

Observe that $\Phi \mathbf{w}_{LS} \in \mathbb{R}^{|X|}$ denotes a vector that contains the approximated function $g(x; \mathbf{w}_{LS})$ for every x . This is the best approximation of f in the span of the features $\phi(x)$, in terms of expected square error. Recalling that Y is a vector of ground truth f values, we view this approximation as a *projection* of Y onto the space spanned by $\Phi \mathbf{w}$, and we can write the projection operator explicitly as:

$$\Pi_{\xi} Y = \Phi \mathbf{w}_{LS} = \Phi (\Phi^T \Xi \Phi)^{-1} \Phi^T \Xi Y.$$

Geometrically, $\Pi_{\xi} Y$ is the vector that is closest to Y on the linear subspace $\Phi \mathbf{w}$, where the distance function is the ξ -weighted Euclidean norm, $\|z\|_{\xi} = \sqrt{\langle z, z \rangle_{\xi}}$, where $\langle z, z' \rangle_{\xi} = z^T \Xi z'$.

We conclude this discussion by noting that although we derived Eq. (12.5) as the expectation of the least square method, we could also take an alternative view: the least squares method in (12.4) and the SGD algorithm are two different *sampling* based approximations to the expected least squares solution in (12.5). We will take this view when we develop our RL algorithms later on.

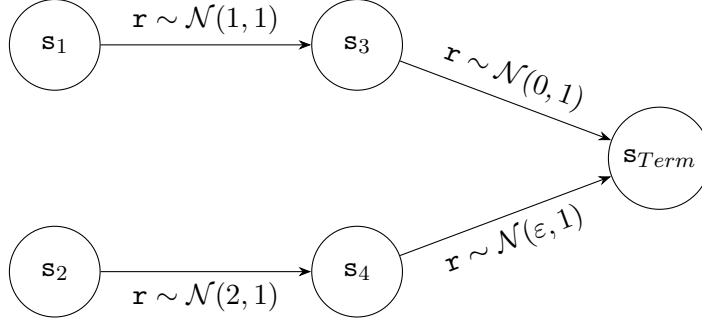


Figure 12.1: Example: MC vs. TD with function approximation.

12.3.3 Approximate Policy Evaluation: Regression

We now consider the simplest value function approximation method – regression, also known as Monte Carlo (MC) sampling. Recall that we are interested in learning the value function of a fixed policy π , $\hat{\mathcal{V}}^\pi$. Based on the least squares method above, all we need to figure out is how to build the sample, namely, how do we set the labels to replace $\mathcal{V}^\pi(\mathbf{s})$. The basic idea is to find an unbiased estimator U_t such that $\mathbb{E}[U_t | \mathbf{s}_t] = \mathcal{V}^\pi(\mathbf{s}_t)$. The Monte-Carlo (MC) estimate, which was introduced in Chapter 11.3, simply sums the observed discounted reward from a state $G_i(\mathbf{s}) = \sum_{\tau=0}^T \gamma^\tau r_\tau$, starting at the first visit of $\mathbf{s}$ in episode i . For large enough T and a bounded reward, the effect of discounted rewards after T is negligible, and we have $\mathbb{E}[G_i(\mathbf{s})] \approx \mathcal{V}^\pi(\mathbf{s})$, so we can set $U_t(\mathbf{s}) = G_i(\mathbf{s})$.

For calculating the approximation, we can apply the various least squares algorithms outlined above. In particular, for a linear approximation, and a large sample, we understand that the solution will approach the projection, $\Phi \mathbf{w} = \Pi_\xi \mathcal{V}^\pi$.

12.3.4 Approximate Policy Evaluation: Bootstrapping

While the MC estimate is intuitive, it turns out that there is a more efficient way of estimating labels for regression, based on the idea bootstrapping (cf. Chapter 11). We motivate this approach with an example.

Consider the episodic MDP in Figure 12.1. In this MDP there are two paths towards a terminal state. The first state in each path gives a large reward that determines most of the total return. The second state on each path gives a small reward that does not affect the total return by much, and therefore, the values of each of these second states are similar. We shall show that this value similarity may be exploited to reduce the variance in the estimates of the initial state values $\mathcal{V}^\pi(\mathbf{s}_1)$

and $\mathcal{V}^\pi(\mathbf{s}_2)$.

Formally, $\mathbf{s}_{Term}$ is a terminal state, and rewards are normally distributed with unit variance, as shown. There are no actions, and therefore for any π , $\mathcal{V}^\pi(\mathbf{s}_1) = 1$, $\mathcal{V}^\pi(\mathbf{s}_2) = 2 + \varepsilon$, $\mathcal{V}^\pi(\mathbf{s}_3) = 0$, and $\mathcal{V}^\pi(\mathbf{s}_4) = \varepsilon$. We will particularly be interested in estimating $\mathcal{V}^\pi(\mathbf{s}_1)$ and $\mathcal{V}^\pi(\mathbf{s}_2)$.

Consider the case of no function approximation. Assume that we have sampled N trajectories, where half start from $\mathbf{s}_1$, and the other half from $\mathbf{s}_2$. In this case, the MC estimates for $\hat{\mathcal{V}}^\pi(\mathbf{s}_1)$ and $\hat{\mathcal{V}}^\pi(\mathbf{s}_2)$ will each be based on $N/2$ samples, and their variance will therefore be $2/(N/2) = 4/N$.

Let us recall the bootstrapping approach. We have that $\mathcal{V}^\pi(\mathbf{s}_1) = \mathbb{E}[\mathbf{r}(\mathbf{s}_1)] + \mathcal{V}^\pi(\mathbf{s}_3)$, and similarly, $\mathcal{V}^\pi(\mathbf{s}_2) = \mathbb{E}[\mathbf{r}(\mathbf{s}_2)] + \mathcal{V}^\pi(\mathbf{s}_4)$. Therefore, we can use the samples to first estimate $\hat{\mathcal{V}}^\pi(\mathbf{s}_3)$ and $\hat{\mathcal{V}}^\pi(\mathbf{s}_4)$, and then plug in to estimate $\hat{\mathcal{V}}^\pi(\mathbf{s}_1)$ and $\hat{\mathcal{V}}^\pi(\mathbf{s}_2)$.

Now, for small ε , we understand that the values $\mathcal{V}^\pi(\mathbf{s}_3)$ and $\mathcal{V}^\pi(\mathbf{s}_4)$ should be similar. One way to take this into account is to use function approximation that approximates $\mathcal{V}^\pi(\mathbf{s}_3)$ and $\mathcal{V}^\pi(\mathbf{s}_4)$ as *the same* value, $\hat{\mathcal{V}}_{3/4}$. In this approximation we effectively use the full N samples to estimate $\hat{\mathcal{V}}_{3/4}$, resulting in variance $1/N$. We can now use bootstrapping to estimate $\hat{\mathcal{V}}^\pi(\mathbf{s}_1)$ and $\hat{\mathcal{V}}^\pi(\mathbf{s}_2)$, which will result in variance $1/N + 1/(N/2) = 3/N$, smaller than the MC estimate!

However, note that for $\varepsilon \neq 0$, the bootstrapping solution will also be biased: taking $N \rightarrow \infty$ we see that $\hat{\mathcal{V}}_{3/4}$ will converge to $\varepsilon/2$, and therefore $\hat{\mathcal{V}}^\pi(\mathbf{s}_1)$ and $\hat{\mathcal{V}}^\pi(\mathbf{s}_2)$ will converge to $1 + \varepsilon/2$ and $2 + \varepsilon/2$, respectively.

Thus, we see that bootstrapping, when combined with function approximation, allowed us to reduce variance by exploiting the similarity between values of different states, but at the cost of a possible bias in the expected solution. As it turns out, this phenomenon is not limited to the example above, but can be shown to hold more generally [74].

In the following, we shall develop a rigorous formulation of bootstrapping with function approximation, and use it to suggest several approximation algorithms. We will also bound the bias incurred by this approach.

12.3.5 The Projected Bellman Equation

Recall from Chapter 11 that without function approximation, TD methods effectively compute a solution to the Bellman equation. With function approximation, this is no longer the case, as clearly whatever solution TD methods converge to (if they converge at all), must depend on the function approximation. We shall start our investigation from a fundamental equation that takes function approximation into account – the projected Bellman equation (PBE). We will use the PBE to define a

particular approximation of the value function, and study its properties. We will later develop TD algorithms that estimate this approximation using sampling.

For technical simplicity, our analysis mainly considers linear function approximation. The algorithms we develop, however, can easily be extended to non-linear approximation, and we will expand on particular cases where relevant. Let $\Phi \in \mathbb{R}^{|\mathcal{S}| \times d}$ denote a matrix in which each row $\mathbf{s}$ is $\phi(\mathbf{s})$, where without loss of generality we assume that the states are ordered as $1, 2, \dots, |\mathcal{S}|$. Let $\hat{\mathcal{S}} = \{\Phi \mathbf{w} : \mathbf{w} \in \mathbb{R}^d\}$ denote the linear subspace spanned by Φ . Recall that $\mathcal{V}^\pi(s) \in \mathbb{R}^{|\mathcal{S}|}$ satisfies the Bellman equation: $\mathcal{V}^\pi = \mathcal{T}^\pi \mathcal{V}^\pi$. However, $\mathcal{V}^\pi$ does not necessarily belong to $\hat{\mathcal{S}}$, as we may not be able to accurately represent the true value function as a linear combination of our features.

To write a ‘Bellman-like’ equation that involves our function approximation, we proceed by projecting the Bellman operator $\mathcal{T}^\pi$ onto $\hat{\mathcal{S}}$, resulting in the PBE:

$$\Phi \mathbf{w} = \Pi_\xi \mathcal{T}^\pi \{\Phi \mathbf{w}\}, \quad (12.6)$$

where Π_ξ is the projection operator onto $\hat{\mathcal{S}}$ under some ξ -weighted Euclidean norm.

Let us try to intuitively interpret the PBE. We are looking for an approximate value function $\Phi \mathbf{w} \in \mathbb{R}^{|\mathcal{S}|}$, which by definition is within our linear approximation space, such that after we apply to it $\mathcal{T}^\pi$, and project the result (which does not necessarily belong to $\hat{\mathcal{S}}$ anymore) back to $\hat{\mathcal{S}}$, we obtain the same approximate value. Since the true value is a fixed point of $\mathcal{T}^\pi$, we have reason to believe that a fixed point of $\Pi_\xi \mathcal{T}^\pi$ may provide a reasonable approximation. In the following, we shall investigate this hypothesis, and build on Eq. (12.6) to develop various learning algorithms. We remark that the PBE is not the only way of defining an approximate value function, and other approaches have been proposed in the literature. However, the PBE is the basis for the most popular RL algorithms today.

Existence, Uniqueness and Error Bound on PBE Solution

We are interested in the following questions:

1. Does the PBE (12.6) have a solution?
2. When is $\Pi_\xi \mathcal{T}^\pi$ a contraction, and what is its fixed point?
3. If $\Pi_\xi \mathcal{T}^\pi$ has a fixed point $\Phi \mathbf{w}^*$, how far is it from the best approximation possible, namely, $\Pi_\xi \mathcal{V}^\pi$?

Answering the first two points will characterize the approximate solution we seek. The third point above relates to the bias of the bootstrapping approach, as described in the example in Section 12.3.4.

Let us assume the following:

Assumption 12.1. *The Markov chain corresponding to π has a single recurrent class and no transient states. We further let*

$$\xi_j = \lim_{N \rightarrow \infty} \frac{1}{N} \sum_{t=1}^N P(\mathbf{s}_t = j | \mathbf{s}_0 = \mathbf{s}) > 0,$$

which is the probability of being in state j when the process reaches its steady state, given any arbitrary $\mathbf{s}_0 = \mathbf{s}$.

We have the following result:

Proposition 12.4. *Under Assumption 12.1 we have that*

1. $\Pi_\xi \mathcal{T}^\pi$ is a contraction operator with modulus γ w.r.t. $\|\cdot\|_\xi$.
2. The unique fixed point $\Phi_{\mathbf{w}^*}$ of $\Pi_\xi \mathcal{T}^\pi$ satisfies,

$$\|\mathcal{V}^\pi - \Phi_{\mathbf{w}^*}\|_\xi \leq \frac{1}{1-\gamma} \|\mathcal{V}^\pi - \Pi_\xi \mathcal{V}^\pi\|_\xi, \quad (12.7)$$

and

$$\|\mathcal{V}^\pi - \Phi_{\mathbf{w}^*}\|_\xi^2 \leq \frac{1}{1-\gamma^2} \|\mathcal{V}^\pi - \Pi_\xi \mathcal{V}^\pi\|_\xi^2. \quad (12.8)$$

We remark that the bound in (12.8) is stronger than the bound in (12.7) (show this!). We nevertheless include the bound (12.7) for didactic purpose, as its proof is slightly different. Proposition 12.4 shows that for the particular projection defined by weighting the Euclidean norm according to the stationary distribution of the Markov chain, we can both guarantee a solution to the PBE, and bound its bias with respect to $\Pi_\xi \mathcal{V}^\pi$ – the best approximation of $\mathcal{V}^\pi$ under the ξ weighting. Fortunately, we shall see that this weighting is suitable for developing on-policy learning algorithms. However, the reader should note that for a different ξ , the conclusions of Proposition 12.4 do not necessarily hold.

Proof. We begin by showing the contraction property. We use two lemmas.

Lemma 12.5. *If P^π is the transition matrix induced by π , then*

$$\forall z \quad \|P^\pi z\|_\xi \leq \|z\|_\xi.$$

Proof. Let p_{ij} be the components of P^π . For all $z \in \mathbb{R}^{|\mathcal{S}|}$:

$$\|P^\pi z\|_\xi^2 = \sum_i \xi_i \left(\sum_j p_{ij} z_j \right)^2 \underset{\text{Jensen}}{\leq} \sum_i \xi_i \sum_j p_{ij} z_j^2 = \sum_j z_j^2 \sum_i \xi_i p_{ij} = \|z\|_\xi^2,$$

where the last equality uses a property of the stationary distribution ξ_i , $\sum_i \xi_i p_{ij} = \xi_j$, and $\sum_{j=1}^n \xi_j z_j^2 = \|z\|_\xi^2$. $\square$

Lemma 12.6. *The projection Π_ξ obeys the Pythagorean theorem:*

$$\forall J \in \mathbb{R}^{|\mathcal{S}|}, \hat{J} \in \hat{\mathcal{S}}: \quad \|J - \hat{J}\|_\xi^2 = \|J - \Pi_\xi J\|_\xi^2 + \|\Pi_\xi J - \hat{J}\|_\xi^2.$$

Proof. Observe that

$$\|J - \hat{J}\|_\xi^2 = \|J - \Pi_\xi J + \Pi_\xi J - \hat{J}\|_\xi^2 = \|J - \Pi_\xi J\|_\xi^2 + \|\Pi_\xi J - \hat{J}\|_\xi^2 + 2 \cdot \langle J - \Pi_\xi J, \Pi_\xi J - \hat{J} \rangle_\xi.$$

We claim that $J - \Pi_\xi J$ and $\Pi_\xi J - \hat{J}$ are orthogonal under $\langle \cdot, \cdot \rangle_\xi$ (this is known as the error orthogonality for weighted Euclidean-norm projections). To see this, recall that

$$\Pi_\xi = \Phi(\Phi^T \Xi \Phi)^{-1} \Phi^T \Xi,$$

so

$$\Xi \Pi_\xi = \Xi \Phi(\Phi^T \Xi \Phi)^{-1} \Phi^T \Xi = \Pi_\xi^T \Xi.$$

Now,

$$\begin{aligned} \langle J - \Pi_\xi J, \Pi_\xi J - \hat{J} \rangle_\xi &= (J - \Pi_\xi J)^T \Xi (\Pi_\xi J - \hat{J}) \\ &= J^T \Xi \Pi_\xi J - J^T \Xi \hat{J} - J^T \Pi_\xi^T \Xi \Pi_\xi J + J^T \Pi_\xi^T \Xi \hat{J} \\ &= J^T \Xi \Pi_\xi J - J^T \Xi \hat{J} - J^T \Xi \Pi_\xi \Pi_\xi J + J^T \Xi \Pi_\xi \hat{J} \\ &= J^T \Xi \Pi_\xi J - J^T \Xi \Pi_\xi J - J^T \Xi \hat{J} + J^T \Xi \hat{J} = 0, \end{aligned}$$

where in the penultimate equality we used that fact that $\Pi_\xi \hat{J} = \hat{J}$, as $\hat{J} \in \hat{\mathcal{S}}$, and that $\Pi_\xi \Pi_\xi = \Pi_\xi$, as projecting a vector that is already in $\hat{\mathcal{S}}$ effects no change to the vector. $\square$

We now claim that that Π_ξ is non-expansive.

Lemma 12.7. *We have that $\forall J_1, J_2 \in \mathbb{R}^{|\mathcal{S}|}$, $\|\Pi_\xi J_1 - \Pi_\xi J_2\|_\xi \leq \|J_1 - J_2\|_\xi$.*

Proof. We have

$$\|\Pi_\xi J_1 - \Pi_\xi J_2\|_\xi^2 = \|\Pi_\xi(J_1 - J_2)\|_\xi^2 \leq \|\Pi_\xi(J_1 - J_2)\|_\xi^2 + \|(I - \Pi_\xi)(J_1 - J_2)\|_\xi^2 = \|J_1 - J_2\|_\xi^2,$$

where the first inequality is by linearity of Π_ξ , and the last equality is by the Pythagorean theorem of Lemma 12.6, where we set $J = J_1 - J_2$ and $\hat{J} = 0$. $\square$

In order to prove the contraction of $\Pi_\xi \mathcal{T}^\pi$, $\forall J_1, J_2 \in \mathbb{R}^{|S|}$:

$$\begin{aligned} \|\Pi_\xi \mathcal{T}^\pi J_1 - \Pi_\xi \mathcal{T}^\pi J_2\|_\xi &\stackrel{\Pi_\xi \text{ non-expansive}}{\leq} \|\mathcal{T}^\pi J_1 - \mathcal{T}^\pi J_2\|_\xi \\ &\stackrel{\text{definition of } \mathcal{T}^\pi}{=} \gamma \|P^\pi(J_1 - J_2)\|_\xi \stackrel{\text{Lemma 12.5}}{\leq} \gamma \|J_1 - J_2\|_\xi, \end{aligned}$$

and therefore $\Pi_\xi \mathcal{T}^\pi$ is a contraction operator.

We now prove the error bound in (12.7).

$$\begin{aligned} \|\mathcal{V}^\pi - \Phi_{\mathbf{w}^*}\|_\xi &\leq \|\mathcal{V}^\pi - \Pi_\xi \mathcal{V}^\pi\|_\xi + \|\Pi_\xi \mathcal{V}^\pi - \Phi_{\mathbf{w}^*}\|_\xi \\ &= \|\mathcal{V}^\pi - \Pi_\xi \mathcal{V}^\pi\|_\xi + \|\Pi_\xi \mathcal{T}^\pi \mathcal{V}^\pi - \Pi_\xi \mathcal{T}^\pi \Phi_{\mathbf{w}^*}\|_\xi \\ &\leq \|\mathcal{V}^\pi - \Pi_\xi \mathcal{V}^\pi\|_\xi + \gamma \|\mathcal{V}^\pi - \Phi_{\mathbf{w}^*}\|_\xi, \end{aligned}$$

where the first inequality is by the triangle inequality, the second equality is since $\mathcal{V}^\pi$ is $\mathcal{T}^\pi$'s fixed point, and $\Phi_{\mathbf{w}^*}$ is $\Pi_\xi \mathcal{T}^\pi$'s fixed point, and the second inequality is by the contraction of $\Pi_\xi \mathcal{T}^\pi$. Rearranging gives (12.7).

We proceed to prove the error bound (12.8).

$$\begin{aligned} \|\mathcal{V}^\pi - \Phi_{\mathbf{w}^*}\|_\xi^2 &= \|\mathcal{V}^\pi - \Pi_\xi \mathcal{V}^\pi\|_\xi^2 + \|\Pi_\xi \mathcal{V}^\pi - \Phi_{\mathbf{w}^*}\|_\xi^2 \\ &= \|\mathcal{V}^\pi - \Pi_\xi \mathcal{V}^\pi\|_\xi^2 + \|\Pi_\xi \mathcal{T}^\pi \mathcal{V}^\pi - \Pi_\xi \mathcal{T}^\pi \Phi_{\mathbf{w}^*}\|_\xi^2 \\ &\leq \|\mathcal{V}^\pi - \Pi_\xi \mathcal{V}^\pi\|_\xi^2 + \gamma^2 \|\mathcal{V}^\pi - \Phi_{\mathbf{w}^*}\|_\xi^2, \end{aligned} \tag{12.9}$$

where the first equality is by the Pythagorean theorem, and the remainder follows similarly to the proof of (12.7) above. $\square$

We now move to solving the projected Bellman equation. In Section 12.3.2 we presented a closed-form solution for linear least squares and an algorithm based on SGD. We use these approaches as inspiration for sampling-based solutions to the PBE. As we shall see, this will yield temporal difference algorithms with function approximation.

Using the explicit formulation of the projection Π_ξ , we see that the PBE solution is some $\hat{\mathcal{V}} = \Phi \mathbf{w}^*$ where $\mathbf{w}^*$ solves

$$\mathbf{w}^* = \operatorname{argmin}_{\mathbf{w} \in \mathbb{R}^d} \|\Phi \mathbf{w} - (R^\pi + \gamma P^\pi \Phi \mathbf{w}^*)\|_\xi^2.$$

Setting the gradient to 0, we get

$$\Phi^T \Xi (\Phi \mathbf{w}^* - (R^\pi + \gamma P^\pi \Phi \mathbf{w}^*)) = 0.$$

Equivalently we can write

$$A \mathbf{w}^* = b,$$

where

$$A = \Phi^T \Xi (I - \gamma P^\pi) \Phi, \quad b = \Phi^T \Xi R^\pi. \quad (12.10)$$

12.3.6 Least Squares Temporal Differences (LSTD)

Our first temporal difference algorithm is based on matrix inversion. We have that

$$\mathbf{w}^* = A^{-1}b.$$

In order to evaluate A, b , we use simulation.

Proposition 12.8. *We have that*

$$\mathbb{E}_{\mathbf{s} \sim \xi} [\phi(\mathbf{s}) r(\mathbf{s}, \pi(\mathbf{s}))] = b,$$

and

$$\mathbb{E}_{\mathbf{s} \sim \xi, \mathbf{s}' \sim P^\pi(\cdot | \mathbf{s})} [\phi(\mathbf{s})(\phi^T(\mathbf{s}) - \gamma \phi^T(\mathbf{s}'))] = A.$$

Proof. We have

$$\mathbb{E}_{\mathbf{s} \sim \xi} [\phi(\mathbf{s}) r(\mathbf{s}, \pi(\mathbf{s}))] = \sum_{\mathbf{s}} \phi(\mathbf{s}) \xi(\mathbf{s}) r(\mathbf{s}, \pi(\mathbf{s})) = \Phi^T \Xi R^\pi = b.$$

Also,

$$\begin{aligned} & \mathbb{E}_{\mathbf{s} \sim \xi, \mathbf{s}' \sim P^\pi(\cdot | \mathbf{s})} [\phi(\mathbf{s})(\phi^T(\mathbf{s}) - \gamma \phi^T(\mathbf{s}'))] \\ &= \sum_{\mathbf{s}, \mathbf{s}'} \xi(\mathbf{s}) P^\pi(\mathbf{s}' | \mathbf{s}) \phi(\mathbf{s})(\phi^T(\mathbf{s}) - \gamma \phi^T(\mathbf{s}')) \\ &= \sum_{\mathbf{s}} \phi(\mathbf{s}) \xi(\mathbf{s}) \phi^T(\mathbf{s}) - \gamma \sum_{\mathbf{s}} \phi(\mathbf{s}) \xi(\mathbf{s}) \sum_{\mathbf{s}'} P^\pi(\mathbf{s}' | \mathbf{s}) \phi^T(\mathbf{s}') \\ &= \Phi^T \Xi \Phi - \gamma \Phi^T \Xi P^\pi \Phi = A. \end{aligned}$$

□

We now propose the following estimates to A and b .

Algorithm 25 Least Squares Temporal Difference (LSTD)

- 1: **Input:** Policy π , discount factor γ , number of steps N
- 2: Initialize $\mathbf{s}_0$ arbitrarily
- 3: **For** $\mathbf{t} = 1$ to N
- 4: Simulate action $\mathbf{a}_{\mathbf{t}} \sim \pi(\cdot \mid \mathbf{s}_{\mathbf{t}})$
- 5: Observe new state $\mathbf{s}_{\mathbf{t}+1}$
- 6: Compute $\hat{b}_N$:

$$\hat{b}_N = \frac{1}{N} \sum_{\mathbf{t}=1}^N \phi(\mathbf{s}_{\mathbf{t}}) \mathbf{r}(\mathbf{s}_{\mathbf{t}}, \pi(\mathbf{s}_{\mathbf{t}}))$$

- 7: Compute $\hat{A}_N$:

$$\hat{A}_N = \frac{1}{N} \sum_{\mathbf{t}=1}^N \phi(\mathbf{s}_{\mathbf{t}}) (\phi^T(\mathbf{s}_{\mathbf{t}}) - \gamma \phi^T(\mathbf{s}_{\mathbf{t}+1}))$$

- 8: **Return** $\mathbf{w}_N = \hat{A}_N^{-1} \hat{b}_N$
-

From the ergodicity property of Markov chains (Theorem 4.9), we have the following result.

Proposition 12.9. *We have that*

$$\lim_{N \rightarrow \infty} \hat{b}_N = b, \quad \lim_{N \rightarrow \infty} \hat{A}_N = A$$

with probability 1.

Remark 12.2. *Note that with tabular features (cf. remark 12.1), the LSTD algorithm is equivalent to computing the value function in an empirical model built from the samples in the data. Recall that Theorem 11.9 established that this solution is also the limit of $TD(0)$ when run on samples from the data.*

12.3.7 Projected Value Iteration

Our second temporal difference algorithm is based on the contraction of the projected Bellman operator. Consider the iterative solution,

$$\Phi \mathbf{w}_{n+1} = \Pi_{\xi} \mathcal{T}^{\pi} \Phi \mathbf{w}_n = \Pi_{\xi} (R^{\pi} + \gamma P^{\pi} \Phi \mathbf{w}_n),$$

which converges to $\mathbf{w}^*$ since $\Pi_\xi \mathcal{T}^\pi$ is a contraction operator.

Recalling that Π_ξ relates to a least squares regression problem, the solution above describes a sequence of least squares regression problems. For the $(n+1)$ 'th regression problem, our independent variable is the state, $\mathbf{s}$, and the dependent variable is $\mathbf{r}(\mathbf{s}) + \gamma \sum_{\mathbf{s}'} P^\pi(\mathbf{s}'|\mathbf{s}) \phi(\mathbf{s}')^T \mathbf{w}_n$.

If we sample trajectories from π , after some mixing time $\mathbf{t}$, a pair of consecutive state $\mathbf{s}_t, \mathbf{s}_{t+1}$ are sampled from $\xi(\mathbf{s})$ and $P^\pi(\mathbf{s}'|\mathbf{s})\xi(\mathbf{s})$, respectively. Therefore, we can define the samples for the least squares regression problem as $\{(\mathbf{s}_t, \mathbf{r}(\mathbf{s}_t) + \gamma \phi(\mathbf{s}_{t+1})^T \mathbf{w}_n), \dots, (\mathbf{s}_{t+N}, \mathbf{r}(\mathbf{s}_{t+N}) + \gamma \phi(\mathbf{s}_{t+N+1})^T \mathbf{w}_n)\}$.

Algorithm 26 Projected Value Iteration

- 1: **Input:** Policy π , discount factor γ , number of samples N , number of iterations K
- 2: Initialize $\mathbf{s}_0, \mathbf{w}_0$ arbitrarily
- 3: **For** $\mathbf{t} = 1$ to N
- 4: Simulate action $\mathbf{a}_t \sim \pi(\cdot | \mathbf{s}_t)$
- 5: Observe new state $\mathbf{s}_{t+1}$
- 6: **For** $n = 0, 1, \dots, K$
- 7: Compute labels for each sample $x_t = \mathbf{s}_t$:

$$y_t = \mathbf{r}(\mathbf{s}_t) + \gamma \phi(\mathbf{s}_{t+1})^T \mathbf{w}_n$$

- 8: Solve least-squares regression for $\{(x_t, y_t)\}$:

$$\mathbf{w}_{n+1} = \arg \min_{\mathbf{w}} \frac{1}{N} \sum_{\mathbf{t}} (\phi(\mathbf{s}_t)^T \mathbf{w} - y_t)^2$$

Remark 12.3. *Projected value iteration can be used with a more general regression algorithm. Let Π_{gen} denote a general regression algorithm, such as a non-linear least squares fit, a neural network, or even a non-parametric regression such as K -nearest neighbors. We can consider the iterative algorithm:*

$$\hat{\mathbf{V}}(\mathbf{w}_{n+1}) = \Pi_{gen} \mathcal{T}^\pi \hat{\mathbf{V}}(\mathbf{w}_n).$$

To realize this algorithm, we use the same samples as above, and only replace the regression algorithm. Note that convergence in this case is not guaranteed, as in general, $\Pi_{gen} \mathcal{T}^\pi$ is not necessarily a contraction in any norm.

12.3.8 TD(0) with Function Approximation

Our third algorithm is based on stochastic approximation. Consider the mean squared error loss function

$$Loss(\mathbf{w}) = \sum_{t=1}^N \left(\hat{\mathcal{V}}(\mathbf{s}_t, \mathbf{w}) - \mathcal{V}^\pi(\mathbf{s}_t) \right)^2,$$

and replace the unknown $\mathcal{V}^\pi(\mathbf{s})$ with a TD-style target $\mathbf{r}(\mathbf{s}, \pi(\mathbf{s})) + \gamma \hat{\mathcal{V}}(\mathbf{s}', \mathbf{w})$,

$$Loss_{TD}(\mathbf{w}) = \sum_{t=1}^N \left(\hat{\mathcal{V}}(\mathbf{s}_t, \mathbf{w}) - \mathbf{r}(\mathbf{s}_t, \pi(\mathbf{s}_t)) - \gamma \hat{\mathcal{V}}(\mathbf{s}_{t+1}, \mathbf{w}) \right)^2.$$

Taking an SGD update on this loss while treating the targets as ‘fixed’ (i.e., we do not differentiate them) yields the TD(0) algorithm (cf. Section 11.5):

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha_t \left(\mathbf{r}(\mathbf{s}_t, \pi(\mathbf{s}_t)) + \gamma \hat{\mathcal{V}}(\mathbf{s}_{t+1}, \mathbf{w}_t) - \hat{\mathcal{V}}(\mathbf{s}_t, \mathbf{w}_t) \right) \nabla_{\mathbf{w}} \hat{\mathcal{V}}(\mathbf{s}_t, \mathbf{w}_t).$$

Notice that the temporal difference term is the approximation (w.r.t. the weights at time t) of $\mathbf{r}(\mathbf{s}_t, \pi(\mathbf{s}_t)) + \gamma \mathcal{V}^\pi(\mathbf{s}_{t+1}) - \mathcal{V}^\pi(\mathbf{s}_t)$.

In the case of linear function approximation, we obtain the following variant, which we shall analyze in some detail.

Algorithm 27 TD(0) with Linear Function Approximation

- 1: **Initialize:** Set $\mathbf{w}_0 = 0$.
- 2: **For** $t = 0, 1, 2, \dots$
- 3: Observe: $(\mathbf{s}_t, \mathbf{a}_t \sim \pi(\mathbf{s}_t), \mathbf{s}_{t+1})$.
- 4: Update:

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha_t \underbrace{\left(\mathbf{r}(\mathbf{s}_t, \pi(\mathbf{s}_t)) + \gamma \phi(\mathbf{s}_{t+1})^\top \mathbf{w}_t - \phi(\mathbf{s}_t)^\top \mathbf{w}_t \right)}_{\text{temporal difference}} \phi(\mathbf{s}_t). \quad (12.11)$$

Remark 12.4. *Note that with tabular features (cf. Remark 12.1), Algorithm 27 is exactly the same as TD(0) without function approximation, Algorithm 18.*

We next relate the TD(0) algorithm 27 to the PBE. A key observation is that the algorithm can be written as a stochastic approximation:

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha_t (b - A\mathbf{w}_t + \omega_t),$$

where A and b are defined in (12.10), and ω_t is a noise term. The corresponding ODE is $\dot{\mathbf{w}} = b - A\mathbf{w}$, with a unique stable fixed point at $\mathbf{w}^* = A^{-1}b$.

Remark 12.5. *In the tabular setting, we proved the convergence of $TD(0)$ using the contraction method for stochastic approximation. Here, we cannot use this approach, as the contraction in $TD(0)$, which follows from the Bellman equation, applies to the values of each state. However, with function approximation, we iterate over the weights $\mathbf{w}_t$ and not over the values for each state, and for these weights the contraction does not necessarily hold. For this reason, we shall seek a convergence proof based on the ODE method.*

We next prove convergence of $TD(0)$. For simplicity, we will consider a somewhat synthetic version of $TD(0)$ where at each iteration t , the state $\mathbf{s}_t$ is drawn i.i.d. from the stationary distribution $\xi(\mathbf{s})$, and the next state $\mathbf{s}_{t+1}$ in the update rule is drawn from $P^\pi(\mathbf{s}'|\mathbf{s} = \mathbf{s}_t)$, respectively. This will allow us to claim that the noise term satisfies $\mathbb{E}[\omega_t|h_{t-1}] = 0$.

Theorem 12.10. *Consider the following iterative algorithm:*

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha_t(\mathbf{r}(\mathbf{s}_t, \pi(\mathbf{s}_t)) + \gamma\phi(\mathbf{s}'_t)^\top \mathbf{w}_t - \phi(\mathbf{s}_t)^\top \mathbf{w}_t)\phi(\mathbf{s}_t),$$

where $\mathbf{s}_t \sim \xi(\mathbf{s})$ i.i.d., and $\mathbf{s}'_t \sim P^\pi(\mathbf{s}'|\mathbf{s} = \mathbf{s}_t)$ independently of the history up to time t . Assume that Φ is full rank. Let the step sizes satisfy $\sum_t \alpha_t = \infty$, and $\sum_t \alpha_t^2 = O(1)$. Then $\mathbf{w}_t$ converges with probability 1 to $\mathbf{w}^* = A^{-1}b$.

Proof. We write Eq. (12.10) as

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha_t(b - A\mathbf{w}_t + \omega_t),$$

where the noise $\omega_t = \mathbf{r}(\mathbf{s}_t, \pi(\mathbf{s}_t))\phi(\mathbf{s}_t) - b + (\gamma\phi(\mathbf{s}'_t)^\top - \phi(\mathbf{s}_t)^\top)\mathbf{w}_t\phi(\mathbf{s}_t) + A\mathbf{w}_t$ satisfies:

$$\mathbb{E}[\omega_t|h_{t-1}] = \mathbb{E}[\omega_t|\mathbf{w}_t] = 0,$$

where the first equality is since the states are drawn i.i.d., and the second is from Proposition 12.8. We would like to use Theorem 11.7 to show convergence. From Proposition 12.4 we already know that $\mathbf{w}^*$ corresponds to the unique fixed point of the linear dynamical system $f(\mathbf{w}) = -A\mathbf{w} + b$. We proceed to show that $\mathbf{w}^*$ is globally asymptotically stable, by showing that the eigenvalues of A have a positive real part. Let $z \in \mathbb{R}^{|\mathcal{S}|}$. We have that

$$\begin{aligned} z^T \Xi P^\pi z &= z^T \Xi^{1/2} \Xi^{1/2} P^\pi z \\ &\leq \|\Xi^{1/2} z\| \|\Xi^{1/2} P^\pi z\| \\ &= \|z\|_\xi \|P^\pi z\|_\xi \\ &\leq \|z\|_\xi \|z\|_\xi = z^T \Xi z. \end{aligned}$$

where the first inequality is by Cauchy-Schwarz, and the second is by Lemma 12.5.

We claim that the matrix $\Xi(I - \gamma P^\pi)$ is positive definite. To see this, observe that for any $z \in \mathbb{R}^{|\mathcal{S}|} \neq 0$ we have

$$z^T \Xi(I - \gamma P^\pi)z = z^T \Xi z - \gamma z^T \Xi P^\pi z \geq z^T \Xi z - \gamma z^T \Xi z = (1 - \gamma) \|z\|_\xi^2 > 0. \quad (12.12)$$

We now claim that $A = \Phi^T \Xi(I - \gamma P^\pi) \Phi$ is positive definite. Assume by negation that for some $\theta \in \mathbb{R}^d$, $\theta \neq 0$ we have $\theta^T \Phi^T \Xi(I - \gamma P^\pi) \Phi \theta \leq 0$. If Φ is full rank, then $z = \Phi \theta \in \mathbb{R}^{|\mathcal{S}|} \neq 0$, contradicting Eq. (12.12). The claim that the eigen-values of A have a positive real part is not immediate from the positive definiteness established above since A is not necessarily symmetric. To show this, let $\lambda \in \mathbb{C} = \alpha + \beta i$ be an eigenvalue of A , and let $v \in \mathbb{C}^d = x + iy$, where $x, y \in \mathbb{R}^d$, be its associated right eigenvector. We have that $(A - \lambda)v = 0$, therefore

$$((A - \alpha) - \beta i)(x + iy) = 0,$$

therefore $(A - \alpha)x + \beta y = 0$ and $(A - \alpha)y - \beta x = 0$. Multiplying these two equations by x^T and y^T , respectively, and summing we obtain

$$x^T(A - \alpha)x + y^T(A - \alpha)y = -x^T \beta y + y^T \beta x = \beta(y^T x - x^T y) = 0.$$

Therefore,

$$\alpha = \frac{x^T A x + y^T A y}{x^T x + y^T y} > 0.$$

□

Remark 12.6. *A similar convergence result holds for the standard TD(0) of Eq. 12.11, using a more sophisticated proof technique that accounts for noise that is correlated (depends on the state). The main idea is to show that since the Markov chain mixes quickly, the average noise is still close to zero with high probability [161].*

12.3.9 Episodic MDPs

We can extend the learning algorithms above to the episodic MDP setting, by removing the discount factor, and explicitly setting the value of a goal state to 0, similarly to Section 11.5.3. For example, the TD(0) algorithm would be modified to,

$$\mathbf{w}_{t+1} = \begin{cases} \mathbf{w}_t + \alpha_t (\mathbf{r}(\mathbf{s}_t, \pi(\mathbf{s}_t)) + \phi(\mathbf{s}_{t+1})^\top \mathbf{w}_t - \phi(\mathbf{s}_t)^\top \mathbf{w}_t) \phi(\mathbf{s}_t), & \mathbf{s}_{t+1} \notin \mathcal{S}_G \\ \mathbf{w}_t + \alpha_t (\mathbf{r}(\mathbf{s}_t, \pi(\mathbf{s}_t)) - \phi(\mathbf{s}_t)^\top \mathbf{w}_t) \phi(\mathbf{s}_t), & \mathbf{s}_{t+1} \in \mathcal{S}_G \end{cases}.$$

Setting the value of goal states to 0 is critical with function approximation (and is a common ‘bug’ in episodic MDP implementations), as with function approximation, updates to the non-goal states will impact the approximation of goal state values, and nothing in the algorithm will push to correct these errors.

12.4 Approximate Policy Iteration

So far we have developed various algorithms for approximating the value of a fixed policy π . Our main interest, however, is finding a *good* policy. Similarly to RL without function approximation, we will consider two different approaches, based on either policy iteration or value iteration. In this section we consider approximate policy iteration.

A general algorithm: as depicted in Algorithm 28, we iterate between projection of $\mathcal{V}^{\pi_k}$ onto $\hat{\mathcal{S}}$ and greedy policy improvement updates w.r.t. the projected $\mathcal{V}^{\pi_k}$.

Algorithm 28 General Approximate Policy Iteration Algorithm

- 1: **Initialize:** Choose an initial policy π_0
 - 2: **For** $k = 0, 1, 2, \dots$
 - 3: **Evaluate:** Compute approximate value function $\hat{\mathcal{V}}_k = \Phi_{\mathbf{w}_k} \approx \mathcal{V}^{\pi_k}$
 - 4: **Improve:** Set $\pi_{k+1} \approx \text{greedy}(\hat{\mathcal{V}}_k)$
-

A key question in approximate policy iteration, is how errors in the value-function approximation, and possibly also errors in the greedy policy update, affect the error in the final policy. The next result shows that if we can guarantee that the approximation errors are bounded at each step of the algorithm, then the error in the final policy will also be bounded. This result suggests that approximate policy iteration is a fundamentally sound idea.

Theorem 12.11. *If for each iteration k the policies are approximated well over $\mathcal{S}$:*

$$\|\hat{\mathcal{V}}_k(\mathbf{s}) - \mathcal{V}^{\pi_k}(\mathbf{s})\|_{\infty} \leq \delta, \quad (12.13)$$

and policy improvement approximates well

$$\|\mathcal{T}^{\pi_{k+1}} \hat{\mathcal{V}}_k - \mathcal{T}^* \hat{\mathcal{V}}_k\|_{\infty} < \varepsilon, \quad (12.14)$$

Then

$$\limsup_k \|\mathcal{V}^{\pi_k}(\mathbf{s}) - \mathcal{V}^*(\mathbf{s})\|_{\infty} \leq \frac{\varepsilon + 2\gamma\delta}{(1 - \gamma)^2}.$$

To understand the policy improvement approximation term (12.14), recall that the Bellman operator $\mathcal{T}^* \hat{\mathcal{V}}_k$ corresponds to a greedy policy improvement step, $\pi_{\text{greedy}}(\hat{\mathcal{V}}_k)(\mathbf{s}) = \arg \max_{\mathbf{a}} \mathbf{r}(\mathbf{s}, \mathbf{a}) + \gamma \sum_{\mathbf{s}'} \mathbf{p}(\mathbf{s}'|\mathbf{s}, \mathbf{a}) \hat{\mathcal{V}}_k(\mathbf{s}')$. Suppose we cannot perform the arg max accurately for every state when computing the next policy π_{k+1} ,

that is, $\pi_{k+1} \neq \pi_{\text{greedy}}(\widehat{\mathcal{V}}_k)$. The condition (12.14) ensures that π_{k+1} is not too far off, in that $\mathbf{r}(\mathbf{s}, \pi_{k+1}(\mathbf{s})) + \gamma \sum_{\mathbf{s}'} \mathbf{p}(\mathbf{s}'|\mathbf{s}, \pi_{k+1}(\mathbf{s})) \widehat{\mathcal{V}}_k(\mathbf{s}')$ is not more than ε away from $\pi_{\text{greedy}}(\widehat{\mathcal{V}}_k)(\mathbf{s})$.

Proof. Let e denote a vector of ones. From the contraction property of $\mathcal{T}^*$ and $\mathcal{T}^{\pi_{k+1}}$ we have that

$$\begin{aligned} \|\mathcal{T}^{\pi_{k+1}} \widehat{\mathcal{V}}_k - \mathcal{T}^{\pi_{k+1}} \mathcal{V}^{\pi_k}\|_{\infty} &\leq \gamma\delta, \\ \|\mathcal{T}^* \widehat{\mathcal{V}}_k - \mathcal{T}^* \mathcal{V}^{\pi_k}\|_{\infty} &\leq \gamma\delta. \end{aligned}$$

Therefore,

$$\mathcal{T}^{\pi_{k+1}} \mathcal{V}^{\pi_k} \geq \mathcal{T}^{\pi_{k+1}} \widehat{\mathcal{V}}_k - \gamma\delta e \geq \mathcal{T}^* \widehat{\mathcal{V}}_k - (\varepsilon + \gamma\delta)e \geq \mathcal{T}^* \mathcal{V}^{\pi_k} - (\varepsilon + 2\gamma\delta)e \quad (12.15)$$

where the inequalities are element-wise, the first inequality is by (12.15), the second inequality is by (12.14), and the last inequality is again by (12.15). Now, by definition of $\mathcal{T}^*$,

$$\mathcal{T}^* \mathcal{V}^{\pi_k} - (\varepsilon + 2\gamma\delta)e \geq \mathcal{T}^{\pi_k} \mathcal{V}^{\pi_k} - (\varepsilon + 2\gamma\delta)e = \mathcal{V}^{\pi_k} - (\varepsilon + 2\gamma\delta)e,$$

so we have

$$\mathcal{T}^{\pi_{k+1}} \mathcal{V}^{\pi_k} \geq \mathcal{V}^{\pi_k} - (\varepsilon + 2\gamma\delta)e. \quad (12.16)$$

Since $\mathcal{T}^{\pi_{k+1}}$ is a contraction and linear, we have that for any $\mathcal{V}$ and a scalar $\alpha > 0$,

$$\|\mathcal{T}^{\pi_{k+1}}(\mathcal{V} - \alpha e) - \mathcal{T}^{\pi_{k+1}} \mathcal{V}\|_{\infty} \leq \gamma\alpha \|e\|_{\infty} = \gamma\alpha.$$

Therefore,

$$\mathcal{T}^{\pi_{k+1}}(\mathcal{V} - \alpha e) \geq \mathcal{T}^{\pi_{k+1}} \mathcal{V} - \gamma\alpha e, \quad (12.17)$$

and

$$\mathcal{T}^{\pi_{k+1}}(\mathcal{V} - \alpha e) \leq \mathcal{T}^{\pi_{k+1}} \mathcal{V} + \gamma\alpha e. \quad (12.18)$$

Recalling that $\mathcal{T}^{\pi_{k+1}}$ is a monotone operator (see proof of Lemma 6.11), applying $\mathcal{T}^{\pi_{k+1}}$ to both sides of (12.16) and using (12.17) we obtain:

$$(\mathcal{T}^{\pi_{k+1}})^2 \mathcal{V}^{\pi_k} \geq \mathcal{T}^{\pi_{k+1}} \mathcal{V}^{\pi_k} - \gamma(\varepsilon + 2\gamma\delta)e.$$

Similarly, applying $(\mathcal{T}^{\pi_{k+1}})^m$ to both sides of (12.16) gives

$$(\mathcal{T}^{\pi_{k+1}})^{m+1} \mathcal{V}^{\pi_k} \geq (\mathcal{T}^{\pi_{k+1}})^m \mathcal{V}^{\pi_k} - \gamma^m(\varepsilon + 2\gamma\delta)e. \quad (12.19)$$

Now, consider the telescoping sum

$$(\mathcal{T}^{\pi_{k+1}})^n \mathcal{V}^{\pi_k} - \mathcal{V}^{\pi_k} = \sum_{m=0}^{n-1} (\mathcal{T}^{\pi_{k+1}})^{m+1} \mathcal{V}^{\pi_k} - (\mathcal{T}^{\pi_{k+1}})^m \mathcal{V}^{\pi_k} \geq \left(\sum_{m=0}^{n-1} \gamma^m \right) (\varepsilon + 2\gamma\delta)e,$$

where in the last equality we plugged (12.19). Taking $n \rightarrow \infty$ we obtain:

$$\mathcal{V}^{\pi_{k+1}} - \mathcal{V}^{\pi_k} \geq \frac{\varepsilon + 2\gamma\delta}{1 - \gamma} e. \quad (12.20)$$

Now,

$$\begin{aligned} -\mathcal{V}^{\pi_{k+1}} &= -\mathcal{T}^{\pi_{k+1}} \mathcal{V}^{\pi_{k+1}} = -\mathcal{T}^{\pi_{k+1}} \mathcal{V}^{\pi_k} - (\mathcal{T}^{\pi_{k+1}} \mathcal{V}^{\pi_{k+1}} - \mathcal{T}^{\pi_{k+1}} \mathcal{V}^{\pi_k}) \\ &\leq -\mathcal{T}^{\pi_{k+1}} \left(\mathcal{V}^{\pi_k} + \frac{\varepsilon + 2\gamma\delta}{1 - \gamma} e \right) \\ &= \mathcal{T}^{\pi_{k+1}} \left(-\mathcal{V}^{\pi_k} - \frac{\varepsilon + 2\gamma\delta}{1 - \gamma} e \right) \\ &\leq -\mathcal{T}^{\pi_{k+1}} \mathcal{V}^{\pi_k} + \gamma \frac{\varepsilon + 2\gamma\delta}{1 - \gamma} e, \end{aligned}$$

where the first inequality is by (12.20) and the second is by (12.18).

We next have,

$$\mathcal{V}^* - \mathcal{V}^{\pi_{k+1}} \leq \mathcal{V}^* - \mathcal{T}^{\pi_{k+1}} \mathcal{V}^{\pi_k} + \gamma \frac{\varepsilon + 2\gamma\delta}{1 - \gamma} e. \quad (12.21)$$

From the contraction of $\mathcal{T}^*$ we have:

$$\mathcal{V}^* - \mathcal{T}^* \mathcal{V}^{\pi_k} = \mathcal{T}^* \mathcal{V}^* - \mathcal{T}^* \mathcal{V}^{\pi_k} \leq \gamma \|\mathcal{V}^* - \mathcal{V}^{\pi_k}\|_\infty e. \quad (12.22)$$

From (12.15), we have

$$\begin{aligned} \mathcal{V}^* - \mathcal{T}^{\pi_{k+1}} \mathcal{V}^{\pi_k} &\leq \mathcal{V}^* - \mathcal{T}^* \mathcal{V}^{\pi_k} + (\varepsilon + 2\gamma\delta)e \\ &\leq (\gamma \|\mathcal{V}^* - \mathcal{V}^{\pi_k}\|_\infty + \varepsilon + 2\gamma\delta) e \end{aligned}$$

Combining this with (12.21) we obtain

$$\begin{aligned} \mathcal{V}^* - \mathcal{V}^{\pi_{k+1}} &\leq \left(\gamma \frac{\varepsilon + 2\gamma\delta}{1 - \gamma} + \gamma \|\mathcal{V}^* - \mathcal{V}^{\pi_k}\|_\infty + \varepsilon + 2\gamma\delta \right) e \\ &= \left(\gamma \|\mathcal{V}^* - \mathcal{V}^{\pi_k}\|_\infty + \frac{\varepsilon + 2\gamma\delta}{1 - \gamma} \right) e. \end{aligned}$$

Since by definition $\mathcal{V}^* \geq \mathcal{V}^{\pi_{k+1}}$, we must have

$$\|\mathcal{V}^* - \mathcal{V}^{\pi_{k+1}}\|_\infty \leq \gamma \|\mathcal{V}^* - \mathcal{V}^{\pi_k}\|_\infty + \frac{\varepsilon + 2\gamma\delta}{1 - \gamma}.$$

The above holds for every k . Taking the limsup of both sides as $k \rightarrow \infty$ (note that the policies and values do not necessarily converge, but limsup always exists) we obtain,

$$(1 - \gamma) \limsup_k \|\mathcal{V}^* - \mathcal{V}^{\pi_k}\|_\infty \leq \frac{\varepsilon + 2\gamma\delta}{1 - \gamma}.$$

□

We next discuss several algorithms that implement approximate policy iteration. That is, methods for approximating the policy evaluation and the greedy policy improvement. We begin with methods that require knowledge of the state transition probabilities P , or a simulator thereof, which we collectively denote by a model M . We shall later consider methods that only require state transition samples.

12.4.1 Classification Based Policy Iteration

We have already discussed approximate policy evaluation in Section 12.3, so let us begin by assuming we have obtained an approximate value function $\hat{\mathcal{V}}^\pi(\mathbf{s}, \mathbf{w})$ for our current policy π . The main question is how obtain an (approximately) greedy policy $\pi_{\text{greedy}}(\hat{\mathcal{V}}^\pi)$ when the state space is too large to iterate over. We would like to treat this as another supervised learning problem. Therefore, we should generate a training set,

$$D = \{(\mathbf{s}_1, \mathbf{a}_{\text{greedy}}(\mathbf{s}_1)), \dots, (\mathbf{s}_m, \mathbf{a}_{\text{greedy}}(\mathbf{s}_m))\},$$

where the label of a sample state $\mathbf{s}$, $\mathbf{a}_{\text{greedy}}(\mathbf{s})$, is an action taken from the greedy policy with respect to $\hat{\mathcal{V}}^\pi(\mathbf{s}, \mathbf{w})$.

We can use this training set to train a machine learning model for the greedy policy using standard supervised learning techniques, i.e., classification if the action space is finite or regression if the action space is continuous. For concreteness, we focus here on a simple parametric method for finite actions; this approach extends naturally to other supervised learning techniques, including neural networks and non-parametric methods. Let $\hat{\pi}(\mathbf{a}|\mathbf{s}; \mathbf{w}')$ be a policy parametrized by a weight vector $\mathbf{w}'$ as follows,

$$\hat{\pi}(\mathbf{a}|\mathbf{s}; \mathbf{w}') = \frac{e^{\mathbf{w}'^T \phi(\mathbf{s}, \mathbf{a})}}{\sum_{\mathbf{a}' \in \mathcal{A}} e^{\mathbf{w}'^T \phi(\mathbf{s}, \mathbf{a}')}},$$

where $\phi(\mathbf{s}, \mathbf{a})$ are some state-action features (cf. the approximate Q function in Section 12.1.1). Note the distinction between the value function parameters $\mathbf{w}$ and policy parameters $\mathbf{w}'$. A standard classification technique for fitting the policy $\hat{\pi}(\mathbf{a}|\mathbf{s}; \mathbf{w}')$ to the data D is maximum likelihood:

$$\mathbf{w}'_{\text{ML}} = \arg \max_{\mathbf{w}'} \sum_{(\mathbf{s}_i, \mathbf{a}_{\text{greedy}}(\mathbf{s}_i)) \in D} \log (\hat{\pi}(\mathbf{a}_{\text{greedy}}(\mathbf{s}_i)) | \mathbf{s}_i; \mathbf{w}'),$$

where the optimization is typically solved using gradient descent methods.

Algorithm 29 Classification Based Approximate Policy Iteration

- 1: **Input:** Model M , policy π_0 , sample size N , approximation architectures $\hat{\mathcal{V}}(\mathbf{s}, \mathbf{w})$, $\hat{\pi}(\mathbf{a}|\mathbf{s}; \mathbf{w}')$
- 2: **For** $k = 0, 1, 2, \dots$
- 3: Collect a set of N samples $\{(\mathbf{s}_t, \mathbf{a}_t, \mathbf{r}_t, \mathbf{s}_{t+1})\}$ under π_k
- 4: Fit value function parameters $\mathbf{w}_k$
- 5: For each sample $\mathbf{s}_t$ calculate

$$\mathbf{a}_{\text{greedy}}(\mathbf{s}_t) = \arg \max_{\mathbf{a}} \left\{ \mathbf{r}(\mathbf{s}_t, \mathbf{a}) + \gamma \sum_{\mathbf{s}'} \mathbf{p}(\mathbf{s}' | \mathbf{s}_t, \mathbf{a}) \hat{\mathcal{V}}(\mathbf{s}', \mathbf{w}_k) \right\}$$

- 6: Fit policy parameters $\mathbf{w}'_k$

$$\mathbf{w}'_k = \arg \max_{\mathbf{w}'} \sum_{(\mathbf{s}_t, \mathbf{a}_{\text{greedy}}(\mathbf{s}_t))} \log \hat{\pi}(\mathbf{a}_{\text{greedy}}(\mathbf{s}_t)) | \mathbf{s}_t; \mathbf{w}')$$

Algorithm 29 gives a template for classification-based approximate policy iteration. For the policy evaluation in Line 4, any approximate policy evaluation method from Section 12.3 can be used. The maximum-likelihood fit in Line 6 can be replaced with different supervised learning methods for fitting a policy to data. Note that we require the state transition probabilities $\mathbf{P}$ for generating the greedy action labels in Line 5, which are input to the algorithm in the model M .

12.4.2 Approximate Policy Iteration with T-step Lookahead

As long as we are generating improved actions for fitting the policy, there is no reason to limit ourselves to greedy actions. We can, for instance, calculate actions that are optimal with respect to a T-step lookahead (whereas greedy corresponds to $T = 1$),

$$\pi_{\text{T-greedy}}(\hat{\mathcal{V}})(\mathbf{s}_t) = \arg \max_{\mathbf{a}_t} \left\{ \max_{\pi} \mathbb{E}^{\pi, \mathbf{a}_t} \left[\sum_{t'=t}^{t+T-1} \gamma^{t'-t} \mathbf{r}(\mathbf{s}_{t'}) + \gamma^T \hat{\mathcal{V}}(\mathbf{s}_{t+T}) \right] \right\}. \quad (12.23)$$

The next theorem show that as we increase the lookahead horizon T , the error bound for approximate policy iteration improves.

Theorem 12.12. *If for each iteration k the policies are approximated well over $\mathcal{S}$:*

$$\|\hat{\mathcal{V}}_k(\mathbf{s}) - \mathcal{V}^{\pi_k}(\mathbf{s})\|_{\infty} \leq \delta, \quad (12.24)$$

and policy improvement approximates well

$$\|\mathcal{T}^{\pi_{k+1}} \hat{\mathcal{V}}_k - (\mathcal{T}^*)^T \hat{\mathcal{V}}_k\|_{\infty} < \varepsilon, \quad (12.25)$$

Then

$$\limsup_k \|\mathcal{V}^{\pi_k}(\mathbf{s}) - \mathcal{V}^*(\mathbf{s})\|_{\infty} \leq \frac{\varepsilon + 2\gamma\delta}{(1-\gamma)(1-\gamma^T)}.$$

Proof. (Sketch) The proof follows the lines of Theorem 12.11, where in Equation 12.22 we replace the γ -contraction of $\mathcal{T}^*$ with the γ^T contraction of $(\mathcal{T}^*)^T$. $\square$

12.4.3 Approximate Policy Iteration with Monte-Carlo Tree Search

Calculating the optimal action for a T -step lookahead in (12.23) is not trivial for $T > 1$, as essentially we need to solve a finite horizon MDP on a potentially large state space. Even if the number of possible state transitions for each state-action pair is bounded by some m , the number of states reachable from $\mathbf{s}_t$ in T steps is bounded by $(\mathcal{A}m)^T$, therefore searching for an optimal policy becomes intractable.

A family of methods that has proven to work very well in practice approximates the optimal action in (12.25) using an approximate tree search algorithm. In particular, we focus here on the Monte-Carlo Tree Search (MCTS) algorithm.

Monte-Carlo Tree Search We can think of the optimization problem in (12.23) as finding an optimal action for each possible state that can be reached from $\mathbf{s}_t$ in T steps.

In MCTS, we sequentially build a tree representing the reachable states and their Q values. The idea is to build the tree in a way that will focus on the more promising

actions, and therefore will provide a good approximation of (12.25) even before the full tree is built. To do this, for each state we encounter, we maintain a Q value estimate, and expand the tree sequentially by selecting actions that maximize,

$$a = \arg \max Q(s, a) + B(s, a),$$

where $B(s, a)$ is an exploration bonus. Thus, most of the search is focused on exploring actions with a high Q estimate, while the bonus term drives exploration of other actions, which is important in case the Q estimate is not accurate.

A common term for the exploration bonus is termed *upper confidence trees (UCT)*,

$$B_{\text{UCT}}(s, a) = \frac{\sqrt{\log \sum_{a'} N(s, a')}}{1 + N(s, a)},$$

where $N(s, a)$ counts how many times a state-action pair was encountered during the tree search, and π is the current policy in the approximate policy iteration. This term gives a higher bonus to actions that were not sampled often. Some motivation for this specific form of exploration-exploitation balance comes from a formal result that we shall establish in Chapter 14 for the upper confidence bound algorithm.

An alternative heuristic for the exploration bonus that was suggested in the popular Alpha Zero study takes into account the policy, and uses it to weigh the bonus term,

$$B_{\text{AlphaZero}}(s, a) = \pi(s, a) \frac{\sqrt{\sum_{a'} N(s, a')}}{1 + N(s, a)}.$$

Algorithm 30 Monte-Carlo Tree Search (MCTS)

```
1: Input: Simulator  $M$ , initial state  $\mathbf{s}_{\text{root}}$ , value estimate  $\hat{\mathcal{V}}(\mathbf{s})$ , horizon  $T$ , budget  $K$ 
2: Initialize Tree =  $\{\mathbf{s}_{\text{root}}\}$ 
3: Initialize  $\forall \mathbf{a} : W(\mathbf{s}_{\text{root}}, \mathbf{a}) = 0, N(\mathbf{s}_{\text{root}}, \mathbf{a}) = 0, Q(\mathbf{s}_{\text{root}}, \mathbf{a}) = 0$ 
4: For  $k = 0, \dots, K$ 
5:   search( $\mathbf{s}, 0$ )
6: Return  $\arg \max_{\mathbf{a}} Q(\mathbf{s}_{\text{root}}, \mathbf{a})$ 
7:
8: function search( $\mathbf{s}, t$ )
9:   If  $\mathbf{s} \notin \text{Tree}$ 
10:    Add  $\mathbf{s}$  to Tree.
11:    Initialize  $\forall \mathbf{a} : W(\mathbf{s}, \mathbf{a}) = 0, N(\mathbf{s}, \mathbf{a}) = 0, Q(\mathbf{s}, \mathbf{a}) = 0$ 
12:    Return  $\hat{\mathcal{V}}(\mathbf{s})$ 
13:   If  $t = T$ :
14:     Return  $\hat{\mathcal{V}}(\mathbf{s})$ 
15:    $\mathbf{a} = \arg \max Q(\mathbf{s}, \mathbf{a}) + B(\mathbf{s}, \mathbf{a})$ 
16:    $\mathbf{s}' \sim p(\cdot | \mathbf{s}, \mathbf{a})$ 
17:    $W(\mathbf{s}, \mathbf{a}) = W(\mathbf{s}, \mathbf{a}) + r(\mathbf{s}, \mathbf{a}) + \gamma \cdot \text{search}(\mathbf{s}', t + 1)$ 
18:    $N(\mathbf{s}, \mathbf{a}) = N(\mathbf{s}, \mathbf{a}) + 1$ 
19:    $Q(\mathbf{s}, \mathbf{a}) = W(\mathbf{s}, \mathbf{a}) / N(\mathbf{s}, \mathbf{a})$ 
20:   Return  $Q(\mathbf{s}, \mathbf{a})$ 
```

Finally, we can use the MCTS algorithm 30 as a replacement of the greedy action selection in Line 5 of the classification based approximate policy iteration algorithm 29, as an approximation of a T -step lookahead action.

Remark 12.7. *In a very influential study [141], a variant of the MCTS-based policy iteration algorithm above termed AlphaGo Zero was used to train a super-human level policy in the challenging game of Go, and followup works extended the results to other board games [140]. These methods used deep neural networks for the policy and value function approximations, and were trained to play games using self-play: the policy was trained to play against itself as an opponent. For a fixed opponent policy, the game can be cast as an MDP as follows: following an action by the player at some state, the next state transition corresponds to first changing the board state according to the player's action, then using the policy to sample an action for the opponent, and subsequently changing board state according to the opponent action. In self-play, every once in a while, the opponent policy was changed to the currently*

trained policy, effectively changing the MDP.

12.4.4 Online Model Free Approximate Policy Iteration – SARSA

We now consider approximate policy iteration methods that do not require the model P to compute the greedy improvement step. The main trick, similarly to the tabular setting, is to consider the state-action value function Q instead of the state values V .

We start with an online method. For a given policy π , we can easily modify the TD(0) algorithm above to learn $\hat{Q}^\pi(\mathbf{s}, \mathbf{a}; \mathbf{w})$.

Algorithm 31 SARSA with Function Approximation

- 1: **Initialize:** Set $\mathbf{w}_0 = 0$, $\mathbf{s}_0, \mathbf{a}_0$ arbitrary.
- 2: **For** $t = 0, 1, 2, \dots$
- 3: Take action $\mathbf{a}_t$ at state $\mathbf{s}_t$
- 4: Observe $\mathbf{r}_t, \mathbf{s}_{t+1}$
- 5: Choose next action: $\mathbf{a}_{t+1}$
- 6: Update:

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha_t \left(r(\mathbf{s}_t, \mathbf{a}_t) + \gamma \hat{Q}(\mathbf{s}_{t+1}, \mathbf{a}_{t+1}; \mathbf{w}_t) - \hat{Q}(\mathbf{s}_t, \mathbf{a}_t; \mathbf{w}_t) \right) \nabla_{\mathbf{w}} \hat{Q}(\mathbf{s}_t, \mathbf{a}_t; \mathbf{w}_t)$$

The actions in Line 5 are typically selected according to an ξ -greedy or softmax rule. Thus, policy evaluation is interleaved with policy improvement.

12.4.5 Batch Model Free Approximate Policy Iteration – Least Squares Policy Iteration (LSPI)

One can also derive an approximate policy iteration algorithm that works on a batch of data. Looking at the SARSA algorithm above, where action selection was updated throughout the run of the algorithm, this may seem a bit surprising – how can we perform policy iteration on data that was collected once, using a single policy? The trick is to recall the Bellman equation for Q . Let π_{greedy} be a policy that is greedy with respect to a value function $\hat{Q}^\pi(\mathbf{s}, \mathbf{a}; \mathbf{w})$, $\pi_{\text{greedy}}(\mathbf{s}) = \arg \max_{\mathbf{a}} \hat{Q}^\pi(\mathbf{s}, \mathbf{a}; \mathbf{w})$. Then $Q^{\pi_{\text{greedy}}}$ satisfies:

$$Q^{\pi_{\text{greedy}}}(\mathbf{s}, \mathbf{a}) = r(\mathbf{s}, \mathbf{a}) + \gamma \mathbb{E} [Q^{\pi_{\text{greedy}}}(\mathbf{s}', \pi_{\text{greedy}}(\mathbf{s}'))].$$

Notice that in the equation above the expectation is with respect to the state transitions, and not with respect to the actions of any policy, therefore we can approximate it using a fixed set of samples. The Least Squares Policy Iteration (LSPI) builds on this idea to iteratively compute a least squares fit to the greedy policy with respect to the current Q . Consider linear function approximation $\hat{Q}^\pi(\mathbf{s}, \mathbf{a}) = \mathbf{w}^T \phi(\mathbf{s}, \mathbf{a})$. The idea is to use LSTD(0) to iteratively fit $\hat{Q}^{\pi_k}$, where π_k is the greedy policy w.r.t. $\hat{Q}^{\pi_{k-1}}$.

Algorithm 32 Least Squares Policy Iteration (LSPI)

- 1: **Input:** Policy π_0
- 2: Collect a set of N samples $\{(\mathbf{s}_t, \mathbf{a}_t, \mathbf{r}_t, \mathbf{s}_{t+1})\}$ under π_0
- 3: **For** $k = 1, 2, \dots$
- 4: Compute:

$$\hat{A}_N^k = \frac{1}{N} \sum_{t=1}^N \phi(\mathbf{s}_t, \mathbf{a}_t) (\phi^T(\mathbf{s}_t, \mathbf{a}_t) - \gamma \phi^T(\mathbf{s}_{t+1}, \mathbf{a}_{t+1}^*)),$$

$$\text{where } \mathbf{a}_{t+1}^* = \arg \max_{\mathbf{a}} \hat{Q}^{\pi_{k-1}}(\mathbf{s}_{t+1}, \mathbf{a}; \mathbf{w}_{k-1}) = \arg \max_{\mathbf{a}} \mathbf{w}_{k-1}^T \phi(\mathbf{s}_{t+1}, \mathbf{a})$$

$$\hat{b}_N^k = \frac{1}{N} \sum_{t=1}^N \phi(\mathbf{s}_t, \mathbf{a}_t) \mathbf{r}(\mathbf{s}_t, \mathbf{a}_t)$$

- 5: Solve:

$$\mathbf{w}_k = (\hat{A}_N^k)^{-1} \hat{b}_N^k$$

It is straightforward to extend the LSPI algorithm to nonlinear function approximation, by replacing LSTD with, e.g., projected value iteration (cf. Remark 12.3). It is also possible to collect data from the modified policy at each iteration k , instead of the initial policy.

12.5 Approximate Value Iteration

Approximate value iteration algorithms directly approximate the optimal value function (or optimal Q function). Let us first consider the linear case. The idea in approximate VI is similar to the PBE, but replacing $\mathcal{T}^\pi$ with $\mathcal{T}^*$. That is, we seek

solutions to the following projected equation:

$$\Phi_{\mathbf{w}} = \Pi \mathcal{T}^* \{\Phi_{\mathbf{w}}\},$$

where Π is some projection, such as the weighted least squares projection Π_{ξ} considered above. Recall that $\mathcal{T}^*$ is a contraction in the $\|\cdot\|_{\infty}$ norm. Unfortunately, Π is not necessarily a contraction in $\|\cdot\|_{\infty}$ for general function approximation, and not even for the weighted least squares projection Π_{ξ} .² On the other hand, $\mathcal{T}^*$ is not a contraction in the $\|\cdot\|_{\xi}$ norm. Thus, we have no guarantee that the projected equation has a solution. Nevertheless, algorithms based on this approach have achieved impressive success in practice.

12.5.1 Online Approximate Value Iteration – Q Learning

The function approximation version of online Q-learning resembles SARSA, only with an additional maximization over the next action:

Algorithm 33 Q-learning with Function Approximation

- 1: **Initialize:** Set $\mathbf{w}_0 = 0$.
- 2: **For** $t = 0, 1, 2, \dots$
- 3: Observe: $\mathbf{s}_t$
- 4: Choose action: $\mathbf{a}_t$
- 5: Observe $\mathbf{r}_t, \mathbf{s}_{t+1}$
- 6: Update:

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha_t \left(\mathbf{r}(\mathbf{s}_t, \mathbf{a}_t) + \gamma \max_{\mathbf{a}} \hat{Q}(\mathbf{s}_{t+1}, \mathbf{a}; \mathbf{w}_t) - \hat{Q}(\mathbf{s}_t, \mathbf{a}_t; \mathbf{w}_t) \right) \nabla_{\mathbf{w}} \hat{Q}(\mathbf{s}_t, \mathbf{a}_t; \mathbf{w}_t).$$

The actions are typically selected according to an ε -greedy or softmax rule, to balance exploration and exploitation.

12.5.2 Batch Approximate Value Iteration – Fitted Q

In this approach, we iteratively project (fit) the Q function based on the projected equation:

$$\hat{Q}(\mathbf{w}_{n+1}) = \Pi \mathcal{T}^* \hat{Q}(\mathbf{w}_n).$$

²A restricted class of function approximators for which contraction does hold is called averagers, as was proposed in [52]. The k-nearest neighbors approximation, for example, is an averager.

Assume we have a data set of samples $\{\mathbf{s}_i, \mathbf{a}_i, \mathbf{s}'_i, \mathbf{r}_i\}$, obtained from some data collection policy. Then, the right hand side of the equation denotes a regression problem where the samples and labels are: $\left\{(\mathbf{s}_i, \mathbf{a}_i), \left(\mathbf{r}_i + \gamma \max_{\mathbf{a}} \widehat{Q}(\mathbf{s}'_i, \mathbf{a}; \mathbf{w}_n)\right)\right\}$. Thus, by solving a sequence of regression problems we approximate a solution to the projected equation.

Algorithm 34 Fitted Q Learning

- 1: **Input:** Data collection policy π_0 , approximation architecture $\widehat{Q}(\mathbf{s}, \mathbf{a}; \mathbf{w})$
- 2: Initialize $\mathbf{w}_0 = 0$
- 3: Collect a set of N samples $\{(\mathbf{s}_t, \mathbf{a}_t, \mathbf{r}_t, \mathbf{s}_{t+1})\}$ under π_0
- 4: **For** $k = 0, 1, 2, \dots$
- 5: Compute labels for each sample $x_t = (\mathbf{s}_t, \mathbf{a}_t)$:

$$y_t = \mathbf{r}_t + \gamma \max_{\mathbf{a}} \widehat{Q}(\mathbf{s}_{t+1}, \mathbf{a}; \mathbf{w}_k)$$

- 6: Solve least-squares regression for $\{(x_t, y_t)\}$:

$$\mathbf{w}_{k+1} = \arg \min_{\mathbf{w}} \frac{1}{N} \sum_{\mathbf{t}} \left(\widehat{Q}(\mathbf{s}_t, \mathbf{a}_t; \mathbf{w}) - y_t \right)^2$$

Note that approximate VI algorithms are *off-policy* algorithms. Thus, in both Q-learning and fitted-Q, the policy that explores the MDP can be arbitrary (assuming of course it explores ‘enough’ interesting states).

Remark 12.8. *It is instructive to compare Fitted Q learning Algorithm 34 with linear function approximation and the LSPI of Algorithm 32. If $\widehat{Q}(\mathbf{s}, \mathbf{a}; \mathbf{w}) = \mathbf{w}^T \phi(\mathbf{s}, \mathbf{a})$, Line 6 in Algorithm 34 would be equivalent to:*

$$\mathbf{w}_{k+1} = \arg \min_{\mathbf{w}} \frac{1}{N} \sum_{\mathbf{t}} \left(\mathbf{w}^T \phi(\mathbf{s}_t, \mathbf{a}_t) - \mathbf{r}_t - \gamma \mathbf{w}_k^T \phi(\mathbf{s}_{t+1}, \mathbf{a}_{t+1}^*) \right)^2,$$

while the LSPI update is

$$\mathbf{w}_{k+1} = \arg \min_{\mathbf{w}} \frac{1}{N} \sum_{\mathbf{t}} \left(\mathbf{w}^T \phi(\mathbf{s}_t, \mathbf{a}_t) - \mathbf{r}_t - \gamma \mathbf{w}^T \phi(\mathbf{s}_{t+1}, \mathbf{a}_{t+1}^*) \right)^2,$$

where $\mathbf{a}^*$ in both cases is the same greedy action with respect to the weights $\mathbf{w}_k$. The difference is subtle but important. In Fitted Q, the weights for the next state value

are ‘frozen’, as we are effectively applying $\Pi\mathcal{T}^*$ to the value function implied by $\mathbf{w}_k$, $\Phi\mathbf{w}_k$. In LSPI, on the other hand, we are trying to approximate the value of the greedy policy, by solving the projected equation $\Phi\mathbf{w} = \Pi\mathcal{T}^{\pi_{\text{greedy}}}\Phi\mathbf{w}$, and therefore the weights of the next state value are part of the optimization. Thus, even though it holds that $\Pi\mathcal{T}^{\pi_{\text{greedy}}} = \Pi\mathcal{T}^*$, the two updates are very different, and relate to the difference between policy iteration and value iteration (cf. Section 6.8). When using tabular features (cf. Remark 12.1), it may be verified that LSPI is equivalent to policy iteration (Algorithm 10), while Fitted Q is equivalent to value iteration (Algorithm 9), both applied to the empirical model from the data.

12.5.3 The Deep Q Learning Algorithm

A notable milestone in RL history is the application of RL to a large collection of Atari games, using the Deep Q Learning (DQN) algorithm [110], which advanced the study of deep RL. DQN employed several techniques to stabilize learning, which were important for its training with deep neural networks. Here, we shall see that these techniques resemble a mix of online Q learning and batch fitted Q.

Replay Buffer The algorithm is online, similarly to the Q learning Algorithm 33. However, instead of updating the Q function weights using the current $(\mathbf{s}, \mathbf{a}, \mathbf{r}, \mathbf{s}')$ sample, as in Q learning, the current sample is stored in a buffer (cf. Definition 11.4), and a random sample is retrieved from the buffer to perform the update. This works to break correlations between subsequent updates to the Q function, which may arise from the algorithm encountering similar states during the agent’s interaction with the environment.

Target Q Network Similarly to the fitted Q algorithm 34, DQN uses the same weight vector $\bar{\mathbf{w}}$ in the labels $\mathbf{r}(\mathbf{s}_t, \mathbf{a}_t) + \gamma \max_{\mathbf{a}} \widehat{Q}(\mathbf{s}_{t+1}, \mathbf{a}; \bar{\mathbf{w}})$ for multiple updates of the algorithm. This works to stabilize the training, as current changes to the Q approximation do not affect the update direction. Every fixed number of updates, the *target* weights $\bar{\mathbf{w}}$ are replaced to be the current weights.

Algorithm 35 DQN

- 1: **Input:** Replay buffer size N , initial weights, $\mathbf{w}_0$, initial state $\mathbf{s}_0$, target update rate K , minibatch size B
- 2: **Initialize:** Set target weights $\bar{\mathbf{w}} = \mathbf{w}_0$, replay buffer $\mathcal{D} = \{\}$
- 3: **For** $t = 0, 1, 2, \dots$
- 4: Observe: $\mathbf{s}_t$
- 5: Choose action: $\mathbf{a}_t$
- 6: Observe $\mathbf{r}_t, \mathbf{s}_{t+1}$
- 7: Add $(\mathbf{s}_t, \mathbf{a}_t, \mathbf{r}_t, \mathbf{s}_{t+1})$ to $\mathcal{D}$
- 8: Sample minibatch $b = \{(\mathbf{s}_i, \mathbf{a}_i, \mathbf{r}_i, \mathbf{s}'_i)\}$ of size B uniformly from $\mathcal{D}$
- 9: Compute labels for each sample $x_i = (\mathbf{s}_i, \mathbf{a}_i)$ in b :

$$y_i = \mathbf{r}_i + \gamma \max_{\mathbf{a}} \hat{Q}(\mathbf{s}'_i, \mathbf{a}; \bar{\mathbf{w}})$$

- 10: Take gradient descent step:

$$\mathbf{w}_{t+1} = \mathbf{w}_t + \alpha_t \nabla_{\mathbf{w}} \left(\frac{1}{B} \sum_{(x_i, y_i) \in b} \left(\hat{Q}(\mathbf{s}_i, \mathbf{a}_i; \mathbf{w}_t) - y_i \right)^2 \right)$$

- 11: Every K iterations set $\bar{\mathbf{w}} = \mathbf{w}_t$
-

As in the Q learning algorithm, action selection in Line 5 of DQN follows an ε -greedy or softmax rule, to balance exploration and exploitation.

12.6 Bibliography Notes

The idea of using function approximation for value functions dates back to the previous century [15, 135, 9, 158]. The error bounds for approximate value iteration and approximate policy iteration presented here were developed in [18]. Error bounds based on the L2 norm were developed by Munos [112].

Least squares regression is a standard statistical method in machine learning [111, 133].

The projected Bellman equation and its analysis can be found in [19]. The LSTD algorithm is due to [24], and was further investigated in [23]. The LSPI algorithm is by [88]. Fitted Q learning and approximate value iteration was proposed in several different studies [52, 164, 122, 116, 44]. The deep Q learning algorithm is by [110]. The convergence of TD(0) with linear function approximation was established in

[162]. Classification based policy iteration was studied in [89, 95, 130]. Approximate PI with multi-step lookahead was proposed in [43].

Monte Carlo Tree Search was developed by [35, 83]. Its extension using RL was applied to Go and other board games by [138, 140].

Chapter 13

Large State Space: Policy Gradient Methods

This chapter continues looking at the case where the MDP models have a large state space. In the previous chapter, we looked at approximating the value function. In this chapter, we will consider learning a policy directly and optimizing it.

Chapter Orientation

Main Prerequisites:

- Episodic MDPs (Chapter 7)
- Value function approximation (Chapter 12)

Main terminology:

- θ parameters (weights) in policy function approximation
- $\pi(\mathbf{a}|\mathbf{s}, \theta)$ parametrized policy
- μ initial state distribution
- $J(\theta)$ expected value for policy parametrized by θ
- θ^* optimal policy parameters, which maximize $J(\theta)$
- $\theta_{\mathbf{s}, \mathbf{a}}$ parameter for state $\mathbf{s}$ and action $\mathbf{a}$ in a simplex policy

- $d^\pi(\mathbf{s})$ visitation frequency of state $\mathbf{s}$ under policy π
- $b(\mathbf{s})$ baseline for state $\mathbf{s}$
- $\mathcal{A}^\pi(\mathbf{s}, \mathbf{a})$ advantage function for state $\mathbf{s}$ and action $\mathbf{a}$ under policy π

13.1 Problem Setting

To describe the problem formally, we shall make an assumption about the policy structure and the optimization objective, as follows. The policy will have a parametrization $\theta \in \mathbb{R}^d$ and we denote by $\pi(\mathbf{a}|\mathbf{s}, \theta)$ the probability of selecting action $\mathbf{a}$ when observing state $\mathbf{s}$, and having a policy parametrization θ .

For technical ease, we consider the stochastic shortest path objective described in Chapter 7:

$$\mathcal{V}^\pi(\mathbf{s}) = \mathbb{E}^\pi \left[\sum_{t=0}^{\tau} \mathbf{r}_t \mid \mathbf{s}_0 = \mathbf{s} \right],$$

where τ is the random variable of termination time, which we will assume to be bounded with probability one, i.e., Assumption 7.1 holds. We are given a distribution over the initial state of the MDP, $\mu(\mathbf{s}_0)$, and define $J(\theta) \triangleq \mathbb{E}[\mathcal{V}^\pi(\mathbf{s}_0)] = \mu^\top \mathcal{V}^\pi$ to be the expected value of the policy (where the expectation is with respect to μ).

The optimization problem we consider is:

$$\theta^* = \arg \max_{\theta} J(\theta). \quad (13.1)$$

This maximization problem can be solved in multiple ways. We will mainly explore gradient-based methods.

In the setting that the MDP is not known, we shall assume that we are allowed to simulate ‘rollouts’ from a given policy, $\mathbf{s}_0, \mathbf{a}_0, \mathbf{r}_0, \dots, \mathbf{s}_\tau, \mathbf{a}_\tau, \mathbf{r}_\tau$, where $\mathbf{s}_0 \sim \mu$, $\mathbf{a}_t \sim \pi(\cdot|\mathbf{s}_t, \theta)$, and $\mathbf{s}_{t+1} \sim P(\cdot|\mathbf{s}_t, \mathbf{a}_t)$. We shall devise algorithms that use such rollouts to modify the policy parameters θ in a way that increases $J(\theta)$.

13.2 Policy Representations

We start with a few examples on how to parameterize the policy.

Log linear policy: We will assume a feature encoding of the state-action pairs, i.e., $\phi(\mathbf{s}, \mathbf{a}) \in \mathbb{R}^d$. Given the parameter θ , the linear part will compute $\xi(\mathbf{s}, \mathbf{a}) = \phi(\mathbf{s}, \mathbf{a})^\top \theta$.

Given the values of $\xi(\mathbf{s}, \mathbf{a})$ for each $\mathbf{a} \in \mathcal{A}$, the policy selects action $\mathbf{a}$ with probability proportional to $e^{\xi(\mathbf{s}, \mathbf{a})}$. Namely,

$$\pi(\mathbf{a}|\mathbf{s}, \theta) = \frac{e^{\xi(\mathbf{s}, \mathbf{a})}}{\sum_{b \in \mathcal{A}} e^{\xi(\mathbf{s}, b)}}$$

Note that this is essentially a softmax selection over $\xi(\mathbf{s}, \mathbf{a})$.

Gaussian linear policy: This policy representation applies when the action space is a real number, i.e., $\mathcal{A} = \mathbb{R}$. The encoding of states is $\phi(\mathbf{s}) \in \mathbb{R}^d$. Given a state $\mathbf{s}$, we compute $\xi(\mathbf{s}) = \phi(\mathbf{s})^\top \theta$. We select an action $\mathbf{a}$ from the normal distribution with mean $\xi(\mathbf{s})$ and variance σ^2 , that is, $\mathcal{N}(\xi(\mathbf{s}), \sigma^2)$ (the Gaussian policy has an additional parameter σ). For an action space with dimension larger than 1, we can consider such a policy with a different parameter vector θ for every dimension.

Non-linear policy: Note that in both the log-linear and Gaussian-linear policies above, the dependence of ξ on θ was linear. It is straightforward to extend these policies such that ξ depends on θ in a more expressive and non-linear manner. A popular parametrization is a feedforward neural network, also called a multi-layer perceptron (MLP). An MLP with d inputs, 2 hidden layers of sizes h_1 , h_2 , and k outputs has parameters $\theta_0 \in \mathbb{R}^{d \times h_1}$, $\theta_1 \in \mathbb{R}^{h_1 \times h_2}$, $\theta_2 \in \mathbb{R}^{h_2 \times k}$. The MLP computes $\xi \in \mathbb{R}^k$ as follows:

$$\xi(\mathbf{s}) = \theta_2^T f_{nl}(\theta_1^T (f_{nl}(\theta_0^T \phi(\mathbf{s})))) \in \mathbb{R}^k,$$

where f_{nl} is some non-linear function that is applied element-wise to each component of a vector, for example, the Rectified Linear Unit (ReLU) defined as $\text{ReLU}(x) = \max(0, x)$. Once ξ is computed, selecting an action proceeds similarly as above, e.g., by sampling from the normal distribution with mean $\xi(\mathbf{s})$ and variance σ^2 .

Simplex policy: This policy representation will be used mostly for pedagogical reasons and can express any Markov stochastic policy. For a finite state and action space, let $\theta \in [0, \infty)^{\mathcal{S} \times \mathcal{A}}$, and denote $\theta_{\mathbf{s}, \mathbf{a}}$ the parameter corresponding to state $\mathbf{s}$ and action $\mathbf{a}$. We define $\pi(\mathbf{a}|\mathbf{s}, \theta) = \frac{\theta_{\mathbf{s}, \mathbf{a}}}{\sum_{\mathbf{a}'} \theta_{\mathbf{s}, \mathbf{a}'}}$. Clearly, any Markov policy $\tilde{\pi}$ can be represented by setting $\theta_{\mathbf{s}, \mathbf{a}} = \tilde{\pi}(\mathbf{a}|\mathbf{s})$.

13.3 The Policy Performance Difference Lemma

Considering the optimization problem (13.1), an important question is how a change in the parameters θ , which induces a change in the policy π , relates to a change in

the performance criterion $J(\theta)$. We shall derive a fundamental result known as the performance difference lemma.

Let $P^\pi(\mathbf{s}'|\mathbf{s})$ denote the state transitions in the Markov chain induced by policy π (following Section 7.3, we only consider transitions between non-goal states). Let us define the visitation frequencies $d^\pi(\mathbf{s}) = \sum_{t=0}^{\infty} \Pr(\mathbf{s}_t = \mathbf{s}|\mu, \pi)$. We first establish the following result.

Proposition 13.1. *We have that $d^\pi = \mu + d^\pi P^\pi$, and therefore $d^\pi = \mu(I - P^\pi)^{-1}$.*

Proof. We have,

$$\begin{aligned} d^\pi(\mathbf{s}) &= \mu(\mathbf{s}) + \sum_{t=1}^{\infty} \Pr(\mathbf{s}_t = \mathbf{s}|\mu, \pi) \\ &= \mu(\mathbf{s}) + \sum_{t=1}^{\infty} \sum_{\mathbf{s}'} \Pr(\mathbf{s}_{t-1} = \mathbf{s}'|\mu, \pi) P^\pi(\mathbf{s}|\mathbf{s}') \\ &= \mu(\mathbf{s}) + \sum_{\mathbf{s}'} P^\pi(\mathbf{s}|\mathbf{s}') \sum_{t=1}^{\infty} \Pr(\mathbf{s}_{t-1} = \mathbf{s}'|\mu, \pi) \\ &= \mu(\mathbf{s}) + \sum_{\mathbf{s}'} d^\pi(\mathbf{s}') P^\pi(\mathbf{s}|\mathbf{s}'). \end{aligned}$$

Writing the result in matrix notation gives the first result. For the second result, Proposition 7.1 showed that $(I - P^\pi)$ is invertible. $\square$

To deal with large state spaces, as we did in previous chapters, we will want to use sampling to approximate quantities that depend on all states. Note that expectations over the state visitation frequencies can be approximated by sampling from policy rollouts.

Proposition 13.2. *Consider a random rollout from the policy $\mathbf{s}_0, \mathbf{a}_0, \mathbf{r}_0, \dots, \mathbf{s}_\tau, \mathbf{a}_\tau, \mathbf{r}_\tau$, where $\mathbf{s}_0 \sim \mu$, $\mathbf{a}_t \sim \pi(\cdot|\mathbf{s}_t, \theta)$, $\mathbf{s}_{t+1} \sim P(\cdot|\mathbf{s}_t, \mathbf{a}_t)$, and τ is the termination time. For some function of states $g(\mathbf{s})$, we have that:*

$$\sum_{\mathbf{s}} d^\pi(\mathbf{s}) g(\mathbf{s}) = \mathbb{E}^\pi \left[\sum_{t=0}^{\tau} g(\mathbf{s}_t) \right].$$

Proof. We have

$$\begin{aligned}
\mathbb{E}^\pi \left[\sum_{t=0}^{\tau} g(\mathbf{s}_t) \right] &= \mathbb{E}^\pi \left[\sum_{t=0}^{\tau} \sum_{\mathbf{s}} \mathbb{I}(\mathbf{s}_t = \mathbf{s}) g(\mathbf{s}_t) \right] \\
&= \sum_{\mathbf{s}} \mathbb{E}^\pi \left[\sum_{t=0}^{\tau} \mathbb{I}(\mathbf{s}_t = \mathbf{s}) g(\mathbf{s}_t) \right] \\
&= \sum_{\mathbf{s}} \mathbb{E}^\pi \left[\sum_{t=0}^{\tau} \mathbb{I}(\mathbf{s}_t = \mathbf{s}) g(\mathbf{s}) \right] \\
&= \sum_{\mathbf{s}} g(\mathbf{s}) \mathbb{E}^\pi \left[\sum_{t=0}^{\tau} \mathbb{I}(\mathbf{s}_t = \mathbf{s}) \right] \\
&= \sum_{\mathbf{s}} g(\mathbf{s}) d^\pi(\mathbf{s}),
\end{aligned}$$

where $\mathbb{I}[\cdot]$ is the indicator function. □

We now state the performance difference lemma.

Lemma 13.3. *For any two policies, π and π' , corresponding to parameters θ and θ' , we have*

$$J(\theta') - J(\theta) = \sum_{\mathbf{s}} d^{\pi'}(\mathbf{s}) \sum_{\mathbf{a}} \pi'(\mathbf{a}|\mathbf{s}) (\mathcal{Q}^\pi(\mathbf{s}, \mathbf{a}) - \mathcal{V}^\pi(\mathbf{s})). \quad (13.2)$$

Proof. We have that $\mathcal{V}^{\pi'} = (I - \mathbf{P}^{\pi'})^{-1} \mathbf{r}$, and therefore

$$\begin{aligned}
\mathcal{V}^{\pi'} - \mathcal{V}^\pi &= (I - \mathbf{P}^{\pi'})^{-1} \mathbf{r} - (I - \mathbf{P}^{\pi'})^{-1} (I - \mathbf{P}^{\pi'}) \mathcal{V}^\pi \\
&= (I - \mathbf{P}^{\pi'})^{-1} \left(\mathbf{r} + \mathbf{P}^{\pi'} \mathcal{V}^\pi - \mathcal{V}^\pi \right).
\end{aligned}$$

Multiplying both sides by μ , and by Proposition 13.1, $d^{\pi'} = \mu(I - \mathbf{P}^{\pi'})^{-1}$, this gives

$$J(\theta') - J(\theta) = d^{\pi'} \left(\mathbf{r} + \mathbf{P}^{\pi'} \mathcal{V}^\pi - \mathcal{V}^\pi \right).$$

Finally, note that $\sum_{\mathbf{a}} \pi'(\mathbf{a}|\mathbf{s}) \mathcal{Q}^\pi(\mathbf{s}, \mathbf{a}) = \mathbf{r}(\mathbf{s}) + \sum_{\mathbf{s}'} \mathbf{P}^{\pi'}(\mathbf{s}'|\mathbf{s}) \mathcal{V}^\pi(\mathbf{s}')$. □

Given some policy $\pi(\mathbf{a}|\mathbf{s})$, an improved policy $\pi'(\mathbf{a}|\mathbf{s})$ must satisfy that the right-hand side of Eq. 13.2 is positive. Let us try to intuitively understand this criterion. First, consider the simplex policy parametrization above, which can express any

Markov policy. Consider the policy iteration update $\pi'(\mathbf{s}) = \arg \max_{\mathbf{a}} \mathcal{Q}^\pi(\mathbf{s}, \mathbf{a})$. Substituting in the right-hand side of Eq. 13.2 yields a non-negative value for every $\mathbf{s}$, and therefore an improved policy as expected.

For some policy parametrizations, however, the terms in the sum in Eq. 13.2 cannot be made positive for all $\mathbf{s}$. To obtain policy improvement, the terms need to be balanced such that a positive sum is obtained. This is not straightforward for two reasons. First, for large state spaces, it is not tractable to compute the sum over $\mathbf{s}$, and sampling must be used to approximate this sum. However, straightforward sampling of states from a fixed policy will not work, as the weights in the sum, $d^{\pi'}(\mathbf{s})$, *depend on the policy π'* ! The basic insight is that when we modify π , we directly influence the action distribution, but we also indirectly change the state distribution, which influences the expected reward.

The following example shows that indeed, balancing the sum with respect to weights that correspond to *the current policy* π does not necessarily lead to a policy improvement.

Example 13.1. Consider the finite horizon MDP in Figure 13.1, where the policy is parametrized by $\theta = (\theta_1, \theta_2) \in [0, 1]^2$ and let π correspond to $\theta_1 = \theta_2 = 1/4$. It is easy to verify that $d^\pi(\mathbf{s}_1) = 1$, $d^\pi(\mathbf{s}_2) = 1/4$, and $d^\pi(\mathbf{s}_3) = 3/4$. Simple calculations give that

$$\begin{aligned} \mathcal{V}^\pi(\mathbf{s}_2) &= 3/4, & \mathcal{V}^\pi(\mathbf{s}_3) &= 1/4, \mathcal{V}^\pi(\mathbf{s}_1) = 3/8, \\ \mathcal{Q}^\pi(\mathbf{s}_2, \text{left}) - \mathcal{V}^\pi(\mathbf{s}_2) &= -3/4, & \mathcal{Q}^\pi(\mathbf{s}_2, \text{right}) - \mathcal{V}^\pi(\mathbf{s}_2) &= 1/4, \\ \mathcal{Q}^\pi(\mathbf{s}_3, \text{left}) - \mathcal{V}^\pi(\mathbf{s}_3) &= 3/4, & \mathcal{Q}^\pi(\mathbf{s}_3, \text{right}) - \mathcal{V}^\pi(\mathbf{s}_3) &= -1/4, \\ \mathcal{Q}^\pi(\mathbf{s}_1, \text{left}) - \mathcal{V}^\pi(\mathbf{s}_1) &= 3/8, & \mathcal{Q}^\pi(\mathbf{s}_1, \text{right}) - \mathcal{V}^\pi(\mathbf{s}_1) &= -1/8. \end{aligned}$$

We want to maximize $\sum_{\mathbf{s}} d^\pi(\mathbf{s}) \sum_{\mathbf{a}} \pi'(\mathbf{a}|\mathbf{s}) (\mathcal{Q}^\pi(\mathbf{s}, \mathbf{a}) - \mathcal{V}^\pi(\mathbf{s}))$. We now need to plug in the three states. For state $\mathbf{s}_1$, we have $\theta_1(3/4 - 3/8) + (1 - \theta_1)(1/4 - 3/8) = \frac{\theta_1}{2} - \frac{1}{8}$. For state $\mathbf{s}_2$ we have $\frac{1}{4} \left(-\frac{3}{4}\theta_2 + (1 - \theta_2)\frac{1}{4} \right) = \frac{1}{16} - \frac{\theta_2}{4}$. For state $\mathbf{s}_3$ we have $\frac{3}{4} \left(\frac{3}{4}\theta_2 + (1 - \theta_2)\frac{-1}{4} \right) = \frac{3\theta_2}{4} - \frac{3}{16}$. Maximizing over θ , we have,

$$\begin{aligned} \arg \max_{\theta_1} \left\{ \frac{\theta_1}{2} - \frac{1}{8} + \frac{1}{16} - \frac{\theta_2}{4} + \frac{3\theta_2}{4} - \frac{3}{16} \right\} &= \arg \max_{\theta_1} \frac{\theta_1}{2} = 1. \\ \arg \max_{\theta_2} \left\{ \frac{\theta_1}{2} - \frac{1}{8} + \frac{1}{16} - \frac{\theta_2}{4} + \frac{3\theta_2}{4} - \frac{3}{16} \right\} &= \arg \max_{\theta_2} \frac{\theta_2}{2} = 1. \end{aligned}$$

However, for π' that corresponds to $\theta' = (1, 1)$, we have that $\mathcal{V}^{\pi'}(\mathbf{s}_1) = 0 < \mathcal{V}^\pi(\mathbf{s}_1)$.

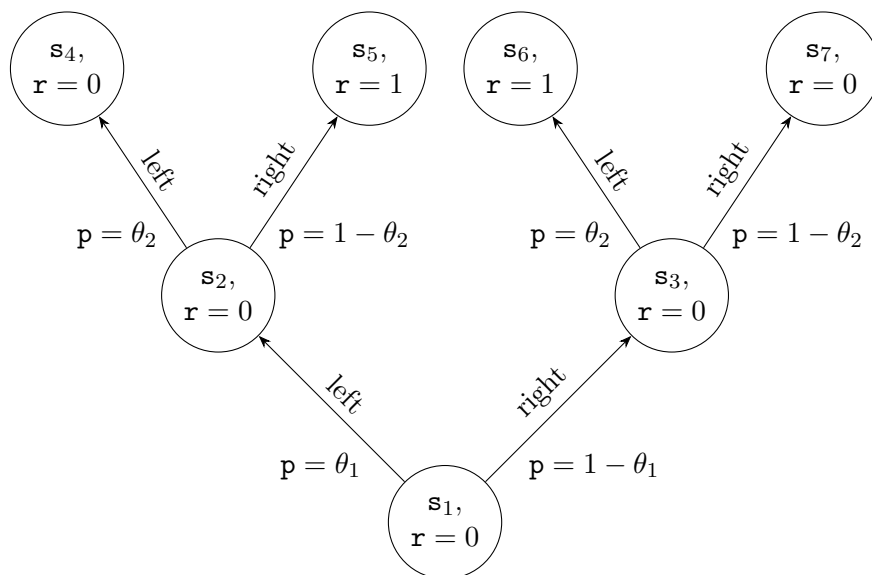


Figure 13.1: Example MDP

Intuitively, we expect that if the difference $\pi' - \pi$ is ‘small’, then the difference in the state visitation frequencies $d^{\pi'} - d^{\pi}$ would also be ‘small’, allowing us to safely replace $d^{\pi'}$ in the right-hand side of Eq. 13.2 with d^{π} . This is the route taken by several algorithmic approaches, which differ in the way of defining a ‘small’ policy perturbation. Of particular interest to us is the case of an infinitesimal perturbation, that is, the *policy gradient* $\nabla_{\theta} J(\theta)$. In the following, we shall describe in detail several algorithms for estimating the policy gradient.

13.4 Gradient-Based Policy Optimization

We would like to use the policy gradient to optimize the expected return $J(\theta)$ of the policy $\pi(\cdot|\cdot, \theta)$. We will compute the gradient of $J(\theta)$, i.e., $\nabla_{\theta} J(\theta)$. The update of the policy parameter θ is by gradient ascent,

$$\theta_{t+1} = \theta_t + \alpha \nabla_{\theta_t} J(\theta_t),$$

where α is a learning rate. For a small enough learning rate, each update is guaranteed to increase $J(\theta)$.

In the following, we shall explore several different approaches for calculating the gradient $\nabla_{\theta} J(\theta)$ using rollouts from the MDP.

13.4.1 Finite Differences Methods

These methods can be used even when we do not have a representation of the gradient of the policy or even the policy itself. This may arise many times when we have, for example, access to an off-the-shelf robot for which the software is already encoded in the robot. In such cases, we can estimate the gradient by introducing perturbations in the parameters.

The simplest case is component-wise gradient estimates, which is also named *coordinate ascent*. Let e_i be a unit vector, i.e., it has a value 1 in the i -th entry and a value 0 in all the other entries. The perturbation that we will add is δe_i for some $\delta > 0$. We will use the following approximation:

$$\frac{\partial}{\partial \theta_i} J(\theta) \approx \frac{\hat{J}(\theta + \delta e_i) - \hat{J}(\theta)}{\delta},$$

where $\hat{J}(\theta)$ is an unbiased estimator of $J(\theta)$. A more symmetric approximation is sometimes better,

$$\frac{\partial}{\partial \theta_i} J(\theta) \approx \frac{\hat{J}(\theta + \delta e_i) - \hat{J}(\theta - \delta e_i)}{2\delta}.$$

The problem is that we need to average many samples of $\hat{J}(\theta \pm \delta e_i)$ to overcome the noise. Another weakness is that we need to perform the computation per dimension. In addition, the selection of δ is also critical. A small δ might have a large noise rate that we need to overcome (by using many samples). A large δ runs the risk of facing the non-linearity of J .

Rather than performing the computation and optimization separately per dimension, we can perform a more global approach and use a least-squares estimation of the gradient. Consider a random vector u_i , then we have

$$J(\theta + \delta u_i) \approx J(\theta) + \delta u_i^\top \nabla J(\theta).$$

We can define the following least-square problem,

$$G = \arg \min_x \sum_i (J(\theta + \delta u_i) - J(\theta) - \delta u_i^\top x)^2,$$

where G is our estimate for $\nabla J(\theta)$.

We can reformulate the problem in matrix notation and define $\Delta J^{(i)} = J(\theta + \delta u_i) - J(\theta)$ and $\Delta J = [\dots, \Delta J^{(i)}, \dots]^\top$. We define $\Delta \theta^{(i)} = \delta u_i$, and the matrix $[\Delta \Theta] = [\dots \Delta \theta^{(i)}, \dots]^\top$, where the i -th row is $\Delta \theta^{(i)}$.

We would like to solve for the gradient, i.e.,

$$\Delta J \approx [\Delta\Theta]x .$$

This is a standard least-square problem and the solution is

$$G = ([\Delta\Theta]^\top [\Delta\Theta])^{-1} [\Delta\Theta]^\top \Delta J .$$

One issue that we neglected is that we actually do not have the value of $J(\theta)$. The solution is to also solve for the value of $J(\theta)$. We can define a matrix $M = [1, [\Delta\Theta]]$, i.e., adding a column of ones, a vector of unknowns $x = [J(\theta), \nabla J(\theta)]$, and have the target be $z = [\dots, J(\theta + \delta u_i), \dots]$. We can now solve for $z \approx Mx$, and this will also recover an estimate for $J(\theta)$.

13.5 Policy Gradient Theorem

The policy gradient theorem will relate the gradient of the expected return $\nabla J(\theta)$ and the gradients of the policy $\nabla \pi(\mathbf{a}|\mathbf{s}, \theta)$. We make the following assumption.

Assumption 13.1. *The gradient $\nabla \pi(\mathbf{a}|\mathbf{s}, \theta)$ exists and is finite for every $\theta \in \mathbb{R}^d$, $\mathbf{s} \in \mathcal{S}$, and $\mathbf{a} \in \mathcal{A}$.*

We will mainly try to make sure that we are able to use it to get estimates, and that the quantities would be indeed observable by the learner.

Theorem 13.4. *Let Assumption 13.1 hold. We have that*

$$\nabla J(\theta) = \sum_{\mathbf{s}} d^\pi(\mathbf{s}) \sum_{\mathbf{a}} \nabla \pi(\mathbf{a}|\mathbf{s}) Q^\pi(\mathbf{s}, \mathbf{a}).$$

Proof. For simplicity, we consider that θ is a scalar; the extension to the vector case is immediate. By definition, we have that

$$\begin{aligned} \frac{\partial J(\theta)}{\partial \theta} &= \lim_{\delta\theta \rightarrow 0} \frac{J(\theta + \delta\theta) - J(\theta)}{\delta\theta} \\ &= \lim_{\delta\theta \rightarrow 0} \frac{\sum_{\mathbf{s}} d^{\pi_{\theta+\delta\theta}}(\mathbf{s}) \sum_{\mathbf{a}} \pi_{\theta+\delta\theta}(\mathbf{a}|\mathbf{s}) (Q^{\pi_{\theta}}(\mathbf{s}, \mathbf{a}) - V^{\pi_{\theta}}(\mathbf{s}))}{\delta\theta} \\ &= \lim_{\delta\theta \rightarrow 0} \frac{\sum_{\mathbf{s}} d^{\pi_{\theta+\delta\theta}}(\mathbf{s}) \sum_{\mathbf{a}} (\pi_{\theta+\delta\theta}(\mathbf{a}|\mathbf{s}) - \pi_{\theta}(\mathbf{a}|\mathbf{s})) Q^{\pi_{\theta}}(\mathbf{s}, \mathbf{a})}{\delta\theta} \\ &= \sum_{\mathbf{s}} d^\pi(\mathbf{s}) \sum_{\mathbf{a}} \frac{\partial \pi(\mathbf{a}|\mathbf{s})}{\partial \theta} Q^\pi(\mathbf{s}, \mathbf{a}), \end{aligned}$$

where the second equality uses Lemma 13.3 and the third equality is since $\sum_{\mathbf{a}} \pi_{\theta+\delta\theta}(\mathbf{a}|\mathbf{s}) \mathcal{V}^{\pi_{\theta}}(\mathbf{s}) = \mathcal{V}^{\pi_{\theta}}(\mathbf{s})$, and $\mathcal{V}^{\pi_{\theta}}(\mathbf{s}) = \sum_{\mathbf{a}} \pi_{\theta}(\mathbf{a}|\mathbf{s}) \mathcal{Q}^{\pi_{\theta}}(\mathbf{s}, \mathbf{a})$. The fourth equality holds by definition of the derivative, and using Assumption 13.1. Note that Assumption 13.1 guarantees that π is continuous in θ , and therefore P^{π} is continuous in θ . By Proposition 13.1 we must have $\lim_{\delta\theta \rightarrow 0} d^{\pi_{\theta+\delta\theta}}(\mathbf{s}) = d^{\pi}(\mathbf{s})$. $\square$

The Policy Gradient Theorem gives us a way to compute the gradient: we can sample states from the distribution d^{π} using the policy π . We still need to resolve the sampling of the action. We are going to observe the outcome of only one action in state $\mathbf{s}$, but the theorem requires summing over all of them! In the following, we will slightly modify the theorem so that we will be able to use only the action $\mathbf{a}$ selected by the policy π , rather than summing over all actions.

Consider the following simple identity,

$$\nabla f(x) = f(x) \frac{\nabla f(x)}{f(x)} = f(x) \nabla \log f(x). \quad (13.3)$$

This implies that we can restate the Policy Gradient Theorem as the following corollary,

Corollary 13.5 (Policy Gradient Corollary). *Consider a random rollout from the policy $\mathbf{s}_0, \mathbf{a}_0, \mathbf{r}_0, \dots, \mathbf{s}_{\tau}, \mathbf{a}_{\tau}, \mathbf{r}_{\tau}$, where $\mathbf{s}_0 \sim \mu$, $\mathbf{a}_t \sim \pi(\cdot|\mathbf{s}_t, \theta)$, $\mathbf{s}_{t+1} \sim \mathcal{P}(\cdot|\mathbf{s}_t, \mathbf{a}_t)$, and τ is the termination time. We have*

$$\begin{aligned} \nabla J(\theta) &= \sum_{\mathbf{s} \in \mathcal{S}} d^{\pi}(\mathbf{s}) \sum_{\mathbf{a} \in \mathcal{A}} \pi(\mathbf{a}|\mathbf{s}) \mathcal{Q}^{\pi}(\mathbf{s}, \mathbf{a}) \nabla \log \pi(\mathbf{a}|\mathbf{s}) \\ &= \mathbb{E}^{\pi} \left[\sum_{t=0}^{\tau} \mathcal{Q}^{\pi}(\mathbf{s}_t, \mathbf{a}_t) \nabla \log \pi(\mathbf{a}_t|\mathbf{s}_t) \right]. \end{aligned}$$

Proof. The first equality is by the identity above, and the second is by definition of $d^{\pi}(\mathbf{s})$, similar to Proposition 13.2. $\square$

Note that in the above corollary, both the state $\mathbf{s}$ and action $\mathbf{a}$ are sampled using the policy π . This avoids the need to sum over all actions and leaves only the action selected by the policy.

We next provide some examples for the policy gradient theorem.

Example 13.2. *Consider an MDP with a single state $\mathbf{s}$ (which is also called a Multi-Arm Bandit, see Chapter 14). Assume we have only two actions, action $\mathbf{a}_1$ has expected reward $\mathbf{r}_1$ and action $\mathbf{a}_2$ has expected reward $\mathbf{r}_2$.*

The policy π is defined with a parameter $\theta = (\theta_1, \theta_2)$, where $\theta_i \in \mathbb{R}$. Given θ , the probability of action $\mathbf{a}_i$ is $p_i = e^{\theta_i} / (e^{\theta_1} + e^{\theta_2})$. We will also select a horizon of length one, i.e., $T = 1$. This implies that $Q^\pi(\mathbf{s}, \mathbf{a}_i) = \mathbf{r}_i$.

In this simple case, we can compute directly $J(\theta)$ and $\nabla J(\theta)$. The expected return is simply,

$$J(\theta) = p_1 \mathbf{r}_1 + p_2 \mathbf{r}_2 = \frac{e^{\theta_1}}{e^{\theta_1} + e^{\theta_2}} \mathbf{r}_1 + \frac{e^{\theta_2}}{e^{\theta_1} + e^{\theta_2}} \mathbf{r}_2$$

Note that $\frac{\partial}{\partial \theta_1} p_1 = p_1 - p_1^2 = p_1(1 - p_1)$ and $\frac{\partial}{\partial \theta_2} p_1 = -p_1 p_2 = -p_1(1 - p_1)$. The gradient is

$$\nabla J(\theta) = \mathbf{r}_1 \begin{pmatrix} p_1(1 - p_1) \\ -p_1(1 - p_1) \end{pmatrix} + \mathbf{r}_2 \begin{pmatrix} -p_1(1 - p_1) \\ p_1(1 - p_1) \end{pmatrix} = (\mathbf{r}_1 - \mathbf{r}_2) p_1(1 - p_1) \begin{pmatrix} +1 \\ -1 \end{pmatrix}.$$

Updating in the direction of the gradient, in the case that $\mathbf{r}_1 > \mathbf{r}_2$, would increase θ_1 and decrease θ_2 and eventually p_1 will converge to 1.

To apply the Policy gradient theorem we need to compute the gradient,

$$\nabla_{\theta} \pi(\mathbf{a}_1 | \mathbf{s}; \theta) = \nabla \begin{pmatrix} p_1 \\ 1 - p_1 \end{pmatrix} = \begin{pmatrix} p_1(1 - p_1) \\ -p_1(1 - p_1) \end{pmatrix}$$

and the policy gradient theorem gives us the same expression,

$$\nabla J(\theta) = \mathbf{r}_1 \nabla \pi(\mathbf{a}_1; \theta) + \mathbf{r}_2 \nabla \pi(\mathbf{a}_2; \theta) = \mathbf{r}_1 \begin{pmatrix} p_1(1 - p_1) \\ -p_1(1 - p_1) \end{pmatrix} + \mathbf{r}_2 \begin{pmatrix} -p_1(1 - p_1) \\ p_1(1 - p_1) \end{pmatrix},$$

where we used the fact that there is only a single state $\mathbf{s}$, and that $Q^\pi(\mathbf{s}, \mathbf{a}_i) = \mathbf{r}_i$.

Example 13.3. Consider the following deterministic MDP. We have states $\mathcal{S} = \{\mathbf{s}_0, \mathbf{s}_1, \mathbf{s}_2, \mathbf{s}_3\}$ and actions $\mathcal{A} = \{\mathbf{a}_0, \mathbf{a}_1\}$. We start at $\mathbf{s}_0$. Action $\mathbf{a}_0$ from any state leads to $\mathbf{s}_3$. Action $\mathbf{a}_1$ moves from $\mathbf{s}_0$ to $\mathbf{s}_1$, from $\mathbf{s}_1$ to $\mathbf{s}_2$, and from $\mathbf{s}_2$ to $\mathbf{s}_3$. All the rewards are zero except the terminal reward at $\mathbf{s}_2$, which is 1. The horizon is $T = 2$. This implies that the optimal policy performs action $\mathbf{a}_1$ in each state and has a return of 1.

We have a log-linear policy parameterized by $\theta \in \mathbb{R}^4$. In state $\mathbf{s}_0$, it selects action $\mathbf{a}_1$ with probability $p_1 = e^{\theta_1} / (e^{\theta_1} + e^{\theta_2})$, and in state $\mathbf{s}_1$ it selects action $\mathbf{a}_1$ with probability $p_2 = e^{\theta_3} / (e^{\theta_3} + e^{\theta_4})$.

For this simple MDP, we can specify the expected return $J(\theta) = p_1 p_2$. We can also compute the gradient and have

$$\nabla J(\theta) = \begin{pmatrix} p_1(1 - p_1)p_2 \\ -p_1(1 - p_1)p_2 \\ p_1 p_2(1 - p_2) \\ -p_1 p_2(1 - p_2) \end{pmatrix} = p_1 p_2 \begin{pmatrix} (1 - p_1) \\ -(1 - p_1) \\ (1 - p_2) \\ -(1 - p_2) \end{pmatrix}.$$

The policy gradient theorem will use the following ingredients. The $\mathcal{Q}^\pi$ is: $\mathcal{Q}^\pi(\mathbf{s}_0, \mathbf{a}_1) = p_2$, $\mathcal{Q}^\pi(\mathbf{s}_1, \mathbf{a}_1) = 1$ and all the other entries are zero. The weights of the states are $d^\pi(\mathbf{s}_0) = 1$, $d^\pi(\mathbf{s}_1) = p_1$, $d^\pi(\mathbf{s}_2) = p_1 p_2$ and $d^\pi(\mathbf{s}_3) = 2 - p_1 - p_1 p_2$. The gradient of the action in each state is:

$$\nabla \pi(\mathbf{a}_1 | \mathbf{s}_0; \theta) = p_1 \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix} - p_1^2 \begin{pmatrix} 1 \\ 0 \\ 0 \\ 0 \end{pmatrix} - p_1(1 - p_1) \begin{pmatrix} 0 \\ 1 \\ 0 \\ 0 \end{pmatrix} = p_1(1 - p_1) \begin{pmatrix} 1 \\ -1 \\ 0 \\ 0 \end{pmatrix}.$$

Similarly

$$\nabla \pi(\mathbf{a}_1 | \mathbf{s}_1; \theta) = p_2 \begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \end{pmatrix} - p_2^2 \begin{pmatrix} 0 \\ 0 \\ 1 \\ 0 \end{pmatrix} - p_2(1 - p_2) \begin{pmatrix} 0 \\ 0 \\ 0 \\ 1 \end{pmatrix} = p_2(1 - p_2) \begin{pmatrix} 0 \\ 0 \\ 1 \\ -1 \end{pmatrix}.$$

The policy gradient theorem states that the expected return gradient is

$$d^\pi(\mathbf{s}_0) \mathcal{Q}^\pi(\mathbf{s}_0, \mathbf{a}_1) \pi(\mathbf{a}_1 | \mathbf{s}_0; \theta) \nabla \log \pi(\mathbf{a}_1 | \mathbf{s}_0; \theta) + \\ d^\pi(\mathbf{s}_1) \mathcal{Q}^\pi(\mathbf{s}_1, \mathbf{a}_1) \pi(\mathbf{a}_1 | \mathbf{s}_1; \theta) \nabla \log \pi(\mathbf{a}_1 | \mathbf{s}_1; \theta),$$

where we dropped all the terms that evaluate to zero. Plugging in our values we have

$$p_2 p_1 (1 - p_1) \begin{pmatrix} 1 \\ -1 \\ 0 \\ 0 \end{pmatrix} + p_1 p_2 (1 - p_2) \begin{pmatrix} 0 \\ 0 \\ 1 \\ -1 \end{pmatrix} = p_1 p_2 \begin{pmatrix} (1 - p_1) \\ -(1 - p_1) \\ (1 - p_2) \\ -(1 - p_2) \end{pmatrix},$$

which is identical to $\nabla J(\theta)$.

Example 13.4. Again, consider an MDP with a single state $\mathbf{s}$ (which is also called a Multi-Arm Bandit, see Chapter 14), and the horizon is $\mathbf{T} = 1$. Consider a continuous action set $\mathcal{A} = \mathbb{R}$. The policy and reward are given as follows:

$$\mathbf{r}(\mathbf{a}) = \mathbf{a}, \\ \pi(\mathbf{a}) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(\mathbf{a} - \theta)^2}{2\sigma^2}\right),$$

where the parameter is $\theta \in \mathbb{R}$ and σ is fixed and known. As in Example 13.2, we have that $\mathcal{Q}^\pi(\mathbf{s}, \mathbf{a}) = \mathbf{a}$. Also, $J(\theta) = \mathbb{E}^\pi[\mathbf{a}] = \theta$, and thus $\nabla J(\theta) = 1$. Using Corollary

13.5, we calculate:

$$\begin{aligned}\nabla \log \pi(\mathbf{a}) &= \frac{\mathbf{a} - \theta}{\sigma^2}, \\ \nabla J(\theta) &= \mathbb{E}^\pi \left[\frac{\mathbf{a}(\mathbf{a} - \theta)}{\sigma^2} \right] = \frac{1}{\sigma^2} (\mathbb{E}^\pi[\mathbf{a}^2] - (\mathbb{E}^\pi[\mathbf{a}])^2) = 1.\end{aligned}$$

Note the intuitive interpretation of the policy gradient here: we average the difference of an action from the mean action $\mathbf{a} - \theta$ and the value it yields $\mathcal{Q}^\pi(\mathbf{s}, \mathbf{a}) = \mathbf{a}$. In this case, actions above the mean lead to higher reward, thereby ‘pushing’ the mean action θ to increase. Note that indeed the optimal value of θ here is infinite.

13.6 Policy Gradient Algorithms

The policy gradient theorem, and Corollary 13.5 provide a straightforward approach to estimating the policy gradient from sample rollouts: all we need to know is how to calculate $\nabla \log \pi(\mathbf{a}|\mathbf{s})$, and $\mathcal{Q}^\pi(\mathbf{s}, \mathbf{a})$. In the following, we show how to compute $\nabla \log \pi(\mathbf{a}|\mathbf{s})$ for several policy classes. Later, we shall discuss how to estimate $\mathcal{Q}^\pi(\mathbf{s}, \mathbf{a})$ and derive practical algorithms.

Log-linear policy For the log-linear policy class, we have

$$\nabla \log \pi(\mathbf{a}|\mathbf{s}; \theta) = \phi(\mathbf{s}, \mathbf{a}) - \sum_{\mathbf{a}'} \pi(\mathbf{a}'|\mathbf{s}; \theta) \phi(\mathbf{s}, \mathbf{a}').$$

Gaussian policy For the Gaussian policy class, we have

$$\nabla \log \pi(\mathbf{a}|\mathbf{s}; \theta) = \left(\frac{\mathbf{a} - \xi(\mathbf{s})}{\sigma^2} \right) \nabla \xi(\mathbf{s}).$$

Simplex policy For the Simplex policy class, we have

$$\nabla_{\theta_{\mathbf{s}, \mathbf{a}}} \log \pi(\mathbf{a}|\mathbf{s}; \theta) = \nabla \log \theta_{\mathbf{s}, \mathbf{a}} - \nabla \log \sum_b \theta_{\mathbf{s}, b} = \frac{1}{\theta_{\mathbf{s}, \mathbf{a}}} - \frac{1}{\sum_b \theta_{\mathbf{s}, b}},$$

and for $b' \neq \mathbf{a}$,

$$\nabla_{\theta_{\mathbf{s}, b'}} \log \pi(\mathbf{a}|\mathbf{s}; \theta) = -\nabla \log \sum_b \theta_{\mathbf{s}, b} = -\frac{1}{\sum_b \theta_{\mathbf{s}, b}}.$$

13.6.1 REINFORCE: Monte-Carlo updates

The REINFORCE algorithm uses Monte-Carlo updates to estimate $\mathcal{Q}^\pi(\mathbf{s}, \mathbf{a})$ in the policy gradient computation. Given a rollout $(\mathbf{s}_0, \mathbf{a}_0, \mathbf{r}_0, \mathbf{s}_1, \mathbf{a}_1, \mathbf{r}_1, \dots, \mathbf{s}_\tau, \mathbf{a}_\tau, \mathbf{r}_\tau)$ from the policy, note that

$$\mathcal{Q}^\pi(\mathbf{s}_t, \mathbf{a}_t) = \mathbb{E}^\pi \left[\sum_{i=t}^{\tau} \mathbf{r}_i \right].$$

Therefore, let $R_{t:\tau} = \sum_{i=t}^{\tau} \mathbf{r}_i$, and at each iteration REINFORCE samples a rollout and updates the policy in the direction $\sum_{t=0}^{\tau} R_{t:\tau} \nabla \log \pi(\mathbf{a}_t | \mathbf{s}_t; \theta)$.

Algorithm 36 REINFORCE

- 1: **Input** step size α
- 2: **Initialize** θ_0 arbitrarily
- 3: **For** $j = 0, 1, 2, \dots$
- 4: Sample rollout $(\mathbf{s}_0, \mathbf{a}_0, \mathbf{r}_0, \dots, \mathbf{s}_\tau, \mathbf{a}_\tau, \mathbf{r}_\tau)$ using policy π_{θ_j} .
- 5: Set $R_{t:\tau} = \sum_{i=t}^{\tau} \mathbf{r}_i$
- 6: Update policy parameters:

$$\theta_{j+1} = \theta_j + \alpha \sum_{t=0}^{\tau} R_{t:\tau} \nabla \log \pi(\mathbf{a}_t | \mathbf{s}_t; \theta_j)$$

Remark 13.1. *The REINFORCE algorithm can be intuitively understood as a trial and error process. Notice that the update*

$$\underbrace{\alpha}_{\text{step size}} \times \sum_{t=0}^{\tau} \underbrace{R_{t:\tau}}_{\text{weight}} \times \underbrace{\nabla \log \pi(\mathbf{a}_t | \mathbf{s}_t; \theta_j)}_{\text{log likelihood of sampled action}}$$

can be understood as a step in maximizing the likelihood of the actions in the rollout multiplied by their return. Thus, actions that were “good” – yielded high return – would be reinforced more than actions that yielded low return.

Baseline function

One caveat with the REINFORCE algorithm as stated above, is that it tends to have high variance in estimating the policy gradient, which in practice leads to slow convergence. A common and elegant technique to reduce variance is to add to REINFORCE a baseline function, also termed a ‘control variate’.

The baseline function $b(\mathbf{s})$ can depend in an arbitrary way on the state, but does not depend on the action. The main observation would be that we can add or subtract any such function from our $Q^\pi(\mathbf{s}, \mathbf{a})$ estimate, and it will still be unbiased. This follows since

$$\sum_{\mathbf{a}} b(\mathbf{s}) \nabla \pi(\mathbf{a}|\mathbf{s}; \theta) = b(\mathbf{s}) \nabla \sum_{\mathbf{a}} \pi(\mathbf{a}|\mathbf{s}; \theta) = b(\mathbf{s}) \nabla 1 = 0. \quad (13.4)$$

Given this, we can restate the Policy Gradient Theorem as,

$$\nabla J(\theta) = \sum_{\mathbf{s} \in \mathcal{S}} d^\pi(\mathbf{s}) \sum_{\mathbf{a} \in \mathcal{A}} \pi(\mathbf{a}|\mathbf{s}) (Q^\pi(\mathbf{s}, \mathbf{a}) - b(\mathbf{s})) \nabla \log \pi(\mathbf{a}|\mathbf{s}).$$

This gives us a degree of freedom to select $b(\mathbf{s})$. Note that by setting $b(\mathbf{s}) = 0$ we get the original theorem. However, we can select $b(\mathbf{s})$ in a way that makes estimating the gradient easier, for example, by reducing the variance of the estimator.

Proposition 13.6. *The variance of $\nabla J(\theta)$ is minimized by $b(\mathbf{s}) = \frac{\sum_{\mathbf{a} \in \mathcal{A}} \pi(\mathbf{a}|\mathbf{s}) (\nabla \log \pi(\mathbf{a}|\mathbf{s}))^2 Q^\pi(\mathbf{s}, \mathbf{a})}{\sum_{\mathbf{a} \in \mathcal{A}} \pi(\mathbf{a}|\mathbf{s}) (\nabla \log \pi(\mathbf{a}|\mathbf{s}))^2}$.*

Proof. Consider the random variable $(Q^\pi(\mathbf{s}, \mathbf{a}) - b(\mathbf{s})) \nabla \log \pi(\mathbf{a}|\mathbf{s})$. Its expectation is $\nabla J(\theta)$, for any $b(\mathbf{s})$. Its variance is

$$\sum_{\mathbf{s} \in \mathcal{S}} d^\pi(\mathbf{s}) \sum_{\mathbf{a} \in \mathcal{A}} \pi(\mathbf{a}|\mathbf{s}) ((Q^\pi(\mathbf{s}, \mathbf{a}) - b(\mathbf{s})) \nabla \log \pi(\mathbf{a}|\mathbf{s}))^2 - (\nabla J(\theta))^2.$$

Taking a gradient with respect to $b(\mathbf{s})$ and equating to 0 gives:

$$\begin{aligned} 0 &= d^\pi(\mathbf{s}) \sum_{\mathbf{a} \in \mathcal{A}} \pi(\mathbf{a}|\mathbf{s}) (\nabla \log \pi(\mathbf{a}|\mathbf{s}))^2 2(b(\mathbf{s}) - Q^\pi(\mathbf{s}, \mathbf{a})) \\ b(\mathbf{s}) &= \frac{\sum_{\mathbf{a} \in \mathcal{A}} \pi(\mathbf{a}|\mathbf{s}) (\nabla \log \pi(\mathbf{a}|\mathbf{s}))^2 Q^\pi(\mathbf{s}, \mathbf{a})}{\sum_{\mathbf{a} \in \mathcal{A}} \pi(\mathbf{a}|\mathbf{s}) (\nabla \log \pi(\mathbf{a}|\mathbf{s}))^2}. \end{aligned}$$

□

In many cases it is reasonable to use for $b(\mathbf{s})$ the value of the state, i.e., $b(\mathbf{s}) = \mathcal{V}^\pi(\mathbf{s})$. If we assume that the magnitude of the gradients $|\nabla \log \pi(\mathbf{a}|\mathbf{s})|$ is similar for all actions $\mathbf{a} \in \mathcal{A}$, the variance is minimized by $b(\mathbf{s}) = \mathbb{E}^\pi[Q^\pi(\mathbf{s}, \mathbf{a})] = \mathcal{V}^\pi(\mathbf{s})$. The following example shows an explicit case for this.

Example 13.5. Consider the Multi-Arm Bandit setting of Example 13.4, where we recall that $\mathbf{r}(\mathbf{a}) = \mathbf{a}$, $\pi_\theta(\mathbf{a}) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp(-\frac{(\mathbf{a}-\theta)^2}{2\sigma^2})$. Find a fixed baseline b that minimizes the variance of the policy gradient estimate.

The policy gradient formula in this case is:

$$\nabla_\theta J(\theta) = \mathbb{E} \left[\frac{(\mathbf{a} - b)(\mathbf{a} - \theta)}{\sigma^2} \right] = 1,$$

and we can calculate the variance

$$\begin{aligned} \frac{1}{\sigma^4} \text{Var}[(\mathbf{a} - b)(\mathbf{a} - \theta)] &= \frac{1}{\sigma^4} \mathbb{E} [((\mathbf{a} - b)(\mathbf{a} - \theta))^2 - 1] \\ &= \frac{1}{\sigma^4} \mathbb{E} [((\mathbf{a} - \theta)(\mathbf{a} - \theta) + (\theta - b)(\mathbf{a} - \theta))^2 - 1] \\ &= \frac{1}{\sigma^4} \mathbb{E} [(\mathbf{a} - \theta)^4 + 2(\theta - b)(\mathbf{a} - \theta)^3 + (\theta - b)^2(\mathbf{a} - \theta)^2 - 1] \\ &= \frac{1}{\sigma^4} \mathbb{E} [(\mathbf{a} - \theta)^4 + (\theta - b)^2(\mathbf{a} - \theta)^2 - 1], \end{aligned}$$

which is minimized for $b = \theta = \mathcal{V}^\pi(\mathbf{s})$.

We are left with the challenge of approximating $\mathcal{V}^\pi(\mathbf{s})$. On the one hand this is part of the learning. On the other hand we have developed tools to address this in the previous chapter on value function approximation (Chapter 12). We can use $\mathcal{V}^\pi(\mathbf{s}) \approx \hat{\mathcal{V}}(\mathbf{s}; \mathbf{w}) = b(\mathbf{s})$. The good news is that any $b(\mathbf{s})$ will keep the estimator unbiased, so we do not depend on $\hat{\mathcal{V}}(\mathbf{s}; \mathbf{w})$ to be unbiased.

We can now describe the REINFORCE algorithm with baseline function. We will use a Monte-Carlo sampling to estimate $\mathcal{V}^\pi(\mathbf{s})$ using a class of value approximation functions $\hat{\mathcal{V}}(\cdot; \mathbf{w})$ and this will define our baseline function $b(\mathbf{s})$.¹ Note that now we have two parameter vectors: θ for the policy, and $\mathbf{w}$ for the value function.

¹We implicitly assume that no state appears twice in the trajectory, and therefore the ‘every visit’ and ‘first visit’ Monte-Carlo updates are equivalent.

Algorithm 37 REINFORCE with Value Baseline

- 1: **Input** step sizes α, β
- 2: **Initialize** $\theta_0, \mathbf{w}_0$ arbitrarily
- 3: **For** $j = 0, 1, 2, \dots$
- 4: Sample rollout $(\mathbf{s}_0, \mathbf{a}_0, \mathbf{r}_0, \dots, \mathbf{s}_\tau, \mathbf{a}_\tau, \mathbf{r}_\tau)$ using policy π_{θ_j} .
- 5: Set $R_{\mathbf{t}:\tau} = \sum_{i=\mathbf{t}}^{\tau} \mathbf{r}_i$
- 6: Set $\Gamma_{\mathbf{t}} = R_{\mathbf{t}:\tau} - \hat{\mathcal{V}}(\mathbf{s}_{\mathbf{t}}; \mathbf{w}_j)$
- 7: Update policy parameters:

$$\theta_{j+1} = \theta_j + \alpha \sum_{\mathbf{t}=0}^{\tau} \Gamma_{\mathbf{t}} \nabla_{\theta} \log \pi(\mathbf{a}_{\mathbf{t}} | \mathbf{s}_{\mathbf{t}}; \theta_j).$$

- 8: Update value parameters:

$$\mathbf{w}_{j+1} = \mathbf{w}_j + \beta \sum_{\mathbf{t}=0}^{\tau} \Gamma_{\mathbf{t}} \nabla_{\mathbf{w}} \hat{\mathcal{V}}(\mathbf{s}_{\mathbf{t}}; \mathbf{w}_j).$$

Note that the update for θ follows the policy gradient theorem with a baseline $\hat{\mathcal{V}}(\mathbf{s}_{\mathbf{t}}; \mathbf{w})$, and the update for $\mathbf{w}$ is a stochastic gradient descent on the mean squared error with step size β .

13.6.2 TD Updates and Compatible Value Functions

In the REINFORCE algorithm above, the term $R_{\mathbf{t}:\tau}$ can be a very noisy estimate of $\mathcal{Q}^{\pi}(\mathbf{s}_{\mathbf{t}}, \mathbf{a}_{\mathbf{t}})$, due to the stochasticity of the state transitions and action selections at times $\mathbf{t} : \tau$. A natural alternative that reduces this variance is to use bootstrapping, i.e., $\mathbf{r}_{\mathbf{t}} + \mathcal{V}^{\pi}(\mathbf{s}_{\mathbf{t}+1})$, which variance is only due to the stochasticity of the state transition at time $\mathbf{t}$. Of course, we cannot expect to have an accurate estimate of $\mathcal{V}^{\pi}(\mathbf{s}_{\mathbf{t}+1})$ and must approximate it which, as we shall see, may induce a bias. In this section we investigate this approach, also known as the actor-critic algorithm.

The critic maintains an approximate value function $\hat{\mathcal{V}}(\mathbf{s}; \mathbf{w})$. For each time $\mathbf{t}$ it defines the TD error to be $\Delta_{\mathbf{t}} = \mathbf{r}_{\mathbf{t}} + \hat{\mathcal{V}}(\mathbf{s}_{\mathbf{t}+1}; \mathbf{w}) - \hat{\mathcal{V}}(\mathbf{s}_{\mathbf{t}}; \mathbf{w})$. The critic's update of the value function parameters, following the TD(0) algorithm will be $\mathbf{w} := \mathbf{w} + \alpha \Delta_{\mathbf{t}} \nabla_{\mathbf{w}} \hat{\mathcal{V}}(\mathbf{s}_{\mathbf{t}}; \mathbf{w})$. The critic sends the TD error to the actor, which maintains a policy π parameterized by θ . Given a TD error $\Delta_{\mathbf{t}}$, the actor updates the policy parameters by $\theta := \theta + \beta \Delta_{\mathbf{t}} \nabla_{\theta} \log \pi(\mathbf{a}_{\mathbf{t}} | \mathbf{s}_{\mathbf{t}}; \theta)$, where β is a learning rate (possibly different from α).

Algorithm 38 ACTOR-CRITIC

- 1: **Input** step sizes α, β
- 2: **Initialize** $\theta_0, \mathbf{w}_0$ arbitrarily
- 3: **For** $j = 0, 1, 2, \dots$
- 4: Sample rollout $(\mathbf{s}_0, \mathbf{a}_0, \mathbf{r}_0, \dots, \mathbf{s}_\tau, \mathbf{a}_\tau, \mathbf{r}_\tau)$ using policy π_{θ_j} .
- 5: Set $\Delta_{\mathbf{t}} = \mathbf{r}_{\mathbf{t}} + \widehat{\mathcal{V}}(\mathbf{s}_{\mathbf{t}+1}; \mathbf{w}_j) - \widehat{\mathcal{V}}(\mathbf{s}_{\mathbf{t}}; \mathbf{w}_j), \quad \forall \mathbf{t} = 0, \dots, \tau - 1$
- 6: ACTOR: Update policy parameters:

$$\theta_{j+1} = \theta_j + \alpha \sum_{\mathbf{t}=0}^{\tau-1} \Delta_{\mathbf{t}} \nabla_{\theta} \log \pi(\mathbf{a}_{\mathbf{t}} | \mathbf{s}_{\mathbf{t}}; \theta_j).$$

- 7: CRITIC: Update value parameters:

$$\mathbf{w}_{j+1} = \mathbf{w}_j + \beta \sum_{\mathbf{t}=0}^{\tau-1} \Delta_{\mathbf{t}} \nabla_{\mathbf{w}} \widehat{\mathcal{V}}(\mathbf{s}_{\mathbf{t}}; \mathbf{w}_j).$$

Note that in contrast to REINFORCE that used the function approximation to estimate only a baseline, here our estimate of $\mathbf{r}_{\mathbf{t}} + \widehat{\mathcal{V}}(\mathbf{s}_{\mathbf{t}+1}; \mathbf{w})$ is used to estimate $\mathcal{Q}^{\pi}(\mathbf{s}_{\mathbf{t}}, \mathbf{a}_{\mathbf{t}})$ directly, and therefore might introduce a bias due to an inaccurate approximation.

Interestingly, there is a special case for which it is guaranteed that we will not have such a bias. The following theorem establishes this result; for simplicity we use an approximation of $\widehat{\mathcal{Q}}(\mathbf{s}_{\mathbf{t}}, \mathbf{a}_{\mathbf{t}}; \mathbf{w})$ instead of $\mathbf{r}_{\mathbf{t}} + \widehat{\mathcal{V}}(\mathbf{s}_{\mathbf{t}+1}; \mathbf{w})$.

Let the expected square error of $\mathbf{w}$ be

$$SE(\mathbf{w}) = \frac{1}{2} \mathbb{E}^{\pi}[(\mathcal{Q}^{\pi}(\mathbf{s}, \mathbf{a}) - \widehat{\mathcal{Q}}(\mathbf{s}, \mathbf{a}; \mathbf{w}))^2].$$

A value function is *compatible* if,

$$\nabla_{\mathbf{w}} \widehat{\mathcal{Q}}(\mathbf{s}, \mathbf{a}; \mathbf{w}) = \nabla_{\theta} \log \pi(\mathbf{a} | \mathbf{s}; \theta).$$

Theorem 13.7. *Assume that $\widehat{\mathcal{Q}}$ is compatible and $\mathbf{w}$ minimizes $SE(\mathbf{w})$, then,*

$$\nabla_{\theta} J(\theta) = \sum_{\mathbf{t}=1}^{\tau} \mathbb{E}^{\pi}[\widehat{\mathcal{Q}}(\mathbf{s}_{\mathbf{t}}, \mathbf{a}_{\mathbf{t}}; \mathbf{w}) \nabla \log \pi(\mathbf{a}_{\mathbf{t}} | \mathbf{s}_{\mathbf{t}}; \theta)].$$

Proof. Since $\mathbf{w}$ minimizes $SE(\mathbf{w})$ we have

$$\begin{aligned} 0 &= \nabla_{\mathbf{w}} SE(\mathbf{w}) \\ &= \nabla_{\mathbf{w}} \mathbb{E}^{\pi}[(\mathcal{Q}^{\pi}(\mathbf{s}, \mathbf{a}) - \widehat{\mathcal{Q}}(\mathbf{s}, \mathbf{a}; \mathbf{w}))^2] \\ &= \mathbb{E}^{\pi}[(\mathcal{Q}^{\pi}(\mathbf{s}, \mathbf{a}) - \widehat{\mathcal{Q}}(\mathbf{s}, \mathbf{a}; \mathbf{w})) \nabla_{\mathbf{w}} \widehat{\mathcal{Q}}(\mathbf{s}, \mathbf{a}; \mathbf{w})]. \end{aligned}$$

Since $\widehat{\mathcal{Q}}$ is compatible, we have $\nabla_{\mathbf{w}} \widehat{\mathcal{Q}}(\mathbf{s}, \mathbf{a}; \mathbf{w}) = \nabla_{\theta} \log \pi(\mathbf{a}|\mathbf{s}; \theta)$ which implies,

$$0 = \mathbb{E}^{\pi}[(\mathcal{Q}^{\pi}(\mathbf{s}, \mathbf{a}) - \widehat{\mathcal{Q}}(\mathbf{s}, \mathbf{a}; \mathbf{w})) \nabla_{\theta} \log \pi(\mathbf{a}|\mathbf{s}; \theta)],$$

and have

$$\mathbb{E}^{\pi}[\mathcal{Q}^{\pi}(\mathbf{s}, \mathbf{a}) \nabla_{\theta} \log \pi(\mathbf{a}|\mathbf{s}; \theta)] = \mathbb{E}^{\pi}[\widehat{\mathcal{Q}}(\mathbf{s}, \mathbf{a}; \mathbf{w}) \nabla_{\theta} \log \pi(\mathbf{a}|\mathbf{s}; \theta)].$$

This implies that by substituting $\mathcal{Q}$ in the policy gradient theorem we have

$$\nabla_{\theta} J(\theta) = \sum_{\mathbf{t}=1}^{\tau} \mathbb{E}^{\pi}[\widehat{\mathcal{Q}}(\mathbf{s}, \mathbf{a}; \mathbf{w}) \nabla \log \pi(\mathbf{a}|\mathbf{s}; \theta)].$$

□

For non-compatible function approximation, we can combine the zero-bias, high-variance REINFORCE update with the biased but low-variance ACTOR-CRITIC update using the idea of TD with multi-step lookahead (see Chapter 11.5.4). Recall that we defined multi-step TD as:

$$\begin{aligned} \Delta_{\mathbf{t}}^{(0)} &= \mathbf{r}_{\mathbf{t}} + \widehat{\mathcal{V}}(\mathbf{s}_{\mathbf{t}+1}; \mathbf{w}) - \widehat{\mathcal{V}}(\mathbf{s}_{\mathbf{t}}; \mathbf{w}), \\ \Delta_{\mathbf{t}}^{(1)} &= \mathbf{r}_{\mathbf{t}} + \mathbf{r}_{\mathbf{t}+1} + \widehat{\mathcal{V}}(\mathbf{s}_{\mathbf{t}+2}; \mathbf{w}) - \widehat{\mathcal{V}}(\mathbf{s}_{\mathbf{t}}; \mathbf{w}), \\ &\dots \\ \Delta_{\mathbf{t}}^{(n)} &= \mathbf{r}_{\mathbf{t}} + \mathbf{r}_{\mathbf{t}+1} + \dots + \mathbf{r}_{\mathbf{t}+n-1} + \widehat{\mathcal{V}}(\mathbf{s}_{\mathbf{t}+n}; \mathbf{w}) - \widehat{\mathcal{V}}(\mathbf{s}_{\mathbf{t}}; \mathbf{w}), \end{aligned}$$

and the TD(λ) update in this case can be defined as $\Delta_{\mathbf{t}}^{\lambda} = \frac{1}{Z_{\mathbf{t}}} \sum_{n=1}^{\tau-\mathbf{t}} \lambda^{n-1} \Delta_{\mathbf{t}}^{(n)}$, where the normalizing constant $Z_{\mathbf{t}} = \sum_{n=1}^{\tau-\mathbf{t}} \lambda^{n-1}$.

Note that for $\lambda = 0$ we obtain the standard ACTOR-CRITIC update above, while for $\lambda = 1$ the update puts almost all of the weight on $\Delta_{\mathbf{t}}^{(n)}$, which is equivalent to REINFORCE (since we consider an episodic MDP and not a discounted setting, the horizon is not infinite and therefore the equivalence of TD(1) to REINFORCE is only an approximation). By choosing λ between 0 and 1 we can balance between bias and variance. Using $\Delta_{\mathbf{t}}^{\lambda}$ instead of $\Delta_{\mathbf{t}}$ in the actor update (line 6 in Algorithm 38) is also known as *generalized advantage estimation*.

13.7 Convergence of Policy Gradient

As our policy optimization in this chapter is based on gradient descent, it is important to understand whether it converges, and what it converges to. As illustrated in Figure 13.2, we can only expect gradient descent to converge to a globally optimal solution for functions that do not have sub-optimal local minima. The policy parametrization itself may induce local optima, and in this case there is no reason to expect convergence to a globally optimal policy. However, let us consider the case where the policy parameterization is expressive enough to not directly add local minima to the loss landscape, such as the simplex policy structure above. Will policy gradient converge to a globally optimal policy in this case?

Convex functions do not have local minima. However, as the following example shows, MDPs are not necessarily convex (or concave) in the policy, regardless of the policy parameterization.

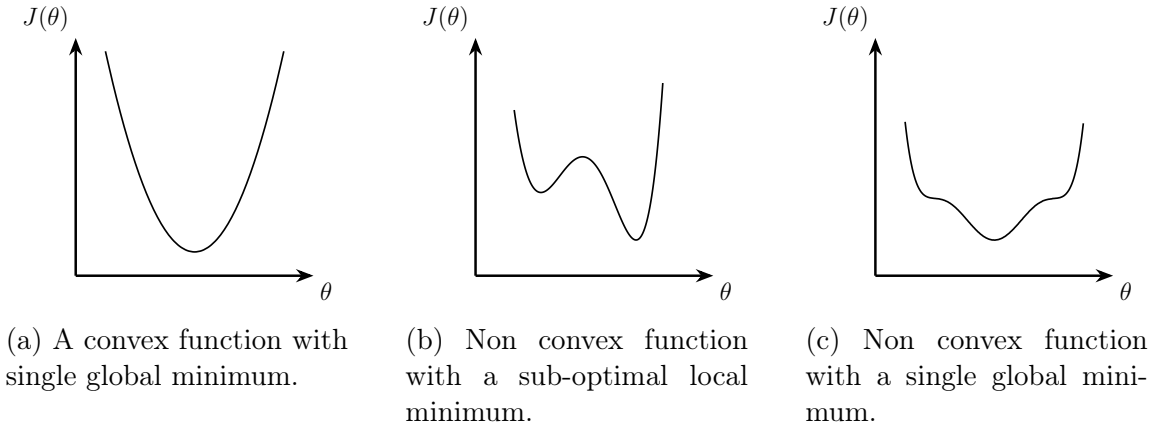


Figure 13.2: Gradient descent with a proper step size will converge to a global optimum in (a) and (c), but not in (b).

Example 13.6. A convex function $f(x)$ satisfies $f(\lambda x_1 + (1 - \lambda)x_2) \leq \lambda f(x_1) + (1 - \lambda)f(x_2)$. A concave function satisfies $f(\lambda x_1 + (1 - \lambda)x_2) \geq \lambda f(x_1) + (1 - \lambda)f(x_2)$. We will show that MDPs are not necessarily convex or concave in the policy.

Consider an MDP with two states $\mathbf{s}_1, \mathbf{s}_2$. In $\mathbf{s}_1$, taking action $\mathbf{a}_1$ transitions to $\mathbf{s}_2$ with 0 reward, while action $\mathbf{a}_2$ terminates with 0 reward. In state $\mathbf{s}_2$, taking action $\mathbf{a}_1$ terminates with reward 0, and taking action $\mathbf{a}_2$ terminates with reward 10. Consider two policies: π_1 chooses $\mathbf{a}_1$ in both states, and π_2 chooses $\mathbf{a}_2$ in both states. We have that $\mathcal{V}^{\pi_1}(\mathbf{s}_1) = 0$ and $\mathcal{V}^{\pi_2}(\mathbf{s}_1) = 0$. Now, consider the policy $\pi_\lambda(\mathbf{a}|\mathbf{s}) = \lambda\pi_1(\mathbf{a}|\mathbf{s}) +$

$(1 - \lambda)\pi_2(\mathbf{a}|\mathbf{s})$. For $\lambda = 0.5$ we have that $\mathcal{V}^{\pi_\lambda}(\mathbf{s}_1) = 2.5 > \lambda\mathcal{V}^{\pi_1}(\mathbf{s}_1) + (1 - \lambda)\mathcal{V}^{\pi_2}(\mathbf{s}_1)$, and therefore the MDP is not convex. By changing the rewards in the example to be their negatives, we can similarly show that the MDP is not concave.

Remark 13.2. Note that the way we combined policies in Example 13.6 is by combining the action probabilities at every state. This is required for establishing convexity of the value in the policy. Perhaps a more intuitive way of combining two policies is by selecting which policy to run at the beginning of an episode, and using only that policy throughout the episode. For such a non-Markovian policy, the expected value will simply be the average of the values of the two policies.

Remark 13.3. From the linear programming formulation in Chapter 8.3, we know that the value is linear (and thereby convex) in the state-action frequencies. While a policy can be inferred from state-action frequencies, this mapping is non-linear, and as the example above shows, renders the mapping from policy to value not necessarily convex.

Following Example 13.6, we should not immediately expect policy gradient algorithms to converge to a globally optimal policy. Interestingly, in the following we shall show that nevertheless, for the simplex policy there are no local optima that are not globally optimal.

Before we show this, however, we must handle a delicate technical issue. The simplex policy is only defined for $\theta_{\mathbf{s},\mathbf{a}} \geq 0$. What happens if some $\theta_{\mathbf{s},\mathbf{a}} = 0$ and $\frac{\partial J(\theta)}{\partial \theta_{\mathbf{s},\mathbf{a}}} < 0$? We shall assume that in this case, the policy gradient algorithm will maintain $\theta_{\mathbf{s},\mathbf{a}} = 0$ (this is in fact a special case of *projected* gradient descent [12]). We can therefore consider a modified gradient at $\theta_{\mathbf{s},\mathbf{a}} = 0$:

$$\left. \frac{\partial \tilde{J}(\theta)}{\partial \theta_{\mathbf{s},\mathbf{a}}} \right|_{\theta_{\mathbf{s},\mathbf{a}}=0} = \max \left\{ 0, \frac{\partial J(\theta)}{\partial \theta_{\mathbf{s},\mathbf{a}}} \right\}, \quad \left. \frac{\partial \tilde{J}(\theta)}{\partial \theta_{\mathbf{s},\mathbf{a}}} \right|_{\theta_{\mathbf{s},\mathbf{a}} \neq 0} = \frac{\partial J(\theta)}{\partial \theta_{\mathbf{s},\mathbf{a}}}.$$

We shall make a further assumption that $d^\pi(\mathbf{s}) > 0$ for all $\mathbf{s}, \pi$. To understand why this is necessary, consider an initial policy π_0 that does not visit a particular state $\mathbf{s}$ at all, and therefore $d^{\pi_0}(\mathbf{s}) = 0$. From the policy gradient theorem, we will have that $\frac{\partial J(\theta)}{\partial \theta_{\mathbf{s},\mathbf{a}}} = 0$, and therefore the policy at $\mathbf{s}$ will not improve. If the optimal policy in other states does not induce a transition to $\mathbf{s}$, we cannot expect convergence to optimal policy in $\mathbf{s}$. In other words, the policy must *explore* enough to cover the state space. Furthermore, for simplicity, we shall assume that the optimal policy is unique.

Let us now calculate the policy gradient for the simplex policy.

$$\frac{\partial \pi(\mathbf{a}'|\mathbf{s})}{\partial \theta_{\mathbf{s},\mathbf{a}}} = \begin{cases} \frac{\sum_{\mathbf{a}''} \theta_{\mathbf{s},\mathbf{a}''} - \theta_{\mathbf{s},\mathbf{a}'}}{(\sum_{\mathbf{a}''} \theta_{\mathbf{s},\mathbf{a}''})^2} & \text{if } \mathbf{a}' = \mathbf{a}, \\ \frac{-\theta_{\mathbf{s},\mathbf{a}'}}{(\sum_{\mathbf{a}''} \theta_{\mathbf{s},\mathbf{a}''})^2} & \text{if } \mathbf{a}' \neq \mathbf{a}. \end{cases}$$

Using the policy gradient theorem,

$$\begin{aligned} \frac{\partial J(\theta)}{\partial \theta_{\mathbf{s},\mathbf{a}}} &= d^\pi(\mathbf{s}) \sum_{\mathbf{a}'} \frac{\partial \pi(\mathbf{a}'|\mathbf{s})}{\partial \theta_{\mathbf{s},\mathbf{a}}} Q^\pi(\mathbf{s}, \mathbf{a}') \\ &= \frac{d^\pi(\mathbf{s})}{(\sum_{\mathbf{a}''} \theta_{\mathbf{s},\mathbf{a}''})^2} \sum_{\mathbf{a}'} (Q^\pi(\mathbf{s}, \mathbf{a}) - Q^\pi(\mathbf{s}, \mathbf{a}')) \theta_{\mathbf{s},\mathbf{a}'} \\ &= \frac{d^\pi(\mathbf{s})}{\sum_{\mathbf{a}''} \theta_{\mathbf{s},\mathbf{a}''}} \sum_{\mathbf{a}'} (Q^\pi(\mathbf{s}, \mathbf{a}) - Q^\pi(\mathbf{s}, \mathbf{a}')) \pi(\mathbf{a}'|\mathbf{s}) \\ &= \frac{d^\pi(\mathbf{s})}{\sum_{\mathbf{a}''} \theta_{\mathbf{s},\mathbf{a}''}} (Q^\pi(\mathbf{s}, \mathbf{a}) - \mathcal{V}^\pi(\mathbf{s})). \end{aligned}$$

Now, assume that π is not optimal, therefore there exists some $\mathbf{s}$ for which $\max_{\mathbf{a}} Q^\pi(\mathbf{s}, \mathbf{a}) > \mathcal{V}^\pi(\mathbf{s})$ (otherwise, $\mathcal{V}^\pi$ would satisfy the Bellman optimality equation and would therefore be optimal). In this case, we have that $\frac{\partial J(\theta)}{\partial \theta_{\mathbf{s},\mathbf{a}}} > 0$ and therefore θ is not a local optimum.

Lastly, we should verify that the optimal policy π^* is indeed a global optimum. The unique optimal policy is deterministic, and satisfies

$$\pi^*(\mathbf{a}|\mathbf{s}) = \begin{cases} 1 & \text{if } Q^{\pi^*}(\mathbf{s}, \mathbf{a}) = \mathcal{V}^{\pi^*}(\mathbf{s}), \\ 0 & \text{else.} \end{cases}$$

Consider any θ^* such that for all $\mathbf{s}, \mathbf{a}$ satisfies

$$\theta_{\mathbf{s},\mathbf{a}}^* = \begin{cases} > 0 & \text{if } Q^{\pi^*}(\mathbf{s}, \mathbf{a}) = \mathcal{V}^{\pi^*}(\mathbf{s}), \\ 0 & \text{else.} \end{cases}$$

By the above, we have that for the optimal action $\frac{\partial J(\theta)}{\partial \theta_{\mathbf{s},\mathbf{a}}} = 0$, and for non-optimal actions $Q^{\pi^*}(\mathbf{s}, \mathbf{a}) - \mathcal{V}^{\pi^*}(\mathbf{s}) < 0$, therefore, $\frac{\partial J(\theta)}{\partial \theta_{\mathbf{s},\mathbf{a}}} < 0$ and $\frac{\partial \tilde{J}(\theta)}{\partial \theta_{\mathbf{s},\mathbf{a}}} = 0$.

13.8 Proximal Policy Optimization

Recall our discussion about the policy difference lemma: if the difference $\pi' - \pi$ is ‘small’, then the difference in the state visitation frequencies $d^{\pi'} - d^\pi$ would also be

‘small’, allowing us to safely replace $d^{\pi'}$ in the right hand side of Eq. 13.2 with d^{π} . The Proximal Policy Optimization (PPO) algorithm is a popular heuristic that takes this approach, and has proved to perform very well empirically.

To simplify our notation we write the advantage function $\mathcal{A}^{\pi}(\mathbf{s}, \mathbf{a}) = \mathcal{Q}^{\pi}(\mathbf{s}, \mathbf{a}) - \mathcal{V}^{\pi}(\mathbf{s})$. The idea is to maximize the policy that leads to policy improvement

$$\max_{\pi' \in \Pi} \sum_{\mathbf{s}} d^{\pi'}(\mathbf{s}) \sum_{\mathbf{a}} \pi'(\mathbf{a}|\mathbf{s}) \mathcal{A}^{\pi}(\mathbf{s}, \mathbf{a}),$$

by replacing $d^{\pi'}$ with the visitation frequencies of the current policy d^{π} , and performing the search over a limited set of policies Π that is similar to π . The main trick in PPO is that this constrained optimization can be done implicitly, by maximizing the following objective:

$$\text{PPO}(\pi) = \max_{\pi'} \sum_{\mathbf{s}} d^{\pi}(\mathbf{s}) \sum_{\mathbf{a}} \pi(\mathbf{a}|\mathbf{s}) \min \left\{ \frac{\pi'(\mathbf{a}|\mathbf{s})}{\pi(\mathbf{a}|\mathbf{s})} \mathcal{A}^{\pi}(\mathbf{s}, \mathbf{a}), \text{clip} \left(\frac{\pi'(\mathbf{a}|\mathbf{s})}{\pi(\mathbf{a}|\mathbf{s})}, 1 - \epsilon, 1 + \epsilon \right) \mathcal{A}^{\pi}(\mathbf{s}, \mathbf{a}) \right\}, \quad (13.5)$$

where $\text{clip}(x, x_{\min}, x_{\max}) = \min\{\max\{x, x_{\min}\}, x_{\max}\}$, and ϵ is some small constant. Intuitively, the clipping in this objective prevents the ratio between the new policy $\pi'(\mathbf{a}|\mathbf{s})$ and the previous policy $\pi(\mathbf{a}|\mathbf{s})$ to grow larger than $1 + \epsilon$, assuring that maximizing the objective indeed leads to an improved policy.

To understand exactly how this works, consider the following two cases:

1. $\mathcal{A}^{\pi}(\mathbf{s}, \mathbf{a}) > 0$: in this case, the max over π' would drive to maximize $\pi'(\mathbf{a}|\mathbf{s})$, but the min in the objective would limit $\pi'(\mathbf{a}|\mathbf{s})$ to at most $(1 + \epsilon) \pi(\mathbf{a}|\mathbf{s})$.
2. $\mathcal{A}^{\pi}(\mathbf{s}, \mathbf{a}) < 0$: in this case, the max over π' would drive to minimize $\pi'(\mathbf{a}|\mathbf{s})$ to 0, but the min in the objective would limit $\pi'(\mathbf{a}|\mathbf{s})$ to be no smaller than $(1 - \epsilon) \pi(\mathbf{a}|\mathbf{s})$.

To optimize the PPO objective using a sample rollout, we let $\hat{\mathcal{A}}_{\mathbf{t}}$ denote an estimate of the advantage at state $\mathbf{s}_{\mathbf{t}}, \mathbf{a}_{\mathbf{t}}$, and take gradient ascent steps on:

$$\nabla_{\theta} \sum_{\mathbf{t}=0}^{\tau} \min \left\{ \frac{\pi'(\mathbf{a}_{\mathbf{t}}|\mathbf{s}_{\mathbf{t}}, \theta)}{\pi(\mathbf{a}_{\mathbf{t}}|\mathbf{s}_{\mathbf{t}})} \hat{\mathcal{A}}_{\mathbf{t}}, \text{clip} \left(\frac{\pi'(\mathbf{a}_{\mathbf{t}}|\mathbf{s}_{\mathbf{t}}, \theta)}{\pi(\mathbf{a}_{\mathbf{t}}|\mathbf{s}_{\mathbf{t}})}, 1 - \epsilon, 1 + \epsilon \right) \hat{\mathcal{A}}_{\mathbf{t}} \right\}.$$

Algorithm 39 PPO

- 1: **Input** step sizes α, β , inner loop optimization steps K , clip parameter ϵ
- 2: **Initialize** $\theta, \mathbf{w}$ arbitrarily
- 3: **For** $j = 0, 1, 2, \dots$
- 4: Sample rollout $(\mathbf{s}_0, \mathbf{a}_0, \mathbf{r}_0, \dots, \mathbf{s}_\tau, \mathbf{a}_\tau, \mathbf{r}_\tau)$ using policy π .
- 5: Set $R_{\mathbf{t}:\tau} = \sum_{i=\mathbf{t}}^{\tau} \mathbf{r}_i$
- 6: Set $\hat{\mathcal{A}}_{\mathbf{t}} = R_{\mathbf{t}:\tau} - \hat{\mathcal{V}}(\mathbf{s}_{\mathbf{t}}; \mathbf{w})$
- 7: Set $\theta_{prev} = \theta$
- 8: **For** $k = 1, \dots, K$
- 9: Update policy parameters:

$$\theta := \theta + \alpha \nabla_{\theta} \sum_{\mathbf{t}=0}^{\tau} \min \left\{ \frac{\pi(\mathbf{a}_{\mathbf{t}}|\mathbf{s}_{\mathbf{t}}, \theta)}{\pi(\mathbf{a}_{\mathbf{t}}|\mathbf{s}_{\mathbf{t}}, \theta_{prev})} \hat{\mathcal{A}}_{\mathbf{t}}, \text{clip} \left(\frac{\pi(\mathbf{a}_{\mathbf{t}}|\mathbf{s}_{\mathbf{t}}, \theta)}{\pi(\mathbf{a}_{\mathbf{t}}|\mathbf{s}_{\mathbf{t}}, \theta_{prev})}, 1 - \epsilon, 1 + \epsilon \right) \hat{\mathcal{A}}_{\mathbf{t}} \right\}$$

- 10: Update value parameters:

$$\mathbf{w} := \mathbf{w} + \beta \sum_{\mathbf{t}=0}^{\tau} \hat{\mathcal{A}}_{\mathbf{t}} \nabla_{\mathbf{w}} \hat{\mathcal{V}}(\mathbf{s}_{\mathbf{t}}; \mathbf{w})$$

13.9 Alternative Proofs for the Policy Gradient Theorem

In this section, for didactic purposes, we show three alternative proofs for the policy gradient theorem (Theorem 13.4). The first proof is based on an elegant idea of unrolling of the value function. The second is based on a trajectory-based view, and will also lead to an interesting insight about partially observed systems. The third is based on importance sampling.

13.9.1 Proof Based on Unrolling the Value Function

The following is an alternative proof of Theorem 13.4.

Proof. For each state $\mathbf{s}$ we have

$$\begin{aligned}
\nabla \mathcal{V}^\pi(\mathbf{s}) &= \nabla \sum_{\mathbf{a}} \pi(\mathbf{a}|\mathbf{s}) \mathcal{Q}^\pi(\mathbf{s}, \mathbf{a}) \\
&= \sum_{\mathbf{a}} \mathcal{Q}^\pi(\mathbf{s}, \mathbf{a}) \nabla \pi(\mathbf{a}|\mathbf{s}) + \pi(\mathbf{a}|\mathbf{s}) \nabla \mathcal{Q}^\pi(\mathbf{s}, \mathbf{a}) \\
&= \sum_{\mathbf{a}} \mathcal{Q}^\pi(\mathbf{s}, \mathbf{a}) \nabla \pi(\mathbf{a}|\mathbf{s}) + \pi(\mathbf{a}|\mathbf{s}) \sum_{\mathbf{s}_1} p(\mathbf{s}_1|\mathbf{s}, \mathbf{a}) \nabla \mathcal{V}^\pi(\mathbf{s}_1) \\
&= \sum_{\mathbf{a}} \mathcal{Q}^\pi(\mathbf{s}, \mathbf{a}) \nabla \pi(\mathbf{a}|\mathbf{s}) + \sum_{\mathbf{s}_1} P^\pi(\mathbf{s}_1|\mathbf{s}) \nabla \mathcal{V}^\pi(\mathbf{s}_1) \\
&= \sum_{\mathbf{a}} \mathcal{Q}^\pi(\mathbf{s}, \mathbf{a}) \nabla \pi(\mathbf{a}|\mathbf{s}) + \sum_{\mathbf{s}_1} P^\pi(\mathbf{s}_1|\mathbf{s}) \sum_{\mathbf{a}} \mathcal{Q}^\pi(\mathbf{s}_1, \mathbf{a}) \nabla \pi(\mathbf{a}|\mathbf{s}_1) \\
&\quad + \sum_{\mathbf{s}_1, \mathbf{s}_2} P^\pi(\mathbf{s}_2|\mathbf{s}_1) P^\pi(\mathbf{s}_1|\mathbf{s}) \nabla \mathcal{V}^\pi(\mathbf{s}_2) \\
&= \sum_{\mathbf{s} \in \mathcal{S}} \sum_{t=0}^{\infty} \Pr(\mathbf{s}_t = \mathbf{s} | \mathbf{s}_0 = \mathbf{s}, \pi) \sum_{\mathbf{a}} \mathcal{Q}^\pi(\mathbf{s}, \mathbf{a}) \nabla \pi(\mathbf{a}|\mathbf{s}),
\end{aligned}$$

where the first identity follows since by averaging $\mathcal{Q}^\pi(\mathbf{s}, \mathbf{a})$ over the actions $\mathbf{a}$, with the probabilities induce by $\pi(\mathbf{a}|\mathbf{s})$, we have both correct expectation of the immediate reward and the next state is distributed correctly. The second equality follows from the gradient of a multiplication, i.e., $\nabla AB = A\nabla B + B\nabla A$. The third follows since $\nabla \mathcal{Q}^\pi(\mathbf{s}, \mathbf{a}) = \nabla [\mathbf{r}(\mathbf{s}, \mathbf{a}) + \sum_{\mathbf{s}'} P(\mathbf{s}'|\mathbf{s}, \mathbf{a}) \mathcal{V}^\pi(\mathbf{s}'|\mathbf{s}, \mathbf{a})]$. The next two identities role the policy one step in to the future. The last identity follows from unrolling $\mathbf{s}_1$ to $\mathbf{s}_2$ etc., and then reorganizing the terms. The term that depends on $\nabla \mathcal{V}^\pi(\mathbf{s}_2)$ vanishes for $t \rightarrow \infty$ because we assume that the termination time is bounded with probability 1.

Using this we have

$$\begin{aligned}
\nabla J(\theta) &= \nabla \sum_{\mathbf{s}} \mu(\mathbf{s}) \mathcal{V}^\pi(\mathbf{s}) \\
&= \sum_{\mathbf{s}} \mu(\mathbf{s}) \left(\sum_{t=0}^{\infty} \Pr(\mathbf{s}_t = \mathbf{s} | \mathbf{s}_0 = \mathbf{s}, \pi) \right) \sum_{\mathbf{a}} \mathcal{Q}^\pi(\mathbf{s}, \mathbf{a}) \nabla \pi(\mathbf{a}|\mathbf{s}) \\
&= \sum_{\mathbf{s}} \left(\sum_{t=0}^{\infty} \Pr(\mathbf{s}_t = \mathbf{s} | \mu, \pi) \right) \sum_{\mathbf{a}} \mathcal{Q}^\pi(\mathbf{s}, \mathbf{a}) \nabla \pi(\mathbf{a}|\mathbf{s}) \\
&= \sum_{\mathbf{s}} d^\pi(\mathbf{s}) \sum_{\mathbf{a}} \nabla \pi(\mathbf{a}|\mathbf{s}) \mathcal{Q}^\pi(\mathbf{s}, \mathbf{a}),
\end{aligned}$$

where the last equality is by definition of d^π . □

13.9.2 Proof Based on the Trajectory View

We next describe yet another proof for the policy gradient theorem, which will provide some interesting insights.

We begin by denoting by $\mathbf{X}$ a random rollout from the policy, $\mathbf{X} = (\mathbf{s}_0, \mathbf{a}_0, \mathbf{r}_0, \dots, \mathbf{s}_{\tau-1}, \mathbf{a}_{\tau-1}, \mathbf{r}_{\tau-1}, \mathbf{s}_G)$, where $\mathbf{s}_0 \sim \mu$, $\mathbf{a}_t \sim \pi(\cdot|\mathbf{s}_t, \theta)$, $\mathbf{s}_{t+1} \sim \mathbf{p}(\cdot|\mathbf{s}_t, \mathbf{a}_t)$, τ is the termination time and $\mathbf{s}_G$ is the unique goal state with zero terminal reward. We also let $\mathbf{r}(\mathbf{X}) = \sum_{t=0}^{\tau-1} \mathbf{r}_t$ denote the accumulated reward in the rollout, and $\Pr(\mathbf{X})$ the probability of observing $\mathbf{X}$, which by our definitions is

$$\Pr(\mathbf{X}) = \mu(\mathbf{s}_0)\pi(\mathbf{a}_0|\mathbf{s}_0, \theta)\mathbf{p}(\mathbf{s}_1|\mathbf{s}_0, \mathbf{a}_0)\pi(\mathbf{a}_1|\mathbf{s}_1, \theta) \cdots \mathbf{p}(\mathbf{s}_G|\mathbf{s}_{\tau-1}, \mathbf{a}_{\tau-1}). \quad (13.6)$$

We therefore have that

$$J(\theta) = \mathbb{E}^\pi [\mathbf{r}(\mathbf{X})] = \sum_{\mathbf{X}} \Pr(\mathbf{X})\mathbf{r}(\mathbf{X}),$$

and, by using a similar trick to (13.3), we have that

$$\nabla J(\theta) = \sum_{\mathbf{X}} \nabla \Pr(\mathbf{X})\mathbf{r}(\mathbf{X}) = \sum_{\mathbf{X}} \frac{\nabla \Pr(\mathbf{X})}{\Pr(\mathbf{X})} \mathbf{r}(\mathbf{X}) \Pr(\mathbf{X}) = \mathbb{E}^\pi [\mathbf{r}(\mathbf{X}) \nabla \log \Pr(\mathbf{X})].$$

We now notice that

$$\begin{aligned} \nabla \log \Pr(\mathbf{X}) &= \nabla (\log \mu(\mathbf{s}_0) + \log \pi(\mathbf{a}_0|\mathbf{s}_0, \theta) \\ &\quad + \log \mathbf{p}(\mathbf{s}_1|\mathbf{s}_0, \mathbf{a}_0) + \cdots + \log \mathbf{p}(\mathbf{s}_G|\mathbf{s}_{\tau-1}, \mathbf{a}_{\tau-1})) \\ &= \sum_{t=0}^{\tau-1} \nabla \log \pi(\mathbf{a}_t|\mathbf{s}_t, \theta), \end{aligned}$$

where the first equality is by (13.6), and the second equality is since the transitions and initial distribution do not depend on θ . We therefore have that

$$\nabla J(\theta) = \mathbb{E}^\pi \left[\sum_{t=0}^{\tau-1} \nabla \log \pi(\mathbf{a}_t|\mathbf{s}_t, \theta) \sum_{t'=0}^{\tau-1} \mathbf{r}(\mathbf{s}_{t'}, \mathbf{a}_{t'}) \right]. \quad (13.7)$$

We next show that in the sums in (13.7), it suffices to only consider rewards that come after $\nabla \log \pi(\mathbf{a}_t|\mathbf{s}_t, \theta)$. For $t' < t$, we have

$$\begin{aligned} \mathbb{E}^\pi [\nabla \log \pi(\mathbf{a}_t|\mathbf{s}_t, \theta) \mathbf{r}(\mathbf{s}_{t'}, \mathbf{a}_{t'})] &= \mathbb{E}^\pi [\mathbb{E}^\pi [\nabla \log \pi(\mathbf{a}_t|\mathbf{s}_t, \theta) \mathbf{r}(\mathbf{s}_{t'}, \mathbf{a}_{t'}) | \mathbf{s}_0, \mathbf{a}_0, \dots, \mathbf{s}_t]] \\ &= \mathbb{E}^\pi [\mathbf{r}(\mathbf{s}_{t'}, \mathbf{a}_{t'}) \mathbb{E}^\pi [\nabla \log \pi(\mathbf{a}_t|\mathbf{s}_t, \theta) | \mathbf{s}_0, \mathbf{a}_0, \dots, \mathbf{s}_t]] \\ &= 0, \end{aligned}$$

where the first equality is from the law of total expectation, and the last is similar to (13.4). So we have

$$\nabla J(\theta) = \mathbb{E}^\pi \left[\sum_{t=0}^{\tau-1} \nabla \log \pi(\mathbf{a}_t | \mathbf{s}_t, \theta) \sum_{t'=t}^{\tau-1} r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) \right]. \quad (13.8)$$

Note that the REINFORCE (Algorithm 36) can be seen as estimating the expectation in (13.8) from a single roll out. To finally obtain the policy gradient theorem, using the law of total expectation again, we have

$$\begin{aligned} & \mathbb{E}^\pi \left[\sum_{t=0}^{\tau-1} \nabla \log \pi(\mathbf{a}_t | \mathbf{s}_t, \theta) \sum_{t'=t}^{\tau-1} r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) \right] \\ &= \mathbb{E}^\pi \left[\sum_{t=0}^{\infty} \nabla \log \pi(\mathbf{a}_t | \mathbf{s}_t, \theta) \sum_{t'=t}^{\infty} r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) \right] \\ &= \sum_{t=0}^{\infty} \mathbb{E}^\pi \left[\nabla \log \pi(\mathbf{a}_t | \mathbf{s}_t, \theta) \sum_{t'=t}^{\infty} r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) \right] \\ &= \sum_{t=0}^{\infty} \mathbb{E}^\pi \left[\mathbb{E}^\pi \left[\nabla \log \pi(\mathbf{a}_t | \mathbf{s}_t, \theta) \sum_{t'=t}^{\infty} r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) \middle| \mathbf{s}_t, \mathbf{a}_t \right] \right] \\ &= \sum_{t=0}^{\infty} \mathbb{E}^\pi \left[\nabla \log \pi(\mathbf{a}_t | \mathbf{s}_t, \theta) \mathbb{E}^\pi \left[\sum_{t'=t}^{\infty} r(\mathbf{s}_{t'}, \mathbf{a}_{t'}) \middle| \mathbf{s}_t, \mathbf{a}_t \right] \right] \\ &= \sum_{t=0}^{\infty} \mathbb{E}^\pi [\nabla \log \pi(\mathbf{a}_t | \mathbf{s}_t, \theta) Q^\pi(\mathbf{s}_t, \mathbf{a}_t)] \\ &= \mathbb{E}^\pi \left[\sum_{t=0}^{\tau-1} \nabla \log \pi(\mathbf{a}_t | \mathbf{s}_t, \theta) Q^\pi(\mathbf{s}_t, \mathbf{a}_t) \right], \end{aligned}$$

which is equivalent to the expression in Corollary 13.5. The first equality is since the terminal state is absorbing, and has reward zero. The justification for exchanging the expectation and infinite sum in the second equality is not straightforward. In this case it holds by the Fubini theorem, using Assumption 7.1.

Partially Observed States We note that the derivation of (13.7) follows through if we consider policies that cannot access the state, but only some encoding ϕ of it, $\pi(\mathbf{a} | \phi(\mathbf{s}))$. Even though the optimal Markov policy in an MDP is deterministic, the encoding may lead to a system that is not Markovian anymore, by coalescing certain

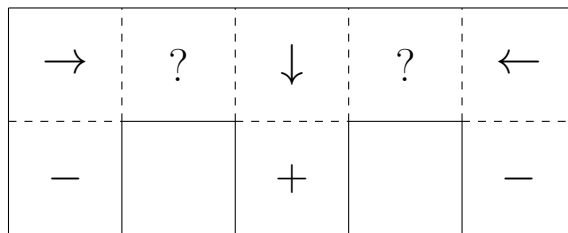


Figure 13.3: Aliased grid-world example

states which have identical encoding. Considering stochastic policies and using a policy gradient approach can be beneficial in such situations, as demonstrated in the following example.

Example 13.7 (Aliased Grid-world from David Silver’s course [137]). *Consider the navigation example in Figure 13.3. The $+$ state is the good goal and the $-$ ones are the bad goals. The encoding of each state is the location of the walls (solid lines). In each state we need to choose a direction. The problem is that we have two states which are indistinguishable, as they have the same wall structure (marked by question mark).*

It is not hard to see that any deterministic policy would fail from some start state (either the left or the right one). Alternatively, we can use a randomized policy in those states, with probability half go right and probability half go left. For such a policy we have a rather short time to reach the $+$ goal state (and avoid the $-$ states).

The issue here was that two different states had the same encoding, and thus violated the Markovian assumption. This can occur when we encode the state with a small set of features, and some (hopefully, similar) states coalesce to a single representation.

Remark 13.4. *The state aliasing example above is a specific instance of a more general decision making problem with partial observability, such as the partially observed MDP (POMDP). While a treatment of POMDPs is not within the scope of this book, we mention that the policy gradient approach applies to such models as well [10].*

13.9.3 Proof Based on Importance Sampling

As above, we let $\mathbf{X}$ denote a trajectory, $\mathbf{X} = (\mathbf{s}_0, \mathbf{a}_0, \mathbf{r}_0, \dots, \mathbf{s}_{\tau-1}, \mathbf{a}_{\tau-1}, \mathbf{r}_{\tau-1}, \mathbf{s}_G)$, and we let $\mathbf{r}(\mathbf{X}) = \sum_{t=0}^{\tau-1} \mathbf{r}_t$ denote the accumulated reward in it. We denote $\Pr_{\theta}(\mathbf{X})$ the probability of observing $\mathbf{X}$ under policy π_{θ} .

Recall from Section 11.6 that given two policy parameters, $\theta, \theta + \delta\theta$, we can estimate the return $J(\theta + \delta\theta)$ using a trajectory sampled from π_θ using importance sampling:

$$J(\theta + \delta\theta) = \mathbb{E}^{\pi_\theta} \left[\frac{\Pr_{\theta+\delta\theta}(\mathbf{X})}{\Pr_\theta(\mathbf{X})} \mathbf{r}(\mathbf{X}) \right],$$

and therefore,

$$J(\theta + \delta\theta) - J(\theta) = \mathbb{E}^{\pi_\theta} \left[\left(\frac{\Pr_{\theta+\delta\theta}(\mathbf{X}) - \Pr_\theta(\mathbf{X})}{\Pr_\theta(\mathbf{X})} \right) \mathbf{r}(\mathbf{X}) \right].$$

Plugging in the derivative definition we obtain:

$$\begin{aligned} \frac{\partial J(\theta)}{\partial \theta} &= \lim_{\delta\theta \rightarrow 0} \frac{J(\theta + \delta\theta) - J(\theta)}{\delta\theta} = \lim_{\delta\theta \rightarrow 0} \mathbb{E}^{\pi_\theta} \left[\frac{1}{\Pr_\theta(\mathbf{X})} \left(\frac{\Pr_{\theta+\delta\theta}(\mathbf{X}) - \Pr_\theta(\mathbf{X})}{\delta} \right) \mathbf{r}(\mathbf{X}) \right] \\ &= \mathbb{E}^{\pi_\theta} \left[\frac{1}{\Pr_\theta(\mathbf{X})} \frac{\partial \Pr_\theta(\mathbf{X})}{\partial \theta} \mathbf{r}(\mathbf{X}) \right] \\ &= \mathbb{E}^{\pi_\theta} \left[\frac{\partial \log \Pr_\theta(\mathbf{X})}{\partial \theta} \mathbf{r}(\mathbf{X}) \right], \end{aligned}$$

which is equivalent to Equation (13.7).

13.10 Bibliography Notes

The policy difference lemma is due to [68], and the proof here is based on [129].

The policy gradient theorem originated in [151], and the proof in Section 13.9 follows the original derivation. Alternative formulations of the theorem appear in [105, 106] and [10]. The proof based on importance sampling is from [64], and [61] extended it with variance reduction techniques.

The REINFORCE algorithm is from [172], which introduced also the baseline functions. Earlier investigations of related ideas in the operations research literature include [51, 49]. Convergence properties of the REINFORCE algorithm were studied in [118, 11, 105]. Optimal variance reduction using baseline functions was studied in [53]. Actor critic algorithms were studied in [84, 21], and multi-step lookahead TD updates for the actor was proposed in [28] and the popular generalized advantage estimation method of [131]. The PPO algorithm is from [132].

Convergence of policy gradient and convergence rates were developed in [134, 20]. The simplified analysis here is to our knowledge novel.

Chapter 14

Regret Minimization

In this chapter we take a closer look into the exploration-exploitation tradeoff under the lens of regret minimization. Most of this chapter concerns a simplified decision making model termed the multi-arm bandit, and we begin by motivating it. Later in the chapter we discuss the extension of the analysis to MDPs.

The multi-arm bandit model (MAB) is a simplified version of the MDP model, and historically predated it. Conceptually, the MAB does not have states (or technically, only a single state), but has multiple actions. Conceptually, our uncertainty is regarding the expected rewards of the various actions. This leads naturally to the well-known tradeoff between *exploration* and *exploitation*. On the one hand, we need to keep exploring the various actions to reduce the uncertainty regarding their expected rewards; this is the *exploration* part. On the other hand, we would like to guarantee a high return by using the information we already gathered, which is the *exploitation* part.

Motivation: Many natural problems can be modeled as MAB problems.

1. **Clinical Trials:** Each patient in the trial is prescribed one treatment from several possible treatments. Each treatment is viewed as an action, and the reward for each patient is the effectiveness of the prescribed treatment. The goal is to maximize the overall effectiveness of the treatments.
2. **Driving route selection:** Consider a navigation application selecting driving routes for users. The actions are the various routes from the initial point to the target place. The cost (negative reward) is the time it takes to drive a given route. The goal is to minimize the total cost of the users.

3. **Online advertisement:** In website advertising, a user visits a webpage, and a learning algorithm selects one of many possible advertisements to display to them. If an advertisement is displayed, the website observes whether the user clicks on it, in which case the advertiser pays the website a prescribed amount (which can depend also on the advertisement). In this case, each advertisement is an action, and the paid amount is the reward.
4. **Gambling:** The name Multi-Arm Bandits comes from a gambling application. Consider a slot-machine with multiple arms (or alternatively, multiple slot-machines, each with a single arm). Each of the arms has a (slightly) different expected reward. The gambler, at each time, needs to decide which arm to pull. The goal of the gambler is to maximize their return, the sum of the rewards.

Chapter Orientation

Main Prerequisites:

- Model based RL (Chapter 10)

Main terminology:

- k number of possible actions (arms)
- $\mathbf{t}$ round in a learning episode
- T length of the episode
- D_i reward distribution for arm i
- μ_i expected reward for arm i
- μ^* maximal expected reward
- $\mathbf{a}^*$ optimal action
- $\mathbf{r}_{\mathbf{t}}(\mathbf{a}_{\mathbf{t}})$ reward at time $\mathbf{t}$ for action $\mathbf{a}_{\mathbf{t}}$ (abbreviated as $\mathbf{r}_{\mathbf{t}}$)
- $n_T(i)$ number of times action i was played during the horizon T

- Δ_i action sub-optimality, i.e., $= \mu^* - \mu_i$
- $\hat{\mu}_t(i)$ empirical average reward for arm i at time t

14.1 Multi-Arm Bandit Model

A MAB is a simplified model of an MDP where there is only a single state and a fixed set $\mathcal{A}$ of k actions (also known as, arms). We consider a finite-horizon problem, where the horizon is T . Clearly, the planning problem is trivial, simply select the action with the highest expected reward. We will concentrate on the learning perspective, where the expected reward of each action is unknown. In the learning setting, we have a *single* episode of length T .

At each round $1 \leq t \leq T$, the learner selects and executes an action. After executing an action, the learner observes the reward of the played action. However, the rewards of the other actions in $\mathcal{A}$ are not revealed to the learner.

The reward for action i at round t is denoted by $r_t(i) \sim D_i$, where the support of the reward distribution D_i is $[0, 1]$. We assume that the rewards are i.i.d. (independent and identically distributed) across time steps, but can be correlated across actions in a single time step.

More formally, we have the following model:

- **Actions:** A finite set of actions $\mathcal{A} = \{\mathbf{a}_1, \dots, \mathbf{a}_k\}$. For simplicity we identify action $\mathbf{a}_i$ with the integer i .
- **Rewards:** Each action $\mathbf{a}_i$ has a reward distribution D_i over $[0, 1]$. The expectation of the reward of action $\mathbf{a}_i$ is:

$$\mu_i = \mathbb{E}_{r(\mathbf{a}_i) \sim D_i} [r(\mathbf{a}_i)]$$

The random variable of the reward of action $\mathbf{a}_i$ at time t is denoted by $r_t(\mathbf{a}_i)$.

- **Best action:** The maximum expected reward is $\mu^* = \max_i \mu_i$, and the corresponding action is $\mathbf{a}^* \in \arg \max_i \mu_i$.
- **Learner's actions and rewards:** At round t , the learner chooses action $\mathbf{a}_t$ and observes its reward $r_t(\mathbf{a}_t)$. When clear from the context, we use r_t for $r_t(\mathbf{a}_t)$.
- **Feedback:** The learner observes either full feedback, the reward for each possible action, or *bandit feedback*, only the reward $r_t(\mathbf{a}_t)$ of the selected action $\mathbf{a}_t$. For most of the chapter, we will consider the bandit setting.

Next, we need to define the objective of the learner. The simple objective is to maximize the cumulative reward collected during the entire episode, namely $\sum_{t=1}^T r_t(a_t)$. Equivalently, we measure the performance by comparing the learner's cumulative reward to the optimal cumulative reward. The difference is called the *regret*. Our goal would be to minimize the regret. Specifically, we would like the average regret to vanish as T goes to infinity. Formally we define the regret as follows,

$$\text{Regret} = \max_{i \in \mathcal{A}} \sum_{t=1}^T \underbrace{r_t(i)}_{\text{Random variable}} - \sum_{t=1}^T \underbrace{r_t(a_t)}_{\text{Random variable}} .$$

The regret as defined above is a random variable and we can consider the expected regret, i.e., $\mathbb{E}[\text{Regret}]$. This regret is a somewhat unachievable objective, since if the learner had known the complete model, and had selected the optimal action at each time, it would still have a regret. This would follow from the difference between the expectation and the realizations of the rewards.¹ For this reason, we will concentrate on the Pseudo Regret, which compares the learner's expected cumulative reward to the maximum expected cumulative reward.

$$\begin{aligned} \text{Pseudo Regret} &= \max_i \mathbb{E} \left[\sum_{t=1}^T r_t(i) \right] - \mathbb{E} \left[\sum_{t=1}^T r_t(a_t) \right] \\ &= \mu^* \cdot T - \mathbb{E} \left[\sum_{t=1}^T \mu_{a_t} \right] . \end{aligned}$$

Note that the difference between the Regret and the Pseudo Regret is related to the difference between taking the expected maximum (in Regret) versus the maximum expectation (in Pseudo Regret). Namely, in regret, our benchmark is $\mathbb{E}[\max_i \sum_{t=1}^T r_t(i)]$, while in pseudo regret, the benchmark is $\max_i \mathbb{E}[\sum_{t=1}^T r_t(i)]$. When considering a horizon T , the two benchmarks can differ by at most $O(\sqrt{T})$, as shown by Lemma 14.1 below. (Note that By Lemma 14.1, with high probability, for all actions $i \in \mathcal{A}$, we have $|\sum_{t=1}^T r_t(i) - \mathbb{E}[\sum_{t=1}^T r_t(i)]| = O(\sqrt{T \log |\mathcal{A}|})$.) In addition, if all the actions have the same expected reward, then any online algorithm will have zero pseudo regret; in contrast, the regret can be non-zero. In this chapter, we will only consider pseudo regret (and sometimes call it simply regret).

An alternative way to represent the pseudo regret is using the expected frequency

¹For example, assume we have two actions, both have a distribution of a Bernoulli r.v. with parameter $1/2$. Any online algorithm will have an expectation of $T/2$, but the expected realization of the better action would be $T/2 + \Theta(\sqrt{T})$.

we play each action, as follows,

$$\begin{aligned}
\text{Pseudo Regret} &= \mu^* \cdot T - \mathbb{E} \left[\sum_{t=1}^T \mu_{\mathbf{a}_t} \right] \\
&= \sum_{t=1}^T \mu^* - \mathbb{E}[\mu_{\mathbf{a}_t}] \\
&= \sum_{i=1}^k \mathbb{E}[n_T(i)](\mu^* - \mu_i) \\
&= \sum_{i=1}^k \mathbb{E}[n_T(i)]\Delta_i
\end{aligned}$$

where $n_T(i)$ is the number of times we played action i during the horizon T , and $\Delta_i = \mu^* - \mu_i$. The equivalence follows since each time we play action i , we incur a regret of $\mu^* - \mu_i = \Delta_i$.

We will use extensively the following concentration bound, which is the additive bound in Lemma 10.2.

Lemma 14.1 (Chernoff-Hoeffding). *Given $X_1, \dots, X_m$ i.i.d. random variables such that $X_i \in [0, 1]$ and $\mathbb{E}[X_i] = \mu$, we have*

$$\Pr \left[\frac{1}{m} \sum_{i=1}^m X_i - \mu \geq \epsilon \right] \leq \exp(-2\epsilon^2 m).$$

Alternatively, for $m \geq \frac{1}{2\epsilon^2} \log(1/\delta)$, with probability $1 - \delta$ we have that $\frac{1}{m} \sum_{i=1}^m X_i - \mu \leq \epsilon$.

14.2 Warmup: Full Information, Two Actions

We start with a simple case where there are two actions, and we observe the reward of both actions at each time t . We will analyze the greedy policy, which selects the action with the higher average reward (so far); see Algorithm 40.

Algorithm 40 Greedy Bandit Policy (k actions)

- 1: Choose $\mathbf{a}_1$ arbitrarily
- 2: **For** $t = 1, \dots, T$
- 3: Observe rewards $\mathbf{r}_t(1), \mathbf{r}_t(2), \dots, \mathbf{r}_t(k)$
- 4: Compute empirical means:

$$\hat{\mu}_t(i) = \frac{1}{t} \sum_{\tau=1}^t \mathbf{r}_\tau(i), \quad i \in \{1, \dots, k\}$$

- 5: Choose next action:

$$\mathbf{a}_{t+1} = \arg \max_{i \in \{1, \dots, k\}} \hat{\mu}_t(i)$$

We now want to compute the expected regret of the greedy policy. Without loss of generality (w.l.o.g.), we assume that $\mu_1 \geq \mu_2$ and define $\Delta = \mu_1 - \mu_2 \geq 0$. Each time we select the suboptimal action, we incur a pseudo regret. For the greedy algorithm, this happens when at time t we have $\hat{\mu}_t(2) \geq \hat{\mu}_t(1)$. The expected pseudo regret is,

$$\text{Pseudo Regret} = \sum_{t=1}^{\infty} (\mu_1 - \mu_2) \Pr[\hat{\mu}_t(2) \geq \hat{\mu}_t(1)].$$

Note that the above is an equivalent formulation of the pseudo regret. In each time step that the greedy algorithm selects the optimal action, clearly the difference is zero, so we can ignore those time steps. In time steps when the greedy algorithm selects the alternative action (action 2) it has a pseudo regret of $\mu_1 - \mu_2$ compared to action 1. This is why we sum over all time steps, the probability that we select action 2 times the regret in that case, i.e., $\mu_1 - \mu_2$. Since we select action 2 at time t when $\hat{\mu}_t(2) \geq \hat{\mu}_t(1)$, the probability that we select action 2 is exactly the probability of the event $\hat{\mu}_t(2) \geq \hat{\mu}_t(1)$.

To bound the pseudo regret, we would like to upper bound the probability of the event $\hat{\mu}_t(2) \geq \hat{\mu}_t(1)$. Clearly, at any time t ,

$$\mathbb{E}[\hat{\mu}_t(2) - \hat{\mu}_t(1)] = \mu_2 - \mu_1 = -\Delta.$$

We can define a random variable $X_t = r_t(2) - r_t(1) + \Delta$, and we have $\mathbb{E}[X_t] = 0$. Since $(1/t) \sum_{t=1}^T X_t = \hat{\mu}_T(2) - \hat{\mu}_T(1) + \Delta$, by Lemma 14.1,

$$\Pr[\hat{\mu}_t(2) \geq \hat{\mu}_t(1)] = \Pr[\hat{\mu}_t(2) - \hat{\mu}_t(1) + \Delta \geq \Delta] \leq e^{-2\Delta^2 t}.$$

We can now bound the expected pseudo regret as follows,

$$\begin{aligned} \mathbb{E}[\text{Pseudo Regret}] &= \sum_{t=1}^T \Delta \Pr[\hat{\mu}_t(2) \geq \hat{\mu}_t(1)] \\ &\leq \sum_{t=1}^{\infty} \Delta e^{-2\Delta^2 t} \\ &\leq \int_0^{\infty} \Delta e^{-2\Delta^2 t} dt \\ &= \left[-\frac{1}{2\Delta} e^{-2\Delta^2 t} \right]_0^{\infty} \\ &= \frac{1}{2\Delta}. \end{aligned}$$

We have established the following theorem.

Theorem 14.2. *In the full-information two-actions multi-arm bandit model, the greedy algorithm guarantees an expected pseudo regret of at most $1/(2\Delta)$, where $\Delta = |\mu_1 - \mu_2|$.*

Notice that this regret bound does not depend on the horizon T . The intuitive reason is that since we calculate the empirical means using the observed rewards, all the empirical means converge quickly to their true means, and the probability of selecting a suboptimal action becomes overwhelmingly small – so small that summing up the probabilities over time is bounded by a constant.

14.3 Stochastic Multi-Arm Bandits: Lower Bound

We now prove that we cannot achieve a regret that does not depend on T for the bandit feedback; that is, when we observe only the reward of the action we selected.

Consider the following example. For action $\mathbf{a}_1$, we have the following distribution,

$$\mathbf{a}_1 \sim Br\left(\frac{1}{2}\right),$$

where $Br(p)$ is a Bernoulli random variable with parameter p .

For action $\mathbf{a}_2$, there are two equally likely alternative distributions, each with probability $1/2$,

$$\mathbf{a}_2 \sim Br\left(\frac{1}{4}\right) \left(w.p.\frac{1}{2}\right) \quad or \quad \mathbf{a}_2 \sim Br\left(\frac{3}{4}\right) \left(w.p.\frac{1}{2}\right).$$

In this setting, since the distribution of action $\mathbf{a}_1$ is known, the optimal policy will select action $\mathbf{a}_2$ for some time M (potentially, $M = T$ is also possible) and then potentially switch to action $\mathbf{a}_1$. The reason is that once we switch to action $\mathbf{a}_1$, we will not receive any new information regarding the optimal action, since the distribution of action $\mathbf{a}_1$ is known.

Assume, by way of contradiction,

$$\mathbb{E}[\text{Pseudo Regret}] = R,$$

where R does not depend on T . By Markov's inequality:

$$\Pr[\text{Pseudo Regret} \geq 2R] \leq \frac{1}{2}$$

Since μ_1 is known, an optimal algorithm will first check $\mathbf{a}_2$ in order to decide which action is better and stick with it.

Assuming $\mu_2 = \frac{1}{4}$ and the algorithm decided to stop playing $\mathbf{a}_2$ after M rounds, Then:

$$\text{Pseudo Regret} = \frac{1}{4}M$$

Thus,

$$\Pr \left[\text{Pseudo Regret} \geq 2R | \mu_2 = \frac{1}{4} \right] = \Pr \left[M \geq 8R | \mu_2 = \frac{1}{4} \right] \leq \frac{1}{2},$$

equivalently,

$$\Pr \left[M < 8R | \mu_2 = \frac{1}{4} \right] > \frac{1}{2}.$$

Hence, the probability that after $8R$ rounds, the algorithm will stop playing $\mathbf{a}_2$ (if $\mu_2 = \frac{1}{4}$) is at least $\frac{1}{2}$. This implies that there is some sequence of $8R$ outcomes that will result in stopping to try action $\mathbf{a}_2$. For simplicity, assume that the sequence is the all-zero sequence. (It is sufficient to note that any sequence of length $8R$ has probability at least 4^{-8R} .)

Assume $\mu_2 = \frac{3}{4}$, but that all $8R$ first rounds playing $\mathbf{a}_2$ yield the value zero (which happens with probability $(\frac{1}{4})^{8R}$). We assumed that after $8R$ zeros for action $\mathbf{a}_2$, the algorithm will stop playing $\mathbf{a}_2$, even though it is the preferred action. In this case, we will get:

$$\text{Pseudo Regret} = \frac{1}{4}(\mathbf{T} - M) \approx \frac{1}{4}\mathbf{T}.$$

The expected Pseudo Regret is,

$$\mathbb{E}[\text{Pseudo Regret}] = R \geq \underbrace{\frac{1}{2}}_{\mathbf{a}_2 \sim \Pr(Br(\frac{3}{4}))} \cdot \underbrace{\left(\frac{1}{4}\right)^{8R}}_{\Pr(\forall t \leq 8R \mathbf{r}_t=0 | \mathbf{a}_2 \sim \Pr(Br(\frac{3}{4})))} \cdot (\mathbf{T} - 8R) \approx e^{-O(R)}\mathbf{T},$$

which implies that,

$$R = \Omega(\log \mathbf{T}).$$

This contradicts the assumption that R does not depend on $\mathbf{T}$. We have established the following theorem,

Theorem 14.3. *For each online algorithm, there is a MAB instance for which the expected pseudo regret is $\Omega(\log \mathbf{T})$*

Discussion: Lower bound

While Theorem 14.3 gives a logarithmic lower bound, there are many additional lower bounds in the literature. The most important distinction is between *instance dependent* and *instance independent* lower bounds.

The instance independent lower bounds hold in the worst case. Namely, they show a difficult instance for which the lower bound holds for any algorithm. For the MAB model, one can show a lower bound of $\Omega(\sqrt{T})$. The lower bound can be derived using two actions. One action has rewards distributed as $Br(\frac{1}{2})$ while the other action has rewards of $Br(\frac{1}{2} + \frac{1}{\sqrt{T}})$ or $Br(\frac{1}{2} - \frac{1}{\sqrt{T}})$, equally likely. The proof uses information theory arguments. The crux of the proof is to show that in order to distinguish between the random variables $Br(\frac{1}{2} - \varepsilon)$ and $Br(\frac{1}{2} + \varepsilon)$, one needs to observe $\Omega(\frac{1}{\varepsilon^2})$ samples. This implies that in the above scenario, the two alternatives are almost indistinguishable in T time steps, which results in $O(\frac{1}{\sqrt{T}})$ regret per time step and overall $\Omega(\sqrt{T})$ regret.

An instance dependent lower bound holds for a specific problem instance with expected rewards $\mu_1, \dots, \mu_K$. Finding a zero regret algorithm for a specific instance is easy – always choose $\arg \max_i \mu_i$. Such a solution, however, will have a linear regret $\Omega(T)$ for any problem instance with a different optimal arm. Therefore, it is common to add a requirement that for any other problem instance the algorithm admits sub-linear regret, say, $O(T^{0.9})$. One can view the lower bound of Theorem 14.3 as an instance dependent lower bound. It is focused on the problem instance $\mu_1 = \frac{1}{2}, \mu_2 = \frac{1}{4}$. To minimize the regret, we minimized the number of times we play the suboptimal second action. However, in order to achieve a sub-linear regret on the other possible instance, $\mu_1 = \frac{1}{2}, \mu_2 = \frac{3}{4}$, we must sample the second action a logarithmic number of times before deciding to stop sampling it ever again. The general instance dependent lower bound is $\Omega(\frac{\log T}{\Delta})$, where Δ is the sub-optimality gap, the difference between the expected reward of the best action and the second-best action.

For more information about the lower bounds, see [144, 94].

14.4 Explore-Then-Exploit

In this section, we develop an algorithm with a vanishing average regret. The algorithm has two phases: In the first phase, the *explore* phase, it plays each action M times. In the second phase, the *exploit* phase, it exploits the information from the explore phase and always plays the action with the highest average reward in the explore phase. The reader may recall that in the model based RL Chapter 10 we discussed a conceptually similar algorithm – explicit explore then exploit. In our

previous discussion, we focused on establishing sufficient coverage of all actions (and states) to yield an approximately optimal policy after learning. Here, we focus on the regret, and effectively measure how many suboptimal decisions we make *during learning*, while guaranteeing that the probability of learning the best action is sufficiently high.

Algorithm 41 Explore-Then-Exploit

- 1: **Input:** Exploration parameter M satisfying $kM \leq T$
 - Explore phase:**
 - 2: For $i = 1, \dots, k$
 - 3: For $j = 1, \dots, M$
 - 4: Choose action i , observe reward $\mathbf{r}_j(i)$
 - 5: Compute empirical mean: $\hat{\mu}(i) = \frac{1}{M} \sum_{j=1}^M \mathbf{r}_j(i)$
 - Exploit phase:**
 - 6: Set $\hat{\mathbf{a}}^* = \arg \max_{i \in \{1, \dots, k\}} \hat{\mu}(i)$
 - 7: Choose $\hat{\mathbf{a}}^*$ in all remaining $T - kM$ rounds
-

Analysis: For each action $j \in \mathcal{A}$ we define:

$$S_j = \{\mathbf{t} : \mathbf{a}_{\mathbf{t}} = j, \mathbf{t} \leq k \cdot M\}$$

$$\hat{\mu}_j = \frac{1}{M} \sum_{\mathbf{t} \in S_j} \mathbf{r}_j(\mathbf{t}),$$

which are the set of rounds where we played action j in the explore phase and the average observed reward.

Recall that $\mu_j = \mathbb{E}[\mathbf{r}_j(\mathbf{t})]$ is the expected reward of action j , and $\Delta_j = \mu^* - \mu_j$ is the suboptimality of action j .

We can now write the regret as a function of those parameters:

$$\mathbb{E} [\text{Pseudo regret}] = \underbrace{\sum_{j=1}^k \Delta_j \cdot M}_{\text{Explore}} + \underbrace{(T - kM) \sum_{j=1}^k \Delta_j \Pr \left[j = \arg \max_i \hat{\mu}_i \right]}_{\text{Exploit}}.$$

For the analysis, define the confidence radius:

$$\lambda = \sqrt{\frac{2 \log T}{M}}.$$

By Lemma 14.1 we have

$$\Pr[|\hat{\mu}_j - \mu_j| \geq \lambda] \leq 2e^{-2\lambda^2 M} = \frac{2}{T^4},$$

which implies (using the union bound) that

$$\Pr[\underbrace{\exists_j : |\hat{\mu}_j - \mu_j| \geq \lambda}_B] \leq \frac{2k}{T^4} \underset{\text{for } k \leq T}{\leq} \frac{2}{T^3}.$$

Define the “bad event” $B = \{\exists_j : |\hat{\mu}_j - \mu_j| \geq \lambda\}$. If B did not happen, then for each action j for which $\hat{\mu}_j \geq \hat{\mu}^*$, we have

$$\mu_j + \lambda \geq \hat{\mu}_j \geq \hat{\mu}^* \geq \mu^* - \lambda.$$

Therefore,

$$2\lambda \geq \mu^* - \mu_j = \Delta_j,$$

and hence,

$$\Delta_j \leq 2\lambda.$$

This implies, given that the bad event B did not occur, that the action $\hat{\mathbf{a}}^*$ selected for the exploit phase has a regret of at most 2λ ; i.e., $\mu^* - \mu_{\hat{\mathbf{a}}^*} \leq 2\lambda$. Now, we can bound the expected regret as follows:

$$\begin{aligned} \mathbb{E}[\text{Pseudo Regret}] &\leq \underbrace{\left(\sum_{j=1}^k \Delta_j\right) M}_{\text{Explore}} + \underbrace{(T - k \cdot M) \cdot 2\lambda}_{\text{B didn't happen}} + \underbrace{\frac{2}{T^3} \cdot T}_{\text{B happened}} \\ &\leq k \cdot M + 2 \cdot \sqrt{\frac{2 \log T}{M}} \cdot T + \frac{2}{T^2}. \end{aligned}$$

We optimize the number of exploration rounds M and choose $M = \left(\frac{T}{k}\right)^{\frac{2}{3}}$, which gives:

$$\mathbb{E}[\text{Pseudo Regret}] \leq k^{1/3} \cdot T^{\frac{2}{3}} + 2 \cdot \sqrt{2 \log T} \cdot T^{\frac{2}{3}} + \frac{2}{T^2}.$$

We have established the following regret bound for the Explore-Exploit algorithm.

Theorem 14.4. *The Explore-Exploit algorithm, with parameter $M = \left(\frac{T}{k}\right)^{\frac{2}{3}}$, has*

$$\mathbb{E}[\text{Pseudo Regret}] \leq k^{1/3} \cdot T^{\frac{2}{3}} + 2 \cdot \sqrt{2 \log T} \cdot T^{\frac{2}{3}} + \frac{2}{T^2} = \tilde{O}(k^{1/3} T^{2/3}),$$

where $\tilde{O}(\cdot)$ ignores logarithmic terms.

The bound we obtained is sub-linear in T , but can we do better? In the following sections, we improve the dependency of the expected regret on the horizon from $T^{2/3}$ to $T^{1/2}$, which is the optimal rate as it matches the lower bound. The key idea that allows this is adaptivity: in Algorithm 41 we choose how many samples to allocate for exploration before collecting any data; thus, both easy-to-identify arms with a very big Δ and hard-to-identify arms with a small Δ get the same number of samples. If we allocate the exploration samples during runtime, based on the actual rewards we observe, we may obtain significantly better performance.

14.5 Improved Regret Minimization Algorithms

We now consider more advanced algorithms that mix exploration and exploitation more elegantly. We define:

$n_t(i)$ - the number of times we chose action i by round t , including round t ; i.e.,
 $n_t(i) = \sum_{\tau=1}^t \mathbb{I}(a_\tau = i)$. Notice that $n_t(i)$ is a random variable, not a number!

$\hat{\mu}_t(i)$ - the average reward of action i so far, that is:

$$\hat{\mu}_t(i) = \frac{1}{n_t(i)} \sum_{\tau=1}^t r_\tau(i) \mathbb{I}(a_\tau = i).$$

In analogy with the confidence radius for a fixed number of exploration actions above, define the confidence radius after observing $n_t(i)$ samples from arm i , $\lambda_t(i) = \sqrt{\frac{2 \log T}{n_t(i)}}$. We would like to get the following result, for each round t and action i :

$$\Pr \left[|\hat{\mu}_t(i) - \mu_i| \leq \underbrace{\sqrt{\frac{2 \log T}{n_t(i)}}}_{\lambda_t(i)} \right] \geq 1 - (\text{quickly diminishing function of } t).$$

Unfortunately, we cannot simply apply the Chernoff-Hoeffding concentration bound (Lemma 14.1) since $n_t(i)$ is a random variable. Our way to overcome this issue is to pre-sample all possible rewards and make sure that *any reward sequence* satisfies the concentration bound. This is a stronger requirement, but one for which we can apply standard concentration bounds, and as we shall see, does not impact the resulting regret bound significantly (This technique is from [144]. We give here a sketch of the main idea; more details can be found in [144]).

Consider sampling each action i for T times, and storing the values in memory. We can later use this sample bank such that when we want to receive the m -th sample of action i in the MAB algorithm, we retrieve the m -th sample for action i stored in the bank. Since the rewards are assumed to be independent of the MAB algorithm, this way of pre-generating samples is statistically identical to sampling the rewards on-the-fly.

Define $\hat{V}_m(i)$ to be the empirical average of the first m reward samples for action i in the sample bank. From Lemma 14.1 we have

$$\Pr \left[\left| \hat{V}_m(i) - \mu_i \right| \leq \sqrt{\frac{2 \log T}{m}} \right] \geq 1 - \frac{2}{T^4} . \quad (14.1)$$

Define the “good event” G as: The condition in Equation 14.1 holds *for all* m, i . Using a union bound, we bound the probability of the good event:

$$\Pr [G] = \Pr \left[\left| \hat{V}_m(i) - \mu_i \right| \leq \sqrt{\frac{2 \log T}{m}}, \forall m \leq T, \forall i \in \mathcal{A} \right] \geq 1 - \frac{2k}{T^3} \geq 1 - \frac{2}{T^2} .$$

Finally, note that if G happened, then by definition $|\hat{\mu}_t(i) - \mu_i| \leq \lambda_t(i)$ for all $i, t \leq T$, as any sequence of empirical averages we encounter during the run will be composed of empirical averages that we already guaranteed to satisfy this requirement in our definition of G .

14.5.1 Refined Confidence Bounds

The confidence radius bounds how far the empirical average can be from the true mean, with high probability. Define the **upper confidence bound**:

$$UCB_t(i) = \hat{\mu}_t(i) + \lambda_t(i) ,$$

and similarly, the **lower confidence bound**:

$$LCB_t(i) = \hat{\mu}_t(i) - \lambda_t(i) .$$

If G holds then:

$$\forall i \forall t \quad \mu_i \in [LCB_t(i), UCB_t(i)] .$$

Therefore:

$$\Pr \left[\forall i \forall t \quad \mu_i \in [LCB_t(i), UCB_t(i)] \right] \geq 1 - \frac{2}{T^2} .$$

14.5.2 Successive Action Elimination

Our first improved algorithm is Successive Action Elimination. As the name suggests, we will eliminate suboptimal actions. The main issue is to do it in a conservative way that would guarantee that once we eliminate an action, it is with high probability a sub-optimal action. For this, we will use the UCB and LCB introduced before. Intuitively, if the UCB of action i is below the LCB of another action j , then we are certain that i cannot be optimal and can eliminate it from further selection. The Successive Action Elimination algorithm maintains a set of active actions S – the set of actions that have not yet been eliminated, and at each phase computes J – the set of actions to eliminate next.

Algorithm 42 Successive Action Elimination

```

1: Initialize: Set  $S_1 = \mathcal{A}$ 
2: For phase  $\mathbf{t} = 1, 2, \dots$ 
3:   Set  $J \leftarrow \emptyset$ 
4:   Play each action  $i \in S_{\mathbf{t}}$  once
5:   For each  $j \in S_{\mathbf{t}}$ 
6:     If there exists  $i \in S_{\mathbf{t}}$  such that  $UCB_{\mathbf{t}}(j) < LCB_{\mathbf{t}}(i)$ :
7:       Add  $j$  to  $J$ 
8:   Set  $S_{\mathbf{t}+1} \leftarrow S_{\mathbf{t}} \setminus J$ 

```

We maintain the following invariants:

- As long as action i is still in $S_{\mathbf{t}}$, we have played action i exactly the same number of times as any other action $j \in S_{\mathbf{t}}$; i.e., $\mathbf{t} - 1$ times.
- The best action, under the assumption that the event G holds, is never eliminated from $S_{\mathbf{t}}$ for any phase $\mathbf{t}$.

To see that the best action is never eliminated under the good event G , note the following for time $\mathbf{t}$. For the best action, we have $\mu^* < UCB_{\mathbf{t}}(\mathbf{a}^*)$, and for any action i , we have $LCB_{\mathbf{t}}(i) \leq \mu_i$. Since $LCB_{\mathbf{t}}(i) \leq \mu_i \leq \mu^* \leq UCB_{\mathbf{t}}(\mathbf{a}^*)$, any optimal action $\mathbf{a}^*$ is never eliminated.

Under the good event G , at phase $\mathbf{t}$, for any optimal action $\mathbf{a}^*$, we have

$$\mu^* - 2\lambda_{\mathbf{t}}(\mathbf{a}^*) \leq \hat{\mu}^* - \lambda_{\mathbf{t}}(\mathbf{a}^*) = LCB_{\mathbf{t}}(\mathbf{a}^*) .$$

Similarly, for any action $i \in S_{\mathbf{t}}$, we have

$$UCB_{\mathbf{t}}(i) = \hat{\mu}_i + \lambda_{\mathbf{t}}(i) \leq \mu_i + 2\lambda_{\mathbf{t}}(i) .$$

If action i is not eliminated at phase $\mathbf{t}$, we have

$$LCB_{\mathbf{t}}(\mathbf{a}^*) < UCB_{\mathbf{t}}(i) .$$

Finally, let $\lambda = \lambda_{\mathbf{t}}(i) = \lambda_{\mathbf{t}}(\mathbf{a}^*)$ since we have chosen action i and any best action $\mathbf{a}^*$, the same number of times so far. Combining all the inequalities we get

$$\mu^* - 2\lambda \leq \hat{\mu}^* - \lambda = LCB_{\mathbf{t}}(\mathbf{a}^*) < UCB_{\mathbf{t}}(i) = \hat{\mu}_i + \lambda \leq \mu_i + 2\lambda .$$

Therefore, assuming the good event G holds,

$$\Delta_i = \mu^* - \mu_i \leq 4\lambda = 4\sqrt{\frac{2 \log T}{n_{\mathbf{t}}(i)}} ,$$

which implies,

$$n_{\mathbf{t}}(i) \leq \frac{32}{\Delta_i^2} \log T ,$$

for any phase $\mathbf{t}$ where action i is played, and therefore bounds the total number of times action i is played.

This implies that we can bound the pseudo regret as follows,

$$\begin{aligned} \mathbb{E} [\text{Pseudo Regret}] &= \mathbb{E} \left[\sum_{i=1}^k \Delta_i n_{\mathbf{t}}(i) \right] \\ &\leq \sum_{i: \Delta_i > 0} \frac{32}{\Delta_i} \log T + \underbrace{\frac{2}{T^2} \cdot T}_{\text{The bad event}} \end{aligned}$$

Theorem 14.5. *The pseudo regret of successive action elimination is bounded by $O(\sum_{i: \Delta_i > 0} \frac{1}{\Delta_i} \log T)$.*

This is an instance dependent upper bound, which matches the order of the instance dependent lower bound (cf. Section 14.3). Note that the bound is unbounded when $\Delta_i \approx 0$. This is not really an issue, since such actions also have very small regret when we use them, and in the following we will use this intuition to derive an instance independent upper bound. Formally, we can partition the actions according to Δ_i . Let $\mathcal{A}_1 = \{i : \Delta_i < \sqrt{(k \log T)/T}\}$ be the set of actions with low Δ_i , and $\mathcal{A}_2 = \{i : \Delta_i \geq \sqrt{(k \log T)/T}\}$ the remaining actions. We can now re-analyze the

pseudo regret, as follows,

$$\begin{aligned}
\mathbb{E}[\text{Pseudo Regret}] &= \sum_{i=1}^k \Delta_i \mathbb{E}[n_{\mathbf{t}}(i)] \\
&= \sum_{i \in \mathcal{A}_1}^k \Delta_i \mathbb{E}[n_i(\mathbf{t})] + \sum_{i \in \mathcal{A}_2}^k \Delta_i \mathbb{E}[n_{\mathbf{t}}(i)] \\
&\leq \sum_{i \in \mathcal{A}_1} \sqrt{\frac{k \log T}{T}} \mathbb{E}[n_{\mathbf{t}}(i)] + \sum_{i \in \mathcal{A}_2} \frac{32}{\Delta_i} \log T + \underbrace{\frac{2}{T^2} \cdot T}_{\text{The bad event}} \\
&\leq \sqrt{kT \log T} + 32k \sqrt{\frac{T}{k \log T}} \log T + \frac{2}{T} \\
&\leq 34 \sqrt{kT \log T}.
\end{aligned}$$

We have established the following instance independent upper bound, which matches the instance independent lower bound up to logarithmic factors.

Theorem 14.6. *The pseudo regret of successive action elimination is bounded by $O(\sqrt{kT \log T})$.*

14.5.3 Upper Confidence Bound (UCB) Algorithm

The elegant UCB algorithm simply selects the actions greedily using the UCB. The most naive method would be to greedily select actions using the empirical mean, but this can lead to very bad performance. Consider the case where we have two actions, and we play each once. For the first action, we observe a reward of 1, and for the second we observe a reward of 0. Assuming the rewards are either 0 or 1, if we select greedily using the empirical mean, we will always select action 1. However, it can be that action 2 is the optimal action.

Algorithm 43 Upper Confidence Bound (UCB)

- 1: **Initialization:** Play each action once (for a total of k rounds)
 - 2: **For** $\mathbf{t} = k + 1, k + 2, \dots, T$
 - 3: Select action $\mathbf{a}_{\mathbf{t}} = \arg \max_{i \in \mathcal{A}} UCB_{\mathbf{t}}(i)$
-

The idea behind the UCB approach is that we use *optimism*. When we select the action with the highest UCB, it is either due to one of the following: (1) the

confidence bound (λ) is large. In this case, we will have an explicit exploration, reducing the confidence bound. (2) the confidence bound (λ) is small. In this case, the empirical mean is large, and we will have an implicit exploitation.

If we choose action i , then, assuming the good event G holds, we have

$$UCB_{\mathbf{t}}(i) \geq UCB_{\mathbf{t}}(\mathbf{a}^*) \geq \mu^*,$$

where $\mathbf{a}^*$ is an optimal action.

Using the definition of the UCB, and the assumption that G holds, we have

$$UCB_{\mathbf{t}}(i) = \hat{\mu}_{\mathbf{t}}(i) + \lambda_{\mathbf{t}}(i) \leq \mu_i + 2\lambda_{\mathbf{t}}(i).$$

Since we selected action i at time $\mathbf{t}$, we have

$$\mu_i + 2\lambda_{\mathbf{t}}(i) \geq \hat{\mu}_{\mathbf{t}}(i) + \lambda_{\mathbf{t}}(i) \geq \hat{\mu}_{\mathbf{t}}(\mathbf{a}^*) + \lambda_{\mathbf{t}}(\mathbf{a}^*) \geq \mu^*.$$

Rearranging, we have,

$$2\lambda_{\mathbf{t}}(i) \geq \mu^* - \mu_i = \Delta_i.$$

Thus, each time we choose action i , our regret is Δ_i , and it is bounded:

$$\Delta_i \leq 2 \cdot \sqrt{\frac{2 \log \mathbf{T}}{n_{\mathbf{t}}(i)}}.$$

Therefore, if i is very far off from the optimal action, we would not choose it too many times. We can bound the number of times action i is used by,

$$n_{\mathbf{t}}(i) \leq \frac{8}{\Delta_i^2} \log \mathbf{T},$$

and overall, we get:

$$\begin{aligned} \mathbb{E}[\text{Pseudo Regret}] &= \sum_{i=1}^k \Delta_i \mathbb{E}[n_{\mathbf{t}}(i)] + \underbrace{\frac{2}{\mathbf{T}^2} \cdot \mathbf{T}}_{\text{The bad event}} \\ &\leq \sum_{i: \Delta_i} \frac{8}{\Delta_i} \cdot \log \mathbf{T} + \frac{2}{\mathbf{T}} \end{aligned}$$

Theorem 14.7. *The pseudo regret of UCB is bounded by $O(\sum_{i: \Delta_i} \frac{1}{\Delta_i} \log \mathbf{T})$.*

Similar to successive action elimination, we can establish the following instance-independent regret bound.

Theorem 14.8. *The pseudo regret of UCB is bounded by $O(\sqrt{k\mathbf{T} \log \mathbf{T}})$.*

14.6 From Multi-Arm Bandits to MDPs

Many of the techniques used in the case of Multi-Arm Bandits can naturally be extended to the case of MDPs. In this section, we sketch a simple extension where the dynamics of the MDPs are known, but the rewards are unknown.

We first need to define the model for online learning in MDPs, which will be very similar to the one in MAB. We will concentrate on the case of a finite horizon return. The learner interacts with the MDP for $\mathcal{K}$ episodes. We assume that $\mathcal{K} > T$ for simplicity, since we are interested in the asymptotic dependency on the number of episodes $\mathcal{K}$.

At each episode $\mathbf{t} \in [\mathcal{K}]$, the learner selects a Markov deterministic ² policy $\pi^{\mathbf{t}}$ and observes a trajectory $(\mathbf{s}_1^{\mathbf{t}}, \mathbf{a}_1^{\mathbf{t}}, \mathbf{r}_1^{\mathbf{t}}, \dots, \mathbf{s}_T^{\mathbf{t}})$, where actions are selected using $\pi^{\mathbf{t}}$; i.e., $\mathbf{a}_\tau^{\mathbf{t}} = \pi_{\mathbf{t}}(\mathbf{s}_\tau^{\mathbf{t}})$.

The goal of the learner is to minimize the pseudo regret. Let $\mathcal{V}^*(\mathbf{s}_1)$ be the optimal value function from the initial state $\mathbf{s}_1$. The pseudo regret is defined as,

$$\mathbb{E}[\text{Pseudo Regret}] = \mathbb{E} \left[\sum_{\mathbf{t} \in [\mathcal{K}]} \left(\mathcal{V}^*(\mathbf{s}_1) - \sum_{\tau=1}^T \mathbf{r}(\mathbf{s}_\tau^{\mathbf{t}}, \mathbf{a}_\tau^{\mathbf{t}}) \right) \right].$$

We now want to introduce a UCB-like algorithm. We assume that the learner knows the dynamics but does not know the rewards. This implies that the learner, given a reward function, can compute an optimal policy.

Let $\mu(\mathbf{s}, \mathbf{a}) = \mathbb{E}[\mathbf{r}(\mathbf{s}, \mathbf{a})]$ be the expected reward for state-action pair $(\mathbf{s}, \mathbf{a})$. As in the case of UCB, we define an Upper Confidence Bound for each reward. Namely, for each state $\mathbf{s}$ and action $\mathbf{a}$ we will maintain an empirical average $\hat{\mu}^{\mathbf{t}}(\mathbf{s}, \mathbf{a})$ and a confidence parameter $\lambda^{\mathbf{t}}(\mathbf{s}, \mathbf{a}) = \sqrt{\frac{2 \log \mathcal{K} |\mathcal{S}| |\mathcal{A}|}{\max(n_{\mathbf{s}, \mathbf{a}}^{\mathbf{t}}, 1)}}$, where $n_{\mathbf{s}, \mathbf{a}}^{\mathbf{t}}$ is the number of times we have visited state $\mathbf{s}$ and performed action $\mathbf{a}$ by the end of episode $\mathbf{t}$.

We define the good event similarly to before:

$$G = \{\forall \mathbf{s}, \mathbf{a}, \mathbf{t} \mid |\hat{\mu}^{\mathbf{t}}(\mathbf{s}, \mathbf{a}) - \mu(\mathbf{s}, \mathbf{a})| \leq \lambda^{\mathbf{t}}(\mathbf{s}, \mathbf{a})\},$$

and similar to the previous analyses, we show that G holds with high probability, specifically $1 - \frac{2}{\mathcal{K}^2}$.

Lemma 14.9. *For $\mathcal{K} > T$, we have that $\Pr[G] \geq 1 - \frac{2}{\mathcal{K}^2}$.*

Proof. This follows similarly to the UCB analysis using Chernoff-Hoeffding bounds (Lemma 14.1). $\square$

²The policy could also be stochastic, we will focus on deterministic ones.

Next, we describe the UCB-RL algorithm. For each episode $\mathbf{t}$ we compute a UCB for each state-action pair, denote the resulting reward function by $\bar{R}^{\mathbf{t}}$, where $\bar{R}^{\mathbf{t}}(\mathbf{s}, \mathbf{a}) = \hat{\mu}^{\mathbf{t}}(\mathbf{s}, \mathbf{a}) + \lambda^{\mathbf{t}}(\mathbf{s}, \mathbf{a})$.³ Let $\pi^{\mathbf{t}}$ be the optimal policy with respect to the rewards $\bar{R}^{\mathbf{t}}$ (the UCB rewards).

The following lemma demonstrates “optimism”, namely, the expected value of $\pi^{\mathbf{t}}$ w.r.t. the reward function $\bar{R}^{\mathbf{t}}$ upper bounds the optimal reward function $\mathcal{V}^*$.

In the following, we use the notation $\mathcal{V}(\cdot|R)$ to imply that we are using the reward function R . We denote the true reward function by R^* , i.e., $R^*(\mathbf{s}, \mathbf{a}) = \mathbb{E}[\mathbf{r}(\mathbf{s}, \mathbf{a})] = \mu(\mathbf{s}, \mathbf{a})$.

Lemma 14.10. *Assume the good event G holds. Then, for any episode $\mathbf{t} \in [\mathcal{K}]$, we have that $\mathcal{V}^{\pi^{\mathbf{t}}}(\mathbf{s}|\bar{R}^{\mathbf{t}}) \geq \mathcal{V}^*(\mathbf{s}|R^*)$, for all $\mathbf{s} \in \mathcal{S}$.*

Proof. Since $\pi^{\mathbf{t}}$ is optimal for the rewards $\bar{R}^{\mathbf{t}}$, we know that $\mathcal{V}^{\pi^{\mathbf{t}}}(\mathbf{s}|\bar{R}^{\mathbf{t}}) \geq \mathcal{V}^{\pi^*}(\mathbf{s}|\bar{R}^{\mathbf{t}})$. Since $\bar{R}^{\mathbf{t}} \geq R^*$, it follows that $\mathcal{V}^{\pi^*}(\mathbf{s}|\bar{R}^{\mathbf{t}}) \geq \mathcal{V}^{\pi^*}(\mathbf{s}|R^*)$. Combining these two inequalities, yields the lemma. $\square$

The optimism is a powerful property, as it lets us bound the pseudo regret as a function of quantities we observe, namely $\bar{R}^{\mathbf{t}}$, rather than unknown quantities, such as the true rewards R^* or the unknown optimal policy π^* .

Lemma 14.11. *Assume the good event G holds. Then,*

$$\mathbb{E}[\text{Pseudo Regret}] \leq \sum_{\mathbf{t} \in [\mathcal{K}]} \sum_{\tau=1}^{\mathbf{T}} \mathbb{E}[2\lambda^{\mathbf{t}}(\mathbf{s}_{\tau}^{\mathbf{t}}, \mathbf{a}_{\tau}^{\mathbf{t}})] ,$$

where $\mathbb{E}[\text{Pseudo Regret}]$ means a conditional expectation over all trajectories given that the good event holds.

Proof. The definition of the pseudo regret is

$$\mathbb{E}[\text{Pseudo Regret}] = \mathbb{E} \left[\sum_{\mathbf{t} \in [\mathcal{K}]} \left(\mathcal{V}^*(\mathbf{s}_1) - \sum_{\tau=1}^{\mathbf{T}} \mathbf{r}^{\mathbf{t}}(\mathbf{s}_{\tau}^{\mathbf{t}}, \mathbf{a}_{\tau}^{\mathbf{t}}) \right) \right] .$$

Using Lemma 14.10, we have,

$$\begin{aligned} \mathbb{E}[\text{Pseudo Regret}] &= \mathbb{E} \left[\sum_{\mathbf{t} \in [\mathcal{K}]} \left(\mathcal{V}^*(\mathbf{s}_1) - \sum_{\tau=1}^{\mathbf{T}} \mathbf{r}^{\mathbf{t}}(\mathbf{s}_{\tau}^{\mathbf{t}}, \mathbf{a}_{\tau}^{\mathbf{t}}) \right) \right] \\ &\leq \sum_{\mathbf{t} \in [\mathcal{K}]} \mathbb{E} \left[\mathcal{V}^{\pi^{\mathbf{t}}}(\mathbf{s}_1|\bar{R}^{\mathbf{t}}) - \sum_{\tau=1}^{\mathbf{T}} \mathbf{r}^{\mathbf{t}}(\mathbf{s}_{\tau}^{\mathbf{t}}, \mathbf{a}_{\tau}^{\mathbf{t}}) \right] . \end{aligned}$$

³In the literature, the additional reward $\bar{R}^{\mathbf{t}}(\mathbf{s}, \mathbf{a}) - \hat{\mu}^{\mathbf{t}}(\mathbf{s}, \mathbf{a})$ is often regarded as *bonuses*.

Since the good event G holds, we have

$$\begin{aligned}
\mathbb{E}[\text{Pseudo Regret}] &\leq \sum_{\mathbf{t} \in [\mathcal{K}]} \mathbb{E} \left[\mathcal{V}^{\pi^{\mathbf{t}}}(\mathbf{s}_1 | \bar{R}^{\mathbf{t}}) - \sum_{\tau=1}^T \mathbf{r}^{\mathbf{t}}(\mathbf{s}_{\tau}^{\mathbf{t}}, \mathbf{a}_{\tau}^{\mathbf{t}}) \right] \\
&\leq \sum_{\mathbf{t} \in [\mathcal{K}]} \mathbb{E} \left[\mathcal{V}^{\pi^{\mathbf{t}}}(\mathbf{s}_1 | \bar{R}^{\mathbf{t}}) - \sum_{\tau=1}^T \bar{R}^{\mathbf{t}}(\mathbf{s}_{\tau}^{\mathbf{t}}, \mathbf{a}_{\tau}^{\mathbf{t}}) + 2\lambda^{\mathbf{t}}(\mathbf{s}_{\tau}^{\mathbf{t}}, \mathbf{a}_{\tau}^{\mathbf{t}}) \right] \\
&= \sum_{\mathbf{t} \in [\mathcal{K}]} \mathbb{E} [2\lambda^{\mathbf{t}}(\mathbf{s}_{\tau}^{\mathbf{t}}, \mathbf{a}_{\tau}^{\mathbf{t}})]
\end{aligned}$$

and so we have,

$$\mathbb{E}[\text{Pseudo Regret}] \leq \mathbb{E} [\mathcal{V}^{\pi^{\mathbf{t}}}(\mathbf{s}_1 | \bar{R}^{\mathbf{t}})] - \mathbb{E} [\mathcal{V}^{\pi^{\mathbf{t}}}(\mathbf{s}_1 | R^*)] = \mathbb{E} \left[\sum_{\tau=1}^T 2\lambda^{\mathbf{t}}(\mathbf{s}_{\tau}^{\mathbf{t}}, \mathbf{a}_{\tau}^{\mathbf{t}}) \right],$$

which completes the proof of the lemma. $\square$

We are now left with only upper bounding the sum of the confidence bounds. We can upper bound this sum regardless of the realization.

Lemma 14.12.

$$\sum_{\mathbf{t} \in [\mathcal{K}]} \sum_{\tau=1}^T \lambda^{\mathbf{t}}(\mathbf{s}_{\tau}^{\mathbf{t}}, \mathbf{a}_{\tau}^{\mathbf{t}}) \leq 2\sqrt{2\mathcal{K}|\mathcal{S}||\mathcal{A}| \log(\mathcal{K}|\mathcal{S}||\mathcal{A}|)}.$$

Proof. We first change the order of summation to be over state-action pairs, as follows,

$$\sum_{\mathbf{t} \in [\mathcal{K}]} \sum_{\tau=1}^T \lambda^{\mathbf{t}}(\mathbf{s}_{\tau}^{\mathbf{t}}, \mathbf{a}_{\tau}^{\mathbf{t}}) = \sum_{\mathbf{s}, \mathbf{a}} \sum_{\tau=1}^{n_{\mathbf{s}, \mathbf{a}}^{\mathcal{K}}} \sqrt{\frac{2 \log(\mathcal{K}|\mathcal{S}||\mathcal{A}|)}{\max(\tau, 1)}}.$$

In the above, τ is the index of the τ -th visit to the state-action pair $(\mathbf{s}, \mathbf{a})$ at some time $\mathbf{t}$. During that visit we have that $n_{\mathbf{s}, \mathbf{a}}^{\mathbf{t}} = \tau$. This explains the expression for the confidence intervals.

We are interested in bounding the sum $\sum_{\tau=1}^N 1/\sqrt{\tau}$. We can do this by using the integral $\sum_{\tau=1}^N 1/\sqrt{\tau} \leq 1 + \int_1^N \frac{1}{\sqrt{x}} dx = 1 + [2\sqrt{x}]_1^N \leq 2\sqrt{N}$. Therefore,

$$\sum_{\mathbf{t} \in [\mathcal{K}]} \sum_{\tau=1}^T \lambda_{\mathbf{s}_{\tau}^{\mathbf{t}}, \mathbf{a}_{\tau}^{\mathbf{t}}}^{\mathbf{t}} \leq \sqrt{2 \log(\mathcal{K}|\mathcal{S}||\mathcal{A}|)} \sum_{\mathbf{s}, \mathbf{a}} 2\sqrt{n_{\mathbf{s}, \mathbf{a}}^{\mathcal{K}}}.$$

Recall that $\sum_{\mathbf{s}, \mathbf{a}} n_{\mathbf{s}, \mathbf{a}}^{\mathcal{K}} = \mathcal{K}T$. Since $\sqrt{\cdot}$ is concave, this implies that $\sum_{\mathbf{s}, \mathbf{a}} \sqrt{n_{\mathbf{s}, \mathbf{a}}^{\mathcal{K}}}$ is maximized when all the $n_{\mathbf{s}, \mathbf{a}}^{\mathcal{K}}$ are equal, i.e., $n_{\mathbf{s}, \mathbf{a}}^{\mathcal{K}} = \mathcal{K}T/(|\mathcal{S}||\mathcal{A}|)$, and thus $\sum_{\mathbf{s}, \mathbf{a}} \sqrt{n_{\mathbf{s}, \mathbf{a}}^{\mathcal{K}}} \leq \sqrt{|\mathcal{S}||\mathcal{A}|\mathcal{K}T}$. Hence,

$$\sum_{\mathbf{t} \in [\mathcal{K}]} \sum_{\tau=1}^T \lambda^{\mathbf{t}}(\mathbf{s}_{\tau}^{\mathbf{t}}, \mathbf{a}_{\tau}^{\mathbf{t}}) \leq 2\sqrt{2|\mathcal{S}||\mathcal{A}|\mathcal{K}T \log(\mathcal{K}|\mathcal{S}||\mathcal{A}|)}.$$

□

We can now derive the upper bound on the pseudo regret

Theorem 14.13.

$$\mathbb{E}[\text{Pseudo Regret}] \leq 4\sqrt{2\mathcal{K}T|\mathcal{S}||\mathcal{A}| \log(\mathcal{K}|\mathcal{S}||\mathcal{A}|)}.$$

14.7 Best Arm Identification

We now switch the objective from minimizing the regret to identifying the best action, or a near-best action. The main conceptual difference is that we are not concerned with the cumulative reward but rather with the number of rounds it would take us to identify the near-optimal action. In some sense, this is similar to a supervised learning scenario, where we aim to identify a hypothesis with low error, and our main concern is the sample complexity. We can define our goal in one of two ways.

PAC Criteria An action i is ϵ -optimal if $\mu_i \geq \mu^* - \epsilon$. The PAC criterion is that, given $\epsilon, \delta > 0$, with probability at least $1 - \delta$, we find an ϵ -optimal action.

Exact Identification Given a parameter $0 < \Delta \leq \min_{i \neq a^*} \mu^* - \mu_i$ (for every suboptimal action i), find the optimal action $\mathbf{a}^*$ with probability at least $1 - \delta$. The number of samples required would depend on Δ and δ .

In this section, we focus on the PAC Criterion, and our goal is to minimize the number of rounds until we output an ϵ -optimal action.

14.7.1 Naive Algorithm (PAC criteria)

The naive approach would simply approximate the expected reward of each action within error at most $\epsilon/2$, and output the action with the highest empirical reward

estimate. Formally, we sample each action i for $m = \frac{8}{\epsilon^2} \log \frac{2k}{\delta}$ times, and return $\hat{\mathbf{a}}^* = \arg \max_i \hat{\mu}_i$.

For rewards in $[0, 1]$, then, by Lemma 14.1, for every action i we have

$$Pr \left[\underbrace{|\hat{\mu}_i - \mu_i|}_{\text{bad event}} > \frac{\epsilon}{2} \right] \leq 2e^{-(\frac{\epsilon}{2})^2 m/2} = \frac{\delta}{k}.$$

By union bound, we get:

$$Pr \left[\exists_i |\hat{\mu}_i - \mu_i| > \frac{\epsilon}{2} \right] \leq \delta.$$

If the bad event $B = \{\exists_i |\hat{\mu}_i - \mu_i| > \frac{\epsilon}{2}\}$ did not happen, then: (1) $\mu^* - \frac{\epsilon}{2} \leq \hat{\mu}^*$ and (2) $\mu_i + \frac{\epsilon}{2} \leq \hat{\mu}_i$. This implies, given that $\hat{\mu}_i \geq \hat{\mu}^*$, that

$$\mu_i + \frac{\epsilon}{2} \geq \hat{\mu}_i \geq \hat{\mu}^* \geq \mu^* - \frac{\epsilon}{2},$$

and hence,

$$\epsilon \geq \mu^* - \mu_i.$$

Therefore $\hat{\mathbf{a}}^* = \arg \max_i \hat{\mu}_i$ is an ϵ -optimal action with probability at least $1 - \delta$. We have established the following theorem.

Theorem 14.14. *There is an algorithm that samples $\frac{8k}{\epsilon^2} \log \frac{2k}{\delta}$ times and returns an ϵ -optimal action with probability at least $1 - \delta$.*

14.7.2 Sequential Halving Algorithm

In this section, we aim to slightly improve the sample size of the naive PAC algorithm from the previous section. Our end result is fairly modest, we reduce the $O(\log \frac{2k}{\delta})$ factor to $O(\log \frac{1}{\delta})$.

The main idea is as follows. The algorithm runs for $l = \lceil \log k \rceil$ phases; after each phase, we eliminate half of the actions. This elimination allows us to sample each remaining action more times in the next phase, which makes eliminating the optimal action less likely.

Algorithm 44 Best Arm Identification

```
1: Input:  $\epsilon, \delta > 0$ 
2: Output:  $\bar{a} \in \mathcal{A}$ 
3: Init:  $S_1 = \mathcal{A}$ ,  $\epsilon_1 = \frac{\epsilon}{4}$ ,  $\delta_1 = \frac{\delta}{2}$ ,  $l = 1$ 
4: repeat
5:   for all  $i \in S_l$  do
6:     Sample action  $i$  for  $m(\epsilon_l, \delta_l) = \frac{1}{(\frac{\epsilon_l}{2})^2} \log\left(\frac{3}{\delta_l}\right)$  times
7:      $\hat{\mu}_i \leftarrow$  average reward of action  $i$  (only of samples during the  $l^{\text{th}}$  phase)
8:   end for
9:    $\text{median}_l \leftarrow \text{median}\{\hat{\mu}_i : i \in S_l\}$ 
10:   $S_{l+1} \leftarrow \{i \in S_l : \hat{\mu}_i \geq \text{median}_l\}$ 
11:   $\epsilon_{l+1} \leftarrow \frac{3}{4}\epsilon_l$ 
12:   $\delta_{l+1} \leftarrow \frac{\delta_l}{2}$ 
13:   $l \leftarrow l + 1$ 
14: until  $|S_l| = 1$ 
15: Output  $\bar{a}$  where  $S_l = \{\bar{a}\}$ 
```

Sample Complexity: During phase l , we have $|S_l| = \frac{k}{2^{l-1}}$ actions. We are setting the accuracy and confidence parameters as follows,

$$\epsilon_l = \frac{3}{4}\epsilon_{l-1} = \frac{\epsilon}{4} \left(\frac{3}{4}\right)^{l-1}, \quad \delta_l = \frac{\delta}{2^l}.$$

This implies that the sum of the accuracy and confidence parameters over all phases is,

$$\sum_l \epsilon_{l=1}^{\log k} \leq \frac{\epsilon}{4} \left(\frac{3}{4}\right)^{l-1} \leq \epsilon, \quad \text{and} \quad \sum_{l=1}^{\log k} \delta_l \leq \sum_{l=1}^{\log k} \frac{\delta}{2^l} \leq \delta$$

In phase l , we have S_l as the set of actions. For each action in S_l , we sample $m(\epsilon_l, \delta_l)$ samples. The total number of samples is

$$\begin{aligned}
\sum_{l=1}^{\log k} |S_l| m(\epsilon_l, \delta_l) &= \sum_{l=1}^{\log k} |S_l| \cdot \frac{4}{\epsilon_l^2} \log \frac{3}{\delta_l} \\
&= \sum_{l=1}^{\log k} \frac{k}{2^{l-1}} \frac{64}{\epsilon^2} \left(\frac{16}{9} \right)^{l-1} \log \frac{3 \cdot 2^l}{\delta} \\
&= \frac{64k}{\epsilon^2} \sum_{l=1}^{\log k} \left(\frac{8}{9} \right)^{l-1} \left[\log \frac{1}{\delta} + \log 3 + l \right] \\
&= O \left(\frac{k}{\epsilon^2} \log \frac{1}{\delta} \right),
\end{aligned}$$

where the last step follows since $\sum_l (8/9)^{l-1} \leq 9$ and $\sum_l (8/9)^{l-1} l \leq 81$.

Correctness: The following lemma is the main tool in establishing the correctness of the algorithm. It shows that when we move from phase l to phase $l+1$, with high probability (i.e., $1 - \delta_l$), the decrease in accuracy is at most ϵ_l .

Lemma 14.15. *Given S_l , we have*

$$\Pr \left[\underbrace{\max_{j \in S_l} \mu_j}_{\text{best action in } S_l} \leq \underbrace{\max_{j \in S_{l+1}} \mu_j}_{\text{best action in } S_{l+1}} + \epsilon_l \right] \geq 1 - \delta_l.$$

Proof. Let $\mu_l^* = \max_{j \in S_l} \mu_j$ be the expected reward of the best action in S_l , and $a_l^* \in \arg \max_{j \in S_l} \mu_j$ be a best action in S_l . Define the bad event $E_l = \{\hat{\mu}_l^* < \mu_l^* - \frac{\epsilon_l}{2}\}$. (Note that E_l depends only on the action a_l^* .) Since we sample a_l^* for $m(\epsilon_l, \delta_l)$ times, we have that $\Pr[E_l] \leq \frac{\delta_l}{3}$. If E_l did not happen, we define a bad set of actions:

$$\text{Bad} = \{j : \mu_l^* - \mu_j > \epsilon_l, \hat{\mu}_j \geq \hat{\mu}^*\}.$$

The set *Bad* includes the actions that have a better empirical average than a_l^* , and the difference in the expectation is more than ϵ_l . We would like to show that $S_{l+1} \not\subseteq \text{Bad}$, and hence includes at least one action that has expectation of at most ϵ_l from μ_l^* .

Consider an action j such that $\mu_l^* - \mu_j > \epsilon_l$, then:

$$\begin{aligned} \Pr \left[\hat{\mu}_j > \hat{\mu}_l^* \mid \underbrace{\hat{\mu}_l^* \geq \mu_l^* - \frac{\epsilon_l}{2}}_{\neg E_l} \right] &\leq \Pr \left[\hat{\mu}_j > \mu_l^* - \frac{\epsilon_l}{2} \mid \neg E_l \right] \\ &\leq \Pr \left[\hat{\mu}_j \geq \mu_j + \frac{\epsilon_l}{2} \mid \neg E_l \right] \\ &\leq \frac{\delta_l}{3}, \end{aligned}$$

where the second inequality follows since $\mu_l^* - \epsilon_l/2 > \mu_j + \epsilon_l/2$, which follows since $\mu_l^* - \mu_j > \epsilon_l$. The last inequality follows from the Chernoff-Hoeffding bound (Lemma 14.1) and the fact that the samples are independent.

Note that the failure probability is not negligible, and our main aim is to avoid a union bound, which introduces an $O(\log k)$ factor. We show that it cannot happen to too many such actions. We bound the expectation of the size of Bad ,

$$\mathbb{E} \left[|\text{Bad}| \mid \neg E_l \right] \leq |\text{Bad}| \frac{\delta_l}{3}.$$

With Markov's inequality we get:

$$\Pr \left[|\text{Bad}| \geq \frac{k}{2} \mid \neg E_l \right] \leq \frac{\mathbb{E} |\text{Bad}|}{|\text{Bad}|/2} = \frac{2}{3} \delta_l.$$

With probability $1 - \delta_l$: $\hat{\mu}_l^* \geq \mu^* - \frac{\epsilon_l}{2}$ and $|\text{Bad}| < \frac{k}{2}$. Therefore, there exists an action $j \in S_{l+1}$ for which $j \notin \text{Bad}$. $\square$

Given the above lemma, we can conclude with the following theorem.

Theorem 14.16. *The Sequential Halving Algorithm guarantees that, with probability at least $1 - \delta$, we have that $\mu^* - \mu_{\hat{a}} \leq \epsilon$.*

Proof. With probability at least $1 - \sum_{l=1}^{\log k} \delta_l \geq 1 - \delta$, we have that during each phase l , it holds that $\max_{j \in S_l} \mu_j \leq \max_{j \in S_{l+1}} \mu_j + \epsilon_l$.

By summing all the inequalities of the different phases, this implies that

$$\mu^* = \max_{j \in A} \mu_j \leq \mu_{\hat{a}} \sum_{l=1}^{\log k} \epsilon_l \leq \mu_{\hat{a}} + \epsilon.$$

Recall that $S_1 = \mathcal{A}$ and $S_{\log k} = \{\hat{a}\}$. $\square$

14.8 Bibliography Notes

Multi-arm bandits date back to Robbins [123], who defined (implicitly) asymptotic vanishing regret for two stochastic actions. The tight upper and lower asymptotic regret bounds were given by Lai and Robbins [90]. The UCB algorithm was presented in [5]. The action elimination is from [45], as is the Sequential Halving algorithm. Our presentation of the analysis of UCB and action elimination borrows from the presentation in [144]. Regret bounds for MDPs using an extension of the UCB idea were first presented by [6]. [26] developed asymptotic regret bounds for MDPs. A recent manuscript [1] extensively covers online regret minimization in MDPs.

There are multiple books that cover online learning and multi-arm bandits. The classical book of Cesa-Bianchi and Lugosi [29] studies adversarial online learning. The book of Slivkins [144] covers mainly stochastic multi-arm bandits. The book of Lattimore and Szepesvári [94] covers both adversarial bandits and online learning. The book of Hazan[57] covers online convex optimization.

Appendix A

Dynamic Programming

In this book, we focused on Dynamic Programming (DP) for solving problems that involve dynamical systems. The DP approach applies more broadly, and in this chapter we briefly describe DP solutions to computational problems of various forms. An in-depth treatment can be found in Chapter 15 of [34].

Dynamic programming is a method for solving complex computational problems by breaking them down into smaller, overlapping sub-problems. It leverages the principle of optimality, where the solution to a larger problem is constructed from the optimal solutions of its sub-problems. The key is to organize the computation so that each sub-problem is solved only once, storing its result for reuse, which avoids redundant calculations and significantly improves efficiency.

We remark that in most cases of interest, the recursive structure is not evident or unique, and its proper identification is part of the DP solution. To illustrate this idea, we present several examples.

Fibonacci Sequence

The Fibonacci sequence is defined by:

$$\begin{aligned}\mathcal{V}_0 &= 0 \\ \mathcal{V}_1 &= 1 \\ \mathcal{V}_t &= \mathcal{V}_{t-2} + \mathcal{V}_{t-1}.\end{aligned}$$

Our ‘problem’ is to calculate the T ’th number in the sequence, $\mathcal{V}_T$. Here, the recursive structure is easy to identify from the problem description, and a DP algorithm for computing $\mathcal{V}_T$ proceeds as follows:

1. Set $\mathcal{V}_0 = 0$, $\mathcal{V}_1 = 1$

2. For $t = 2, \dots, T$, set

$$\mathcal{V}_t = \mathcal{V}_{t-2} + \mathcal{V}_{t-1}.$$

Our choice of notation here matches the finite horizon DP problems in Chapter 3: the effective ‘size’ of the problem T is similar to the horizon length, and the quantity that we keep track of for each sub-problem $\mathcal{V}$ is similar to the value function. Note that by ordering the computation in increasing t , each element in the sequence is computed *exactly once*, and the complexity of this algorithm is therefore $O(T)$.

We will next discuss problems where the DP structure is less obvious.

Maximum Contiguous Sum

We are given a (long) sequence of T real numbers $x_1, x_2, \dots, x_T$, which could be positive or negative. Our goal is to find the maximal contiguous sum, namely,

$$\mathcal{V}^* = \max_{1 \leq t_1 \leq t_2 \leq T} \sum_{\ell=t_1}^{t_2} x_\ell.$$

An exhaustive search needs to examine $O(T^2)$ sums. We will now devise a more efficient DP solution. Let

$$\mathcal{V}_t = \max_{1 \leq t' \leq t} \sum_{\ell=t'}^t x_\ell$$

denote the maximal sum over all contiguous subsequences that end exactly at x_t . We have that:

$$\mathcal{V}_1 = x_1,$$

and

$$\mathcal{V}_t = \max\{\mathcal{V}_{t-1} + x_t, x_t\}.$$

Our DP algorithm thus proceeds as follows:

1. Set $\mathcal{V}_1 = x_1$, $\pi_1 = 1$
2. For $t = 2, \dots, T$, set

$$\begin{aligned} \mathcal{V}_t &= \max\{\mathcal{V}_{t-1} + x_t, x_t\}, \\ \pi_t &= \begin{cases} \pi_{t-1}, & \text{if } \mathcal{V}_{t-1} + x_t > x_t \\ t & \text{else.} \end{cases} \end{aligned}$$

3. Set $t^* = \arg \max_{1 \leq t \leq T} \mathcal{V}_t$

4. Return $\mathcal{V}^* = \mathcal{V}_{\mathbf{t}^*}$, $\mathbf{t}_{\text{start}} = \pi_{\mathbf{t}^*}$, $\mathbf{t}_{\text{end}} = \mathbf{t}^*$.

This algorithm requires only $O(T)$ calculations, i.e., linear time. Note also that in order to return the range of elements that make up the maximal contiguous sum $[\mathbf{t}_{\text{start}}, \mathbf{t}_{\text{end}}]$, we keep track of $\pi_{\mathbf{t}}$ – the index of the first element in the maximal sum that ends exactly at $x_{\mathbf{t}}$.

Longest Increasing Subsequence

We are given a sequence of T real numbers $x_1, x_2, \dots, x_T$. Our goal is to find the length of the longest strictly increasing subsequence (not necessarily contiguous). E.g, for the sequence $(3, 1, 5, 3, 4)$, the longest strictly increasing subsequence is $(1, 3, 4)$, and the solution is 3. Observe that the number of subsequences is 2^T , therefore an exhaustive search is inefficient.

We next develop a DP solution. Define $\mathcal{V}_{\mathbf{t}}$ to be the length of the longest strictly increasing subsequence ending at position $\mathbf{t}$. Then

$$\mathcal{V}_1 = 1,$$

$$\mathcal{V}_{\mathbf{t}} = \begin{cases} 1, & \text{if } x_{\mathbf{t}'} \geq x_{\mathbf{t}} \text{ for all } \mathbf{t}' < \mathbf{t}, \\ \max \{ \mathcal{V}_{\mathbf{t}'} : \mathbf{t}' < \mathbf{t}, x_{\mathbf{t}'} < x_{\mathbf{t}} \} + 1, & \text{else.} \end{cases}$$

The length of the longest subsequence is then $\mathcal{V}^* = \max_{1 \leq \mathbf{t} \leq T} (\mathcal{V}_{\mathbf{t}})$. Computing $\mathcal{V}_{\mathbf{t}}$ recursively gives the result with a running time of $O(T^2)$.¹

An Integer Knapsack Problem

We are given a knapsack (bag) of integer capacity $C > 0$, and a set of T items with respective sizes $\mathbf{s}_1, \dots, \mathbf{s}_T$ and values (worth) $\mathbf{r}_1, \dots, \mathbf{r}_T$. The sizes are positive and integer-valued. Our goal is to fill the knapsack to maximize the total value. That is, find the subset $A \subset \{1, \dots, T\}$ of items that maximizes

$$\sum_{\mathbf{t} \in A} \mathbf{r}_{\mathbf{t}},$$

subject to

$$\sum_{\mathbf{t} \in A} \mathbf{s}_{\mathbf{t}} \leq C.$$

Note that the number of item subsets is 2^T . We will now devise a DP solution. Let $\mathcal{V}(\mathbf{t}, \mathbf{t}')$ denote the maximal value for filling exactly capacity $\mathbf{t}'$ with items from

¹We note that this can be further improved to $O(T \log T)$. See Chapter 15 of [34].

the set $\{1, \dots, \mathfrak{t}\}$. If the capacity $\mathfrak{t}'$ cannot be matched by any such subset, set $\mathcal{V}(\mathfrak{t}, \mathfrak{t}') = -\infty$. Also set $\mathcal{V}(0, 0) = 0$, and $\mathcal{V}(0, \mathfrak{t}') = -\infty$ for $\mathfrak{t}' \geq 1$. Then

$$\mathcal{V}(\mathfrak{t}, \mathfrak{t}') = \max\{\mathcal{V}(\mathfrak{t} - 1, \mathfrak{t}'), \mathcal{V}(\mathfrak{t} - 1, \mathfrak{t}' - \mathfrak{s}_{\mathfrak{t}}) + \mathfrak{r}_{\mathfrak{t}}\},$$

which can be computed recursively for $\mathfrak{t} = 1 : T$, $\mathfrak{t}' = 1 : C$. The required value is obtained by $\mathcal{V}^* = \max_{0 \leq \mathfrak{t}' \leq C} \mathcal{V}(T, \mathfrak{t}')$. The running time of this algorithm is $O(TC)$. We note that the recursive computation of $\mathcal{V}(\mathfrak{t}, \mathfrak{t}')$ requires $O(C)$ space. To obtain the indices of the terms in the optimal subset some additional book-keeping is needed, which requires $O(TC)$ space.

Longest Common Subsequence

We are given two sequences (or strings) $X(1 : T_1)$, $Y(1 : T_2)$, of length T_1 and T_2 , respectively. We define a *subsequence* of X as the string that remains after deleting some number (zero or more) of elements of X . We wish to find the longest common subsequence (LCS) of X and Y , namely, a sequence of maximal length that is a subsequence of both X and Y . For example:

$$\begin{aligned} X &= \underline{A} \underline{V} \underline{B} \underline{V} \underline{A} \underline{M} \underline{C} \underline{D}, \\ Y &= \underline{A} \underline{Z} \underline{B} \underline{Q} \underline{A} \underline{C} \underline{L} \underline{D}. \end{aligned}$$

We next devise a DP solution. Let $\mathcal{V}(\mathfrak{t}_1, \mathfrak{t}_2)$ denote the length of an LCS of the prefix subsequences $X(1 : \mathfrak{t}_1)$, $Y(1 : \mathfrak{t}_2)$. Set $\mathcal{V}(\mathfrak{t}_1, \mathfrak{t}_2) = 0$ if $\mathfrak{t}_1 = 0$ or $\mathfrak{t}_2 = 0$. Then, for $\mathfrak{t}_1, \mathfrak{t}_2 > 0$, we have:

$$\mathcal{V}(\mathfrak{t}_1, \mathfrak{t}_2) = \begin{cases} \mathcal{V}(\mathfrak{t}_1 - 1, \mathfrak{t}_2 - 1) + 1 & : X(\mathfrak{t}_1) = Y(\mathfrak{t}_2) \\ \max\{\mathcal{V}(\mathfrak{t}_1, \mathfrak{t}_2 - 1), \mathcal{V}(\mathfrak{t}_1 - 1, \mathfrak{t}_2)\} & : X(\mathfrak{t}_1) \neq Y(\mathfrak{t}_2) \end{cases}$$

We can now compute $\mathcal{V}(T_1, T_2)$ recursively, using a row-first or column-first order, in $O(T_1 T_2)$ computations.

Further examples

Additional important DP problems include, among others:

- The Edit-Distance problem: find the distance (or similarity) between two strings, by counting the minimal number of “basic operations” that are needed to transform one string to another. A common set of basic operations is: delete character, add character, change character. This problem is frequently encountered in natural language processing and bio-informatics (e.g., DNA sequencing) applications, among others.

- The Matrix-Chain Multiplication problem: Find the optimal order to compute a matrix multiplication $M_1 M_2 \cdots M_n$ (for non-square matrices).

Appendix B

Ordinary Differential Equations

Ordinary differential equations (ODEs) are fundamental tools in mathematical modeling, used to describe dynamics and processes in various scientific fields. This chapter provides an introduction to ODEs, with a focus on linear systems of ODEs, their solutions, and stability analysis.

B.1 Definitions and Fundamental Results

An ordinary differential equation (ODE) is an equation that involves a function of one independent variable and its derivatives. The most general form of an ODE can be expressed as:

$$F(x, y, y', y'', \dots, y^{(n)}) = 0$$

where $y = y(x)$ is the unknown function, and $y', y'', \dots, y^{(n)}$ represent the first through n -th derivatives of y with respect to x .

A linear ODE has the form:

$$a_n(x)y^{(n)} + a_{n-1}(x)y^{(n-1)} + \dots + a_1(x)y' + a_0(x)y = g(x),$$

where $a_0, a_1, \dots, a_n$ and g are continuous functions on a given interval.

Typically, given an ODE, the goal is to find a set of functions y that solve it. We may also be interested in other properties of the set of solutions, such as their limits.

Example B.1. *Consider the first-order linear ODE*

$$y' = ay + b,$$

where $a \neq 0$ and b are constants. This equation can be solved using an integrating factor. The integrating factor, $\mu(x)$, is given by $\mu(x) = e^{-ax}$. Multiplying through by this integrating factor, the equation becomes:

$$e^{-ax}y' = ae^{-ax}y + be^{-ax}.$$

This simplifies to:

$$(e^{-ax}y)' = be^{-ax}.$$

Integrating both sides with respect to x gives:

$$e^{-ax}y = -\frac{b}{a}e^{-ax} + C,$$

where C is the constant of integration. Solving for y , we obtain:

$$y(x) = -\frac{b}{a} + Ce^{ax}.$$

Note that if $a < 0$, we have that $\lim_{x \rightarrow \infty} y(x) = -\frac{b}{a}$ for all the solutions of the ODE.

A fundamental result in ODE theory is the Picard-Lindelöf theorem, also known as the existence and uniqueness theorem.

Theorem B.1 (Existence and Uniqueness). *Consider the ODE given by*

$$y'(x) = f(x, y(x)), \quad y(x_0) = y_0,$$

where $f : [a, b] \times \mathbb{R} \rightarrow \mathbb{R}$ is a function. Assume that f satisfies the following conditions: (1) f is **continuous** on the domain $[a, b] \times \mathbb{R}$. (2) f satisfies a **Lipschitz condition** with respect to y in the domain, i.e., there exists a constant $L > 0$ such that $|f(x, y_1) - f(x, y_2)| \leq L|y_1 - y_2|$ for all $x \in [a, b]$ and $y_1, y_2 \in \mathbb{R}$. Then, there exists a **unique** function $y : [a, b] \rightarrow \mathbb{R}$ that solves the ODE on some interval containing x_0 .

The proof of this theorem involves constructing a sequence of approximate solutions using the method of successive approximations and showing that this sequence converges to the actual solution of the differential equation. For a detailed proof and further exploration of this theorem, refer to classic texts on differential equations such as [58].

B.1.1 Systems of Linear Differential Equations

When dealing with multiple interdependent variables, we can extend the concept of linear ODEs to systems of equations. These are particularly useful in modeling multiple phenomena that influence each other.

Consider a system of linear differential equations represented in matrix form as follows:

$$\mathbf{y}' = A\mathbf{y} + \mathbf{b}, \quad (\text{B.1})$$

where $\mathbf{y}$ is a vector of N unknown functions, $A \in \mathbb{R}^{N \times N}$ is a matrix of coefficients, and $\mathbf{b} \in \mathbb{R}^N$ is a vector of constants. This compact form encapsulates a system where each derivative of the component functions in $\mathbf{y}$ depends linearly on all other functions in $\mathbf{y}$ and possibly some external inputs $\mathbf{b}$. We shall now present the general solution to the ODE in Equation (B.1).

Let us first define the matrix exponential.

Definition B.1. *The matrix exponential, e^{Ax} , where A is a matrix, is defined similarly to the scalar exponential function but extended to matrices,*

$$e^{Ax} = \sum_{k=0}^{\infty} \frac{x^k A^k}{k!}.$$

The matrix exponential is fundamental in systems theory and control engineering, as it provides a straightforward method for solving linear systems of differential equations.

Proposition B.2. *The solutions of the system of linear differential equations in (B.1) are given by $\mathbf{y}(x) = e^{Ax}\mathbf{y}_0 + \mathbf{y}_p$, where $\mathbf{y}_p$ is such that $A\mathbf{y}_p = -\mathbf{b}$.*

Proof. Let us first consider the homogeneous case $\mathbf{b} = 0$. To prove that $\mathbf{y}(x) = e^{Ax}\mathbf{y}_0$ solves $\mathbf{y}' = A\mathbf{y}$, we differentiate $\mathbf{y}(x)$ with respect to x and show that it satisfies the differential equation $\mathbf{y}' = A\mathbf{y}$.

The derivative of $\mathbf{y}(x)$ with respect to x is given by:

$$\frac{d}{dx}\mathbf{y}(x) = \frac{d}{dx}(e^{Ax}\mathbf{y}_0).$$

Applying the derivative to the series expansion of e^{Ax} , we get:

$$\frac{d}{dx}(e^{Ax}) = \frac{d}{dx} \left(\sum_{k=0}^{\infty} \frac{(Ax)^k}{k!} \right) = \sum_{k=0}^{\infty} \frac{d}{dx} \left(\frac{(Ax)^k}{k!} \right).$$

Using the power rule and the properties of matrix multiplication, we find:

$$\sum_{k=1}^{\infty} \frac{A(Ax)^{k-1}k}{k!} = A \sum_{k=1}^{\infty} \frac{(Ax)^{k-1}}{(k-1)!} = Ae^{Ax}.$$

Therefore,

$$\frac{d}{dx}\mathbf{y}(x) = Ae^{Ax}\mathbf{y}_0.$$

Substituting $\mathbf{y}(x)$ back into the original differential equation:

$$\mathbf{y}' = A\mathbf{y} \quad \Rightarrow \quad Ae^{Ax}\mathbf{y}_0 = A\mathbf{y}.$$

Since $\mathbf{y}(x) = e^{Ax}\mathbf{y}_0$, it follows that:

$$Ae^{Ax}\mathbf{y}_0 = A\mathbf{y}(x).$$

To show that $e^{Ax}\mathbf{y}_0$ is the only possible solution, note that at $x = 0$, $e^{Ax}\mathbf{y}_0 = \mathbf{y}_0$. Therefore, for any initial condition, we have found a solution, and the uniqueness follows from Theorem B.1.

Now, for the case $\mathbf{b} \neq 0$, let $\mathbf{y}_p$ such that $A\mathbf{y}_p = -\mathbf{b}$. We have that for $\mathbf{y}(x) = e^{Ax}\mathbf{y}_0 + \mathbf{y}_p$,

$$\mathbf{y}'(x) = Ae^{Ax}\mathbf{y}_0 = Ae^{Ax}\mathbf{y}_0 + A\mathbf{y}_p - A\mathbf{y}_p = A\mathbf{y}(x) + \mathbf{b}.$$

□

B.2 Asymptotic Stability

We will be interested in the asymptotic behavior of ODE solutions. In particular, we shall be interested in the following stability definitions.

Definition B.2 (Stability). *A solution $y(x)$ of a differential equation is called **stable** if, for every $\epsilon > 0$, there exists a $\delta > 0$ such that for any other solution $\tilde{y}(x)$ with $|\tilde{y}(0) - y(0)| < \delta$, it holds that $|\tilde{y}(x) - y(x)| < \epsilon$ for all $x \geq 0$.*

Definition B.3 (Asymptotic Stability). *A solution $y(x)$ of a differential equation is called **asymptotically stable** if it is stable and additionally, there exists a $\delta > 0$ such that if $|\tilde{y}(0) - y(0)| < \delta$, then $\lim_{x \rightarrow \infty} |\tilde{y}(x) - y(x)| = 0$.*

Intuitively, asymptotic stability means that not only do perturbations remain small, but they also decay to zero as x progresses, causing the perturbed solutions to eventually converge to the stable solution. A stronger condition is global asymptotic stability, under which the system converges for every initial condition.

Definition B.4 (Global Asymptotic Stability). *A solution $y(x)$ of a differential equation is called **globally asymptotically stable** if for any other solution $\tilde{y}(x)$, we have $\lim_{x \rightarrow \infty} |\tilde{y}(x) - y(x)| = 0$.*

We have the following result for the system of linear differential equations in (B.1).

Theorem B.3. *Consider the ODE in Equation (B.1), and let $A \in \mathbb{R}^{N \times N}$ be diagonalizable. If all the eigenvalues of A have a negative real part, and let $\mathbf{y}_p$ such that $A\mathbf{y}_p = -\mathbf{b}$, then $y = \mathbf{y}_p$ is a globally asymptotically stable solution.*

Proof. We have already established that every solution is of the form $\mathbf{y}(x) = e^{Ax}\mathbf{y}_0 + \mathbf{y}_p$. Let λ_i, v_i denote the eigenvalues and eigenvectors of A . Since A is diagonalizable, we can write $Ay_0 = \sum_{i=1}^N \lambda_i v_i^T y_0$, so

$$e^{Ax}y_0 = \sum_{k=0}^{\infty} \frac{x^k A^k y_0}{k!} = \sum_{k=0}^{\infty} \sum_{i=1}^N \frac{x^k \lambda_i^k v_i^T y_0}{k!} = \sum_{i=1}^N e^{\lambda_i x} v_i^T y_0.$$

If λ_i has a negative real part, then $\lim_{x \rightarrow \infty} e^{\lambda_i x} = 0$. Thus, if all the eigenvalues of A have a negative real part, $\lim_{x \rightarrow \infty} e^{Ax}y_0 = 0$ for all y_0 , and the claim follows. $\square$

A similar result can be shown to hold for general (not necessarily diagonalizable) matrices. We state here a general theorem (see, e.g., Theorem 4.5 in [77]) without proof.

Theorem B.4. *Consider the ODE $\mathbf{y}' = A\mathbf{y}$, where $A \in \mathbb{R}^{N \times N}$. The solution $y = 0$ is globally asymptotically stable if and only if all the eigenvalues of A have a negative real part.*

Bibliography

- [1] Alekh Agarwal, Nan Jiang, Sham M Kakade, and Wen Sun. Reinforcement learning: Theory and algorithms. *CS Dept., UW Seattle, Seattle, WA, USA, Tech. Rep*, 32:96, 2019.
- [2] Alekh Agarwal, Sham M. Kakade, and Lin F. Yang. Model-based reinforcement learning with a generative model is minimax optimal. In Jacob D. Abernethy and Shivani Agarwal, editors, *Conference on Learning Theory, COLT*, 2020.
- [3] Eitan Altman. *Constrained Markov decision processes*. Routledge, 2021.
- [4] K.J. Åström and B. Wittenmark. *Adaptive Control*. Dover Books on Electrical Engineering. Dover Publications, 2008.
- [5] Peter Auer, Nicolò Cesa-Bianchi, and Paul Fischer. Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47(2-3):235–256, 2002.
- [6] Peter Auer, Thomas Jaksch, and Ronald Ortner. Near-optimal regret bounds for reinforcement learning. *Advances in neural information processing systems*, 21, 2008.
- [7] Mohammad Gheshlaghi Azar, Rémi Munos, and Hilbert J. Kappen. Minimax PAC bounds on the sample complexity of reinforcement learning with a generative model. *Machine Learning*, 91(3):325–349, 2013.
- [8] Andrew G. Barto and Michael O. Duff. Monte carlo matrix inversion and reinforcement learning. In Jack D. Cowan, Gerald Tesauro, and Joshua Alspecter, editors, *Advances in Neural Information Processing Systems 6*, pages 687–694. Morgan Kaufmann, 1993.
- [9] Andrew G Barto, Richard S Sutton, and Charles W Anderson. Neuronlike adaptive elements that can solve difficult learning control problems. *IEEE transactions on systems, man, and cybernetics*, 13(5):834–846, 1983.

- [10] Jonathan Baxter and Peter L. Bartlett. Infinite-horizon policy-gradient estimation. *J. Artificial Intelligence Research*, 15:319–350, 2001.
- [11] Jonathan Baxter and Peter L Bartlett. Infinite-horizon policy-gradient estimation. *journal of artificial intelligence research*, 15:319–350, 2001.
- [12] Amir Beck. *First-Order Methods in Optimization*. SIAM, Philadelphia, 2017.
- [13] Jacob Beck, Risto Vuorio, Evan Zheran Liu, Zheng Xiong, Luisa Zintgraf, Chelsea Finn, and Shimon Whiteson. A survey of meta-reinforcement learning. *arXiv preprint arXiv:2301.08028*, 2023.
- [14] Richard Bellman. *Dynamic Programming*. Dover Publications, 1957.
- [15] Richard Bellman and Stuart Dreyfus. Functional approximations and dynamic programming. *Mathematical Tables and Other Aids to Computation*, pages 247–251, 1959.
- [16] Alberto Bemporad and Manfred Morari. Control of systems integrating logic, dynamics, and constraints. *Automatica*, 35(3):407–427, 1999.
- [17] Albert Benveniste, Michel Métivier, and Pierre Priouret. *Adaptive algorithms and stochastic approximations*, volume 22. Springer Science & Business Media, 2012.
- [18] D. P. Bertsekas and J. N. Tsitsiklis. *Neuro-dynamic programming*. Athena Scientific, 1996.
- [19] Dimitri P. Bertsekas. *Dynamic programming and optimal control, 3rd Edition*. Athena Scientific, 2005.
- [20] Jalaj Bhandari and Daniel Russo. Global optimality guarantees for policy gradient methods. *Operations Research*, 72(5):1906–1927, 2024.
- [21] Shalabh Bhatnagar, Mohammad Ghavamzadeh, Mark Lee, and Richard S Sutton. Incremental natural actor-critic algorithms. *Advances in neural information processing systems*, 20, 2007.
- [22] Vivek S Borkar. *Stochastic approximation: a dynamical systems viewpoint*, volume 48. Springer, 2009.
- [23] Justin A Boyan. Technical update: Least-squares temporal difference learning. *Machine learning*, 49:233–246, 2002.

- [24] Steven J Bradtke and Andrew G Barto. Linear least-squares algorithms for temporal difference learning. *Machine learning*, 22(1):33–57, 1996.
- [25] Ronen I. Brafman and Moshe Tennenholtz. R-MAX - A general polynomial time algorithm for near-optimal reinforcement learning. *Journal of Machine Learning Research*, 3:213–231, 2002.
- [26] Apostolos N Burnetas and Michael N Katehakis. Optimal adaptive policies for Markov decision processes. *Mathematics of Operations Research*, 22(1):222–255, 1997.
- [27] Murray Campbell, A. Joseph Hoane, and Feng hsiung Hsu. Deep blue. *Artificial Intelligence*, 134(1):57–83, 2002.
- [28] Dotan Di Castro and Ron Meir. A convergent online single time scale actor critic algorithm. *The Journal of Machine Learning Research*, 11:367–410, 2010.
- [29] Nicolò Cesa-Bianchi and Gábor Lugosi. *Prediction, learning, and games*. Cambridge University Press, 2006.
- [30] Mmanu Chaturvedi and Ross M. McConnell. A note on finding minimum mean cycle. *Inf. Process. Lett.*, 127:21–22, 2017.
- [31] Lili Chen, Kevin Lu, Aravind Rajeswaran, Kimin Lee, Aditya Grover, Misha Laskin, Pieter Abbeel, Aravind Srinivas, and Igor Mordatch. Decision transformer: Reinforcement learning via sequence modeling. *Advances in neural information processing systems*, 34:15084–15097, 2021.
- [32] Vášek Chvátal. *Linear programming*. Macmillan, 1983.
- [33] Thomas H Cormen, Charles E Leiserson, Ronald L Rivest, and Clifford Stein. *Introduction to algorithms*. MIT press, 2009.
- [34] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms, 3rd Edition*. MIT Press, 2009.
- [35] Rémi Coulom. Efficient selectivity and backup operators in monte-carlo tree search. In *International conference on computers and games*, pages 72–83. Springer, 2006.
- [36] Christoph Dann and Emma Brunskill. Sample complexity of episodic fixed-horizon reinforcement learning. In *Neural Information Processing Systems (NeurIPS)*, 2015.

- [37] Sanjoy Dasgupta, Christos H. Papadimitriou, and Umesh V. Vazirani. *Algorithms*. McGraw-Hill, 2008.
- [38] Peter Dayan. The convergence of $\text{td}(\lambda)$ for general λ . *Machine Learning*, 8:341–362, 1992.
- [39] Peter Dayan and Terrence J. Sejnowski. $\text{Td}(\lambda)$ converges with probability 1. *Machine Learning*, 14(1):295–301, 1994.
- [40] Daniela Pucci de Farias and Benjamin Van Roy. The linear programming approach to approximate dynamic programming. *Oper. Res.*, 51(6):850–865, 2003.
- [41] Vijay Desai, Vivek Farias, and Ciamac C Moallemi. A smoothed approximate linear program. *Advances in Neural Information Processing Systems*, 22, 2009.
- [42] EW Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1:269–271, 1959.
- [43] Yonathan Efroni, Gal Dalal, Bruno Scherrer, and Shie Mannor. Beyond the one-step greedy approach in reinforcement learning. In *International Conference on Machine Learning*, pages 1387–1396. PMLR, 2018.
- [44] Damien Ernst, Mevludin Glavic, Pierre Geurts, and Louis Wehenkel. Approximate value iteration in the reinforcement learning context. application to electrical power system control. *International Journal of Emerging Electric Power Systems*, 3(1), 2005.
- [45] Eyal Even-Dar, Shie Mannor, and Yishay Mansour. Action elimination and stopping conditions for the multi-armed bandit and reinforcement learning problems. *J. Machine Learning Research*, 7:1079–1105, 2006.
- [46] Eyal Even-Dar and Yishay Mansour. Learning rates for q-learning. *Journal of Machine Learning Research*, 5:1–25, 2003.
- [47] John Fearnley. Exponential lower bounds for policy iteration. In *Automata, Languages and Programming (ICALP)*, volume 6199, pages 551–562, 2010.
- [48] Claude-Nicolas Fiechter. Efficient reinforcement learning. In *Computational Learning Theory (COLT)*, 1994.
- [49] Michael C Fu. Optimization via simulation: A review. *Annals of operations research*, 53:199–247, 1994.

- [50] Mohammad Ghavamzadeh, Shie Mannor, Joelle Pineau, Aviv Tamar, et al. Bayesian reinforcement learning: A survey. *Foundations and Trends® in Machine Learning*, 8(5-6):359–483, 2015.
- [51] Peter W Glynn. Likelihood ratio gradient estimation for stochastic systems. *Communications of the ACM*, 33(10):75–84, 1990.
- [52] Geoffrey J Gordon. Stable function approximation in dynamic programming. In *Machine learning proceedings 1995*, pages 261–268. Elsevier, 1995.
- [53] Evan Greensmith, Peter L Bartlett, and Jonathan Baxter. Variance reduction techniques for gradient estimates in reinforcement learning. *Journal of Machine Learning Research*, 5(9), 2004.
- [54] Assaf Hallak, Dotan Di Castro, and Shie Mannor. Contextual Markov decision processes. *arXiv preprint arXiv:1502.02259*, 2015.
- [55] Thomas Dueholm Hansen, Peter Bro Miltersen, and Uri Zwick. Strategy iteration is strongly polynomial for 2-player turn-based stochastic games with a constant discount factor. *J. ACM*, 60(1):1:1–1:16, 2013.
- [56] Peter E Hart, Nils J Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968.
- [57] Elad Hazan. Introduction to online convex optimization. *Foundations and Trends® in Optimization*, 2(3-4):157–325, 2016.
- [58] Morris W Hirsch, Stephen Smale, and Robert L Devaney. *Differential equations, dynamical systems, and an introduction to chaos*. Academic press, 2013.
- [59] Romain Hollanders, Jean-Charles Delvenne, and Raphaël M. Jungers. The complexity of policy iteration is exponential for discounted Markov decision processes. In *Proceedings of the 51th IEEE Conference on Decision and Control (CDC)*, pages 5997–6002, 2012.
- [60] R. A. Howard. *Dynamic Programming and Markov Processes*. MIT Press, 1960.
- [61] Jiawei Huang and Nan Jiang. From importance sampling to doubly robust policy gradient. In *International Conference on Machine Learning*, pages 4434–4443. PMLR, 2020.

- [62] Tommi S. Jaakkola, Michael I. Jordan, and Satinder P. Singh. On the convergence of stochastic iterative dynamic programming algorithms. *Neural Comput.*, 6(6):1185–1201, 1994.
- [63] Donald E. Jacobson and David Q. Mayne. *Differential Dynamic Programming*. American Elsevier Publishing Company, New York, 1970.
- [64] Tang Jie and Pieter Abbeel. On a connection between importance sampling and the likelihood ratio policy gradient. *Advances in Neural Information Processing Systems*, 23, 2010.
- [65] Chi Jin, Akshay Krishnamurthy, Max Simchowitz, and Tiancheng Yu. Reward-free exploration for reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2020.
- [66] Leslie Pack Kaelbling. Learning to achieve goals. In *IJCAI*, volume 2, pages 1094–8. Citeseer, 1993.
- [67] Sham Kakade. *On the sample complexity of reinforcement learning*. PhD thesis, University College London, 2003.
- [68] Sham Kakade and John Langford. Approximately optimal approximate reinforcement learning. In *Proceedings of the Nineteenth International Conference on Machine Learning*, pages 267–274, 2002.
- [69] Samuel Karlin and Howard M. Taylor. *A First Course in Stochastic Processes*. Academic Press, 2nd edition, 1975.
- [70] Howard Karloff. *Linear programming*. Birkhauser Boston Inc., USA, 1991.
- [71] Richard M. Karp. A characterization of the minimum cycle mean in a digraph. *Discret. Math.*, 23(3):309–311, 1978.
- [72] Elia Kaufmann, Leonard Bauersfeld, Antonio Loquercio, Matthias Müller, Vladlen Koltun, and Davide Scaramuzza. Champion-level drone racing using deep reinforcement learning. *Nature*, 620(7976):982–987, 2023.
- [73] Emilie Kaufmann, Pierre Ménard, Omar Darwiche Domingues, Anders Jonsson, Edouard Leurent, and Michal Valko. Adaptive reward-free exploration. In *Algorithmic Learning Theory (ALT)*, 2021.
- [74] Michael J Kearns and Satinder Singh. Bias-variance error bounds for temporal difference updates. In *COLT*, pages 142–147, 2000.

- [75] Michael J. Kearns and Satinder P. Singh. Finite-sample convergence rates for q-learning and indirect algorithms. In *Advances in Neural Information Processing Systems*, pages 996–1002, 1998.
- [76] Michael J. Kearns and Satinder P. Singh. Near-optimal reinforcement learning in polynomial time. *Machine Learning*, 49(2-3):209–232, 2002.
- [77] H.K. Khalil. *Nonlinear Systems*. Pearson Education. Prentice Hall, 2002.
- [78] IS Khalil, JC Doyle, and K Glover. *Robust and optimal control*, volume 2. Prentice hall, 1996.
- [79] Khimya Khetarpal, Matthew Riemer, Irina Rish, and Doina Precup. Towards continual reinforcement learning: A review and perspectives. *Journal of Artificial Intelligence Research*, 75:1401–1476, 2022.
- [80] Donald E Kirk. *Optimal control theory: an introduction*. Courier Corporation, 2004.
- [81] Robert Kirk, Amy Zhang, Edward Grefenstette, and Tim Rocktäschel. A survey of zero-shot generalisation in deep reinforcement learning. *Journal of Artificial Intelligence Research*, 76:201–264, 2023.
- [82] Jon Kleinberg and Éva Tardos. *Algorithm Design*. Addison Wesley, 2006.
- [83] Levente Kocsis and Csaba Szepesvári. Bandit based monte-carlo planning. In *European conference on machine learning*, pages 282–293. Springer, 2006.
- [84] Vijay Konda and John Tsitsiklis. Actor-critic algorithms. *Advances in neural information processing systems*, 12, 1999.
- [85] H. J. Kushner. *Approximation and Weak Convergence Methods for Random Processes*. MIT press Cambridge, MA, 1984.
- [86] Harold J. Kushner and G. George Yin. *Stochastic Approximation and Recursive Algorithms and Applications*, volume 35 of *Stochastic Modelling and Applied Probability*. Springer, New York, 2 edition, 2003.
- [87] H.J. Kushner and D.S. Clark. *Stochastic Approximation Methods for Constrained and Unconstrained Systems*. Springer-Verlag, New York, 1978.
- [88] Michail G Lagoudakis and Ronald Parr. Least-squares policy iteration. *Journal of machine learning research*, 4(Dec):1107–1149, 2003.

- [89] Michail G Lagoudakis and Ronald Parr. Reinforcement learning as classification: Leveraging modern classifiers. In *Proceedings of the 20th International Conference on Machine Learning (ICML-03)*, pages 424–431, 2003.
- [90] T.L Lai and H. Robbins. Asymptotically efficient adaptive allocation rules. *Advances in Applied Mathematics*, 6(1):4–22, 1985.
- [91] Brenden M Lake, Tomer D Ullman, Joshua B Tenenbaum, and Samuel J Gershman. Building machines that learn and think like people. *Behavioral and brain sciences*, 40:e253, 2017.
- [92] Abdul Latif. Banach contraction principle and its generalizations. *Topics in fixed point theory*, pages 33–64, 2014.
- [93] Tor Lattimore and Marcus Hutter. Near-optimal PAC bounds for discounted mdps. *Theor. Comput. Sci.*, 558:125–143, 2014.
- [94] Tor Lattimore and Csaba Szepesvári. *Bandit Algorithms*. Cambridge University Press, 2020.
- [95] Alessandro Lazaric, Mohammad Ghavamzadeh, and Rémi Munos. Analysis of a classification-based policy iteration algorithm. In *ICML-27th International Conference on Machine Learning*, pages 607–614. Omnipress, 2010.
- [96] David A Levin and Yuval Peres. *Markov chains and mixing times*, volume 107. American Mathematical Soc., 2017.
- [97] Sergey Levine, Chelsea Finn, Trevor Darrell, and Pieter Abbeel. End-to-end training of deep visuomotor policies. *Journal of Machine Learning Research*, 17(39):1–40, 2016.
- [98] Lihong Li. *Sample Complexity Bounds of Exploration*, pages 175–204. Springer Berlin Heidelberg, Berlin, Heidelberg, 2012.
- [99] L. Ljung. Analysis of recursive stochastic algorithms. *IEEE Transactions on Automatic Control*, 22(4):551–575, 1977.
- [100] L. Ljung and T. Söderström. *Theory and practice of recursive identification*. MIT press Cambridge, MA, 1983.
- [101] Omid Madani, Mikkel Thorup, and Uri Zwick. Discounted deterministic Markov decision processes and discounted all-pairs shortest paths. *ACM Trans. Algorithms*, 6(2):33:1–33:25, 2010.

- [102] Shie Mannor and Nahum Shimkin. A geometric approach to multi-criterion reinforcement learning. *The Journal of Machine Learning Research*, 5:325–360, 2004.
- [103] Shie Mannor and John N Tsitsiklis. Algorithmic aspects of mean-variance optimization in Markov decision processes. *European Journal of Operational Research*, 231(3):645–653, 2013.
- [104] Yishay Mansour and Satinder Singh. On the complexity of policy iteration. In *Conference on Uncertainty in Artificial Intelligence (UAI)*, pages 401–408, 1999.
- [105] Peter Marbach and John N Tsitsiklis. Simulation-based optimization of Markov reward processes. *IEEE Transactions on Automatic Control*, 46(2):191–209, 2002.
- [106] Peter Marbach and John N. Tsitsiklis. Approximate gradient methods in policy-space optimization of Markov reward processes. *Discret. Event Dyn. Syst.*, 13(1-2):111–148, 2003.
- [107] Mary Melekopoglou and Anne Condon. On the complexity of the policy improvement algorithm for Markov decision processes. *INFORMS J. Comput.*, 6(2):188–192, 1994.
- [108] Pierre Ménard, Omar Darwiche Domingues, Anders Jonsson, Emilie Kaufmann, Edouard Leurent, and Michal Valko. Fast active learning for pure exploration in reinforcement learning. In *International Conference on Machine Learning (ICML)*, 2021.
- [109] N. Metropolis and S. Ulam. The monte carlo method. *Journal of the American Statistical Association*, 44:335–341, 1949.
- [110] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Andrei A Rusu, Joel Veness, Marc G Bellemare, Alex Graves, Martin Riedmiller, Andreas K Fidjeland, Georg Ostrovski, et al. Human-level control through deep reinforcement learning. *Nature*, 518(7540):529–533, 2015.
- [111] Mehryar Mohri, Afshin Rostamizadeh, and Ameet Talwalkar. *Foundations of Machine Learning*. MIT Press, Cambridge, MA, 2 edition, 2018.
- [112] Rémi Munos. Performance bounds in l_p -norm for approximate value iteration. *SIAM journal on control and optimization*, 46(2):541–561, 2007.

- [113] Andrew Y Ng, Daishi Harada, and Stuart Russell. Policy invariance under reward transformations: Theory and application to reward shaping. In *International Conference on Machine Learning*, volume 99, pages 278–287, 1999.
- [114] Andrew Y Ng, Stuart Russell, et al. Algorithms for inverse reinforcement learning. In *ICML*, pages 663–670, 2000.
- [115] Arnab Nilim and Laurent El Ghaoui. Robust control of Markov decision processes with uncertain transition matrices. *Operations Research*, 53(5):780–798, 2005.
- [116] Dirk Ormoneit and Šaunak Sen. Kernel-based reinforcement learning. *Machine learning*, 49:161–178, 2002.
- [117] Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, et al. Training language models to follow instructions with human feedback. *Advances in neural information processing systems*, 35:27730–27744, 2022.
- [118] Vijay V. Phansalkar and M. A. L. Thathachar. Local and global optimization algorithms for generalized learning automata. *Neural Comput.*, 7(5):950–973, 1995.
- [119] Ian Post and Yinyu Ye. The simplex method is strongly polynomial for deterministic Markov decision processes. In *Symposium on Discrete Algorithms (SODA)*, pages 1465–1473. SIAM, 2013.
- [120] Warren B Powell. *Approximate Dynamic Programming: Solving the curses of dimensionality*, volume 703. John Wiley & Sons, 2007.
- [121] Martin L Puterman. *Markov decision processes: discrete stochastic dynamic programming*. John Wiley & Sons, 2014.
- [122] Martin Riedmiller. Neural fitted q iteration—first experiences with a data efficient neural reinforcement learning method. In *Machine learning: ECML 2005: 16th European conference on machine learning, Porto, Portugal, October 3-7, 2005. proceedings 16*, pages 317–328. Springer, 2005.
- [123] H. Robbins. Some aspects of the sequential design of experiments. *Bulletin of the American Mathematical Society*, 58(5):527–535, 1952.

- [124] Herbert Robbins and Sutton Monro. A Stochastic Approximation Method. *The Annals of Mathematical Statistics*, 22(3):400 – 407, 1951.
- [125] Gavin A Rummery and Mahesan Niranjan. *On-line Q-learning using connectionist systems*, volume 37. University of Cambridge, Department of Engineering Cambridge, UK, 1994.
- [126] Stuart J Russell and Peter Norvig. *Artificial intelligence: a modern approach*. Pearson, 2016.
- [127] Arthur L. Samuel. Artificial intelligence - a frontier of automation. *Elektron. Rechenanlagen*, 4(4):173–177, 1962.
- [128] Herbert Scarf. The optimality of (s, S) policies in the dynamic inventory problem. In Kenneth J. Arrow, Samuel Karlin, and Patrick Suppes, editors, *Mathematical Methods in the Social Sciences*, chapter 13, pages 196–202. Stanford University Press, Stanford, CA, 1959.
- [129] Bruno Scherrer and Matthieu Geist. Local policy search in a convex space and conservative policy iteration as boosted policy search. In *Machine Learning and Knowledge Discovery in Databases: European Conference, ECML PKDD 2014, Nancy, France, September 15-19, 2014. Proceedings, Part III 14*, pages 35–50. Springer, 2014.
- [130] Bruno Scherrer, Mohammad Ghavamzadeh, Victor Gabillon, Boris Lesner, and Matthieu Geist. Approximate modified policy iteration and its application to the game of tetris. *J. Machine Learning Research*, 16(49):1629–1676, 2015.
- [131] John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations (ICLR)*, 2016.
- [132] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [133] Shai Shalev-Shwartz and Shai Ben-David. *Understanding machine learning: From theory to algorithms*. Cambridge university press, 2014.

- [134] Lior Shani, Yonathan Efroni, and Shie Mannor. Adaptive trust region policy optimization: Global convergence and faster rates for regularized MDPs. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 5668–5675, 2020.
- [135] Claude E Shannon. Xxii. programming a computer for playing chess. *The London, Edinburgh, and Dublin Philosophical Magazine and Journal of Science*, 41(314):256–275, 1950.
- [136] L. S. Shapley. Stochastic games. *Proc Natl Acad Sci USA*, 39:1095–1100, 1953.
- [137] David Silver. UCL course on RL, 2015. <https://www.davidsilver.uk/teaching/>.
- [138] David Silver, Aja Huang, Chris J Maddison, Arthur Guez, Laurent Sifre, George Van Den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Veda Panneershelvam, Marc Lanctot, et al. Mastering the game of go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016.
- [139] David Silver, Aja Huang, Chris J. Maddison, Arthur Guez, Laurent Sifre, George van den Driessche, Julian Schrittwieser, Ioannis Antonoglou, Vedavyas Panneershelvam, Marc Lanctot, Sander Dieleman, Dominik Grewe, John Nham, Nal Kalchbrenner, Ilya Sutskever, Timothy P. Lillicrap, Madeleine Leach, Koray Kavukcuoglu, Thore Graepel, and Demis Hassabis. Mastering the game of Go with deep neural networks and tree search. *Nature*, 529(7587):484–489, 2016.
- [140] David Silver, Thomas Hubert, Julian Schrittwieser, Ioannis Antonoglou, Matthew Lai, Arthur Guez, Marc Lanctot, Laurent Sifre, Dhharshan Kumaran, Thore Graepel, et al. A general reinforcement learning algorithm that masters chess, shogi, and go through self-play. *Science*, 362(6419):1140–1144, 2018.
- [141] David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, et al. Mastering the game of go without human knowledge. *Nature*, 550(7676):354–359, 2017.
- [142] Satinder Singh, Tommi S. Jaakkola, Michael L. Littman, and Csaba Szepesvári. Convergence results for single-step on-policy reinforcement-learning algorithms. *Machine Learning*, 38(3):287–308, 2000.

- [143] Satinder P. Singh and Richard S. Sutton. Reinforcement learning with replacing eligibility traces. *Machine Learning*, 22(1-3):123–158, 1996.
- [144] Aleksandrs Slivkins. Introduction to multi-armed bandits. *Found. Trends Machine Learning*, 12(1-2):1–286, 2019.
- [145] Rupesh Kumar Srivastava, Pranav Shyam, Filipe Mutz, Wojciech Jaśkowski, and Jürgen Schmidhuber. Training agents using upside-down reinforcement learning. *arXiv preprint arXiv:1912.02877*, 2019.
- [146] Alexander L. Strehl, Lihong Li, and Michael L. Littman. Reinforcement learning in finite MDPs: PAC analysis. *Journal of Machine Learning Research*, 10:2413–2444, 2009.
- [147] Alexander L. Strehl and Michael L. Littman. An analysis of model-based interval estimation for Markov decision processes. *J. Comput. Syst. Sci.*, 74(8):1309–1331, 2008.
- [148] Richard S. Sutton. Learning to predict by the methods of temporal differences. *Machine Learning*, 3:9–44, 1988.
- [149] Richard S. Sutton and Andrew G. Barto. *Reinforcement learning - an introduction*. Adaptive computation and machine learning. MIT Press, 1998.
- [150] Richard S Sutton, Andrew G Barto, and Ronald J Williams. Reinforcement learning is direct adaptive optimal control. *IEEE control systems magazine*, 12(2):19–22, 1992.
- [151] Richard S. Sutton, David A. McAllester, Satinder Singh, and Yishay Mansour. Policy gradient methods for reinforcement learning with function approximation. In *NIPS*, pages 1057–1063, 1999.
- [152] Csaba Szepesvári. *Algorithms for Reinforcement Learning*. Synthesis Lectures on Artificial Intelligence and Machine Learning. Morgan & Claypool Publishers, 2010.
- [153] Istvan Szita and András Lörincz. Optimistic initialization and greediness lead to polynomial time learning in factored mdps. In Andrea Pohoreckýj Danyluk, Léon Bottou, and Michael L. Littman, editors, *International Conference on Machine Learning (ICML)*, 2009.

- [154] Istvan Szita and Csaba Szepesvári. Model-based reinforcement learning with nearly tight exploration complexity bounds. In *International Conference on Machine Learning (ICML)*, 2010.
- [155] Aviv Tamar, Daniel Soudry, and Ev Zisselman. Regularization guarantees generalization in bayesian reinforcement learning through algorithmic stability. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 8423–8431, 2022.
- [156] Matthew E Taylor and Peter Stone. Transfer learning for reinforcement learning domains: A survey. *Journal of Machine Learning Research*, 10(7), 2009.
- [157] Gerald Tesauro. Temporal difference learning and td-gammon. *Commun. ACM*, 38(3):58–68, 1995.
- [158] Gerald Tesauro. Programming backgammon using self-teaching neural nets. *Artificial Intelligence*, 134(1-2):181–199, 2002.
- [159] Josh Tobin, Rachel Fong, Alex Ray, Jonas Schneider, Wojciech Zaremba, and Pieter Abbeel. Domain randomization for transferring deep neural networks from simulation to the real world. In *2017 IEEE/RSJ international conference on intelligent robots and systems (IROS)*, pages 23–30. IEEE, 2017.
- [160] Emanuel Todorov and Weiwei Li. A generalized iterative lqg method for locally-optimal feedback control of constrained nonlinear stochastic systems. In *Proceedings of the 2005, American Control Conference, 2005.*, pages 300–306. IEEE, 2005.
- [161] J. Tsitsiklis and B. Van Roy. An analysis of temporal-difference learning with function approximation. *IEEE Trans. on Automatic Control*, 42(5):674–690, 1997.
- [162] John Tsitsiklis and Benjamin Van Roy. Analysis of temporal-difference learning with function approximation. *Advances in neural information processing systems*, 9, 1996.
- [163] John N. Tsitsiklis. Asynchronous stochastic approximation and Q-learning. *Machine Learning*, 16(3):185–202, 1994.
- [164] John N Tsitsiklis and Benjamin Van Roy. Feature-based methods for large scale dynamic programming. *Machine Learning*, 22(1):59–94, 1996.

- [165] AW van der Vaart, A.W. van der Vaart, A. van der Vaart, and J. Wellner. *Weak Convergence and Empirical Processes: With Applications to Statistics*. Springer Series in Statistics. Springer, 1996.
- [166] Harm van Seijen, Ashique Rupam Mahmood, Patrick M. Pilarski, Marlos C. Machado, and Richard S. Sutton. True online temporal-difference learning. *J. Machine Learning Research*, 17:145:1–145:40, 2016.
- [167] Harm van Seijen and Richard S. Sutton. True online TD(λ). In *Proceedings of the 31th International Conference on Machine Learning, ICML*, volume 32 of *JMLR Workshop and Conference Proceedings*, pages 692–700. JMLR.org, 2014.
- [168] Harm van Seijen, Hado van Hasselt, Shimon Whiteson, and Marco A. Wiering. A theoretical and empirical analysis of expected sarsa. In *IEEE Symposium on Adaptive Dynamic Programming and Reinforcement Learning, ADPRL 2009, Nashville, TN, USA, March 31 - April 1, 2009*, pages 177–184, 2009.
- [169] Robert J Vanderbei. Linear programming: foundations and extensions. *Journal of the Operational Research Society*, 49(1):94–94, 1998.
- [170] Oriol Vinyals, Igor Babuschkin, Wojciech M Czarnecki, Michaël Mathieu, Andrew Dudzik, Junyoung Chung, David H Choi, Richard Powell, Timo Ewalds, Petko Georgiev, et al. Grandmaster level in starcraft ii using multi-agent reinforcement learning. *nature*, 575(7782):350–354, 2019.
- [171] Christopher J. C. H. Watkins and Peter Dayan. Q-learning. *Machine Learning*, 8:279–292, 1992.
- [172] Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8:229–256, 1992.
- [173] Yinyu Ye. The simplex and policy-iteration methods are strongly polynomial for the Markov decision problem with a fixed discount rate. *Math. Oper. Res.*, 36(4):593–603, 2011.
- [174] Kaiqing Zhang, Zhuoran Yang, and Tamer Başar. Multi-agent reinforcement learning: A selective overview of theories and algorithms. In Kyriakos G. Vamvoudakis, Yan Wan, Frank L. Lewis, and Derya Cansever, editors, *Handbook of Reinforcement Learning and Control*, pages 321–384. Springer International Publishing, Cham, 2021.

- [175] Shun Zhang, Zhenfang Chen, Yikang Shen, Mingyu Ding, Joshua B. Tenenbaum, and Chuang Gan. Planning with large language models for code generation. In *Proceedings of the International Conference on Learning Representations (ICLR)*, 2023.

Index

- Alpha Zero, 240
- Approximate model, 149
- Approximate optimal policy, 146
- Approximate Policy Evaluation, 218
- Approximate policy iteration, 234
- Approximate policy iteration,
 lookahead, 238
- Approximate Value Iteration, 152
- Approximate value iteration, 243
- Approximation approaches, 215
- Asymptotic Stability, 314
- Average return, 80

- Banach Fixed Point Theorem, 98
- Bellman equation, projected, 223
- Bellman operator, 99
- Bellman's optimality equation, 95
- Bellman's Optimality Equation Proof,
 101
- Bootstrapping, 222
- Bootstrapping Algorithms, 189
- Bretagnolle Huber-Carol, 150

- Chernoff-Hoeffding, 142
- Classification based policy iteration,
 237
- Compatible Value Functions, 265
- Contraction Operators, 97
- Convergence of Policy Gradient, 268
- Curse of dimensionality, 213

- Deep Q learning, 246
- Design matrix, 220
- Discounted MDP Problem Statement,
 90
- Discounted MDP Value Function, 91
- Discounted MDPs effective horizon,
 140
- discounted return, 80
- DQN, 246
- Dynamic programming operators, 99

- E^3 , 156, 158
- Effective horizon, 140
- Estimating transition probabilities,
 147
- Euclidean norm, weighted, 221
- Explicit Explore or Exploit, 156, 158

- Fibonacci Sequence, 305
- Finite Differences Methods, 256
- Finite Horizon
 - Dynamic Programming, 82
 - Markov policies, 82
 - Policy evaluation, 83
 - Policy optimization, 84
 - Q-function, 87
- Finite Horizon return, 77
- Fitted Q learning, 244
- Fixed point theorem, 98
- Fixed policy value iteration, 93

- Gaussian linear policy, 251
- Hoeffding, 142
- Importance sampling, 209
- Integer Knapsack, 307
- Least squares policy iteration, 242
- Least Squares Regression, 219
- Linear equations for value function, 91
- Linear least squares, 220
- Linear ODE, 311
- Linear programming, 121
 - Discounted, 126
 - Finite horizon, 122
 - background, 121
- Log linear policy, 250
- Longest Common Subsequence, 308
- Longest Increasing Subsequence, 307
- Lookahead policy, 215
- LSPI algorithm, 242
- LSTD algorithm, 228
- Matrix exponential, 313
- Maximum Contiguous Sum, 306
- Maximum likelihood, 176
- MCTS, 239
- MDP
 - definition, 73
 - policies, 75
 - return function, 77
- Mean estimation, 142
- Model predictive control, 215
- Monte Carlo tree search, 239
- Monte Carlo, policy evaluation, 222
- Monte-Carlo algorithms, 173
 - control, 180
 - Every visit, 177
 - First visit, 174
- Multi-Layer Perceptron policy, 251
- Myopic approximation, 215
- ODE, 311
- ODE, Existence and Uniqueness, 312
- Off-Policy Model-Based Learning, 141
- On-Policy Model Based Learning, 154
- Optimistic on-policy model based RL, 157, 162
- Ordinary differential equation, 311
- Picard-Lindelöf theorem, 312
- Policy approximation, 215
- Policy Gradient Theorem, 257
- Policy improvement lemma, 104
- Policy iteration, 96, 104
- Policy iteration comparison with value iteration, 105
- Policy Performance Difference Lemma, 251
- Policy Representations, 250
- Projected Bellman equation, 223
- Projected operator, contraction, 225
- Projected value iteration, 229, 243
- Projection, 221
- Projection, non-expansive, 226
- Proximal Policy Optimization, 270
- Pythagorean theorem, 226
- Q learning, function approximation, 244
- Q-learning
 - DDP, 169
 - MDP, 194
- R-max algorithm, 157, 162
- Regression, policy evaluation, 222
- Regression, projected value iteration, 230
- REINFORCE algorithm, 262
- Replay buffer, 246

- Reward estimation errors, 143
- Rolling horizon policy, 215
- SARSA, 197
- SARSA(λ), 208
- SARSA, function approximation, 242
- SGD, 220
- Simplex policy, 251
- Simulation lemma, 149
- Stochastic approximation, 182
 - convergence (via contraction), 183
 - convergence (via ODE), 184
- Stochastic gradient descent, 220
- Stochastic Shortest-Path, 81
- Stopping rules, 103
- Tabular features, 217
- Target network, 246
- TD(0), 190
- TD(0), convergence with function approximation, 232
- TD(0), episodic MDPs, 233
- TD(0), function approximation, 231
- Temporal differences, 222
- Temporal differences, function approximation, 231
- Temporal differences, least squares, 228
- UCT, 240
- Value approximation, 215
- Value approximation architectures, 216
- Value approximation error, 217
- Value iteration, 95, 102
- Value iteration comparison with policy iteration, 105
- Value iteration error bounds, 103
- Value iteration stopping rules, 103
- Value iteration, projected, 229
- Weighted Euclidean norm, 221